
Foundations of Computer Vision

Mathematics, Models, and Research

Ashish Papanai

Foundations of Computer Vision

Mathematics, Models, and Research

Ashish Papanai

Contents

How to use this book	i
Acknowledgements	iv
Edition and correction notes	v
A field guide to new problems	vi
Notation	xiii
I Mathematical and statistical foundations	1
1 Geometry, matrix calculus and stable computation	2
1.1 A concrete problem	2
1.2 How we got here	3
1.3 Assumptions and notation	3
1.4 Mathematical formulation	4
1.5 Derivation, with reasons for each step	5
1.6 Numerical worked example	8
1.7 Small implementation and checks	8
1.8 Experiment and expected observations	9
1.9 Moving pieces: diagnosing and fixing	11
1.10 Application, limitations and failure cases	12
1.11 Bridge to a research paper	12
1.12 Memory map	13
1.13 Exercises	14
1.14 Self-check and common mistakes	15
2 SVD and PCA	16
2.1 A concrete problem	16
2.2 How we got here	16
2.3 Assumptions and notation	17
2.4 Mathematical formulation	18
2.5 Derivation, with reasons for each step	18
2.6 Numerical worked example	21
2.7 Small implementation and checks	22
2.8 Experiment and expected observations	22
2.9 Moving pieces: diagnosing and fixing	24
2.10 Application, limitations and failure cases	25

2.11	Bridge to a research paper	26
2.12	Memory map	26
2.13	Exercises	26
2.14	Self-check and common mistakes	28
3	Regression, classification and regularization	30
3.1	A concrete problem	30
3.2	How we got here	31
3.3	Assumptions and notation	31
3.4	Mathematical formulation	32
3.5	Derivation, with reasons for each step	32
3.6	Numerical worked example	36
3.7	Small implementation and checks	36
3.8	Experiment and expected observations	37
3.9	Moving pieces: diagnosing and fixing	39
3.10	Application, limitations and failure cases	40
3.11	Bridge to a research paper	40
3.12	Memory map	40
3.13	Exercises	42
3.14	Self-check and common mistakes	43
4	Duality and kernels	44
4.1	A concrete problem	44
4.2	How we got here	44
4.3	Assumptions and notation	45
4.4	Mathematical formulation	46
4.5	Derivation, with reasons for each step	47
4.6	Numerical worked example	50
4.7	Small implementation and checks	50
4.8	Experiment and expected observations	51
4.9	Moving pieces: diagnosing and fixing	53
4.10	Application, limitations and failure cases	53
4.11	Bridge to a research paper	54
4.12	Memory map	54
4.13	Exercises	54
4.14	Self-check and common mistakes	56
5	Risk, generalization and experimental design	58
5.1	A concrete problem	58
5.2	How we got here	59
5.3	Assumptions and notation	59
5.4	Mathematical formulation	60
5.5	Derivation, with reasons for each step	60
5.6	Numerical worked example	65
5.7	Small implementation and checks	65
5.8	Experiment and expected observations	65
5.9	Moving pieces: diagnosing and fixing	68
5.10	Application, limitations and failure cases	68
5.11	Bridge to a research paper	69
5.12	Memory map	69
5.13	Exercises	69

5.14	Self-check and common mistakes	71
II	Probabilistic learning	73
6	Likelihood, priors and exponential families	74
6.1	A concrete problem	74
6.2	How we got here	75
6.3	Assumptions and notation	75
6.4	Mathematical formulation	76
6.5	Derivation, with reasons for each step	76
6.6	Numerical worked example	81
6.7	Small implementation and checks	81
6.8	Experiment and expected observations	81
6.9	Moving pieces: diagnosing and fixing	82
6.10	Application, limitations and failure cases	83
6.11	Bridge to a research paper	83
6.12	Memory map	83
6.13	Exercises	85
6.14	Self-check and common mistakes	85
7	Gaussian conditioning and Gaussian processes	87
7.1	A concrete problem	87
7.2	How we got here	87
7.3	Assumptions and notation	88
7.4	Mathematical formulation	89
7.5	Derivation, with reasons for each step	89
7.6	Numerical worked example	92
7.7	Small implementation and checks	92
7.8	Experiment and expected observations	93
7.9	Moving pieces: diagnosing and fixing	94
7.10	Application, limitations and failure cases	94
7.11	Bridge to a research paper	95
7.12	Memory map	95
7.13	Exercises	97
7.14	Self-check and common mistakes	97
8	Latent variables and EM	99
8.1	A concrete problem	99
8.2	How we got here	100
8.3	Assumptions and notation	101
8.4	Mathematical formulation	101
8.5	Derivation, with reasons for each step	102
8.6	Numerical worked example	106
8.7	Small implementation and checks	106
8.8	Experiment and expected observations	108
8.9	Moving pieces: diagnosing and fixing	108
8.10	Application, limitations and failure cases	110
8.11	Bridge to a research paper	110
8.12	Memory map	111
8.13	Exercises	111
8.14	Self-check and common mistakes	113

9	KL divergence, variational inference and the ELBO	115
9.1	A concrete problem	115
9.2	How we got here	116
9.3	Assumptions and notation	116
9.4	Mathematical formulation	117
9.5	Derivation, with reasons for each step	117
9.6	Numerical worked example	122
9.7	Small implementation and checks	122
9.8	Experiment and expected observations	123
9.9	Moving pieces: diagnosing and fixing	123
9.10	Application, limitations and failure cases	125
9.11	Bridge to a research paper	125
9.12	Memory map	126
9.13	Exercises	126
9.14	Self-check and common mistakes	128
10	VAEs and generative objectives	130
10.1	A concrete problem	130
10.2	How we got here	130
10.3	Assumptions and notation	132
10.4	Mathematical formulation	132
10.5	Derivation, with reasons for each step	133
10.6	Numerical worked example	136
10.7	Small implementation and checks	137
10.8	Experiment and expected observations	137
10.9	Moving pieces: diagnosing and fixing	139
10.10	Application, limitations and failure cases	140
10.11	Bridge to a research paper	141
10.12	Memory map	143
10.13	Exercises	143
10.14	Self-check and common mistakes	144
III	Deep learning and visual tasks	146
11	Backpropagation and optimization	147
11.1	A concrete problem	147
11.2	How we got here	148
11.3	Assumptions and notation	148
11.4	Mathematical formulation	149
11.5	Derivation, with reasons for each step	149
11.6	Numerical worked example	152
11.7	Small implementation and checks	153
11.8	Experiment and expected observations	153
11.9	Moving pieces: diagnosing and fixing	154
11.10	Application, limitations and failure cases	157
11.11	Bridge to a research paper	157
11.12	Memory map	158
11.13	Exercises	159
11.14	Self-check and common mistakes	160
12	Convolutions and residual networks	162

12.1 A concrete problem	162
12.2 How we got here	163
12.3 Assumptions and notation	163
12.4 Mathematical formulation	164
12.5 Derivation, with reasons for each step	165
12.6 Numerical worked example	167
12.7 Small implementation and checks	167
12.8 Experiment and expected observations	168
12.9 Moving pieces: diagnosing and fixing	169
12.10 Application, limitations and failure cases	170
12.11 Bridge to a research paper	171
12.12 Memory map	172
12.13 Exercises	172
12.14 Self-check and common mistakes	174
13 Detection and segmentation	176
13.1 A concrete problem	176
13.2 How we got here	177
13.3 Assumptions and notation	177
13.4 Mathematical formulation	178
13.5 Derivation, with reasons for each step	179
13.6 Numerical worked example	181
13.7 Small implementation and checks	183
13.8 Experiment and expected observations	183
13.9 Moving pieces: diagnosing and fixing	185
13.10 Application, limitations and failure cases	186
13.11 Bridge to a research paper	186
13.12 Memory map	187
13.13 Exercises	187
13.14 Self-check and common mistakes	189
14 Attention and vision transformers	191
14.1 A concrete problem	191
14.2 How we got here	191
14.3 Assumptions and notation	192
14.4 Mathematical formulation	193
14.5 Derivation, with reasons for each step	194
14.6 Numerical worked example	196
14.7 Small implementation and checks	197
14.8 Experiment and expected observations	197
14.9 Moving pieces: diagnosing and fixing	198
14.10 Application, limitations and failure cases	199
14.11 Bridge to a research paper	200
14.12 Memory map	201
14.13 Exercises	202
14.14 Self-check and common mistakes	202
IV Visual representation learning	204
15 Contrastive learning	205
15.1 A concrete problem	205

15.2	How we got here	205
15.3	Assumptions and notation	206
15.4	Mathematical formulation	207
15.5	Derivation, with reasons for each step	208
15.6	Numerical worked example	210
15.7	Small implementation and checks	210
15.8	Experiment and expected observations	211
15.9	Moving pieces: diagnosing and fixing	213
15.10	Application, limitations and failure cases	214
15.11	Bridge to a research paper	214
15.12	Memory map	215
15.13	Exercises	216
15.14	Self-check and common mistakes	216
16	Teacher–student and masked learning	218
16.1	A concrete problem	218
16.2	How we got here	219
16.3	Assumptions and notation	219
16.4	Mathematical formulation	220
16.5	Derivation, with reasons for each step	220
16.6	Numerical worked example	223
16.7	Small implementation and checks	224
16.8	Experiment and expected observations	224
16.9	Moving pieces: diagnosing and fixing	226
16.10	Application, limitations and failure cases	227
16.11	Bridge to a research paper	227
16.12	Memory map	228
16.13	Exercises	229
16.14	Self-check and common mistakes	229
17	JEPA and video	231
17.1	A concrete problem	231
17.2	How we got here	232
17.3	Assumptions and notation	232
17.4	Mathematical formulation	233
17.5	Derivation, with reasons for each step	233
17.6	Numerical worked example	236
17.7	Small implementation and checks	236
17.8	Experiment and expected observations	236
17.9	Moving pieces: diagnosing and fixing	238
17.10	Application, limitations and failure cases	239
17.11	Bridge to a research paper	239
17.12	Memory map	240
17.13	Exercises	240
17.14	Self-check and common mistakes	242
18	Transfer and representation evaluation	244
18.1	A concrete problem	244
18.2	How we got here	245
18.3	Assumptions and notation	245
18.4	Mathematical formulation	246

18.5 Derivation, with reasons for each step	246
18.6 Numerical worked example	248
18.7 Small implementation and checks	248
18.8 Experiment and expected observations	249
18.9 Moving pieces: diagnosing and fixing	250
18.10 Application, limitations and failure cases	251
18.11 Bridge to a research paper	251
18.12 Memory map	252
18.13 Exercises	252
18.14 Self-check and common mistakes	254
V Multimodal and generative vision	256
19 Image–text alignment	257
19.1 A concrete problem	257
19.2 How we got here	258
19.3 Assumptions and notation	258
19.4 Mathematical formulation	259
19.5 Derivation, with reasons for each step	259
19.6 Numerical worked example	261
19.7 Small implementation and checks	261
19.8 Experiment and expected observations	261
19.9 Moving pieces: diagnosing and fixing	262
19.10 Application, limitations and failure cases	263
19.11 Bridge to a research paper	264
19.12 Memory map	264
19.13 Exercises	266
19.14 Self-check and common mistakes	267
20 Instruction-tuned VLMs and adaptation	268
20.1 A concrete problem	268
20.2 How we got here	269
20.3 Assumptions and notation	269
20.4 Mathematical formulation	270
20.5 Derivation, with reasons for each step	271
20.6 Numerical worked example	272
20.7 Small implementation and checks	273
20.8 Experiment and expected observations	273
20.9 Moving pieces: diagnosing and fixing	275
20.10 Application, limitations and failure cases	276
20.11 Bridge to a research paper	276
20.12 Memory map	277
20.13 Exercises	278
20.14 Self-check and common mistakes	279
21 Video and omni-modal perception	280
21.1 A concrete problem	280
21.2 How we got here	281
21.3 Assumptions and notation	281
21.4 Mathematical formulation	282
21.5 Derivation, with reasons for each step	282

21.6 Numerical worked example	284
21.7 Small implementation and checks	284
21.8 Experiment and expected observations	285
21.9 Moving pieces: diagnosing and fixing	285
21.10 Application, limitations and failure cases	287
21.11 Bridge to a research paper	287
21.12 Memory map	288
21.13 Exercises	289
21.14 Self-check and common mistakes	290
22 Diffusion in pixel and latent spaces	291
22.1 A concrete problem	291
22.2 How we got here	292
22.3 Assumptions and notation	292
22.4 Mathematical formulation	293
22.5 Derivation, with reasons for each step	295
22.6 Numerical worked example	297
22.7 Small implementation and checks	297
22.8 Experiment and expected observations	298
22.9 Moving pieces: diagnosing and fixing	299
22.10 Application, limitations and failure cases	300
22.11 Bridge to a research paper	302
22.12 Memory map	302
22.13 Exercises	302
22.14 Self-check and common mistakes	304
VI Entering research	307
23 Geometry and world-model introduction	308
23.1 A concrete problem	308
23.2 How we got here	309
23.3 Assumptions and notation	309
23.4 Mathematical formulation	310
23.5 Derivation, with reasons for each step	311
23.6 Numerical worked example	312
23.7 Small implementation and checks	312
23.8 Experiment and expected observations	313
23.9 Moving pieces: diagnosing and fixing	316
23.10 Application, limitations and failure cases	317
23.11 Bridge to a research paper	317
23.12 Memory map	318
23.13 Exercises	319
23.14 Self-check and common mistakes	320
24 Reproduction, ablations and scientific writing	322
24.1 A concrete problem	322
24.2 How we got here	322
24.3 Assumptions and notation	322
24.4 Mathematical formulation	323
24.5 Derivation, with reasons for each step	323
24.6 Numerical worked example	324

24.7 Small implementation and checks	324
24.8 Experiment and expected observations	325
24.9 Moving pieces: diagnosing and fixing	325
24.10 Application, limitations and failure cases	326
24.11 Bridge to a research paper	326
24.12 Memory map	327
24.13 Exercises	328
24.14 Self-check and common mistakes	328
Appendix A. Prerequisites and self-diagnostic	330
Appendix C. Stable numerical computation	332
Glossary	333
References	334
About the author	339

How to use this book

Machine learning research begins with a question about data, assumptions or behaviour. A model is useful only when we can say what it learns, how it learns it, and what evidence supports our claims about it. This book builds those abilities step by step, from fitting a straight line to reading and testing claims about modern vision, vision–language and world models. Its recurring rule is simple:

Derive the mechanism, implement a small version, test an explanation.

The book is meant to be a one-stop companion. Every chapter explains the idea in plain words before the mathematics, derives every step it uses, works numbers you can check by hand, runs a small experiment you can reproduce on a laptop, and then shows how the idea breaks and how to repair it.

Who this book is for

You need basic Python, matrix multiplication, derivatives and introductory probability. Appendix A has a short self-diagnostic and refreshers; you do not need a full undergraduate mathematics course first. Readers who already know the basics can skim Part I, but should still read the “Moving pieces” sections, which are where most practical skill lives.

How every chapter is built

Every full chapter follows the same fourteen-step arc, so you always know where to look.

Section	What it gives you
N.1 A concrete problem	a situation you can picture, and why the obvious approach is not enough
N.2 How we got here	the history of the idea: what was tried before, what failed, what this idea fixed, and what it still cannot do
N.3 Assumptions and notation	every symbol with its shape, and the assumptions written so they can be checked
N.4 Mathematical formulation	the problem as an equation
N.5 Derivation	every step, with the <i>reason</i> for the step
N.6 Numerical worked example	numbers small enough to do by hand
N.7 Small implementation and checks	NumPy code in <code>labs/</code> and the tests that check it
N.8 Experiment	a run you can repeat, with the numbers it prints and what to notice

Section	What it gives you
N.9 Moving pieces	the knobs, what goes wrong with each, how to diagnose it, how to fix it
N.10 Application and failure cases	where the idea is used, and how it fails in practice
N.11 Bridge to a research paper	how to read one paper through this chapter, often with its architecture redrawn
N.12 Memory map	a one-page mind map of the chapter, with the same six branches every time
N.13 Exercises	conceptual, derivation, implementation, interpretation and open questions, with answer discussions
N.14 Self-check	a checklist and the most common mistakes

The **memory maps** always have the same six branches — *Problem*, *Key idea*, *Math*, *Moving pieces*, *Failure modes*, *Connects to* — so that after a few chapters you can rebuild a map from memory. That is the point: draw the map yourself before looking at it.

The **How we got here** sections are there because ideas are easier to understand when you know what problem forced them into existence. Attention, for example, makes much more sense once you have felt the limits of a fixed receptive field. The lineage map in the field guide (figure F.3) puts all of these stories on one page.

A way to read actively

For each chapter:

1. Explain the problem to yourself without equations.
2. Read *How we got here* and predict what the new idea must change.
3. Cover the right-hand side of each important derivation and rebuild it.
4. Do the worked example by hand before running the lab.
5. Run the lab, then change one thing and predict the result before you look.
6. Draw the memory map from memory, then compare with the book's.
7. Explain one failure case and how you would detect it.

Recognising a formula is weaker evidence of understanding than being able to rebuild its assumptions and interpret its output.

Exercises, answers and labs

Exercises have stable identifiers such as 7.2. Each exercise section ends with short answer discussions for self-study.

The **labs/** directory holds a runnable lab for each chapter that has one. Labs use NumPy on synthetic data, run on a laptop CPU in seconds, need no downloads, and come with tests that check properties derived in the text. Each lab README declares its compute, memory, run time, seeds and data provenance. The examples are deliberately small so you can see what is happening; they do not claim to reproduce a published benchmark.

Getting the code. The labs, tests and figure sources live in the book's companion repository, which is shared privately with the closed group this draft is circulated to. Paths

such as `labs/ch10-vae/vae.py` are relative to the root of that repository. If you have the book but not the repository, ask the person who shared the book with you, or write to founder@blankpageml.tech. Python 3 with NumPy and pytest is enough to run every lab (`pip install -r requirements.txt`, then `make check`).

Reading paths

Reader	Path
Foundations	front matter, chapters 1–10, chapter 11 as a bridge, chapter 24 for the capstone
Computer vision research	field guide, chapters 11–21, then 23–24; earlier chapters as short bridge readings; chapter 22 is an extension
Self-study	in order; spend extra time on worked examples and the moving-pieces tables
Practitioner in a hurry	field guide, then for any chapter: N.1, N.2, N.9, N.12

Track markers

Material is marked **core** (assigned and assessed), **bridge** (short prerequisites or connections) or **extension** (optional full derivations, extra experiments or advanced papers). The book covers more than a single course built from it would assign.

Attribution

The explanations, derivations, worked examples and diagrams are written for this book. Architecture diagrams of published models are *redrawn* from the papers' descriptions and say which figure they follow; paper figures are never copied. Notation follows Goodfellow, Bengio and Courville (2016). Selected 2023–2025 research examples illustrate recent practice as of October 2026; inclusion does not imply they are the newest or best models. Report errata with a chapter, equation and a reproducible counterexample.

Acknowledgements

AI (Claude, by Anthropic) was used as an aid for language checks, code evaluation, keeping experiments consistent with the text, and converting handwritten notes to LaTeX. The cover image was generated with GPT (OpenAI).

This is a draft, and there can be rough edges. Corrections, suggestions and feedback are welcome at founder@blankpageml.tech.

Edition and correction notes

Edition. Draft for closed group circulation, built on the date shown on the title page. It is not peer reviewed and is not a final edition.

Corrections in this build.

- *Likelihood bounds.* The β -VAE objective is a bound on negative log-likelihood for every $\beta \geq 1$, not only $\beta = 1$ (chapter 10). Importance-weighted bounds hold in expectation; a single estimate can exceed the log evidence (chapters 9 and 10).
- *Temporal pooling.* Mean pooling is invariant to rearranging already-computed features; temporal encoders can retain motion information before pooling. The lab’s conclusion is now limited to its per-frame encoder (chapter 17).
- *EMA indexing and sharpening.* The EMA teacher is written as a finite sum whose weights add to one; the account of DINO’s teacher temperature no longer claims the target is never uniform (chapter 16).
- *PCA from the SVD.* Directions are recovered only for positive singular values (chapter 2).
- *SVM support vectors.* The complementary-slackness cases are stated as one-way implications (chapter 4).
- *Optimisation error.* The population-risk optimisation term is signed; the empirical gap is not (chapter 5, figure 5.1).
- *Architecture attribution.* The detector diagram is labelled as a modern Faster R-CNN variant, with FPN and RoIAlign credited to their own papers (chapter 13).
- *Compute counts, calibration, and modality interventions* (chapters 14, 18 and 21).
- *Build.* Chapter 4’s exercise heading and chapter 16’s first figure were missing from the previous PDF because of build problems; both now render. Missing appendices are no longer referenced.

What was checked. The structural checker, all 159 unit tests and the smoke run of every lab pass. Full default-seed runs of the six labs behind the reworded claims (chapters 9, 10, 16, 17, 18, 21) reproduce every number quoted in those passages. A new check compares the built book with its sources: every chapter has its fourteen sections and all of its figures, in order. Pages with changed text or figures were rendered and inspected by eye.

Limits. These checks show that the code runs and that quoted numbers match the labs. They do not show that every explanation, derivation or citation is correct.

Reporting an error. Please send the chapter, section, and the text or equation concerned, with a counterexample or source where possible, to founder@blankpageml.tech.

A field guide to new problems

Most of this book is about particular ideas: PCA, Gaussian processes, attention, contrastive learning, diffusion. This guide is about what they have in common. Every learning system, from a least-squares fit to a vision–language model, is made of the same **moving pieces**. When something goes wrong, one of those pieces is at fault. When a new idea appears in a paper, it almost always changes one or two pieces to fix a failure of an older idea. Learn to see the pieces and you can approach a problem you have never seen before.

Read this guide once before chapter 1, and come back to it whenever an experiment surprises you.

F.1 The moving pieces of a learning system

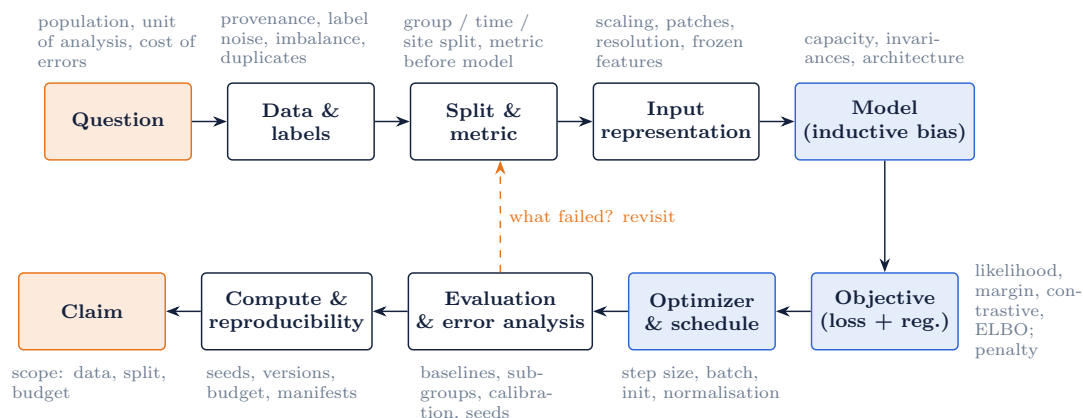


Figure F.1: The moving pieces of any learning system, with the knobs on each. The dashed arrow is the loop that does most of the work: evaluation tells you which piece to revisit.

1. **The question.** What decision or understanding is the model for? Who or what is the population? What is the unit of analysis: an image, a patient, a field, a video? What does each kind of error cost? A vague question produces an experiment that cannot fail, which means it cannot teach you anything.
2. **Data and labels.** Where did the examples come from (provenance)? How were labels produced, and how noisy are they? Are classes imbalanced? Are there near-duplicates? Most “model” problems are data problems in disguise.
3. **Split and metric.** The split decides *which* generalisation you are measuring: new images of familiar patients, or new patients; next week, or last week; a new farm, or the same farm. Choose the split and the metric **before** choosing a model (chapter 5).
4. **Input representation.** Raw pixels, normalised pixels, patches, frozen features from a pretrained network, a text prompt. Representation decides what is easy to learn.

5. **Model and its inductive bias.** Every model assumes something: linearity (chapter 3), smoothness (chapter 7), locality and translation equivariance (chapter 12), content-based mixing (chapter 14). An assumption that matches the data saves examples; one that does not costs accuracy no matter how long you train.
6. **Objective.** What the optimiser actually minimises: a likelihood (chapter 6), a margin, an ELBO (chapter 9), a contrastive loss (chapter 15), a denoising loss (chapter 22), plus regularisation. The objective is a precise statement of what you *asked* for, which is not always what you *wanted*.
7. **Optimiser and schedule.** Step size, batch size, initialisation, normalisation, number of steps. Optimisation can fail silently: a model that never reached the minimum of its objective tells you nothing about that objective.
8. **Evaluation and error analysis.** Baselines, per-subgroup errors, calibration, seeds, looking at the actual failures. This is where you learn which piece to change next.
9. **Compute and reproducibility.** Seeds, versions, data manifests, run budget. A result you cannot reproduce is an anecdote.
10. **The claim.** What the evidence supports, on which data, split and budget — and nothing more.

F.2 A routine for a problem you have never seen

1. **Write the question in one sentence**, including the population and the decision. “Detect late blight in potato leaves *photographed on farms we have not seen*” is a different problem from “... in this dataset”.
2. **Look at the data before modelling.** Print twenty examples with their labels. Plot the label distribution. Search for duplicates. Note anything a model could use as a shortcut (background, scanner type, timestamp, file name).
3. **Fix the split and metric.** Group by the unit of analysis. Decide the primary metric and how much difference would matter. Write both down.
4. **Run trivial baselines.** Predict the majority class; predict the mean; nearest neighbour on raw inputs; a linear model on standard features. Any later model must beat these, and the gap tells you how much there is to learn.
5. **Build the simplest model that could work**, then make it **overfit a tiny batch** (10–50 examples). If it cannot, the problem is a bug or the optimiser, not the data.
6. **Scale up, one change at a time.** More data, more capacity, regularisation, augmentation — each as a separate, recorded run.
7. **Do error analysis.** Sort errors by subgroup, by confidence, by input property. Look at the worst twenty. Ask which moving piece explains them.
8. **Ablate.** Remove each component you added. If removing it does not hurt, you have not shown it helps.
9. **Repeat with seeds** and report spread, not just the best run.
10. **Write the claim with its scope** (chapter 24).

Most of the time spent on a successful project goes into steps 2, 3, 7 and 8, not into choosing an architecture.

F.3 Diagnose by symptom

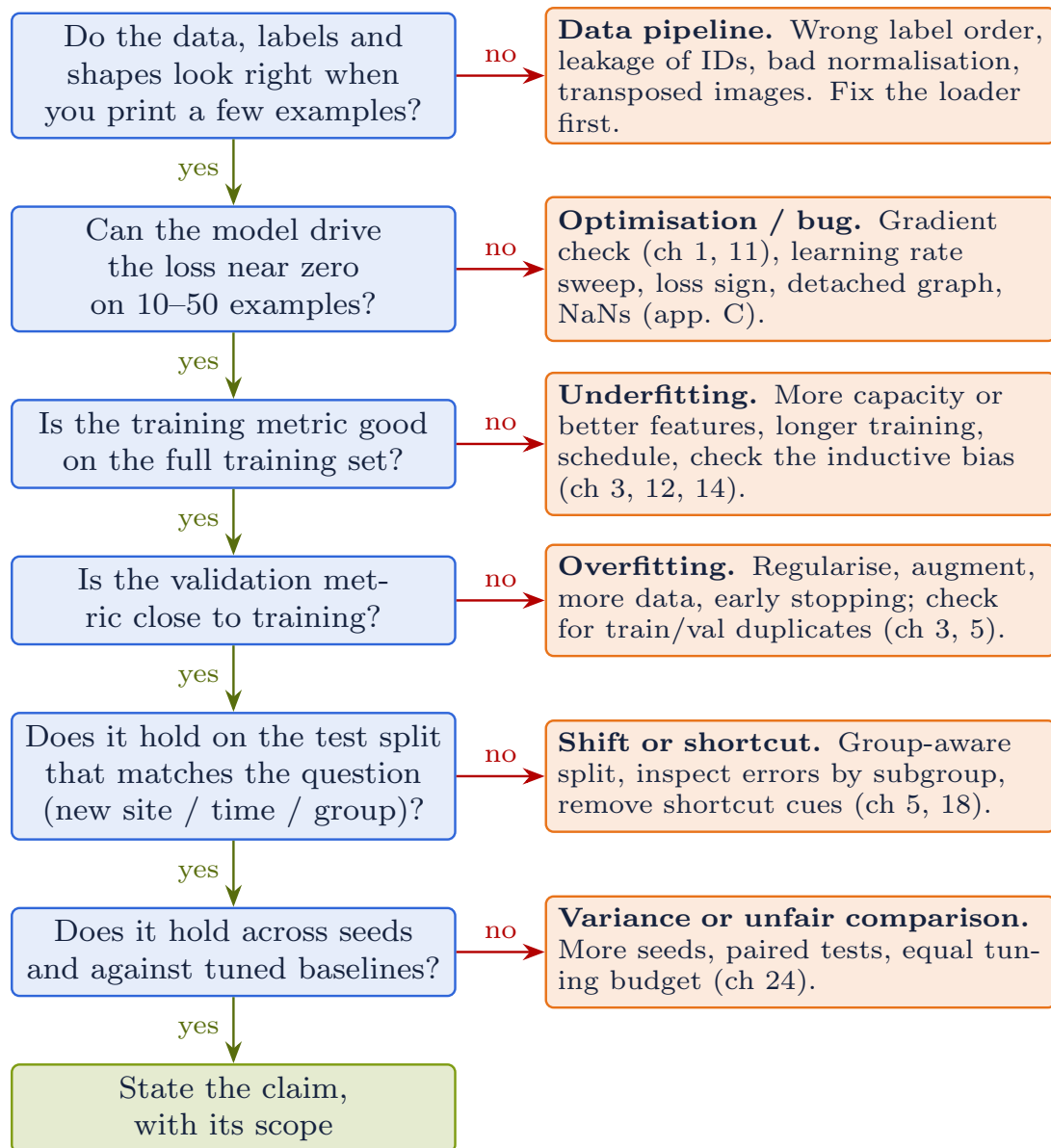


Figure F.2: A diagnosis flow. Walk down the left column; the first “no” tells you which moving piece to inspect, and the box on the right says what to try.

Symptom	Most likely moving piece	How to confirm	First fix
Loss is NaN or explodes	optimiser, numerics	print loss per step; check for exp of large numbers, division by zero, log of zero	lower learning rate; stable softmax/log-sum-exp (appendix C); gradient clipping
Training loss does not decrease	optimiser or a bug	overfit 10 examples; gradient check (chapters 1 and 11)	fix the bug; sweep learning rate over powers of ten
Training loss decreases, accuracy stays at chance	labels or objective	shuffle labels — if nothing changes, labels never reach the loss	fix label alignment; check the loss target format
Good training, poor validation	capacity vs. data	learning curves; duplicates across splits	regularise, augment, more data, early stopping
Good validation, poor test	split mismatch or tuning on test	compare split construction; count test evaluations	rebuild the split to match the question; tune on validation only
Good on random split, poor on new site or time	shortcut or distribution shift	group-aware split; errors by group	remove shortcut cues; collect diverse data; evaluate honestly (chapters 5, 18)
Metric improves but outputs look wrong	metric does not measure the goal	look at examples the metric scores highly	change or add metrics; human inspection protocol
Probabilities are over-confident	calibration	reliability diagram; NLL vs. accuracy	temperature scaling (chapter 5); proper scoring rules
Representation collapses (all embeddings equal)	self-supervised objective	embedding variance across a batch	stop-gradient, EMA teacher, negatives, variance terms (chapters 15–17)
Results change a lot with seed	variance	run 3–5 seeds; paired comparison	report spread; average; larger validation set
Works in notebook, fails in script	data pipeline	compare a batch byte for byte	one shared preprocessing function

F.4 How ideas move: the pattern behind the history

Look at any arrow in figure F.3 and the same pattern appears:

1. An existing method works well enough to be used widely.
2. Wide use exposes a **specific failure**, tied to one moving piece.
3. A new idea changes that piece — and only that piece, at first.

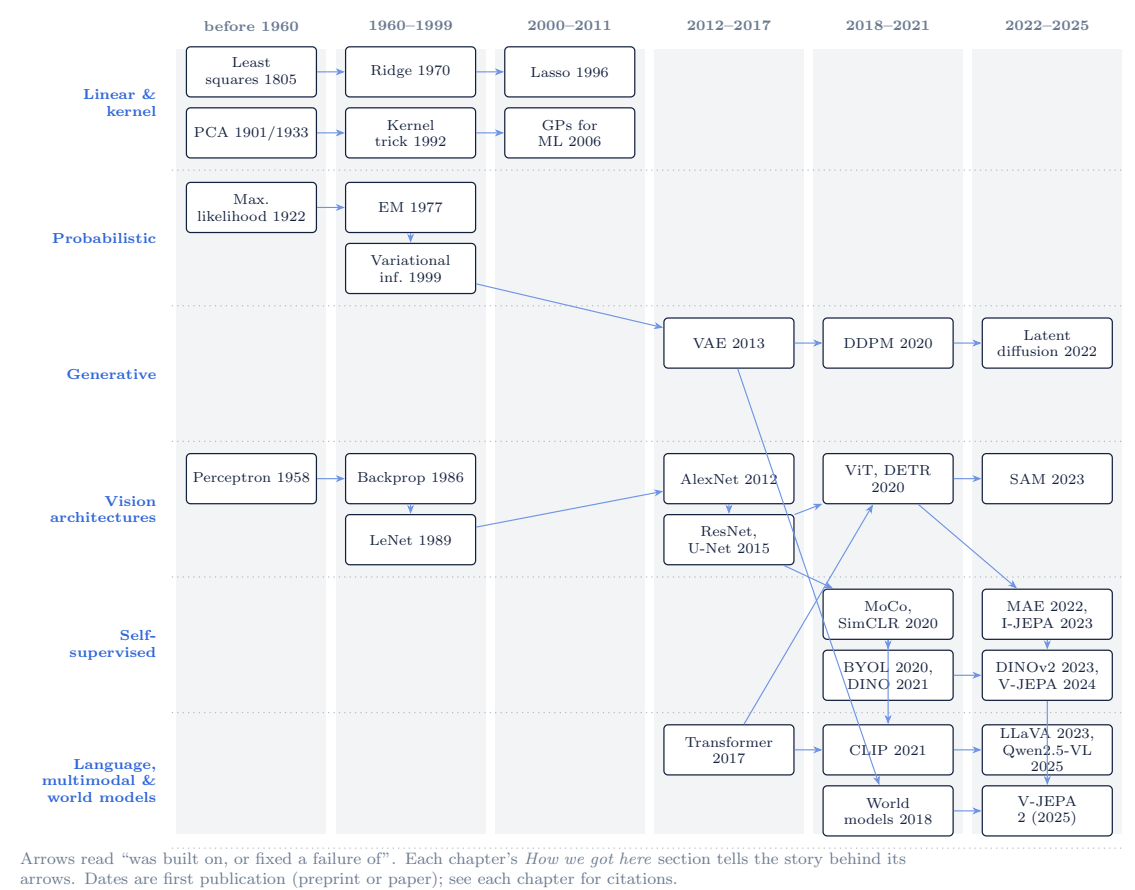


Figure F.3: How the ideas in this book grew out of one another. Each arrow is a story about a moving piece that failed and was replaced.

4. The new idea has its own failure, which becomes the next problem.

A few of the stories told in later chapters:

Older idea	What failed	Which piece changed	New idea	Chapter
Least squares	unstable weights when features are correlated	objective (add a penalty)	ridge, Lasso	3
Linear models	can only draw straight boundaries	representation (implicit features)	kernels	4
Kernel ridge	no uncertainty	model (a prior over functions)	Gaussian processes	7
Exact likelihood	intractable with latent variables	objective (a lower bound)	EM, then variational inference	8, 9
Variational inference	one optimisation per data point	model (an encoder amortises inference)	VAE	10
Fully connected nets	too many parameters for images, no translation structure	model (local, shared weights)	CNNs	12
Very deep CNNs	deeper networks trained <i>worse</i>	model (identity shortcuts)	ResNet	12
CNNs	long-range context needs many layers	model (content-based mixing)	attention, ViT	14
Supervised pretraining	labels are expensive and narrow	objective (agreement between views)	contrastive learning	15
Contrastive learning	needs many negatives; pushes apart near-duplicates	objective (teacher–student, stop-gradient)	BYOL, DINO	16
Pixel reconstruction	wastes capacity on unpredictable detail	objective (predict in latent space)	JEPA	17
Fixed-label classifiers	cannot name unseen categories	data and objective (image–text pairs)	CLIP	19
VAEs	blurry samples	model and objective (many small denoising steps)	diffusion	22

When you meet a new paper, ask the same questions: *which older method is it fixing? which failure? which moving piece did it change? what new failure might that change introduce?* Most papers answer the first three in their introduction. The fourth is usually where your own research question is.

F.5 Thinking tools that work everywhere

- **Shapes.** Write the shape of every tensor. Most bugs are shape bugs that broadcasting hid.
- **Units and scale.** If you double every input, what should happen to the output, the loss, the gradient?
- **Limits.** Push a knob to zero and to infinity. Ridge with $\lambda \rightarrow 0$ is least squares; with $\lambda \rightarrow \infty$ every weight is zero. A GP with length-scale $\ell \rightarrow \infty$ is almost constant. Limits are free sanity checks.
- **Toy cases.** One example, one dimension, two classes. If you cannot predict the answer there, you will not understand it in a thousand dimensions.
- **Invariances.** Which transformations of the input should leave the output unchanged (translation, permutation, colour jitter)? Test them.
- **Conservation checks.** Probabilities sum to one; attention rows sum to one; a reconstruction with every component is exact; a gradient check matches.
- **Baselines and ablations.** The value of a component is the difference it makes when removed, measured over seeds.

F.6 How the book is organised

Part	Chapters	The moving pieces it teaches
I. Mathematical and statistical foundations	1–5	representation, objective, numerics, split and metric
II. Probabilistic learning	6–10	objectives from probability, uncertainty, latent variables, generative models
III. Deep learning and visual tasks	11–14	optimiser, architectures and their inductive biases, dense prediction
IV. Visual representation learning	15–18	self-supervised objectives, collapse, evaluating representations
V. Multimodal and generative vision	19–22	language as supervision, adaptation, many modalities, diffusion
VI. Entering research	23–24	world models, reproduction, ablations and writing

Each chapter’s memory map ends with a *Connects to* branch. Following those branches is another way to read the book: as one long story about which moving piece to change next.

Notation

This section provides a concise reference describing the notation used throughout this book. It follows the notation of Goodfellow, Bengio and Courville (2016), released at https://github.com/goodfeli/dlbook_notation, extended for topics that book does not cover.

Numbers and arrays

Symbol	Meaning
a	a scalar (integer or real)
$\mathbf{a}$	a vector (column vector unless stated)
$\mathbf{A}$	a matrix
$\mathbf{A}$	a tensor
$\mathbf{I}_n$	identity matrix with n rows and n columns
$\mathbf{I}$	identity matrix with dimensionality implied by context
$\mathbf{e}^{(i)}$	standard basis vector with a 1 at position i
$\text{diag}(\mathbf{a})$	square diagonal matrix with diagonal $\mathbf{a}$
a	a scalar random variable
$\mathbf{a}$	a vector-valued random variable
$\mathbf{A}$	a matrix-valued random variable

Bold Greek letters are vectors or matrices too: $\boldsymbol{\mu}$, $\boldsymbol{\theta}$, $\boldsymbol{\Sigma}$.

Sets and graphs

Symbol	Meaning
$\mathbb{A}$	a set
$\mathbb{R}$	the set of real numbers
$\{0, 1, \dots, n\}$	the integers from 0 to n
$[a, b]$, $(a, b]$	real intervals including / excluding an end point
$\mathbb{A} \setminus \mathbb{B}$	set subtraction
$\mathcal{G}$	a graph

Do not write $\mathbb{R}^+$. Write $[0, \infty)$ or $(0, \infty)$.

Indexing

Symbol	Meaning
a_i	element i of vector $\mathbf{a}$, indexing from 1
a_{-i}	all elements of $\mathbf{a}$ except element i
$A_{i,j}$	element i, j of matrix $\mathbf{A}$
$\mathbf{A}_{i,:}$	row i of matrix $\mathbf{A}$
$\mathbf{A}_{:,i}$	column i of matrix $\mathbf{A}$
$A_{i,j,k}$	element (i, j, k) of a 3-D tensor
$\mathbf{A}_{::,i}$	2-D slice of a 3-D tensor
$\mathbf{a}_i$	element i of the random vector $\mathbf{a}$

Elements of vectors and matrices are scalars, so they are **not** bold.

Linear algebra operations

Symbol	Meaning
$\mathbf{A}^\top$	transpose
$\mathbf{A}^+$	Moore–Penrose pseudoinverse
$\mathbf{A} \odot \mathbf{B}$	element-wise (Hadamard) product
$\det(\mathbf{A})$	determinant
$\text{Tr}(\mathbf{A})$	trace

Calculus

Symbol	Meaning
$\frac{dy}{dx}, \frac{\partial y}{\partial x}$	derivative, partial derivative
$\nabla_{\mathbf{x}} y$	gradient of y with respect to $\mathbf{x}$
$\nabla_{\mathbf{X}} y$	matrix of derivatives of y with respect to $\mathbf{X}$ (same shape as $\mathbf{X}$)
$\nabla_{\mathbf{X}} y$	tensor of derivatives of y with respect to $\mathbf{X}$
$\frac{\partial f}{\partial \mathbf{x}}$	Jacobian $\mathbf{J} \in \mathbb{R}^{m \times n}$ of $f : \mathbb{R}^n \rightarrow \mathbb{R}^m$
$\nabla_{\mathbf{x}}^2 f(\mathbf{x})$ or $\mathbf{H}(f)(\mathbf{x})$	Hessian of f at $\mathbf{x}$
$\int f(\mathbf{x}) d\mathbf{x}, \int_{\mathbb{S}} f(\mathbf{x}) d\mathbf{x}$	definite integral over the domain, over $\mathbb{S}$

Probability and information theory

Symbol	Meaning
$\mathbf{a} \perp \mathbf{b}, \mathbf{a} \perp \mathbf{b} \mid \mathbf{c}$	independence, conditional independence
$P(\mathbf{a})$	distribution over a discrete variable
$p(\mathbf{a})$	distribution over a continuous (or unspecified) variable
$\mathbf{a} \sim P$	random variable $\mathbf{a}$ has distribution P ; the left side is always a random variable
$\mathbb{E}_{\mathbf{x} \sim P}[f(\mathbf{x})]$ or $\mathbb{E}f(\mathbf{x})$	expectation of $f(\mathbf{x})$ with respect to $P(\mathbf{x})$

Symbol	Meaning
$\text{Var}(f(x)),$ $\text{Cov}(f(x), g(x))$	variance, covariance under $P(x)$
$H(x)$	Shannon entropy
$D_{\text{KL}}(P\ Q)$	Kullback–Leibler divergence
$\mathcal{N}(x; \mu, \Sigma)$	Gaussian over x with mean μ and covariance Σ

A density is written with its argument and a semicolon: $p(x) = \mathcal{N}(x; \mu, \Sigma)$. Parameters that are not random variables follow a semicolon: $p(y | X; \theta)$.

Functions

Symbol	Meaning
$f : \mathbb{A} \rightarrow \mathbb{B}$	function with domain $\mathbb{A}$ and range $\mathbb{B}$
$f \circ g$	composition
$f(x; \theta)$	function of x parametrised by θ (sometimes $f(x)$)
$\log x$	natural logarithm
$\sigma(x)$	logistic sigmoid $1/(1 + \exp(-x))$
$\zeta(x)$	softplus $\log(1 + \exp(x))$
$\ x\ _p, \ x\ $	L^p norm, L^2 norm
x^+	positive part $\max(0, x)$
$\mathbf{1}_{\text{condition}}$	1 if the condition is true, 0 otherwise

A scalar function applied to an array acts element-wise: if $\mathbf{C} = \sigma(\mathbf{X})$, then $C_{i,j,k} = \sigma(X_{i,j,k})$. σ with a subscript or a square is a standard deviation or variance (σ_n^2, σ_f^2), never the sigmoid.

Datasets and distributions

Symbol	Meaning
p_{data}	the data-generating distribution
$\hat{p}_{\text{data}}$	the empirical distribution of the training set
$\mathbb{X}$	a set of training examples
$x^{(i)}$	the i -th example (input) from a dataset
$y^{(i)}$ or $\mathbf{y}^{(i)}$	the target for $x^{(i)}$
$\mathbf{X}$	the $m \times n$ design matrix, with example $x^{(i)}$ in row $\mathbf{X}_{i,:}$

So m **counts examples** and n **counts features** throughout. Code follows the same layout: arrays have shape (m, n) , and shapes are written in comments as `# (m, n)`.

Book-specific conventions

These extend the reference notation for topics it does not cover. Where possible they match the Deep Learning book's own chapters.

Symbol	Meaning	Chapter
$\mathbf{X}_c$	centred design matrix, rows $(\mathbf{x}^{(i)} - \boldsymbol{\mu})^\top$	2
$\mathbf{D} \in \mathbb{R}^{n \times l}$, $\mathbf{c}$	PCA decoding matrix with orthonormal columns; code $\mathbf{c} \in \mathbb{R}^l$	2
$\mathbf{USV}^\top$	thin SVD, singular values $s_1 \geq s_2 \geq \dots$	2
λ_j	eigenvalues in decreasing order (also a regularisation weight in chapters 3–4, stated there)	2, 3, 4
$k(\mathbf{x}, \mathbf{x}')$, $\mathbf{K}$	kernel function and Gram matrix	4, 7
$\mathbf{X}_*$, $\mathbf{f}_*$, m_*	GP query inputs, latent values at them, number of queries	7
ℓ	kernel length-scale (distinct from the PCA code size l)	7
τ	number of tokens in a sequence (as for sequence length in the Deep Learning book)	14
$\mathbf{Q}, \mathbf{K}, \mathbf{V}, \mathbf{A}$	attention queries, keys, values, weights	14
d, d_k, d_v, h	model width, key and value widths, number of heads	14
$\mathbf{O}^{[j]}$	quantity belonging to head (or layer) j ; square brackets keep $^{(i)}$ for examples	14
L	a scalar loss; $\nabla_{\mathbf{Z}} L$ has the shape of $\mathbf{Z}$	11, 14

Known clash: $\mathbf{K}$ is the Gram matrix in Chapters 4 and 7 and the key matrix in Chapter 14. Each chapter says which one it means.

Part I

Mathematical and statistical foundations

Chapter 1

Geometry, matrix calculus and stable computation

Prerequisites: appendix A (self-diagnostic); matrix multiplication and derivatives.

Learning outcomes: after this chapter you can

1. read a linear predictor geometrically: as a hyperplane, and as a projection onto a column space;
2. differentiate scalar functions of vectors and matrices, keeping track of shapes;
3. derive the least-squares normal equations and say when the solution is unique;
4. measure how sensitive a solution is (the condition number), and choose a solver that does not make the sensitivity worse;
5. check any gradient with finite differences, choosing the step size for a reason.

1.1 A concrete problem

A plant scientist photographs leaves and wants to predict each leaf’s chlorophyll content from three numbers measured on the image: the mean red, green and blue intensity. A linear model is the natural first try: chlorophyll $\approx$ a weighted sum of the three colours plus a constant. With 200 photographed leaves and lab measurements of chlorophyll, we want the weights.

Two surprises usually follow.

- **The weights jump around.** Add a few more leaves, or re-run the lab measurement with slightly different noise, and the fitted weight on green changes from 2 to -1 , while the predictions barely change. Red and green intensity move together under the same lighting, so the data cannot tell them apart well.
- **Different software gives different answers.** One library returns sensible weights; a hand-written formula $(\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top \mathbf{y}$ returns garbage or crashes with “singular matrix”.

Neither is a bug in the usual sense. Both come from the *geometry* of the problem and from how computers store numbers. This chapter gives the language to see both: vectors and subspaces, gradients of functions of vectors, and the condition number. Everything later in the book — PCA, kernels, Gaussian processes, backpropagation — is built from these pieces.

1.2 How we got here

Fitting a line to astronomical data (1805–1809). In January 1801 Giuseppe Piazzi discovered the dwarf planet Ceres, followed it for about six weeks, and lost it behind the Sun. Carl Friedrich Gauss computed an orbit from those few noisy observations, and astronomers found Ceres again at the end of 1801 close to where he predicted. Adrien-Marie Legendre published the *method of least squares* in 1805 as a way to combine more observations than unknowns; Gauss published his version in 1809 and said he had used it since 1795. The problem they faced is ours: more equations than unknowns, every equation slightly wrong, and a need for one best compromise. Least squares picks the compromise that minimises the sum of squared errors, and Gauss later showed (the Gauss–Markov theorem, 1821–1823) why that compromise is the best unbiased linear one when the errors are uncorrelated with equal variance.

Solving the equations by hand, then by machine (1920s–1960s). Minimising squared error leads to the *normal equations* $\mathbf{X}^\top \mathbf{X} \mathbf{w} = \mathbf{X}^\top \mathbf{y}$ (section 1.5). For hand computation they were ideal: a small symmetric system. André-Louis Cholesky devised a fast way to solve such systems for geodetic surveys; it was published after his death, in 1924. When electronic computers arrived, people found that the normal equations could lose accuracy badly. Alan Turing (1948) and John von Neumann and Herman Goldstine (1947) studied rounding errors in matrix computations; Turing coined the term **condition number**. James Wilkinson’s backward error analysis (1960s) explained which algorithms are trustworthy. Alston Householder’s reflections (1958) and Gene Golub’s use of them for least squares (1965) gave the **QR** method, which works with $\mathbf{X}$ itself instead of $\mathbf{X}^\top \mathbf{X}$; Golub and William Kahan (1965) made the **singular value decomposition** (SVD) computable. The standardisation of floating-point arithmetic (IEEE 754, 1985) finally made “the same computation” mean the same thing on different machines.

Derivatives by machine (1964–1986). Training models later in this book means following gradients of losses with millions of parameters. Writing those derivatives by hand does not scale, and finite differences cost one function evaluation per parameter. *Automatic differentiation* — forward mode (Wengert, 1964) and reverse mode (Linnainmaa, 1970) — computes exact derivatives of a program; reverse mode, popularised for neural networks as *backpropagation* (Rumelhart, Hinton and Williams, 1986), returns the whole gradient of a scalar loss for roughly the cost of a few evaluations of the loss. Finite differences survived in a new role: as the *check* that a hand-derived or automatic gradient is right (section 1.5).

What this chapter cannot fix. A stable solver computes the least-squares answer accurately. It cannot make the answer *meaningful* when the data do not determine it, as with the red and green weights above. That needs a change to the objective — a penalty that prefers small or sparse weights — which is chapter 3. And a straight line cannot fit a curved relationship however well it is computed; chapter 4 changes the representation instead.

1.3 Assumptions and notation

Notation follows the notation page. The pieces used in this chapter:

Symbol	Shape	Meaning
m, n	scalars	number of examples, number of features

Symbol	Shape	Meaning
$\mathbf{x}^{(i)}$	n	features of example i (a column vector)
$\mathbf{X}$	$m \times n$	design matrix, $\mathbf{X}_{i,:} = \mathbf{x}^{(i)\top}$
$\mathbf{y}$	m	targets, $y_i = y^{(i)}$
$\mathbf{w}, b$	n , scalar	weights and intercept
$\hat{\mathbf{y}}$	m	predictions $\mathbf{X}\mathbf{w}$ (plus b)
$\mathbf{r}$	m	residual $\mathbf{y} - \hat{\mathbf{y}}$
$\ \mathbf{v}\ $	scalar	Euclidean (L^2) norm $\sqrt{\mathbf{v}^\top \mathbf{v}}$
$s_1 \geq \dots \geq s_r > 0$	scalars	nonzero singular values of $\mathbf{X}$, $r = \text{rank}(\mathbf{X})$
$\kappa(\mathbf{X})$	scalar	condition number s_1/s_r
u	scalar	unit roundoff of the floating-point format ($u \approx 1.1 \times 10^{-16}$ in float64)

We handle the intercept by appending a column of ones to $\mathbf{X}$, so that the constant becomes one more weight. When we regularise later (chapter 3) this matters, because the intercept is usually *not* penalised.

Assumptions of this chapter:

1. **A linear relationship is worth fitting.** We do not yet assume it is *true*; least squares gives the best linear approximation in the squared-error sense either way.
2. **Squared error measures what we care about.** It treats over- and under-prediction alike and penalises large errors heavily.
3. **The numbers are stored in floating point.** Every arithmetic operation may introduce a relative error up to u .

1.4 Mathematical formulation

Two pictures of a linear predictor. A linear score is $f(\mathbf{x}) = \mathbf{w}^\top \mathbf{x} + b$.

- *Row picture (one example at a time).* For classification by the sign of f , the **decision boundary** $\{\mathbf{x} : \mathbf{w}^\top \mathbf{x} + b = 0\}$ is a hyperplane. The vector $\mathbf{w}$ is perpendicular to it, and the signed distance from a point to the boundary is $(\mathbf{w}^\top \mathbf{x} + b)/\|\mathbf{w}\|$.
- *Column picture (all examples at once).* The prediction vector $\hat{\mathbf{y}} = \mathbf{X}\mathbf{w} = \sum_j w_j \mathbf{X}_{:,j}$ is a weighted sum of the columns of $\mathbf{X}$. Every possible prediction vector lies in the **column space** of $\mathbf{X}$, a subspace of $\mathbb{R}^m$ of dimension $\text{rank}(\mathbf{X}) \leq n$.

Least squares. Choose $\mathbf{w}$ to minimise

$$J(\mathbf{w}) = \frac{1}{2} \|\mathbf{X}\mathbf{w} - \mathbf{y}\|^2 = \frac{1}{2} \sum_{i=1}^m (\mathbf{x}^{(i)\top} \mathbf{w} - y^{(i)})^2.$$

In the column picture: find the point of the column space closest to $\mathbf{y}$. The factor $\frac{1}{2}$ only tidies the gradient. Whether you sum or average over examples does not change the minimiser here, but it does change the meaning of a regularisation weight later (chapter 3), so always say which one you use.

Sensitivity. We also want to know how much the minimiser moves when $\mathbf{y}$ or $\mathbf{X}$ is perturbed slightly — by measurement noise, or by rounding. The answer is governed by the condition number $\kappa(\mathbf{X})$.

1.5 Derivation, with reasons for each step

1.5.1 Step 1: a toolkit for gradients of functions of vectors

For a scalar function $f : \mathbb{R}^n \rightarrow \mathbb{R}$, the **gradient** $\nabla_{\mathbf{w}} f$ is the vector of partial derivatives $\partial f / \partial w_j$. We use the convention of the notation page: **the gradient has the same shape as the variable**. That rule alone catches most mistakes.

Four rules cover almost everything in this book. Each follows by writing the function with indices and differentiating one coordinate at a time.

1. **Linear function.** $f(\mathbf{w}) = \mathbf{a}^\top \mathbf{w} = \sum_j a_j w_j$. Then $\partial f / \partial w_k = a_k$, so $\nabla_{\mathbf{w}}(\mathbf{a}^\top \mathbf{w}) = \mathbf{a}$.
2. **Quadratic form.** $f(\mathbf{w}) = \mathbf{w}^\top \mathbf{A} \mathbf{w} = \sum_{i,j} w_i A_{i,j} w_j$. Differentiating with respect to w_k picks up the terms with $i = k$ and the terms with $j = k$: $\partial f / \partial w_k = \sum_j A_{k,j} w_j + \sum_i w_i A_{i,k}$. Hence $\nabla_{\mathbf{w}}(\mathbf{w}^\top \mathbf{A} \mathbf{w}) = (\mathbf{A} + \mathbf{A}^\top) \mathbf{w}$, which is $2\mathbf{A} \mathbf{w}$ when $\mathbf{A}$ is symmetric.
3. **Chain rule.** If $\mathbf{z} = g(\mathbf{w}) \in \mathbb{R}^k$ and $f = h(\mathbf{z})$, then $\nabla_{\mathbf{w}} f = \mathbf{J}^\top \nabla_{\mathbf{z}} h$, where $\mathbf{J} = \partial \mathbf{z} / \partial \mathbf{w} \in \mathbb{R}^{k \times n}$ is the Jacobian. *Check the shapes:* $(n \times k)(k) = (n)$, the shape of $\mathbf{w}$.
4. **Linear map inside.** If $\mathbf{z} = \mathbf{X} \mathbf{w}$, the Jacobian is $\mathbf{X}$, so $\nabla_{\mathbf{w}} h(\mathbf{X} \mathbf{w}) = \mathbf{X}^\top \nabla_{\mathbf{z}} h$.

Rule 4 is the whole of backpropagation through a linear layer (chapter 11): multiply the upstream gradient by the transpose of the matrix.

1.5.2 Step 2: the gradient of the least-squares loss

Write $J(\mathbf{w}) = h(\mathbf{z})$ with $\mathbf{z} = \mathbf{X} \mathbf{w} - \mathbf{y}$ and $h(\mathbf{z}) = \frac{1}{2} \mathbf{z}^\top \mathbf{z}$. Rule 2 with $\mathbf{A} = \frac{1}{2} \mathbf{I}$ gives $\nabla_{\mathbf{z}} h = \mathbf{z}$, and rule 4 (the constant $\mathbf{y}$ does not change the Jacobian) gives

$$\nabla_{\mathbf{w}} J = \mathbf{X}^\top (\mathbf{X} \mathbf{w} - \mathbf{y}) = -\mathbf{X}^\top \mathbf{r}.$$

Shape check: $(n \times m)(m) = (n)$. *Index check:* $\partial J / \partial w_k = \sum_i X_{i,k} (\sum_j X_{i,j} w_j - y_i)$, the k -th entry of $\mathbf{X}^\top (\mathbf{X} \mathbf{w} - \mathbf{y})$.

1.5.3 Step 3: the normal equations, and the projection they describe

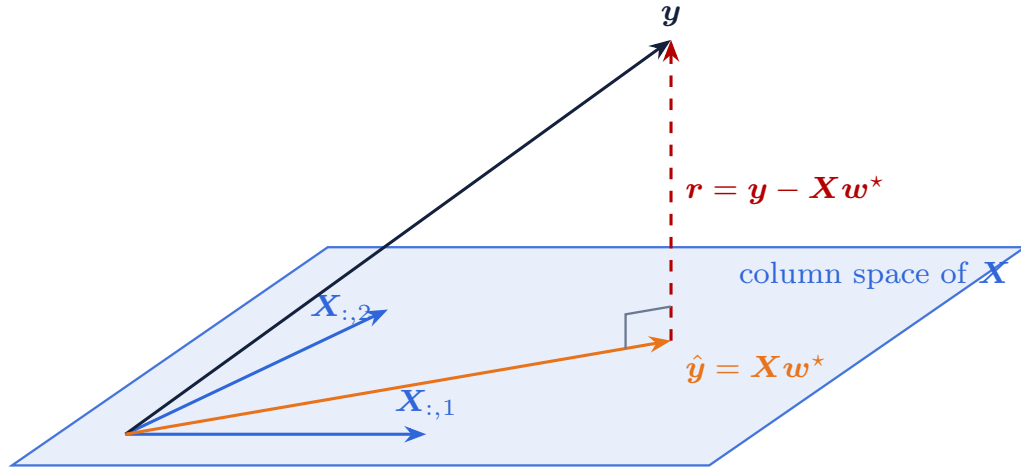
At a minimum the gradient is zero:

$$\mathbf{X}^\top \mathbf{X} \mathbf{w}^* = \mathbf{X}^\top \mathbf{y} \iff \mathbf{X}^\top \mathbf{r}^* = \mathbf{0}.$$

The right-hand form is the geometry: the residual is **perpendicular to every column** of $\mathbf{X}$, hence to the whole column space. The fitted vector $\hat{\mathbf{y}} = \mathbf{X} \mathbf{w}^*$ is the **orthogonal projection** of $\mathbf{y}$ onto that space (figure 1.1). Two consequences are worth remembering:

- **Pythagoras:** $\|\mathbf{y}\|^2 = \|\hat{\mathbf{y}}\|^2 + \|\mathbf{r}^*\|^2$, because $\hat{\mathbf{y}} \perp \mathbf{r}^*$. Chapter 2 reuses exactly this identity.
- **With an intercept column of ones, the training residuals sum to zero**, because the ones vector is one of the columns the residual is perpendicular to. That is algebra, not evidence that future predictions are unbiased.

When $\mathbf{X}$ has full column rank, $\mathbf{X}^\top \mathbf{X}$ is invertible and the projector is $\mathbf{P} = \mathbf{X}(\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top$. You can check that $\mathbf{P}^\top = \mathbf{P}$ and $\mathbf{P}^2 = \mathbf{P}$: projecting twice changes nothing.



The fitted vector $\hat{\mathbf{y}}$ is the point of the plane closest to $\mathbf{y}$, so the residual is perpendicular to every column: $\mathbf{X}^\top \mathbf{r} = \mathbf{0}$ (the normal equations).

Figure 1.1: Least squares as orthogonal projection. The column space of $\mathbf{X}$ is a plane inside $\mathbb{R}^m$; the fitted vector is the point of the plane closest to $\mathbf{y}$, and the residual meets the plane at a right angle.

1.5.4 Step 4: when is the solution unique?

The Hessian of J is $\nabla_w^2 J = \mathbf{X}^\top \mathbf{X}$ (differentiate the gradient once more). For any vector $\mathbf{v}$,

$$\mathbf{v}^\top \mathbf{X}^\top \mathbf{X} \mathbf{v} = \|\mathbf{X} \mathbf{v}\|^2 \geq 0,$$

so the Hessian is positive semidefinite and J is convex: every stationary point is a global minimum. It is positive *definite* exactly when $\mathbf{X} \mathbf{v} \neq \mathbf{0}$ for every $\mathbf{v} \neq \mathbf{0}$, that is, when the columns are linearly independent. Then the minimiser is unique.

If two columns are identical (or one is a combination of others), there is a direction $\mathbf{v}$ with $\mathbf{X} \mathbf{v} = \mathbf{0}$. Adding any multiple of $\mathbf{v}$ to a solution leaves the predictions unchanged: **the predictions are unique, the weights are not**. The SVD picks the solution of smallest norm, through the **Moore–Penrose pseudoinverse** $\mathbf{X}^+ = \mathbf{V} \mathbf{S}^{-1} \mathbf{U}^\top$ (keeping only nonzero singular values): $\mathbf{w}^* = \mathbf{X}^+ \mathbf{y}$. With two identical columns it splits the weight equally between them — a choice, not a discovery.

1.5.5 Step 5: the condition number, and why the normal equations can hurt

Write the thin SVD $\mathbf{X} = \mathbf{U} \mathbf{S} \mathbf{V}^\top$ (chapter 2 derives it in detail). The least-squares solution is

$$\mathbf{w}^* = \mathbf{V} \mathbf{S}^{-1} \mathbf{U}^\top \mathbf{y} = \sum_{j=1}^r \frac{\mathbf{U}_{:,j}^\top \mathbf{y}}{s_j} \mathbf{V}_{:,j}.$$

Read the formula as a recipe: measure how much of $\mathbf{y}$ lies along each output direction $\mathbf{U}_{:,j}$, **divide by** s_j , and use the result as the weight along the input direction $\mathbf{V}_{:,j}$. A small singular value means a direction in which the inputs barely vary, and dividing by it amplifies whatever noise lies along $\mathbf{U}_{:,j}$. The ratio between the most and least amplified directions is the **condition number** $\kappa(\mathbf{X}) = s_1/s_r$. A standard bound makes this precise: for a full-rank least-squares problem with a small residual, a relative change ϵ in $\mathbf{y}$ can change $\mathbf{w}^*$ by a relative amount up to about $\kappa(\mathbf{X}) \epsilon$.

The statistical version. If the targets carry independent noise of variance σ^2 , the fitted weights have covariance $\text{Cov}(\mathbf{w}^*) = \sigma^2 (\mathbf{X}^\top \mathbf{X})^{-1} = \sigma^2 \mathbf{V} \mathbf{S}^{-2} \mathbf{V}^\top$. Along the direction with the smallest singular value the standard deviation is σ/s_r . For two standardised features with correlation ρ and m examples, the weight on each has standard deviation about $\sigma/\sqrt{m(1-\rho^2)}$, which blows up as $\rho \rightarrow 1$. *This is the red-green problem of section 1.1: the data contain little information about the difference between the two weights.*

The numerical version. Forming $\mathbf{X}^\top \mathbf{X} = \mathbf{V} \mathbf{S}^2 \mathbf{V}^\top$ squares every singular value, so $\kappa(\mathbf{X}^\top \mathbf{X}) = \kappa(\mathbf{X})^2$. In float64, with $u \approx 10^{-16}$, a problem with $\kappa(\mathbf{X}) = 10^8$ is still solvable from $\mathbf{X}$ (error around $\kappa u = 10^{-8}$) but hopeless through $\mathbf{X}^\top \mathbf{X}$ (error around $\kappa^2 u = 1$). **QR** avoids the squaring: with $\mathbf{X} = \mathbf{Q} \mathbf{R}$, $\mathbf{Q}$ with orthonormal columns and $\mathbf{R}$ upper triangular, the normal equations become $\mathbf{R}^\top \mathbf{R} \mathbf{w} = \mathbf{R}^\top \mathbf{Q}^\top \mathbf{y}$, and cancelling the invertible $\mathbf{R}^\top$ leaves the triangular system $\mathbf{R} \mathbf{w} = \mathbf{Q}^\top \mathbf{y}$, whose condition number is $\kappa(\mathbf{X})$.

Keep the two versions apart. A better *solver* fixes the numerical problem. Only more or different *data*, or a different *objective* (chapter 3), fixes the statistical one.

1.5.6 Step 6: floating point, and the log-sum-exp trick

A float64 number has about 16 significant digits; numbers above about 1.8×10^{308} overflow to infinity. Two habits prevent most numerical failures:

- **Do not subtract nearly equal numbers if you can avoid it** (catastrophic cancellation: the leading digits cancel and only rounding error remains).
- **Shift before exponentiating.** For any constant c ,

$$\log \sum_j \exp(a_j) = c + \log \sum_j \exp(a_j - c).$$

Proof: factor $\exp(c)$ out of the sum. Choosing $c = \max_j a_j$ makes every exponent ≤ 0 , so nothing overflows and at least one term equals 1, so nothing underflows to $\log 0$. This identity, **log-sum-exp**, underlies every stable softmax and cross-entropy in this book (chapters 3, 14 and appendix C).

1.5.7 Step 7: checking gradients with finite differences

Every derivation in this book can be checked numerically. The **central difference**

$$\frac{\partial J}{\partial w_j} \approx \frac{J(\mathbf{w} + \epsilon \mathbf{e}^{(j)}) - J(\mathbf{w} - \epsilon \mathbf{e}^{(j)})}{2\epsilon}$$

has two error sources that pull in opposite directions:

- **Truncation.** Taylor-expand both evaluations: the even terms cancel and the error is $\frac{\epsilon^2}{6} \partial^3 J / \partial w_j^3 + O(\epsilon^4)$ — it *shrinks* as ϵ^2 . The one-sided **forward difference** $(J(\mathbf{w} + \epsilon \mathbf{e}^{(j)}) - J(\mathbf{w})) / \epsilon$ keeps the $\frac{\epsilon}{2} \partial^2 J / \partial w_j^2$ term and shrinks only as ϵ .
- **Rounding.** Each evaluation of J carries a relative error of about u ; dividing by ϵ magnifies it to about $u|J|/\epsilon$ — it *grows* as ϵ shrinks.

Minimising the sum of the two gives the best step: for forward differences $\epsilon \sim \sqrt{u} \approx 10^{-8}$ with error $\sim \sqrt{u} \approx 10^{-8}$; for central differences $\epsilon \sim u^{1/3} \approx 5 \times 10^{-6}$ with error $\sim u^{2/3} \approx 10^{-11}$. Section 1.8 measures both, and they match.

A gradient check compares the analytic gradient $\mathbf{g}$ with the numerical one $\tilde{\mathbf{g}}$ through a **relative error** $\|\mathbf{g} - \tilde{\mathbf{g}}\| / \max(\|\mathbf{g}\|, \|\tilde{\mathbf{g}}\|)$. In float64 with central differences, values below about 10^{-7} mean “correct”; values around 10^{-2} mean a real bug.

1.5.8 Step 8: distance to a hyperplane

For completeness, the row picture. Let $\mathbf{x}_0$ be any point. Its closest point on the hyperplane $\mathbf{w}^\top \mathbf{x} + b = 0$ is reached by moving along the normal $\mathbf{w}$: $\mathbf{x}_0 - t\mathbf{w}$ for the t that solves $\mathbf{w}^\top (\mathbf{x}_0 - t\mathbf{w}) + b = 0$, i.e. $t = (\mathbf{w}^\top \mathbf{x}_0 + b) / \|\mathbf{w}\|^2$. The distance moved is $t\|\mathbf{w}\| = (\mathbf{w}^\top \mathbf{x}_0 + b) / \|\mathbf{w}\|$, signed by the side of the boundary. Multiplying $\mathbf{w}$ and b by any positive constant changes the *score* but not the boundary or the distance: **scores are not distances, and they are not probabilities** until we choose a scale (chapter 3).

1.6 Numerical worked example

A projection by hand. Fit $y = wx$ (no intercept) to $\mathbf{x} = (1, 2)^\top$, $\mathbf{y} = (1, 3)^\top$. Here $\mathbf{X} = \mathbf{x}$ is a single column, so $\mathbf{X}^\top \mathbf{X} = 1 + 4 = 5$ and $\mathbf{X}^\top \mathbf{y} = 1 + 6 = 7$, giving $w^* = 7/5 = 1.4$. The predictions are $(1.4, 2.8)$, the residuals $\mathbf{r} = (-0.4, 0.2)$, and $\mathbf{X}^\top \mathbf{r} = -0.4 + 0.4 = 0$ — perpendicular, as step 3 promised. The squared error is $0.16 + 0.04 = 0.20$, and Pythagoras holds: $\|\mathbf{y}\|^2 = 10 = \|\hat{\mathbf{y}}\|^2 + \|\mathbf{r}\|^2 = 9.8 + 0.2$.

A gradient by hand. At $w = 1$, the gradient is $\mathbf{X}^\top (\mathbf{X}w - \mathbf{y}) = (1, 2) \cdot (0, -1) = -2$. A central difference with $\epsilon = 0.1$: $J(1.1) = \frac{1}{2}[(0.1)^2 + (-0.8)^2] = 0.325$ and $J(0.9) = \frac{1}{2}[(-0.1)^2 + (-1.2)^2] = 0.725$, so $(0.325 - 0.725)/0.2 = -2.0$, exact because J is quadratic (its third derivative is zero).

A sensitivity by hand. Take $\mathbf{X} = \text{diag}(1, 10^{-6})$ and $\mathbf{y} = (1, 10^{-6})^\top$, so $\mathbf{w}^* = (1, 1)^\top$. Change the second target by only 10^{-6} : the second weight becomes 2. The condition number is 10^6 : the data say almost nothing about w_2 . No solver can fix that.

A distance by hand. For $\mathbf{w} = (1, -1)^\top$, $b = 0$, the boundary is $x_1 = x_2$. The point $(3, 1)$ has score 2 and signed distance $2/\sqrt{2} = \sqrt{2} \approx 1.41$. Doubling $\mathbf{w}$ doubles the score to 4 but leaves the distance at $\sqrt{2}$.

Log-sum-exp by hand. $\log(e^{1000} + e^{1000})$ overflows if computed directly. With $c = 1000$: $1000 + \log(1 + 1) = 1000.693$.

1.7 Small implementation and checks

The lab is `labs/ch01-foundations/foundations.py`. It implements three solvers side by side:

```
def solve_normal(X, y):    # squares the condition number
    return np.linalg.solve(X.T @ X, X.T @ y)
```

```
def solve_qr(X, y):          # works with X itself
    Q, R = np.linalg.qr(X)
    return np.linalg.solve(R, Q.T @ y)

def solve_svd(X, y):         # minimum-norm solution via the pseudoinverse
    U, s, Vt = np.linalg.svd(X, full_matrices=False)
    keep = s > 1e-12 * s[0]
    return Vt[keep].T @ ((U[:, keep].T @ y) / s[keep])
```

In practice, `np.linalg.lstsq` (an SVD-based solver) is the right default; the lab writes the three out so you can compare them. The tests in `test_foundations.py` check what the derivation predicts:

- the residual is orthogonal to every column ($\mathbf{X}^\top \mathbf{r} \approx \mathbf{0}$);
- on a well-conditioned problem the three solvers agree;
- with two identical columns the SVD solution splits the weight equally (minimum norm);
- the analytic gradient matches central differences;
- the finite-difference error has an interior optimum in ϵ ;
- as two features become collinear, $\kappa(\mathbf{X})$ grows and so does the spread of the fitted weights;
- on the Lauchli matrix the normal equations fail while QR succeeds;
- log-sum-exp is finite and correct at ± 1000 .

1.8 Experiment and expected observations

Run `python labs/ch01-foundations/foundations.py` (seed 0, about one second).

Experiment 1: the gradient check. On a random least-squares problem with $m = 30$, $n = 4$, central differences at $\epsilon = 10^{-6}$ agree with $\mathbf{X}^\top(\mathbf{X}\mathbf{w} - \mathbf{y})$ to a relative error of 2.5×10^{-10} .

Experiment 2: choosing the step size. On a non-quadratic loss (a sum of log cosh terms, so that truncation error is visible), sweep ϵ from 10^{-12} to 10^{-1} :

The best forward step is $\epsilon = 3 \times 10^{-8}$ with error 2.9×10^{-8} ; the best central step is $\epsilon = 3 \times 10^{-6}$ with error 2.0×10^{-11} . Both match step 7’s predictions ($\sqrt{u} \approx 10^{-8}$ and $u^{1/3} \approx 5 \times 10^{-6}$). The slopes of the two right-hand branches are 1 and 2 on the log-log plot, the orders of the two truncation errors. **Lesson:** “smaller ϵ is more accurate” is false.

Experiment 3: collinear features. Two standardised features with correlation ρ plus an intercept, $m = 200$, noise standard deviation 0.1, true weights (1, 2, -1):

ρ	$\kappa(\mathbf{X})$	$\kappa(\mathbf{X}^\top \mathbf{X})$	normal vs. QR	error of QR fit	sd of weights
0	1.1	1.3	1.6×10^{-15}	0.009	0.007
0.9	4.0	16	1.7×10^{-15}	0.015	0.016
0.99	13.1	173	9.2×10^{-16}	0.055	0.048
0.999	41.9	1750	1.1×10^{-12}	0.183	0.153
0.9999	132.8	1.8×10^4	2.9×10^{-12}	0.590	0.482
0.99999	420.1	1.8×10^5	2.9×10^{-11}	1.877	1.525

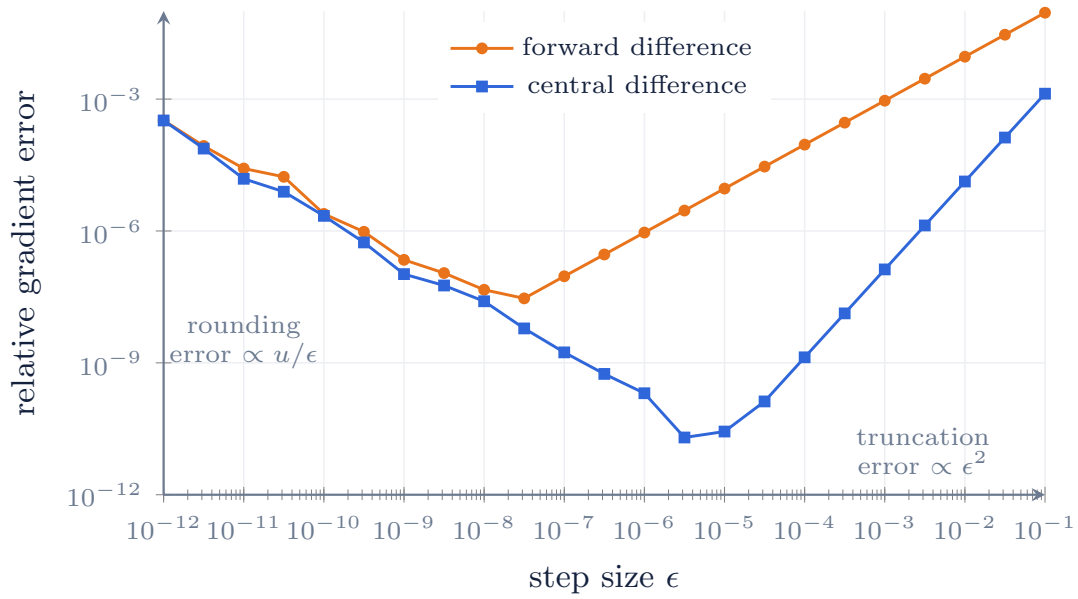
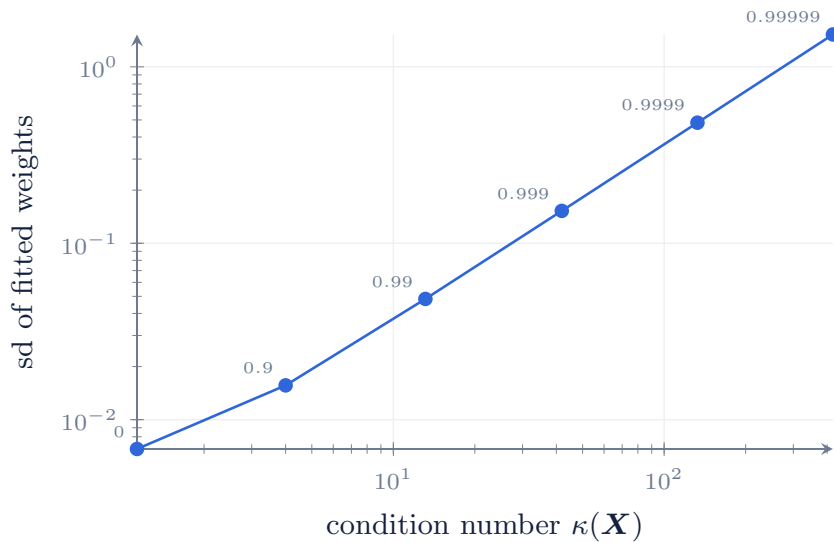


Figure 1.2: Relative error of finite-difference gradients against the step size. Left of each minimum, rounding error ($\propto u/\epsilon$) dominates; right of it, truncation error ($\propto \epsilon$ for forward, $\propto \epsilon^2$ for central differences) dominates.



Labels: correlation ρ between the two features; sd over 20 label-noise draws.

Figure 1.3: The spread of fitted weights over 20 redraws of the label noise grows in proportion to the condition number. The two solvers agree throughout: the instability is statistical, not numerical.

What to notice:

1. **The two solvers agree** to 10^{-11} or better in every row. At these condition numbers floating point is not the problem.
2. **The weights become unreliable anyway.** At $\rho = 0.99999$ the fitted weights are off by about 1.9 and vary by 1.5 between noise redraws, although the true weights are 1, 2, -1 . Step 5's formula predicts the spread: $0.1/\sqrt{200(1-0.99999^2)} \approx 1.58$, against 1.53 measured.
3. **The spread is proportional to $\kappa(\mathbf{X})$:** ten times the condition number, ten times the spread (figure 1.3).

Experiment 4: when the solver does matter. The Läuchli matrix $\mathbf{X} = \begin{pmatrix} 1 & 1 \\ \delta & 0 \\ 0 & \delta \end{pmatrix}$ with $\mathbf{y} = \mathbf{X}(1, 1)^\top$ has no noise at all. With $\delta = 10^{-4}$ ($\kappa(\mathbf{X}) = 1.4 \times 10^4$) the normal equations lose six digits (error 7.8×10^{-10}) while QR is exact to 3×10^{-16} . With $\delta = 10^{-8}$ ($\kappa = 1.4 \times 10^8$), $1 + \delta^2$ rounds to 1, $\mathbf{X}^\top \mathbf{X}$ becomes exactly singular in float64, and the normal equations fail; QR and SVD still return (1, 1).

1.9 Moving pieces: diagnosing and fixing

Moving piece	Symptom when it is wrong	How to confirm	Fix
Feature scale	one weight tiny, another huge; regularisation (chapter 3) hits features unevenly	print column standard deviations	standardise with <i>training</i> statistics
Intercept	residuals not centred; predictions offset	is there a column of ones? is it penalised?	append a ones column; exclude it from penalties
Collinearity	weights swing between fits, predictions stable	$\kappa(\mathbf{X})$; correlation matrix; bootstrap the weights	more varied data; drop or combine features; ridge (chapter 3); report predictions, not weights
Solver	“singular matrix”, NaN, or results that differ between libraries	compare <code>lstsq</code> with your solver; compute $\kappa(\mathbf{X})$	QR or SVD (<code>lstsq</code>); never <code>inv(X.T @ X)</code>
Rank deficiency	infinitely many solutions; solver picks one silently	count singular values above $10^{-12}s_1$	minimum-norm solution; say which one you report
Precision	results change between float32 and float64	rerun in float64	float64 for small linear algebra; scale inputs
Sum vs. mean loss	a regularisation weight from one codebase does not transfer	duplicate every example: does the fit change?	write the full objective down

Moving piece	Symptom when it is wrong	How to confirm	Fix
Finite-difference step	gradient check “fails” on a correct gradient	sweep ϵ (figure 1.2)	central differences, $\epsilon \approx 10^{-6}$ in float64, relative error
Overflow in exponentials	<code>inf</code> or <code>nan</code> in losses	find the largest exponent	log-sum-exp; work in log space

First things to try when a linear fit misbehaves: (1) print the shapes and the first rows of $\mathbf{X}$ and $\mathbf{y}$; (2) compute $\kappa(\mathbf{X})$; (3) solve with `np.linalg.lstsq`; (4) refit on bootstrap resamples and look at the spread of each weight; (5) plot residuals against predictions and against each feature.

1.10 Application, limitations and failure cases

Application (educational case study). Image-derived regression — chlorophyll from leaf colour, biomass from vegetation indices in remote sensing, age from simple face measurements — usually starts with a linear baseline. Correlated inputs are the norm, because many measurements respond to the same lighting, camera or background. A careful baseline (1) splits the data first, (2) standardises with training statistics, (3) solves with QR or SVD, (4) reports held-out error, and (5) plots residuals against predictions (curvature suggests nonlinearity; a funnel shape suggests noise that grows with the target). Neither residual pattern is conclusive from one plot, but each gives a testable next question.

Failure cases to recognise:

- **Interpreting weights causally.** With correlated features, a weight measures the effect of a feature *holding the others fixed* — a situation the data may never contain. Predictions can be trustworthy while individual weights are not.
- **Training-set algebra mistaken for evidence.** Residuals that sum to zero, or are orthogonal to the features, say nothing about future data.
- **Extrapolation.** A linear fit is a local approximation. Outside the range of the training inputs it can be arbitrarily wrong.
- **Outliers.** Squared error makes one wildly wrong target pull the whole fit. Look at the largest residuals before trusting a fit.

1.11 Bridge to a research paper

Baydin, Pearlmutter, Radul and Siskind (2018), *Automatic differentiation in machine learning: a survey* (JMLR 18(153)).

- **Problem:** how to obtain exact derivatives of programs efficiently.
- **Inputs and outputs:** a program computing $f : \mathbb{R}^n \rightarrow \mathbb{R}^k$; its Jacobian-vector or vector-Jacobian products.
- **Central claim:** reverse mode computes the gradient of a scalar function at a cost that is a small constant multiple of evaluating the function, *independent of n* , whereas finite differences need $n + 1$ (forward) or $2n$ (central) evaluations and are only approximate.
- **Assumptions to question:** memory — reverse mode stores intermediate values; the constant factor; non-differentiable points such as ReLU at zero, where “the” derivative is a convention.

- **Affordable experiment:** for least squares with $n \in \{10, 100, 1000\}$, time the analytic gradient $\mathbf{X}^\top(\mathbf{X}\mathbf{w} - \mathbf{y})$ against central differences. Plot time against n . Then compare both with a reverse-mode library (PyTorch or JAX) if you have one, and check all three agree to 10^{-8} .

1.12 Memory map



Figure 1.4: Memory map of chapter 1.

Summary. A linear predictor is a hyperplane in input space and a point in the column space of $\mathbf{X}$. Least squares projects $\mathbf{y}$ onto that space, so the residual is perpendicular to every column; that single fact gives the normal equations, Pythagoras for errors, and uniqueness exactly when the columns are independent. How much the solution can be trusted is measured by the condition number: it controls both the statistical spread of the weights (fix with data or penalties) and the numerical error of the solver (fix with QR or SVD rather than $\mathbf{X}^\top \mathbf{X}$). Gradients follow four rules, keep the shape of the variable,

and are checked with central differences at a step chosen to balance truncation against rounding.

1.13 Exercises

1. *Conceptual.* Show that $\mathbf{A}^\top \mathbf{A}$ is always positive semidefinite. When is it positive definite?
2. *Conceptual.* For $\mathbf{w} = (2, 0)^\top$, $b = -4$, give the decision boundary and the distance of $(3, 5)$ from it.
3. *Derivation.* Show that $\mathbf{P} = \mathbf{X}(\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top$ is symmetric and idempotent, and that $\mathbf{I} - \mathbf{P}$ projects onto the orthogonal complement of the column space.
4. *Derivation.* Derive $\nabla_{\mathbf{W}} \frac{1}{2} \|\mathbf{X}\mathbf{W} - \mathbf{Y}\|_F^2$ for a weight *matrix* $\mathbf{W} \in \mathbb{R}^{n \times k}$ (several outputs at once), and check its shape.
5. *Derivation.* Show that with an intercept column the training residuals sum to zero. Give an example where the *test* residuals do not.
6. *Implementation.* Add a ridge solver (chapter 3 preview): $(\mathbf{X}^\top \mathbf{X} + \lambda \mathbf{I})\mathbf{w} = \mathbf{X}^\top \mathbf{y}$. Rerun experiment 3 with $\lambda = 1$ and compare the spread of the weights at $\rho = 0.99999$.
7. *Implementation.* Time the analytic gradient against central differences for $n \in \{10, 100, 1000\}$ (the experiment of section 1.11).
8. *Interpretation.* A training loss decreases, but the weights change dramatically after tiny changes to the labels. Give two hypotheses and the measurement that would distinguish them.
9. *Failure analysis.* Duplicate every training example. What happens to the least-squares solution? To a ridge solution with a fixed summed-loss λ ? What does this tell you about comparing regularisation weights across codebases?
10. *Open question.* Image features measured under varying light are often correlated. Propose an experiment that separates “the model needs more capacity” from “the data do not identify the weights”.

1.13.1 Answer discussions

1. $\mathbf{v}^\top \mathbf{A}^\top \mathbf{A} \mathbf{v} = \|\mathbf{A}\mathbf{v}\|^2 \geq 0$. Positive definite exactly when $\mathbf{A}$ has a trivial null space (independent columns).
2. The boundary is $2x_1 - 4 = 0$, the line $x_1 = 2$. Distance $(2 \cdot 3 - 4)/2 = 1$.
3. $\mathbf{P}^\top = \mathbf{X}(\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top = \mathbf{P}$ since $\mathbf{X}^\top \mathbf{X}$ is symmetric; and $\mathbf{P}^2 = \mathbf{P}$, because the middle factor $(\mathbf{X}^\top \mathbf{X})^{-1}(\mathbf{X}^\top \mathbf{X})$ in $\mathbf{P}^2$ is the identity. $(\mathbf{I} - \mathbf{P})\mathbf{v}$ satisfies $\mathbf{X}^\top (\mathbf{I} - \mathbf{P})\mathbf{v} = \mathbf{0}$.
4. $\mathbf{X}^\top (\mathbf{X}\mathbf{W} - \mathbf{Y})$, of shape $n \times k$ like $\mathbf{W}$; each output column is an independent least-squares problem.
5. The ones vector is a column, so $\mathbf{1}^\top \mathbf{r} = 0$. Test residuals need not sum to zero: fit on data with mean target 0, test on data with mean target 5.
6. The spread falls sharply (ridge damps the $1/s_r$ amplification); the fitted weights move towards splitting the effect between the two features — a bias you have chosen.
7. Central differences scale linearly in n (each evaluation costs $O(mn)$, so $O(mn^2)$ in total); the analytic gradient is $O(mn)$.
8. Ill-conditioning (check $\kappa(\mathbf{X})$ and bootstrap the weights) versus a label or pipeline bug (check that labels are aligned and the loss decreases on a fixed tiny set). Lower training loss alone does not distinguish them.
9. Unregularised least squares is unchanged (both sides of the normal equations double). With a fixed summed-loss ridge λ , the data term doubles but the penalty does not, so the effective regularisation halves. Regularisation weights are meaningless without

the full objective.

10. Fix the model and vary the data: refit on bootstrap resamples (weight spread reveals identifiability) and compare held-out error of linear and more flexible models on the same split (a gap reveals missing capacity).

1.14 Self-check and common mistakes

- ☐ I can draw figure 1.1 and explain why the residual is perpendicular to the columns.
- ☐ I can derive $\nabla_{\mathbf{w}} \frac{1}{2} \|\mathbf{X}\mathbf{w} - \mathbf{y}\|^2$ and check its shape.
- ☐ I know why $\kappa(\mathbf{X}^\top \mathbf{X}) = \kappa(\mathbf{X})^2$ and what that means in float64.
- ☐ I can tell a statistical instability from a numerical one, and name the fix for each.
- ☐ I can choose a finite-difference step and justify it.

Common mistakes: inverting $\mathbf{X}^\top \mathbf{X}$ explicitly; reading weights causally when features are correlated; forgetting the intercept or penalising it; comparing regularisation weights without checking sum vs. mean; making the finite-difference step as small as possible; absolute instead of relative error in gradient checks; checking a gradient only at zero, where many terms vanish.

References

- I. Goodfellow, Y. Bengio and A. Courville. *Deep Learning*. MIT Press, 2016, chapter 2 (linear algebra) and section 4.1 (numerical computation).
- G. H. Golub and C. F. Van Loan. *Matrix Computations*, 4th edition. Johns Hopkins University Press, 2013, chapter 5 (least squares).
- N. J. Higham. *Accuracy and Stability of Numerical Algorithms*, 2nd edition. SIAM, 2002.
- A. G. Baydin, B. A. Pearlmutter, A. A. Radul and J. M. Siskind. Automatic differentiation in machine learning: a survey. *JMLR*, 18(153):1–43, 2018.
- A. M. Turing. Rounding-off errors in matrix processes. *Quarterly Journal of Mechanics and Applied Mathematics*, 1:287–308, 1948.
- G. Golub. Numerical methods for solving linear least squares problems. *Numerische Mathematik*, 7:206–216, 1965.
- S. M. Stigler. Gauss and the invention of least squares. *Annals of Statistics*, 9(3):465–474, 1981.

Chapter 2

SVD and PCA

Prerequisites: chapter 1 (projections, orthonormal bases, gradients); eigenvalues of symmetric matrices (appendix A). No CNN knowledge is required.

Learning outcomes: after this chapter you can

1. state the two equivalent goals of PCA (maximum retained variance, minimum reconstruction error) and the assumptions behind them;
2. derive the principal directions as eigenvectors of the sample covariance and compute them stably from the SVD of the centred design matrix;
3. state the Eckart–Young theorem and use singular values to predict reconstruction error before looking at any image;
4. choose the number of components with held-out error, a noise threshold and the downstream task, and explain why “keep 95% of the variance” can mislead;
5. diagnose the common failures: unscaled features, outliers, leakage, nonlinear structure, and variance that is not relevance.

2.1 A concrete problem

A 12×12 greyscale image is a point in $\mathbb{R}^{144}$. Suppose we have 800 such images and suspect they are built from only a handful of recurring patterns plus sensor noise. Two practical questions follow:

- **Compression.** Can each image be stored with $l \ll 144$ numbers and rebuilt with small error?
- **Understanding.** What are the recurring patterns, and how many are there?

Storing a few coordinates in a well-chosen basis answers both. We need a principled way to choose that basis from data. Picking pixels to keep does not work: every pattern touches many pixels. We need *directions* in pixel space, not pixels.

The same question appears everywhere in vision: summarising the correlated bands of a multispectral satellite image, finding the main modes of variation in aligned faces, reducing 768-dimensional network features before a nearest-neighbour search, or simply *looking at* a high-dimensional dataset in two dimensions.

2.2 How we got here

Lines of closest fit (1901). Karl Pearson asked a geometric question: given a cloud of points, which line or plane passes closest to them, measuring distance *perpendicular* to the

line rather than vertically as in least squares? His answer was the direction of greatest spread. Least squares (chapter 1) treats one variable as the target and the others as exact; Pearson’s fit treats all coordinates symmetrically. That is what you want when no variable is “the answer” — every pixel is just a measurement.

Principal components (1933). Harold Hotelling, analysing batteries of psychological test scores, asked a statistical question: which few combinations of the scores capture most of their variance? He named them *principal components* and showed they are eigenvectors of the covariance matrix. Pearson’s geometry and Hotelling’s statistics turn out to be the same problem (section 2.5, step 1).

Best low-rank approximation (1936). Carl Eckart and Gale Young, also motivated by psychometrics, proved that truncating the singular value decomposition gives the best approximation of a matrix by one of lower rank. The SVD itself had been found by Eugenio Beltrami (1873) and Camille Jordan (1874), but computing it reliably had to wait for Golub and Kahan (1965). Signal processing rediscovered the same idea as the Karhunen–Loève transform.

Faces (1987–1991). Lawrence Sirovich and Michael Kirby represented aligned face images with a few dozen principal components, and introduced the “snapshot” trick of computing them from the small $m \times m$ matrix of image inner products when there are far more pixels than images. Matthew Turk and Alex Pentland (1991) used these “eigenfaces” for recognition — the paper bridge of this chapter.

What broke, and what came next. PCA is linear, so curved structure needs many components (kernel PCA, chapter 4; autoencoders and VAEs, chapter 10). It has no noise model, so it cannot say how uncertain a code is (probabilistic PCA, Tipping and Bishop, 1999 — chapters 6 and 10). It is sensitive to outliers because it is built on squared error (robust PCA). And it is unsupervised: the directions of greatest variance need not be the ones a task needs (chapters 3 and 18). Each of these limitations is the starting point of a later idea.

2.3 Assumptions and notation

Notation follows the book-wide conventions (see the notation page): m examples, n features, and the example $\mathbf{x}^{(i)}$ in row i of the design matrix.

Symbol	Shape	Meaning
m	scalar	number of images (examples)
n	scalar	number of pixels per image (features)
$\mathbf{x}^{(i)}$	n	the i -th image, flattened (a column vector)
$\mathbf{X}$	$m \times n$	design matrix, $\mathbf{X}_{i,:} = \mathbf{x}^{(i)\top}$
$\boldsymbol{\mu}$	n	sample mean $\frac{1}{m} \sum_i \mathbf{x}^{(i)}$
$\tilde{\mathbf{x}}^{(i)}$	n	centred example $\mathbf{x}^{(i)} - \boldsymbol{\mu}$
$\mathbf{X}_c$	$m \times n$	centred design matrix, rows $\tilde{\mathbf{x}}^{(i)\top}$
$\boldsymbol{\Sigma}$	$n \times n$	sample covariance $\frac{1}{m-1} \mathbf{X}_c^\top \mathbf{X}_c$
$\lambda_1 \geq \dots \geq \lambda_n$	scalars	eigenvalues of $\boldsymbol{\Sigma}$

Symbol	Shape	Meaning
l	scalar	number of components (code size)
$\mathbf{D}$	$n \times l$	decoding matrix with orthonormal columns, $\mathbf{D}^\top \mathbf{D} = \mathbf{I}_l$
$\mathbf{c}^{(i)}$	l	code of image i : $\mathbf{c}^{(i)} = \mathbf{D}^\top \tilde{\mathbf{x}}^{(i)}$
$\mathbf{C}$	$m \times l$	all codes, $\mathbf{C} = \mathbf{X}_c \mathbf{D}$

The letters $\mathbf{D}$, $\mathbf{c}$ and l are the ones the Deep Learning book uses for PCA (Goodfellow et al., 2016, section 2.12).

Assumptions, stated so that we can later check them:

1. **Linearity.** The structure of interest is well described by an affine subspace $\boldsymbol{\mu} + \text{span}(\mathbf{D})$, the span of the columns of $\mathbf{D}$ shifted by the mean.
2. **Variance means signal.** Directions with large variance carry the information we care about; small-variance directions are mostly noise.
3. **Squared error is the right loss.** Errors in all pixels are equally important and penalised quadratically. This also makes PCA sensitive to feature scale and to outliers.
4. **Examples are exchangeable.** The mean and covariance estimated from the training images describe the images we will encode later.

2.4 Mathematical formulation

Reconstruction view. Encode $\mathbf{c}^{(i)} = \mathbf{D}^\top \tilde{\mathbf{x}}^{(i)}$ and decode $r(\mathbf{x}^{(i)}) = \boldsymbol{\mu} + \mathbf{D}\mathbf{c}^{(i)}$. **Principal component analysis (PCA)** chooses $\mathbf{D}$ to minimise

$$J(\mathbf{D}) = \frac{1}{m} \sum_{i=1}^m \|\tilde{\mathbf{x}}^{(i)} - \mathbf{D}\mathbf{D}^\top \tilde{\mathbf{x}}^{(i)}\|^2 \quad \text{subject to } \mathbf{D}^\top \mathbf{D} = \mathbf{I}_l.$$

$\mathbf{P} = \mathbf{D}\mathbf{D}^\top$ is the orthogonal projector onto the chosen subspace (chapter 1). This is Pearson's view: the subspace closest to the points.

Variance view. Choose orthonormal directions that retain as much variance of the projected data as possible:

$$\max_{\mathbf{D}^\top \mathbf{D} = \mathbf{I}_l} \text{Tr}(\mathbf{D}^\top \boldsymbol{\Sigma} \mathbf{D}).$$

This is Hotelling's view. **Matrix view.** Find the rank- l matrix closest to $\mathbf{X}_c$ in Frobenius norm. This is Eckart and Young's view. Section 2.5 shows all three have the same solution.

2.5 Derivation, with reasons for each step

Step 1: the reconstruction and variance views are equivalent. Because $\mathbf{P}$ is an orthogonal projector, $\tilde{\mathbf{x}}^{(i)} = \mathbf{P}\tilde{\mathbf{x}}^{(i)} + (\mathbf{I} - \mathbf{P})\tilde{\mathbf{x}}^{(i)}$ with orthogonal parts, so by Pythagoras (as in section 1.5)

$$\|\tilde{\mathbf{x}}^{(i)}\|^2 = \|\mathbf{P}\tilde{\mathbf{x}}^{(i)}\|^2 + \|(\mathbf{I} - \mathbf{P})\tilde{\mathbf{x}}^{(i)}\|^2.$$

The left side does not depend on $\mathbf{D}$. Minimising the residual $\sum_i \|(\mathbf{I} - \mathbf{P})\tilde{\mathbf{x}}^{(i)}\|^2$ is therefore the same as maximising the retained $\sum_i \|\mathbf{P}\tilde{\mathbf{x}}^{(i)}\|^2 = \sum_i \|\mathbf{D}^\top \tilde{\mathbf{x}}^{(i)}\|^2 = (m-1) \text{Tr}(\mathbf{D}^\top \Sigma \mathbf{D})$. *Why this matters:* “compress well” and “keep the most variance” are one objective, so we may use whichever is easier to analyse.

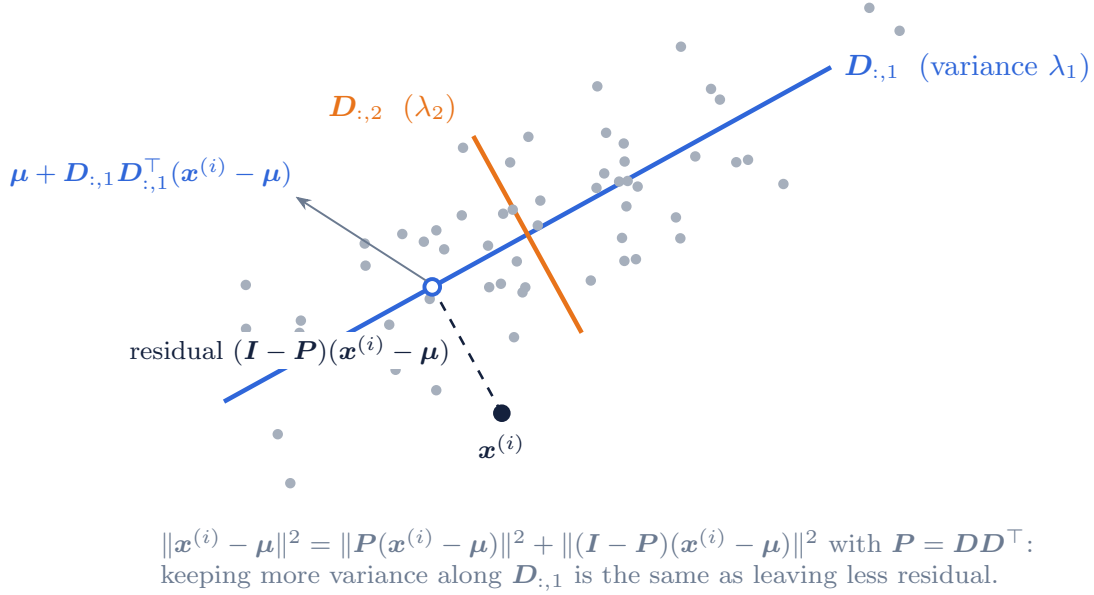


Figure 2.1: Sixty points from a correlated 2-D Gaussian with their principal axes, drawn to ± 2 standard deviations. The dashed segment is the residual of one point after projection onto the first principal direction; step 1 trades this residual against retained variance.

Step 2: one direction. For $l = 1$ with unit vector $\mathbf{d}$, maximise $\mathbf{d}^\top \Sigma \mathbf{d}$ subject to $\mathbf{d}^\top \mathbf{d} = 1$. The Lagrangian is $\mathcal{L}(\mathbf{d}, \lambda) = \mathbf{d}^\top \Sigma \mathbf{d} - \lambda(\mathbf{d}^\top \mathbf{d} - 1)$. Setting the gradient to zero (rule 2 of section 1.5: $\nabla_{\mathbf{d}} \mathbf{d}^\top \Sigma \mathbf{d} = 2\Sigma \mathbf{d}$ for symmetric Σ) gives

$$\Sigma \mathbf{d} = \lambda \mathbf{d}.$$

So every stationary point is an eigenvector, and its objective value is $\mathbf{d}^\top \Sigma \mathbf{d} = \lambda$: **the variance along an eigenvector is its eigenvalue**. The maximum is the eigenvector with the **largest** eigenvalue. *Why the constraint is necessary:* without $\|\mathbf{d}\| = 1$ the objective grows without bound by scaling $\mathbf{d}$.

Step 3: l directions. Σ is symmetric positive semidefinite (because $\mathbf{v}^\top \Sigma \mathbf{v}$ is the sample variance of $\mathbf{v}^\top \tilde{\mathbf{x}}$, which cannot be negative), so $\Sigma = \mathbf{Q}\mathbf{\Lambda}\mathbf{Q}^\top$ with orthonormal eigenvectors and $\lambda_1 \geq \dots \geq \lambda_n \geq 0$. Write any candidate $\mathbf{D}$ in this eigenbasis as $\mathbf{D} = \mathbf{Q}\mathbf{B}$ with $\mathbf{B}^\top \mathbf{B} = \mathbf{I}_l$. Then

$$\text{Tr}(\mathbf{D}^\top \Sigma \mathbf{D}) = \text{Tr}(\mathbf{B}^\top \mathbf{\Lambda} \mathbf{B}) = \sum_{j=1}^n \lambda_j w_j, \quad w_j = \|\mathbf{B}_{j,:}\|^2.$$

The weights w_j lie in $[0, 1]$ (each is the squared length of the projection of a unit basis vector onto the subspace) and sum to l (the squared Frobenius norm of $\mathbf{B}$). The sum $\sum_j \lambda_j w_j$ is

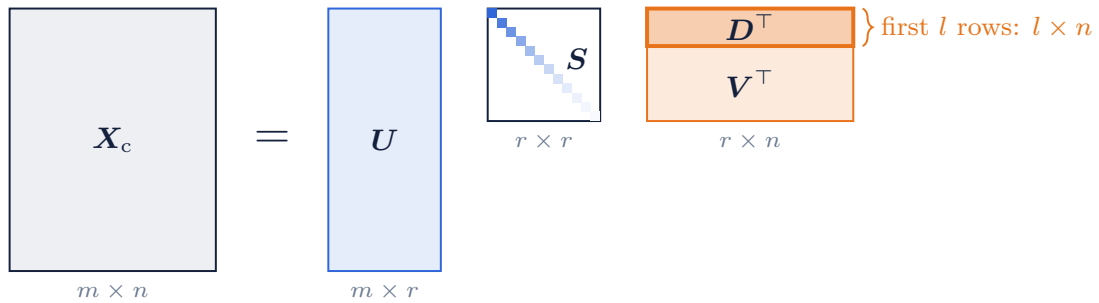
largest when the weight 1 goes to the l largest eigenvalues, giving $\lambda_1 + \dots + \lambda_l$ (Ky Fan's inequality), attained by the top l eigenvectors. These are the **principal directions**, and the coordinates $c_j^{(i)}$ are the **principal components**. The optimal subspace is unique when $\lambda_l > \lambda_{l+1}$. The columns of $\mathbf{D}$ are determined only up to rotation inside the subspace, and each up to sign when the eigenvalues are distinct.

Step 4: compute it from the SVD, not the covariance. Take the thin **singular value decomposition** (SVD) $\mathbf{X}_c = \mathbf{U}\mathbf{S}\mathbf{V}^\top$ with $\mathbf{U} \in \mathbb{R}^{m \times r}$, $\mathbf{S} = \text{diag}(s_1, \dots, s_r)$, $s_1 \geq \dots \geq s_r \geq 0$, $\mathbf{V} \in \mathbb{R}^{n \times r}$ and $r = \min(m, n)$. Here r is the number of columns kept by the reduced SVD, not the rank. The rank k is the number of positive singular values, and $k \leq r$. Centring makes the rows of $\mathbf{X}_c$ sum to zero, so $k \leq \min(m-1, n)$: with $m \leq n$ at least one singular value is zero. Then

$$\mathbf{\Sigma} = \frac{1}{m-1} \mathbf{V} \mathbf{S} \mathbf{U}^\top \mathbf{U} \mathbf{S} \mathbf{V}^\top = \mathbf{V} \frac{\mathbf{S}^2}{m-1} \mathbf{V}^\top.$$

The columns of $\mathbf{V}$ are the eigenvectors of $\mathbf{\Sigma}$, with $\lambda_j = s_j^2/(m-1)$, and $\mathbf{D} = \mathbf{V}_{:,1:l}$. The codes come for free: $\mathbf{C} = \mathbf{X}_c \mathbf{D} = \mathbf{U}_{:,1:l} \mathbf{S}_{1:l,1:l}$. *Why prefer the SVD:* forming $\mathbf{X}_c^\top \mathbf{X}_c$ squares the condition number, so small directions lose relative precision (section 1.5 and appendix C). The SVD works on $\mathbf{X}_c$ directly.

When $n \gg m$ (a few hundred images of a million pixels each), even the thin SVD is fine, but there is an older trick worth knowing because it reappears in chapter 4: the $m \times m$ matrix $\mathbf{X}_c \mathbf{X}_c^\top = \mathbf{U} \mathbf{S}^2 \mathbf{U}^\top$ has the same nonzero eigenvalues, and for the k positive singular values $\mathbf{V}_{:,1:k} = \mathbf{X}_c^\top \mathbf{U}_{:,1:k} \text{diag}(s_1, \dots, s_k)^{-1}$ recovers the directions. Columns with $s_j = 0$ cannot be recovered this way (there is nothing to divide by, and $\mathbf{X}_c^\top \mathbf{U}_{:,j} = \mathbf{0}$); they carry zero variance and PCA never needs them. Equivalently, $\mathbf{V}_{:,1:k} \mathbf{V}_{:,1:k}^\top = \mathbf{X}_c^\dagger \mathbf{X}_c$ with the pseudoinverse, which inverts only positive singular values. In practice, drop singular values below a tolerance such as $\max(m, n) \epsilon_{\text{mach}} s_1$ before dividing. Everything PCA needs can be computed from **inner products between examples** — the observation behind kernel methods.



$$r = \min(m, n) \quad s_1 \geq s_2 \geq \dots \geq s_r \geq 0 \quad \lambda_j = s_j^2/(m-1)$$

Encode: $\mathbf{C} = \mathbf{X}_c \mathbf{D} \quad (m \times l)$ Decode: $\hat{\mathbf{X}} = \mathbf{C} \mathbf{D}^\top + \mathbf{1} \boldsymbol{\mu}^\top \quad (m \times n)$

Figure 2.2: Shapes in the thin SVD of the centred design matrix. The first l columns of $\mathbf{V}$ (the first l rows of $\mathbf{V}^\top$) form the decoding matrix $\mathbf{D}$.

Step 5: the error that remains, and the Eckart–Young theorem. Substituting the optimum back,

$$J_l^* = \frac{m-1}{m} \sum_{j>l} \lambda_j = \frac{1}{m} \sum_{j>l} s_j^2,$$

so the discarded eigenvalues are exactly the average squared reconstruction error. In matrix form, the reconstruction $\hat{\mathbf{X}}_c = \mathbf{X}_c \mathbf{D} \mathbf{D}^\top = \sum_{j \leq l} s_j \mathbf{U}_{:,j} \mathbf{V}_{:,j}^\top$ is the SVD truncated after l terms, and the **Eckart–Young theorem** says it is the best rank- l approximation of $\mathbf{X}_c$ in Frobenius norm (and in spectral norm), with errors

$$\|\mathbf{X}_c - \hat{\mathbf{X}}_c\|_F^2 = \sum_{j>l} s_j^2, \quad \|\mathbf{X}_c - \hat{\mathbf{X}}_c\|_2 = s_{l+1}.$$

Why it holds (sketch): any rank- l matrix $\mathbf{B}$ has a null space of dimension $n-l$, which must intersect the $(l+1)$ -dimensional span of $\mathbf{V}_{:,1:l+1}$; on a unit vector in that intersection, $\mathbf{X}_c$ has norm at least s_{l+1} while $\mathbf{B}$ gives zero. So no rank- l matrix does better than s_{l+1} in spectral norm, and the truncated SVD achieves it. The Frobenius case follows by applying the same argument to each s_{l+k} .

The **explained variance ratio** of l components is $\sum_{j \leq l} \lambda_j / \sum_j \lambda_j$. Because the singular values predict the error exactly, you can say how good a reconstruction will be *before* looking at a single reconstructed image.

Step 6: what noise looks like in the spectrum. If the data were pure isotropic noise with variance σ^2 per pixel, the population covariance would be $\sigma^2 \mathbf{I}$ and every eigenvalue would equal σ^2 . With finitely many samples they spread out. When m and n are both large, the Marchenko–Pastur law (1967) says the sample eigenvalues of pure noise fill the interval $[\sigma^2(1 - \sqrt{n/m})^2, \sigma^2(1 + \sqrt{n/m})^2]$. Eigenvalues clearly above the upper edge are evidence of signal; those inside are indistinguishable from noise. Section 2.8 checks this on the lab’s images.

Step 7: scaling, standardising and whitening. PCA depends on the units of each feature, because variance does. Three transformations are easy to confuse:

- **Centring** subtracts the training mean. It is required (exercise 1).
- **Standardising** also divides each feature by its training standard deviation, so PCA works on the correlation matrix. It stops a feature measured in large units from dominating, but it can amplify a nearly constant, noisy feature. For images whose pixels share one intensity scale, standardising per pixel is a modelling choice, not an automatic improvement.
- **Whitening** divides each code by the square root of its eigenvalue, $\mathbf{c} \mapsto \mathbf{\Lambda}_l^{-1/2} \mathbf{c}$, so the training codes have identity covariance. Directions with tiny eigenvalues are mostly noise, so whitening amplifies noise; a regularised version divides by $\sqrt{\lambda_j + \varepsilon}$. Whitening discards variance scale, which a downstream task might need.

All three are **fitted on training data only** and then applied unchanged to validation and test data.

2.6 Numerical worked example

Four points. Take $(2, 0), (0, 1), (-2, 0), (0, -1)$. The mean is zero, so $\mathbf{X}_c = \mathbf{X}$, and

$$\mathbf{\Sigma} = \frac{1}{3} \mathbf{X}^\top \mathbf{X} = \frac{1}{3} \begin{pmatrix} 8 & 0 \\ 0 & 2 \end{pmatrix}, \quad \lambda_1 = \frac{8}{3}, \quad \lambda_2 = \frac{2}{3}.$$

The first principal direction is $\mathbf{d} = (1, 0)^\top$ and explains $8/10 = 80\%$ of the variance. Projecting onto $\mathbf{d}$ keeps $(2, 0)$ and $(-2, 0)$ exactly and sends $(0, \pm 1)$ to the origin. The mean squared error is $(0 + 1 + 0 + 1)/4 = 0.5$. Step 5 predicts $\frac{m-1}{m}\lambda_2 = \frac{3}{4} \cdot \frac{2}{3} = 0.5$, which matches.

A tilted covariance. Suppose a centred covariance is $\Sigma = \begin{pmatrix} 2 & 1 \\ 1 & 2 \end{pmatrix}$. Its eigenpairs are $(3, (1, 1)^\top/\sqrt{2})$ and $(1, (1, -1)^\top/\sqrt{2})$ — check: $\Sigma(1, 1)^\top = (3, 3)^\top$. One component keeps $3/4$ of the variance. The centred point $(2, 0)^\top$ has code $\mathbf{d}^\top(2, 0)^\top = \sqrt{2}$ and reconstruction $\sqrt{2} \cdot (1, 1)^\top/\sqrt{2} = (1, 1)^\top$, with squared error $(2 - 1)^2 + (0 - 1)^2 = 2$. The point $(2, 2)^\top$ lies on the retained diagonal and reconstructs exactly.

Predicting error from singular values. If $s = (5, 3, 1)$, the total squared energy is $25 + 9 + 1 = 35$. One component leaves error $9 + 1 = 10$ (keeps 71.4%); two leave 1 (keep 97.1%). You know the reconstruction quality before computing any reconstruction.

Check the sign ambiguity: $\mathbf{d} = (-1, 0)^\top$ in the first example gives codes of opposite sign and identical reconstructions. Any test that compares principal directions must allow for this, and any figure that shows a component should say its sign is arbitrary.

2.7 Small implementation and checks

The lab is `labs/ch02-pca/pca.py`. The core is four lines:

```
mu = X.mean(axis=0)                # (n,)
_, s, Vt = np.linalg.svd(X - mu, full_matrices=False)
D = Vt[:l].T                       # (n, l) decoding matrix
C = (X - mu) @ D                   # (m, l) codes; X_hat = C @ D.T
    ↪ + mu
```

`test_pca.py` checks the properties derived above:

- $\mathbf{D}^\top \mathbf{D} = \mathbf{I}_l$ (orthonormality, step 3);
- $s_j^2/(m-1)$ equals the eigenvalues of `np.cov` (step 4);
- with $l = \min(m, n)$ the reconstruction is exact;
- training error never increases as l grows (nested subspaces);
- the subspace angle between $\mathbf{D}$ and $\mathbf{D}\mathbf{Q}$ is zero for any rotation $\mathbf{Q}$ (the subspace, not the basis, is what PCA determines);
- a single outlier image replaces a whole principal direction;
- an unseen pattern has larger reconstruction error than typical test images.

Run `python -m pytest labs/ch02-pca`. A check that fails is a bug to find, not a tolerance to loosen.

2.8 Experiment and expected observations

The synthetic images mix four templates (horizontal bar, vertical bar, diagonal, square) with random non-negative weights. Each pixel gets independent Gaussian noise with standard deviation $\sigma = 0.15$, so the noise variance is 0.0225. Running `python labs/ch02-pca/pca.py` (seed 0, 800 training and 200 test images) prints:

l	1	2	3	4	8	16
cumulative explained variance	0.363	0.528	0.639	0.718	0.733	0.762
train MSE per pixel	0.0491	0.0364	0.0278	0.0218	0.0206	0.0184
test MSE per pixel	0.0490	0.0376	0.0283	0.0222	0.0215	0.0203

(The lab prints cumulative variance for $l \leq 8$; the value at $l = 16$ comes from `np.cumsum(run(...)[“evr”])`. MSE per pixel is J/n in the notation of step 5.)

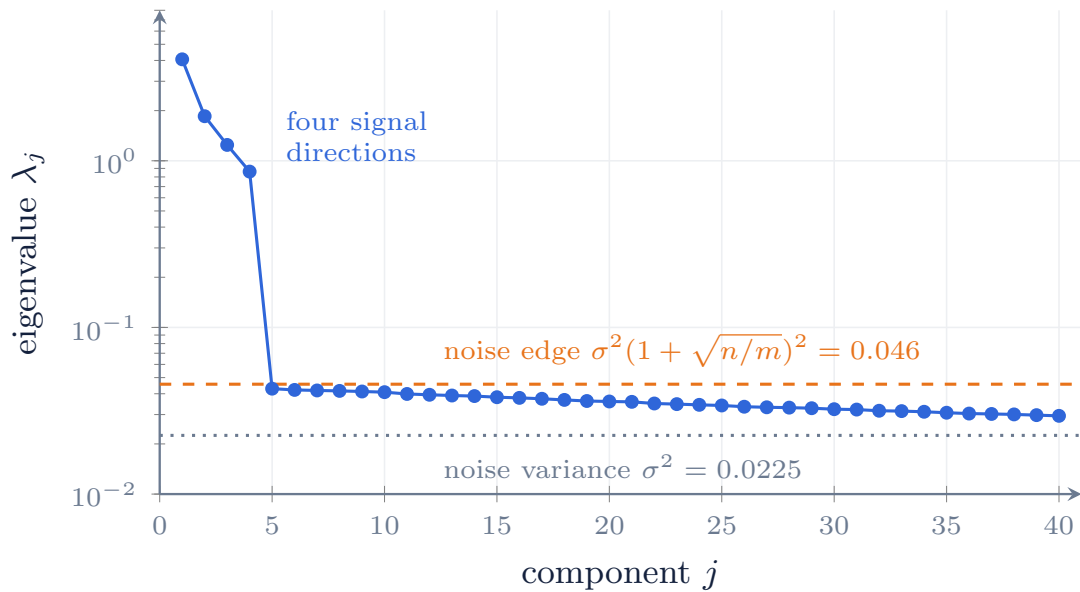


Figure 2.3: Scree plot of the lab’s images on a log scale. Four eigenvalues stand far above the rest; the remaining 140 sit just below the Marchenko–Pastur noise edge, not at the noise variance itself.

What to notice:

1. **An elbow at $l = 4$** , the true number of templates. The top eigenvalues are 4.06, 1.85, 1.25 and 0.86; the fifth is 0.043. After the elbow, each extra component adds about 0.4% of variance.
2. **The noise eigenvalues are not σ^2** . The largest noise eigenvalue is 0.043, almost twice $\sigma^2 = 0.0225$. Step 6 explains why: with $n/m = 144/800$ the noise edge is $0.0225(1 + \sqrt{0.18})^2 = 0.046$. A threshold at σ^2 would wrongly call dozens of noise directions “signal”.
3. **Test error levels off near σ^2** . With the four signal directions kept, the residual holds the noise in the other 140 directions. Its expected size is $\sigma^2(n - 4)/n \approx 0.0219$ per pixel, close to the measured values. Components beyond four mostly describe noise. They still lower training error, and the gap between training and test error widens. This is the first overfitting signal in the book.
4. **Explained variance stops at 72% even with the right l** . Isotropic noise spreads variance over all 144 directions. A fixed rule such as “keep 95% of variance” would

ask for far too many components here.

Two more measurements (printed by the full run):

- **Outliers.** Add *one* image with a single pixel set to 50 to the 800 training images. The largest principal angle between the clean and contaminated top-4 subspaces is 89.9° : one whole principal direction now points at that pixel. The reason is arithmetic: that pixel's variance becomes about $50^2/801 \approx 3.1$, more than the fourth signal eigenvalue (0.86).
- **Anomalies.** With $l = 4$, normal test images have mean reconstruction error 3.11 (summed over pixels). A cross-shaped pattern never seen in training scores 8.03. An extreme but *in-subspace* image (one template at three times its usual intensity) scores only 0.19. Reconstruction error flags what lies *outside* the subspace, not what is unusual within it.

Re-run with `--seed 1`. The numbers change in the third decimal place and the conclusions do not. Report this kind of seed check whenever you make a claim.

2.9 Moving pieces: diagnosing and fixing

Moving piece	Symptom when it is wrong	How to confirm	Fix
Centring	first component points at the mean image; explained variance looks huge	is μ subtracted? does component 1 look like the average image?	subtract the training mean
Feature scale	one feature (in large units) owns component 1	compare feature variances	standardise, or rescale to meaningful common units
Number of components l	too few: blurry reconstructions; too many: noise fitted, test error rises	held-out error curve; scree plot against the noise edge; downstream validation	choose l by the purpose: held-out reconstruction, noise threshold, or task metric
Outliers	a component looks like a single pixel or a single image	plot the largest codes; look at the images with extreme codes	remove or down-weight; robust PCA; clip values
Leakage	test reconstruction suspiciously good	was PCA fitted before the split?	fit μ , scaling and D on training data only
Sign and order	components “flip” between runs or libraries	compare subspaces with principal angles, not vectors	fix a sign convention; compare subspaces
Nonlinear structure	many components needed, components look like blurry averages	reconstructions of individual images; curvature in 2-D plots of codes	kernel PCA (chapter 4), autoencoders (chapter 10)

Moving piece	Symptom when it is wrong	How to confirm	Fix
Variance $\neq$ relevance	good reconstruction, poor downstream accuracy	linear probe accuracy against l	supervised projection (chapter 3), or learned features (chapter 18)
Whitening	noisy, unstable downstream behaviour	look at the smallest kept eigenvalues	regularise with $\sqrt{\lambda_j} + \varepsilon$; whiten fewer components
Computation	slow or inaccurate on wide data	timing; compare with <code>np.cov</code> eigenvalues	thin SVD; snapshot method; randomised SVD for very large data

First things to try when PCA output looks wrong: (1) plot the mean image and the top components as images; (2) plot the scree on a log scale; (3) look at the ten examples with the largest codes on each component; (4) check that everything was fitted on the training split only.

2.10 Application, limitations and failure cases

Application (educational case study). PCA is a standard first baseline for any image representation. Fit it on training images only, then report held-out reconstruction error and a linear probe on the codes (chapter 18). If a deep representation cannot beat this baseline, the deep model needs a closer look. For multispectral or hyperspectral imagery, PCA summarises correlated bands and helps visualise the data, but each component mixes bands and loses physical meaning; compare against the original bands before treating compression as a scientific finding. The reusable research question is: *which variations does the representation keep, and are those the useful ones?*

Failure cases to recognise:

- **Unscaled features.** Variance depends on units. Mixing pixel intensities with, say, temperature in kelvin lets one feature dominate.
- **Nonlinear structure.** A digit rotated through many angles lies on a curved manifold. PCA needs many components to approximate it, and the components look like blurred averages.
- **Variance is not relevance.** The largest variance in a photo collection is often global illumination, not the object class. In medical images, lighting or scanner differences can be kept before a subtle disease cue.
- **Leakage.** Fitting μ and D on all data before splitting leaks test information into the encoder (chapter 5), even though no labels were used.
- **Outliers.** One corrupted image can rotate a whole direction (section 2.8).
- **Anomaly detection by reconstruction error** catches departures from the subspace, not every kind of unusual input.
- **Interpreting components.** A component describes a direction of variation in this dataset, not necessarily one physical factor you could control. Show both signs.

2.11 Bridge to a research paper

Eigenfaces (Turk and Pentland, 1991) applies this chapter to face recognition.

- **Inputs:** aligned greyscale face images. **Outputs:** an identity label, or “not a face”.
- **Learned components:** the mean face and the top principal directions (“eigenfaces”), computed with the snapshot trick of step 4.
- **Objective:** PCA reconstruction (unsupervised), then nearest neighbour in code space; distance from the “face space” (reconstruction error) flags non-faces.
- **Assumptions:** alignment, controlled lighting, and identity carried by the leading linear directions.
- **Central claim:** a low-dimensional linear code supports recognition and face detection.
- **What later work found:** the leading components often encode lighting rather than identity; discarding the first few can *improve* recognition. That is assumption 2 failing in practice.
- **Affordable experiment:** on a public face dataset with an identity-disjoint split, compare nearest-neighbour accuracy with codes of size $l \in \{10, 50, 150\}$ against raw pixels. Then drop the first three components and repeat; then add brightness jitter.

Read the paper for its assumptions and its evaluation protocol. Its accuracy numbers are not targets for this exercise.

2.12 Memory map

Summary. PCA finds the subspace closest to the centred data, which is the same as the subspace of greatest variance and the same as the truncated SVD. The principal directions are eigenvectors of the covariance, computed stably as right singular vectors of $\mathbf{X}_c$; the discarded eigenvalues are exactly the reconstruction error. Choose l by purpose — held-out error, a noise threshold that accounts for sample size, or the downstream task — and fit everything on training data. PCA assumes linearity, squared error and “variance means signal”; when those fail, the fixes are the subject of chapters 4, 6, 10 and 18.

2.13 Exercises

1. *Conceptual.* Why must the data be centred? Give a two-point example where uncentred “PCA” returns a direction that only points at the mean.
2. *Conceptual.* What happens to the singular values, eigenvalues, directions and explained-variance ratios if $\mathbf{X}_c$ is multiplied by 2?
3. *Derivation.* Prove step 1 for l components. Then complete the Ky Fan argument of step 3: show $0 \leq w_j \leq 1$ and $\sum_j w_j = l$.
4. *Derivation.* Show that the codes $\mathbf{C} = \mathbf{X}_c \mathbf{D}$ have a diagonal sample covariance. What are its entries?
5. *Derivation.* Verify the snapshot trick: if $\mathbf{X}_c \mathbf{X}_c^\top \mathbf{u} = s^2 \mathbf{u}$ with $\|\mathbf{u}\| = 1$ and $s > 0$, then $\mathbf{v} = \mathbf{X}_c^\top \mathbf{u} / s$ is a unit eigenvector of $\mathbf{X}_c^\top \mathbf{X}_c$.
6. *Implementation.* Implement PCA through `np.linalg.eigh(np.cov(X, rowvar=False))` and compare it with the SVD version. Then scale one pixel by 10^6 and report what happens to each method’s leading direction and to its numerical agreement.
7. *Interpretation.* Plot the test MSE against l for noise levels $\sigma \in \{0.05, 0.15, 0.4\}$. Explain the position of the elbow and the level of the plateau. At which σ does the

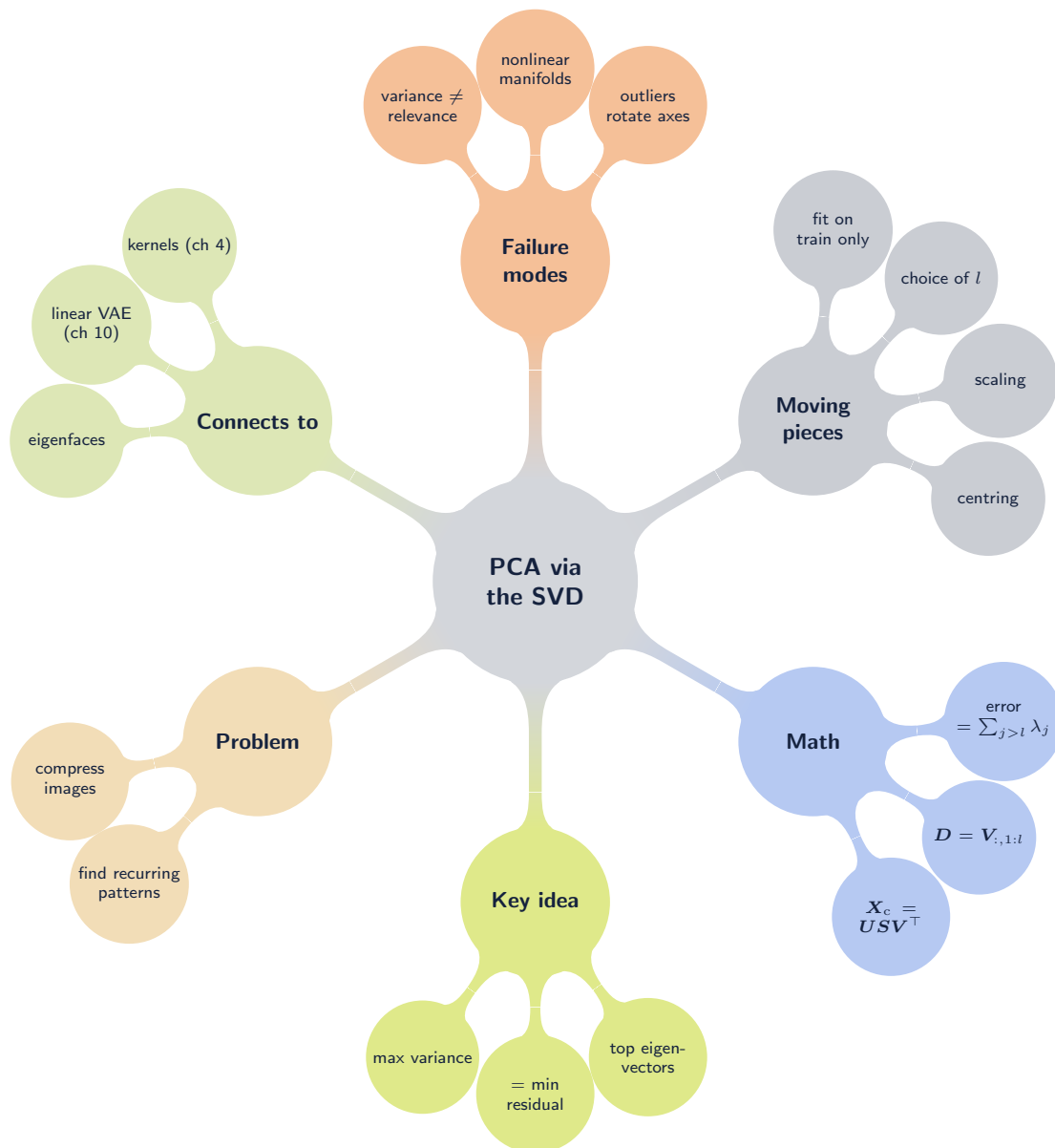


Figure 2.4: Memory map of chapter 2.

fourth signal eigenvalue sink below the noise edge?

8. *Failure analysis.* Reproduce the outlier experiment of section 2.8. How large must the outlier pixel be before it takes over a direction? Propose and test one robust alternative (for example, clipping each pixel at its training 99th percentile).
9. *Open question.* PCA codes are linear functions of pixels. Which property of natural images would make a *nonlinear* encoder worth its cost? Design a falsifiable test.

2.13.1 Answer discussions

1. Uncentred, the first direction maximises $\frac{1}{m} \sum (\mathbf{d}^\top \mathbf{x}^{(i)})^2$, which includes $(\mathbf{d}^\top \boldsymbol{\mu})^2$. For the points $(10, 1)$ and $(10, -1)$, the spread is vertical but the uncentred first direction is close to $(1, 0)$, pointing at the mean.
2. Singular values double, eigenvalues quadruple; directions and ratios are unchanged.
3. $w_j = \|\mathbf{B}_{j,:}\|^2 = \mathbf{e}^{(j)\top} \mathbf{B} \mathbf{B}^\top \mathbf{e}^{(j)}$ and $\mathbf{B} \mathbf{B}^\top$ is a projector, so $0 \leq w_j \leq 1$; $\sum_j w_j = \text{Tr}(\mathbf{B} \mathbf{B}^\top) = \text{Tr}(\mathbf{B}^\top \mathbf{B}) = l$.
4. $\frac{1}{m-1} \mathbf{C}^\top \mathbf{C} = \frac{1}{m-1} \mathbf{D}^\top \mathbf{X}_c^\top \mathbf{X}_c \mathbf{D} = \text{diag}(\lambda_1, \dots, \lambda_l)$: principal components are uncorrelated, with variances equal to the eigenvalues.
5. $\mathbf{X}_c^\top \mathbf{X}_c \mathbf{v} = \mathbf{X}_c^\top (\mathbf{X}_c \mathbf{X}_c^\top \mathbf{u})/s = s^2 \mathbf{v}$, and $\|\mathbf{v}\|^2 = \mathbf{u}^\top \mathbf{X}_c \mathbf{X}_c^\top \mathbf{u}/s^2 = 1$.
6. Both find the scaled pixel as the first direction. The covariance route loses accuracy in the small directions first, because it squares the condition number.
7. The elbow stays at 4 while the signal eigenvalues exceed the noise edge; the plateau tracks $\sigma^2(n-4)/n$. With $\sigma = 0.4$ the edge is about $0.16 \cdot 2.03 \approx 0.33$, still below 0.86, so all four survive; they fail once the edge passes 0.86 (around $\sigma \approx 0.65$).
8. Roughly when v^2/m exceeds $\lambda_4 \approx 0.86$, i.e. $v \approx 26$ for $m = 800$. Clipping at a training percentile removes the effect of single saturated pixels.
9. Natural images lie near curved manifolds (pose, lighting, deformation). Test: at equal code size, compare held-out reconstruction error and linear-probe accuracy of PCA and a small autoencoder on rotated digits; PCA should need many more components.

2.14 Self-check and common mistakes

- ☐ I can state the shape of $\mathbf{X}$, $\mathbf{D}$, $\mathbf{C}$ and $\hat{\mathbf{X}}$ without looking.
- ☐ I can explain why the SVD is preferred to forming $\mathbf{X}_c^\top \mathbf{X}_c$.
- ☐ I can predict the reconstruction error from the singular values.
- ☐ I can say why the noise eigenvalues exceed σ^2 in a finite sample.
- ☐ I fit $\boldsymbol{\mu}$, any scaling, and $\mathbf{D}$ on training data only.

Common mistakes: using the columns of $\mathbf{U}$ (rather than $\mathbf{V}$) as directions; confusing s_j with λ_j ; forgetting the $m-1$; comparing components without allowing for sign flips; choosing l by a variance threshold without held-out error; standardising pixel data by reflex; reporting an MSE without saying whether it is per pixel or per image.

References

- I. Goodfellow, Y. Bengio and A. Courville. *Deep Learning*. MIT Press, 2016, section 2.12 (PCA) and section 2.8 (SVD).
- K. Pearson. On lines and planes of closest fit to systems of points in space. *Philosophical Magazine*, 2(11):559–572, 1901.
- H. Hotelling. Analysis of a complex of statistical variables into principal components. *Journal of Educational Psychology*, 24:417–441, 1933.

- C. Eckart and G. Young. The approximation of one matrix by another of lower rank. *Psychometrika*, 1:211–218, 1936.
- L. Sirovich and M. Kirby. Low-dimensional procedure for the characterization of human faces. *Journal of the Optical Society of America A*, 4(3):519–524, 1987.
- M. Turk and A. Pentland. Eigenfaces for recognition. *Journal of Cognitive Neuroscience*, 3(1):71–86, 1991.
- V. A. Marchenko and L. A. Pastur. Distribution of eigenvalues for some sets of random matrices. *Mathematics of the USSR-Sbornik*, 1(4):457–483, 1967.
- M. E. Tipping and C. M. Bishop. Probabilistic principal component analysis. *Journal of the Royal Statistical Society B*, 61(3):611–622, 1999.
- I. T. Jolliffe and J. Cadima. Principal component analysis: a review and recent developments. *Phil. Trans. R. Soc. A*, 374:20150202, 2016.

Chapter 3

Regression, classification and regularization

Prerequisites: chapters 1–2 (least squares, the SVD).

Learning outcomes: after this chapter you can

1. explain regularisation as a stated preference over solutions, and say which preference ridge and Lasso express;
2. derive ridge in closed form, read it as shrinkage along singular directions, and compute its bias and variance;
3. derive the Lasso’s soft-thresholding and proximal-gradient update, and explain geometrically why it produces exact zeros;
4. derive logistic regression from the Bernoulli likelihood, its gradient and Hessian, and say when its optimum exists and is unique;
5. separate three decisions that are often blurred: estimating probabilities, choosing a decision threshold, and reweighting classes.

3.1 A concrete problem

An agronomist has 60 labelled photographs of potato leaves and wants to flag late blight. From each photograph a pipeline extracts 20 numbers: colour statistics, texture measures, lesion counts. Many of these numbers are correlated with each other, and only a few probably matter. Two models are natural:

- a **regression** predicting a severity score, and
- a **classifier** predicting the probability that the leaf is diseased.

With 20 features and 60 examples, least squares (chapter 1) fits the training data well and predicts new leaves poorly; its weights change wildly from one sample to the next. And only 5% of leaves in the field are diseased, so a classifier that always says “healthy” is 95% accurate and useless.

This chapter adds two ideas to the linear model. **Regularisation** changes the objective to prefer some solutions over others — small weights, or few nonzero weights — trading a little fit for a lot of stability. **Probabilistic classification** replaces “fit the labels” with “model the probability of each label”, which makes room for decisions that weigh different errors differently.

3.2 How we got here

Ill-posed problems (1943–1970). Andrey Tikhonov, working on inverse problems in geophysics, showed that adding a penalty $\lambda\|\mathbf{w}\|^2$ turns an unstable inverse problem into a stable one (1943; fully developed in 1963). Statisticians met the same instability as **multicollinearity**: correlated predictors make least-squares weights swing wildly (section 1.8). Arthur Hoerl and Robert Kennard (1970) proposed the same penalty in regression and called it **ridge regression**, after the “ridge” of the penalised objective. Their key observation: a small, deliberate bias can reduce the variance of the weights so much that the total error falls.

Choosing variables (1970s–1996). Ridge keeps every feature. Scientists often want to know *which* features matter, and the classical answer — searching over subsets of variables — is combinatorial and itself unstable: adding one example can change the chosen subset completely. Robert Tibshirani (1996) proposed the **Lasso** (“least absolute shrinkage and selection operator”): penalise $\|\mathbf{w}\|_1$ instead. The L^1 penalty shrinks like ridge *and* sets some weights exactly to zero, giving selection through a convex problem (related L^1 ideas had appeared in geophysics in the 1980s and in signal processing as basis pursuit). Fast algorithms followed: least angle regression traced the whole path of solutions (Efron et al., 2004); iterative soft thresholding (Daubechies, Defrise and De Mol, 2004) and its accelerated version FISTA (Beck and Teboulle, 2009) handled large problems; coordinate descent (Friedman, Hastie and Tibshirani, 2010) became the standard. The elastic net (Zou and Hastie, 2005) mixed the two penalties to keep groups of correlated features together.

From lines to probabilities (1838–1972). The logistic curve was introduced by Pierre-François Verhulst (1838) for population growth. Joseph Berkson (1944) named its inverse the *logit* and used it for dose–response data, and David Cox (1958) developed logistic regression for binary outcomes. John Nelder and Robert Wedderburn (1972) placed linear, logistic and Poisson regression in one family — generalised linear models — fitted by the same Newton-type algorithm (iteratively reweighted least squares). Fitting squared error to 0/1 labels had obvious problems (predictions below 0 or above 1, and large penalties for confidently correct points); modelling the *probability* through the logistic function fixed them.

What this chapter cannot fix. Regularisation makes a linear model stable; it does not make it flexible. A straight decision boundary cannot separate classes that need a curved one — chapter 4 changes the representation. And the regularisation weight λ is chosen from data, which needs an honest split (chapter 5). Chapter 6 shows that ridge and Lasso are what you get from Gaussian and Laplace *priors*, turning the “preference” of this chapter into a probabilistic assumption.

3.3 Assumptions and notation

Symbol	Shape	Meaning
$\mathbf{X}, \mathbf{y}$	$m \times n, m$	design matrix and targets (regression)
$y^{(i)} \in \{0, 1\}$	scalar	binary label (classification)
$\mathbf{w}$	n	weights; intercept handled by a ones column, not penalised
$\lambda \geq 0$	scalar	regularisation weight

Symbol	Shape	Meaning
$\ \mathbf{w}\ _1$	scalar	$\sum_j w_j $
$\sigma(a)$	scalar	logistic sigmoid $1/(1 + \exp(-a))$
$a^{(i)}$	scalar	logit (score) $\mathbf{w}^\top \mathbf{x}^{(i)}$
$p^{(i)}$	scalar	predicted probability $\sigma(a^{(i)})$
$\mathbf{p}$	m	all predicted probabilities
$\mathbf{USV}^\top$	—	thin SVD of $\mathbf{X}$ (chapter 2), singular values s_j

Convention. We use the **summed** data loss, $\frac{1}{2}\|\mathbf{X}\mathbf{w} - \mathbf{y}\|^2$, not the mean. With the mean, the same estimator needs λ divided by m . Libraries differ (scikit-learn’s **Ridge** uses the sum, its **Lasso** the mean), so a value of λ means nothing without the full objective.

Assumptions:

1. A linear score $\mathbf{w}^\top \mathbf{x}$ is a useful model, after any feature engineering.
2. Features are on comparable scales (standardised with training statistics), because the penalties treat all weights alike.
3. For classification, the label depends on $\mathbf{x}$ only through $\mathbf{w}^\top \mathbf{x}$ via the logistic link, and examples are independent.

3.4 Mathematical formulation

$$\begin{aligned}
 \text{ridge: } J(\mathbf{w}) &= \frac{1}{2}\|\mathbf{X}\mathbf{w} - \mathbf{y}\|^2 + \frac{\lambda}{2}\|\mathbf{w}\|^2, \\
 \text{Lasso: } J(\mathbf{w}) &= \frac{1}{2}\|\mathbf{X}\mathbf{w} - \mathbf{y}\|^2 + \lambda\|\mathbf{w}\|_1, \\
 \text{elastic net: } J(\mathbf{w}) &= \frac{1}{2}\|\mathbf{X}\mathbf{w} - \mathbf{y}\|^2 + \lambda_1\|\mathbf{w}\|_1 + \frac{\lambda_2}{2}\|\mathbf{w}\|^2, \\
 \text{logistic: } J(\mathbf{w}) &= \sum_{i=1}^m \left[-y^{(i)} \log p^{(i)} - (1 - y^{(i)}) \log(1 - p^{(i)}) \right] + \frac{\lambda}{2}\|\mathbf{w}\|^2.
 \end{aligned}$$

Each penalised problem has an equivalent constrained form — minimise the data loss subject to $\|\mathbf{w}\|_2 \leq t$ or $\|\mathbf{w}\|_1 \leq t$ — with a one-to-one correspondence between λ and t along the solution path. The constrained form gives the geometry of figure 3.1.

For K classes, logistic regression becomes **softmax regression**: logits $a_k = \mathbf{W}_{:,k}^\top \mathbf{x}$, probabilities $p_k = \exp(a_k) / \sum_{k'} \exp(a_{k'})$, and the loss is the cross-entropy $-\sum_k \mathbf{1}_{y=k} \log p_k$.

3.5 Derivation, with reasons for each step

3.5.1 Step 1: ridge has a unique closed-form solution

The gradient of the ridge objective is $\mathbf{X}^\top(\mathbf{X}\mathbf{w} - \mathbf{y}) + \lambda\mathbf{w}$ (rules 2 and 4 of section 1.5). Setting it to zero:

$$(\mathbf{X}^\top \mathbf{X} + \lambda \mathbf{I}) \mathbf{w}_\lambda = \mathbf{X}^\top \mathbf{y}.$$

The Hessian is $\mathbf{X}^\top \mathbf{X} + \lambda \mathbf{I}$, and for $\mathbf{v} \neq \mathbf{0}$, $\mathbf{v}^\top (\mathbf{X}^\top \mathbf{X} + \lambda \mathbf{I}) \mathbf{v} = \|\mathbf{X}\mathbf{v}\|^2 + \lambda\|\mathbf{v}\|^2 > 0$ when $\lambda > 0$. So **ridge has exactly one minimiser even when the columns of $\mathbf{X}$ are dependent**, and even when $n > m$. That is the precise sense in which the penalty “stabilises” the problem.

3.5.2 Step 2: ridge is shrinkage along singular directions

Substitute $\mathbf{X} = \mathbf{U}\mathbf{S}\mathbf{V}^\top$:

$$\mathbf{w}_\lambda = \sum_j \frac{s_j}{s_j^2 + \lambda} (\mathbf{U}_{:,j}^\top \mathbf{y}) \mathbf{V}_{:,j}, \quad \hat{\mathbf{y}}_\lambda = \mathbf{X}\mathbf{w}_\lambda = \sum_j \underbrace{\frac{s_j^2}{s_j^2 + \lambda}}_{\text{shrinkage}} (\mathbf{U}_{:,j}^\top \mathbf{y}) \mathbf{U}_{:,j}.$$

Compare least squares (section 1.5), which divides by s_j and so amplifies noise in directions with small s_j . Ridge multiplies each direction's least-squares fit by $s_j^2/(s_j^2 + \lambda)$: close to 1 where the data vary a lot ($s_j^2 \gg \lambda$), close to 0 where they barely vary ($s_j^2 \ll \lambda$). **Ridge quietly ignores the directions the data cannot pin down.** It is PCA (chapter 2) with soft rather than hard truncation.

The sum of the shrinkage factors, $\text{df}(\lambda) = \sum_j s_j^2/(s_j^2 + \lambda)$, is the **effective number of parameters**: n at $\lambda = 0$, falling to 0 as $\lambda \rightarrow \infty$.

3.5.3 Step 3: the bias and variance of ridge

Hold the inputs $\mathbf{X}$ fixed, let $\mathbf{y} = \mathbf{X}\mathbf{w}^\circ + \boldsymbol{\epsilon}$ with noise of mean zero and covariance $\sigma^2 \mathbf{I}$, and write $\mathbf{A}_\lambda = (\mathbf{X}^\top \mathbf{X} + \lambda \mathbf{I})^{-1} \mathbf{X}^\top$ so that $\mathbf{w}_\lambda = \mathbf{A}_\lambda \mathbf{y}$.

- **Bias.** $\mathbb{E}[\mathbf{w}_\lambda] = \mathbf{A}_\lambda \mathbf{X}\mathbf{w}^\circ$, and using $\mathbf{X}^\top \mathbf{X} = (\mathbf{X}^\top \mathbf{X} + \lambda \mathbf{I}) - \lambda \mathbf{I}$,

$$\mathbb{E}[\mathbf{w}_\lambda] - \mathbf{w}^\circ = -\lambda(\mathbf{X}^\top \mathbf{X} + \lambda \mathbf{I})^{-1} \mathbf{w}^\circ.$$

Zero at $\lambda = 0$; growing with λ ; pulling the weights towards zero.

- **Variance.** $\text{Cov}(\mathbf{w}_\lambda) = \sigma^2 \mathbf{A}_\lambda \mathbf{A}_\lambda^\top$, and the summed variance of the fitted values is

$$\sum_i \text{Var}(\hat{y}_\lambda^{(i)}) = \sigma^2 \sum_j \left(\frac{s_j^2}{s_j^2 + \lambda} \right)^2.$$

At $\lambda = 0$ this is $\sigma^2 n$ (with $n \leq m$ independent columns): every parameter adds σ^2 of variance. Averaged over the m training points, that is $\sigma^2 n/m$.

The expected squared error of the fitted values decomposes into **squared bias + variance** (the noise in the targets adds a constant on top for new observations). As λ grows, bias rises and variance falls; their sum usually has an interior minimum (figure 3.3). This is the **bias–variance trade-off** in its cleanest form; chapter 5 treats it generally.

3.5.4 Step 4: the Lasso, one coordinate at a time

The absolute value is not differentiable at zero, and that is exactly where the interesting behaviour lives. For a convex function, the **subdifferential** $\partial f(w)$ is the set of slopes of lines that touch f at w and stay below it: for $|w|$ it is $\{\text{sign}(w)\}$ when $w \neq 0$ and the whole interval $[-1, 1]$ at $w = 0$. A point minimises a convex function exactly when $0 \in \partial f$.

Take the scalar problem $\min_w \frac{1}{2}(w-a)^2 + \lambda|w|$. The optimality condition is $0 \in w - a + \lambda \partial|w|$:

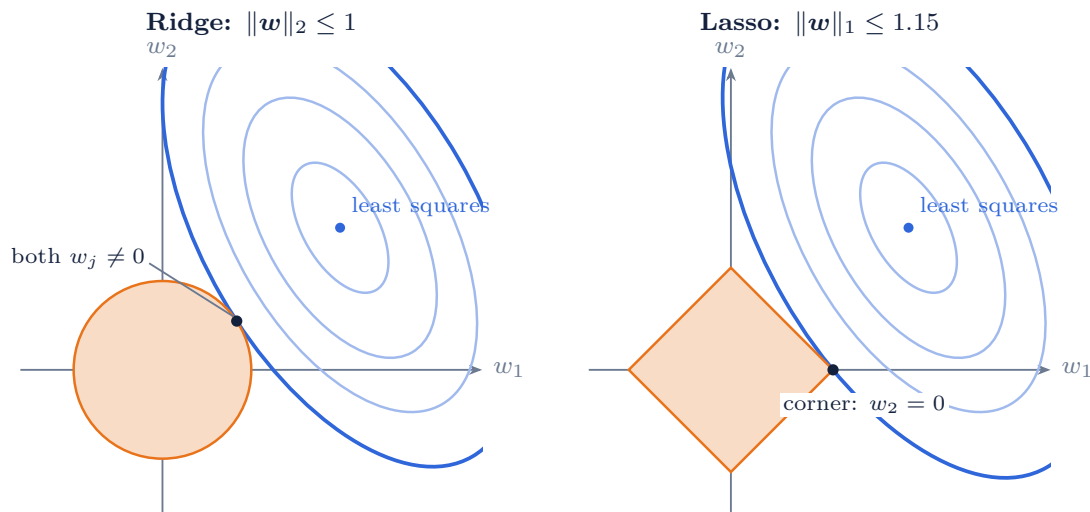
- if $w > 0$: $w = a - \lambda$, consistent only if $a > \lambda$;
- if $w < 0$: $w = a + \lambda$, consistent only if $a < -\lambda$;
- if $w = 0$: need $a \in [-\lambda, \lambda]$.

Together:

$$w^* = \text{soft}(a, \lambda) = \text{sign}(a) \max(|a| - \lambda, 0).$$

This is **soft thresholding**: move a towards zero by λ , and stop at zero. Every input with $|a| \leq \lambda$ maps to *exactly* zero — the source of sparsity.

Geometry. In the constrained form, the solution is where the smallest contour of the squared error touches the constraint region. The L^2 ball is round, so the contact point generically has all coordinates nonzero. The L^1 ball is a diamond (a cross-polytope in higher dimensions) whose corners lie on the coordinate axes; contours often touch it first at a corner or an edge, where some coordinates are zero (figure 3.1).



The penalised solution is where the smallest error contour first touches the constraint region (dark ellipse). A disc has no corners, so the contact point generically has every coordinate nonzero; a diamond's corners lie on the axes, so contact at a corner sets coordinates exactly to zero.

Figure 3.1: Why the L^1 penalty produces exact zeros. Both panels show the same squared-error contours around the least-squares solution; the dark contour is the one that touches the constraint region. Touch points are computed exactly.

Optimality conditions for the full Lasso. Applying $0 \in \partial J$ coordinate-wise, w is optimal exactly when, for every j ,

$$\mathbf{X}_{:,j}^\top(\mathbf{y} - \mathbf{X}\mathbf{w}) = \lambda \text{sign}(w_j) \text{ if } w_j \neq 0, \quad |\mathbf{X}_{:,j}^\top(\mathbf{y} - \mathbf{X}\mathbf{w})| \leq \lambda \text{ if } w_j = 0.$$

Read it: an active feature's correlation with the residual is pinned at exactly $\pm\lambda$; an inactive feature's correlation is too weak to be worth its penalty. At $w = \mathbf{0}$ the residual is $\mathbf{y}$, so **all weights are zero once** $\lambda \geq \lambda_{\max} = \|\mathbf{X}^\top \mathbf{y}\|_\infty$. These conditions are also a certificate: the lab checks any computed solution against them.

3.5.5 Step 5: proximal gradient descent (ISTA)

The Lasso objective is smooth plus non-smooth. The smooth part $g(w) = \frac{1}{2}\|\mathbf{X}w - \mathbf{y}\|^2$ has a gradient that changes no faster than $L = \|\mathbf{X}\|_2^2 = s_1^2$ (its Lipschitz constant). Around

the current point $\mathbf{w}_t$, g is therefore bounded above by the quadratic $g(\mathbf{w}_t) + \nabla g(\mathbf{w}_t)^\top (\mathbf{w} - \mathbf{w}_t) + \frac{1}{2\eta} \|\mathbf{w} - \mathbf{w}_t\|^2$ for any step $\eta \leq 1/L$. Minimising this upper bound plus the penalty separates into one scalar problem per coordinate — each a soft threshold:

$$\mathbf{v} = \mathbf{w}_t - \eta \mathbf{X}^\top (\mathbf{X} \mathbf{w}_t - \mathbf{y}), \quad \mathbf{w}_{t+1} = \text{soft}(\mathbf{v}, \eta \lambda).$$

A gradient step on the smooth part, then a shrink-and-clip. Because each step minimises an upper bound that touches the objective at $\mathbf{w}_t$, the objective never increases. This *majorise-minimise* pattern reappears in EM (chapter 8).

3.5.6 Step 6: logistic regression from the Bernoulli likelihood

Model $P(y = 1 \mid \mathbf{x}) = \sigma(\mathbf{w}^\top \mathbf{x})$. The probability of an observed label $y \in \{0, 1\}$ is $p^y(1-p)^{1-y}$, so the negative log-likelihood of one example is $\ell = -y \log p - (1-y) \log(1-p)$ — the **binary cross-entropy**.

Differentiate through the logit first. The sigmoid satisfies $\sigma'(a) = \sigma(a)(1-\sigma(a))$ (differentiate $1/(1+e^{-a})$). Then

$$\frac{\partial \ell}{\partial a} = -\frac{y}{p} p(1-p) + \frac{1-y}{1-p} p(1-p) = p - y.$$

Prediction minus target — the same form as squared error’s residual. By rule 4 of section 1.5, summed over examples and with the ridge penalty,

$$\nabla_{\mathbf{w}} J = \mathbf{X}^\top (\mathbf{p} - \mathbf{y}) + \lambda \mathbf{w}, \quad \nabla_{\mathbf{w}}^2 J = \mathbf{X}^\top \mathbf{\Lambda} \mathbf{X} + \lambda \mathbf{I}, \quad \mathbf{\Lambda} = \text{diag}(p^{(i)}(1-p^{(i)})).$$

For $\mathbf{v} \neq \mathbf{0}$, $\mathbf{v}^\top \nabla^2 J \mathbf{v} = \sum_i p^{(i)}(1-p^{(i)})(\mathbf{x}^{(i)\top} \mathbf{v})^2 + \lambda \|\mathbf{v}\|^2 > 0$ when $\lambda > 0$. So the penalised objective is strictly convex; it also grows without bound as $\|\mathbf{w}\| \rightarrow \infty$ (the penalty does), so a minimiser exists and is unique. **Newton’s method** uses the Hessian directly, $\mathbf{w} \leftarrow \mathbf{w} - (\nabla^2 J)^{-1} \nabla J$, and is the iteratively reweighted least squares of generalised linear models.

Without regularisation, the minimiser may not exist. If some $\mathbf{w}$ separates the classes perfectly ($\mathbf{w}^\top \mathbf{x}^{(i)} > 0$ exactly for the positives), then scaling it up by $c \rightarrow \infty$ drives every $p^{(i)}$ towards its label and the loss towards zero, which is never reached. Gradient descent then keeps increasing $\|\mathbf{w}\|$ forever (section 3.8). Its *direction* converges to the maximum-margin separator (Soudry et al., 2018) — an early example of an optimiser’s **implicit bias**.

Softmax. For K classes the same calculation gives $\partial \ell / \partial a_k = p_k - \mathbf{1}_{y=k}$. Adding a constant to every logit leaves the probabilities unchanged, so softmax weights are only determined up to a shared shift — a non-identifiability that regularisation or a reference class removes. Compute softmax and cross-entropy with log-sum-exp (section 1.5).

3.5.7 Step 7: probabilities, thresholds and class weights are three different things

Thresholds from costs. Suppose a false positive costs c_{FP} , a false negative c_{FN} , and correct decisions cost nothing. At an input with calibrated probability p of being positive, predicting positive has expected cost $c_{\text{FP}}(1-p)$ and predicting negative has expected cost $c_{\text{FN}}p$. Predict positive when

$$p > \frac{c_{FP}}{c_{FP} + c_{FN}}.$$

Equal costs give 0.5. If missing a diseased leaf costs four times a false alarm, the threshold is 0.2. **The model's probabilities did not change; the decision did.**

Class weights change the probabilities. Weighting the loss as $-ay \log p - b(1 - y) \log(1 - p)$ moves its population minimiser. At an input where the true probability is q , the expected weighted loss $-aq \log p - b(1 - q) \log(1 - p)$ is minimised where its derivative $-aq/p + b(1 - q)/(1 - p)$ vanishes:

$$p^* = \frac{aq}{aq + b(1 - q)} \neq q \quad \text{unless } a = b.$$

With $a = 4$, $b = 1$ and $q = 0.2$, the weighted model reports $p^* = 0.5$. Reweighting (or balanced sampling) can help a decision, but it **mis-calibrates** the probabilities unless you correct them. Estimate calibrated probabilities first; then choose the threshold from costs; use weights only for a reason you can state.

3.6 Numerical worked example

Ridge in one dimension. Reuse $\mathbf{x} = (1, 2)^\top$, $\mathbf{y} = (1, 3)^\top$ (section 1.6). Least squares gives $w = 7/5 = 1.4$. With $\lambda = 2$: $w = 7/(5 + 2) = 1$. Training squared error rises from 0.20 to $(1 - 1)^2 + (2 - 3)^2 = 1$, and the weight shrinks by the factor $s^2/(s^2 + \lambda) = 5/7$. Whether this helps on new data is a question for a validation set.

Soft thresholding. $\text{soft}(-3, 1) = -2$; $\text{soft}(0.5, 1) = 0$; $\text{soft}(2, 1) = 1$.

One proximal step. With $\mathbf{X}$ the single column above, $L = 5$, $\eta = 0.2$, $\lambda = 2$, starting at $w_0 = 0$: $v = 0 - 0.2 \cdot (0 - 7) = 1.4$, then $w_1 = \text{soft}(1.4, 0.4) = 1.0$. The next step from $w_1 = 1$: $v = 1 - 0.2(5 - 7) = 1.4$, $w_2 = 1.0$ — a fixed point. Check the optimality condition: $\mathbf{x}^\top(\mathbf{y} - \mathbf{x}w) = 7 - 5 = 2 = \lambda \text{sign}(1)$. ✓

A logistic gradient. One example with $\mathbf{x} = (1, 2)^\top$, $y = 1$ and current $p = 0.25$: the gradient is $(p - y)\mathbf{x} = (-0.75, -1.5)^\top$. A gradient *descent* step moves $\mathbf{w}$ along $+(0.75, 1.5)$, increasing the logit and so p — the right direction for a positive example.

A threshold. $c_{FN} = 4$, $c_{FP} = 1$: threshold 0.2. A leaf with $p = 0.3$ is “healthy” at the default 0.5 but flagged under the cost rule.

3.7 Small implementation and checks

The lab is `labs/ch03-regularization/regularization.py`. The three central functions:

```
def ridge(X, y, lam):                                     # SVD shrinkage, step 2
    U, s, Vt = np.linalg.svd(X, full_matrices=False)
    return Vt.T @ ((s / (s**2 + lam)) * (U.T @ y))

def soft_threshold(v, t):                                  # step 4
    return np.sign(v) * np.maximum(np.abs(v) - t, 0.0)

def lasso_ista(X, y, lam, iters=5000):                    # step 5
    eta = 1.0 / np.linalg.norm(X, 2) ** 2
```



```

w = np.zeros(X.shape[1])
for _ in range(iters):
    w = soft_threshold(w - eta * X.T @ (X @ w - y), eta * lam)
return w

```

The logistic loss is written with `np.logaddexp(0, a) - y * a`, which equals the binary cross-entropy and never takes the log of zero. The tests in `test_regularization.py` check:

- ridge via the SVD equals the normal-equation solution, and has the right limits ($\lambda \rightarrow 0$: least squares; $\lambda \rightarrow \infty$: zero; $\text{df}(0) = n$);
- soft thresholding on hand-computed values;
- the Lasso solution satisfies the optimality conditions of step 4 to 10^{-5} , is sparse, and is all zeros above $\lambda_{\max}$;
- bias rises, variance falls, and their sum has an interior minimum;
- the logistic gradient *and Hessian* match finite differences, and the sigmoid is finite at ± 1000 ;
- Newton’s method reaches a stationary point;
- on separable data the weight norm keeps growing while the loss keeps falling;
- the threshold and class-weight formulas of step 7.

3.8 Experiment and expected observations

Run `python labs/ch03-regularization/regularization.py` (seed 0, under a second). The regression data have $m = 60$ examples and $n = 20$ features with pairwise correlation 0.5; only the first four weights are nonzero, $(3, -2, 1.5, 1)$.

Selection. The Lasso with $\lambda = 10$ keeps exactly features 0–3 and sets the other sixteen to zero, with optimality-condition violation 5.5×10^{-8} . Its weights, $(2.77, -1.39, 1.16, 0.99)$, are all pulled towards zero: the price of selection is **shrinkage bias** on the features kept (a common fix is to refit least squares on the selected features — the “relaxed Lasso”). Ridge with the same λ keeps all twenty, with irrelevant weights up to 0.42 in size and 13.9 effective parameters out of 20.

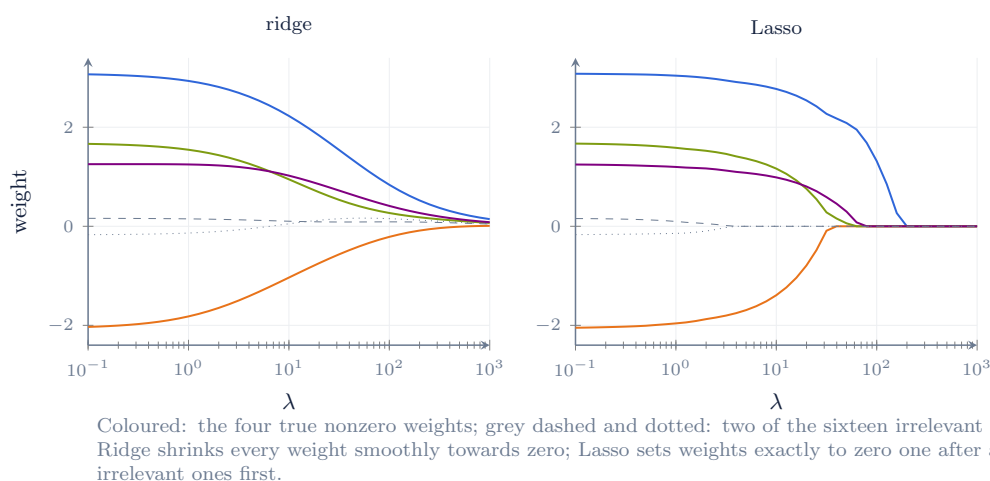


Figure 3.2: Coefficient paths as λ grows. Ridge shrinks everything smoothly; the Lasso drops weights to exactly zero one at a time.

Bias and variance. Now a noisier regime: $m = 30$, the same 20 features, noise standard deviation 1.5. Hold the inputs fixed, redraw the noise 200 times, and average over the training inputs:

λ	0.01	1	3	10	100
squared bias	0.010	0.070	0.282	0.983	4.252
variance	1.471	1.122	0.849	0.488	0.072
sum	1.481	1.192	1.131	1.471	4.324

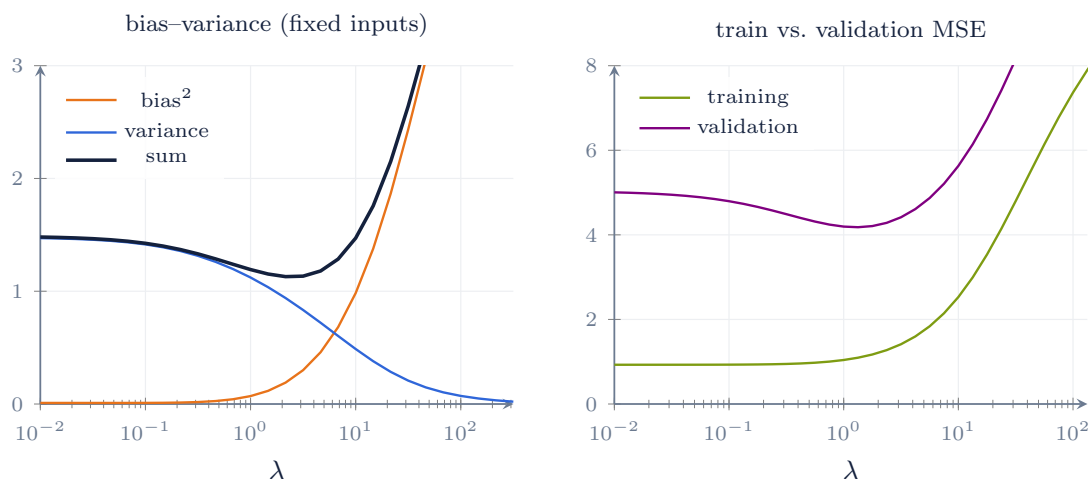


Figure 3.3: Left: bias–variance decomposition of ridge on fixed inputs. Right: training and validation error for one training set of 30 examples. Both have their minimum at a moderate λ .

What to notice:

1. **At $\lambda \approx 0$ the variance is $\sigma^2 n/m$,** as step 3 predicts: $2.25 \times 20/30 = 1.5$, against 1.47 measured.
2. **A moderate penalty cuts the total error by about a quarter** (1.48 to 1.13), mostly by removing variance.
3. **Training error only rises with λ** (figure 3.3, right). It cannot choose λ . The validation curve has its minimum at $\lambda \approx 1.6$, with validation MSE 4.19 against 5.01 for least squares. (Validation MSE includes the noise variance 2.25 that no model can remove.)

Logistic regression. On 200 examples drawn from true weights $(-0.5, 2, -1)$, Newton’s method with a tiny penalty converges to $(-0.40, 1.66, -0.92)$ with gradient norm 5×10^{-15} in a handful of iterations. On a *separable* dataset, unregularised gradient descent gives:

steps	10	100	1000	10000
$\ \mathbf{w}\ $	1.36	2.41	4.12	7.02
mean loss	4.4×10^{-2}	1.2×10^{-2}	3.2×10^{-3}	5.6×10^{-4}

The weights never settle. Any “converged” weights you report are an artefact of when you stopped; the probabilities become absurdly confident. A small λ fixes both.

3.9 Moving pieces: diagnosing and fixing

Moving piece	Symptom when it is wrong	How to confirm	Fix
Value of λ	too small: unstable weights, big train-validation gap; too large: everything near zero, high training error	validation curve over a log grid ($10^{-3} \dots 10^3$)	choose by validation (or cross-validation, chapter 5); report the grid
Scale of λ (sum vs. mean)	a λ copied from a paper or library gives a very different fit	write the objective; duplicate the data and refit	convert: mean-loss λ = summed-loss λ/m
Feature scaling	penalty punishes features in small units hardest; selection depends on units	compare fits before and after standardising	standardise with training statistics; penalise standardised weights
Intercept	predictions pulled towards zero; probabilities biased	is the ones column penalised?	exclude the intercept from the penalty
Correlated features (Lasso)	the selected subset changes between bootstrap samples	selection frequency over 100 bootstraps	elastic net; report stability, not one support
Lasso shrinkage bias	kept weights systematically too small	compare with least squares refit on the support	relaxed Lasso / refit; report both
Separable classes	logistic weights grow without bound; probabilities near 0/1	weight norm keeps growing with iterations	any $\lambda > 0$; early stopping as implicit regularisation
Optimisation	slow or oscillating ISTA / gradient descent	objective per iteration; step size vs. $1/L$	step $\leq 1/\ \mathbf{X}\ _2^2$; FISTA; coordinate descent; Newton for small n
Class imbalance	high accuracy, no positives found	confusion matrix; per-class recall	choose the threshold from costs; report recall and precision (chapter 5)
Class weights / resampling	good recall, over-confident positive probabilities	reliability diagram (chapter 5)	recalibrate after weighting, or threshold a calibrated model instead

First things to try when a regularised model disappoints: (1) standardise and refit; (2) plot the validation curve over a wide log grid; (3) bootstrap the fit and look at how stable the weights and the selected support are; (4) for classifiers, look at the confusion matrix at the threshold you will actually use.

3.10 Application, limitations and failure cases

Application (educational case study): crop disease with few labels. With a handful of labelled field photographs, a regularised linear model on fixed image features (colour, texture, or frozen deep features from chapter 18) is a strong, honest baseline. Choose λ by validation, report per-class recall rather than accuracy, and choose the decision threshold from the stated cost of missing a diseased plant. If a feature such as “photographed at farm 3” or the image’s background colour gets a large weight, suspect a **shortcut**: the model may be recognising the farm, not the disease. Regularisation does not establish causality; only an evaluation on new farms does (chapter 5).

Failure cases:

- **Lasso picks one of a correlated group arbitrarily.** If two features carry the same information, which one survives depends on noise. Do not interpret the chosen subset as “the” important features without a stability check.
- **Penalised weights are biased.** Confidence intervals that ignore the penalty are wrong.
- **Calibration after reweighting.** A model trained with class weights or balanced batches reports inflated probabilities for the up-weighted class (step 7).
- **Linear boundaries.** No amount of regularisation lets a linear classifier separate classes that need a curved boundary (chapter 4).

3.11 Bridge to a research paper

Tibshirani (1996), *Regression shrinkage and selection via the lasso* (Journal of the Royal Statistical Society B, 58(1):267–288).

- **Problem:** least squares has high variance and keeps every variable; subset selection is discrete and unstable.
- **Inputs and outputs:** a design matrix and targets; a sparse weight vector.
- **Objective:** least squares subject to $\|\mathbf{w}\|_1 \leq t$.
- **Central claim:** the Lasso combines the stability of ridge with the interpretability of subset selection.
- **Assumptions to question:** how correlated may the irrelevant features be with the relevant ones before the right support cannot be recovered? (Later theory names this the *irrepresentable condition*.)
- **Affordable experiment:** using the lab’s `sparse_regression`, vary the feature correlation $\rho \in \{0, 0.5, 0.8, 0.95\}$ and record, over 100 bootstrap samples, how often each feature is selected at the validation-chosen λ . Plot selection frequency against ρ , separately for relevant and irrelevant features.

3.12 Memory map

Summary. A penalty is a stated preference among solutions. Ridge prefers small weights: it has a unique closed-form solution, shrinks each singular direction by $s_j^2/(s_j^2 + \lambda)$, and trades bias for variance. The Lasso prefers few nonzero weights: soft thresholding, explained by subgradients or by the corners of the L^1 ball, sets weak features exactly to zero, at the cost of shrinking the kept ones. Logistic regression is the Bernoulli negative log-likelihood; its gradient is $\mathbf{X}^\top(\mathbf{p} - \mathbf{y})$, its penalised optimum is unique, and on separable data it needs a penalty to have one at all. Probabilities, thresholds and class weights answer different questions; keep them separate.

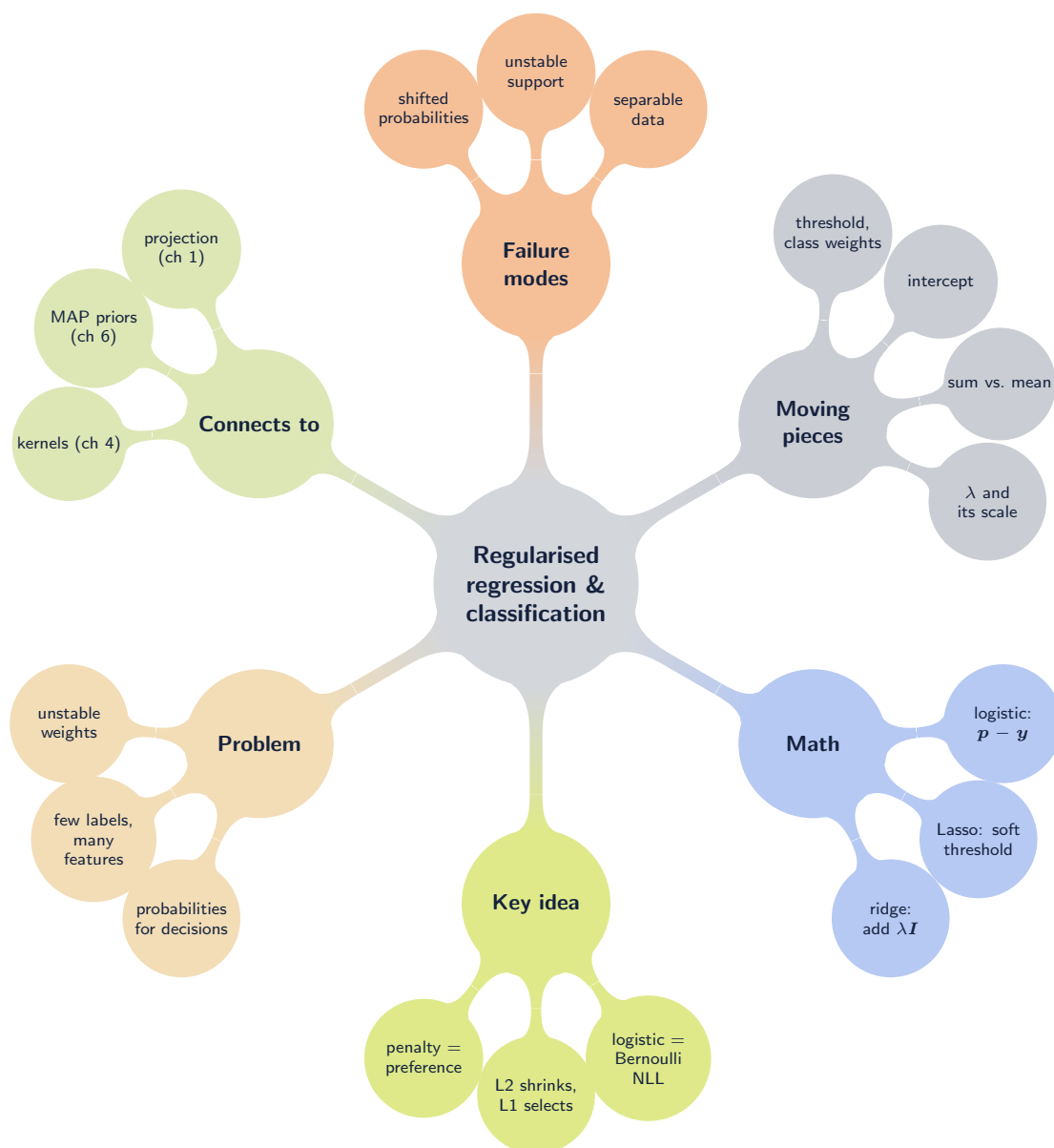


Figure 3.4: Memory map of chapter 3.

3.13 Exercises

1. *Conceptual.* Soft-threshold $a = -3$ at $\lambda = 1$ and $a = 0.5$ at $\lambda = 1$.
2. *Conceptual.* Why can two implementations with the same numerical λ produce different fits? List three reasons.
3. *Derivation.* Show that ridge with $\lambda > 0$ has a unique solution even when $n > m$, and write it in the “dual” form $\mathbf{w}_\lambda = \mathbf{X}^\top (\mathbf{X} \mathbf{X}^\top + \lambda \mathbf{I})^{-1} \mathbf{y}$. (This form is the start of chapter 4.)
4. *Derivation.* Derive the summed variance $\sigma^2 \sum_j (s_j^2 / (s_j^2 + \lambda))^2$ of the ridge fitted values.
5. *Derivation.* Show that $\lambda_{\max} = \|\mathbf{X}^\top \mathbf{y}\|_\infty$ is the smallest λ for which $\mathbf{w} = \mathbf{0}$ solves the Lasso.
6. *Derivation.* Derive $\partial \ell / \partial a_k = p_k - \mathbf{1}_{y=k}$ for softmax cross-entropy.
7. *Implementation.* Implement coordinate descent for the Lasso (update one w_j at a time by soft thresholding its partial residual) and compare its speed and solution with ISTA.
8. *Implementation.* Add FISTA (ISTA with momentum) and plot the objective against iterations for both.
9. *Interpretation.* A logistic example has $\mathbf{x} = (1, 2)^\top$, $y = 1$, $p = 0.25$. Give its unregularised gradient and say which way a descent step moves the logit.
10. *Failure analysis.* Train logistic regression with class weights 4:1 on data where the true positive rate is 20%. Plot a reliability diagram (chapter 5). Then recalibrate with the formula of step 7 inverted and plot again.
11. *Open question.* For crop-disease features from several farms, design an experiment to tell whether a large weight on a colour feature reflects disease or a farm-specific background.

3.13.1 Answer discussions

1. -2 and 0 .
2. Sum vs. mean data loss; different feature standardisation; whether the intercept is penalised (also: $\lambda/2$ vs. λ in front of the penalty).
3. $\mathbf{X}^\top \mathbf{X} + \lambda \mathbf{I}$ is positive definite (step 1). The identity $(\mathbf{X}^\top \mathbf{X} + \lambda \mathbf{I}) \mathbf{X}^\top = \mathbf{X}^\top (\mathbf{X} \mathbf{X}^\top + \lambda \mathbf{I})$ gives the dual form.
4. $\hat{\mathbf{y}}_\lambda = \mathbf{U} \text{diag}(s_j^2 / (s_j^2 + \lambda)) \mathbf{U}^\top \mathbf{y}$; its covariance is σ^2 times the square of that matrix, whose trace is the stated sum.
5. At $\mathbf{w} = \mathbf{0}$ the residual is $\mathbf{y}$; the zero-weight conditions require $|\mathbf{X}_{:,j}^\top \mathbf{y}| \leq \lambda$ for all j .
6. $\ell = -a_y + \log \sum_{k'} \exp(a_{k'})$; differentiate.
7. Coordinate descent is usually much faster on standardised features and gives the same solution up to tolerance.
8. FISTA’s objective gap falls as $O(1/t^2)$ instead of $O(1/t)$, but the objective is not monotone.
9. $(-0.75, -1.5)^\top$; descent increases the logit.
10. The weighted model’s predicted probabilities sit above the diagonal for positives; inverting $p^* = 4q / (4q + 1 - q)$ gives $q = p^* / (4 - 3p^*)$ and restores calibration.
11. Evaluate on held-out farms; mask or crop backgrounds; test whether the weight survives within-farm comparisons. A weight that vanishes when farm is controlled for was a shortcut.

3.14 Self-check and common mistakes

- ☐ I can derive the ridge solution and its shrinkage factors from the SVD.
- ☐ I can derive soft thresholding from the subgradient condition.
- ☐ I can write and check the Lasso optimality conditions for a computed solution.
- ☐ I can derive the logistic gradient $\mathbf{X}^\top(\mathbf{p} - \mathbf{y})$ in three lines.
- ☐ I can explain why class weights change probabilities but cost thresholds do not.

Common mistakes: comparing λ across codebases without checking sum vs. mean; penalising the intercept; not standardising before penalising; choosing λ on the test set; reading a Lasso support as causal; reporting weights from unregularised logistic regression on separable data; using a 0.5 threshold by default; trusting probabilities from a model trained with class weights.

References

- I. Goodfellow, Y. Bengio and A. Courville. *Deep Learning*. MIT Press, 2016, sections 5.2 (capacity, regularisation) and 7.1 (parameter norm penalties).
- A. N. Tikhonov. Solution of incorrectly formulated problems and the regularization method. *Soviet Mathematics Doklady*, 4:1035–1038, 1963.
- A. E. Hoerl and R. W. Kennard. Ridge regression: biased estimation for nonorthogonal problems. *Technometrics*, 12(1):55–67, 1970.
- R. Tibshirani. Regression shrinkage and selection via the lasso. *JRSS B*, 58(1):267–288, 1996.
- I. Daubechies, M. Defrise and C. De Mol. An iterative thresholding algorithm for linear inverse problems with a sparsity constraint. *Comm. Pure Appl. Math.*, 57(11):1413–1457, 2004.
- A. Beck and M. Teboulle. A fast iterative shrinkage-thresholding algorithm for linear inverse problems. *SIAM J. Imaging Sciences*, 2(1):183–202, 2009.
- J. Friedman, T. Hastie and R. Tibshirani. Regularization paths for generalized linear models via coordinate descent. *Journal of Statistical Software*, 33(1), 2010.
- H. Zou and T. Hastie. Regularization and variable selection via the elastic net. *JRSS B*, 67(2):301–320, 2005.
- D. R. Cox. The regression analysis of binary sequences. *JRSS B*, 20(2):215–242, 1958.
- J. A. Nelder and R. W. M. Wedderburn. Generalized linear models. *JRSS A*, 135(3):370–384, 1972.
- D. Soudry, E. Hoffer, M. S. Nacson, S. Gunasekar and N. Srebro. The implicit bias of gradient descent on separable data. *JMLR*, 19(70):1–57, 2018.
- T. Hastie, R. Tibshirani and J. Friedman. *The Elements of Statistical Learning*, 2nd edition. Springer, 2009, chapters 3–4.

Chapter 4

Duality and kernels

Prerequisites: chapter 3 (ridge, logistic regression). Familiarity with constrained optimisation (Boyd and Vandenberghe, 2004, chapter 5) is helpful, but this chapter derives what it uses.

Learning outcomes: after this chapter you can

1. explain how a linear model in a feature space produces a nonlinear model in input space, and why only inner products are needed;
2. derive the dual form of ridge regression twice — by a matrix identity and by Lagrangian duality — and compute kernel ridge predictions;
3. decide whether a function is a valid kernel, using the PSD definition and closure rules, and recognise invalid ones;
4. state and prove the representer theorem, and derive the dual of the soft-margin SVM and the eigenproblem of kernel PCA;
5. weigh the costs ($O(m^3)$ time, $O(m^2)$ memory) and use random features to scale.

4.1 A concrete problem

A microscope image pipeline measures two numbers for each cell: its size and its stain intensity, both centred on the population average. Healthy cells cluster near the average; abnormal cells sit on a ring around them — too large *or* too small, too pale *or* too dark. In the plane of the two features the classes are concentric circles. No straight line separates them, and logistic regression (chapter 3) does no better than a coin toss.

The fix is to change the representation. Add the features x_1^2 , x_2^2 and x_1x_2 , and the circle $x_1^2 + x_2^2 = r^2$ becomes a *straight* boundary in the new coordinates. A linear method in the bigger space is a nonlinear method in the original one. Two questions follow. How do we choose the new features when we do not know the right shape? And what if the useful feature space is enormous — or infinite? Kernels answer both: they let us work in such spaces by computing only **inner products between examples**.

4.2 How we got here

Positive definite functions (1909–1950). James Mercer (1909), studying integral equations, characterised the functions $k(x, x')$ that behave like inner products: those whose matrices $[k(x_i, x_j)]$ are always positive semidefinite. Nachman Aronszajn (1950) built the theory of *reproducing kernel Hilbert spaces* (RKHS), the function spaces such kernels define.

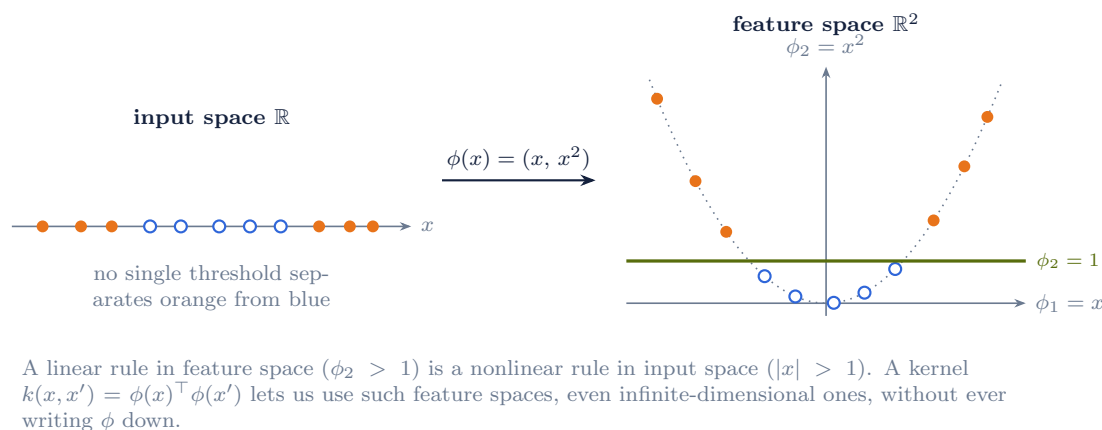


Figure 4.1: A feature map makes a nonlinear problem linear: points labelled by $|x| > 1$ become separable by a horizontal line after $\phi(x) = (x, x^2)$.

The kernel trick (1964–1992). Mark Aizerman, Emmanuel Braverman and Lev Rozonoer (1964) used kernels as “potential functions” in pattern recognition, effectively running the perceptron in a feature space. George Kimeldorf and Grace Wahba (1970–1971) proved that spline smoothing problems have solutions that are finite combinations of kernel functions centred at the data — the **representer theorem**. The idea became central in 1992, when Bernhard Boser, Isabelle Guyon and Vladimir Vapnik combined it with the maximum-margin classifier: the optimal separating hyperplane depends on the data only through inner products, so replacing them by a kernel gives a nonlinear classifier with the same convex optimisation. Corinna Cortes and Vapnik (1995) added the soft margin, and the **support vector machine** (SVM) was born.

Kernelising everything (1998–2007). If an algorithm can be written with inner products alone, it can be kernelised: kernel PCA (Schölkopf, Smola and Müller, 1998), kernel ridge regression (Saunders, Gammernan and Vovk, 1998), Gaussian processes (chapter 7). Through the 2000s kernel methods, with hand-designed features such as SIFT and HOG, were the state of the art in many vision benchmarks.

What broke. Two things. *Cost:* the Gram matrix has m^2 entries and solving with it costs $O(m^3)$, which is painful beyond tens of thousands of examples. Ali Rahimi and Ben Recht (2007) answered with random features that approximate a kernel cheaply — this chapter’s paper bridge. *Representation:* a kernel on raw pixels measures similarity by Euclidean distance between images, which is a poor notion of visual similarity (a one-pixel shift moves an image far away). From 2012, deep networks *learned* the feature map instead of fixing it (chapters 11–14). Kernels did not disappear: they live on in Gaussian processes (chapter 7), in attention, which is a kernel-weighted average (chapter 14), and in theory, where the neural tangent kernel (Jacot, Gabriel and Hongler, 2018) describes very wide networks as kernel machines.

4.3 Assumptions and notation

Symbol	Shape	Meaning
$\phi(\mathbf{x})$	N	feature map into a (possibly huge) feature space

Symbol	Shape	Meaning
Φ	$m \times N$	feature matrix, $\Phi_{i,:} = \phi(\mathbf{x}^{(i)})^\top$
$k(\mathbf{x}, \mathbf{x}')$	scalar	kernel, $k(\mathbf{x}, \mathbf{x}') = \phi(\mathbf{x})^\top \phi(\mathbf{x}')$
K	$m \times m$	Gram matrix, $K_{i,j} = k(\mathbf{x}^{(i)}, \mathbf{x}^{(j)})$, so $K = \Phi \Phi^\top$
k_*	m	kernel between a new point and the training points
α	m	dual coefficients, one per training example
λ	scalar	ridge regularisation weight (summed loss, chapter 3)
ℓ	scalar	RBF length-scale

Assumptions:

1. A good model is linear in *some* feature space, and we can express similarity between examples through a kernel that matches that space.
2. The data set is small or moderate (m up to roughly 10^4 for exact methods).
3. The kernel's notion of similarity is meaningful for the inputs — e.g. Euclidean distance between *good features*, not necessarily between raw pixels.

4.4 Mathematical formulation

Ridge in feature space (primal, N unknowns):

$$\min_{\mathbf{w} \in \mathbb{R}^N} \frac{1}{2} \|\Phi \mathbf{w} - \mathbf{y}\|^2 + \frac{\lambda}{2} \|\mathbf{w}\|^2, \quad f(\mathbf{x}) = \mathbf{w}^\top \phi(\mathbf{x}).$$

Kernel ridge (dual, m unknowns):

$$\alpha = (K + \lambda I)^{-1} \mathbf{y}, \quad f(\mathbf{x}) = \sum_{i=1}^m \alpha_i k(\mathbf{x}^{(i)}, \mathbf{x}) = \mathbf{k}_*^\top \alpha.$$

Common kernels.

Kernel	$k(\mathbf{x}, \mathbf{x}')$	Feature space	Assumption it encodes
linear	$\mathbf{x}^\top \mathbf{x}'$	the inputs themselves	linear structure
polynomial	$(\mathbf{x}^\top \mathbf{x}' + c)^d$	all monomials up to degree d	interactions up to order d
RBF (Gaussian)	$\sigma_f^2 \exp(-\ \mathbf{x} - \mathbf{x}'\ ^2 / 2\ell^2)$	infinite-dimensional	smooth functions; nearby inputs behave alike
Laplacian	$\exp(-\ \mathbf{x} - \mathbf{x}'\ / \ell)$	infinite-dimensional	rougher, less smooth functions

Kernel	$k(\mathbf{x}, \mathbf{x}')$	Feature space	Assumption it encodes
periodic	$\sigma_f^2 \exp(-2 \sin^2(\pi t - t' /p)/\ell^2)$	infinite-dimensional	repetition with period p

The SVM (for labels $y^{(i)} \in \{-1, +1\}$):

$$\min_{\mathbf{w}, b} \frac{1}{2} \|\mathbf{w}\|^2 + C \sum_i \max(0, 1 - y^{(i)}(\mathbf{w}^\top \phi(\mathbf{x}^{(i)} + b)),$$

the **hinge loss** plus a penalty that, for separable data, maximises the geometric margin $1/\|\mathbf{w}\|$ (section 1.5, step 8).

4.5 Derivation, with reasons for each step

4.5.1 Step 1: the dual form of ridge, by a matrix identity

Chapter 3 gave $\mathbf{w} = (\Phi^\top \Phi + \lambda \mathbf{I}_N)^{-1} \Phi^\top \mathbf{y}$. The identity

$$(\Phi^\top \Phi + \lambda \mathbf{I}_N) \Phi^\top = \Phi^\top \Phi \Phi^\top + \lambda \Phi^\top = \Phi^\top (\Phi \Phi^\top + \lambda \mathbf{I}_m)$$

lets us move the inverse across: multiply on the left by $(\Phi^\top \Phi + \lambda \mathbf{I}_N)^{-1}$ and on the right by $(\Phi \Phi^\top + \lambda \mathbf{I}_m)^{-1}$ (both invertible for $\lambda > 0$):

$$\mathbf{w} = \Phi^\top (K + \lambda \mathbf{I}_m)^{-1} \mathbf{y} = \Phi^\top \boldsymbol{\alpha}.$$

Two things happened. **The weight vector is a combination of training feature vectors** ($\mathbf{w} = \sum_i \alpha_i \phi(\mathbf{x}^{(i)})$). And **everything we need is an inner product**: fitting needs $K = \Phi \Phi^\top$, predicting needs $\mathbf{w}^\top \phi(\mathbf{x}) = \boldsymbol{\alpha}^\top \Phi \phi(\mathbf{x}) = \boldsymbol{\alpha}^\top \mathbf{k}_*$. If we can compute k directly, we never need ϕ . That is the **kernel trick**.

Which form is cheaper? The primal solves an $N \times N$ system, the dual an $m \times m$ one. With few features and many examples ($N \ll m$), use the primal. With many (or infinitely many) features, use the dual. “Kernel” does not automatically mean “faster”.

4.5.2 Step 2: the same result by Lagrangian duality

The matrix identity is quick but hides a general tool. Rewrite ridge with explicit residuals $\boldsymbol{\xi}$:

$$\min_{\mathbf{w}, \boldsymbol{\xi}} \frac{1}{2} \|\boldsymbol{\xi}\|^2 + \frac{\lambda}{2} \|\mathbf{w}\|^2 \quad \text{subject to} \quad \boldsymbol{\xi} = \mathbf{y} - \Phi \mathbf{w}.$$

Attach a **Lagrange multiplier** α_i to each constraint:

$$\mathcal{L}(\mathbf{w}, \boldsymbol{\xi}, \boldsymbol{\alpha}) = \frac{1}{2} \|\boldsymbol{\xi}\|^2 + \frac{\lambda}{2} \|\mathbf{w}\|^2 + \boldsymbol{\alpha}^\top (\mathbf{y} - \Phi \mathbf{w} - \boldsymbol{\xi}).$$

For fixed $\boldsymbol{\alpha}$, minimise over the primal variables. Setting gradients to zero: $\nabla_{\boldsymbol{\xi}} \mathcal{L} = \boldsymbol{\xi} - \boldsymbol{\alpha} = \mathbf{0}$ and $\nabla_{\mathbf{w}} \mathcal{L} = \lambda \mathbf{w} - \Phi^\top \boldsymbol{\alpha} = \mathbf{0}$, so

$$\boldsymbol{\xi} = \boldsymbol{\alpha}, \quad \boldsymbol{w} = \frac{1}{\lambda} \boldsymbol{\Phi}^\top \boldsymbol{\alpha}.$$

Substituting back gives the **dual function** $g(\boldsymbol{\alpha}) = -\frac{1}{2}\|\boldsymbol{\alpha}\|^2 - \frac{1}{2\lambda}\boldsymbol{\alpha}^\top \boldsymbol{K}\boldsymbol{\alpha} + \boldsymbol{\alpha}^\top \boldsymbol{y}$. For any $\boldsymbol{\alpha}$, $g(\boldsymbol{\alpha})$ is a lower bound on the primal optimum (*weak duality*: the Lagrangian equals the objective on feasible points and we minimised over a larger set). For convex problems with linear constraints like this one the best lower bound is tight (*strong duality*). Maximising g : $\boldsymbol{y} - \boldsymbol{\alpha} - \boldsymbol{K}\boldsymbol{\alpha}/\lambda = \mathbf{0}$, i.e. $\boldsymbol{\alpha} = \lambda(\boldsymbol{K} + \lambda\boldsymbol{I})^{-1}\boldsymbol{y}$, and then $\boldsymbol{w} = \boldsymbol{\Phi}^\top(\boldsymbol{K} + \lambda\boldsymbol{I})^{-1}\boldsymbol{y}$ — step 1 again (the multipliers here are λ times step 1's $\boldsymbol{\alpha}$).

Why the detour: the multipliers have a meaning. $\alpha_i = \xi_i$ is example i 's **residual**: examples the model fits badly get large dual weights. In the SVM below, the same machinery makes most multipliers *exactly zero*.

4.5.3 Step 3: which functions are kernels?

A symmetric function k is a **valid (positive semidefinite) kernel** if every Gram matrix it produces is positive semidefinite: $\boldsymbol{c}^\top \boldsymbol{K} \boldsymbol{c} \geq 0$ for every finite set of inputs and every $\boldsymbol{c} \in \mathbb{R}^m$.

Any inner product of features is valid: $\boldsymbol{c}^\top \boldsymbol{K} \boldsymbol{c} = \sum_{i,j} c_i c_j \phi(\boldsymbol{x}^{(i)})^\top \phi(\boldsymbol{x}^{(j)}) = \|\sum_i c_i \phi(\boldsymbol{x}^{(i)})\|^2 \geq 0$. The converse — every valid kernel is an inner product in *some* feature space — is Mercer's and Aronszajn's theorem. So “valid kernel” and “inner product of features” are the same thing.

Closure rules let you build kernels from kernels. If k_1, k_2 are valid, so are:

1. $a k_1$ for $a \geq 0$ (scale the quadratic form);
2. $k_1 + k_2$ (add the quadratic forms; feature map = concatenation);
3. $k_1 \cdot k_2$ (pointwise product; the Gram matrix is the element-wise product of PSD matrices, which is PSD by the Schur product theorem; feature map = all pairwise products);
4. $f(\boldsymbol{x}) k_1(\boldsymbol{x}, \boldsymbol{x}') f(\boldsymbol{x}')$ for any function f ;
5. $\exp(k_1)$ (a limit of polynomials with nonnegative coefficients, by rules 1–3).

The RBF kernel is valid: $\exp(-\|\boldsymbol{x} - \boldsymbol{x}'\|^2/2\ell^2) = e^{-\|\boldsymbol{x}\|^2/2\ell^2} \exp(\boldsymbol{x}^\top \boldsymbol{x}'/\ell^2) e^{-\|\boldsymbol{x}'\|^2/2\ell^2}$ — rule 5 applied to the linear kernel, then rule 4.

Invalid “kernels”. $k(\boldsymbol{x}, \boldsymbol{x}') = -\boldsymbol{x}^\top \boldsymbol{x}'$ has a negative diagonal entry at any nonzero point. The popular sigmoid “kernel” $\tanh(\alpha \boldsymbol{x}^\top \boldsymbol{x}' + c)$ is not PSD in general (section 4.8 finds an eigenvalue of -21.9). Symmetry alone is not enough.

In floating point, a valid kernel's Gram matrix can show eigenvalues like -10^{-14} . That is rounding, not invalidity. Compare against $u\|\boldsymbol{K}\|$; add a small jitter $\epsilon \boldsymbol{I}$ if a factorisation needs it (appendix C).

4.5.4 Step 4: the representer theorem

Step 1 found $\boldsymbol{w}$ in the span of the training features for ridge. This is general.

Theorem. For any loss L of the predictions $f(\boldsymbol{x}^{(1)}), \dots, f(\boldsymbol{x}^{(m)})$ and any strictly increasing function Ω , every minimiser of $L(\boldsymbol{w}^\top \phi(\boldsymbol{x}^{(1)}), \dots, \boldsymbol{w}^\top \phi(\boldsymbol{x}^{(m)})) + \Omega(\|\boldsymbol{w}\|)$ has the form $\boldsymbol{w} = \sum_i \alpha_i \phi(\boldsymbol{x}^{(i)})$.

Proof. Split any $\boldsymbol{w}$ into a part in the span of the training features and a part $\boldsymbol{w}_\perp$ orthogonal to all of them. The predictions $\boldsymbol{w}^\top \phi(\boldsymbol{x}^{(i)})$ do not depend on $\boldsymbol{w}_\perp$, so the loss does not

either; but $\|\mathbf{w}\|^2 = \|\mathbf{w}_{\parallel}\|^2 + \|\mathbf{w}_{\perp}\|^2$ (Pythagoras again). Setting $\mathbf{w}_{\perp} = \mathbf{0}$ keeps the loss and strictly lowers the penalty unless it was already zero. $\square$

Why it matters: an optimisation over an infinite-dimensional space reduces to m numbers. Logistic regression, SVMs, kernel ridge — all have finite-dimensional duals.

4.5.5 Step 5: the soft-margin SVM and its support vectors

Introduce slack variables $\xi_i \geq 0$ for the hinge terms: $\min \frac{1}{2}\|\mathbf{w}\|^2 + C \sum_i \xi_i$ subject to $y^{(i)}(\mathbf{w}^{\top} \phi(\mathbf{x}^{(i)}) + b) \geq 1 - \xi_i$ and $\xi_i \geq 0$. With multipliers $\alpha_i \geq 0$ and $\mu_i \geq 0$ for the two constraint families, stationarity gives

$$\mathbf{w} = \sum_i \alpha_i y^{(i)} \phi(\mathbf{x}^{(i)}), \quad \sum_i \alpha_i y^{(i)} = 0, \quad \alpha_i = C - \mu_i,$$

and the dual problem is

$$\max_{\alpha} \sum_i \alpha_i - \frac{1}{2} \sum_{i,j} \alpha_i \alpha_j y^{(i)} y^{(j)} k(\mathbf{x}^{(i)}, \mathbf{x}^{(j)}) \quad \text{s.t.} \quad 0 \leq \alpha_i \leq C, \quad \sum_i \alpha_i y^{(i)} = 0.$$

The **complementary slackness** conditions ($\alpha_i \times$ constraint slack = 0 at the optimum) constrain where each training point can lie. Write $y^{(i)} f(\mathbf{x}^{(i)})$ for its functional margin. For $C > 0$:

- $\alpha_i = 0$ implies $y^{(i)} f(\mathbf{x}^{(i)}) \geq 1$: on or beyond the margin, correctly classified. These points **do not appear in the solution**;
- $0 < \alpha_i < C$ implies $y^{(i)} f(\mathbf{x}^{(i)}) = 1$: exactly on the margin;
- $\alpha_i = C$ permits $y^{(i)} f(\mathbf{x}^{(i)}) \leq 1$: on the margin, inside it, or misclassified.

The implications run one way. A point exactly on the margin can have $\alpha_i = 0$, $\alpha_i = C$ or any value in between, so the three cases are not a strict partition of the points by position; in practice most points fall clearly into one case.

The points with $\alpha_i > 0$ are the **support vectors**. The decision function $f(\mathbf{x}) = \sum_i \alpha_i y^{(i)} k(\mathbf{x}^{(i)}, \mathbf{x}) + b$ needs only them — a sparse dual, unlike kernel ridge where every α_i is generally nonzero.

4.5.6 Step 6: kernel PCA

PCA (chapter 2) needs eigenvectors of the feature covariance. In feature space that matrix may be infinite, but the snapshot trick of section 2.5 needs only the $m \times m$ inner-product matrix. Let $\mathbf{K}_c$ be the Gram matrix of **centred** features. If $\mathbf{K}_c \mathbf{u} = \nu \mathbf{u}$ with $\|\mathbf{u}\| = 1$ and $\nu > 0$, then the feature-space direction $\mathbf{v} = \Phi_c^{\top} \mathbf{u} / \sqrt{\nu}$ is a unit principal direction (exercise 2.5), and the training codes along it are $\Phi_c \mathbf{v} = \mathbf{K}_c \mathbf{u} / \sqrt{\nu} = \sqrt{\nu} \mathbf{u}$. A new point gets code $\mathbf{k}_{c,*}^{\top} \mathbf{u} / \sqrt{\nu}$. Everything is kernels. Eigenvectors with $\nu = 0$ are excluded: they give $\Phi_c^{\top} \mathbf{u} = \mathbf{0}$, carry no variance, and there is nothing to divide by. Centring always creates at least one (the constant vector), and in practice eigenvalues below a small tolerance are dropped too.

4.5.7 Step 7: centring without features

Centring the features at their training mean $\bar{\phi} = \frac{1}{m} \sum_i \phi(\mathbf{x}^{(i)})$ changes inner products to $(\phi(\mathbf{x}) - \bar{\phi})^{\top} (\phi(\mathbf{x}') - \bar{\phi})$. Expanding, with $\mathbf{H} = \mathbf{I} - \mathbf{1}\mathbf{1}^{\top}/m$:

$K_c = HKH$ (subtract row means and column means, add the grand mean),

and for a new point, using the **training** means only, $k_{c,*}(i) = k_*(i) - \frac{1}{m} \sum_j k_*(j) - \frac{1}{m} \sum_j K_{j,i} + \frac{1}{m^2} \sum_{j,l} K_{j,l}$. Centring with the test set's own mean would change the model and leak information (chapter 5).

4.5.8 Step 8: cost, and random features

Exact kernel ridge needs the $m \times m$ Gram matrix ($O(m^2)$ memory, $O(m^2n)$ to build) and a Cholesky factorisation ($O(m^3)$). At $m = 10^5$ that is 80 GB of float64 and about 10^{15} operations. **Random Fourier features** avoid it. Bochner's theorem says a shift-invariant kernel $k(\mathbf{x} - \mathbf{x}')$ is the Fourier transform of a probability distribution $p(\boldsymbol{\omega})$; for the RBF kernel with length-scale ℓ , $p(\boldsymbol{\omega}) = \mathcal{N}(\boldsymbol{\omega}; \mathbf{0}, \ell^{-2}\mathbf{I})$. Then

$$k(\mathbf{x}, \mathbf{x}') = \mathbb{E}_{\boldsymbol{\omega}, b} [2 \cos(\boldsymbol{\omega}^\top \mathbf{x} + b) \cos(\boldsymbol{\omega}^\top \mathbf{x}' + b)], \quad b \sim \text{Uniform}[0, 2\pi],$$

and averaging D samples gives features $\mathbf{z}(\mathbf{x}) = \sqrt{2/D} (\cos(\boldsymbol{\omega}_d^\top \mathbf{x} + b_d))_{d=1}^D$ with $\mathbf{z}(\mathbf{x})^\top \mathbf{z}(\mathbf{x}') \approx k(\mathbf{x}, \mathbf{x}')$, error shrinking like $1/\sqrt{D}$. Ordinary (primal) ridge on $\mathbf{z}$ costs $O(mD^2)$ — linear in m .

4.6 Numerical worked example

A feature map by hand. For 2-D inputs, $k(\mathbf{x}, \mathbf{x}') = (\mathbf{x}^\top \mathbf{x}')^2$ has feature map $\phi(\mathbf{x}) = (x_1^2, \sqrt{2}x_1x_2, x_2^2)$. For $\mathbf{x} = (1, 2)$, $\mathbf{x}' = (3, 4)$: the raw inner product is 11, so $k = 121$. Explicitly: $\phi(\mathbf{x}) = (1, 2\sqrt{2}, 4)$, $\phi(\mathbf{x}') = (9, 12\sqrt{2}, 16)$, and $9 + 48 + 64 = 121$. The $\sqrt{2}$ accounts for the two equal cross terms in $(x_1x'_1 + x_2x'_2)^2$.

A dual solve by hand. Let $\mathbf{K} = \begin{pmatrix} 1 & 0.5 \\ 0.5 & 1 \end{pmatrix}$, $\mathbf{y} = (1, -1)^\top$, $\lambda = 0.5$. Since $\mathbf{K}\mathbf{y} = (0.5, -0.5)^\top = 0.5\mathbf{y}$, $\mathbf{y}$ is an eigenvector with eigenvalue 0.5, so $(\mathbf{K} + 0.5\mathbf{I})\mathbf{y} = \mathbf{y}$ and $\boldsymbol{\alpha} = \mathbf{y}$. A new point with $\mathbf{k}_* = (0.8, 0.2)^\top$ — closer to the first training point — gets prediction $0.8 - 0.2 = 0.6$. Nonlinear in the inputs, but fitted by one convex linear solve.

An RBF entry. With $\ell = 1$, two points at distance 2 have $k = \exp(-4/2) = e^{-2} \approx 0.135$; at distance 0.5, $e^{-0.125} \approx 0.88$. With $\ell = 0.1$ the second becomes $e^{-12.5} \approx 4 \times 10^{-6}$: almost every pair of points looks unrelated, and the model can only memorise.

Centring. For $\mathbf{K} = \begin{pmatrix} 2 & 0 \\ 0 & 0 \end{pmatrix}$ (features $\phi_1 = (\sqrt{2}, 0)$, $\phi_2 = (0, 0)$), the mean feature is $(\sqrt{2}/2, 0)$ and the centred features are $\pm(\sqrt{2}/2, 0)$, with Gram matrix $\begin{pmatrix} 0.5 & -0.5 \\ -0.5 & 0.5 \end{pmatrix}$. The formula HKH gives the same.

4.7 Small implementation and checks

The lab is `labs/ch04-kernels/kernels.py`:

```
def kernel_ridge_fit(K, y, lam):                                # alpha = (K + lam I)^{-1} y,
    ↪ via Cholesky
    L = np.linalg.cholesky(K + lam * np.eye(K.shape[0]))
```

```

    return np.linalg.solve(L.T, np.linalg.solve(L, y))

def kernel_pca(K, l=2):                                # step 6
    Kc = center_gram(K)
    lam, U = np.linalg.eigh(Kc)
    idx = np.argsort(lam)[::-1][:l]
    return U[:, idx] * np.sqrt(np.maximum(lam[idx], 0.0))

```

The tests in `test_kernels.py` check:

- the quadratic kernel equals explicit features (the 121 above);
- **primal and dual ridge agree**, and $\mathbf{w} = \Phi^\top \alpha$;
- RBF, sums and products of kernels are PSD; the tanh kernel is not;
- kernel centring matches centring explicit features, for training and test points;
- the length-scale has an interior optimum, and a tiny length-scale fits noise;
- kernel PCA makes concentric circles linearly separable;
- random features approximate the RBF kernel better as D grows.

4.8 Experiment and expected observations

Run `python labs/ch04-kernels/kernels.py` (seed 0, well under a second).

Primal equals dual. With 30 points, the quadratic feature map and $\lambda = 0.5$, primal predictions, dual predictions and $\Phi^\top \alpha$ agree to machine precision.

Validity. On 40 random points, the smallest eigenvalue of the RBF Gram matrix is 6×10^{-8} (positive, small because many points are similar), of the quadratic kernel -2.8×10^{-14} (rounding), and of the tanh “kernel” -21.9 (genuinely invalid).

Length-scale. Kernel ridge with an RBF kernel and $\lambda = 0.1$ on 40 noisy samples of $\sin 2x + 0.3x$:

ℓ	0.05	0.2	0.5	1	3	10
training MSE	0.011	0.026	0.037	0.046	0.404	0.544
MSE against the true curve	0.164	0.038	0.025	0.014	0.324	0.456

Training error falls monotonically as ℓ shrinks; error against the truth has its minimum at $\ell = 1$. The length-scale is a capacity knob, chosen by validation (chapter 5) or by marginal likelihood (chapter 7).

Kernel PCA. Two concentric noisy circles (radii 1 and 3, 100 points each): a linear classifier on the raw coordinates reaches 53% training accuracy. On the first two RBF kernel principal components ($\ell = 1$) it reaches 100%.

Random features. The largest entry-wise error of the random-feature approximation to the 40×40 RBF Gram matrix is 0.66 with $D = 10$, 0.18 with $D = 100$, 0.053 with $D = 1000$ and 0.032 with $D = 10^4$ — falling roughly like $1/\sqrt{D}$, as Monte Carlo averages do.

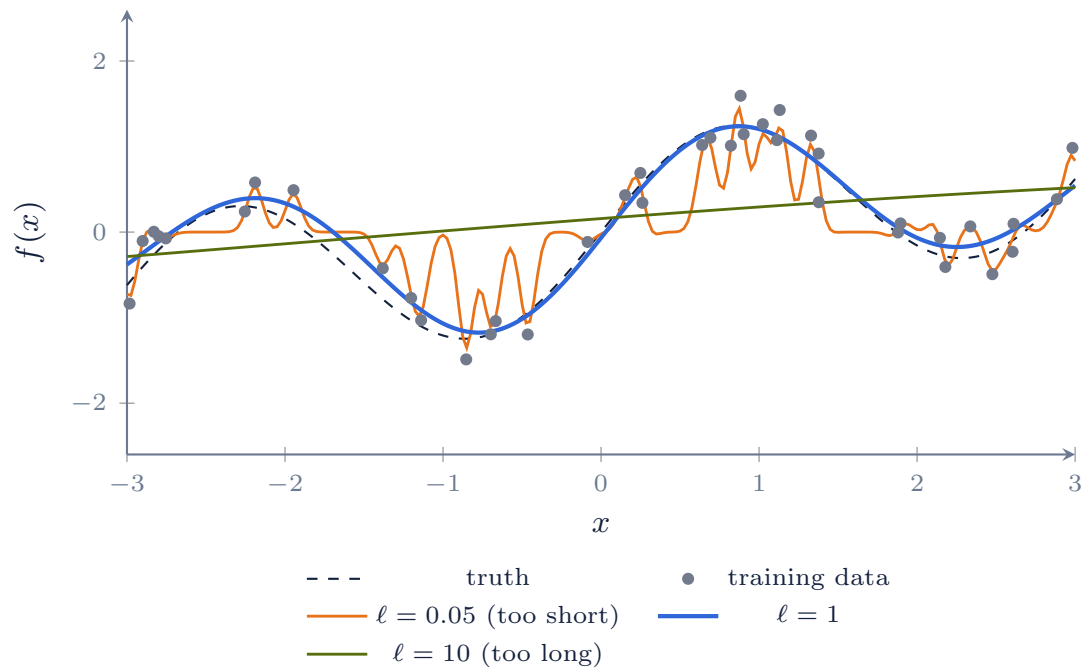


Figure 4.2: Kernel ridge fits at three length-scales. Too short: the fit chases each point and drops back to zero between them. Too long: everything looks similar and the fit is nearly a straight line.

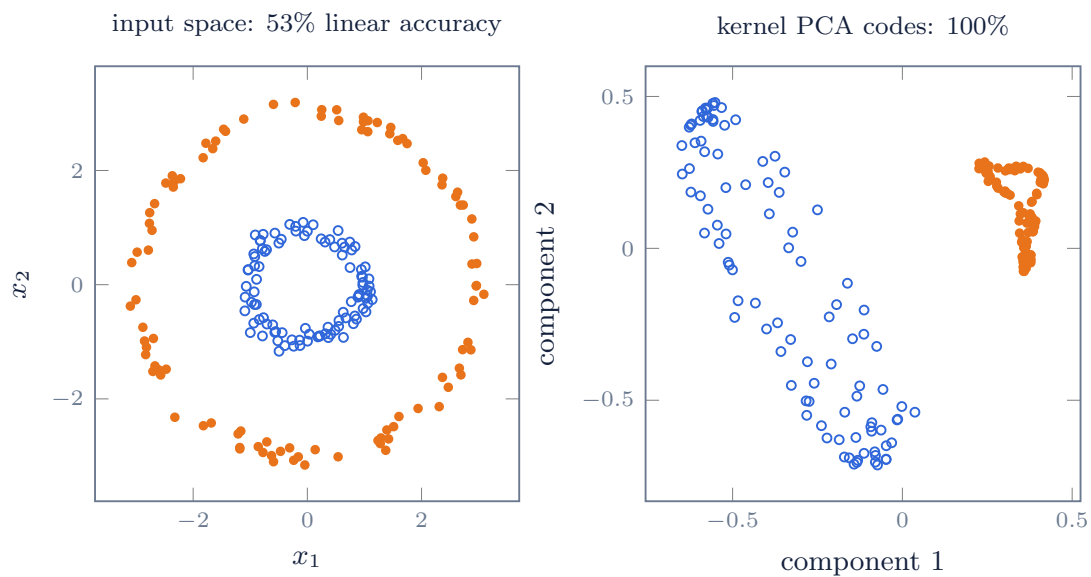


Figure 4.3: Concentric circles before and after RBF kernel PCA. In the kernel feature space the first component already separates the rings.

4.9 Moving pieces: diagnosing and fixing

Moving piece	Symptom when it is wrong	How to confirm	Fix
Kernel family	good training fit, poor validation; or systematic misfit (wrong smoothness, missing periodicity)	residual plots; compare kernel families on validation	match the kernel to the structure: RBF for smooth, periodic for repeating, sums/products for combinations
Length-scale ℓ	too short: spiky fit, memorisation; too long: almost linear fit	validation curve over a log grid; look at typical pairwise distances	set ℓ near the median pairwise distance, then tune; marginal likelihood (chapter 7)
Feature scaling	one input dominates the distance	per-feature standard deviations	standardise; or per-dimension length-scales (ARD)
λ	as in chapter 3	validation curve	tune jointly with ℓ
Kernel validity	Cholesky fails; negative “variances”	smallest eigenvalue vs. $u\ \mathbf{K}\ $	use a valid kernel; jitter only for rounding-level negatives
Centring	intercept-like offsets; kernel PCA component 1 = mean	compare centred and uncentred results	HKH with training means; test kernel centred with training means
Scale (m)	memory errors, hours of compute	$m^2 \times 8$ bytes	random features, Nyström, inducing points (chapter 7)
Input representation	kernel on raw pixels fails on shifted or relit images	nearest neighbours of a query: are they semantically similar?	kernel on good features (frozen deep features, chapter 18)
Tuning budget	“kernel beats linear” with a much bigger search	count configurations tried per method	equal tuning budgets for every baseline

First things to try: (1) standardise inputs; (2) set ℓ to the median pairwise distance; (3) sweep ℓ and λ on a log grid with a validation set; (4) check the smallest eigenvalue of $\mathbf{K}$; (5) look at nearest neighbours under the kernel.

4.10 Application, limitations and failure cases

Application (educational case study): small data, frozen features. With a few hundred labelled images, a kernel method on frozen features from a pretrained network (chapter 18) is a strong baseline: the network provides a good representation, the kernel provides a flexible but well-regularised decision function, and the convex fit is reproducible. Give the linear baseline the same tuning budget; an improvement may come from useful nonlinear boundaries or simply from more tuning.

Limitations and failures:

- **Cost.** Exact methods stop at roughly 10^4 – 10^5 examples.
- **Kernels on pixels.** Euclidean distance between raw images is not visual similarity.
- **The kernel is the inductive bias.** A poor kernel gives confident nonsense, especially far from the training data, where an RBF prediction decays to the mean (zero).
- **Interpretation.** Dual coefficients say which training examples matter, not which input features do.

4.11 Bridge to a research paper

Rahimi and Recht (2007), *Random features for large-scale kernel machines* (NeurIPS 2007).

- **Problem:** exact kernel machines scale as $O(m^2)$ – $O(m^3)$.
- **Inputs and outputs:** data and a shift-invariant kernel; explicit randomised features whose inner products approximate the kernel.
- **Key mechanism:** Bochner’s theorem (step 8): sample frequencies from the kernel’s spectral distribution.
- **Central claim:** linear models on D random features approach kernel-machine accuracy with cost linear in m .
- **Assumptions to question:** how large must D be for a given accuracy? does approximating the kernel well imply predicting well?
- **Affordable experiment:** on the lab’s curve and on a 10 000-point synthetic 2-D classification problem, compare exact kernel ridge and random-feature ridge for $D \in \{10, 100, 1000\}$: test error and wall-clock time against m .

4.12 Memory map

Summary. A linear model in a feature space is a nonlinear model in input space (figure 4.1). Ridge’s solution lies in the span of the training features, so it can be written with m dual coefficients and computed from inner products alone; the representer theorem makes this general, and Lagrangian duality explains the coefficients (residuals for ridge, support vectors for SVMs). A function is a kernel exactly when all its Gram matrices are PSD; closure rules build new kernels from old. The kernel and its length-scale are the model’s inductive bias. The costs, $O(m^2)$ memory and $O(m^3)$ time, are eased by random features, and the same ideas reappear in GPs and attention.

4.13 Exercises

1. *Conceptual.* Why is $k(\mathbf{x}, \mathbf{x}') = -\mathbf{x}^\top \mathbf{x}'$ invalid on any domain containing a nonzero point?
2. *Conceptual.* A kernel matrix is numerically symmetric but has eigenvalue -10^{-12} . Does this prove the kernel is invalid?
3. *Derivation.* Prove that the sum of two valid kernels is valid using the quadratic form, and give its feature map.
4. *Derivation.* Show that $\mathbf{H}\mathbf{K}\mathbf{H}$ is the Gram matrix of centred features.
5. *Derivation.* Derive the SVM dual from its Lagrangian, including the box constraint $0 \leq \alpha_i \leq C$.
6. *Derivation.* Show that for the RBF kernel, kernel ridge predictions tend to 0 far from all training points. What does this imply for extrapolation?

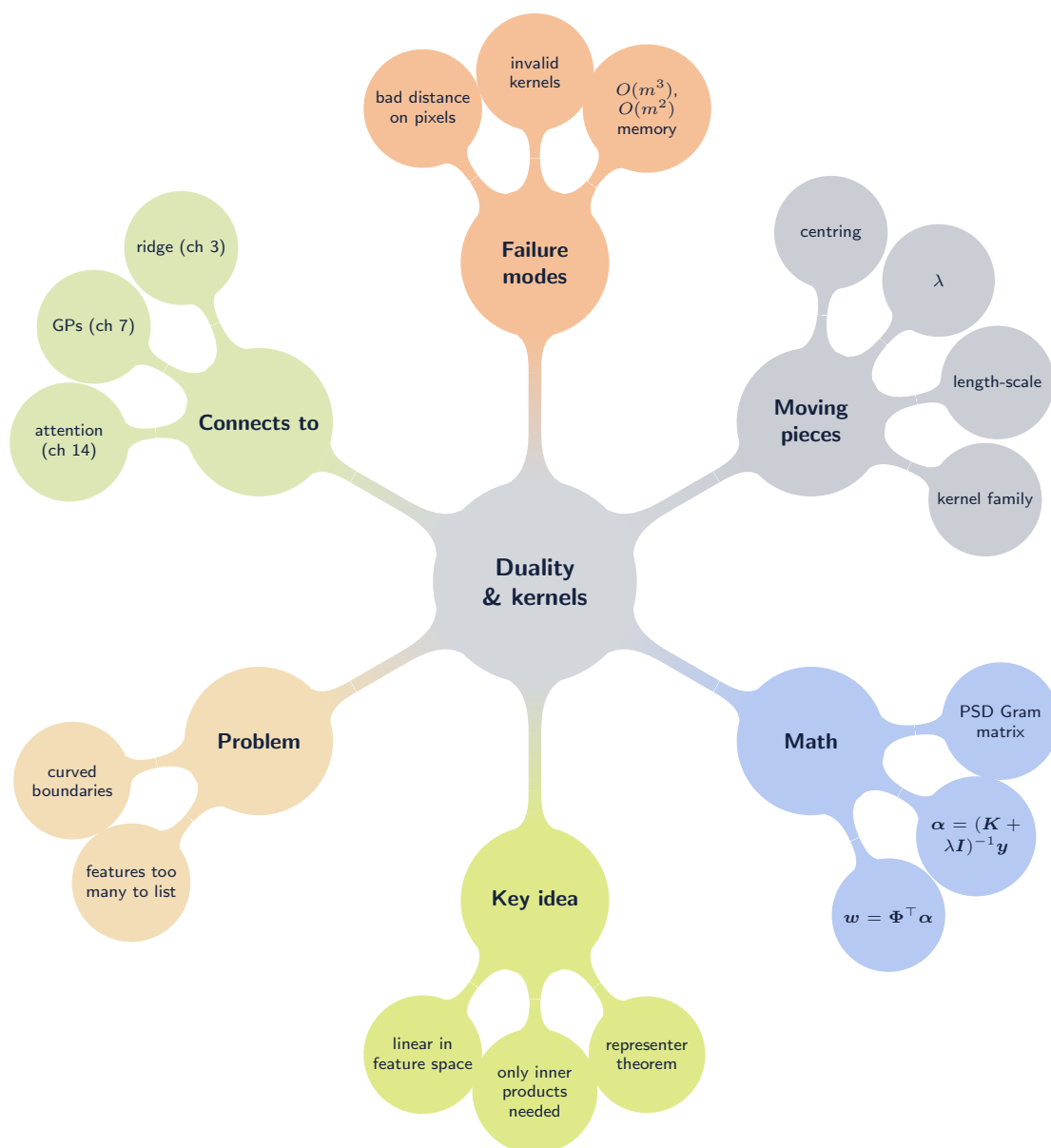


Figure 4.4: Memory map of chapter 4.

7. *Implementation.* Implement the Nyström approximation (sample q landmark points, use $\mathbf{K}_{mq}\mathbf{K}_{qq}^{-1}\mathbf{K}_{qm}$) and compare its error with random features at equal feature count.
8. *Implementation.* Fit a kernel SVM with an off-the-shelf solver on the circles data and count the support vectors as C varies.
9. *Interpretation.* On fixed image embeddings, a kernel classifier beats a linear one by 3 points. List two explanations other than “nonlinearity helps” and how to rule them out.
10. *Open question.* Attention (chapter 14) computes $\text{softmax}(\mathbf{q}^\top \mathbf{k}_j)$ weights over keys. In what sense is it a kernel smoother, and in what sense is it not?

4.13.1 Answer discussions

1. Its diagonal is $-\|\mathbf{x}\|^2 < 0$; take $\mathbf{c} = \mathbf{e}^{(i)}$.
2. No. Rounding of order $u\|\mathbf{K}\|$ can produce it. Compare the magnitude with the matrix norm; a substantially negative eigenvalue needs investigation, not silent clipping.
3. $\mathbf{c}^\top(\mathbf{K}_1 + \mathbf{K}_2)\mathbf{c} = \mathbf{c}^\top\mathbf{K}_1\mathbf{c} + \mathbf{c}^\top\mathbf{K}_2\mathbf{c} \geq 0$; the feature map is the concatenation $(\phi_1(\mathbf{x}), \phi_2(\mathbf{x}))$.
4. $\Phi_c = \mathbf{H}\Phi$, so $\Phi_c\Phi_c^\top = \mathbf{H}\mathbf{K}\mathbf{H}$ ($\mathbf{H}$ is symmetric).
5. Stationarity in ξ_i gives $C - \alpha_i - \mu_i = 0$; with $\mu_i \geq 0$ this is $\alpha_i \leq C$.
6. $\mathbf{k}_* \rightarrow \mathbf{0}$, so $f \rightarrow 0$: the model reverts to the prior mean. Extrapolation with an RBF kernel is not a continuation of the trend; centre the targets or add a mean function.
7. Nyström usually wins for a fixed feature count when the data lie near a low-dimensional set; random features are data-independent.
8. Small C : many support vectors (wide margin); large C : fewer, near the boundary.
9. A larger tuning budget; a better-regularised fit rather than nonlinearity. Equalise tuning budgets, and compare against a linear model on random features of the same dimension.
10. Attention is a normalised kernel-weighted average of values with kernel $\exp(\mathbf{q}^\top \mathbf{k}/\sqrt{d})$, but the queries and keys are *learned* functions of the input, so the similarity itself is learned, and it is not symmetric.

4.14 Self-check and common mistakes

- ☐ I can derive $\mathbf{w} = \Phi^\top(\mathbf{K} + \lambda\mathbf{I})^{-1}\mathbf{y}$ two ways.
- ☐ I can prove the representer theorem in four lines.
- ☐ I can test a candidate kernel for validity and build new kernels from old.
- ☐ I can say when primal is cheaper than dual.
- ☐ I can explain support vectors from complementary slackness.

Common mistakes: assuming “kernel” means “faster”; centring test kernels with test means; treating tiny negative eigenvalues as invalidity (or large ones as rounding); using an RBF kernel on unscaled features; tuning the kernel model far more than the baseline; expecting RBF models to extrapolate trends.

References

- J. Mercer. Functions of positive and negative type, and their connection with the theory of integral equations. *Phil. Trans. R. Soc. A*, 209:415–446, 1909.
- N. Aronszajn. Theory of reproducing kernels. *Trans. AMS*, 68(3):337–404, 1950.

- M. A. Aizerman, E. M. Braverman and L. I. Rozonoer. Theoretical foundations of the potential function method in pattern recognition learning. *Automation and Remote Control*, 25:821–837, 1964.
- G. Kimeldorf and G. Wahba. Some results on Tchebycheffian spline functions. *J. Math. Anal. Appl.*, 33(1):82–95, 1971.
- B. E. Boser, I. M. Guyon and V. N. Vapnik. A training algorithm for optimal margin classifiers. *COLT*, 1992.
- C. Cortes and V. Vapnik. Support-vector networks. *Machine Learning*, 20:273–297, 1995.
- B. Schölkopf, A. Smola and K.-R. Müller. Nonlinear component analysis as a kernel eigenvalue problem. *Neural Computation*, 10(5):1299–1319, 1998.
- A. Rahimi and B. Recht. Random features for large-scale kernel machines. *NeurIPS*, 2007.
- A. Jacot, F. Gabriel and C. Hongler. Neural tangent kernel: convergence and generalization in neural networks. *NeurIPS*, 2018.
- B. Schölkopf and A. J. Smola. *Learning with Kernels*. MIT Press, 2002.
- S. Boyd and L. Vandenberghe. *Convex Optimization*. Cambridge University Press, 2004.

Chapter 5

Risk, generalization and experimental design

Prerequisites: chapter 3 (regularisation, logistic regression, thresholds).

Learning outcomes: after this chapter you can

1. distinguish population, empirical, validation and test risk, and say what each one estimates;
2. derive the Bayes-optimal predictor for squared and zero-one loss, and decompose excess risk into approximation, estimation and optimisation error;
3. derive a finite-class generalisation bound from Hoeffding’s inequality and the union bound, and use the same reasoning to explain why choosing on the test set inflates results;
4. design a split that answers a stated question (new images, new sites, the future), and audit a pipeline for leakage and shortcuts;
5. report uncertainty at the right unit, and evaluate probabilities with proper scores and calibration.

5.1 A concrete problem

A team trains a plant-disease classifier on 1 200 leaf photographs from 20 farms. They shuffle the photographs, hold out a quarter, and report **94% test accuracy**. Deployed on a new farm, it gets about 30% right — worse than guessing.

Nothing was wrong with the arithmetic. The test set answered a different question from the one the team cared about. Leaves on each farm were photographed against that farm’s background, and on these farms the background happened to correlate with disease. The model learned the background. A random split puts photographs from the same farms in training and test, so the shortcut works on both. The question that mattered — *how well does this work on a farm we have never seen, where backgrounds mean something else?* — needed a different test.

This chapter is about choosing the right question and estimating the answer honestly: what “risk” is, why training error underestimates it, how far a test estimate can be trusted, how to build splits that match the question, and how to report uncertainty and probabilities. It is the most important chapter for doing research that holds up.

5.2 How we got here

Designing experiments (1920s–1930s). Ronald Fisher’s work at the Rothamsted agricultural station — *Statistical Methods for Research Workers* (1925) and *The Design of Experiments* (1935) — introduced the principles every ML evaluation still relies on: *randomise* to break hidden correlations, *replicate* to measure variability, and *block* by known sources of variation such as field plots. A machine-learning split is an experimental design, and the farm in section 5.1 is exactly Fisher’s “block”.

Estimating error honestly (1960s–1979). Wassily Hoeffding (1963) bounded how far an average of bounded random variables can stray from its mean — the inequality behind every test-set error bar. Mervyn Stone (1974) and Seymour Geisser (1975) formalised **cross-validation** for choosing models without fooling yourself. Bradley Efron (1979) introduced the **bootstrap**: resample the data to see how much a statistic would vary.

Learning theory (1971–1984). Vladimir Vapnik and Alexey Chervonenkis (1971) asked when the *empirical* error of a whole class of models converges uniformly to the true error, and answered with the **VC dimension**. Leslie Valiant (1984) framed learning as “probably approximately correct” (PAC). These results explain why training error is optimistic and how much data a model class needs.

Probabilities, not just labels (1950–2017). Glenn Brier (1950), verifying weather forecasts, proposed the squared-error score for probabilities; Morris DeGroot and Stephen Fienberg (1983) formalised **calibration**. Chuan Guo and colleagues (2017) showed that modern deep networks are badly over-confident and that a single learned **temperature** fixes much of it.

When the test set lies (2011–2023). As benchmarks grew, failures of the “shuffle and split” habit became famous. Antonio Torralba and Alexei Efros (2011) showed that classifiers can tell which *dataset* an image came from. Sara Beery, Grant Van Horn and Pietro Perona (2018) found that camera-trap classifiers collapse at new locations — this chapter’s paper bridge. Benjamin Recht and colleagues (2019) rebuilt an ImageNet test set and found accuracy drops of 11–14 points, partly from years of community tuning against one test set. Robert Geirhos and colleagues (2020) named the general phenomenon **shortcut learning**, WILDS (Koh et al., 2021) collected real distribution-shift benchmarks, and Sayash Kapoor and Arvind Narayanan (2023) documented **data leakage** across hundreds of published ML-based studies.

What this chapter cannot fix. Good evaluation tells you *that* a model fails under a shift and roughly *why*. Making models that transfer is the subject of chapters 15–18; this chapter gives the measuring instruments.

5.3 Assumptions and notation

Symbol	Meaning
$p_{\text{data}}(\mathbf{x}, y)$	the data-generating distribution — the population we care about
$\ell(f(\mathbf{x}), y)$	loss of predictor f on one example
$R(f) = \mathbb{E}_{(\mathbf{x}, y) \sim p_{\text{data}}}[\ell(f(\mathbf{x}), y)]$	population risk
$\hat{R}(f) = \frac{1}{m} \sum_i \ell(f(\mathbf{x}^{(i)}), y^{(i)})$	empirical risk on m examples
$\mathcal{F}$	the model class (all predictors the method can return)

Symbol	Meaning
f^*, R^*	the Bayes predictor (best of <i>all</i> functions) and its risk
$f_{\mathcal{F}}^*$	the best predictor in $\mathcal{F}$
$\hat{f}$	the empirical risk minimiser in $\mathcal{F}$
$\tilde{f}$	what the optimiser actually returned
$\mathbb{X}_{\text{train}}, \mathbb{X}_{\text{val}}, \mathbb{X}_{\text{test}}$	training, validation and test sets

Assumptions to state in every study:

1. **Which population.** The test set must be a sample from the population the claim is about. “New images from known farms” and “images from new farms” are different populations.
2. **Which unit is independent.** Images, patients, farms, video clips. Error bars and splits must respect the unit.
3. **Which choices touched which data.** Training fits parameters, validation chooses hyperparameters and models, the test set is used once, at the end.

5.4 Mathematical formulation

Generalisation is the gap between what we can measure and what we want: $R(\tilde{f}) - \hat{R}(\tilde{f})$. Training error estimates it badly because $\tilde{f}$ was chosen *to make* $\hat{R}$ small.

A clean evaluation protocol:

1. fit parameters on $\mathbb{X}_{\text{train}}$ (minimising $\hat{R}_{\text{train}}$ plus any penalty);
2. choose hyperparameters, architectures and thresholds by $\hat{R}_{\text{val}}$;
3. evaluate the single, final, declared procedure once on $\mathbb{X}_{\text{test}}$.

Because $\mathbb{X}_{\text{test}}$ played no part in choosing $\tilde{f}$, the test error is an unbiased estimate of $R(\tilde{f})$ — **for the population the test set was drawn from.**

Probabilistic predictions $p = \hat{P}(y = 1 \mid \mathbf{x})$ are evaluated by **proper scoring rules** — the negative log-likelihood (NLL) $-y \log p - (1 - y) \log(1 - p)$ and the **Brier score** $(p - y)^2$ — and by **calibration**: among inputs given probability p , a fraction p should be positive.

5.5 Derivation, with reasons for each step

5.5.1 Step 1: the best possible predictor

Squared loss. Condition on $\mathbf{x} = \mathbf{x}$ and let $\mu(\mathbf{x}) = \mathbb{E}[y \mid \mathbf{x}]$. For any prediction a ,

$$\mathbb{E}[(a - y)^2 \mid \mathbf{x}] = (a - \mu(\mathbf{x}))^2 + \text{Var}(y \mid \mathbf{x}),$$

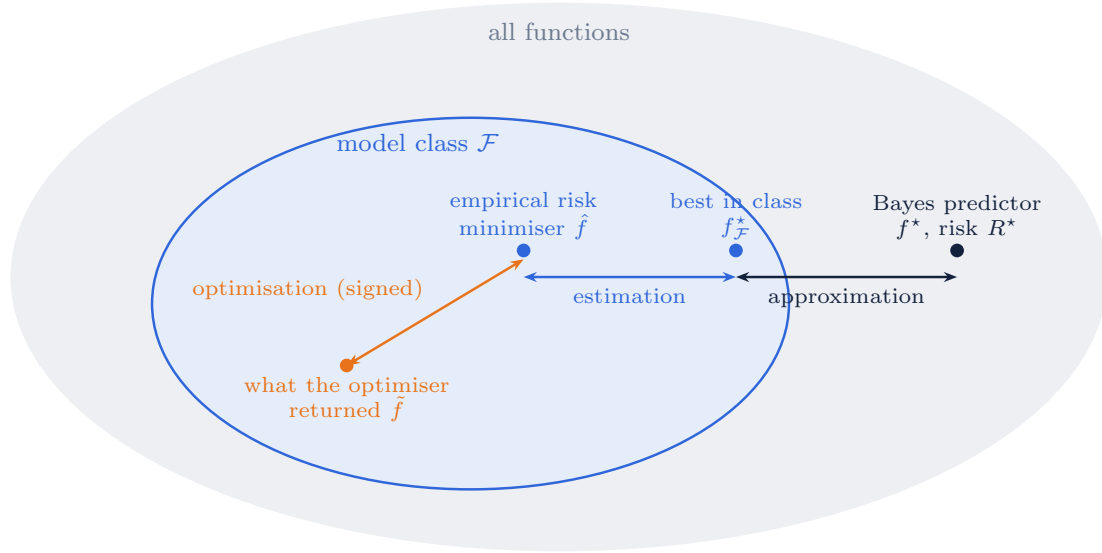
because the cross term $2(a - \mu) \mathbb{E}[\mu - y \mid \mathbf{x}]$ vanishes. The best prediction is the **conditional mean**, and even it pays the irreducible noise $\text{Var}(y \mid \mathbf{x})$.

Zero-one loss. Predicting class k is wrong with probability $1 - P(y = k \mid \mathbf{x})$, so the best prediction is the class with the **largest posterior probability**. With costs it is the threshold rule of section 3.5. These are the **Bayes predictors** f^* ; their risk R^* is the floor no method can beat. A restricted class $\mathcal{F}$ (say, linear classifiers) may not contain them.

5.5.2 Step 2: three kinds of error

Write the excess risk of the predictor we actually got as a telescoping sum:

$$R(\tilde{f}) - R^* = \underbrace{R(\tilde{f}) - R(\hat{f})}_{\text{optimisation}} + \underbrace{R(\hat{f}) - R(f_{\mathcal{F}}^*)}_{\text{estimation}} + \underbrace{R(f_{\mathcal{F}}^*) - R^*}_{\text{approximation}}.$$



$R(\tilde{f}) - R^* = \underbrace{R(\tilde{f}) - R(\hat{f})}_{\text{optimisation}} + \underbrace{R(\hat{f}) - R(f_{\mathcal{F}}^*)}_{\text{estimation}} + \underbrace{R(f_{\mathcal{F}}^*) - R^*}_{\text{approximation}}.$ The approximation and estimation terms are ≥ 0 ; the optimisation term can be negative, because an approximate minimiser can have lower population risk than $\hat{f}$. The empirical optimisation gap $\hat{R}(\tilde{f}) - \hat{R}(\hat{f})$ is always ≥ 0 .

Figure 5.1: Three terms between the predictor we get and the best possible one. The approximation and estimation terms are nonnegative; the optimisation term is a signed difference of population risks.

Each term points at a different moving piece (field guide, section F.1):

- **Approximation** error belongs to the model class: a bigger or better-suited class (chapters 4, 12, 14) shrinks it.
- **Estimation** error belongs to the data: with finitely many examples, the empirical minimiser is not the class’s best. More data or a smaller effective class (regularisation, chapter 3) shrinks it.
- **Optimisation** error belongs to the optimiser: non-convex training (chapter 11) may not find $\hat{f}$. Unlike the other two terms, $R(\tilde{f}) - R(\hat{f})$ is a difference of *population* risks and can be negative: an approximate empirical minimiser (for example, one stopped early) can generalise better than the exact one. What the optimiser is responsible for, and what is always nonnegative, is the *empirical* optimisation gap $\hat{R}(\tilde{f}) - \hat{R}(\hat{f}) \geq 0$, since $\hat{f}$ minimises $\hat{R}$ over $\mathcal{F}$. The identity above stays true; only the reading of each term as a “cost” needs this care.

The classical trade-off is between the first two. (Very large networks complicate the picture: they can fit noise and still generalise — “benign overfitting” and “double descent” — which is an active research area, not a contradiction of the decomposition.)

5.5.3 Step 3: bias and variance across datasets

Chapter 3 decomposed the error of ridge on fixed inputs. In general, let $\hat{f}_{\mathbb{D}}$ be the predictor fitted on a random training set $\mathbb{D}$. At a fixed input $\mathbf{x}$, for a fresh target,

$$\mathbb{E}_{\mathbb{D}}\mathbb{E}[(\hat{f}_{\mathbb{D}}(\mathbf{x}) - y)^2 \mid \mathbf{x}] = \underbrace{(\mathbb{E}_{\mathbb{D}}\hat{f}_{\mathbb{D}}(\mathbf{x}) - \mu(\mathbf{x}))^2}_{\text{bias}^2} + \underbrace{\text{Var}_{\mathbb{D}}(\hat{f}_{\mathbb{D}}(\mathbf{x}))}_{\text{variance}} + \underbrace{\text{Var}(y \mid \mathbf{x})}_{\text{noise}}.$$

Derivation: add and subtract $\mathbb{E}_{\mathbb{D}}\hat{f}_{\mathbb{D}}(\mathbf{x})$ and $\mu(\mathbf{x})$; the cross terms vanish because the fresh target is independent of $\mathbb{D}$. This decomposition is about the *distribution of fitted predictors* over datasets. You cannot read it off one model’s training and test curves.

5.5.4 Step 4: why a test set can be trusted — and when it cannot

One fixed predictor. If f was chosen *before* looking at the test set, its test losses $\ell^{(1)}, \dots, \ell^{(m)}$ are independent draws with mean $R(f)$. For losses in $[0, 1]$, **Hoeffding’s inequality** gives

$$P(|\hat{R}(f) - R(f)| > \epsilon) \leq 2 \exp(-2m\epsilon^2).$$

(The proof bounds the moment generating function of a bounded variable; see Hoeffding (1963) or Shalev-Shwartz and Ben-David (2014), appendix B.) With $m = 1000$ test images, the probability that test accuracy is off by more than 5 points is at most $2e^{-5} \approx 0.013$.

Many predictors — the union bound. If we evaluate M predictors and want *all* their estimates to be accurate at once, the probability that *at least one* is off is at most the sum of the individual probabilities:

$$P\left(\max_{f \in \mathcal{F}} |\hat{R}(f) - R(f)| > \epsilon\right) \leq 2M \exp(-2m\epsilon^2).$$

Setting the right side to δ and solving, $\epsilon = \sqrt{\log(2M/\delta)/(2m)}$. On this event, the empirical minimiser satisfies $R(\hat{f}) \leq \hat{R}(\hat{f}) + \epsilon \leq \hat{R}(f_{\mathcal{F}}^*) + \epsilon \leq R(f_{\mathcal{F}}^*) + 2\epsilon$: one ϵ to move from the population to the sample for $\hat{f}$, one to move back for $f_{\mathcal{F}}^*$. The bound grows only with $\log M$ — which is why learning from finite data is possible at all.

The same argument, read backwards, is the winner’s curse. If you evaluate M models on the *test* set and report the best, the reported number is the maximum of M noisy estimates, biased upwards by roughly the ϵ above. Section 5.8 picks the best of 1000 coin-flip classifiers on a 500-image test set and reports 57% accuracy for models that are 50% accurate. **Every decision made by looking at test results turns the test set into a validation set.**

5.5.5 Step 5: infinite classes and the VC dimension

Linear classifiers form an infinite class, so the union bound over M predictors does not apply directly. What matters is how many *different labellings* the class can produce on m points. A class **shatters** a set of points if it realises all 2^m labellings; its **VC dimension** is the size of the largest set it can shatter.

Example: intervals on the line. Any labelling of two points is realised by some interval (including “neither”, by an interval containing no point). For three ordered points, the

labelling $(1, 0, 1)$ is impossible — an interval containing both ends contains the middle. So the VC dimension is 2. Linear classifiers in $\mathbb{R}^n$ have VC dimension $n + 1$.

The VC bound replaces $\log M$ by roughly $d_{VC} \log(m/d_{VC})$. For deep networks these bounds are far too loose to predict actual test error — another reason the empirical protocol of section 5.4 matters more in practice. Shalev-Shwartz and Ben-David (2014), chapters 2–6, give the full statements and proofs.

5.5.6 Step 6: a split is a question

The protocol in section 5.4 guarantees honesty only *for the population the test set represents*. Choosing the split is choosing that population (figure 5.2):

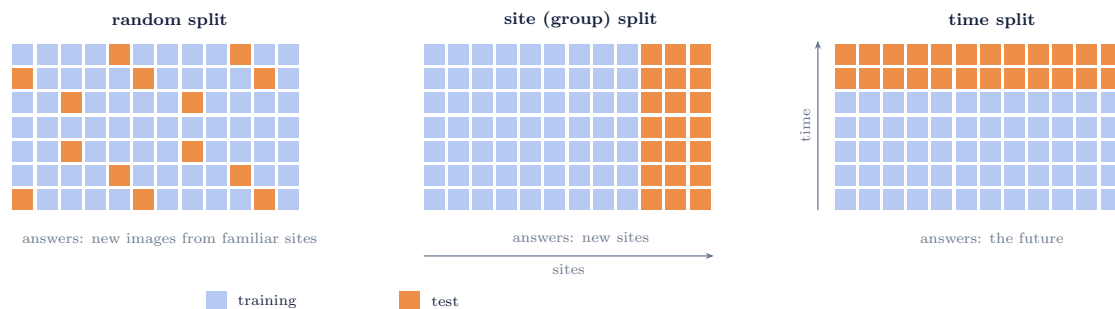


Figure 5.2: Three ways to split the same images, answering three different questions.

Question	Split	Unit held out
new images from the same sites and conditions	random	images
new patients / farms / cameras / videos	group	the whole group
the future	time	everything after a cut-off date
a new region, sensor or hospital system	domain	the whole domain

Leakage is any path by which test information reaches training or model selection. The common forms:

- **Group leakage:** frames of one video, scans of one patient, tiles of one field, or near-duplicate images on both sides of a random split.
- **Temporal leakage:** training on data from after the test period.
- **Preprocessing leakage:** normalisation statistics, PCA (chapter 2), vocabularies or feature selection fitted on all data before splitting.
- **Augmentation before splitting:** an image and its augmented copy on opposite sides.
- **Target leakage:** a feature that is a consequence of the label (a “treatment started” flag predicting disease).
- **Selection leakage:** hyperparameters, thresholds or early stopping chosen on the test set (step 4).

Shortcuts are a related, subtler failure: features that predict the label in the training population but not in the population of interest (backgrounds, scanner marks, hospital-specific text in an image). A split can only expose a shortcut that it *breaks*: a site split exposes site-specific shortcuts, but a shortcut shared by all training sites survives it and appears only under a new population (section 5.8).

5.5.7 Step 7: uncertainty at the right unit

For accuracy $\hat{a}$ on m **independent** test examples, the standard error is about $\sqrt{\hat{a}(1-\hat{a})/m}$. With $\hat{a} = 0.9$ and $m = 400$, that is 0.015: a 95% interval of roughly ± 3 points.

Dependence shrinks the effective sample size. If test images come in clusters of k (frames of a clip, scans of a patient) with intra-cluster correlation ρ , the variance of the mean is inflated by the **design effect** $1 + (k-1)\rho$. With $k = 25$ frames per clip and $\rho = 0.08$, the design effect is about 2.9: 1 000 frames carry the information of about 350 independent ones. **Bootstrap the independent unit** — resample clips, not frames.

Comparing two models, evaluate them on the *same* examples and seeds and analyse the *paired differences*; this cancels the shared difficulty of each example and is far more sensitive than comparing two independent averages. **Seeds** are a source of variance too: report the spread over several training runs, not the best run.

5.5.8 Step 8: scoring probabilities

A scoring rule is **proper** if, when the true probability is q , the expected score is minimised by reporting $p = q$. For the Brier score,

$$\mathbb{E}[(p - y)^2] = q(1 - p)^2 + (1 - q)p^2, \quad \frac{d}{dp} = -2q(1 - p) + 2(1 - q)p = 0 \iff p = q.$$

The NLL is proper by the same calculation (chapter 3, step 7 with $a = b$). Accuracy is not a proper score for probabilities: it only sees which side of 0.5 they fall on.

Calibration asks whether predicted probabilities match frequencies. A **reliability diagram** bins predictions by p and plots the observed positive rate in each bin; the **expected calibration error** (ECE) is the bin-size-weighted average gap $\sum_b \frac{|B_b|}{m} |\bar{p}_b - \bar{y}_b|$. ECE depends on the binning and can be gamed (a model that predicts the base rate everywhere is perfectly calibrated and useless), so report it *with* a proper score and a discrimination metric.

Temperature scaling fixes over-confidence with one parameter: replace logits a by a/T and choose T by minimising NLL on the **validation** set. Dividing every logit by the same positive number keeps their order, so **accuracy and ranking do not change**; only the confidence does.

5.5.9 Step 9: metrics when classes are imbalanced

With 5% diseased leaves, “always healthy” scores 95% accuracy. Report the **confusion matrix** and metrics tied to the decision:

- **recall** (sensitivity) = $TP/(TP + FN)$: of the diseased, how many were found;
- **precision** = $TP/(TP + FP)$: of the flagged, how many were diseased;
- **balanced accuracy**: the mean of per-class recalls;
- **ROC AUC**: the probability that a random positive scores above a random negative (threshold-free, insensitive to prevalence); the **precision–recall curve** is more informative when positives are rare;
- **macro** averages weight classes equally, **micro** averages weight examples equally.

5.6 Numerical worked example

A finite-class bound. $M = 100$ candidate models, $m = 1000$ examples, $\delta = 0.05$: $\epsilon = \sqrt{\log(4000)/2000} = \sqrt{8.29/2000} = 0.0644$. With probability at least 95%, every candidate's empirical risk is within 6.4 points of its true risk, and the empirical minimiser is within 12.9 points of the best candidate. It is a guarantee under the stated assumptions, not a prediction of the actual gap.

A standard error. 90% accuracy on 400 independent images: $\sqrt{0.9 \cdot 0.1/400} = 0.015$. Two models at 90% and 91.5% on *different* 400-image test sets are not distinguishable.

A design effect. 40 clips $\times$ 25 frames = 1 000 frames, accuracy 72%, intra-clip correlation 0.08: design effect $1 + 24 \times 0.08 = 2.9$, effective sample size about 350, standard error $\sqrt{0.72 \cdot 0.28/350} \approx 0.024$ instead of 0.014.

A confusion matrix. 1 000 leaves, 50 diseased. A model flags 60 leaves, 40 of them diseased. Accuracy = $(40 + 930)/1000 = 0.97$; recall = $40/50 = 0.80$; precision = $40/60 = 0.67$; balanced accuracy = $(0.80 + 930/950)/2 = 0.89$. “Always healthy” has accuracy 0.95, recall 0 and balanced accuracy 0.50.

A Brier score. Predicting 0.8 for three positives and one negative: $\frac{1}{4}[3(0.2)^2 + (0.8)^2] = \frac{1}{4}[0.12 + 0.64] = 0.19$.

5.7 Small implementation and checks

The lab is `labs/ch05-evaluation/evaluation.py`. Every dataset in it is simulated, so the right answer is known. Two functions carry the chapter's main lessons:

```
def bootstrap_ci(C, unit="frame"):          # C is (clips, frames) of 0/1
    ↪ correctness
    if unit == "frame": resample individual frames
    else:                resample whole clips (rows)          # the
    ↪ independent unit

def fit_temperature(logits, y):             # on VALIDATION data only
    return argmin over T of NLL(sigmoid(logits / T), y)
```

The tests in `test_evaluation.py` check that:

- a global shortcut beats the symptom model on random *and* site-held-out splits but fails on a population where the shortcut flips;
- a site-identity shortcut's gain disappears on held-out sites;
- Hoeffding's bound holds in simulation, and the finite-class ϵ is 0.0644;
- choosing the best of 100 useless models inflates reported accuracy;
- proper scores behave correctly on perfect and uninformative predictions;
- temperature scaling lowers NLL and ECE without changing accuracy;
- the frame-level bootstrap interval is narrower than the clip-level one.

5.8 Experiment and expected observations

Run `python labs/ch05-evaluation/evaluation.py` (seed 0, about two seconds).

1. Shortcuts and splits. Twenty simulated farms, 60 leaves each. A noisy but stable *symptom* feature, and a *background* feature that agrees with the label 90% of the time on

every training farm.

Model	random split	held-out farms	new population (background flips)
symptom + background	0.943	0.917	0.291
symptom only	0.787	0.783	0.777

The background shortcut wins by 15 points on the random split *and still wins on held-out farms*, because every farm shares it. Only a population where the background means something else exposes it: accuracy falls to 29%, far below chance. The symptom-only model is worse in-distribution and the only one that survives.

A second simulation has a *site-specific* shortcut: farms differ in disease prevalence (15% or 85%) and the model may use a one-hot farm identity. On a random split, adding farm identity raises accuracy from 0.773 to 0.863; on held-out farms both models score 0.743. The apparent gain was the model memorising “farm 7 is mostly diseased”. **A group split exposes group-specific shortcuts; only a new population exposes shared ones.**

2. Concentration and the winner’s curse. For a fixed classifier with true error 0.3 and $m = 200$ test examples, the simulated probability of an estimation error above 5 points is 0.105; Hoeffding’s bound says at most 0.736 — valid, and loose, as worst-case bounds are. Now choose the best of K coin-flip classifiers on 500 test images:

K	1	10	100	1000
reported accuracy of the “best”	0.501	0.534	0.556	0.572
true accuracy	0.500	0.500	0.500	0.500

A seven-point “improvement” from nothing but selection on the test set.

3. Calibration. A simulated classifier reports $\sigma(3a)$ where the true probability is $\sigma(a)$ — three times too confident. Temperature scaling fitted on 2 000 validation predictions finds $T = 2.87$ (the truth is 3). On 2 000 test predictions:

	accuracy	NLL	Brier	ECE
raw	0.732	0.756	0.208	0.161
after temperature scaling	0.732	0.531	0.178	0.018

Accuracy is identical — temperature does not change any decision at threshold 0.5 — while NLL falls by 30% and ECE by a factor of nine.

4. The unit of resampling. Forty simulated clips of 25 frames each, with clip-level accuracy varying around 75%. Measured accuracy 0.722. The 95% bootstrap interval is (0.694, 0.750) when resampling frames and (0.679, 0.766) when resampling clips — about 1.55 times wider, close to the $\sqrt{2.9} \approx 1.7$ predicted by the design effect. Over 200 simulated studies, the frame-level interval contains the true mean only **73%** of the time; the clip-level interval **94%**, close to the promised 95%.

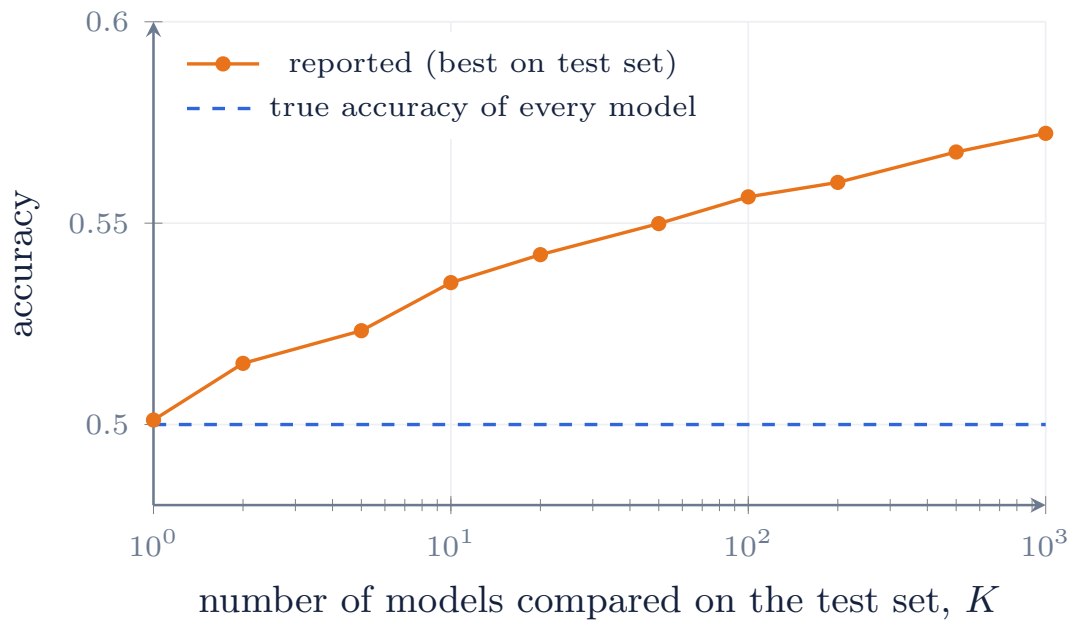


Figure 5.3: The winner's curse: the reported accuracy of the best of K useless models grows like the union bound's ϵ , roughly $\sqrt{\log K}$.

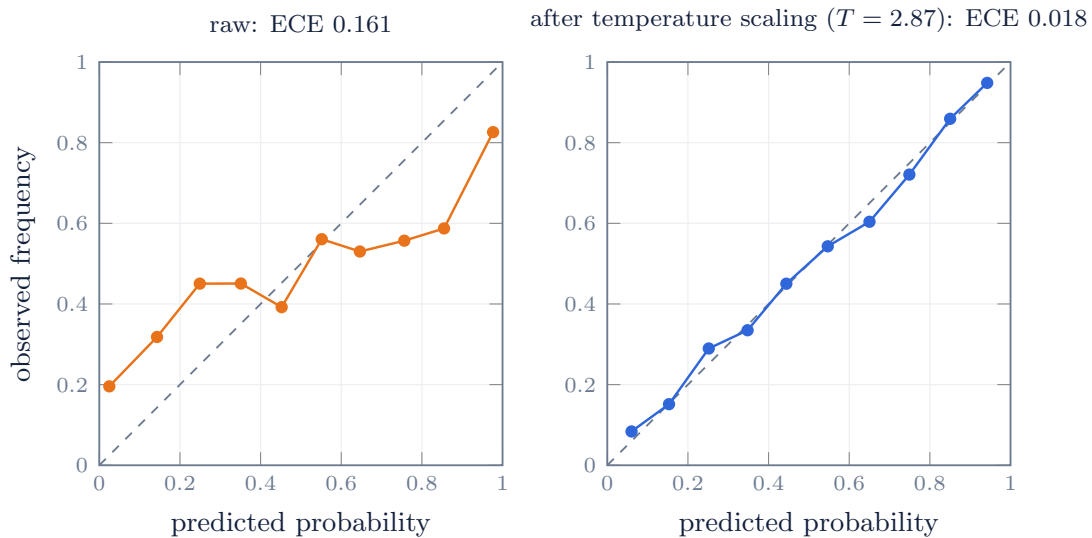


Figure 5.4: Reliability diagrams. Raw predictions are too extreme (below the diagonal at high confidence, above it at low confidence); after temperature scaling they follow the diagonal.

5.9 Moving pieces: diagnosing and fixing

Moving piece	Symptom when it is wrong	How to confirm	Fix
Split unit	great test score, poor deployment	does any group (patient, farm, video, camera) appear on both sides?	split by the independent unit, before any augmentation or preprocessing
Split question	the claim says “new sites” but the split is random	write the question next to the split	choose random / group / time / domain split to match the claim
Preprocessing	small but suspicious test gains	which statistics were fitted on which data?	fit every learned transform on training data only; pipelines
Duplicates	near-perfect test accuracy on some classes	perceptual hashing; nearest-neighbour search across splits	deduplicate before splitting
Shortcuts	errors concentrated in one site or condition; saliency on backgrounds	per-group metrics; counterfactual edits (swap backgrounds)	group splits; data balancing; new-population tests
Test-set reuse	results that do not replicate	count how many decisions used the test set	validation for all choices; a fresh test set for the final claim
Metric	high accuracy, useless model	confusion matrix; per-class recall	metrics tied to the decision; balanced accuracy; PR curves
Calibration	confident wrong predictions	reliability diagram; NLL vs. accuracy	temperature scaling on validation; report NLL/Brier
Uncertainty	non-replicating “significant” gains	bootstrap at the right unit; seed spread	clustered bootstrap; paired comparisons; more seeds
Tuning budget	new method beats an untuned baseline	count configurations per method	equal budgets; report the search

First things to try before trusting any number: (1) write the population and the unit of analysis; (2) check every learned transform for where it was fitted; (3) look for groups and duplicates across splits; (4) compute per-group metrics; (5) bootstrap at the independent unit.

5.10 Application, limitations and failure cases

Application (educational case studies).

- **Remote sensing.** Neighbouring image tiles are strongly correlated. A random tile

split measures interpolation within known regions. Use spatial blocks (hold out whole regions) for “new regions” and a time split for “next season”. Expect the scores to drop, and report all of them.

- **Medical imaging.** Several images per patient, several patients per hospital. Split by patient at minimum, by hospital for claims about new hospitals. Report calibration, not just AUC, because decisions use the probabilities.
- **Video.** Split by clip or by recording session, never by frame.

Limitations:

- No split can test a population you did not sample. A model evaluated on three hospitals says little about a fourth with a different scanner.
- Learning-theory bounds explain *why* honest evaluation is possible; they rarely give useful numbers for deep networks.
- Calibration is population-specific: a model calibrated on one site can be badly mis-calibrated on another.

5.11 Bridge to a research paper

Beery, Van Horn and Perona (2018), *Recognition in Terra Incognita* (ECCV).

- **Problem:** do species classifiers for camera-trap images work at new locations?
- **Inputs and outputs:** camera-trap images; species labels.
- **Evaluation design:** separate test sets for *seen* camera locations and *unseen* locations — exactly the split distinction of step 6.
- **Central claim:** accuracy drops substantially at unseen locations, and models rely on background and location-specific context.
- **Assumptions to question:** how much of the drop is due to different species frequencies at new locations (label shift) rather than appearance? Would per-location calibration help?
- **Affordable experiment:** with the lab’s simulator, vary how many farms share the shortcut (from all to half) and plot the gap between random-split and held-out-farm accuracy. Then, if you have access to a public camera-trap subset, compare a frozen-feature linear probe under random and location splits.

5.12 Memory map

Summary. We want low risk on a population; we can measure only empirical risk on samples. Training error is optimistic because the model was chosen to make it small; a test set used once is honest, but only about the population it was drawn from, and selection on it inflates results by roughly the union bound’s ϵ . Excess risk splits into approximation, estimation and optimisation error, each fixed by a different moving piece. The split is the question: random, group, time and domain splits measure different things, and only a split that breaks a shortcut can expose it. Report uncertainty at the independent unit, compare models in pairs, and judge probabilities with proper scores and calibration.

5.13 Exercises

1. *Conceptual.* What two operations create the finite-class bound of step 4?
2. *Conceptual.* An interval realises the labelling $(1, 1, 0)$ on three points. Are the points shattered?



Figure 5.5: Memory map of chapter 5.

3. *Derivation.* Derive the bias–variance–noise decomposition of step 3, showing why each cross term vanishes.
4. *Derivation.* Show that the NLL is a proper scoring rule.
5. *Derivation.* Show that dividing logits by $T > 0$ never changes the predicted class, and compute the derivative of the validation NLL with respect to $\log T$.
6. *Implementation.* Add a nested cross-validation routine to the lab: an inner loop chooses λ for ridge, an outer loop estimates the error of the whole procedure. Compare it with the (optimistic) error of the best inner configuration.
7. *Implementation.* Implement a deduplication audit: flag pairs of examples across train and test whose feature vectors are within a small distance. Plant 20 duplicates and check you find them.
8. *Interpretation.* A method improves mean accuracy by 0.5 points with a seed standard deviation of 2 points. What should the report include?
9. *Failure analysis.* In the site-prevalence simulation, increase the regularisation λ on the site-identity model. Does the random-split advantage disappear before the held-out-site one? Explain.
10. *Open question.* Design an evaluation for a crop-disease model meant for a new country. Which splits would you report, what would each tell you, and what could none of them tell you?

5.13.1 Answer discussions

1. Concentration (Hoeffding) for one fixed predictor, then the union bound over the class.
2. No: all eight labellings must be realisable, and $(1, 0, 1)$ is not.
3. Expand around $\mathbb{E}_{\mathbb{D}} \hat{f}$ and μ ; one cross term has $\mathbb{E}_{\mathbb{D}}[\hat{f} - \mathbb{E} \hat{f}] = 0$, the other $\mathbb{E}[\mu - y \mid \mathbf{x}] = 0$ with y independent of $\mathbb{D}$.
4. $\mathbb{E}[\text{NLL}] = -q \log p - (1 - q) \log(1 - p)$; its derivative $-q/p + (1 - q)/(1 - p)$ vanishes at $p = q$, and the function is convex in p .
5. $\sigma(a/T) > 0.5 \iff a > 0$. With $u = \log T$, $\frac{\partial}{\partial u} \text{NLL} = \sum_i (p_i - y_i) \cdot (-a_i/T)$.
6. The best inner configuration's error is optimistic (it is a winner's-curse estimate); the outer loop estimates the error of the *procedure* honestly, at extra compute.
7. Normalise features first; use a threshold chosen from the distribution of within-train nearest-neighbour distances.
8. Per-seed results, a paired comparison with a baseline under identical conditions, an uncertainty estimate suited to the design, and cautious language: the improvement is within seed noise.
9. Stronger regularisation shrinks the site weights, reducing the random-split advantage; held-out sites were already unaffected (their one-hot features are unseen).
10. At least: in-country held-out regions (spatial), next season (time), and a different camera or phone model (domain). None can tell you about conditions absent from all collected data (a new disease strain, a new crop variety).

5.14 Self-check and common mistakes

- ☐ I can say which population my test set represents and which unit is independent.
- ☐ I can derive the finite-class bound and explain the winner's curse with it.
- ☐ I can list six kinds of leakage and how to audit for each.
- ☐ I can compute a standard error and a design effect, and bootstrap the right unit.
- ☐ I can explain why temperature scaling changes NLL but not accuracy.

Common mistakes: random splits for claims about new groups; preprocessing before splitting; tuning thresholds, early stopping or prompts on the test set; reporting accuracy for imbalanced classes; frame-level error bars for video; reporting the best seed; treating ECE as sufficient; assuming a group split catches every shortcut.

References

- I. Goodfellow, Y. Bengio and A. Courville. *Deep Learning*. MIT Press, 2016, sections 5.2–5.4 (capacity, estimators, bias and variance) and 11.1–11.3 (practical methodology).
- R. A. Fisher. *The Design of Experiments*. Oliver and Boyd, 1935.
- W. Hoeffding. Probability inequalities for sums of bounded random variables. *JASA*, 58(301):13–30, 1963.
- V. N. Vapnik and A. Ya. Chervonenkis. On the uniform convergence of relative frequencies of events to their probabilities. *Theory of Probability and its Applications*, 16(2):264–280, 1971.
- M. Stone. Cross-validated choice and assessment of statistical predictions. *JRSS B*, 36(2):111–147, 1974.
- B. Efron. Bootstrap methods: another look at the jackknife. *Annals of Statistics*, 7(1):1–26, 1979.
- G. W. Brier. Verification of forecasts expressed in terms of probability. *Monthly Weather Review*, 78(1):1–3, 1950.
- C. Guo, G. Pleiss, Y. Sun and K. Q. Weinberger. On calibration of modern neural networks. *ICML*, 2017.
- A. Torralba and A. A. Efros. Unbiased look at dataset bias. *CVPR*, 2011.
- S. Beery, G. Van Horn and P. Perona. Recognition in terra incognita. *ECCV*, 2018.
- B. Recht, R. Roelofs, L. Schmidt and V. Shankar. Do ImageNet classifiers generalize to ImageNet? *ICML*, 2019.
- R. Geirhos et al. Shortcut learning in deep neural networks. *Nature Machine Intelligence*, 2:665–673, 2020.
- P. W. Koh et al. WILDS: a benchmark of in-the-wild distribution shifts. *ICML*, 2021.
- S. Kapoor and A. Narayanan. Leakage and the reproducibility crisis in machine-learning-based science. *Patterns*, 4(9), 2023.
- S. Shalev-Shwartz and S. Ben-David. *Understanding Machine Learning: From Theory to Algorithms*. Cambridge University Press, 2014.

Part II

Probabilistic learning

Chapter 6

Likelihood, priors and exponential families

Prerequisites: chapters 1 and 3 (least squares, ridge, logistic regression); probability refresher in appendix A.

Learning outcomes: after this chapter you can

1. build a training objective from a probability model, and say which noise model a given loss assumes;
2. compute and distinguish the MLE, the MAP estimate, the posterior mean and the posterior predictive, and explain why they differ;
3. derive conjugate Bernoulli–Beta and Gaussian–Gaussian updates, and Bayesian linear regression, by completing the square;
4. recognise exponential families, derive their mean and variance from the log-partition function, and fit generalised linear models;
5. use the Laplace approximation and the marginal likelihood, and state their assumptions.

6.1 A concrete problem

Three questions from a crop-health project:

1. Of 10 leaves photographed in a new field, 8 show blight. What is the blight rate in the field? And in another field, 0 of 3 photographed leaves show blight — is the rate zero?
2. A detector counts lesions per leaf. Squared error on counts treats predicting 2 instead of 0 the same as 102 instead of 100. Is that sensible?
3. A classifier outputs 0.9. Is that a probability, and how sure should we be of it when the model was trained on 60 images?

Chapters 1–5 chose losses (squared error, cross-entropy) and penalties (ridge, Lasso) and justified them by their behaviour. This chapter derives them from **probability models**: say what is random and how, and the loss follows. That turns every modelling choice into an assumption you can state and test — and it gives uncertainty, not just point estimates.

6.2 How we got here

Inverse probability (1763–1812). Thomas Bayes’s essay, published posthumously by Richard Price in 1763, asked how to reason from observed outcomes back to an unknown probability. Pierre-Simon Laplace developed the idea into a general method (1774–1812) and used it on problems from birth ratios to astronomy; his “rule of succession” — after s successes in n trials, predict the next success with probability $(s + 1)/(n + 2)$ — is the posterior predictive of section 6.5 with a uniform prior.

Likelihood (1912–1925). Ronald Fisher rejected priors as unscientific and proposed choosing parameters to **maximise the likelihood** — the probability of the observed data as a function of the parameters. He defined it carefully (1922), proved properties of the maximum-likelihood estimator, and introduced **sufficiency** and **Fisher information**.

Families and conjugacy (1935–1972). George Darmois, Bernard Koopman and Edwin Pitman (1935–1936) independently showed that only distributions of a particular form — the **exponential family** — have sufficient statistics of fixed size. Howard Raiffa and Robert Schlaifer (1961) systematised **conjugate priors**, for which the posterior stays in the prior’s family. John Nelder and Robert Wedderburn (1972) built generalised linear models on exponential families.

Bayes becomes computable (1953–1992). Markov chain Monte Carlo (Metropolis and colleagues, 1953; Gelfand and Smith, 1990) made posteriors computable for models without conjugate structure. David MacKay (1992) applied Bayesian reasoning to neural networks — Laplace approximations of the posterior and the marginal likelihood (“evidence”) for choosing regularisation — this chapter’s paper bridge.

Why it matters for deep learning. Every standard deep-learning loss is a negative log-likelihood (figure 6.2), and weight decay is a Gaussian prior. What deep learning usually drops is the rest of the Bayesian picture — the posterior over weights — and that is why uncertainty in large networks needs the special tools of chapters 7, 9 and 10.

6.3 Assumptions and notation

Symbol	Meaning
$\mathbb{D}$	the observed data, e.g. $\{(\mathbf{x}^{(i)}, y^{(i)})\}_{i=1}^m$
$\boldsymbol{\theta}$	parameters (weights, rates, means)
$p(\mathbb{D} \mid \boldsymbol{\theta})$	likelihood : probability (density) of the data, as a function of $\boldsymbol{\theta}$
$p(\boldsymbol{\theta})$	prior over parameters
$p(\boldsymbol{\theta} \mid \mathbb{D})$	posterior
$p(\mathbb{D})$	marginal likelihood or evidence, $\int p(\mathbb{D} \mid \boldsymbol{\theta})p(\boldsymbol{\theta}) d\boldsymbol{\theta}$
$\hat{\boldsymbol{\theta}}_{\text{MLE}}, \hat{\boldsymbol{\theta}}_{\text{MAP}}$	maximum-likelihood and maximum a posteriori estimates
$\boldsymbol{\eta}, T(\mathbf{x}), A(\boldsymbol{\eta})$	natural parameter, sufficient statistic, log-partition function

Assumptions:

1. **Conditional independence.** Given $\boldsymbol{\theta}$ (and the inputs), examples are independent, so the likelihood is a product.

2. **A correct-enough model.** The family $p(\cdot | \boldsymbol{\theta})$ is close to how the data arise. When it is not, MLE converges to the member *closest* (in KL divergence, chapter 9) to the truth.
3. **A prior you can state.** For Bayesian answers, the prior is part of the model and must be reported like any other assumption.

6.4 Mathematical formulation

$$\begin{aligned}\hat{\boldsymbol{\theta}}_{\text{MLE}} &= \arg \max_{\boldsymbol{\theta}} \sum_{i=1}^m \log p(y^{(i)} | \mathbf{x}^{(i)}; \boldsymbol{\theta}), \\ p(\boldsymbol{\theta} | \mathbb{D}) &= \frac{p(\mathbb{D} | \boldsymbol{\theta}) p(\boldsymbol{\theta})}{p(\mathbb{D})} \quad (\text{Bayes' rule}), \\ \hat{\boldsymbol{\theta}}_{\text{MAP}} &= \arg \max_{\boldsymbol{\theta}} [\log p(\mathbb{D} | \boldsymbol{\theta}) + \log p(\boldsymbol{\theta})], \\ p(y_* | \mathbf{x}_*, \mathbb{D}) &= \int p(y_* | \mathbf{x}_*; \boldsymbol{\theta}) p(\boldsymbol{\theta} | \mathbb{D}) d\boldsymbol{\theta} \quad (\text{posterior predictive}).\end{aligned}$$

An **exponential-family** distribution has the form

$$p(\mathbf{x} | \boldsymbol{\eta}) = h(\mathbf{x}) \exp(\boldsymbol{\eta}^\top T(\mathbf{x}) - A(\boldsymbol{\eta})), \quad A(\boldsymbol{\eta}) = \log \int h(\mathbf{x}) \exp(\boldsymbol{\eta}^\top T(\mathbf{x})) d\mathbf{x}.$$

6.5 Derivation, with reasons for each step

6.5.1 Step 1: what a likelihood is, and is not

Once the data are observed, $L(\boldsymbol{\theta}) = p(\mathbb{D} | \boldsymbol{\theta})$ is a function of the parameters. It is **not** a probability distribution over $\boldsymbol{\theta}$: it need not integrate to one over $\boldsymbol{\theta}$, and maximising it says nothing about how plausible other values are. Under conditional independence it is a product, and its logarithm a sum: $\log L(\boldsymbol{\theta}) = \sum_i \log p(y^{(i)} | \mathbf{x}^{(i)}; \boldsymbol{\theta})$. Logs turn products of many small numbers (which underflow) into sums, and the maximiser is unchanged because log is increasing.

6.5.2 Step 2: a Bernoulli rate — MLE, MAP, mean and prediction

Let n independent leaves contain s diseased ones, with unknown rate p . Ignoring factors that do not depend on p , $\log L(p) = s \log p + (n - s) \log(1 - p)$. Setting the derivative to zero,

$$\frac{s}{p} - \frac{n - s}{1 - p} = 0 \quad \implies \quad \hat{p}_{\text{MLE}} = \frac{s}{n}.$$

At $s = 0$ the MLE is 0: after three healthy leaves, the MLE claims blight is *impossible*.

A conjugate prior. Take a Beta prior $p(p) \propto p^{a-1}(1 - p)^{b-1}$. Multiplying by the likelihood just adds exponents:

$$p(p | \mathbb{D}) \propto p^{a+s-1}(1 - p)^{b+n-s-1} \quad \implies \quad \text{Beta}(a + s, b + n - s).$$

The prior acts like $a - 1$ imaginary successes and $b - 1$ imaginary failures (“pseudo-counts”). Three different summaries of this posterior are often confused:

- **MAP** (mode): $\frac{a + s - 1}{a + b + n - 2}$, when both shape parameters exceed 1;
- **posterior mean**: $\frac{a + s}{a + b + n}$;
- **posterior predictive** probability that the next leaf is diseased: $\int p \cdot p(p \mid \mathbb{D}) dp =$ the posterior mean.

With a uniform prior ($a = b = 1$) the predictive is $(s + 1)/(n + 2)$ — Laplace’s rule of succession. As n grows, all summaries converge to s/n and the posterior narrows like $1/\sqrt{n}$: the data overwhelm any reasonable prior (figure 6.1).

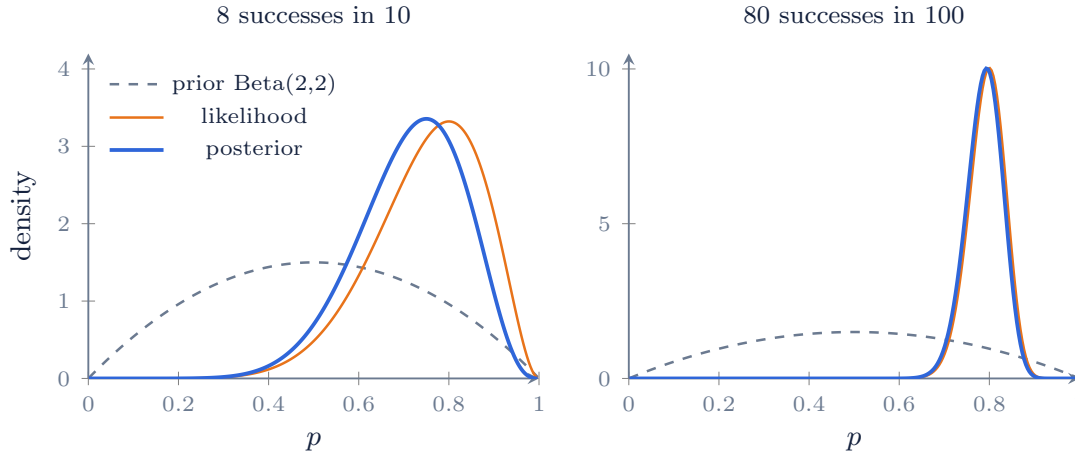


Figure 6.1: Prior times likelihood gives the posterior. With 10 observations the Beta(2, 2) prior visibly pulls the posterior towards 0.5; with 100 it barely matters.

6.5.3 Step 3: losses are noise models

Choose a conditional distribution for the target and take the negative log-likelihood:

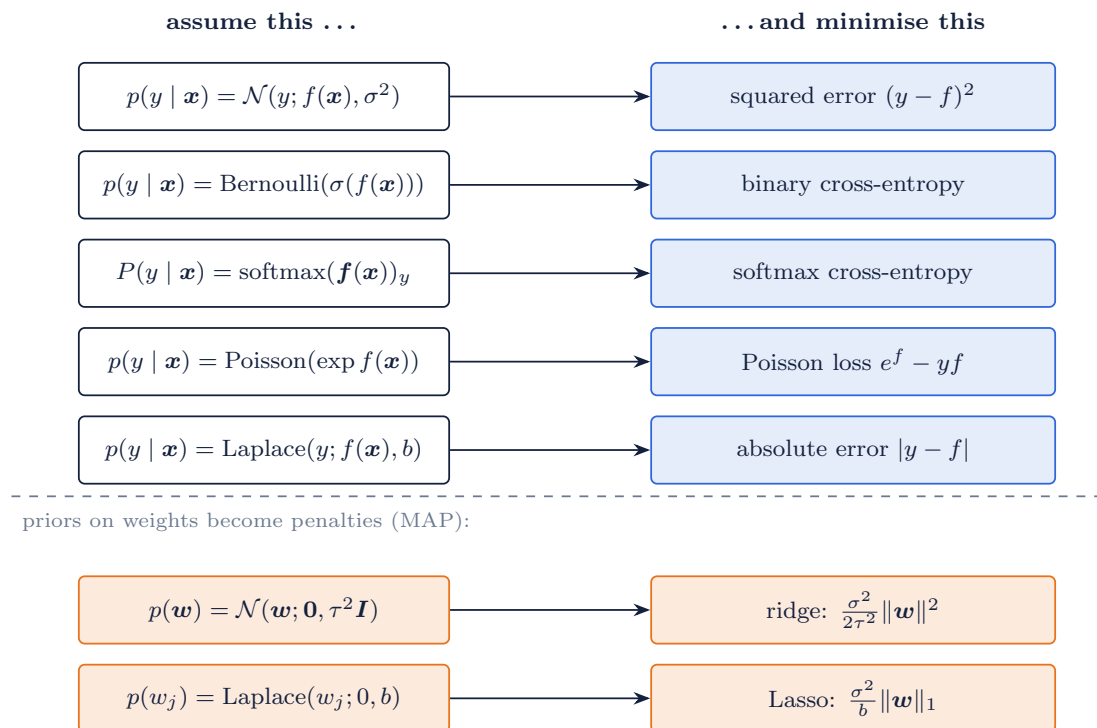
- **Gaussian** $p(y \mid \mathbf{x}) = \mathcal{N}(y; f(\mathbf{x}), \sigma^2)$ gives $-\log p = \frac{1}{2\sigma^2}(y - f(\mathbf{x}))^2 + \frac{1}{2}\log(2\pi\sigma^2)$: **squared error**. Maximising over σ^2 as well gives $\hat{\sigma}^2 = \frac{1}{m} \sum_i (y^{(i)} - f(\mathbf{x}^{(i)}))^2$.
- **Bernoulli** with $p = \sigma(f(\mathbf{x}))$ gives **binary cross-entropy** (chapter 3).
- **Categorical** with softmax probabilities gives **softmax cross-entropy**.
- **Poisson** with rate $\exp f(\mathbf{x})$: $-\log p = e^f - yf + \log y!$, the **Poisson loss**, right for counts (non-negative, variance growing with the mean).
- **Laplace** $p(y \mid \mathbf{x}) \propto \exp(-|y - f(\mathbf{x})|/b)$ gives **absolute error**, which is robust to outliers because its penalty grows linearly, not quadratically.

So the answer to question 2 of section 6.1 is no: squared error on counts assumes Gaussian noise of constant variance, which counts violate. Same backbone network, different likelihood, different head.

6.5.4 Step 4: penalties are priors

Let $y^{(i)} = \mathbf{w}^\top \mathbf{x}^{(i)} + \epsilon_i$ with ϵ_i Gaussian of variance σ^2 , and put a prior $p(\mathbf{w}) = \mathcal{N}(\mathbf{w}; \mathbf{0}, \tau^2 \mathbf{I})$. The negative log posterior is, up to constants,

$$\frac{1}{2\sigma^2} \|\mathbf{X}\mathbf{w} - \mathbf{y}\|^2 + \frac{1}{2\tau^2} \|\mathbf{w}\|^2.$$



Every loss on the right is a negative log-likelihood (up to constants) of the model on the left. Choosing a loss is choosing a noise model; choosing a penalty is choosing a prior.

Figure 6.2: A dictionary from probability models to training objectives. Choosing a loss is choosing a noise model; choosing a penalty is choosing a prior.

Multiplying by σ^2 gives ridge (chapter 3) with $\lambda = \sigma^2/\tau^2$: **MAP with a Gaussian prior is ridge**. A broad prior (large τ^2) means weak regularisation; noisy data (large σ^2) means strong regularisation. A Laplace prior $p(w_j) \propto \exp(-|w_j|/b)$ gives the Lasso with $\lambda = \sigma^2/b$.

This does not mean every penalty is a *justified* prior for a particular dataset. It means the objective embodies a preference that can be stated, criticised and changed.

6.5.5 Step 5: Gaussian updates by completing the square

Scalar. Let $p(\theta) = \mathcal{N}(\theta; \mu_0, v_0)$ and one observation $p(x | \theta) = \mathcal{N}(x; \theta, v)$. The log posterior is $-\frac{(\theta-\mu_0)^2}{2v_0} - \frac{(x-\theta)^2}{2v} + \text{const}$, a quadratic in θ . A quadratic $-\frac{1}{2}a\theta^2 + b\theta$ is the log of a Gaussian with variance $1/a$ and mean b/a (“completing the square”). Reading off a and b :

$$\frac{1}{v_{\text{post}}} = \frac{1}{v_0} + \frac{1}{v}, \quad \mu_{\text{post}} = v_{\text{post}} \left(\frac{\mu_0}{v_0} + \frac{x}{v} \right).$$

Precisions add, and the posterior mean is the precision-weighted average of the prior mean and the observation. Because the posterior is Gaussian, its mean, mode and median coincide.

Bayesian linear regression. The same calculation with vectors: the log posterior for $\mathbf{w}$ is $-\frac{1}{2\sigma^2} \|\mathbf{X}\mathbf{w} - \mathbf{y}\|^2 - \frac{1}{2\tau^2} \|\mathbf{w}\|^2$. Collecting the quadratic and linear terms in $\mathbf{w}$,

$$p(\mathbf{w} | \mathbb{D}) = \mathcal{N}(\mathbf{w}; \mathbf{m}, \mathbf{S}), \quad \mathbf{S}^{-1} = \tau^{-2} \mathbf{I} + \sigma^{-2} \mathbf{X}^\top \mathbf{X}, \quad \mathbf{m} = \sigma^{-2} \mathbf{S} \mathbf{X}^\top \mathbf{y}.$$

The posterior mean $\mathbf{m}$ is the ridge solution. The **posterior predictive** at a new input $\mathbf{x}_*$ is Gaussian with

$$\text{mean } \mathbf{x}_*^\top \mathbf{m}, \quad \text{variance } \underbrace{\mathbf{x}_*^\top \mathbf{S} \mathbf{x}_*}_{\text{uncertainty about } \mathbf{w}} + \underbrace{\sigma^2}_{\text{noise}}.$$

The first term shrinks as data accumulate; the second does not. Replacing the posterior by its mode (the “plug-in” prediction) drops the first term and is over-confident wherever the data are thin. Chapter 7 is this calculation with infinitely many features.

6.5.6 Step 6: exponential families

Many familiar distributions share one form, $p(\mathbf{x} | \boldsymbol{\eta}) = h(\mathbf{x}) \exp(\boldsymbol{\eta}^\top T(\mathbf{x}) - A(\boldsymbol{\eta}))$:

Distribution	$T(\mathbf{x})$	natural parameter $\boldsymbol{\eta}$	$A(\boldsymbol{\eta})$	mean
Bernoulli(p)	x	$\log \frac{p}{1-p}$ (logit)	$\log(1 + e^\eta)$	$\sigma(\eta)$
Poisson(λ)	x	$\log \lambda$	e^η	e^η
Gaussian(μ , known σ^2)	x	μ/σ^2	$\sigma^2 \eta^2/2$	$\sigma^2 \eta$
Categorical	one-hot x	log- probabilities (up to a constant)	$\log \sum_k e^{\eta_k}$	$\text{softmax}(\boldsymbol{\eta})$

The log-partition function generates moments. Differentiate $A(\boldsymbol{\eta}) = \log \int h e^{\boldsymbol{\eta}^\top T}$ under the integral:

$$\nabla A(\boldsymbol{\eta}) = \frac{\int T h e^{\boldsymbol{\eta}^\top T}}{\int h e^{\boldsymbol{\eta}^\top T}} = \mathbb{E}[T(\mathbf{x})], \quad \nabla^2 A(\boldsymbol{\eta}) = \text{Cov}(T(\mathbf{x})).$$

Three consequences:

1. **The NLL is convex in $\boldsymbol{\eta}$.** Its Hessian is a covariance, hence positive semidefinite. This is *why* logistic and softmax regression are convex.
2. **Maximum likelihood is moment matching.** Setting the gradient of $\sum_i [\boldsymbol{\eta}^\top T(\mathbf{x}^{(i)}) - A(\boldsymbol{\eta})]$ to zero gives $\mathbb{E}_{\boldsymbol{\eta}}[T] = \frac{1}{m} \sum_i T(\mathbf{x}^{(i)})$: choose the parameter whose expected sufficient statistic equals the observed average.
3. **Sufficiency.** The data enter the likelihood only through $\sum_i T(\mathbf{x}^{(i)})$; nothing else about the sample matters for $\boldsymbol{\eta}$.

Conjugate priors have the same exponential form in $\boldsymbol{\eta}$: $p(\boldsymbol{\eta}) \propto \exp(\boldsymbol{\eta}^\top \boldsymbol{\chi} - \nu A(\boldsymbol{\eta}))$; the posterior simply adds $\sum_i T(\mathbf{x}^{(i)})$ to $\boldsymbol{\chi}$ and m to ν — pseudo-observations again.

Generalised linear models. Let the natural parameter be linear in the inputs, $\eta^{(i)} = \mathbf{w}^\top \mathbf{x}^{(i)}$. The NLL gradient is $\sum_i (\mathbb{E}[T \mid \eta^{(i)}] - T(y^{(i)})) \mathbf{x}^{(i)} = \mathbf{X}^\top (\boldsymbol{\mu} - \mathbf{y})$ — prediction minus target, for *every* exponential family: linear regression, logistic regression and Poisson regression share one gradient formula and one Newton algorithm.

6.5.7 Step 7: how good is the MLE, and the Laplace approximation

Under regularity conditions, as $m \rightarrow \infty$ the MLE is **consistent** (it converges to the true parameter if the model is correct) and **asymptotically normal**, with covariance equal to the inverse **Fisher information** $\mathcal{I}(\boldsymbol{\theta})^{-1}/m$, where $\mathcal{I} = \mathbb{E}[-\nabla^2 \log p(\mathbf{x} \mid \boldsymbol{\theta})]$. No unbiased estimator does better (the Cramér–Rao bound). These are large-sample statements; with 10 leaves they are optimistic.

The **Laplace approximation** applies the same idea to a posterior. Let $g(\boldsymbol{\theta}) = \log p(\boldsymbol{\theta} \mid \mathbb{D})$ have a maximum at $\hat{\boldsymbol{\theta}}$. A second-order Taylor expansion (the gradient vanishes at a maximum) gives $g(\boldsymbol{\theta}) \approx g(\hat{\boldsymbol{\theta}}) - \frac{1}{2}(\boldsymbol{\theta} - \hat{\boldsymbol{\theta}})^\top \mathbf{H}(\boldsymbol{\theta} - \hat{\boldsymbol{\theta}})$ with $\mathbf{H} = -\nabla^2 g(\hat{\boldsymbol{\theta}})$, so

$$p(\boldsymbol{\theta} \mid \mathbb{D}) \approx \mathcal{N}(\boldsymbol{\theta}; \hat{\boldsymbol{\theta}}, \mathbf{H}^{-1}).$$

Exact when the log posterior is quadratic (Gaussian models); poor when the posterior is skewed, bounded or multimodal. It is centred at the **mode**, not the mean.

6.5.8 Step 8: the marginal likelihood compares models

The denominator of Bayes' rule, $p(\mathbb{D}) = \int p(\mathbb{D} \mid \boldsymbol{\theta}) p(\boldsymbol{\theta}) d\boldsymbol{\theta}$, is the probability a *model* (prior and likelihood together) assigns to the data. A flexible model spreads its probability over many possible datasets and so gives each one little; a model too rigid to fit gives the actual data little. The marginal likelihood therefore prefers the simplest model that fits — the Occam effect used to set GP hyperparameters in chapter 7. For Bayesian linear regression it is Gaussian: $p(\mathbf{y} \mid \mathbf{X}) = \mathcal{N}(\mathbf{y}; \mathbf{0}, \tau^2 \mathbf{X} \mathbf{X}^\top + \sigma^2 \mathbf{I})$, so it can choose τ^2 — and hence the ridge λ — without a validation set (section 6.11).

6.6 Numerical worked example

A Bernoulli rate. $s = 8$, $n = 10$, prior $\text{Beta}(2, 2)$: MLE 0.8; MAP $9/12 = 0.75$; posterior mean $10/14 \approx 0.714$. The prior pulls towards 0.5 but does not insert observations into the data. For $s = 0$, $n = 3$: MLE 0, MAP $1/5 = 0.2$, posterior mean $2/7 \approx 0.286$.

A Gaussian update. Prior $\mathcal{N}(\theta; 0, 1)$, one observation $x = 2$ with noise variance 0.25 (precision 4): posterior precision $1 + 4 = 5$, variance 0.2, mean $0.2 \times (0 + 4 \cdot 2) = 1.6$. Equivalently, $\theta^2 + 4(x - \theta)^2 = 5(\theta - 0.8x)^2 + 0.8x^2$.

MAP as ridge. $\sigma^2 = 4$, $\tau^2 = 2$: $\lambda = 4/2 = 2$ in the summed-loss convention.

Exponential-family moments. Bernoulli with $\eta = 0.7$: $A'(0.7) = \sigma(0.7) = 0.668$ (the mean) and $A''(0.7) = 0.668 \times 0.332 = 0.222$ (the variance). Poisson with $\eta = \log 3$: both derivatives of e^η equal 3, the Poisson's mean *and* variance — a restrictive assumption real counts often violate (over-dispersion).

6.7 Small implementation and checks

The lab is `labs/ch06-probability/probability.py`. The key routines are a few lines each:

```
def gaussian_update(x, prior_mean=0.0, prior_var=1.0, noise_var=0.25):
    prec = 1 / prior_var + 1 / noise_var                # precisions
    ↪ add
    return (prior_mean / prior_var + x / noise_var) / prec, 1 / prec

def poisson_regression(X, y, lam=1e-3):                # GLM, Newton's
    ↪ method
    ... mu = exp(X @ w); g = X.T @ (mu - y) + lam * w; H = X.T @ (mu *
    ↪ X) + lam * I
```

The tests in `test_probability.py` check the worked examples, that the prior rescues zero counts, that the posterior is normalised and narrows with data, that MAP equals ridge, that numerical derivatives of A equal the mean and variance, that Poisson regression recovers its weights, and that the Laplace approximation is centred at the mode.

6.8 Experiment and expected observations

Run `python labs/ch06-probability/probability.py` (seed 0).

data	MLE	MAP	posterior mean	95% credible interval
8 of 10	0.800	0.750	0.714	(0.462, 0.909)
0 of 3	0.000	0.200	0.286	(0.043, 0.641)
80 of 100	0.800	0.794	0.788	(0.705, 0.861)

What to notice:

1. **With little data the summaries disagree**, and the interval is wide: 8 of 10 is consistent with rates from about 0.46 to 0.91. Reporting “80%” alone hides that.
2. **Zero counts.** The MLE says blight is impossible after three healthy leaves; the posterior says the rate is probably below 0.64. Small subgroups in evaluation (chapter 5) have the same problem.

3. **With more data the prior fades.** At 80 of 100 all estimates are within 0.012 of each other.
4. **The Laplace approximation is centred at the mode.** For 8 of 10 it gives $\mathcal{N}(\theta; 0.750, 0.125^2)$; the exact $\text{Beta}(10, 4)$ posterior has mean 0.714 and standard deviation 0.117. At 80 of 100 it gives $\mathcal{N}(\theta; 0.794, 0.040^2)$, close to exact.
5. **Predictive variance has two parts.** In the Bayesian regression example, the predictive variance at a test input is 0.162 (uncertainty about the weights) plus 4.0 (noise). More data shrink the first, never the second.
6. **Poisson regression** on 300 simulated lesion counts recovers weights (0.48, 0.78, -0.32) against true (0.5, 0.8, -0.4) with the same Newton code as logistic regression.

6.9 Moving pieces: diagnosing and fixing

Moving piece	Symptom when it is wrong	How to confirm	Fix
Noise model (likelihood)	residuals with structure: variance growing with the mean, heavy tails, counts below zero	residual vs. prediction plots; histogram of residuals	choose the likelihood for the target type (figure 6.2); heteroscedastic or robust likelihoods
Prior strength	estimates stuck near the prior (too strong) or wild with few data (too weak)	prior predictive checks: sample data from the prior and look at it	set priors in interpretable units; sensitivity analysis over prior scales
Point vs. predictive	confident predictions where data are thin	compare plug-in and posterior predictive variance	report the predictive distribution; ensembles or Bayesian methods
Mode vs. mean	reported value moves when the parameterisation changes	compute both	say which summary you report; prefer the posterior mean or predictive for decisions
Small counts	MLE of 0 or 1	count per subgroup	Beta prior / Laplace smoothing; report intervals
Over-dispersion	Poisson model's intervals too narrow	compare variance and mean of counts	negative binomial likelihood
Laplace approximation	intervals symmetric where truth is skewed; outside parameter bounds	compare with exact or sampled posterior	transform to an unbounded parameter (logit, log); sampling
Model misspecification	good fit, poor calibration on new data	calibration on held-out data (chapter 5)	richer likelihood; accept and report it

First things to try: (1) write down the likelihood your loss implies and check it against the target type; (2) plot residuals; (3) for small groups, report a posterior interval, not a ratio; (4) sample data from your prior to see whether it is sensible.

6.10 Application, limitations and failure cases

Application (educational case study). A vision model that predicts disease probability, lesion counts and leaf area shares one backbone but needs three different heads and losses: Bernoulli cross-entropy, a Poisson (or negative binomial) loss, and a Gaussian or log-normal loss. For per-field disease rates estimated from a handful of photographs, report a Beta posterior interval and use a weak prior centred on the regional rate. When comparing subgroups in an evaluation (chapter 5), use the same machinery to avoid declaring a rate of 0% from three examples.

Limitations:

- A prior is an assumption; its influence must be checked, especially with small data.
- Conjugacy is a convenience, not a law of nature; most models need approximations (chapters 8–9).
- MLE’s large-sample guarantees say little about small datasets or misspecified models.
- In deep networks, the MAP estimate (weights trained with weight decay) carries no uncertainty by itself.

6.11 Bridge to a research paper

MacKay (1992), *A practical Bayesian framework for backpropagation networks* (Neural Computation, 4(3):448–472).

- **Problem:** how to choose regularisation constants and compare network architectures without a separate validation set.
- **Mechanism:** a Gaussian prior on weights (weight decay), a Laplace approximation of the posterior around the trained weights, and the marginal likelihood (“evidence”) for choosing the prior scale and comparing models.
- **Central claim:** the evidence chooses sensible regularisation and correlates with generalisation.
- **Assumptions to question:** is the posterior near-Gaussian around one mode? What happens when the model is misspecified?
- **Affordable experiment:** in Bayesian linear regression (where the evidence is exact, step 8), compute $\log p(\mathbf{y} \mid \mathbf{X}; \tau^2)$ over a grid of prior variances, and plot it next to validation error for the corresponding ridge $\lambda = \sigma^2/\tau^2$. Do the two criteria pick similar values? Repeat with heavy-tailed noise.

6.12 Memory map

Summary. Say what is random and how: the negative log-likelihood is then the loss (Gaussian gives squared error, Bernoulli cross-entropy, Poisson the Poisson loss), and a prior becomes a penalty (Gaussian gives ridge, Laplace the Lasso). Bayes’ rule turns prior and likelihood into a posterior; the MLE, the MAP estimate, the posterior mean and the predictive are different summaries that agree only with plenty of data. Conjugate updates add pseudo-counts, and Gaussian updates add precisions. Exponential families unify the common distributions: the log-partition function gives mean and variance, makes the NLL convex, turns maximum likelihood into moment matching, and gives every GLM the gradient “prediction minus target”. The Laplace approximation and the marginal likelihood extend these ideas when exact posteriors are unavailable.



Figure 6.3: Memory map of chapter 6.

6.13 Exercises

1. *Conceptual.* Why may we omit a constant from MAP optimisation but not from a normalised density?
2. *Conceptual.* Does multiplying a likelihood by a prior always give a Gaussian posterior?
3. *Derivation.* With $\sigma^2 = 4$ and $\tau^2 = 2$, what is the equivalent summed-loss ridge coefficient? What if the data loss is averaged over $m = 100$ examples?
4. *Derivation.* Derive the posterior predictive of a Beta–Bernoulli model and show it equals the posterior mean.
5. *Derivation.* Prove $\nabla^2 A(\boldsymbol{\eta}) = \text{Cov}(T)$ and conclude that the NLL of a GLM is convex.
6. *Derivation.* Derive the Bayesian linear regression posterior $\mathbf{m}$, $\mathbf{S}$ by completing the square.
7. *Implementation.* Fit a negative binomial regression to over-dispersed simulated counts and compare its predictive intervals with Poisson regression.
8. *Implementation.* Carry out the experiment of section 6.11.
9. *Interpretation.* A model reports a 2% error rate on a subgroup of 50 images. Give a 95% interval with a uniform prior and explain what it means for the claim.
10. *Open question.* Weight decay in a deep network is a Gaussian prior. What would it take to turn a trained network’s MAP estimate into useful uncertainty? (chapters 7, 9 and 10 give some answers.)

6.13.1 Answer discussions

1. A parameter-independent constant does not move the maximiser; a density must still integrate to one.
2. No. Conjugacy needs a likelihood that is Gaussian in the parameter; a logistic likelihood gives a non-Gaussian posterior.
3. $\lambda = 2$; with an averaged loss the equivalent coefficient is $2/100 = 0.02$.
4. $P(x_{\text{new}} = 1 \mid \mathbb{D}) = \int p \text{Beta}(p; a + s, b + n - s) dp = (a + s)/(a + b + n)$.
5. Differentiate $\nabla A = \mathbb{E}[T]$ once more under the integral; the NLL Hessian $\mathbf{X}^\top \text{diag}(\text{Var}[T \mid \eta^{(i)}]) \mathbf{X}$ is PSD.
6. Expand both quadratics; the coefficient of $\mathbf{w}^\top \mathbf{w}$ -type terms gives $\mathbf{S}^{-1}$, the linear term gives $\mathbf{S}^{-1} \mathbf{m} = \sigma^{-2} \mathbf{X}^\top \mathbf{y}$.
7. The negative binomial’s intervals widen to cover the extra variance; Poisson intervals under-cover.
8. Both criteria usually pick similar λ under Gaussian noise; with heavy tails the evidence can be misled because the likelihood is wrong.
9. One error in 50: posterior Beta(2, 50), 95% interval roughly (0.5%, 10%). The data are consistent with an error rate five times larger than reported.
10. A posterior or an approximation to it: Laplace around the trained weights, variational inference, ensembles, or Monte Carlo methods — each with its own cost and assumptions.

6.14 Self-check and common mistakes

- ☐ I can name the noise model behind squared error, cross-entropy and the Poisson loss.
- ☐ I can compute MLE, MAP, posterior mean and predictive for a Beta–Bernoulli model.
- ☐ I can derive a Gaussian update by completing the square.
- ☐ I can derive $\nabla A = \mathbb{E}[T]$ and use it.
- ☐ I can state when the Laplace approximation is exact and when it misleads.

Common mistakes: calling the likelihood a distribution over parameters; reporting a MAP

estimate as “the probability”; using squared error for counts or probabilities; forgetting that weight decay values depend on the loss scale; reporting rates from tiny subgroups without intervals; treating the Laplace approximation’s symmetry as a property of the true posterior.

References

- I. Goodfellow, Y. Bengio and A. Courville. *Deep Learning*. MIT Press, 2016, chapter 3 and sections 5.5–5.6 (maximum likelihood and Bayesian statistics).
- T. Bayes. An essay towards solving a problem in the doctrine of chances. *Phil. Trans. R. Soc.*, 53:370–418, 1763.
- R. A. Fisher. On the mathematical foundations of theoretical statistics. *Phil. Trans. R. Soc. A*, 222:309–368, 1922.
- B. O. Koopman. On distributions admitting a sufficient statistic. *Trans. AMS*, 39(3):399–409, 1936.
- H. Raiffa and R. Schlaifer. *Applied Statistical Decision Theory*. Harvard University, 1961.
- J. A. Nelder and R. W. M. Wedderburn. Generalized linear models. *JRSS A*, 135(3):370–384, 1972.
- D. J. C. MacKay. A practical Bayesian framework for backpropagation networks. *Neural Computation*, 4(3):448–472, 1992.
- C. M. Bishop. *Pattern Recognition and Machine Learning*. Springer, 2006, chapters 2–3.
- K. P. Murphy. *Probabilistic Machine Learning: An Introduction*. MIT Press, 2022, chapters 3–4.
- A. Gelman, J. B. Carlin, H. S. Stern, D. B. Dunson, A. Vehtari and D. B. Rubin. *Bayesian Data Analysis*, 3rd edition. CRC Press, 2013.

Chapter 7

Gaussian conditioning and Gaussian processes

Prerequisites: chapter 4 (kernels and Gram matrices), chapter 6 (likelihood, priors, MAP); multivariate Gaussian density and Cholesky factorisation (appendices A and C).

Learning outcomes: after this chapter you can

1. derive the conditional distribution of a partitioned Gaussian;
2. state a GP prior, sample functions from it, and explain what the kernel’s hyperparameters assume;
3. derive the GP regression posterior mean and covariance two ways — by conditioning (function space) and by Bayesian linear regression (weight space) — and compute them with a Cholesky factorisation;
4. use the log marginal likelihood for model selection, and say what it does not measure;
5. check whether predictive intervals are calibrated, and know how GPs are extended to classification, large data and Bayesian optimisation.

7.1 A concrete problem

A sensor reports a smooth signal at 30 irregular times, with a gap where it was offline. We want two things at any query point: a prediction, and an honest statement of how unsure that prediction is. Inside the gap the answer should be “less sure”.

Ridge regression (chapter 3) and kernel ridge regression (chapter 4) give a prediction but no uncertainty. Chapter 6 gave uncertainty over a few parameters. Here we put a probability distribution directly over *functions*. Because every quantity we need is jointly Gaussian, the whole posterior has a closed form.

The same need appears in vision and beyond: interpolating a sparse depth map, predicting crop yield between surveyed fields, or choosing which hyperparameters of a large network to try next when each training run costs a day (Bayesian optimisation, section 7.10).

7.2 How we got here

Predicting random processes (1940s). Norbert Wiener and Andrey Kolmogorov independently solved the problem of predicting a stationary random signal from its past: the best linear predictor is determined entirely by the signal’s **covariance function**. That

is the core of a GP: specify how values at two points co-vary, and optimal prediction follows.

Finding gold (1951–1963). Danie Krige, a South African mining engineer, estimated ore grades between boreholes by weighting nearby samples according to how grades co-varied with distance. Georges Matheron (1963) put the method on a mathematical footing and named it **kriging**. Geostatistics has used GP regression ever since, under that name.

From geostatistics to machine learning (1978–2006). Anthony O’Hagan (1978) used Gaussian-process priors for Bayesian curve fitting. Radford Neal (1996) showed that a neural network with one hidden layer becomes a GP as its width goes to infinity, which made GPs interesting to the neural-network community. Christopher Williams and Carl Rasmussen (1996) then presented GP regression as a practical machine-learning method, with hyperparameters set by the marginal likelihood; their 2006 book is the standard reference and this chapter’s paper bridge.

What broke, and what came next. *Cost:* exact GPs scale as $O(m^3)$, which led to sparse approximations with $M \ll m$ inducing points (Snelson and Ghahramani, 2006; Titsias, 2009) and GPU-friendly iterative solvers (GPpyTorch, Gardner et al., 2018). *Non-Gaussian likelihoods:* for classification the posterior is no longer Gaussian and needs approximation (Laplace, expectation propagation, or the variational methods of chapter 9). *The kernel:* a hand-chosen kernel on raw inputs is a weak prior for images, which is where learned features (chapters 11–18) took over — although GPs on top of learned features, and GPs for deciding *which experiment to run next* (Bayesian optimisation, Jones et al. 1998; Snoek, Larochelle and Adams, 2012), remain widely used.

7.3 Assumptions and notation

Symbol	Shape	Meaning
$\mathbf{X}$	$m \times n$	training inputs, $\mathbf{X}_{i,:} = \mathbf{x}^{(i)\top}$
$\mathbf{y}$	m	noisy training targets, $y_i = y^{(i)}$
$\mathbf{X}_*$	$m_* \times n$	query inputs
$\mathbf{f}, \mathbf{f}_*$	m, m_*	latent function values at $\mathbf{X}$ and at $\mathbf{X}_*$
$k(\mathbf{x}, \mathbf{x}')$	scalar	positive semidefinite kernel (chapter 4)
$\mathbf{K}$	$m \times m$	Gram matrix, $K_{i,j} = k(\mathbf{x}^{(i)}, \mathbf{x}^{(j)})$
$\mathbf{K}_*$	$m \times m_*$	kernel between training and query inputs
$\mathbf{K}_{**}$	$m_* \times m_*$	kernel among query inputs
σ_n^2	scalar	observation-noise variance
θ	—	hyperparameters $(\ell, \sigma_f, \sigma_n)$

Assumptions:

1. **Prior.** A **Gaussian process** (GP) prior with mean zero and kernel k : for any finite set of inputs, the function values are jointly Gaussian, $p(\mathbf{f}) = \mathcal{N}(\mathbf{f}; \mathbf{0}, \mathbf{K})$. A zero mean is a modelling choice. Centre $\mathbf{y}$ first, or add a mean function.

2. **Likelihood.** $y^{(i)} = f(\mathbf{x}^{(i)}) + \epsilon_i$ with independent Gaussian noise of variance σ_n^2 , so $p(\mathbf{y} | \mathbf{f}) = \mathcal{N}(\mathbf{y}; \mathbf{f}, \sigma_n^2 \mathbf{I})$.
3. **Kernel encodes smoothness.** With the squared-exponential (RBF) kernel $k(\mathbf{x}, \mathbf{x}') = \sigma_f^2 \exp(-\|\mathbf{x} - \mathbf{x}'\|^2 / 2\ell^2)$, the **length-scale** ℓ sets how far apart two inputs can be and still have strongly correlated outputs, and σ_f^2 is the prior variance of f at any point.

What a prior over functions *looks like* is easiest to see by sampling from it: pick a grid of inputs, form $\mathbf{K}$, factor $\mathbf{K} = \mathbf{L}\mathbf{L}^\top$, and set $\mathbf{f} = \mathbf{L}\mathbf{z}$ with $\mathbf{z}$ standard normal (its covariance is $\mathbf{L}\mathbf{L}^\top = \mathbf{K}$).

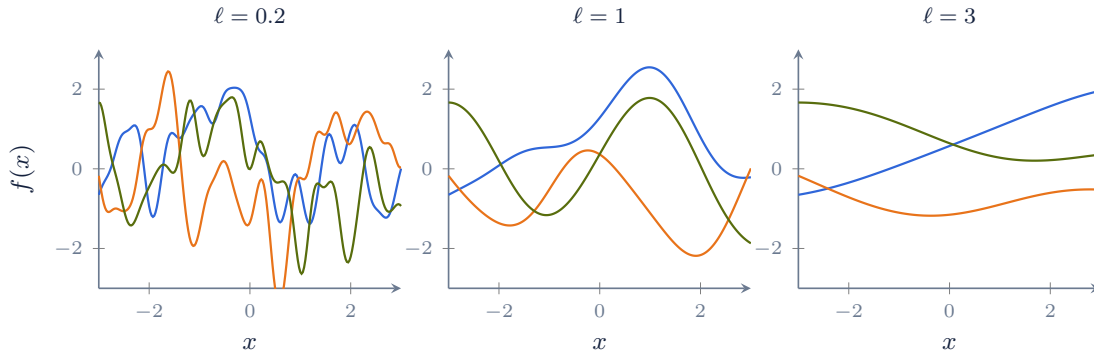


Figure 7.1: Three functions drawn from an RBF-kernel GP prior at each of three length-scales. The length-scale is the typical horizontal distance over which a function changes appreciably; σ_f sets the typical vertical size.

7.4 Mathematical formulation

Under the assumptions, training targets and query values are jointly Gaussian:

$$p\begin{pmatrix} \mathbf{y} \\ \mathbf{f}_* \end{pmatrix} = \mathcal{N}\left(\begin{pmatrix} \mathbf{y} \\ \mathbf{f}_* \end{pmatrix}; \begin{pmatrix} \mathbf{0} \\ \mathbf{0} \end{pmatrix}, \begin{pmatrix} \mathbf{K} + \sigma_n^2 \mathbf{I} & \mathbf{K}_* \\ \mathbf{K}_*^\top & \mathbf{K}_{**} \end{pmatrix}\right).$$

Prediction means computing $p(\mathbf{f}_* | \mathbf{y})$. Model selection means choosing $\boldsymbol{\theta}$, for instance by maximising $\log p(\mathbf{y} | \mathbf{X}; \boldsymbol{\theta})$.

7.5 Derivation, with reasons for each step

Step 1: conditioning a partitioned Gaussian. Let the random vectors $\mathbf{a}$ and $\mathbf{b}$ be jointly Gaussian,

$$p\begin{pmatrix} \mathbf{a} \\ \mathbf{b} \end{pmatrix} = \mathcal{N}\left(\begin{pmatrix} \mathbf{a} \\ \mathbf{b} \end{pmatrix}; \begin{pmatrix} \boldsymbol{\mu}_a \\ \boldsymbol{\mu}_b \end{pmatrix}, \begin{pmatrix} \mathbf{A} & \mathbf{C} \\ \mathbf{C}^\top & \mathbf{B} \end{pmatrix}\right),$$

with $\mathbf{A}$ invertible. Define $\mathbf{r} = \mathbf{b} - \mathbf{C}^\top \mathbf{A}^{-1} \mathbf{a}$. Then

$$\text{Cov}(\mathbf{r}, \mathbf{a}) = \mathbf{C}^\top - \mathbf{C}^\top \mathbf{A}^{-1} \mathbf{A} = \mathbf{0}.$$

Why this step: $\mathbf{r}$ and $\mathbf{a}$ are jointly Gaussian and uncorrelated, so they are independent, $\mathbf{r} \perp \mathbf{a}$. Knowing $\mathbf{a}$ therefore tells us nothing new about $\mathbf{r}$. Write $\mathbf{b} = \mathbf{r} + \mathbf{C}^\top \mathbf{A}^{-1} \mathbf{a}$. Given $\mathbf{a} = \mathbf{a}$, the second term is a constant and $\mathbf{r}$ keeps its marginal distribution, with mean $\boldsymbol{\mu}_b - \mathbf{C}^\top \mathbf{A}^{-1} \boldsymbol{\mu}_a$ and covariance $\mathbf{B} - \mathbf{C}^\top \mathbf{A}^{-1} \mathbf{C}$ (expand $\text{Cov}(\mathbf{b} - \mathbf{C}^\top \mathbf{A}^{-1} \mathbf{a})$). Hence

$$p(\mathbf{b} \mid \mathbf{a}) = \mathcal{N}(\mathbf{b}; \boldsymbol{\mu}_b + \mathbf{C}^\top \mathbf{A}^{-1}(\mathbf{a} - \boldsymbol{\mu}_a), \mathbf{B} - \mathbf{C}^\top \mathbf{A}^{-1} \mathbf{C}).$$

The conditional covariance $\mathbf{B} - \mathbf{C}^\top \mathbf{A}^{-1} \mathbf{C}$ is the **Schur complement** of $\mathbf{A}$. It does not depend on the observed value $\mathbf{a}$. That is a special property of Gaussians, and it fails for most other distributions.

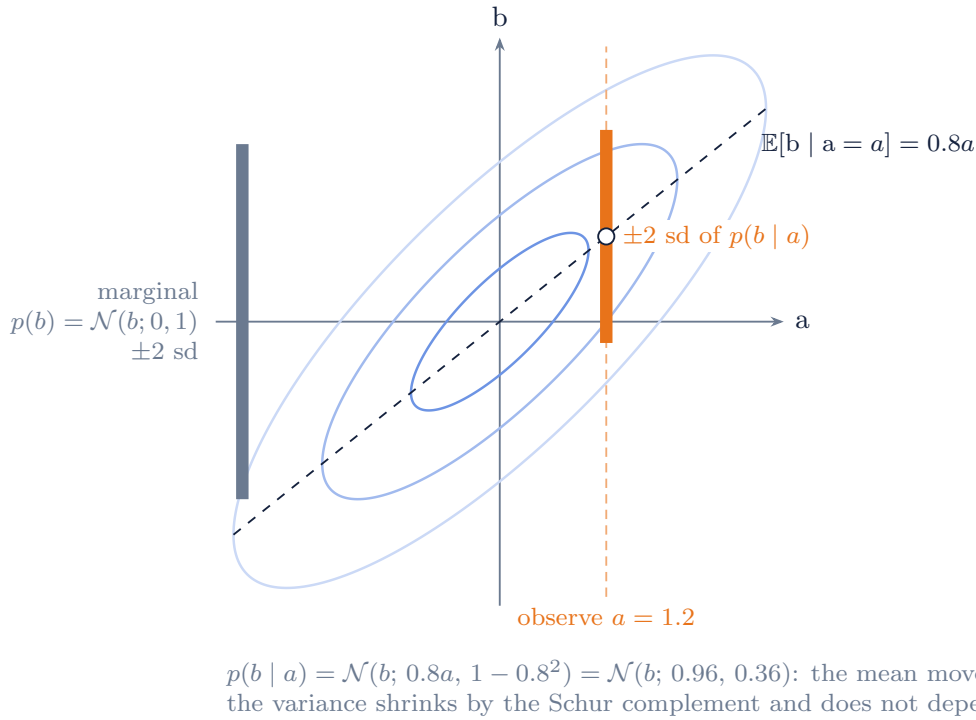


Figure 7.2: Conditioning a 2-D Gaussian over (a, b) with unit variances and correlation $\rho = 0.8$ on $a = 1.2$. The conditional mean lies on the regression line ρa , and the conditional interval is narrower than the marginal one.

Step 2: the GP posterior. Substitute $\mathbf{a} = \mathbf{y}$, $\mathbf{b} = \mathbf{f}_*$, $\mathbf{A} = \mathbf{K} + \sigma_n^2 \mathbf{I}$, $\mathbf{C} = \mathbf{K}_*$, $\mathbf{B} = \mathbf{K}_{**}$ and zero means. Then $p(\mathbf{f}_* \mid \mathbf{y}) = \mathcal{N}(\mathbf{f}_*; \bar{\mathbf{f}}_*, \boldsymbol{\Sigma}_*)$ with

$$\bar{\mathbf{f}}_* = \mathbf{K}_*^\top (\mathbf{K} + \sigma_n^2 \mathbf{I})^{-1} \mathbf{y}, \quad \boldsymbol{\Sigma}_* = \mathbf{K}_{**} - \mathbf{K}_*^\top (\mathbf{K} + \sigma_n^2 \mathbf{I})^{-1} \mathbf{K}_*$$

Read the two terms. The mean is a weighted combination of training targets, with weights $\boldsymbol{\alpha} = (\mathbf{K} + \sigma_n^2 \mathbf{I})^{-1} \mathbf{y}$. It is the kernel ridge regression solution with regularisation weight $\lambda = \sigma_n^2$ (chapter 4): **the GP mean is kernel ridge, and the GP adds the variance**. The covariance starts at the prior $\mathbf{K}_{**}$ and is *reduced* by the information the data carry about $\mathbf{f}_*$. Far from all training inputs $\mathbf{K}_* \approx \mathbf{0}$, so the posterior falls back to the prior: mean 0, variance σ_f^2 . The predictive variance for a *noisy observation* y_* adds σ_n^2 .

Step 3: the same answer from weight space. Where does a GP come from? Take a linear model on fixed features, $f(\mathbf{x}) = \phi(\mathbf{x})^\top \mathbf{w}$, with a Gaussian prior $p(\mathbf{w}) = \mathcal{N}(\mathbf{w}; \mathbf{0}, \sigma_w^2 \mathbf{I})$

(chapter 6). Then $\mathbf{f} = \mathbf{\Phi}\mathbf{w}$ is a linear function of a Gaussian vector, so it is Gaussian with mean $\mathbf{0}$ and covariance $\sigma_w^2 \mathbf{\Phi}\mathbf{\Phi}^\top$. That is a GP with kernel $k(\mathbf{x}, \mathbf{x}') = \sigma_w^2 \phi(\mathbf{x})^\top \phi(\mathbf{x}')$.

The weight-space posterior (chapter 6) is $\mathcal{N}(\mathbf{w}; \mathbf{m}, \mathbf{S})$ with $\mathbf{S}^{-1} = \sigma_w^{-2} \mathbf{I} + \sigma_n^{-2} \mathbf{\Phi}^\top \mathbf{\Phi}$ and $\mathbf{m} = \sigma_n^{-2} \mathbf{S} \mathbf{\Phi}^\top \mathbf{y}$, so a prediction at $\mathbf{x}_*$ has mean $\phi(\mathbf{x}_*)^\top \mathbf{m}$ and variance $\phi(\mathbf{x}_*)^\top \mathbf{S} \phi(\mathbf{x}_*)$. Applying the matrix identity of section 4.5 (step 1) shows these equal the GP formulas of step 2 — the lab checks it to 10^{-13} . Two consequences:

- **The function-space view needs only the kernel**, so it works even when the feature space is infinite (the RBF kernel corresponds to infinitely many basis functions). That is the GP's advantage over Bayesian linear regression.
- **Infinitely wide neural networks with random weights are GPs** (Neal, 1996): the output is a sum of many independent hidden units, and the central limit theorem makes it Gaussian.

Step 4: the marginal likelihood. Since $p(\mathbf{y} | \mathbf{X}; \boldsymbol{\theta}) = \mathcal{N}(\mathbf{y}; \mathbf{0}, \mathbf{K}_y)$ with $\mathbf{K}_y = \mathbf{K} + \sigma_n^2 \mathbf{I}$, the **log marginal likelihood** is

$$\log p(\mathbf{y} | \mathbf{X}; \boldsymbol{\theta}) = \underbrace{-\frac{1}{2} \mathbf{y}^\top \mathbf{K}_y^{-1} \mathbf{y}}_{\text{data fit}} \underbrace{-\frac{1}{2} \log \det(\mathbf{K}_y)}_{\text{complexity penalty}} - \frac{m}{2} \log 2\pi.$$

Why both terms: a very short length-scale fits the data well but makes $\det(\mathbf{K}_y)$ large, because the model could have explained many other datasets equally well — it spreads its probability thinly. A very long length-scale is simple but cannot fit. Their sum is an automatic trade-off, a form of **Occam's razor**. It is not a held-out error, and it assumes the model family is correct.

Step 5: learning hyperparameters by gradient. Differentiating with the identities $\partial \mathbf{K}_y^{-1} = -\mathbf{K}_y^{-1} (\partial \mathbf{K}_y) \mathbf{K}_y^{-1}$ and $\partial \log \det \mathbf{K}_y = \text{Tr}(\mathbf{K}_y^{-1} \partial \mathbf{K}_y)$,

$$\frac{\partial}{\partial \theta_j} \log p(\mathbf{y} | \mathbf{X}; \boldsymbol{\theta}) = \frac{1}{2} \text{Tr} \left((\boldsymbol{\alpha} \boldsymbol{\alpha}^\top - \mathbf{K}_y^{-1}) \frac{\partial \mathbf{K}_y}{\partial \theta_j} \right), \quad \boldsymbol{\alpha} = \mathbf{K}_y^{-1} \mathbf{y}.$$

Optimise over $\log \theta_j$ so that positivity is automatic. The objective is non-convex; restart from several initial values. Treating hyperparameters this way is called **type-II maximum likelihood** or **empirical Bayes**.

Step 6: compute stably with Cholesky. Never form $\mathbf{K}_y^{-1}$. Factor $\mathbf{K}_y = \mathbf{L}\mathbf{L}^\top$ once, with $\mathbf{L}$ lower triangular, at $O(m^3)$ cost. Then

- $\boldsymbol{\alpha} = \mathbf{L}^\top \backslash (\mathbf{L} \backslash \mathbf{y})$ and $\bar{\mathbf{f}}_* = \mathbf{K}_*^\top \boldsymbol{\alpha}$;
- $\mathbf{Z} = \mathbf{L} \backslash \mathbf{K}_*$ and $(\boldsymbol{\Sigma}_*)_{j,j} = (\mathbf{K}_{**})_{j,j} - \sum_i Z_{i,j}^2$;
- $\frac{1}{2} \log \det(\mathbf{K}_y) = \sum_i \log L_{i,i}$.

Here $\mathbf{L} \backslash \mathbf{b}$ means solving the triangular system $\mathbf{L}\mathbf{z} = \mathbf{b}$. Each solve costs $O(m^2)$. The log-determinant comes almost for free, and nothing overflows (appendix C). Adding a small jitter (for example $10^{-8} \mathbf{I}$) before factorising is standard when σ_n is tiny.

Step 7: designing kernels. The closure rules of section 4.5 (step 3) become modelling tools. A sum $k_1 + k_2$ models a function that is a sum of two independent parts (a smooth trend plus a periodic component). A product $k_1 k_2$ models interaction (a periodic pattern whose shape drifts slowly). Separate length-scales per input dimension (“automatic relevance determination”) let the marginal likelihood switch off irrelevant inputs by making

their length-scales very long. Read a fitted GP by asking: what is the mean, what are the components of the covariance, and what length-scales did the data choose?

Step 8: beyond Gaussian likelihoods and small data.

- **Classification.** With $p(y | f) = \sigma(f)$, the posterior over $\mathbf{f}$ is no longer Gaussian. The Laplace approximation (a Gaussian at the posterior mode, appendix and chapter 9), expectation propagation, or variational inference (chapter 9) give practical approximations.
- **Large m .** Sparse GPs summarise the data through M inducing points, giving $O(mM^2)$ cost; iterative solvers (conjugate gradients) use only matrix–vector products and run well on GPUs.

7.6 Numerical worked example

One training point $x^{(1)} = 0$ with $y^{(1)} = 1$. Use the RBF kernel with $\sigma_f^2 = 1$ and $\ell = 1$, and noise $\sigma_n^2 = 0.25$. Query $x_* = 1$.

- $K_y = 1 + 0.25 = 1.25$, $K_* = e^{-1/2} \approx 0.607$, $K_{**} = 1$.
- Mean: $\bar{f}_* = 0.607 \times 1/1.25 \approx 0.485$.
- Variance: $1 - 0.607^2/1.25 = 1 - 0.294 \approx 0.706$.

At the training input itself ($x_* = 0$): mean $1/1.25 = 0.8$, variance $1 - 1/1.25 = 0.2$. The mean is pulled towards the observation but not all the way, because the observation is noisy. The variance shrinks but not to zero. Halve σ_n^2 and repeat: the mean moves closer to 1 and the variance shrinks further.

A scalar Gaussian update, by completing the square. Let $p(\theta) = \mathcal{N}(\theta; 0, 1)$ and $p(x | \theta) = \mathcal{N}(x; \theta, 0.25)$. The posterior is proportional to $\exp[-\frac{1}{2}\{\theta^2 + 4(x - \theta)^2\}]$, and $\theta^2 + 4(x - \theta)^2 = 5(\theta - 0.8x)^2 + 0.8x^2$. So $p(\theta | x) = \mathcal{N}(\theta; 0.8x, 0.2)$: **precisions add** ($1 + 4 = 5$), and the mean is the precision-weighted average of the prior mean (0) and the observation (x). Step 1 gives the same answer with $\mathbf{A} = 1.25$, $\mathbf{C} = \mathbf{B} = 1$.

7.7 Small implementation and checks

The lab is `labs/ch07-gp/gp.py`. The function `gp_posterior` implements step 6; `sample_prior` draws functions as in figure 7.1; `weight_space_vs_function_space` compares the two derivations of step 3. The tests in `test_gp.py` check:

- the Gram matrix is symmetric with eigenvalues ≥ 0 up to round-off;
- the Cholesky mean and variance match a naive explicit-inverse implementation;
- the log marginal likelihood matches the multivariate normal log-density computed by `slogdet`;
- uncertainty inside the data gap is more than five times larger than near data;
- far from data the posterior equals the prior (mean 0, variance σ_f^2);
- prior samples have the kernel as their covariance (20 000 draws);
- Bayesian linear regression and the GP with the induced kernel give identical means and variances;
- the marginal likelihood picks the true noise level from a grid.

7.8 Experiment and expected observations

Data: $f(x) = \sin 3x + 0.3x$ on $[-2, 2]$. Training inputs are deliberately missing from $[0.5, 1.5]$. There are $m = 30$ training points with $\sigma_n = 0.1$, and 500 held-out noisy test points. Running `python labs/ch07-gp/gp.py` (seed 0, $\sigma_f^2 = 1$ fixed) prints:

length-scale ℓ	0.1	0.3	0.5	1.0	3.0
log marginal likelihood	-8.73	3.87	8.63	-29.93	-438.00
test RMSE	0.393	0.208	0.134	0.127	0.636
95% interval coverage	0.996	0.970	0.954	0.954	0.190

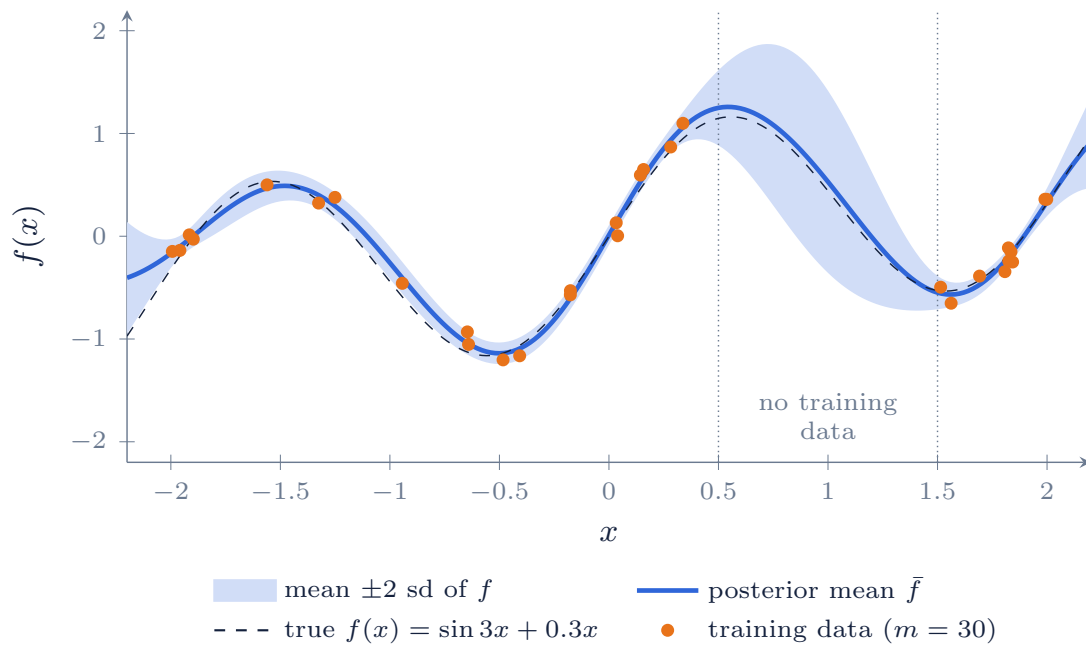


Figure 7.3: Posterior mean and ± 2 standard deviations of f from the lab (seed 0, $\ell = 0.5$). The band widens inside the data gap and narrows near observations.

What to notice:

1. **The marginal likelihood picks a sensible length-scale**, but not the one with the lowest test RMSE. They are different criteria. On seeds 1 and 2, $\ell = 0.5$ again has the highest marginal likelihood, while the RMSE ranking of $\ell = 0.5$ and $\ell = 1.0$ changes with the seed. Treat differences in the third decimal place as noise.
2. **Too short a length-scale is under-confident.** Coverage near 1.0 means the intervals are wider than they need to be.
3. **Too long a length-scale is over-confident and wrong.** Coverage of 0.19 means the model is very sure about a function far smoother than the truth. This is the more dangerous failure, because the intervals look tight.
4. **Uncertainty follows the data** (figure 7.3). At $\ell = 0.5$ the posterior standard deviation is 0.048 at $x = 0$, where there is data nearby, and 0.443 at $x = 1$, inside the gap.
5. **Hyperparameters can be learned.** A grid search of the marginal likelihood over seven length-scales and five noise levels picks $\ell = 0.5$ and $\sigma_n = 0.1$ — the true noise level.

6. **The two derivations agree.** With cubic features, weight-space and function-space predictions differ by at most 6.5×10^{-13} in the mean and 2.9×10^{-14} in the variance.

7.9 Moving pieces: diagnosing and fixing

Moving piece	Symptom when it is wrong	How to confirm	Fix
Length-scale ℓ	too short: wiggly mean, wide bands everywhere; too long: smooth mean that misses structure, tight wrong bands	coverage on held-out data; residual plots	marginal likelihood with restarts; per-dimension length-scales
Signal variance σ_f^2	bands far from data are the wrong width	prior samples vs. data range	centre and scale $\mathbf{y}$; learn σ_f
Noise σ_n^2	too small: interpolates noise, Cholesky failures; too large: over-smooth	learned σ_n vs. known measurement noise	learn it; lower-bound it; add jitter
Kernel family	systematic residual patterns (periodicity, trends)	plot residuals against inputs	sum/product kernels; mean functions (step 7)
Mean function	predictions decay to 0 far from data	look far outside the data	centre targets; constant or linear mean
Non-Gaussian noise	outliers distort mean and bands	residual histogram; coverage by region	Student- t likelihood with approximate inference (chapter 9)
Hyperparameter optimisation	different restarts give different answers	compare local optima's marginal likelihood and held-out error	several restarts; priors on hyperparameters
Scale	memory errors beyond $\sim 10^4$ points	m^2 memory	sparse / inducing-point GPs; random features (chapter 4)
Input representation	poor performance on images or text	nearest neighbours under the kernel	GP on learned features (deep kernel learning)

First things to try: (1) centre and scale the targets; (2) initialise ℓ at the median pairwise distance and σ_n at a tenth of the target standard deviation; (3) optimise the marginal likelihood from several starts; (4) check coverage on held-out data, not just RMSE.

7.10 Application, limitations and failure cases

Application (educational case studies).

- **Image-derived regression with uncertainty.** Use a GP over a small set of fixed image features (for example the PCA codes of chapter 2) to predict a continuous label, and report calibration on a group-aware split (chapter 5). In medical imaging, a confident wrong prediction costs more than an honest “unsure”.
- **Bayesian optimisation.** Training a large vision model is expensive, so its validation error as a function of the hyperparameters is an expensive black box. Fit a GP to the runs so far and choose the next hyperparameters by an **acquisition function** that balances exploitation (a low predicted error) and exploration (high uncertainty) — for example the lower confidence bound $\bar{f}(\mathbf{x}) - \beta\sigma(\mathbf{x})$. This is the GP’s most common use in deep learning practice.

Limitations and failures:

- **Cost.** Exact inference is $O(m^3)$ time and $O(m^2)$ memory.
- **The kernel is the model.** An RBF kernel assumes smooth, stationary functions. Raw pixels make Euclidean distance a weak similarity.
- **Calibration holds only under the assumptions.** Coverage measured on one distribution says nothing about shifted data.
- **Multiple optima** in the marginal likelihood.
- **Gaussian noise is assumed.** Heavy-tailed outliers distort both mean and variance.

7.11 Bridge to a research paper

Rasmussen and Williams (2006), *Gaussian Processes for Machine Learning*, chapter 2 and algorithm 2.1 — the standard reference for this chapter, which derives that algorithm. Read it with the paper-analysis sheet in **research/**:

- **Inputs:** $(\mathbf{X}, \mathbf{y})$ and a kernel family. **Outputs:** predictive mean and variance.
- **Learned components:** hyperparameters $\boldsymbol{\theta}$ only. The weights $\boldsymbol{\alpha}$ are computed, not trained.
- **Objective:** the log marginal likelihood (their section 5.4).
- **Central claim to test:** the marginal likelihood balances fit and complexity automatically.
- **Affordable experiment:** sweep ℓ and σ_n on a grid. Plot the data-fit and complexity terms separately. Then check whether the maximiser also gives good held-out coverage when the true noise is heavy-tailed rather than Gaussian.

7.12 Memory map

Summary. A GP is a prior over functions defined by a kernel: any finite set of function values is jointly Gaussian. Conditioning a partitioned Gaussian — remove the part correlated with the observations, and the rest is independent — gives the posterior in closed form: the mean is kernel ridge, the covariance is the prior minus what the data explain. The same answer follows from Bayesian linear regression in weight space, which is why GPs extend to infinite feature spaces. The marginal likelihood trades data fit against complexity and sets the hyperparameters. Compute with Cholesky; check coverage, not just error; and remember the $O(m^3)$ cost and the Gaussian assumptions.

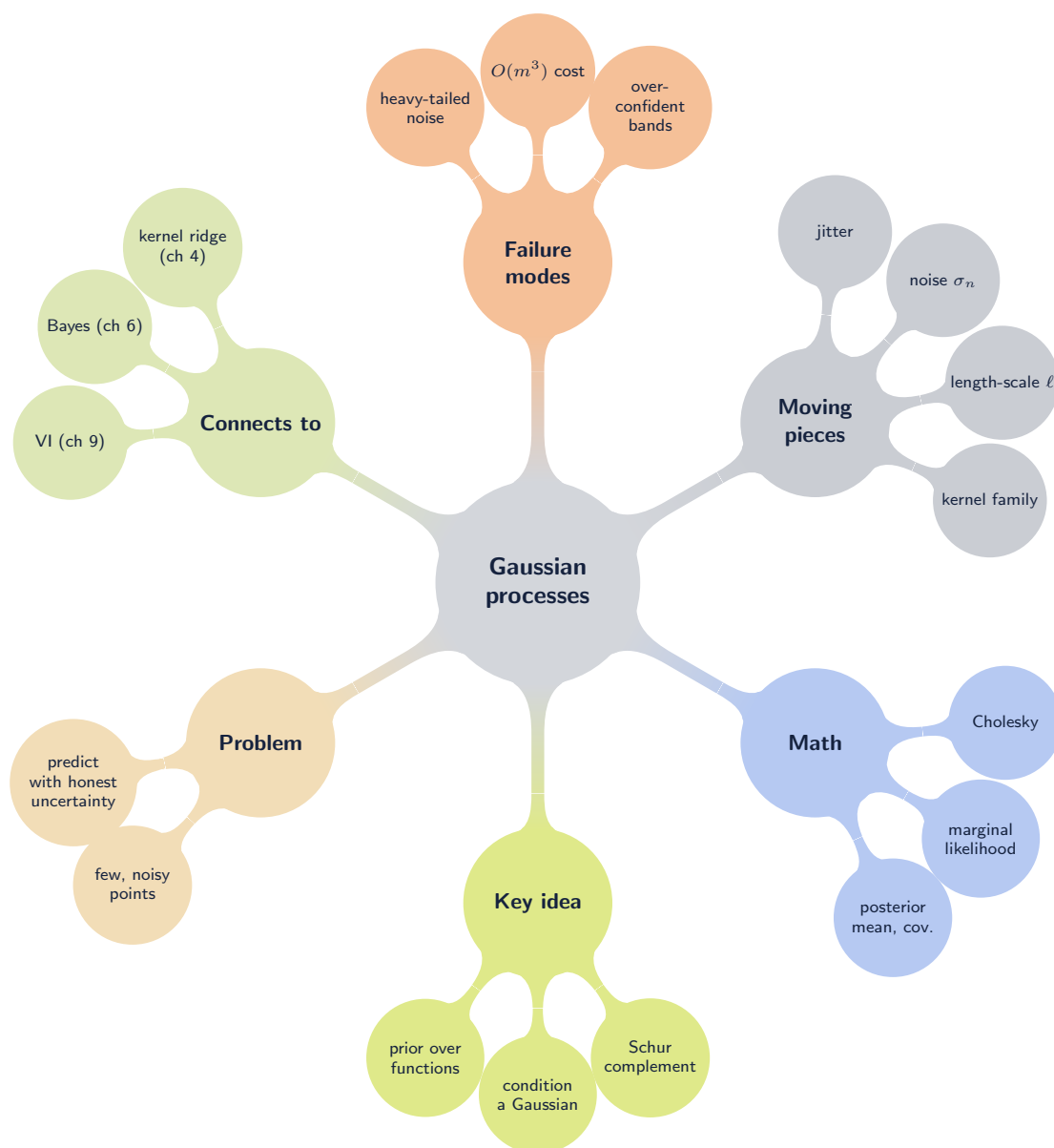


Figure 7.4: Memory map of chapter 7.

7.13 Exercises

1. *Conceptual.* Why does conditioning on $\mathbf{y}$ *reduce* the posterior variance whatever value $\mathbf{y}$ takes? Which property of step 1 is responsible?
2. *Conceptual.* Why may we omit a constant from MAP optimisation but not from the log marginal likelihood when comparing kernels?
3. *Derivation.* Derive step 1 a second way, by completing the square in the joint density, and identify the precision-matrix form of the conditional.
4. *Derivation.* Show that the GP posterior mean equals the kernel ridge regression predictor of chapter 4 with $\lambda = \sigma_n^2$. What does the GP add?
5. *Derivation.* Prove the weight-space/function-space equivalence of step 3 for the mean.
6. *Implementation.* Add gradient-based optimisation of $\log \ell$ and $\log \sigma_n$ using step 5's gradient. Check it with finite differences.
7. *Implementation.* Implement a one-dimensional Bayesian optimisation loop with the lower confidence bound and use it to minimise $\sin 3x + 0.3x$ on $[-2, 2]$ in ten evaluations.
8. *Interpretation.* Repeat the experiment with $m \in \{10, 30, 100\}$. How do the marginal-likelihood optimum and the coverage change?
9. *Failure analysis.* Replace 10% of the training noise with draws from a Student- t distribution with 2 degrees of freedom. What happens to coverage, and why?
10. *Open question.* Propose a kernel over small images that respects translation. How would you test whether it improves calibration and not just RMSE?

7.13.1 Answer discussions

1. The conditional covariance $\mathbf{B} - \mathbf{C}^\top \mathbf{A}^{-1} \mathbf{C}$ does not depend on the observed value, and the subtracted term is positive semidefinite.
2. The constant is the same for every parameter value of one model (so irrelevant to MAP) but the normalising terms, including $\log \det \mathbf{K}_y$, differ between kernels.
3. The joint precision matrix $\mathbf{\Lambda}$ gives $p(\mathbf{b} \mid \mathbf{a}) = \mathcal{N}(\mathbf{b}; \boldsymbol{\mu}_b - \mathbf{\Lambda}_{bb}^{-1} \mathbf{\Lambda}_{ba}(\mathbf{a} - \boldsymbol{\mu}_a), \mathbf{\Lambda}_{bb}^{-1})$; block inversion shows $\mathbf{\Lambda}_{bb}^{-1}$ equals the Schur complement.
4. Both are $\mathbf{k}_*^\top (\mathbf{K} + \sigma_n^2 \mathbf{I})^{-1} \mathbf{y}$; the GP adds the posterior variance.
5. $\phi_*^\top \mathbf{m} = \sigma_n^{-2} \phi_*^\top \mathbf{S} \Phi^\top \mathbf{y}$; the push-through identity turns $\mathbf{S} \Phi^\top$ into $\sigma_w^2 \Phi^\top (\sigma_w^2 \Phi \Phi^\top + \sigma_n^2 \mathbf{I})^{-1} \sigma_n^2$.
6. Agreement to 10^{-6} relative error in float64 with central differences.
7. The acquisition first explores the edges and the gap, then concentrates near the minimum.
8. With more data the optimum length-scale stabilises and coverage approaches 0.95; with 10 points, multiple optima appear.
9. Coverage drops near the outliers and the noise estimate inflates, widening bands elsewhere; the Gaussian likelihood assumption failed.
10. A kernel on translation-invariant features (e.g. pooled filter responses), or a kernel averaged over shifts. Compare coverage and NLL, not only RMSE, on shifted test images.

7.14 Self-check and common mistakes

- ☐ I can write the joint Gaussian of $(\mathbf{y}, \mathbf{f}_*)$ and condition it from memory.
- ☐ I know which variance to report: latent $\mathbf{f}_*$ or noisy $\mathbf{y}_*$.
- ☐ I can derive the GP from Bayesian linear regression.
- ☐ I compute with Cholesky solves, never with `inv`.

□ I can explain why the marginal likelihood penalises complexity.

Common mistakes: forgetting σ_n^2 on the diagonal, or adding it to $\mathbf{K}_{**}$ when predicting $\mathbf{f}_*$; not centring $\mathbf{y}$ under a zero-mean prior; reading a high marginal likelihood as proof of good generalisation; reporting intervals without checking coverage; optimising hyperparameters from a single start.

References

- C. E. Rasmussen and C. K. I. Williams. *Gaussian Processes for Machine Learning*. MIT Press, 2006. Chapters 2 and 5.
- C. K. I. Williams and C. E. Rasmussen. Gaussian processes for regression. *NeurIPS*, 1996.
- R. M. Neal. *Bayesian Learning for Neural Networks*. Springer, 1996.
- G. Matheron. Principles of geostatistics. *Economic Geology*, 58(8):1246–1266, 1963.
- A. O’Hagan. Curve fitting and optimal design for prediction. *JRSS B*, 40(1):1–42, 1978.
- E. Snelson and Z. Ghahramani. Sparse Gaussian processes using pseudo-inputs. *NeurIPS*, 2006.
- M. Titsias. Variational learning of inducing variables in sparse Gaussian processes. *AISTATS*, 2009.
- J. R. Gardner, G. Pleiss, D. Bindel, K. Q. Weinberger and A. G. Wilson. GPyTorch: blackbox matrix-matrix Gaussian process inference with GPU acceleration. *NeurIPS*, 2018.
- D. R. Jones, M. Schonlau and W. J. Welch. Efficient global optimization of expensive black-box functions. *Journal of Global Optimization*, 13:455–492, 1998.
- J. Snoek, H. Larochelle and R. P. Adams. Practical Bayesian optimization of machine learning algorithms. *NeurIPS*, 2012.
- K. P. Murphy. *Probabilistic Machine Learning: An Introduction*. MIT Press, 2022, section 3.2 (Gaussian conditioning) and chapter 17 (GPs).
- I. Goodfellow, Y. Bengio and A. Courville. *Deep Learning*. MIT Press, 2016, section 3.9.3 (the Gaussian distribution) and the notation page.

Chapter 8

Latent variables and EM

Prerequisites: chapters 6–7 (likelihood, Bayes’ rule, Gaussians); matrix calculus from chapter 1.

Learning outcomes: after this chapter you can

1. write a model with hidden (latent) variables as a generative story and as a likelihood, and explain why its log-likelihood is hard to maximise directly;
2. derive the E-step (Bayes’ rule) and the M-step (weighted maximum likelihood) for a Gaussian mixture, including the covariance update;
3. prove that EM never decreases the likelihood, using the decomposition into a lower bound and a KL divergence — the same bound that becomes the ELBO of chapter 9;
4. explain k-means as the hard limit of EM, and soft assignments as uncertainty;
5. diagnose and fix EM’s failure modes: local optima, collapsing components, label switching and the choice of K .

8.1 A concrete problem

A crop-health team has thousands of unlabelled field photographs. Each image is cut into patches, and each patch is described by a small feature vector (two colour statistics in our lab; an embedding from a pretrained network in practice). The team suspects the patches come from a few **sources** — soil, healthy leaf, lesion — but nobody has labelled them.

Questions:

1. Can we recover the sources and their proportions from the unlabelled features?
2. For a given patch, how sure are we which source produced it?
3. How many sources are there?

A single Gaussian cannot describe three groups. If we *knew* each patch’s source, we would fit one Gaussian per source by chapter 6’s maximum likelihood — just averages and covariances. The difficulty is entirely that the source is **hidden**. This chapter turns that difficulty into an algorithm, **expectation–maximisation (EM)**: guess the hidden variables softly, refit as if the guesses were data, repeat. The same idea — a hidden variable, a lower bound, alternating improvement — reappears in variational inference (chapter 9), VAEs (chapter 10), clustering-based self-supervised learning (chapter 15) and diffusion models (chapter 22).

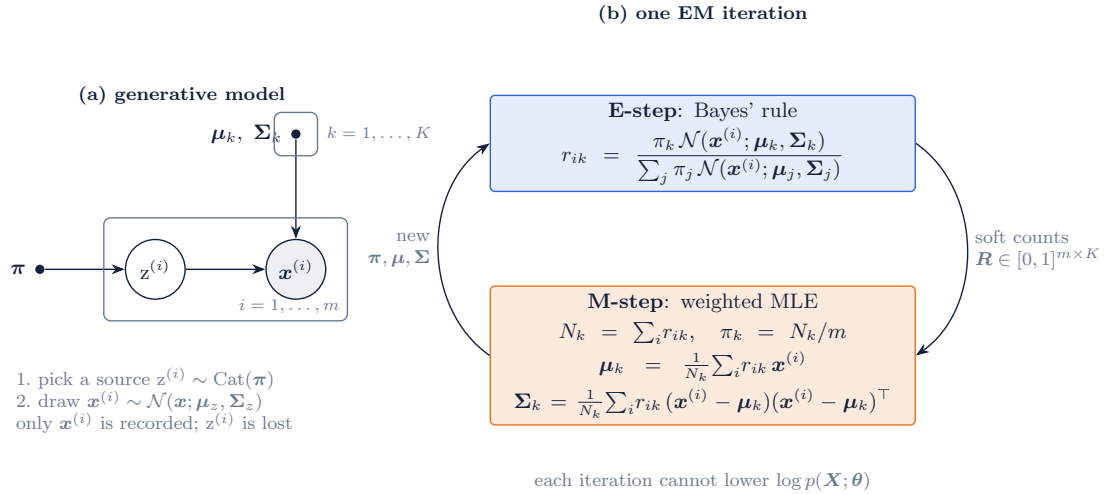


Figure 8.1: (a) The Gaussian mixture as a generative story in plate notation: shaded nodes are observed, plain nodes are hidden, and a plate repeats its contents. (b) One EM iteration: the E-step computes soft memberships by Bayes' rule; the M-step refits each component by weighted maximum likelihood.

8.2 How we got here

Mixtures before algorithms (1894). Karl Pearson fitted a mixture of two normal distributions to the forehead-to-body-length ratios of 1,000 Naples crabs, suspecting two subspecies. With no computer he used the method of moments and solved a ninth-degree polynomial by hand. The question — which hidden group did each observation come from? — was already the right one.

Special cases of EM (1950s–1970). Maximum-likelihood fits for missing data and mixtures were found case by case: for genetics (Ceppellini, Siniscalco and Smith, 1955), for mixtures (Hasselblad, 1966; Day, 1969; Wolfe, 1970), and — most influentially — for hidden Markov models: the Baum–Welch algorithm (Baum and colleagues, 1970) is EM for sequences and later powered speech recognition.

The general algorithm (1977). Arthur Dempster, Nan Laird and Donald Rubin named and unified the method — *maximum likelihood from incomplete data via the EM algorithm* — and proved that each iteration cannot lower the likelihood. Their proof contained an error about convergence to a maximum; C. F. Jeff Wu (1983) gave the correct statement: EM converges to a **stationary point**, which may be a local maximum or a saddle.

Hard clustering in parallel (1957–1982). Stuart Lloyd's algorithm for quantising signals (Bell Labs memo, 1957; published 1982) and James MacQueen's "k-means" (1967) assign each point to the nearest centre and re-average. Section 8.5 shows k-means is EM in a limit.

The bound view (1998). Radford Neal and Geoffrey Hinton recast EM as coordinate ascent on a single function of two arguments — a distribution q over the hidden variables and the parameters θ — borrowed from statistical physics ("free energy"). That view justified incremental and partial EM variants and is precisely the **evidence lower bound (ELBO)** that variational inference (chapter 9) and VAEs (chapter 10) optimise with gradients.

Mixtures in computer vision. Stauffer and Grimson (1999) modelled each pixel’s colour over time as a Gaussian mixture to separate moving objects from background — this chapter’s paper bridge. Sivic and Zisserman (2003) clustered local descriptors with k-means into “visual words”; Perronnin and Dance (2007) replaced the hard clustering with a GMM and encoded images by gradients of its log-likelihood (Fisher vectors), the state of the art for image classification until 2012. Modern descendants keep the soft-assignment idea: DeepCluster (2018) and SwAV (2020) cluster network features to create training targets (chapter 15), and Slot Attention (2020) runs a learnable, EM-like soft clustering inside a network to discover objects.

8.3 Assumptions and notation

Symbol	Meaning
$\mathbf{X} \in \mathbb{R}^{m \times n}$	data matrix, row i is $\mathbf{x}^{(i)\top}$
K	number of mixture components
$z^{(i)} \in \{1, \dots, K\}$	hidden component (source) of example i ; value $z^{(i)}$
π_k	mixing weight, $P(z = k)$; $\pi_k \geq 0$, $\sum_k \pi_k = 1$
$\boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k$	mean and covariance of component k
$\boldsymbol{\theta}$	all parameters $\{\pi_k, \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k\}_{k=1}^K$
$r_{ik}, \mathbf{R} \in [0, 1]^{m \times K}$	responsibility $P(z^{(i)} = k \mid \mathbf{x}^{(i)}; \boldsymbol{\theta})$; rows sum to one
$N_k = \sum_i r_{ik}$	effective number of examples assigned to component k
$q_i(z)$	any distribution over the hidden value of example i
$\mathcal{L}(q, \boldsymbol{\theta})$	the lower bound on the log-likelihood (the ELBO)

Assumptions:

1. **Independent examples.** Given $\boldsymbol{\theta}$, the pairs $(\mathbf{x}^{(i)}, z^{(i)})$ are independent.
2. **A known number of components K** (section 8.5, step 9, discusses choosing it).
3. **Gaussian components.** Each source produces a Gaussian cloud. Real clusters need not be Gaussian; a non-Gaussian cluster is then split across several components.
4. **Latent means unobserved, not unknowable.** EM assumes the hidden values are *missing at random* — whether a value is hidden does not depend on what it is.

8.4 Mathematical formulation

Generative story. For each example, pick a source, then draw a point from it:

$$P(z^{(i)} = k) = \pi_k, \quad p(\mathbf{x}^{(i)} \mid z^{(i)} = k) = \mathcal{N}(\mathbf{x}^{(i)}; \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k).$$

Marginal likelihood of the observed data. The hidden value is summed out:

$$p(\mathbf{x}; \boldsymbol{\theta}) = \sum_{k=1}^K \pi_k \mathcal{N}(\mathbf{x}; \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k), \quad \ell(\boldsymbol{\theta}) = \sum_{i=1}^m \log \sum_{k=1}^K \pi_k \mathcal{N}(\mathbf{x}^{(i)}; \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k).$$

Goal: maximise $\ell(\boldsymbol{\theta})$ over $\boldsymbol{\theta}$ with $\boldsymbol{\Sigma}_k \succ 0$ and $\boldsymbol{\pi}$ on the simplex. The obstacle is visible in the formula: a **log of a sum**. With known sources the log would act on a single Gaussian and everything would separate per component.

The EM algorithm repeats, from an initial $\boldsymbol{\theta}^{(0)}$:

$$\begin{aligned} \text{E-step: } r_{ik} &= \frac{\pi_k \mathcal{N}(\mathbf{x}^{(i)}; \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k)}{\sum_j \pi_j \mathcal{N}(\mathbf{x}^{(i)}; \boldsymbol{\mu}_j, \boldsymbol{\Sigma}_j)}, \\ \text{M-step: } N_k &= \sum_i r_{ik}, \quad \pi_k = \frac{N_k}{m}, \quad \boldsymbol{\mu}_k = \frac{1}{N_k} \sum_i r_{ik} \mathbf{x}^{(i)}, \\ \boldsymbol{\Sigma}_k &= \frac{1}{N_k} \sum_i r_{ik} (\mathbf{x}^{(i)} - \boldsymbol{\mu}_k)(\mathbf{x}^{(i)} - \boldsymbol{\mu}_k)^\top, \end{aligned}$$

until ℓ stops increasing. Figure 8.2 shows the iterations on the lab's data.

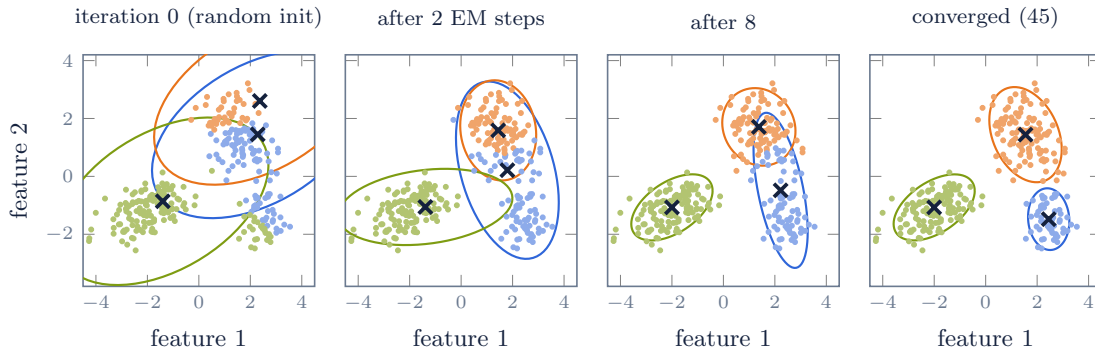


Figure 8.2: EM on the lab's patch features from a poor random start (seed 4; section 8.8 runs it). Points are coloured by their most responsible component, ellipses are two-standard-deviation contours, crosses are means. Two components begin in the same cluster; the E-step lets them share it softly, and over iterations one migrates to the unclaimed cluster.

The general form, for any latent-variable model $p(\mathbf{x}, \mathbf{z}; \boldsymbol{\theta})$, is

$$\text{E: } q_i(\mathbf{z}) = p(\mathbf{z} \mid \mathbf{x}^{(i)}; \boldsymbol{\theta}^{\text{old}}), \quad \text{M: } \boldsymbol{\theta}^{\text{new}} = \arg \max_{\boldsymbol{\theta}} \sum_i \mathbb{E}_{q_i} [\log p(\mathbf{x}^{(i)}, \mathbf{z}; \boldsymbol{\theta})].$$

8.5 Derivation, with reasons for each step

Step 1 — why direct maximisation is awkward, and what the gradient already tells us. Differentiate ℓ with respect to $\boldsymbol{\mu}_k$. Only one term in each inner sum depends on it:

$$\nabla_{\boldsymbol{\mu}_k} \ell = \sum_i \frac{\pi_k \mathcal{N}(\mathbf{x}^{(i)}; \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k)}{\sum_j \pi_j \mathcal{N}(\mathbf{x}^{(i)}; \boldsymbol{\mu}_j, \boldsymbol{\Sigma}_j)} \boldsymbol{\Sigma}_k^{-1} (\mathbf{x}^{(i)} - \boldsymbol{\mu}_k) = \boldsymbol{\Sigma}_k^{-1} \sum_i r_{ik} (\mathbf{x}^{(i)} - \boldsymbol{\mu}_k).$$

Reason: $\nabla \log f = \nabla f / f$, and $\nabla_{\boldsymbol{\mu}} \mathcal{N}(\mathbf{x}; \boldsymbol{\mu}, \boldsymbol{\Sigma}) = \mathcal{N}(\mathbf{x}; \boldsymbol{\mu}, \boldsymbol{\Sigma}) \boldsymbol{\Sigma}^{-1} (\mathbf{x} - \boldsymbol{\mu})$. The fraction is the responsibility. Setting the gradient to zero gives $\boldsymbol{\mu}_k = \sum_i r_{ik} \mathbf{x}^{(i)} / N_k$ — but r_{ik} itself depends on $\boldsymbol{\mu}_k$. There is no closed form, only a **fixed-point equation**. EM is the natural

way to iterate it: freeze r_{ik} , solve, recompute r_{ik} , repeat. The rest of the derivation shows that this iteration is not a heuristic but provably improves ℓ .

Step 2 — the E-step is Bayes' rule. With the parameters fixed, the posterior over the hidden source is

$$P(z^{(i)} = k \mid \mathbf{x}^{(i)}; \boldsymbol{\theta}) = \frac{P(z = k) p(\mathbf{x}^{(i)} \mid z = k)}{p(\mathbf{x}^{(i)})} = \frac{\pi_k \mathcal{N}(\mathbf{x}^{(i)}; \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k)}{\sum_j \pi_j \mathcal{N}(\mathbf{x}^{(i)}; \boldsymbol{\mu}_j, \boldsymbol{\Sigma}_j)} = r_{ik}.$$

Reason: chapter 6's prior $\times$ likelihood / evidence, with prior π_k . Each row of $\mathbf{R}$ sums to one. A responsibility of 0.6 is not “60% of the patch is lesion”; it is 60% belief that the whole patch came from the lesion component.

Computation: evaluate $a_{ik} = \log \pi_k + \log \mathcal{N}(\mathbf{x}^{(i)}; \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k)$ and normalise with log-sum-exp (chapter 1): $r_{ik} = \exp(a_{ik} - \text{logsumexp}_j a_{ij})$. The row log-sum-exp is also example i 's contribution to ℓ , so the E-step returns the log-likelihood for free. Far from every component the raw densities underflow to zero and 0/0 appears; in log space nothing does.

Step 3 — if we knew the sources: the complete-data log-likelihood. With one-hot indicators $z_{ik} = \mathbb{I}[z^{(i)} = k]$,

$$\log p(\mathbf{X}, \mathbf{Z}; \boldsymbol{\theta}) = \sum_i \sum_k z_{ik} \left[\log \pi_k + \log \mathcal{N}(\mathbf{x}^{(i)}; \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k) \right].$$

Reason: the product over components becomes a sum of logs because exactly one indicator per example is 1. The log now acts on single Gaussians, and the problem separates into K ordinary Gaussian fits plus a categorical fit.

We do not know z_{ik} , so we use its expectation under the E-step posterior, $\mathbb{E}[z_{ik}] = r_{ik}$:

$$Q(\boldsymbol{\theta}; \boldsymbol{\theta}^{\text{old}}) = \sum_i \sum_k r_{ik} \left[\log \pi_k - \frac{1}{2} \log \det(2\pi \boldsymbol{\Sigma}_k) - \frac{1}{2} (\mathbf{x}^{(i)} - \boldsymbol{\mu}_k)^\top \boldsymbol{\Sigma}_k^{-1} (\mathbf{x}^{(i)} - \boldsymbol{\mu}_k) \right].$$

Step 4 — the M-step for the mixing weights. Maximise $\sum_k N_k \log \pi_k$ subject to $\sum_k \pi_k = 1$ with a Lagrange multiplier:

$$\frac{\partial}{\partial \pi_k} \left[\sum_j N_j \log \pi_j - \lambda (\sum_j \pi_j - 1) \right] = \frac{N_k}{\pi_k} - \lambda = 0 \Rightarrow \pi_k = \frac{N_k}{\lambda}, \quad \sum_k \pi_k = 1 \Rightarrow \lambda = \sum_k N_k = m.$$

Reason: rows of $\mathbf{R}$ sum to one, so $\sum_k N_k = m$. The answer is the fraction of the data each component is responsible for; positivity holds automatically.

Step 5 — the M-step for the means. $\nabla_{\boldsymbol{\mu}_k} Q = \boldsymbol{\Sigma}_k^{-1} \sum_i r_{ik} (\mathbf{x}^{(i)} - \boldsymbol{\mu}_k) = \mathbf{0}$ gives the weighted average $\boldsymbol{\mu}_k = \sum_i r_{ik} \mathbf{x}^{(i)} / N_k$. *Reason:* the same equation as step 1, but now r_{ik} is frozen, so it is a closed form.

Step 6 — the M-step for the covariances. Write $\boldsymbol{\Lambda}_k = \boldsymbol{\Sigma}_k^{-1}$ (the precision) and $\mathbf{S}_k = \frac{1}{N_k} \sum_i r_{ik} (\mathbf{x}^{(i)} - \boldsymbol{\mu}_k)(\mathbf{x}^{(i)} - \boldsymbol{\mu}_k)^\top$. The terms of Q that depend on $\boldsymbol{\Lambda}_k$ are

$$\frac{N_k}{2} \left[\log \det \boldsymbol{\Lambda}_k - \text{Tr}(\boldsymbol{\Lambda}_k \mathbf{S}_k) \right].$$

Reason: $\log \det \Sigma^{-1} = -\log \det \Sigma$, and a quadratic form is a trace, $\mathbf{d}^\top \Lambda \mathbf{d} = \text{Tr}(\Lambda \mathbf{d} \mathbf{d}^\top)$, so the weighted sum of quadratic forms is $N_k \text{Tr}(\Lambda_k \mathbf{S}_k)$. Using $\nabla_\Lambda \log \det \Lambda = \Lambda^{-1}$ and $\nabla_\Lambda \text{Tr}(\Lambda \mathbf{S}) = \mathbf{S}$ (appendix A), the gradient is $\frac{N_k}{2}(\Lambda_k^{-1} - \mathbf{S}_k)$, so

$$\Sigma_k = \mathbf{S}_k = \frac{1}{N_k} \sum_i r_{ik} (\mathbf{x}^{(i)} - \boldsymbol{\mu}_k)(\mathbf{x}^{(i)} - \boldsymbol{\mu}_k)^\top,$$

with the *new* mean. The objective is concave in Λ_k (log det is concave, the trace is linear), so this stationary point is the maximum. The divisor is N_k , not $N_k - 1$: this is a maximum-likelihood estimate, not an unbiased one.

Step 7 — why each iteration cannot lower the likelihood. This is the central argument. For any distribution q_i over example i 's hidden value with $q_i(z) > 0$ wherever $p(\mathbf{x}^{(i)}, z) > 0$,

$$\log p(\mathbf{x}^{(i)}; \boldsymbol{\theta}) = \log \sum_z q_i(z) \frac{p(\mathbf{x}^{(i)}, z; \boldsymbol{\theta})}{q_i(z)} \geq \sum_z q_i(z) \log \frac{p(\mathbf{x}^{(i)}, z; \boldsymbol{\theta})}{q_i(z)}.$$

Reason: multiply and divide by $q_i(z)$; then log is concave, so the log of an average is at least the average of the logs (Jensen's inequality). Summing over examples defines the **lower bound**

$$\mathcal{L}(q, \boldsymbol{\theta}) = \sum_i \mathbb{E}_{q_i} [\log p(\mathbf{x}^{(i)}, z; \boldsymbol{\theta})] + \sum_i H(q_i) \leq \ell(\boldsymbol{\theta}).$$

How loose is it? Write $p(\mathbf{x}, z) = p(z | \mathbf{x}) p(\mathbf{x})$ inside the bound:

$$\ell(\boldsymbol{\theta}) = \mathcal{L}(q, \boldsymbol{\theta}) + \sum_i D_{\text{KL}}(q_i(z) \| p(z | \mathbf{x}^{(i)}; \boldsymbol{\theta})).$$

Reason: $\sum_z q \log \frac{p(z|\mathbf{x})p(\mathbf{x})}{q} = \log p(\mathbf{x}) - \sum_z q \log \frac{q}{p(z|\mathbf{x})}$ because $\sum_z q = 1$. The gap is exactly a KL divergence, which is non-negative and zero only when q_i equals the posterior. This identity is the whole of EM:

- **E-step** = maximise $\mathcal{L}$ over q with $\boldsymbol{\theta}$ fixed. Since $\ell(\boldsymbol{\theta})$ does not depend on q , maximising $\mathcal{L}$ means making the KL zero: $q_i = p(\cdot | \mathbf{x}^{(i)})$. The bound now **touches** the log-likelihood at $\boldsymbol{\theta}^{\text{old}}$.
- **M-step** = maximise $\mathcal{L}$ over $\boldsymbol{\theta}$ with q fixed. The entropy term does not depend on $\boldsymbol{\theta}$, so this is maximising Q (steps 4–6).

Chain them:

$$\ell(\boldsymbol{\theta}^{\text{new}}) \geq \mathcal{L}(q^{\text{old}}, \boldsymbol{\theta}^{\text{new}}) \geq \mathcal{L}(q^{\text{old}}, \boldsymbol{\theta}^{\text{old}}) = \ell(\boldsymbol{\theta}^{\text{old}}).$$

The first inequality is the bound, the second is the M-step, the equality is the tight E-step. Figure 8.3 draws this for one parameter.

What the argument does **not** say: that EM reaches the global maximum (it reaches a stationary point, figure 8.4a), or that it converges fast (near a maximum it converges linearly, with a rate set by the fraction of information that is missing — slow when components overlap heavily).

Step 8 — k-means is EM with hard, isotropic components. Fix $\pi_k = 1/K$ and $\Sigma_k = \varepsilon \mathbf{I}$. Then

$$r_{ik} = \frac{\exp(-\|\mathbf{x}^{(i)} - \boldsymbol{\mu}_k\|^2/2\varepsilon)}{\sum_j \exp(-\|\mathbf{x}^{(i)} - \boldsymbol{\mu}_j\|^2/2\varepsilon)} \xrightarrow{\varepsilon \rightarrow 0} \mathbb{I}[k = \arg \min_j \|\mathbf{x}^{(i)} - \boldsymbol{\mu}_j\|].$$

Reason: a softmax with temperature ε becomes an argmax as $\varepsilon \rightarrow 0$. The M-step mean is then the average of the assigned points: Lloyd’s algorithm. k-means therefore inherits EM’s monotonicity (of the within-cluster squared distance) and its local optima, but loses everything the covariances and soft responsibilities carry — elongated clusters, unequal sizes, and uncertainty at the boundaries. The same temperature knob reappears as the softmax temperature in attention (chapter 14) and contrastive losses (chapter 15).

Step 9 — the likelihood is unbounded: degeneracy, and the MAP fix. Place one component exactly on a data point, $\boldsymbol{\mu}_k = \mathbf{x}^{(1)}$, with $\Sigma_k = \sigma^2 \mathbf{I}$. Its density at that point is $(2\pi\sigma^2)^{-n/2}$, so example 1’s term in ℓ contains $\log(\pi_k(2\pi\sigma^2)^{-n/2} + \dots) \geq \log \pi_k - n \log \sigma - \frac{n}{2} \log 2\pi$, while every other example is still explained by the remaining components. As $\sigma \rightarrow 0$, $\ell \rightarrow +\infty$ (figure 8.4b). **The maximum-likelihood estimate of an unconstrained Gaussian mixture does not exist**; what EM finds is a good *local* maximum, and a run that wanders into a collapse can report an ever-growing likelihood for a useless model.

Three standard fixes, each a statement about the model:

- **Covariance floor** $\Sigma_k \leftarrow \mathbf{S}_k + \epsilon \mathbf{I}$. This is exactly the MAP M-step under a weak inverse-Wishart prior on Σ_k (exercise 5) — chapter 6’s “penalty = prior”.
- **Re-seed small components:** if N_k falls below a threshold (a few examples), move the component elsewhere. This addresses parking on a single point, which a floor alone does not (section 8.8).
- **Constrain the covariance type:** shared (“tied”), diagonal or spherical covariances have fewer parameters, need fewer examples per component, and degenerate less.

Step 10 — label switching and identifiability. Permuting the component indices gives the same $p(\mathbf{x}; \boldsymbol{\theta})$, so every maximum has $K!$ copies. Within one run this is harmless; across runs or seeds, “component 2” means nothing until you match components (by Hungarian matching of means, or by agreement with known labels after fitting). Never average parameters from different runs without matching them first.

Step 11 — choosing K . The training likelihood always rises with K (more components can only fit better), so it cannot choose K . Options: held-out log-likelihood; the **Bayesian information criterion**

$$\text{BIC}(K) = -2\ell(\hat{\boldsymbol{\theta}}_K) + p_K \log m, \quad p_K = (K-1) + Kn + K \frac{n(n+1)}{2},$$

which approximates $-2\log$ of the marginal likelihood (chapter 6, step 8) by a Laplace argument; or a prior over the number of components (Dirichlet-process mixtures). And remember that the number of *density* clusters need not equal the number of *semantic* categories.

Step 12 — EM beyond mixtures. The bound argument of step 7 never used Gaussians. It applies whenever the complete-data problem is easy:

- **Missing values:** hidden entries of $\mathbf{x}$ are the latent variables; the E-step computes their conditional Gaussian (chapter 7) moments.

- **Probabilistic PCA and factor analysis** (Tipping and Bishop, 1999): $\mathbf{x} = \mathbf{W}\mathbf{h} + \boldsymbol{\mu} + \text{noise}$ with a hidden Gaussian code $\mathbf{h}$. As the noise goes to zero, EM recovers PCA's subspace (chapter 2) without forming the covariance matrix.
- **Hidden Markov models**: the E-step is the forward-backward algorithm.
- **When the E-step is intractable** — the posterior has no closed form, as in deep latent-variable models — replace the exact posterior with an approximation q from a tractable family. The bound of step 7 still holds, but is no longer tight. That is **variational inference** (chapter 9); a neural network that outputs q is a **VAE** (chapter 10).

8.6 Numerical worked example

E-step. Two one-dimensional components with equal weights, unit variances and means 0 and 2. At $x = 0$ the unnormalised densities are proportional to e^0 and $e^{-(0-2)^2/2} = e^{-2}$, so

$$r_{.1} = \frac{1}{1 + e^{-2}} = 0.881, \quad r_{.2} = 0.119.$$

At $x = 1$ the distances are equal and $r = (0.5, 0.5)$. Hard nearest-centre assignment would call both points certain.

M-step. Two points $x = (0, 2)$ with responsibilities $(0.75, 0.25)$ for component 1 and $(0.25, 0.75)$ for component 2. Then $N_1 = 1$, $\pi_1 = 1/2$, $\mu_1 = (0.75 \cdot 0 + 0.25 \cdot 2)/1 = 0.5$, and

$$\sigma_1^2 = 0.75(0 - 0.5)^2 + 0.25(2 - 0.5)^2 = 0.1875 + 0.5625 = 0.75.$$

By symmetry $\mu_2 = 1.5$ and $\sigma_2^2 = 0.75$. Both match the lab's `worked_m_step`.

One step as bound maximisation (figure 8.3). In the lab's one-dimensional example, starting from $\mu_1^{\text{old}} = -0.2$ the log-likelihood is -235.6 . After the E-step the bound touches it there (gap 0 to machine precision). The bound's maximum is at $\mu_1^{\text{new}} = -1.195$ where the bound is -183.6 and the log-likelihood -174.0 : one M-step gained 61.6 nats, and the remaining gap of 9.6 nats is the KL divergence that the next E-step will close.

8.7 Small implementation and checks

The lab `labs/ch08-em/gmm.py` implements EM in about 60 lines of NumPy. The two steps:

```
def e_step(X, pi, mu, Sigma):
    J = log_joint(X, pi, mu, Sigma)          # m x K: log pi_k + log
    ↪ N(x_i; mu_k, Sigma_k)
    lse = logsumexp(J)                        # log p(x_i), per example
    return np.exp(J - lse[:, None]), lse.sum() # responsibilities,
    ↪ log-likelihood

def m_step(X, R, reg=1e-3, min_count=1e-8):
    Nk = R.sum(0); pi = Nk / m; mu = (R.T @ X) / Nk[:, None]
    Sigma[k] = (R[:, k, None] * D).T @ D / Nk[k] + reg * I    # D = X -
    ↪ mu[k]
    # components with Nk < min_count are re-seeded at a random data point
```

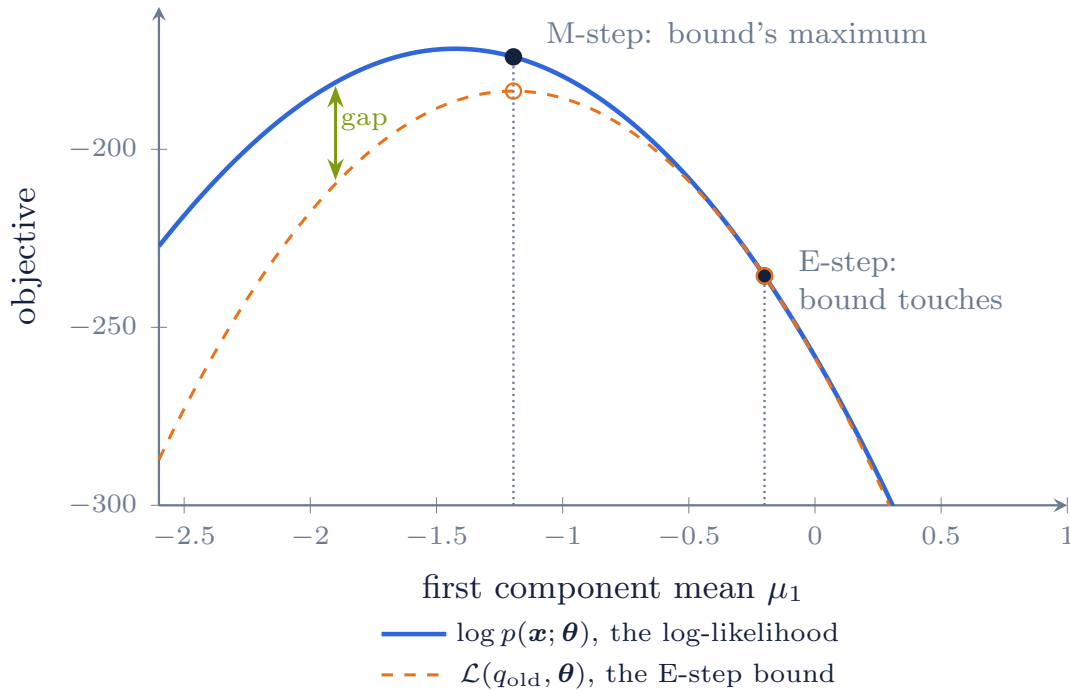


Figure 8.3: One EM step as bound maximisation, for a one-dimensional two-component mixture with only the first mean varying (lab function `bound_curve`). The E-step bound (dashed) touches the log-likelihood at the current value; the M-step moves to the bound's maximum, where the log-likelihood is at least as high.

`log_gauss` uses a Cholesky factor $\mathbf{L}$ of $\boldsymbol{\Sigma}$: the quadratic form is $\|\mathbf{L}^{-1}(\mathbf{x} - \boldsymbol{\mu})\|^2$ and $\log \det \boldsymbol{\Sigma} = 2 \sum_j \log L_{jj}$ — no inverse, no determinant, and a failed factorisation is an immediate signal that a covariance has become singular.

Checks, in `test_gmm.py`:

1. `log_gauss` agrees with the textbook formula on random inputs.
2. The worked E- and M-steps of section 8.6.
3. Responsibilities stay finite and sum to one for a point 700 units from every component.
4. The log-likelihood trace never decreases, and the fit recovers the hidden sources (up to relabelling) and their weights.
5. The bound equals the log-likelihood at the old parameters, lies below it everywhere else, and is maximised by the M-step.
6. Permuting component labels leaves the likelihood unchanged.
7. The degenerate component's log-likelihood grows as $\sigma \rightarrow 0$; EM with no floor breaks; a floor plus re-seeding recovers the good solution.
8. Different initialisations reach different optima; BIC chooses $K = 3$.
9. With all responsibilities equal to one the M-step is the ordinary sample mean and (biased) covariance.

Debugging habits that transfer to every iterative fit: log the objective every iteration and assert it never decreases (for exact EM, any decrease is a bug); log $\min_k N_k$ and the smallest covariance eigenvalue; check that responsibilities sum to one; keep everything in log space.

8.8 Experiment and expected observations

Run `python labs/ch08-em/gmm.py` (seed 0, 600 points, three hidden sources with weights (0.5, 0.35, 0.15)).

quantity	value
iterations to converge (k-means++ initialisation)	32
log-likelihood, start $\rightarrow$ end	$-2393.1 \rightarrow -1754.5$, monotone
fitted weights (sorted by mean)	(0.455, 0.380, 0.164)
agreement with hidden labels, best relabelling	0.987 (k-means: 0.975)
examples with max responsibility < 0.9	2.8%
ring data, $K = 6$, 20 random initialisations	3 distinct optima; 18 of 20 reach the best (-1322.1), worst -1377.3
BIC for $K = 1, \dots, 5$	4619.0, 4109.0, 3617.7 , 3649.0, 3679.2

What to notice:

1. **Monotone, then flat.** The log-likelihood jumps in the first few iterations and creeps afterwards. A stopping rule on relative change (10^{-8} here) matters more than the iteration cap.
2. **Soft assignments are informative.** 97% of patches are assigned with confidence above 0.9; the rest sit between clusters. In an application, those are the patches to send for labelling.
3. **Weights are not exact.** The fitted 0.455 for a true 0.5 is sampling noise plus overlap, not a bug: with 600 points, a proportion has a standard error of about 0.02.
4. **Local optima (figure 8.4a).** On six clusters arranged in a ring, two of 20 random starts converge to a solution where one component covers two clusters and another splits one. Remedy: several restarts, keep the highest likelihood — and report how many you used.
5. **Collapse.** Starting one component on a single point with variance 10^{-4} : with no floor, the first M-step produces an exactly singular covariance and EM stops. With a floor of 10^{-3} the run continues but the component stays parked on that point ($N_k = 0.70$) and the final log-likelihood is -2018.8 , far below -1754.5 , because the other two components must cover three clusters. Adding re-seeding of components with $N_k < 3$ recovers -1754.5 .
6. **BIC chooses three.** The training likelihood keeps rising with K , but BIC's penalty of $\log 600 \approx 6.4$ per parameter makes $K = 3$ the minimum.

8.9 Moving pieces: diagnosing and fixing

Moving piece	Symptom when it is wrong	How to confirm	Fix
Initialisation	different runs give different clusters and likelihoods	final ℓ across 10–20 seeds	k-means++ seeding; several restarts, keep the best ℓ (on training data)

Moving piece	Symptom when it is wrong	How to confirm	Fix
Number of components K	one cluster split in two, or two merged	BIC or held-out ℓ across K ; inspect N_k	choose K by a stated criterion; merge or split components
Covariance floor / prior	ℓ keeps growing; a covariance eigenvalue $\rightarrow 0$; Cholesky fails	log min eigenvalue per iteration	add $\epsilon \mathbf{I}$ (MAP with inverse-Wishart prior); scale features first
Small components	N_k of a few points; a component parked on outliers	log $\min_k N_k$	re-seed components with tiny N_k ; remove outliers
Covariance type	too many parameters for the data; unstable fits in high dimension	count $Kn(n+1)/2$ against m	diagonal, tied or spherical covariances; reduce n with PCA (chapter 2)
Feature scaling	one feature dominates; spherical clusters in one axis	per-feature standard deviations	standardise; whiten with PCA
Numerics	NaN responsibilities; $\ell = -\infty$	test a far-away point	log-space E-step with log-sum-exp; Cholesky
Convergence test	stops too early or never	plot the trace	log-densities relative-change tolerance plus an iteration cap; a decrease means a bug
Label identity	comparisons across runs are meaningless	means move between indices across seeds	match components (Hungarian) before comparing
Model family	clusters are curved or heavy-tailed; many components describe one cluster	residuals; compare with a non-Gaussian model	more components, heavier-tailed (Student-t) mixtures, or a learned feature space
Semantic mismatch	clusters capture lighting or background, not the target	inspect examples per cluster against labels after fitting	change the features (better embeddings), not the clustering

First things to try: (1) assert the log-likelihood never decreases; (2) run 10 seeds and compare final ℓ ; (3) log $\min_k N_k$ and the smallest covariance eigenvalue; (4) standardise the features; (5) choose K by BIC or held-out likelihood, then look at examples from each cluster.

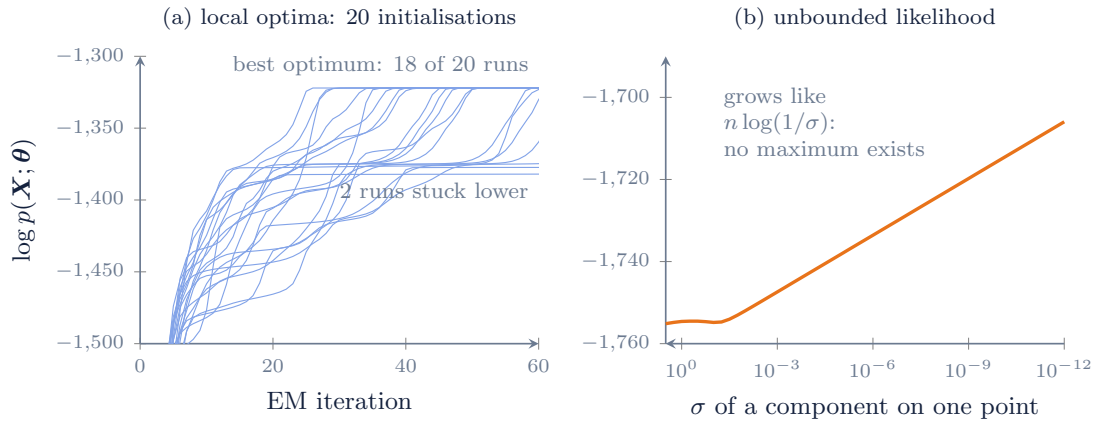


Figure 8.4: Two ways EM’s answer goes wrong. (a) Log-likelihood traces from 20 random initialisations on six clusters in a ring: every trace is monotone, but they end on different plateaus. (b) The log-likelihood with an extra component of weight $1/m$ placed on one data point, as its standard deviation σ shrinks: it grows without bound.

8.10 Application, limitations and failure cases

Application (educational case study). For the crop-health project, fit a GMM to embeddings of image patches from a pretrained network, reduced to 20–50 dimensions with PCA, with diagonal or tied covariances. Use the result in three ways: (1) **triage** — patches with low maximum responsibility, or low $p(\mathbf{x})$ under the whole mixture, are ambiguous or unusual and go first to annotators; (2) **anomaly detection** — a new field whose patches have much lower average $\log p(\mathbf{x})$ than the training fields signals a distribution shift (chapter 5); (3) **pseudo-labels** — clusters inspected and named by an expert seed a small labelled set. In the classic video setting, a per-pixel mixture separates static background from moving objects (section 8.11).

Limitations:

- Clusters found by density are not categories found by meaning. A cluster can capture lighting, camera or background — the shortcut problem of chapter 5 in unsupervised form. Validate with labels *after* fitting.
- In high dimensions full covariances need $O(n^2)$ parameters per component; distances concentrate and every point is far from every mean. Reduce dimension first.
- EM’s guarantees are local. Restarts help but do not certify a global optimum.
- The Gaussian-component assumption is strong. Heavy tails or curved clusters force extra components.

8.11 Bridge to a research paper

Stauffer and Grimson (1999), *Adaptive background mixture models for real-time tracking* (CVPR 1999, pp. 246–252).

- **Problem:** separate moving foreground from background in video with changing lighting, swaying trees and repetitive motion, in real time.
- **Mechanism:** model the colour history of each pixel as a mixture of K (3–5) Gaussians with diagonal covariances. Instead of batch EM, each new frame does an **online, hard-assignment update**: match the pixel to the first component

within 2.5 standard deviations, update that component’s weight, mean and variance with a learning rate, and replace the least probable component if nothing matches. Components with high weight and low variance are declared background.

- **Central claim:** a mixture per pixel handles multimodal backgrounds (a flickering monitor, leaves) that a single Gaussian cannot, with constant cost per frame.
- **Assumptions to question:** pixels are independent (no spatial smoothness); background is the most frequent and least variable mode (a car parked for a long time becomes background); the learning rate trades adaptation against absorbing slow-moving objects.
- **Affordable experiment:** simulate a 32×32 video where each background pixel alternates between two colours (a “flickering” region) and a small square moves across. Compare per-pixel single-Gaussian and three-component models on foreground precision and recall; then vary the learning rate and measure how long a stopped object takes to be absorbed.

8.12 Memory map

Summary. A latent-variable model explains data through hidden causes; its likelihood sums over them, and the log of a sum couples all parameters. EM separates them again: the E-step computes the posterior over hidden variables by Bayes’ rule (responsibilities); the M-step refits each component by weighted maximum likelihood, in closed form. The identity $\log p(\mathbf{x}) = \mathcal{L}(q, \boldsymbol{\theta}) + D_{\text{KL}}(q \| p(z | \mathbf{x}))$ explains why each step cannot lower the likelihood, and — when the posterior is intractable — becomes the ELBO of variational inference. k-means is the hard, isotropic limit. In practice EM’s answers depend on initialisation, on K , and on a covariance floor, because the unconstrained likelihood is unbounded; and clusters are only as meaningful as the features they are computed on.

8.13 Exercises

1. *Conceptual.* Why does a responsibility of 0.6 not mean that 60% of a patch is lesion?
2. *Conceptual.* The training log-likelihood increases from $K = 3$ to $K = 4$. Does that justify choosing four components?
3. *Derivation.* Derive the M-step for a mixture of K Bernoulli vectors (binary features), and for a GMM with one shared covariance $\boldsymbol{\Sigma}$.
4. *Derivation.* Show that the gap $\ell(\boldsymbol{\theta}) - \mathcal{L}(q, \boldsymbol{\theta})$ is the KL divergence from q to the posterior, and conclude that EM’s iterates have non-decreasing ℓ .
5. *Derivation.* With an inverse-Wishart prior on $\boldsymbol{\Sigma}_k$, show the MAP M-step has the form $(N_k \mathbf{S}_k + \boldsymbol{\Psi}) / (N_k + \nu')$, and choose $\boldsymbol{\Psi}, \nu'$ to obtain a floor $\mathbf{S}_k + \epsilon \mathbf{I}$ approximately when N_k is large.
6. *Derivation.* Show that k-means never increases $\sum_i \|\mathbf{x}^{(i)} - \boldsymbol{\mu}_{z^{(i)}}\|^2$, and that it terminates in finitely many steps.
7. *Implementation.* Add a diagonal-covariance option to the lab and compare BIC curves for full and diagonal models on the patch data.
8. *Implementation.* Use EM to fill in missing entries: delete 20% of the coordinates of the patch data at random, and write the E-step that computes the conditional mean and covariance of the missing coordinates given the observed ones (chapter 7’s Gaussian conditioning).
9. *Interpretation.* A GMM on image embeddings finds a cluster containing all photographs from one camera, healthy and diseased alike. What does that tell you about the embedding and about the downstream classifier?

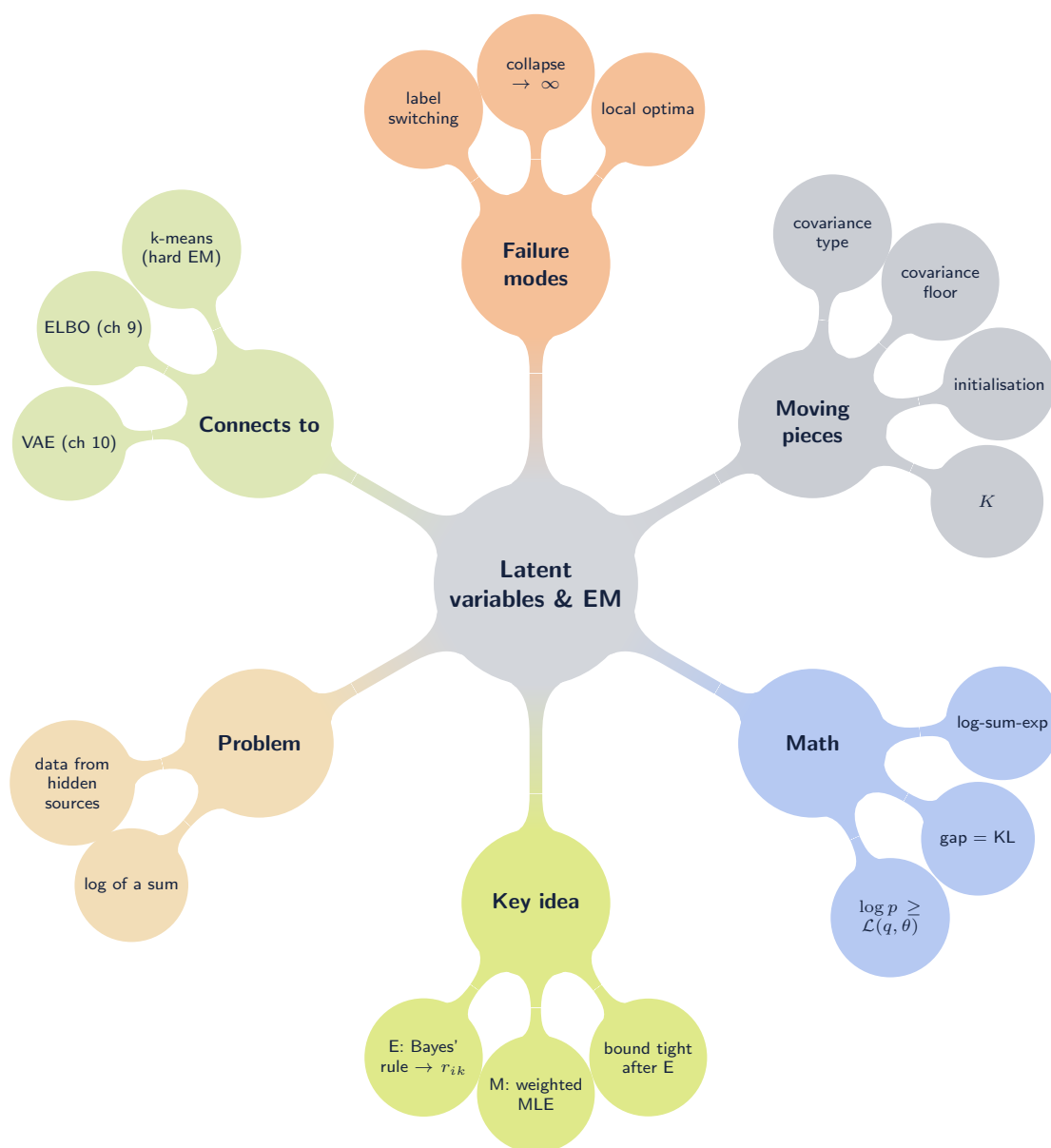


Figure 8.5: Memory map of chapter 8.

10. *Open question.* SwAV (chapter 15) assigns features to prototypes with soft assignments forced to use all prototypes equally. Which EM failure mode does the equal-use constraint prevent, and what does it cost?

8.13.1 Answer discussions

1. The model says the *whole* patch came from one source; 0.6 is the posterior belief that the source was the lesion component. A patch that is literally part lesion is a violation of the model.
2. No. More components can only fit the training data better (the $K = 3$ solution is a special case of $K = 4$). Use held-out likelihood or BIC, and inspect whether the extra component is meaningful.
3. Bernoulli: $\pi_k = N_k/m$ and $\mathbf{p}_k = \sum_i r_{ik} \mathbf{x}^{(i)} / N_k$ (weighted feature frequencies). Shared covariance: $\mathbf{\Sigma} = \frac{1}{m} \sum_i \sum_k r_{ik} (\mathbf{x}^{(i)} - \boldsymbol{\mu}_k)(\mathbf{x}^{(i)} - \boldsymbol{\mu}_k)^\top$.
4. Use $p(\mathbf{x}, z) = p(z | \mathbf{x})p(\mathbf{x})$ and $\sum_z q(z) = 1$, as in step 7; then chain the three (in)equalities.
5. The posterior over $\mathbf{\Sigma}_k$ is inverse-Wishart with scale $\mathbf{\Psi} + N_k \mathbf{S}_k$; its mode divides by $N_k + \nu + n + 1$. Taking $\mathbf{\Psi} = \epsilon N_k \mathbf{I}$ (or a fixed $\mathbf{\Psi} = \epsilon \mathbf{I}$ with small ν) gives approximately $\mathbf{S}_k + \epsilon \mathbf{I}$. The floor is a prior belief that covariances are not smaller than ϵ .
6. The assignment step cannot increase the objective (each point picks its nearest mean) and the update step cannot either (the mean minimises squared distance); there are finitely many partitions and none repeats while the objective strictly decreases.
7. Diagonal models have far fewer parameters; for these correlated clusters full covariances should still win on BIC, but the gap narrows after PCA whitening.
8. For each example, partition $\mathbf{x}$ into observed and missing parts; the E-step uses $\boldsymbol{\mu}_m + \boldsymbol{\Sigma}_{mo} \boldsymbol{\Sigma}_{oo}^{-1} (\mathbf{x}_o - \boldsymbol{\mu}_o)$ and the Schur complement; the M-step adds the conditional covariance to the outer products.
9. The embedding encodes camera identity strongly — a shortcut. A classifier trained on it may learn the camera rather than the disease; test with a camera- or site-grouped split (chapter 5).
10. Collapse to a few prototypes (all features assigned to one cluster) — a trivial solution of the joint feature-and-cluster objective. The cost is a bias toward equal-sized clusters, which may not match the data.

8.14 Self-check and common mistakes

- ☐ I can write a mixture's generative story, its likelihood and its complete-data likelihood.
- ☐ I can derive the E-step from Bayes' rule and the three M-step updates.
- ☐ I can prove EM's monotonicity from the bound-plus-KL identity.
- ☐ I can explain why the GMM likelihood is unbounded and name two fixes.
- ☐ I can say how k-means relates to EM and what it loses.

Common mistakes: computing responsibilities with raw densities (underflow, NaN); using the old mean in the covariance update; dividing by $N_k - 1$; forgetting the covariance floor; declaring convergence on a fixed number of iterations without checking the trace; reporting one random seed; comparing component indices across runs; choosing K by training likelihood; interpreting clusters as categories without checking against labels.

References

- I. Goodfellow, Y. Bengio and A. Courville. *Deep Learning*. MIT Press, 2016, sections 3.9.6, 16.5 and 19.2 (mixtures, latent variables and EM).
- K. Pearson. Contributions to the mathematical theory of evolution. *Phil. Trans. R. Soc. A*, 185:71–110, 1894.
- A. P. Dempster, N. M. Laird and D. B. Rubin. Maximum likelihood from incomplete data via the EM algorithm. *JRSS B*, 39(1):1–38, 1977.
- C. F. J. Wu. On the convergence properties of the EM algorithm. *Annals of Statistics*, 11(1):95–103, 1983.
- S. P. Lloyd. Least squares quantization in PCM. *IEEE Trans. Information Theory*, 28(2):129–137, 1982.
- R. M. Neal and G. E. Hinton. A view of the EM algorithm that justifies incremental, sparse, and other variants. In *Learning in Graphical Models*, Kluwer, 1998.
- M. E. Tipping and C. M. Bishop. Probabilistic principal component analysis. *JRSS B*, 61(3):611–622, 1999.
- C. Stauffer and W. E. L. Grimson. Adaptive background mixture models for real-time tracking. In *CVPR*, 1999.
- J. Sivic and A. Zisserman. Video Google: a text retrieval approach to object matching in videos. In *ICCV*, 2003.
- F. Perronnin and C. Dance. Fisher kernels on visual vocabularies for image categorization. In *CVPR*, 2007.
- D. Arthur and S. Vassilvitskii. k-means++: the advantages of careful seeding. In *SODA*, 2007.
- C. M. Bishop. *Pattern Recognition and Machine Learning*. Springer, 2006, chapter 9.

Chapter 9

KL divergence, variational inference and the ELBO

Prerequisites: chapter 8 (EM and its lower bound); chapter 6 (posteriors, Laplace approximation); chapter 7 (Gaussian algebra).

Learning outcomes: after this chapter you can

1. define the KL divergence, prove it is non-negative, and predict how the direction of a KL fit shapes an approximation (mode-seeking versus mass-covering);
2. derive the evidence lower bound (ELBO) in three equivalent forms and say what each term means;
3. turn posterior inference into optimisation over a family $\mathcal{Q}$, and derive coordinate-ascent mean-field updates;
4. derive and compare score-function and reparameterisation gradient estimators;
5. run Gaussian variational inference for a model with no closed-form posterior and check it against an exact answer.

9.1 A concrete problem

The crop-health team has only 30 labelled leaf images for a new disease. A logistic-regression classifier on two image features fits easily, but with so few examples the team wants **uncertainty in the weights**: how much would the decision boundary move with different data, and how confident should the prediction for a borderline leaf be?

Chapter 6 gave the recipe — prior $\times$ likelihood, normalise — but for logistic regression the normalising constant

$$p(\mathbf{y} \mid \mathbf{X}) = \int \prod_{i=1}^m \sigma(\mathbf{w}^\top \mathbf{x}^{(i)})^{y^{(i)}} (1 - \sigma(\mathbf{w}^\top \mathbf{x}^{(i)}))^{1-y^{(i)}} \mathcal{N}(\mathbf{w}; \mathbf{0}, \tau^2 \mathbf{I}) d\mathbf{w}$$

has no closed form. With two weights we can integrate on a grid; with a million weights, or with a latent code for every image (chapter 10), we cannot. Chapter 8’s EM needed the exact posterior in its E-step; here it is unavailable.

Variational inference (VI) changes the question from “compute the posterior” to “find the closest member of a tractable family”, and “closest” is measured by a **KL divergence**. Inference becomes optimisation — and optimisation is something deep learning does at

scale. This chapter is the bridge from classical probabilistic modelling to VAEs (chapter 10) and diffusion models (chapter 22), whose training objectives are both ELBOs.

9.2 How we got here

Information and divergence (1948–1951). Claude Shannon defined entropy as the average number of bits needed to encode outcomes. Solomon Kullback and Richard Leibler (1951) defined the “information for discrimination” between two distributions — now the KL divergence: the expected number of *extra* nats paid for encoding samples from p with a code optimised for q . That reading explains why it is asymmetric and non-negative.

Variational principles in physics (1900s–1970s). Physicists approximated intractable many-particle distributions by factorised ones — Pierre Weiss’s mean-field theory of magnetism (1907) — and the Gibbs–Bogoliubov–Feynman inequality bounds a free energy by an expectation under a simpler distribution. The ELBO is the negative of that variational free energy; the word “variational” comes from optimising over a function (the distribution q), as in the calculus of variations.

Into machine learning (1987–1999). Peterson and Anderson (1987) used mean-field theory to train Boltzmann machines. Hinton and van Camp (1993) put a factorised Gaussian over a network’s weights and minimised a description length — an ELBO. Neal and Hinton (1998) showed that EM is coordinate ascent on the same bound (chapter 8). Jordan, Ghahramani, Jaakkola and Saul (1999) wrote the introduction that made VI a standard tool for graphical models. Minka (2001) proposed expectation propagation, which uses the other KL direction.

Scaling up (2012–2015). Hoffman, Blei, Wang and Paisley (2013) made VI stochastic, using minibatches like SGD. Two gradient estimators made VI “black-box”: the **score-function** estimator (Williams’s REINFORCE, 1992; Ranganath, Gerrish and Blei, 2014) works for any model; the **reparameterisation trick** (Kingma and Welling, 2013; Rezende, Mohamed and Wierstra, 2014; Titsias and Lázaro-Gredilla, 2014) gives far lower variance for continuous variables and made VAEs possible. Blundell and colleagues (2015) applied it to weights (“Bayes by Backprop”), this chapter’s paper bridge; Gal and Ghahramani (2016) reinterpreted dropout as approximate VI.

Why it matters. The same identity — $\log p(\mathbf{x}) = \text{ELBO} + \text{KL}$ — trains VAEs (chapter 10), underlies diffusion models’ loss (chapter 22), and explains the KL penalties in reinforcement learning from human feedback and in knowledge distillation. Reading those papers is much easier once this chapter is familiar.

9.3 Assumptions and notation

Symbol	Meaning
$\mathbf{x}, \mathbf{z}$	observed and latent variables (for Bayesian models, $\mathbf{z}$ = parameters $\mathbf{w}$)
$p(\mathbf{x}, \mathbf{z}) = p(\mathbf{z})p(\mathbf{x} \mathbf{z})$	the model: prior times likelihood
$p(\mathbf{z} \mathbf{x})$	the posterior, usually intractable
$p(\mathbf{x})$	the evidence (marginal likelihood)
$q(\mathbf{z}), q_\phi$	approximating distribution with variational parameters ϕ
$\mathcal{Q}$	the variational family

Symbol	Meaning
$D_{\text{KL}}(q\ p) = \mathbb{E}_q[\log q - \log p]$	KL divergence from q to p
$H(q) = -\mathbb{E}_q[\log q]$	entropy
$\mathcal{L}(q)$	the ELBO
ϵ	parameter-free noise used in reparameterisation
L	number of Monte Carlo samples per gradient

Assumptions:

1. **We can evaluate** $\log p(\mathbf{x}, \mathbf{z})$ for any $\mathbf{z}$ (prior $\times$ likelihood), and its gradient in $\mathbf{z}$ for reparameterisation. We cannot evaluate $p(\mathbf{x})$.
2. **Support.** $q(\mathbf{z}) > 0$ only where $p(\mathbf{z} \mid \mathbf{x}) > 0$; otherwise $D_{\text{KL}}(q\|p) = \infty$.
3. **The family is tractable:** we can sample from q , evaluate $\log q$, and (ideally) its entropy in closed form.

9.4 Mathematical formulation

KL divergence. For densities q and p on the same space,

$$D_{\text{KL}}(q\|p) = \int q(\mathbf{z}) \log \frac{q(\mathbf{z})}{p(\mathbf{z})} d\mathbf{z} \geq 0, \quad \text{with equality iff } q = p \text{ almost everywhere.}$$

Variational inference solves

$$q^* = \arg \min_{q \in \mathcal{Q}} D_{\text{KL}}(q(\mathbf{z}) \parallel p(\mathbf{z} \mid \mathbf{x})) = \arg \max_{q \in \mathcal{Q}} \mathcal{L}(q),$$

where the **evidence lower bound (ELBO)** has three equivalent forms:

$$\begin{aligned} \mathcal{L}(q) &= \mathbb{E}_q[\log p(\mathbf{x}, \mathbf{z})] + H(q) && \text{(energy + entropy)} \\ &= \mathbb{E}_q[\log p(\mathbf{x} \mid \mathbf{z})] - D_{\text{KL}}(q(\mathbf{z}) \parallel p(\mathbf{z})) && \text{(fit to data - distance from prior)} \\ &= \log p(\mathbf{x}) - D_{\text{KL}}(q(\mathbf{z}) \parallel p(\mathbf{z} \mid \mathbf{x})) && \text{(evidence - approximation gap).} \end{aligned}$$

The first form is what we compute; the second is how VAE papers write it; the third is why it works.

9.5 Derivation, with reasons for each step

Step 1 — KL is non-negative (Gibbs' inequality).

$$-D_{\text{KL}}(q\|p) = \mathbb{E}_q\left[\log \frac{p(\mathbf{z})}{q(\mathbf{z})}\right] \leq \log \mathbb{E}_q\left[\frac{p(\mathbf{z})}{q(\mathbf{z})}\right] = \log \int_{q>0} p(\mathbf{z}) d\mathbf{z} \leq \log 1 = 0.$$

Reason: Jensen's inequality for the concave log; the ratio's expectation under q cancels q . Equality needs p/q constant on the support of q , i.e. $q = p$. KL is **not a distance**: it is

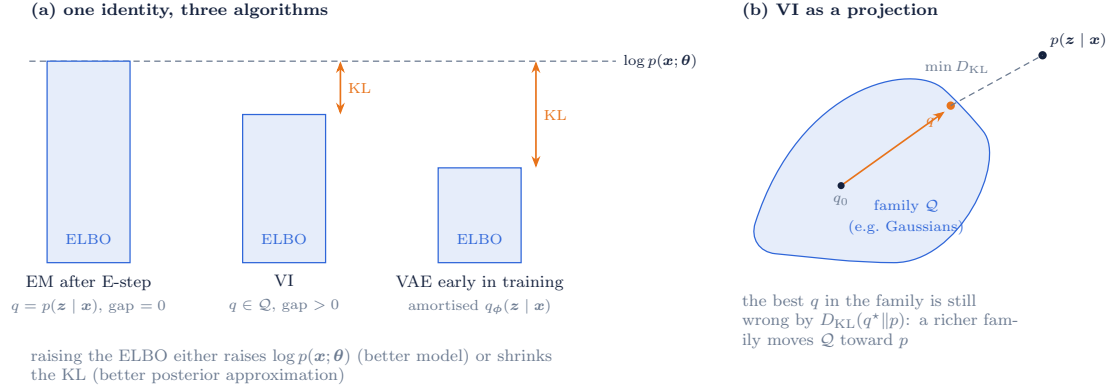


Figure 9.1: (a) For every q , the log evidence splits into the ELBO and a KL gap. EM closes the gap exactly in its E-step (chapter 8); VI closes it as far as the family allows; a VAE amortises q with an encoder network (chapter 10). (b) VI as a projection: the optimiser moves q inside the family $\mathcal{Q}$ to the member closest to the posterior in reverse KL.

not symmetric and does not satisfy the triangle inequality. For two Gaussians it has the closed form

$$D_{\text{KL}}(\mathcal{N}(\mathbf{z}; \boldsymbol{\mu}_1, \boldsymbol{\Sigma}_1) \| \mathcal{N}(\mathbf{z}; \boldsymbol{\mu}_2, \boldsymbol{\Sigma}_2)) = \frac{1}{2} \left[\text{Tr}(\boldsymbol{\Sigma}_2^{-1} \boldsymbol{\Sigma}_1) + (\boldsymbol{\mu}_2 - \boldsymbol{\mu}_1)^\top \boldsymbol{\Sigma}_2^{-1} (\boldsymbol{\mu}_2 - \boldsymbol{\mu}_1) - n + \log \frac{\det \boldsymbol{\Sigma}_2}{\det \boldsymbol{\Sigma}_1} \right],$$

obtained by taking the expectation of the log-density difference under the first Gaussian ($\mathbb{E}[\mathbf{d}^\top \mathbf{A} \mathbf{d}] = \text{Tr}(\mathbf{A} \boldsymbol{\Sigma}) + \dots$). The mean difference is measured in the *second* distribution's metric — the algebraic source of the asymmetry.

Step 2 — the direction matters: mode-seeking and mass-covering. Look at where each direction pays a penalty.

- **Reverse KL**, $D_{\text{KL}}(q \| p) = \mathbb{E}_q[\log q - \log p]$, averages over q 's samples. Where $q > 0$ but $p \approx 0$, $\log p \rightarrow -\infty$: a huge penalty. Where $p > 0$ but $q \approx 0$: no penalty at all. So q avoids putting mass where p has none (“zero-forcing”) and is content to cover just one mode. VI uses this direction.
- **Forward KL**, $D_{\text{KL}}(p \| q)$, averages over p 's samples. Where $p > 0$ but $q \approx 0$, the penalty is huge; so q must cover every region where p has mass (“zero-avoiding”), even if that puts q 's mass in valleys between modes. For an exponential-family q , minimising forward KL is **moment matching**: $\mathbb{E}_q[T] = \mathbb{E}_p[T]$ (chapter 6's maximum-likelihood argument).

Connection: maximum likelihood is forward-KL minimisation from the data distribution, $\arg \max_{\theta} \mathbb{E}_{p_{\text{data}}}[\log p_{\theta}] = \arg \min_{\theta} D_{\text{KL}}(p_{\text{data}} \| p_{\theta})$. That is why likelihood-trained generative models tend to spread mass (blurry samples), while reverse-KL or adversarial objectives tend to drop modes (sharp but less diverse samples) — a theme in chapters 10 and 22.

Step 3 — the ELBO, first derivation (Jensen). Introduce q by multiplying and dividing:

$$\log p(\mathbf{x}) = \log \int q(\mathbf{z}) \frac{p(\mathbf{x}, \mathbf{z})}{q(\mathbf{z})} d\mathbf{z} \geq \int q(\mathbf{z}) \log \frac{p(\mathbf{x}, \mathbf{z})}{q(\mathbf{z})} d\mathbf{z} = \mathcal{L}(q).$$

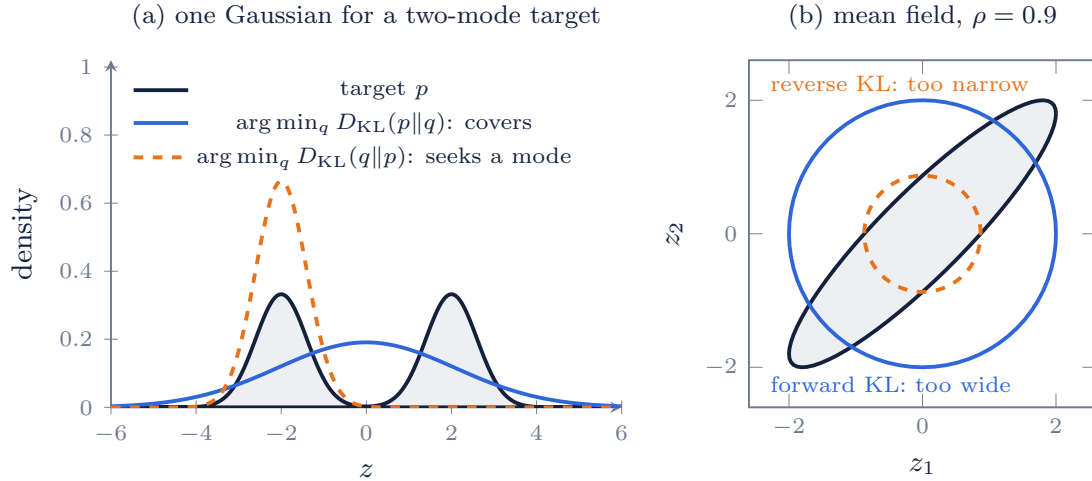


Figure 9.2: (a) The best single Gaussian for a two-mode target: forward KL covers both modes with one wide Gaussian (mean 0, sd 2.09); reverse KL locks onto one mode (mean -2 , sd 0.60). (b) Factorised (mean-field) Gaussians for a correlated Gaussian target: reverse KL gives sd 0.44 per axis, too narrow; forward KL gives the marginal sd 1, which puts mass far off the diagonal ridge.

Reason: identical to chapter 8, step 7, with an integral in place of a sum. This shows it is a bound but not how loose.

Step 4 — the ELBO, second derivation (exact identity). Start from the quantity we want to make small and substitute $p(\mathbf{z} | \mathbf{x}) = p(\mathbf{x}, \mathbf{z})/p(\mathbf{x})$:

$$D_{\text{KL}}(q \| p(\mathbf{z} | \mathbf{x})) = \mathbb{E}_q[\log q(\mathbf{z}) - \log p(\mathbf{x}, \mathbf{z}) + \log p(\mathbf{x})] = -\mathcal{L}(q) + \log p(\mathbf{x}).$$

Reason: $\log p(\mathbf{x})$ does not depend on $\mathbf{z}$, so its expectation is itself. Rearranged: $\log p(\mathbf{x}) = \mathcal{L}(q) + D_{\text{KL}}(q \| p(\mathbf{z} | \mathbf{x}))$. Three consequences:

1. Because $\log p(\mathbf{x})$ is a constant in q , **maximising the ELBO is the same as minimising the reverse KL to the posterior** — without ever computing the posterior or the evidence.
2. The gap is the **KL to the posterior**, not the KL to the prior that appears in the second form. Confusing the two is the most common mistake in reading VAE papers.
3. When q is the exact posterior, the ELBO *equals* the log evidence. When the family excludes the posterior, a gap remains that no optimiser can remove (section 9.6).

Step 5 — the third form. Split $\log p(\mathbf{x}, \mathbf{z}) = \log p(\mathbf{x} | \mathbf{z}) + \log p(\mathbf{z})$:

$$\mathcal{L}(q) = \underbrace{\mathbb{E}_q[\log p(\mathbf{x} | \mathbf{z})]}_{\text{expected fit}} - \underbrace{D_{\text{KL}}(q(\mathbf{z}) \| p(\mathbf{z}))}_{\text{complexity}}.$$

Reading: explain the data well on average over q , but don't move far from the prior. This is a data-fit-plus-regulariser objective, like ridge (chapter 3) — and with q a point mass the first term is the log-likelihood at that point while the KL is infinite, which is why point estimates fall outside this framework and why MAP is not VI.

Step 6 — mean-field coordinate ascent. Take a factorised family $q(\mathbf{z}) = \prod_j q_j(z_j)$ and optimise one factor with the others fixed. Collecting the terms of $\mathcal{L}$ that involve q_j :

$$\mathcal{L} = \int q_j(z_j) \mathbb{E}_{-j}[\log p(\mathbf{x}, \mathbf{z})] dz_j - \int q_j \log q_j dz_j + \text{const},$$

where $\mathbb{E}_{-j}$ averages over all other factors. This is $-D_{\text{KL}}(q_j \| \tilde{p}_j) + \text{const}$ with $\log \tilde{p}_j(z_j) = \mathbb{E}_{-j}[\log p(\mathbf{x}, \mathbf{z})]$, so the optimum is

$$\log q_j^*(z_j) = \mathbb{E}_{-j}[\log p(\mathbf{x}, \mathbf{z})] + \text{const}.$$

Reason: a KL is minimised by setting its arguments equal. Cycling through j is **coordinate-ascent VI (CAVI)**; each update cannot lower $\mathcal{L}$.

Gaussian example. For a target $\mathcal{N}(\mathbf{z}; \boldsymbol{\mu}, \boldsymbol{\Lambda}^{-1})$, $\log p$ is quadratic, and $\mathbb{E}_{-j}[\log p]$ is a quadratic in z_j with curvature Λ_{jj} :

$$q_j^*(z_j) = \mathcal{N}\left(z_j; \mu_j - \frac{1}{\Lambda_{jj}} \sum_{k \neq j} \Lambda_{jk}(m_k - \mu_k), \frac{1}{\Lambda_{jj}}\right).$$

The means converge to the exact $\boldsymbol{\mu}$; the variances are $1/\Lambda_{jj}$, the **conditional** variance, which is smaller than the marginal Σ_{jj} whenever variables are correlated. For $\rho = 0.9$, $1/\Lambda_{jj} = 1 - \rho^2 = 0.19$: sd 0.44 instead of 1 (figure 9.2b). Mean-field VI under-estimates uncertainty — systematically, not by accident.

Step 7 — gradients of an expectation: the score-function estimator. In general $\mathcal{L}$ is an expectation we cannot compute, $\mathcal{L}(\phi) = \mathbb{E}_{q_\phi}[f(\mathbf{z})]$ with $f = \log p(\mathbf{x}, \mathbf{z}) - \log q_\phi(\mathbf{z})$. The difficulty: the distribution itself depends on ϕ . Use $\nabla_\phi q = q \nabla_\phi \log q$:

$$\nabla_\phi \mathbb{E}_{q_\phi}[f(\mathbf{z})] = \int f(\mathbf{z}) \nabla_\phi q_\phi(\mathbf{z}) d\mathbf{z} = \mathbb{E}_{q_\phi}[f(\mathbf{z}) \nabla_\phi \log q_\phi(\mathbf{z})] \approx \frac{1}{L} \sum_{l=1}^L f(\mathbf{z}^{(l)}) \nabla_\phi \log q_\phi(\mathbf{z}^{(l)}).$$

(The extra term from f 's own dependence on ϕ has expectation zero, because $\mathbb{E}_q[\nabla \log q] = \nabla \int q = 0$.) It needs only samples and $\log q$, so it works for discrete variables and non-differentiable f — it is REINFORCE, used in reinforcement learning. Its weakness is **variance**: f multiplies a zero-mean score, so a large constant offset in f adds noise. The same zero-mean property allows subtracting a **baseline** b : $\mathbb{E}[(f - b) \nabla \log q] = \mathbb{E}[f \nabla \log q]$ for any constant b .

Step 8 — the reparameterisation estimator. If $\mathbf{z}$ can be written as a differentiable function of ϕ and parameter-free noise — for a Gaussian, $\mathbf{z} = \mathbf{m} + \mathbf{L}\boldsymbol{\epsilon}$, $\boldsymbol{\epsilon} \sim \mathcal{N}(\mathbf{z}; \mathbf{0}, \mathbf{I})$ — then the expectation is over a fixed distribution and the gradient moves inside:

$$\nabla_\phi \mathbb{E}_{q_\phi}[f(\mathbf{z})] = \mathbb{E}_\epsilon[\nabla_\phi f(\mathbf{m} + \mathbf{L}\boldsymbol{\epsilon})] = \mathbb{E}_\epsilon\left[(\nabla_{\mathbf{z}} f)^\top \frac{\partial \mathbf{z}}{\partial \phi}\right].$$

Reason: the chain rule, once the sampling no longer depends on ϕ . It uses the *gradient* of f — how f changes as $\mathbf{z}$ moves — rather than only its value, and that extra information is why its variance is usually much lower (figure 9.3). For $f(\mathbf{z}) = z^2$, $\mathbf{z} \sim \mathcal{N}(z; 1, 1)$ and $L = 10$ samples, the lab measures standard deviations of 1.70 (score function), 1.31 (with baseline) and 0.63 (reparameterisation), for the same true gradient 2. It requires continuous

z and differentiable f ; for discrete variables use score functions with control variates, or continuous relaxations (Gumbel-softmax).

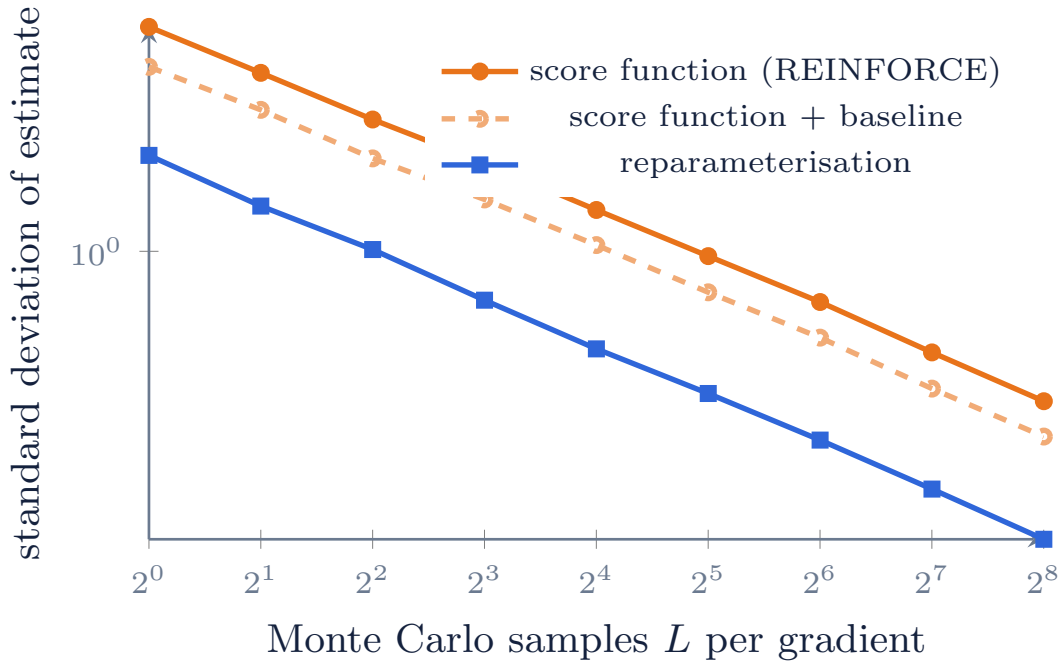


Figure 9.3: Standard deviation of three unbiased estimators of $\frac{d}{d\mu}\mathbb{E}[z^2]$, $z \sim \mathcal{N}(z; 1, 1)$, against the number of samples. All fall as $1/\sqrt{L}$; reparameterisation is about 2.5 times less noisy than the plain score function at every L — equivalent to about six times as many samples.

Step 9 — Gaussian VI in practice. For $q = \mathcal{N}(z; \mathbf{m}, \mathbf{L}\mathbf{L}^\top)$ with lower-triangular $\mathbf{L}$ in n dimensions, the entropy is exact, $H(q) = \sum_j \log |L_{jj}| + \frac{n}{2}(1 + \log 2\pi)$, and only the energy term needs sampling. With $\mathbf{g}^{(l)} = \nabla_z \log p(\mathbf{x}, \mathbf{z}^{(l)})$ at $\mathbf{z}^{(l)} = \mathbf{m} + \mathbf{L}\boldsymbol{\epsilon}^{(l)}$:

$$\nabla_{\mathbf{m}} \mathcal{L} \approx \frac{1}{L} \sum_l \mathbf{g}^{(l)}, \quad \nabla_{\mathbf{L}} \mathcal{L} \approx \frac{1}{L} \sum_l \mathbf{g}^{(l)} \boldsymbol{\epsilon}^{(l)\top} + \text{diag}(1/L_{jj}).$$

Reason: $\partial \mathbf{z} / \partial \mathbf{m} = \mathbf{I}$ and $\partial z_a / \partial L_{ab} = \epsilon_b$; the entropy's derivative is $1/L_{jj}$ on the diagonal. Parameterise $L_{jj} = e^{\rho_j}$ to keep it positive. A diagonal $\mathbf{L}$ is mean-field; a full $\mathbf{L}$ captures correlations at $O(n^2)$ cost.

Step 10 — scaling to large data and to many latent variables.

- **Minibatches.** For a Bayesian model with i.i.d. data, the second form of the ELBO is $\sum_{i=1}^m \mathbb{E}_q[\log p(y^{(i)} | \mathbf{x}^{(i)}, \mathbf{w})] - D_{\text{KL}}(q \| p(\mathbf{w}))$. An unbiased estimate uses a minibatch B and rescales its sum by $m/|B|$ — stochastic VI. Forgetting the rescaling silently changes the prior's strength.
- **Amortisation.** For a model with a latent $\mathbf{z}^{(i)}$ *per example* (chapter 8's mixtures, chapter 10's VAEs), fitting separate variational parameters for every example is expensive and does not generalise to new examples. Instead, train a network $q_\phi(\mathbf{z} | \mathbf{x})$ that outputs them — an **encoder**. The price is an **amortisation gap**: the encoder's q can be worse than the best per-example q .

Step 11 — what the ELBO can and cannot tell you. It is a lower bound on $\log p(\mathbf{x})$, so a higher ELBO for model A than for model B does not prove A has higher evidence: B's bound may simply be looser. Report it as a bound; when evidence matters, estimate it with importance sampling, using q as the proposal: $\log \frac{1}{K} \sum_k p(\mathbf{x}, \mathbf{z}^{(k)})/q(\mathbf{z}^{(k)})$. Its *expectation* is a lower bound on $\log p(\mathbf{x})$ that does not loosen as K grows (the IWAE bound of chapter 10), provided q is positive wherever $p(\mathbf{x}, \mathbf{z})$ is. A single estimate is random and can exceed $\log p(\mathbf{x})$; average many.

9.6 Numerical worked example

A Gaussian posterior found by optimisation. Chapter 6's model: $\mathbf{z} \sim \mathcal{N}(\mathbf{z}; 0, 1)$, $\mathbf{x} | \mathbf{z} \sim \mathcal{N}(\mathbf{x}; \mathbf{z}, 0.25)$, observed $x = 2$. Exact posterior $\mathcal{N}(z; 1.6, 0.2)$; evidence $\mathcal{N}(2; 0, 1.25)$, so $\log p(x) = -\frac{1}{2} \log(2\pi \cdot 1.25) - \frac{4}{2 \cdot 5} = -2.6305$. For $q = \mathcal{N}(z; m, v)$, using $\mathbb{E}_q[(x - z)^2] = (x - m)^2 + v$ and step 1's KL formula,

$$\mathcal{L}(m, v) = -\frac{1}{2} \log(2\pi \cdot 0.25) - 2[(2 - m)^2 + v] - \frac{1}{2}(m^2 + v - 1 - \log v).$$

Setting derivatives to zero: $\partial_m \mathcal{L} = 4(2 - m) - m = 0 \Rightarrow m = 1.6$; $\partial_v \mathcal{L} = -2 - \frac{1}{2} + \frac{1}{2v} = 0 \Rightarrow v = 0.2$. VI recovers the exact posterior and $\mathcal{L}(1.6, 0.2) = -2.6305 = \log p(x)$: the family contains the answer, so the bound is tight.

A restricted family leaves a gap. Fix $v = 1$. The best mean is still 1.6, but $\mathcal{L}(1.6, 1) = -3.8258$, a gap of

$$D_{\text{KL}}(\mathcal{N}(z; 1.6, 1) \| \mathcal{N}(z; 1.6, 0.2)) = \frac{1}{2}[1/0.2 - 1 + \log 0.2] = 1.1953 \text{ nats}.$$

No optimiser can remove this; only a richer family can.

KL asymmetry by hand. $q = \mathcal{N}(\mathbf{z}; \mathbf{0}, \mathbf{I})$, $p = \mathcal{N}(\mathbf{z}; (1, 0), \text{diag}(2, 0.5))$ in two dimensions: $D_{\text{KL}}(q \| p) = \frac{1}{2}[(1/2 + 2) + 1/2 - 2 + \log 1] = 0.5$, while $D_{\text{KL}}(p \| q) = \frac{1}{2}[2.5 + 1 - 2 - \log 1] = 0.75$. The lab's Monte Carlo estimate of the first is 0.4996.

9.7 Small implementation and checks

The lab `labs/ch09-vi/vi.py` is NumPy only. The core of Gaussian VI for Bayesian logistic regression:

```
eps = rng.standard_normal((L, 2))
W = m + eps @ Lc.T                                     # reparameterised samples
→ of the weights
G = grad_log_joint(W, X, y)                             # L x 2: d/dw [log
→ p(y|X, w) + log p(w)]
g_m = G.mean(0)                                         # gradient for the mean
g_L = (G[:, :, None] * eps[:, None, :]).mean(0)       # E[g eps^T]
g_L[diag] += 1 / Lc[diag]                             # entropy term (exact)
```

followed by an Adam step (chapter 11). The ELBO for monitoring is evaluated with a *fixed* set of 2000 noise draws so that its curve is smooth; training resamples noise at every step, so the optimiser follows the expected objective rather than one noise realisation.

Checks, in `test_vi.py`:

1. KL of a distribution with itself is zero; KL is positive and asymmetric; closed form matches Monte Carlo.
2. Forward KL moment-matches the two-mode target; reverse KL locks onto one mode.
3. Mean-field variances equal $1 - \rho^2$; CAVI converges to the exact mean.
4. All three gradient estimators are unbiased; their spreads are ordered $\text{reparam} < \text{baseline} < \text{score}$.
5. $\text{ELBO} + \text{KL} = \log \text{evidence}$ exactly on a conjugate model, for an arbitrary q and for the posterior.
6. Scalar example: grid maximum at $(1.6, 0.2)$ with $\text{ELBO} = \log p(x)$; the fixed-variance gap is 1.1953.
7. For logistic regression: the ELBO stays below the exact log evidence, full covariance beats mean field, mean-field variances are too small, and VI's mean is closer to the posterior mean than Laplace's.

9.8 Experiment and expected observations

Run `python labs/ch09-vi/vi.py` (seed 0). Bayesian logistic regression with two weights, 30 points, prior $\mathcal{N}(\mathbf{w}; \mathbf{0}, 4\mathbf{I})$; the exact posterior is computed on a 141×141 grid.

method	mean	sd	ELBO / log evidence
exact (grid)	$(2.592, -1.869)$	$(0.778, 0.732)$, corr -0.65	$\log p(\mathbf{y} \mathbf{X}) = -14.773$
Laplace (chapter 6)	$(2.403, -1.743)$	$(0.774, 0.719)$	—
VI, full covariance	$(2.635, -1.885)$	$(0.754, 0.701)$	-14.804
VI, mean field	$(2.603, -1.797)$	$(0.585, 0.539)$	-15.092

What to notice:

1. **The bound is a bound.** Both ELBO curves stay below the dashed log evidence. The curves are Monte Carlo estimates (2,000 fixed samples from q), so they estimate the gap $D_{\text{KL}}(q||p(\mathbf{w} | \mathbb{D}))$ of step 4 (0.03 and 0.32 nats) up to sampling error; with few samples, an estimate could cross the line.
2. **Mean field is over-confident.** Its standard deviations (0.59, 0.54) are about 25% too small, because it cannot represent the -0.65 correlation (step 6). Predictions made with it will be too confident near the decision boundary.
3. **Laplace and VI disagree about the centre.** The logistic posterior is skewed; Laplace sits at the mode, VI near the mean. Which is better depends on the use; for predictive averages the mean matters.
4. **Noise in the ELBO trace.** The curves wobble by a few hundredths of a nat even with a fixed evaluation set, because the parameters themselves move with noisy gradients. Use a decaying step size, more samples, or average the iterates for a final answer.
5. **Asymmetric KL in one picture.** Figure 9.2a: forward KL gives mean 0, sd 2.09 — it covers both modes but puts most mass in the valley; reverse KL gives mean -2.00 , sd 0.60 — one mode, exactly. Which run of VI finds which mode depends on the initialisation.

9.9 Moving pieces: diagnosing and fixing

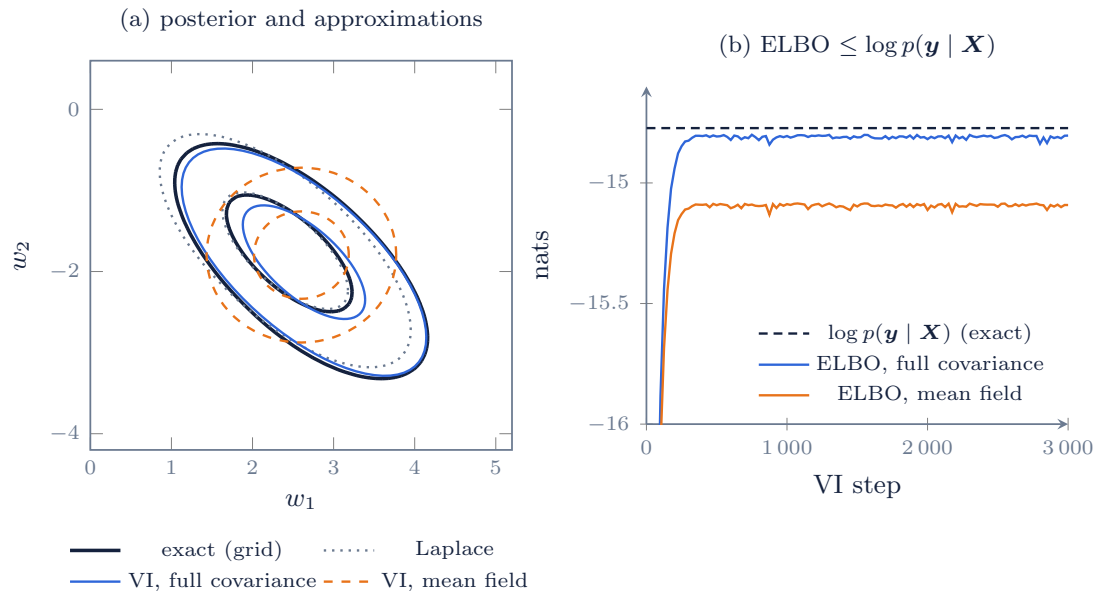


Figure 9.4: Bayesian logistic regression with two weights. (a) The exact posterior (black, grid) is skewed away from the origin; Laplace is centred at the mode, full-covariance VI near the mean, and mean-field VI cannot tilt and is too narrow. (b) The ELBO rises to just below the exact log evidence; the remaining gap is the KL from q to the posterior: 0.03 nats for full covariance, 0.32 for mean field.

Moving piece	Symptom when it is wrong	How to confirm	Fix
Variational family	over-confident predictions; ELBO plateaus well below a better method's	compare variances with an exact or sampled posterior on a small problem; compare full vs. mean field	full or low-rank covariance; mixtures; normalising flows
KL direction	a whole mode missing; or mass placed between modes	multiple restarts land on different modes	accept (report it); restarts; mixtures; forward-KL methods (EP) or sampling
Gradient estimator	ELBO trace extremely noisy; slow or no progress	gradient variance across repeated draws	reparameterisation where possible; baselines/control variates; more samples L
Step size / optimiser	ELBO diverges or oscillates	trace of ELBO; NaN in log scale parameters	smaller or decaying learning rate; Adam; clip $\rho = \log \sigma$
Variance parameterisation	negative or zero variances; NaN	inspect σ	$\sigma = \exp(\rho)$ or softplus; initialise small but not tiny

Moving piece	Symptom when it is wrong	How to confirm	Fix
Minibatch scaling	posterior too wide or too narrow as data grow	compare full-batch and minibatch ELBO	rescale the data term by $m/ B $
Prior	posterior dominated by an arbitrary prior scale	vary the prior scale	prior predictive checks (chapter 6); report sensitivity
Amortisation	encoder's q worse than per-example optimisation	refine q for a few examples by direct optimisation	larger encoder; iterative refinement
Monitoring	claims about “likelihood” made from an ELBO	read the definition	report it as a bound; importance-weighted estimate for comparisons

First things to try: (1) on a two- or three-dimensional version of your model, compare VI with an exact grid or long MCMC run; (2) plot the ELBO trace with a fixed evaluation set; (3) switch from mean-field to full covariance and see whether the conclusions change; (4) restart from several initialisations and compare.

9.10 Application, limitations and failure cases

Application (educational case study). With 30 labelled leaf images, the team fits Bayesian logistic regression on two or three features with full-covariance VI. For a new leaf, the **posterior predictive** probability $\mathbb{E}_{q(\mathbf{w})}[\sigma(\mathbf{w}^\top \mathbf{x})]$ (estimated by sampling $\mathbf{w}$) is less extreme than the plug-in prediction at the mean weights, especially for leaves unlike the training set; leaves whose predictive probability is near 0.5 *and* whose weight-sampled predictions disagree most are sent for expert review. With a deep feature extractor, the same idea applies to the last layer only (“Bayesian last layer”), which is cheap and often captures most of the useful uncertainty.

Limitations:

- **VI under-estimates uncertainty** with mean-field families and reverse KL; it can miss modes entirely. It is an approximation with a known bias, not a sampler.
- **The ELBO is not the likelihood.** Model comparisons by ELBO can be misleading when bounds differ in tightness.
- **Gradient noise** makes convergence slow and its diagnosis fuzzy; “converged” means the trace is flat within noise, not that q equals the posterior.
- **In large networks**, mean-field weight posteriors often under-fit (the KL term dominates) and need tempering or other adjustments — an active and contested area.

9.11 Bridge to a research paper

Blundell, Cornebise, Kavukcuoglu and Wierstra (2015), *Weight uncertainty in neural networks* (ICML 2015) — “Bayes by Backprop”.

- **Problem:** neural networks trained by maximum likelihood or MAP give no weight uncertainty, are over-confident, and explore poorly in decision problems.

- **Mechanism:** a mean-field Gaussian $q(\mathbf{w}) = \prod_j \mathcal{N}(w_j; \mu_j, \sigma_j^2)$ over all weights with $\sigma_j = \log(1 + e^{\rho_j})$; the ELBO's data term estimated by reparameterised samples (step 8) and backpropagation; minibatch scaling of the KL term (step 10); a scale-mixture prior of two Gaussians.
- **Central claim:** comparable accuracy to dropout on MNIST, better-calibrated uncertainty in regression away from data, and better exploration in a contextual bandit.
- **Assumptions to question:** is mean field adequate for millions of correlated weights (step 6)? How much does the prior choice matter? Do the reported gains survive careful tuning of the deterministic baseline?
- **Affordable experiment:** reuse the lab's logistic-regression VI as a one-layer "network". Generate data only on $x_1 < 0$, then compare predictive uncertainty for points with $x_1 > 0$ under (a) MAP, (b) mean-field VI, (c) full-covariance VI and (d) the exact grid posterior. Measure how far each predictive probability is from the exact one.

9.12 Memory map

Summary. When the posterior cannot be computed, choose a tractable family and find its member closest to the posterior. "Closest" is the reverse KL divergence, which can be minimised without the evidence because $\log p(\mathbf{x}) = \text{ELBO}(q) + D_{\text{KL}}(q \| p(\mathbf{z} | \mathbf{x}))$: raising the ELBO lowers the KL. Reverse KL is mode-seeking and, with factorised families, under-estimates variance; forward KL is mass-covering and is what maximum likelihood minimises. Gradients of the ELBO are estimated by sampling: score-function estimators work everywhere but are noisy; reparameterisation uses gradients of the model and is far less noisy. Minibatching scales VI to large data; amortisation with an encoder scales it to a latent variable per example, which is the VAE.

9.13 Exercises

1. *Conceptual.* Which KL is the ELBO's gap, and which KL appears inside the ELBO? Are they the same?
2. *Conceptual.* If q equals the exact posterior, what is the ELBO? What if the family excludes it?
3. *Conceptual.* Why does VI use $D_{\text{KL}}(q \| p)$ rather than $D_{\text{KL}}(p \| q)$?
4. *Derivation.* Derive the closed-form KL between two univariate Gaussians, and evaluate both directions for $\mathcal{N}(z; 0, 1)$ and $\mathcal{N}(z; 1, 4)$.
5. *Derivation.* Show that minimising $D_{\text{KL}}(p \| q)$ over an exponential family q matches the expected sufficient statistics.
6. *Derivation.* Derive the CAVI update for a two-dimensional Gaussian target and prove the variance result $1/\Lambda_{jj} \leq \Sigma_{jj}$.
7. *Derivation.* Show the score function has zero mean, and that subtracting a constant baseline leaves the estimator unbiased. Find the variance-minimising constant baseline.
8. *Implementation.* Replace the reparameterisation gradient in the lab's logistic-regression VI by a score-function gradient with the same number of samples. How many more steps does it need to reach the same ELBO?
9. *Implementation.* Estimate the log evidence of the logistic-regression model by importance sampling with q as the proposal, for $K = 1, 10, 100, 1000$ samples, and compare with the grid value.



Figure 9.5: Memory map of chapter 9.

10. *Open question.* Knowledge distillation trains a student to match a teacher’s softmax with a KL divergence. Which direction is usually used, and what behaviour would the other direction produce?

9.13.1 Answer discussions

1. The gap is $D_{\text{KL}}(q||p(\mathbf{z} | \mathbf{x}))$, to the posterior; inside the ELBO’s second form is $D_{\text{KL}}(q||p(\mathbf{z}))$, to the prior. They differ; only the first measures approximation quality.
2. Exactly $\log p(\mathbf{x})$. Otherwise the ELBO’s maximum is $\log p(\mathbf{x})$ minus the smallest achievable KL, a gap that better optimisation cannot remove.
3. It needs expectations only under q (which we can sample) and p only up to its normalising constant. Forward KL needs expectations under the posterior — the thing we cannot compute.
4. $D_{\text{KL}} = \frac{1}{2}[v_1/v_2 + (m_1 - m_2)^2/v_2 - 1 + \log(v_2/v_1)]$. $q = \mathcal{N}(z; 0, 1)||\mathcal{N}(z; 1, 4)$: $\frac{1}{2}[0.25 + 0.25 - 1 + \log 4] = 0.443$; the reverse: $\frac{1}{2}[4 + 1 - 1 - \log 4] = 1.307$.
5. $D_{\text{KL}}(p||q) = -\mathbb{E}_p[\boldsymbol{\eta}^\top T - A(\boldsymbol{\eta})] + \text{const}$; the gradient is $-\mathbb{E}_p[T] + \nabla A = -\mathbb{E}_p[T] + \mathbb{E}_q[T]$.
6. As in step 6; $1/\Lambda_{jj}$ is the conditional variance of z_j given the others, and conditioning never increases a Gaussian variance (Schur complement, chapter 7).
7. $\mathbb{E}_q[\nabla \log q] = \int \nabla q = \nabla 1 = 0$. The optimal scalar baseline is $\mathbb{E}[f \|\nabla \log q\|^2] / \mathbb{E}[\|\nabla \log q\|^2]$, close to $\mathbb{E}[f]$ when f and the score magnitude are weakly related.
8. Expect several times more steps (or samples) for the same ELBO, roughly in proportion to the variance ratio of figure 9.3 — and sensitivity to the step size.
9. The estimates increase towards -14.773 as K grows ($K = 1$ is the ELBO); with a good proposal, $K = 100$ should be within a few hundredths.
10. Usually forward KL from teacher to student, $D_{\text{KL}}(p_{\text{teacher}}||q_{\text{student}})$ (cross-entropy with soft targets), so the student covers all classes the teacher considers plausible. Reverse KL would let the student concentrate on the teacher’s top class, losing the “dark knowledge” in the other probabilities.

9.14 Self-check and common mistakes

- ☐ I can prove $\text{KL} \geq 0$ and compute it for two Gaussians.
- ☐ I can derive the ELBO two ways and write its three forms.
- ☐ I can explain mode-seeking versus mass-covering with a picture.
- ☐ I can derive the score-function and reparameterisation gradients.
- ☐ I can explain why mean-field VI under-estimates variance.

Common mistakes: calling KL a distance; confusing the KL to the prior with the KL to the posterior; reporting an ELBO as a log-likelihood; comparing models by ELBOs with different tightness; using fresh random noise to evaluate the objective every step and reading the noise as progress; forgetting to rescale the minibatch data term; parameterising a standard deviation without a positivity constraint; trusting mean-field uncertainty for correlated parameters.

References

- I. Goodfellow, Y. Bengio and A. Courville. *Deep Learning*. MIT Press, 2016, sections 3.13 and chapter 19 (approximate inference).
- S. Kullback and R. A. Leibler. On information and sufficiency. *Annals of Mathematical Statistics*, 22(1):79–86, 1951.

- G. E. Hinton and D. van Camp. Keeping neural networks simple by minimizing the description length of the weights. In *COLT*, 1993.
- M. I. Jordan, Z. Ghahramani, T. S. Jaakkola and L. K. Saul. An introduction to variational methods for graphical models. *Machine Learning*, 37:183–233, 1999.
- R. J. Williams. Simple statistical gradient-following algorithms for connectionist reinforcement learning. *Machine Learning*, 8:229–256, 1992.
- M. D. Hoffman, D. M. Blei, C. Wang and J. Paisley. Stochastic variational inference. *JMLR*, 14:1303–1347, 2013.
- R. Ranganath, S. Gerrish and D. M. Blei. Black box variational inference. In *AISTATS*, 2014.
- D. P. Kingma and M. Welling. Auto-encoding variational Bayes. In *ICLR*, 2014. arXiv:1312.6114.
- D. J. Rezende, S. Mohamed and D. Wierstra. Stochastic backpropagation and approximate inference in deep generative models. In *ICML*, 2014.
- C. Blundell, J. Cornebise, K. Kavukcuoglu and D. Wierstra. Weight uncertainty in neural networks. In *ICML*, 2015.
- Y. Gal and Z. Ghahramani. Dropout as a Bayesian approximation. In *ICML*, 2016.
- D. M. Blei, A. Kucukelbir and J. D. McAuliffe. Variational inference: a review for statisticians. *JASA*, 112(518):859–877, 2017.
- C. M. Bishop. *Pattern Recognition and Machine Learning*. Springer, 2006, chapter 10.

Chapter 10

VAEs and generative objectives

Prerequisites: chapter 9 (ELBO, reparameterisation); chapter 8 (latent variables); backpropagation is used here and derived in full in chapter 11.

Learning outcomes: after this chapter you can

1. specify a VAE as a probabilistic model plus an amortised inference network, and trace its training and generation paths separately;
2. derive its loss — the Gaussian KL in closed form, the reconstruction term for a stated decoder likelihood — and its reparameterised gradients;
3. read the loss as a rate–distortion trade-off and explain what β and the decoder variance do;
4. diagnose posterior collapse, unused latent dimensions, blurry samples and holes between the aggregate posterior and the prior;
5. place VAEs among the other generative objectives — autoregressive, flows, GANs, diffusion — and explain why modern latent diffusion uses a VAE-like autoencoder as its first stage.

10.1 A concrete problem

The crop-health team wants two things from thousands of unlabelled leaf-patch images:

1. a **compact code** for each patch — a few numbers that capture where and how large a lesion is — to search, cluster and visualise the data; and
2. a **generator** that produces new, realistic patches, for example to stress-test a detector on lesions in unusual positions.

PCA (chapter 2) gives a linear code but cannot generate realistic images from it. A Gaussian mixture (chapter 8) generates, but in 64 or 1,000,000 pixel dimensions its components are hopeless. What is needed is a latent-variable model $p(\mathbf{z})p_{\theta}(\mathbf{x} | \mathbf{z})$ whose decoder is a neural network — expressive enough for images — together with a way to *infer* $\mathbf{z}$ from $\mathbf{x}$. Exact inference is intractable (chapter 9); the **variational autoencoder (VAE)** trains an encoder network to do approximate inference for every image at once.

10.2 How we got here

Autoencoders (1986–2006). Networks trained to reproduce their input through a narrow middle layer appear in Rumelhart, Hinton and Williams (1986). A linear autoencoder with squared error learns PCA’s subspace (Baldi and Hornik, 1989). Hinton and Salakhutdinov

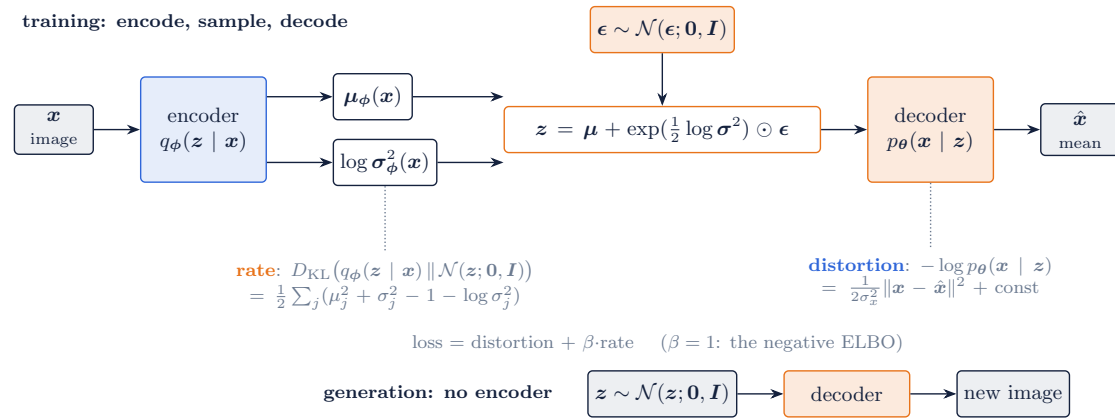


Figure 10.1: The VAE’s two paths. Training: encode an image to a Gaussian over codes, sample a code with the reparameterisation, decode, and pay a distortion term plus a rate term. Generation: sample a code from the prior and decode — the encoder is not used. Standard construction of Kingma and Welling (2014) and Rezende, Mohamed and Wierstra (2014), redrawn.

(2006) trained deep autoencoders layer by layer and got codes far better than PCA. But an autoencoder is not a generative model: there is no distribution over codes to sample from, and nearby codes need not decode to anything sensible.

Recognition models (1995). The Helmholtz machine (Dayan, Hinton, Neal and Zemel, 1995) paired a generative network with a *recognition* network that approximates the posterior — amortised inference — trained by the wake–sleep algorithm, which optimised two different objectives and was not guaranteed to improve either.

The VAE (2013–2014). Diederik Kingma and Max Welling, and independently Danilo Rezende, Shakir Mohamed and Daan Wierstra, combined three pieces: an encoder that outputs a Gaussian $q_\phi(z | x)$, a single objective — the ELBO — for both networks, and the reparameterisation gradient (chapter 9). For the first time a deep latent-variable model could be trained by backpropagation end to end.

Understanding the objective (2016–2019). Bowman and colleagues (2016) found that with powerful decoders the KL term drives the encoder to ignore the input — **posterior collapse** — and proposed KL annealing. Burda, Grosse and Salakhutdinov (2016) tightened the bound with importance weighting (IWAE). Higgins and colleagues (2017) up-weighted the KL (β -VAE) for more “disentangled” codes; Alemi and colleagues (2018) showed that the ELBO is one point on a **rate–distortion** curve and that different points suit different goals. Hoffman and Johnson (2016) showed the KL term contains a mutual information between data and code.

Sharper samples (2017–2022). VAEs were long criticised for blurry samples. VQ-VAE (van den Oord, Vinyals and Kavukcuoglu, 2017) used discrete codes with a learned prior; hierarchical VAEs (NVAE, Vahdat and Kautz, 2020; very deep VAEs, Child, 2021) stacked many latent layers. The most consequential step was a division of labour: **latent diffusion** (Rombach and colleagues, 2022) trains a VAE-like autoencoder only to compress images, with a tiny KL weight, and lets a diffusion model (chapter 22) learn the distribution of its codes. Stable Diffusion and most current image and video generators use this pattern — this chapter’s paper bridge.

Parallel families. Generative adversarial networks (Goodfellow and colleagues, 2014)

avoid likelihoods altogether; normalising flows (Dinh and colleagues, 2015–2017; Rezende and Mohamed, 2015) make likelihood exact with invertible networks; autoregressive models (PixelCNN, 2016; transformers) factorise the likelihood pixel by pixel or token by token. Section 10.5, step 11, compares them.

10.3 Assumptions and notation

Symbol	Meaning
$\mathbf{x} \in \mathbb{R}^n$	an image (here $n = 64$ pixels)
$\mathbf{z} \in \mathbb{R}^d$	latent code (here $d = 8$)
$p(\mathbf{z}) = \mathcal{N}(\mathbf{z}; \mathbf{0}, \mathbf{I})$	the prior
$p_{\theta}(\mathbf{x} \mid \mathbf{z})$	decoder (generative network); here $\mathcal{N}(\mathbf{x}; \hat{\mathbf{x}}_{\theta}(\mathbf{z}), \sigma_x^2 \mathbf{I})$
$q_{\phi}(\mathbf{z} \mid \mathbf{x})$	encoder (inference network); $\mathcal{N}(\mathbf{z}; \boldsymbol{\mu}_{\phi}(\mathbf{x}), \text{diag}(\boldsymbol{\sigma}_{\phi}^2(\mathbf{x})))$
$\mathbf{l} = \log \boldsymbol{\sigma}^2$	the encoder’s log-variance output
β	weight on the KL term ($\beta = 1$: the ELBO)
R, D	rate (KL term) and distortion (reconstruction term), in nats
$q(\mathbf{z}) = \mathbb{E}_{\mathbf{x} \sim p_{\text{data}}} q_{\phi}(\mathbf{z} \mid \mathbf{x})$	the aggregate posterior

Assumptions:

1. **Continuous latent codes** with a standard Gaussian prior, so reparameterisation applies.
2. **A stated likelihood for pixels.** Here: Gaussian with fixed $\sigma_x = 0.1$. This choice is part of the model, not a detail of the loss (step 4).
3. **A diagonal Gaussian posterior family** — a mean-field assumption (chapter 9), applied per image.

10.4 Mathematical formulation

For one image, the VAE maximises the ELBO jointly over decoder parameters $\boldsymbol{\theta}$ and encoder parameters $\boldsymbol{\phi}$:

$$\mathcal{L}(\boldsymbol{\theta}, \boldsymbol{\phi}; \mathbf{x}) = \mathbb{E}_{q_{\phi}(\mathbf{z} \mid \mathbf{x})} [\log p_{\theta}(\mathbf{x} \mid \mathbf{z})] - D_{\text{KL}}(q_{\phi}(\mathbf{z} \mid \mathbf{x}) \parallel p(\mathbf{z})) \leq \log p_{\theta}(\mathbf{x}).$$

The training loss is the negative ELBO averaged over the data, optionally with a KL weight:

$$J(\boldsymbol{\theta}, \boldsymbol{\phi}) = \frac{1}{m} \sum_{i=1}^m \left[\underbrace{-\mathbb{E}_{q_{\phi}(\mathbf{z} \mid \mathbf{x}^{(i)})} \log p_{\theta}(\mathbf{x}^{(i)} \mid \mathbf{z})}_{D: \text{distortion}} + \beta \underbrace{D_{\text{KL}}(q_{\phi}(\mathbf{z} \mid \mathbf{x}^{(i)}) \parallel p(\mathbf{z}))}_{R: \text{rate}} \right].$$

With a diagonal Gaussian encoder and a Gaussian decoder of fixed variance, and one sample $\mathbf{z} = \boldsymbol{\mu} + \exp(\frac{1}{2}\mathbf{l}) \odot \boldsymbol{\epsilon}$ per image:

$$D \approx \frac{1}{2\sigma_x^2} \|\mathbf{x} - \hat{\mathbf{x}}_{\theta}(\mathbf{z})\|^2 + \frac{n}{2} \log(2\pi\sigma_x^2), \quad R = \frac{1}{2} \sum_{j=1}^d (\mu_j^2 + e^{l_j} - 1 - l_j).$$

Generation samples $\mathbf{z} \sim \mathcal{N}(\mathbf{z}; \mathbf{0}, \mathbf{I})$ and outputs $\hat{\mathbf{x}}_{\theta}(\mathbf{z})$ (the decoder mean; adding the σ_x pixel noise would only add speckle). **Reconstruction** encodes first and decodes $\mu_{\phi}(\mathbf{x})$. They are different operations and must be evaluated separately.

10.5 Derivation, with reasons for each step

Step 1 — why a latent-variable model with a neural decoder. The marginal $p_{\theta}(\mathbf{x}) = \int p(\mathbf{z}) p_{\theta}(\mathbf{x} | \mathbf{z}) d\mathbf{z}$ is an *infinite* mixture of Gaussians whose means lie on a curved d -dimensional surface $\{\hat{\mathbf{x}}_{\theta}(\mathbf{z})\}$ in pixel space — the “image manifold”. Chapter 8’s mixture had K separate means; here the network ties infinitely many together smoothly. The integral has no closed form once $\hat{\mathbf{x}}_{\theta}$ is nonlinear, and neither does the posterior $p_{\theta}(\mathbf{z} | \mathbf{x})$.

Step 2 — amortised inference and what the joint objective does. Chapter 9 would fit separate variational parameters $(\mu^{(i)}, \mathbf{l}^{(i)})$ for every image. The VAE replaces them with a function: $(\mu, \mathbf{l}) = \text{encoder}_{\phi}(\mathbf{x})$. Averaging the identity of chapter 9, step 4, over the data,

$$\frac{1}{m} \sum_i \mathcal{L}(\theta, \phi; \mathbf{x}^{(i)}) = \frac{1}{m} \sum_i \log p_{\theta}(\mathbf{x}^{(i)}) - \frac{1}{m} \sum_i D_{\text{KL}}(q_{\phi}(\mathbf{z} | \mathbf{x}^{(i)}) \| p_{\theta}(\mathbf{z} | \mathbf{x}^{(i)})).$$

Reading: raising the objective either raises the data’s likelihood (a better model, via θ) or makes the encoder a better posterior approximation (via ϕ — and also via θ , which can make its own posterior easier to approximate). This is EM’s two steps done simultaneously by gradients. Two gaps separate the trained encoder from the true posterior: the **approximation gap** (the best diagonal Gaussian is not the posterior) and the **amortisation gap** (the encoder does not output the best diagonal Gaussian for each image). The lab measures the second (section 10.8).

Step 3 — the Gaussian KL in closed form. For $q = \mathcal{N}(\mathbf{z}; \mu, \text{diag}(\sigma^2))$ and $p = \mathcal{N}(\mathbf{z}; \mathbf{0}, \mathbf{I})$, both factorise over dimensions, so the KL is a sum. For one dimension,

$$\mathbb{E}_q[\log q(z_j) - \log p(z_j)] = \mathbb{E}_q\left[-\frac{1}{2} \log \sigma_j^2 - \frac{(z_j - \mu_j)^2}{2\sigma_j^2} + \frac{z_j^2}{2}\right] = -\frac{1}{2} \log \sigma_j^2 - \frac{1}{2} + \frac{1}{2}(\mu_j^2 + \sigma_j^2).$$

Reason: the $\log 2\pi$ terms cancel; $\mathbb{E}_q[(z_j - \mu_j)^2] = \sigma_j^2$ and $\mathbb{E}_q[z_j^2] = \mu_j^2 + \sigma_j^2$. Summing gives $R = \frac{1}{2} \sum_j (\mu_j^2 + \sigma_j^2 - 1 - \log \sigma_j^2)$: zero exactly when $\mu_j = 0$ and $\sigma_j = 1$, i.e. when the code carries no information about $\mathbf{x}$. Example: $\mu = (1, 0)$, $\sigma^2 = (1, 4)$ gives $\frac{1}{2}[1 + 4 - 1 - \log 4] = 1.307$ nats.

Step 4 — the reconstruction term is a likelihood, and its scale matters. With $p_{\theta}(\mathbf{x} | \mathbf{z}) = \mathcal{N}(\mathbf{x}; \hat{\mathbf{x}}, \sigma_x^2 \mathbf{I})$,

$$-\log p_{\theta}(\mathbf{x} | \mathbf{z}) = \frac{1}{2\sigma_x^2} \|\mathbf{x} - \hat{\mathbf{x}}\|^2 + \frac{n}{2} \log(2\pi\sigma_x^2).$$

Three consequences that trip up most first implementations:

- **σ_x is a KL weight in disguise.** Multiplying J by $2\sigma_x^2$ shows that a VAE with decoder variance σ_x^2 and $\beta = 1$ has the same optimum as the squared error $\|\mathbf{x} - \hat{\mathbf{x}}\|^2$ plus a KL weight $2\sigma_x^2$. “MSE + KL” with no stated σ_x silently assumes $\sigma_x^2 = \frac{1}{2}$ — for pixels in $[0, 1]$, an enormous noise level that makes the code nearly worthless.

- **Sum over pixels, then average over images** — the per-image ELBO is a sum over all n pixels. Averaging the reconstruction over pixels but not the KL changes the relative weight by a factor of n (64 here, $\approx 200,000$ for a 256×256 colour image).
- **A Bernoulli decoder** (binary cross-entropy) is the right likelihood for binary pixels. Using it on grey levels in $[0, 1]$ is a common shortcut but is not a valid density for them, so the resulting “ELBO” is not a bound on anything; say so if you do it.

A continuous density can exceed one, so D can be negative: in the lab, $D = -82.2$ nats per image means the decoder assigns high density to the held-out pixels, which is good.

Step 5 — gradients through the sample. With $\mathbf{z} = \boldsymbol{\mu} + \mathbf{s} \odot \boldsymbol{\epsilon}$, $\mathbf{s} = \exp(\frac{1}{2}\mathbf{l})$, and $\mathbf{g}_z = \partial D / \partial \mathbf{z}$ obtained by backpropagating through the decoder (chapter 11), the chain rule gives

$$\frac{\partial J}{\partial \boldsymbol{\mu}} = \mathbf{g}_z + \beta \boldsymbol{\mu}, \quad \frac{\partial J}{\partial \mathbf{l}} = \frac{1}{2} \mathbf{g}_z \odot \mathbf{s} \odot \boldsymbol{\epsilon} + \frac{\beta}{2} (e^{\mathbf{l}} - \mathbf{1}).$$

Reason: $\partial \mathbf{z} / \partial \boldsymbol{\mu} = \mathbf{I}$; $\partial z_j / \partial l_j = \frac{1}{2} s_j \epsilon_j$; and from step 3, $\partial R / \partial \mu_j = \mu_j$, $\partial R / \partial l_j = \frac{1}{2} (e^{l_j} - 1)$. The encoder’s gradient then flows back from $(\boldsymbol{\mu}, \mathbf{l})$ through its own layers. Two classic bugs live here: using $\exp(\mathbf{l})$ instead of $\exp(\frac{1}{2}\mathbf{l})$ as the standard deviation, and forgetting the $\frac{1}{2}$ in $\partial \mathbf{z} / \partial \mathbf{l}$. The lab’s gradient check (relative error 2×10^{-9}) holds $\boldsymbol{\epsilon}$ fixed so the finite differences compare the same random objective.

Step 6 — rate and distortion, and what β does. Write the loss as $D + \beta R$. Changing β moves along a trade-off curve (figure 10.4b): small β buys low distortion with a high rate (codes carry many nats of information about each image); large β forces codes toward the prior, losing detail. Why “rate”? Averaged over the data, the KL term decomposes (Hoffman and Johnson, 2016) as

$$\frac{1}{m} \sum_i D_{\text{KL}}(q_\phi(\mathbf{z} | \mathbf{x}^{(i)}) \| p(\mathbf{z})) = I_q(\mathbf{x}; \mathbf{z}) + D_{\text{KL}}(q(\mathbf{z}) \| p(\mathbf{z})),$$

where I_q is the mutual information between an image index and its code under the encoder, and $q(\mathbf{z})$ is the aggregate posterior. So R upper-bounds the number of nats the code transmits about the image: it is literally a channel rate.

Which values of β still give a bound. Write $J_\beta = D + \beta R$, with D the full negative log-likelihood of the decoder, including its normalising constant (for a Gaussian decoder, the $\frac{1}{2} \log(2\pi\sigma_x^2)$ term per pixel), and $R \geq 0$ the KL. Then

$$J_\beta = J_1 + (\beta - 1)R,$$

and J_1 is the standard negative ELBO, an upper bound on $-\log p_\theta(\mathbf{x})$ (averaged over the data). For $\beta \geq 1$, $J_\beta \geq J_1$, so J_β is still an upper bound, only a looser one. For $\beta < 1$ there is in general no such guarantee for the original model: J_β can fall below $-\log p_\theta(\mathbf{x})$. Only $\beta = 1$ gives the standard ELBO objective; any $\beta \neq 1$ is a different, legitimate objective that should be named as such, and its value should not be reported as an ELBO.

Step 7 — posterior collapse and switched-off dimensions. The rate term is minimised by $q_\phi(\mathbf{z} | \mathbf{x}) = p(\mathbf{z})$ for every image. If the decoder can model the data without the code — because it is autoregressive and can predict each pixel from the previous ones, or because the data are simple — that is the optimum: the code is ignored and $R = 0$. This is **posterior collapse**. In a milder per-dimension form it is useful: a dimension that

is not worth its KL cost is switched off ($\mu_j \approx 0$, $\sigma_j \approx 1$, $\text{KL} \approx 0$). In the lab, three of eight dimensions are switched off at $\beta = 1$ (figure 10.3b) because three factors of variation do not need eight numbers. Diagnose with KL per dimension and **active units** — dimension j is active if $\text{Var}_{\mathbf{x}}(\mathbb{E}_q[z_j | \mathbf{x}]) > 0.01$. Fixes when collapse is unwanted: anneal β from 0 to 1 early in training, give each dimension “free bits” of KL that are not penalised, or weaken the decoder.

Step 8 — the aggregate posterior and holes in the prior. Training only ever decodes codes drawn from $q_\phi(\mathbf{z} | \mathbf{x}^{(i)})$ for training images. Generation decodes codes from $p(\mathbf{z})$. These agree only if the aggregate posterior $q(\mathbf{z}) = \frac{1}{m} \sum_i q_\phi(\mathbf{z} | \mathbf{x}^{(i)})$ — a mixture of m Gaussians — fills the prior. The second term of step 6 measures the mismatch. Where the prior has mass but $q(\mathbf{z})$ does not — “holes” — the decoder was never trained, and samples are atypical: in the lab, prior samples are dimmer and more diffuse than data (mean maximum pixel 0.52 against 0.73; figure 10.2). Fixes: a learned, more flexible prior (a mixture, a VampPrior); fit a second model to $q(\mathbf{z})$ (two-stage VAEs) — which is exactly what latent diffusion does with a diffusion model (section 10.11).

Step 9 — why samples are blurry. With a fixed-variance Gaussian decoder, the decoder mean $\hat{\mathbf{x}}_\theta(\mathbf{z})$ that minimises squared error is the **average** of all images consistent with $\mathbf{z}$. If the code does not pin down fine detail (texture, exact edge position), the average blurs it. A larger rate (smaller β or σ_x) sharpens reconstructions but enlarges the holes of step 8, which hurts samples. Remedies change the likelihood (perceptual or adversarial losses, as in latent diffusion’s autoencoder), make the latent hierarchical, or hand fine detail to another model.

Step 10 — a tighter bound: importance weighting. For K samples $\mathbf{z}^{(k)} \sim q_\phi(\mathbf{z} | \mathbf{x})$ with weights $w_k = p_\theta(\mathbf{x}, \mathbf{z}^{(k)})/q_\phi(\mathbf{z}^{(k)} | \mathbf{x})$,

$$\mathcal{L}_K = \mathbb{E} \left[\log \frac{1}{K} \sum_{k=1}^K w_k \right] \leq \log \mathbb{E} \left[\frac{1}{K} \sum_{k=1}^K w_k \right] = \log p_\theta(\mathbf{x}).$$

Reason: each w_k has expectation $p_\theta(\mathbf{x})$ under q (importance sampling), and Jensen applies to the average. $\mathcal{L}_1$ is the ELBO; $\mathcal{L}_K$ does not decrease with K (an average of K weights is a less noisy estimate, so less is lost to Jensen) and, when the weights are bounded, tends to $\log p_\theta(\mathbf{x})$. The bound holds for the *expectation*. One draw of $\log \frac{1}{K} \sum_k w_k$ is a random estimate of $\mathcal{L}_K$ and can exceed $\log p_\theta(\mathbf{x})$; estimates computed with different K need not increase monotonically. The importance-sampling argument also needs q to be positive wherever $p_\theta(\mathbf{x}, \mathbf{z})$ is. Use $\mathcal{L}_K$ to *evaluate* VAEs, averaged over many held-out images: the lab’s Monte Carlo estimates, each averaged over 200 held-out images, rise from 74.86 ($K = 1$) to 75.74 nats ($K = 1000$).

Step 11 — a map of generative objectives. Every family answers the same question — how to make p_θ close to p_{data} — with a different tractability trade:

Family	Objective	Likelihood	Sampling	Typical weakness
Autoregressive (PixelCNN, transformers)	exact $\log p = \sum_t \log p(x_t x_{<t})$	exact	slow, sequential	slow sampling; order dependence
Normalising flow	exact $\log p$ via change of variables	exact	fast	invertibility limits architecture

Family	Objective	Likelihood	Sampling	Typical weakness
VAE	ELBO (lower bound)	bound	fast, one pass	blur; prior holes; collapse
GAN	adversarial game (a divergence estimated by a critic)	none	fast	unstable training; mode dropping
Diffusion (chapter 22)	ELBO of a hierarchical VAE with a fixed encoder; a weighted denoising loss	bound	slow, many steps (distillable)	compute at sampling
Latent diffusion	autoencoder + diffusion on codes	bound on codes	moderate	autoencoder limits fidelity

The connection to chapter 9: maximum-likelihood families (rows 1–3, 5) minimise forward KL from the data and so tend to cover all modes; GANs can minimise other divergences and tend to drop modes but produce sharp samples.

Step 12 — towards diffusion. Stack latent variables: $\mathbf{x} \rightarrow \mathbf{z}_1 \rightarrow \mathbf{z}_2 \rightarrow \dots \rightarrow \mathbf{z}_T$, where the “encoder” from each layer to the next is *fixed* — add a little Gaussian noise — and only the decoder from $\mathbf{z}_t$ back to $\mathbf{z}_{t-1}$ is learned. The ELBO of this hierarchical VAE splits into one KL term per step, each between two Gaussians (step 3), and simplifies to a denoising loss. That model is a diffusion model (Ho, Jain and Abbeel, 2020; Kingma and colleagues, 2021). Chapter 22 derives it; everything needed is in this chapter and the last.

10.6 Numerical worked example

One forward pass by hand ($d = 2$ latent dimensions, one pixel, to keep it small). Encoder output for an image: $\boldsymbol{\mu} = (1, 0)$, $\mathbf{l} = (0, \log 4)$, so $\boldsymbol{\sigma} = (1, 2)$ — note $e^{l/2} = 2$, not $e^l = 4$. Draw $\boldsymbol{\epsilon} = (0.5, -1)$: $\mathbf{z} = (1+0.5, 0-2) = (1.5, -2)$. Rate: $R = \frac{1}{2}[(1+1-1-0) + (0+4-1-\log 4)] = \frac{1}{2}[1+1.614] = 1.307$ nats. Suppose the decoder maps $\mathbf{z}$ to $\hat{x} = 0.62$ for a true pixel $x = 0.70$ with $\sigma_x = 0.1$: distortion $D = \frac{(0.08)^2}{2 \cdot 0.01} + \frac{1}{2} \log(2\pi \cdot 0.01) = 0.32 - 1.383 = -1.063$ nats. Loss at $\beta = 1$: $-1.063 + 1.307 = 0.244$.

Gradient for the encoder. Suppose backpropagation gives $\mathbf{g}_z = (0.2, -0.1)$. Then $\partial J / \partial \boldsymbol{\mu} = (0.2 + 1, -0.1 + 0) = (1.2, -0.1)$ and $\partial J / \partial \mathbf{l} = \frac{1}{2}(0.2 \cdot 1 \cdot 0.5, -0.1 \cdot 2 \cdot (-1)) + \frac{1}{2}(e^0 - 1, 4 - 1) = (0.05, 0.1) + (0, 1.5) = (0.05, 1.6)$. The KL part pushes the second dimension’s variance back toward 1; the data part pushes its mean only slightly.

σ_x as β . Training with $\sigma_x = 0.1$ and $\beta = 1$ has the same minimiser as $\|\mathbf{x} - \hat{\mathbf{x}}\|^2 + 0.02 R$ (multiply J by $2\sigma_x^2$): an “MSE + KL” implementation would need a KL weight of 0.02 to match. With weight 1 it would correspond to $\sigma_x \approx 0.71$.

10.7 Small implementation and checks

The lab `labs/ch10-vae/vae.py` is about 150 lines of NumPy with every gradient written by hand: one-hidden-layer tanh encoder and decoder (96 units), 8 latent dimensions, $\sigma_x = 0.1$, Adam, batches of 64, 6,000 steps. The heart of it:

```
mu, lv, h = encode(p, X)
s = np.exp(0.5 * lv); Z = mu + s * eps          # reparameterise: sd is
→ exp(lv / 2)
xhat, g = decode(p, Z)
rec = 0.5 * ((xhat - X)**2).sum(1) / sx**2 + 0.5 * n * np.log(2 * np.pi *
→ sx**2)
kl = 0.5 * (mu**2 + np.exp(lv) - 1 - lv).sum(1)
# backward: d_z from the decoder, then
d_mu = d_z + beta * mu / B
d_lv = d_z * 0.5 * s * eps + beta * 0.5 * (np.exp(lv) - 1) / B
```

Checks, in `test_vae.py`:

1. Hand gradients agree with central differences (fixed ϵ) to 10^{-6} .
2. The closed-form KL matches a Monte Carlo estimate, and the sampled standard deviation is $e^{l/2}$.
3. Training lowers the loss; at least one dimension is switched off and at least three are used; codes predict the spot position linearly with $R^2 > 0.8$.
4. The IWAE estimate with $K = 100$ exceeds the $K = 1$ estimate (an ELBO estimate); per-example refinement does not worsen the ELBO.
5. Raising β lowers the rate and raises the distortion.

Monitoring habits: log D and R separately (their sum hides collapse), KL per dimension, and both reconstructions and prior samples; evaluate on held-out images with fixed noise.

10.8 Experiment and expected observations

Run `python labs/ch10-vae/vae.py` (seed 0; 3,000 training images, 500 held out).

quantity ($\beta = 1$, held out)	value
–ELBO = $D + R$	$-74.9 = -82.2 + 7.29$ nats
KL per dimension	0.96, 0.00, 0.00, 0.00, 1.60, 1.89, 1.56, 1.28
active units	5 of 8
linear R^2 from codes to $(c_x, c_y, \text{brightness})$	0.90, 0.94, 0.14
reconstruction MSE per pixel	0.0013
mean maximum pixel: prior samples	0.52 vs. 0.73
vs. data	
IWAE estimate (200 held-out images, one draw each), $K = 1, 10, 100, 1000$	74.86, 75.57, 75.72, 75.74
–ELBO before / after per-example refinement (amortisation gap)	$-74.87 / -75.28$

β	distortion D	rate R	active units
0.25	–84.2	13.95	8

β	distortion D	rate R	active units
1	-82.2	7.29	5
4	-75.8	4.16	5
16	-60.2	1.82	2

Figure 10.2: Held-out images (top), their reconstructions (middle) and samples from the prior (bottom), $\beta = 1$. Reconstructions are faithful; prior samples are dimmer and more diffuse because some prior codes fall in regions the decoder never saw during training.

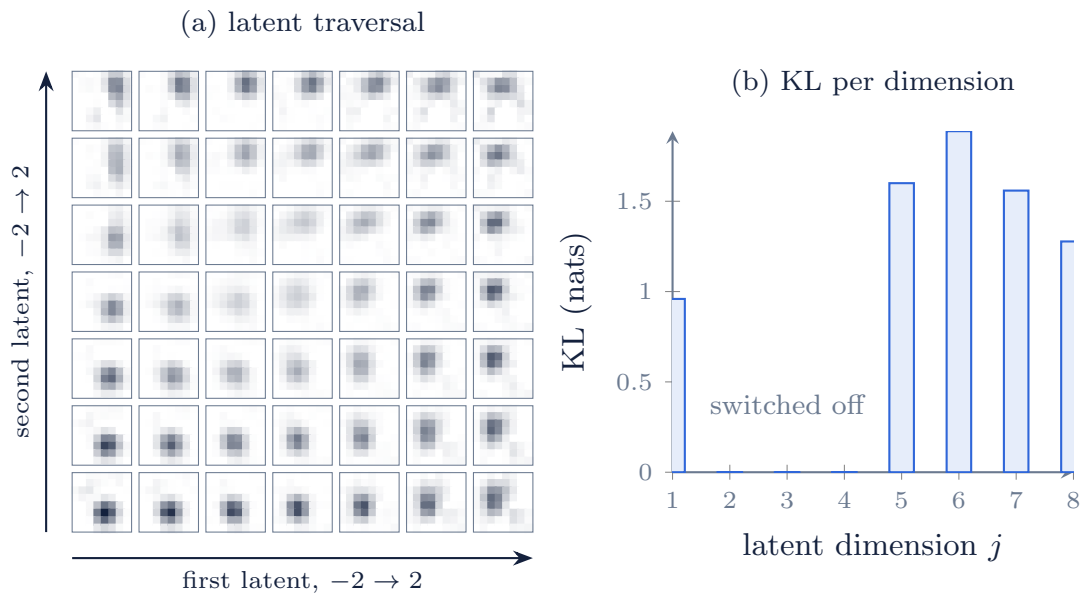


Figure 10.3: (a) Decoding a grid of codes along the two dimensions with the largest KL (others at 0): the spot moves smoothly across the image — the code has learned position without labels. (b) KL per latent dimension on held-out data: three dimensions are switched off.

What to notice:

1. **The code learns position without labels.** Two factors — the spot's x and y — are recovered with linear R^2 of 0.90 and 0.94. Brightness ($R^2 = 0.14$) is mostly *not* encoded: at $\sigma_x = 0.1$ the brightness range costs less in distortion than it would in rate. What a VAE encodes is decided by the trade-off, not by what we find interesting.
2. **Unused dimensions switch off.** Five dimensions carry the information; three sit

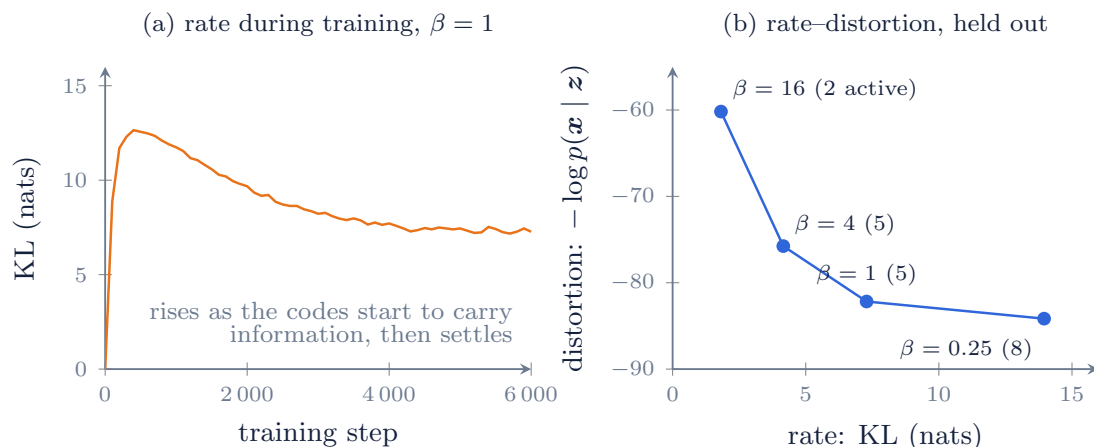


Figure 10.4: (a) The rate during training at $\beta = 1$: it rises as the codes begin to carry information, then settles as unused dimensions switch off. (b) Held-out rate against distortion for four values of β , with the number of active units: one model family, four different compromises.

exactly at the prior. More latent dimensions than needed is mostly harmless; fewer is not.

3. **Reconstructions are better than samples.** The decoder reproduces held-out images with per-pixel MSE 0.0013, but prior samples are atypical. Always show both, never only reconstructions.
4. **β moves along a curve, not to a better model.** $\beta = 16$ uses 1.8 nats and two dimensions and reconstructs poorly; $\beta = 0.25$ uses all eight dimensions. Which is “best” depends on the goal: compression, sampling, or a compact code for a downstream task.
5. **The ELBO is a loose-ish bound; the encoder is nearly optimal.** Importance weighting raises the bound by 0.9 nats, so the diagonal-Gaussian approximation costs about that much; per-example refinement recovers only 0.4 nats, so the encoder’s amortisation gap is small here.

10.9 Moving pieces: diagnosing and fixing

Moving piece	Symptom when it is wrong	How to confirm	Fix
Decoder likelihood / σ_x	blurry reconstructions (too large) or overfitting codes, poor samples (too small)	vary σ_x ; check D against R	state the likelihood; learn σ_x or set it from the noise level
Loss normalisation	KL dominates or vanishes when image size changes	write the per-image ELBO on paper	sum over pixels, average over images, consistently
β / KL weight	collapse (too large), prior holes (too small)	rate–distortion plot; active units	choose β for the goal; anneal from 0; free bits

Moving piece	Symptom when it is wrong	How to confirm	Fix
Posterior collapse	$R \approx 0$; reconstructions independent of input	KL per dimension; decode with shuffled codes	KL annealing; free bits; weaker or non-autoregressive decoder
Latent dimension d	all dimensions active and still poor D (too few)	KL per dimension	increase d ; extra dimensions switch off cheaply
Log-variance head	NaN or huge losses; σ exploding	min/max of $\mathbf{l}$	clamp $\mathbf{l}$ to a range; initialise its weights small
Reparameterisation	slow learning; wrong posterior width	gradient check with fixed ϵ ; sample sd vs. $e^{l/2}$	use $\exp(\frac{1}{2}\mathbf{l})$; resample ϵ each step
Prior vs. aggregate posterior	good reconstructions, bad samples	fit a Gaussian to encoder means; compare with $\mathcal{N}(\mathbf{z}; \mathbf{0}, \mathbf{I})$	learned prior; two-stage model; latent diffusion
Evaluation	claims based on reconstructions only	show prior samples, held-out ELBO and IWAE	report D , R , IWAE, sample quality metrics, nearest training neighbours

First things to try: (1) gradient-check the reparameterised loss with fixed noise; (2) plot D and R separately over training; (3) print KL per dimension; (4) look at prior samples next to reconstructions; (5) state σ_x and the normalisation of both terms.

10.10 Application, limitations and failure cases

Application (educational case study). For the leaf-patch archive, train a convolutional VAE on 64×64 patches with a moderate β . Use the encoder means as a compact code for retrieval and visualisation; check which factors they capture by regressing labelled attributes (lesion size, position, colour) on the codes after training. Use the decoder to generate *variations* of real patches — encode, perturb one active dimension, decode — rather than free samples from the prior, which may be atypical (step 8).

Failure case — **likelihood is not typicality**. It is tempting to flag unusual images by a low ELBO. Nalisnick and colleagues (2019) showed that deep generative models (VAEs, flows, autoregressive models) trained on one dataset can assign *higher* likelihood to a different, simpler dataset (models trained on CIFAR-10 rate SVHN digits as more likely). Likelihood is dominated by low-level statistics such as smoothness, not by semantic content. Validate any anomaly detector on real anomalies and on shifted but normal data (chapter 5).

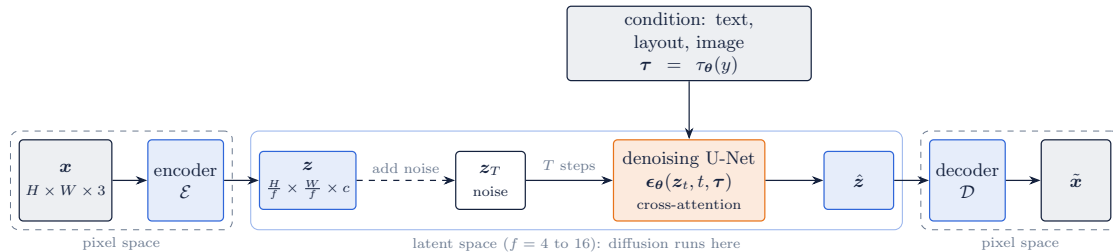
Limitations:

- The diagonal Gaussian posterior and standard-normal prior are strong assumptions; their gaps show up as blur and prior holes.
- “Disentanglement” by raising β is not guaranteed: without supervision or inductive biases, axis-aligned factors are not identifiable (Locatello and colleagues, 2019).

- Reconstruction quality says little about generation quality, and neither says how useful the codes are for a downstream task (chapter 18).

10.11 Bridge to a research paper

Rombach, Blattmann, Lorenz, Esser and Ommer (2022), *High-resolution image synthesis with latent diffusion models* (CVPR 2022).



stage 1: train $\mathcal{E}, \mathcal{D}$ as an autoencoder with a KL (or vector-quantisation) penalty of tiny weight, plus perceptual and adversarial losses — a VAE with $\beta \approx 10^{-6}$, chosen for reconstruction, not for sampling.
 stage 2: freeze $\mathcal{E}, \mathcal{D}$ and train the diffusion model on $\mathbf{z} = \mathcal{E}(\mathbf{x})$; to generate, sample $\mathbf{z}_T$, denoise, decode.

Figure 10.5: Latent diffusion, redrawn after Fig. 3 of Rombach et al. (2022). An autoencoder compresses images by a factor f in each spatial dimension; a diffusion model with a denoising U-Net learns the distribution of the codes, conditioned through cross-attention; the decoder maps generated codes back to pixels. Paper: [arXiv:2112.10752](https://arxiv.org/abs/2112.10752).

- **Problem:** pixel-space diffusion models produce excellent images but spend most of their compute modelling imperceptible detail; training and sampling at high resolution are very expensive.
- **Mechanism:** split the work. Stage 1 trains an autoencoder whose encoder outputs a diagonal Gaussian (a VAE) with a *tiny* KL weight ($\approx 10^{-6}$) — enough to keep the latent space smooth and bounded — or a vector-quantised code, with perceptual and adversarial reconstruction losses so that reconstructions are sharp (step 9). Stage 2 trains a diffusion model on the frozen codes. Generation samples codes by diffusion and decodes them once.
- **Central claim:** with downsampling factors $f = 4$ to 8 , image quality matches or beats pixel-space diffusion at a fraction of the compute; cross-attention conditioning handles text, layouts and images.
- **How it uses this chapter:** the VAE’s two weaknesses are separated out. Blur is handled by the reconstruction loss; prior holes (step 8) disappear because sampling uses a learned model of the aggregate posterior, not $\mathcal{N}(\mathbf{z}; \mathbf{0}, \mathbf{I})$. The tiny β is a deliberate choice of a high-rate point on figure 10.4b.
- **Assumptions to question:** the autoencoder bounds achievable fidelity (small text and faces suffer); the KL weight and f trade compression against detail; evaluation by FID depends on a particular feature extractor.
- **Affordable experiment:** with the lab’s VAE at $\beta = 0.25$ (high rate), fit a Gaussian mixture (chapter 8) with 10 components to the encoder means of the training images, sample codes from the mixture instead of the prior, and decode. Compare the mean maximum pixel and visual quality of these samples with prior samples — a two-stage model in miniature.

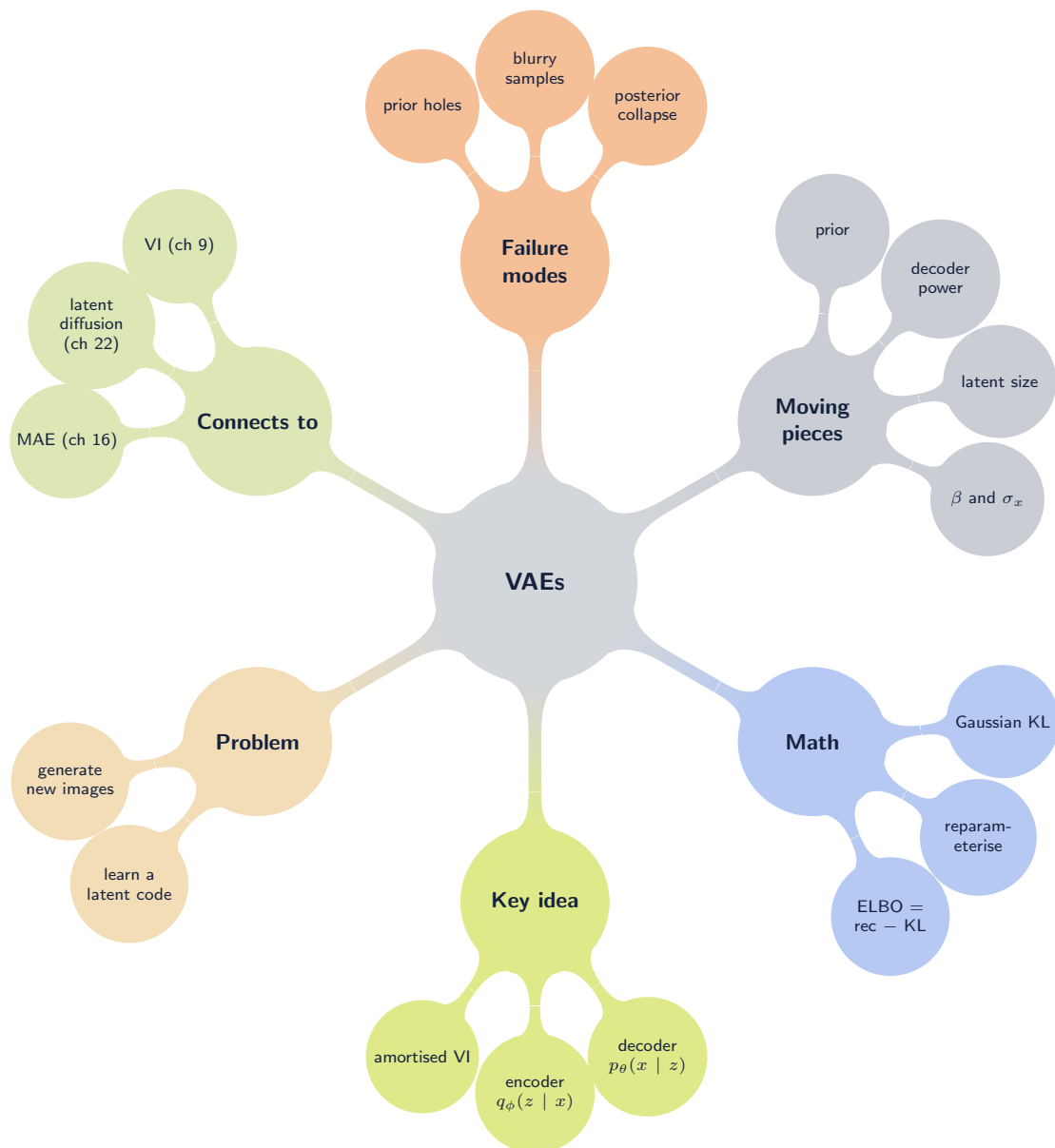


Figure 10.6: Memory map of chapter 10.

10.12 Memory map

Summary. A VAE is a latent-variable model with a neural decoder, trained by amortised variational inference: an encoder outputs a diagonal Gaussian over codes, a reparameterised sample is decoded, and the loss is distortion (a stated likelihood of the pixels) plus rate (a closed-form KL to the prior). β and the decoder variance choose a point on the rate–distortion curve. Unused dimensions switch off; with powerful decoders the whole code can collapse. Reconstructions and prior samples answer different questions, and holes between the aggregate posterior and the prior explain poor samples. The remedies — better likelihoods, learned priors, two-stage models — lead directly to latent diffusion, and stacking fixed noising steps turns the VAE’s ELBO into the diffusion loss of chapter 22.

10.13 Exercises

1. *Conceptual.* An encoder emits a log-variance $l = \log 4$. What standard deviation should be used for sampling?
2. *Conceptual.* Why is the decoder mean not the same as a sample from the decoder, and which should be shown as “a generated image”?
3. *Conceptual.* What is wrong with calling a $\beta = 4$ training loss “the ELBO”?
4. *Derivation.* Derive the KL between $\mathcal{N}(\mathbf{z}; \boldsymbol{\mu}, \boldsymbol{\Sigma})$ and $\mathcal{N}(\mathbf{z}; \mathbf{0}, \mathbf{I})$ for a full covariance, and check it reduces to step 3 for a diagonal one.
5. *Derivation.* Show that for a linear decoder $\hat{\mathbf{x}} = \mathbf{W}\mathbf{z} + \mathbf{b}$ and Gaussian likelihood the VAE’s generative model is probabilistic PCA (chapter 8, step 12), and that its marginal likelihood is available in closed form.
6. *Derivation.* Prove that $\mathcal{L}_K \leq \mathcal{L}_{K+1}$ for the importance-weighted bound.
7. *Implementation.* Add KL annealing (β rising linearly from 0 to 1 over the first 2,000 steps) and compare the number of active units with and without it.
8. *Implementation.* Carry out the two-stage experiment of section 10.11.
9. *Interpretation.* A colleague reports a VAE with “excellent” generation and shows reconstructions of training images. What would you ask for?
10. *Open question.* Latent diffusion uses $\beta \approx 10^{-6}$. Why not $\beta = 1$? What would happen to the diffusion model’s job, and to reconstruction quality?

10.13.1 Answer discussions

1. $e^{l/2} = 2$. Using $e^l = 4$ samples four times too widely in standard deviation.
2. The mean averages out pixel noise; a sample adds independent noise to each pixel (σ_x), which for images looks like speckle. Papers show decoder means (or modes); say which.
3. The KL has been reweighted, so it is not the standard ELBO. Since $J_4 = J_1 + 3R \geq J_1$, its value is still an upper bound on $-\log p(\mathbf{x})$, but a looser one, and the parameters it selects differ from the ELBO’s. Call it a β -weighted objective and do not compare its value with ELBOs.
4. $\frac{1}{2}[\text{Tr}\boldsymbol{\Sigma} + \boldsymbol{\mu}^\top \boldsymbol{\mu} - d - \log \det \boldsymbol{\Sigma}]$; with diagonal $\boldsymbol{\Sigma}$ the trace and log determinant split into sums.
5. $\mathbf{x} = \mathbf{W}\mathbf{z} + \mathbf{b} + \text{noise}$ with Gaussian $\mathbf{z}$ and noise gives $\mathbf{x} \sim \mathcal{N}(\mathbf{x}; \mathbf{b}, \mathbf{W}\mathbf{W}^\top + \sigma_x^2 \mathbf{I})$; the posterior is Gaussian too, so a Gaussian encoder can be exact (though a *diagonal* one may not be).
6. Write the average of $K + 1$ weights as the average over the $K + 1$ leave-one-out averages of K weights, then apply Jensen’s inequality to the concave log.
7. Annealing usually keeps more dimensions active early and can end with one or two

more active units; at the end of training the objective is the same, so the final difference can be small.

8. Mixture samples should be brighter and closer to real spots (mean maximum pixel nearer 0.73), because they avoid the prior’s holes.
9. Prior samples (many, unselected), held-out ELBO or IWAE, a sample-quality metric, and nearest training neighbours of samples to rule out memorisation.
10. With $\beta = 1$ the autoencoder would discard detail to keep the rate low (blurry reconstructions) and its codes would already be close to $\mathcal{N}(\mathbf{z}; \mathbf{0}, \mathbf{I})$; the diffusion model would have little to do, and the decoder would cap quality. A tiny β keeps detail and leaves the distribution of codes to the diffusion model.

10.14 Self-check and common mistakes

- ☐ I can draw the VAE’s training and generation paths and say where the encoder is used.
- ☐ I can derive the diagonal Gaussian KL and the reparameterised gradients.
- ☐ I can explain what β and σ_x do in rate–distortion terms.
- ☐ I can diagnose posterior collapse and prior holes from logged quantities.
- ☐ I can place VAEs, flows, GANs, autoregressive models and diffusion in one table.

Common mistakes: sampling with $\exp(l)$ instead of $\exp(l/2)$; averaging reconstruction over pixels but not the KL; “MSE + KL” with no stated decoder variance; calling a β -weighted loss an ELBO; reporting only reconstructions; reading a small KL as success; using binary cross-entropy on grey levels and calling the result a likelihood; using ELBOs from different models as likelihoods; treating low likelihood as evidence that an input is unusual.

References

- I. Goodfellow, Y. Bengio and A. Courville. *Deep Learning*. MIT Press, 2016, sections 14 (autoencoders) and 20.10.3 (variational autoencoders).
- D. P. Kingma and M. Welling. Auto-encoding variational Bayes. In *ICLR*, 2014. arXiv:1312.6114.
- D. J. Rezende, S. Mohamed and D. Wierstra. Stochastic backpropagation and approximate inference in deep generative models. In *ICML*, 2014.
- P. Dayan, G. E. Hinton, R. M. Neal and R. S. Zemel. The Helmholtz machine. *Neural Computation*, 7(5):889–904, 1995.
- G. E. Hinton and R. R. Salakhutdinov. Reducing the dimensionality of data with neural networks. *Science*, 313(5786):504–507, 2006.
- S. R. Bowman, L. Vilnis, O. Vinyals, A. M. Dai, R. Jozefowicz and S. Bengio. Generating sentences from a continuous space. In *CoNLL*, 2016.
- Y. Burda, R. Grosse and R. Salakhutdinov. Importance weighted autoencoders. In *ICLR*, 2016.
- M. D. Hoffman and M. J. Johnson. ELBO surgery: yet another way to carve up the variational evidence lower bound. NIPS Workshop on Advances in Approximate Bayesian Inference, 2016.
- I. Higgins et al. β -VAE: learning basic visual concepts with a constrained variational framework. In *ICLR*, 2017.
- A. A. Alemi, B. Poole, I. Fischer, J. V. Dillon, R. A. Saurous and K. Murphy. Fixing a broken ELBO. In *ICML*, 2018.
- A. van den Oord, O. Vinyals and K. Kavukcuoglu. Neural discrete representation learning. In *NeurIPS*,

2017.

- E. Nalisnick, A. Matsukawa, Y. W. Teh, D. Gorur and B. Lakshminarayanan. Do deep generative models know what they don't know? In *ICLR*, 2019.
- F. Locatello et al. Challenging common assumptions in the unsupervised learning of disentangled representations. In *ICML*, 2019.
- A. Vahdat and J. Kautz. NVAE: a deep hierarchical variational autoencoder. In *NeurIPS*, 2020.
- J. Ho, A. Jain and P. Abbeel. Denoising diffusion probabilistic models. In *NeurIPS*, 2020.
- R. Rombach, A. Blattmann, D. Lorenz, P. Esser and B. Ommer. High-resolution image synthesis with latent diffusion models. In *CVPR*, 2022. arXiv:2112.10752.

Part III

Deep learning and visual tasks

Chapter 11

Backpropagation and optimization

Prerequisites: chapter 1 (matrix calculus, finite differences, conditioning); chapter 3 (logistic regression and its gradient).

Learning outcomes: after this chapter you can

1. draw a network as a computational graph and derive every gradient of a two-layer classifier by the chain rule, in matrix form, with shapes;
2. explain why reverse-mode differentiation costs about as much as one forward pass, and implement a small automatic-differentiation engine;
3. analyse gradient descent on a quadratic — the $2/\lambda_{\max}$ limit, the role of the condition number — and explain what momentum and Adam change;
4. derive the He initialisation and explain vanishing and exploding signals, normalisation and residual connections;
5. debug a training run with a fixed sequence of checks, and interpret a network that memorises random labels.

11.1 A concrete problem

A lesion classifier has three classes (healthy, early blight, late blight) and two image features that are not linearly separable — in the lab, three interleaved spirals. Logistic regression (chapter 3) cannot separate them. A network with one hidden layer of 64 ReLU units can: it bends the feature space before the final linear classifier.

Training it raises three questions that this chapter answers and that every later chapter assumes:

1. **How do we get the gradient** of the loss with respect to thousands (later, billions) of weights, cheaply and correctly?
2. **How do we use it?** Which step size, which optimiser, and why does a run sometimes diverge or stall?
3. **Why does depth make it harder**, and what do initialisation, normalisation and residual connections do about it?

The answers are general — the same backpropagation trains the VAE of chapter 10, the convolutional networks of chapter 12 and the transformers of chapter 14.

11.2 How we got here

Steepest descent and stochastic approximation (1847–1951). Cauchy proposed gradient descent in 1847. Herbert Robbins and Sutton Monro (1951) showed that noisy gradient steps with decreasing step sizes still converge — stochastic gradient descent (SGD). Boris Polyak (1964) added momentum (“heavy ball”), and Yurii Nesterov (1983) an accelerated variant with optimal rates on convex problems.

Reverse-mode differentiation (1970–1986). Seppo Linnainmaa (1970) described reverse-mode automatic differentiation in his master’s thesis: the gradient of a scalar output with respect to all inputs, at a cost proportional to one evaluation. Paul Werbos (1974) proposed applying it to neural networks. Rumelhart, Hinton and Williams (1986) demonstrated “backpropagation” learning useful hidden representations, and the method spread. Earlier, the perceptron (Rosenblatt, 1958) had trained only one layer, and Minsky and Papert (1969) had shown what one layer cannot do.

Why deep nets were hard (1991–2010). Sepp Hochreiter (1991) analysed vanishing and exploding gradients in recurrent networks; the same products of Jacobians plague deep feed-forward networks. Xavier Glorot and Yoshua Bengio (2010) derived an initialisation that keeps signal variance constant for symmetric activations, and recommended ReLU-like units over saturating sigmoids.

The modern toolkit (2011–2019). AdaGrad (Duchi, Hazan and Singer, 2011), RMSProp (Hinton’s lecture notes, 2012) and Adam (Kingma and Ba, 2014) adapted step sizes per coordinate. He, Zhang, Ren and Sun (2015) derived the initialisation for ReLU networks — this chapter’s paper bridge. Batch normalisation (Ioffe and Szegedy, 2015), residual connections (He and colleagues, 2016; chapter 12) and layer normalisation (Ba, Kiros and Hinton, 2016) made depth routine. Goyal and colleagues (2017) showed how to scale learning rate with batch size using warm-up; Loshchilov and Hutter introduced cosine schedules (2017) and decoupled weight decay, AdamW (2019).

Software (2010–2018). Theano (2010), TensorFlow (2015), PyTorch (2017) and JAX (2018) made reverse-mode differentiation automatic. Writing gradients by hand became an exercise — but debugging still requires knowing what the engine computes, which is why the lab builds one in 60 lines.

11.3 Assumptions and notation

Symbol	Meaning
$\mathbf{X} \in \mathbb{R}^{B \times n}$	a minibatch, one example per row
$\mathbf{W}_1 \in \mathbb{R}^{n \times h}$, $\mathbf{b}_1$; $\mathbf{W}_2 \in \mathbb{R}^{h \times K}$, $\mathbf{b}_2$	weights and biases
$\mathbf{S} = \mathbf{XW}_1 + \mathbf{1b}_1^\top$, $\mathbf{H} = \text{ReLU}(\mathbf{S})$	hidden pre-activations and activations, $B \times h$
$\mathbf{A} = \mathbf{HW}_2 + \mathbf{1b}_2^\top$, $\mathbf{P} = \text{softmax}(\mathbf{A})$	logits and probabilities, $B \times K$ (softmax per row)
$\mathbf{Y}$	one-hot targets, $B \times K$
$\mathbf{G}_Z = \partial L / \partial \mathbf{Z}$	gradient of the scalar loss with respect to any array $\mathbf{Z}$; same shape as $\mathbf{Z}$
η	learning rate (step size)
$\boldsymbol{\theta}$	all parameters; $\mathbf{g}_t$ a (stochastic) gradient at step t
$\kappa = \lambda_{\max} / \lambda_{\min}$	condition number of a Hessian

Symbol	Meaning
--------	---------

Assumptions: the loss is a scalar average over the minibatch; functions are differentiable almost everywhere (ReLU's kink at zero is assigned derivative 0, which is harmless); minibatches are sampled uniformly, so their mean gradient is an unbiased estimate of the full-batch gradient.

11.4 Mathematical formulation

The model and loss.

$$\mathbf{H} = \text{ReLU}(\mathbf{X}\mathbf{W}_1 + \mathbf{1}\mathbf{b}_1^\top), \quad \mathbf{A} = \mathbf{H}\mathbf{W}_2 + \mathbf{1}\mathbf{b}_2^\top, \quad L = -\frac{1}{B} \sum_{i=1}^B \log P_{i,y^{(i)}}, \quad P_{ik} = \frac{e^{A_{ik}}}{\sum_j e^{A_{ij}}}.$$

The backward pass (derived in section 11.5):

$$\mathbf{G}_A = \frac{1}{B}(\mathbf{P} - \mathbf{Y}), \quad \mathbf{G}_{W_2} = \mathbf{H}^\top \mathbf{G}_A, \quad \mathbf{G}_H = \mathbf{G}_A \mathbf{W}_2^\top,$$

$$\mathbf{G}_S = \mathbf{G}_H \odot \mathbb{K}[\mathbf{S} > 0], \quad \mathbf{G}_{W_1} = \mathbf{X}^\top \mathbf{G}_S,$$

with bias gradients the column sums of $\mathbf{G}_A$ and $\mathbf{G}_S$.

Optimisers. With gradient $\mathbf{g}_t$ of a minibatch loss:

$$\begin{aligned} \text{SGD: } \boldsymbol{\theta}_{t+1} &= \boldsymbol{\theta}_t - \eta \mathbf{g}_t \\ \text{heavy-ball momentum: } \mathbf{m}_t &= \beta \mathbf{m}_{t-1} + \mathbf{g}_t, \quad \boldsymbol{\theta}_{t+1} = \boldsymbol{\theta}_t - \eta \mathbf{m}_t \\ \text{Adam: } \mathbf{m}_t &= \beta_1 \mathbf{m}_{t-1} + (1 - \beta_1) \mathbf{g}_t, \quad \mathbf{v}_t = \beta_2 \mathbf{v}_{t-1} + (1 - \beta_2) \mathbf{g}_t^{\odot 2}, \\ \boldsymbol{\theta}_{t+1} &= \boldsymbol{\theta}_t - \eta \frac{\mathbf{m}_t / (1 - \beta_1^t)}{\sqrt{\mathbf{v}_t / (1 - \beta_2^t) + \epsilon}} \quad (\text{elementwise}). \end{aligned}$$

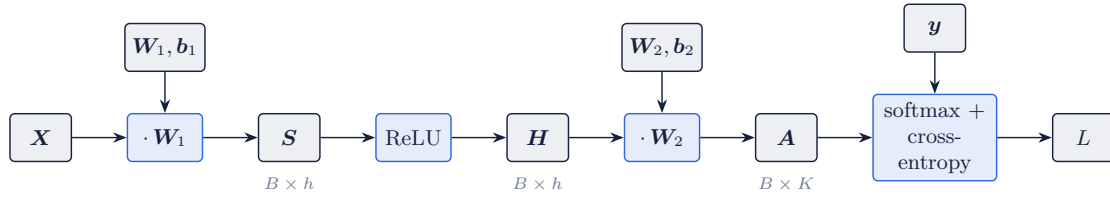
11.5 Derivation, with reasons for each step

Step 1 — without nonlinearities, depth adds nothing. $\mathbf{X}\mathbf{W}_1\mathbf{W}_2 \cdots \mathbf{W}_L = \mathbf{X}\mathbf{W}$ for $\mathbf{W} = \prod_l \mathbf{W}_l$: any stack of linear (or affine) layers is one linear map. ReLU makes the network piecewise linear, with a different linear map in each region where the pattern of active units is fixed — and the number of such regions can grow exponentially with depth. That is the source of the spirals classifier's curved boundary.

Step 2 — the chain rule on a computational graph. Write the computation as a directed acyclic graph whose nodes are arrays and whose edges are simple functions. If L depends on $\mathbf{Z}$ only through its children $\mathbf{U}_1, \dots, \mathbf{U}_r$, then

$$\mathbf{G}_Z = \sum_{c=1}^r \left(\frac{\partial U_c}{\partial \mathbf{Z}} \right)^\top \mathbf{G}_{U_c}.$$

forward: compute and store



backward: chain rule, right to left

$$\begin{aligned}
 G_S = G_H \odot \mathbb{I}[S > 0] &\leftarrow G_H = G_A W_2^\top \leftarrow G_A = \frac{1}{B}(P - Y) \leftarrow 1 \\
 G_{W_1} = X^\top G_S &\quad G_{b_1} = \sum_i (G_S)_{i,:} \\
 G_{W_2} = H^\top G_A &\quad G_{b_2} = \sum_i (G_A)_{i,:}
 \end{aligned}$$

each backward step multiplies by a local Jacobian and reuses a value stored on the forward pass (H, S, X, P): the cost is a small constant times the forward pass, whatever the number of parameters.

Figure 11.1: The two-layer classifier as a computational graph. The forward pass computes and stores intermediate arrays; the backward pass applies the chain rule from the loss back to every parameter, multiplying by local Jacobians and reusing the stored arrays.

Reason: the multivariable chain rule; contributions along different paths add. Processing nodes in reverse topological order guarantees every G_{U_c} is ready when needed. The Jacobians are never formed: each node only needs a rule for the product “Jacobian-transpose times incoming gradient” (a **vector–Jacobian product**). For $U = ZW$ these are $G_Z = G_U W^\top$ and $G_W = Z^\top G_U$ — check: shapes $(B \times h)(h \times n)$ and $(n \times B)(B \times h)$. For an elementwise $U = f(Z)$: $G_Z = G_U \odot f'(Z)$. For a bias broadcast to B rows: sum the incoming gradient over rows (the bias was used B times). The lab’s **Tensor** class is exactly these rules plus a topological sort.

Step 3 — why reverse mode is cheap. Each node’s backward rule costs about the same as its forward computation (a matrix product for a matrix product), so the whole gradient costs a small constant times one forward pass — typically $2\text{--}3\times$ — **regardless of the number of parameters**. Forward-mode differentiation (propagating derivatives from inputs) would need one pass per parameter. Finite differences need two evaluations per parameter and lose precision (chapter 1). The price of reverse mode is **memory**: stored activations (X, S, H, P here) grow with depth and batch size. Activation checkpointing trades memory back for recomputation.

Step 4 — the softmax cross-entropy gradient. For one row with logits $\mathbf{a}$ and target class c , $\ell = -a_c + \log \sum_k e^{a_k}$, so

$$\frac{\partial \ell}{\partial a_j} = -\mathbb{I}[j = c] + \frac{e^{a_j}}{\sum_k e^{a_k}} = p_j - \mathbb{I}[j = c].$$

Reason: differentiate the log-sum-exp. Averaging over the batch gives $G_A = (P - Y)/B$ — “prediction minus target”, as for every exponential-family model (chapter 6). Each row sums to zero, consistent with the invariance of softmax to adding a constant to all logits. Computing the combined expression avoids forming the softmax Jacobian $\partial p_i / \partial a_j = p_i(\mathbb{I}[i = j] - p_j)$ and is numerically stable when written with log-sum-exp.

Step 5 — through the layers. Apply step 2’s rules right to left: $G_{W_2} = H^\top G_A$, $G_H = G_A W_2^\top$, $G_S = G_H \odot \mathbb{I}[S > 0]$, $G_{W_1} = X^\top G_S$. The ReLU mask is where learning

can stop: a unit whose pre-activation is negative for every example passes no gradient to its incoming weights and can stay **dead** for ever. Large learning rates and large negative biases cause dead units; leaky ReLU and GELU always pass some gradient.

Step 6 — gradient descent on a quadratic: the $2/\lambda_{\max}$ limit. Near a minimum, a loss looks like $f(\mathbf{w}) = \frac{1}{2}\mathbf{w}^\top \mathbf{H}\mathbf{w}$. Gradient descent gives $\mathbf{w}_{t+1} = (\mathbf{I} - \eta\mathbf{H})\mathbf{w}_t$. In the eigenbasis of $\mathbf{H}$ each coordinate evolves independently: $u_{t+1} = (1 - \eta\lambda_i)u_t$. So:

- **Stability** requires $|1 - \eta\lambda_i| < 1$ for all i , i.e. $\eta < 2/\lambda_{\max}$. Above it, the steepest direction oscillates with growing amplitude: divergence. The lab finds the norm after 100 steps is 0.08 at $\eta = 0.079$ and 11.8 at $\eta = 0.081$ for $\lambda_{\max} = 25$.
- **Speed** is set by the slowest direction, $|1 - \eta\lambda_{\min}|$. The best fixed step, $\eta = 2/(\lambda_{\min} + \lambda_{\max})$, gives contraction $(\kappa - 1)/(\kappa + 1)$ per step — 0.92 for $\kappa = 25$, close to 1 for badly conditioned problems.

The learning rate is therefore bounded by the **sharpest** direction and its progress by the **flattest**. That single fact explains most learning-rate behaviour: why runs diverge abruptly, why loss curves have a fast phase then a slow one, and why preconditioning and normalisation help (they reduce κ).

Step 7 — momentum. Heavy-ball momentum averages recent gradients: in a valley, components along the floor accumulate while oscillations across it cancel. On the quadratic, with $\eta = 4/(\sqrt{\lambda_{\max}} + \sqrt{\lambda_{\min}})^2$ and $\beta = ((\sqrt{\kappa} - 1)/(\sqrt{\kappa} + 1))^2$ (Polyak), the contraction per step becomes $(\sqrt{\kappa} - 1)/(\sqrt{\kappa} + 1)$: 0.67 for $\kappa = 25$. The dependence on κ becomes a dependence on $\sqrt{\kappa}$ — 66 steps to $f < 10^{-3}$ for gradient descent, 22 for momentum in the lab (figure 11.2).

Step 8 — Adam: per-coordinate scaling and bias correction. Adam divides an average gradient by a running root-mean-square of the gradient, coordinate by coordinate. If the curvature in coordinate i is roughly proportional to its typical gradient size, this is a cheap diagonal preconditioner, and it makes the step size roughly invariant to rescaling each parameter. Bias correction: with $\mathbf{m}_0 = \mathbf{0}$, $\mathbb{E}[\mathbf{m}_t] = (1 - \beta_1^t)\mathbb{E}[\mathbf{g}]$ for a stationary gradient, so dividing by $1 - \beta_1^t$ removes the early shrinkage. Adam’s steps have magnitude about η per coordinate regardless of the gradient’s scale — a reason it tolerates poorly scaled problems, and a reason it keeps “jittering” near a minimum unless η decays (figure 11.2b). Two practical notes: an L2 penalty added to the loss is rescaled by Adam’s denominator and so does not act as weight decay; **AdamW** applies decay directly to the weights. And Adam fixes none of: a wrong loss, leaked data, a bad split.

Step 9 — minibatch noise, batch size and schedules. A minibatch gradient is unbiased with variance $\propto 1/B$. Noise is not only a cost: it lets SGD escape sharp regions and acts as a regulariser. For small learning rates, SGD with step η and batch B behaves like a process whose noise scale is about η/B , which motivates the **linear scaling rule** — multiply η by k when multiplying B by k — valid until η approaches the stability limit of step 6. **Warm-up** (increasing η over the first steps) avoids early divergence while curvature is high and Adam’s statistics are unreliable; **decay** (step, cosine) shrinks the noise ball at the end of training.

Step 10 — signal propagation and the He initialisation. Consider layer l with fan-in n : $s_j = \sum_{k=1}^n w_{jk}h_k$ with independent zero-mean weights of variance σ_w^2 , independent of the inputs. Then

$$\text{Var}[s_j] = n \sigma_w^2 \mathbb{E}[h_k^2].$$

Reason: a sum of n independent zero-mean products, each with variance $\mathbb{E}[w^2]\mathbb{E}[h^2]$. For $h = \text{ReLU}(s')$ with s' symmetric about zero, half of the second moment survives: $\mathbb{E}[h^2] = \frac{1}{2}\text{Var}[s']$. So $\text{Var}[s^{(l)}] = \frac{n}{2}\sigma_w^2 \text{Var}[s^{(l-1)}]$, and over L layers the variance is multiplied by $(\frac{n}{2}\sigma_w^2)^L$. Keeping it constant requires

$$\sigma_w^2 = \frac{2}{n} \quad (\text{He et al., 2015}).$$

Xavier's $1/n$ was derived for symmetric activations (tanh near zero); with ReLU it shrinks the signal by $\sqrt{1/2}$ per layer — 3.5×10^{-5} after 30 layers in the lab — and a scale of 0.01 shrinks it to 10^{-29} . The same argument applied to the backward pass, where $\mathbf{G}_H$ is multiplied by $\mathbf{W}^\top$ and masked, gives the same condition with fan-out: gradients vanish or explode geometrically with depth unless the per-layer gain is one (figure 11.4).

Step 11 — normalisation and residual connections. Initialisation sets the gain at step 0; training changes the weights. Two architectural fixes keep signals healthy throughout:

- **Normalisation** rescales activations to zero mean and unit variance — per channel over the batch (batch norm) or per example over features (layer norm, chapter 14) — followed by a learned scale and shift. It removes the dependence of each layer's input scale on all earlier weights and lets larger learning rates be used. Batch norm behaves differently in training (batch statistics) and evaluation (running averages): forgetting to switch modes is a classic bug.
- **Residual connections** $\mathbf{h}_{l+1} = \mathbf{h}_l + F(\mathbf{h}_l)$ make the Jacobian $\mathbf{I} + \partial F/\partial \mathbf{h}$: the identity path carries signal and gradient through any depth even if F is poorly scaled. With the branch scaled by $1/\sqrt{L}$, the lab's 30-layer network keeps both activations and gradients at order one even with Xavier initialisation. Chapter 12 develops residual networks.

Step 12 — capacity and memorisation. A network with more parameters than examples can fit any labelling, including random ones (Zhang and colleagues, 2017). The lab's network reaches 100% training accuracy on both true and shuffled labels; only validation accuracy tells them apart (figure 11.5). Two lessons: training loss is not evidence of learning anything general; and what a network learns first is the structure that is easiest to fit — true labels are fitted in 90 epochs, random ones in 388.

11.6 Numerical worked example

A scalar chain with every derivative visible. $h = \max(0, wx)$, $\hat{y} = vh$, $L = \frac{1}{2}(\hat{y} - y)^2$ with $x = 2$, $w = 0.5$, $v = 3$, $y = 1$. Forward: $wx = 1$, $h = 1$, $\hat{y} = 3$, $L = 2$. Backward: $\partial L/\partial \hat{y} = \hat{y} - y = 2$; $\partial L/\partial v = 2 \cdot h = 2$; $\partial L/\partial h = 2 \cdot v = 6$; $\partial L/\partial w = 6 \cdot \mathcal{K}[wx > 0] \cdot x = 12$. A step with $\eta = 0.01$ gives $v = 2.98$, $w = 0.38$; then $h = 0.76$, $\hat{y} = 2.265$, $L = 0.800$ — lower. At $w = -0.5$ instead, $wx < 0$, $h = 0$, and $\partial L/\partial w = 0$: the unit is dead for this input.

Softmax gradient. Probabilities (0.7, 0.2, 0.1), target class 1: logit gradient (−0.3, 0.2, 0.1); it sums to zero, and a descent step raises the correct logit relative to the others.

Stability. $f(\mathbf{w}) = \frac{1}{2}(w_1^2 + 25w_2^2)$: $\lambda_{\max} = 25$, so $\eta < 0.08$. At $\eta = 0.07$ the w_2 coordinate is multiplied by $1 - 0.07 \cdot 25 = -0.75$ per step (oscillating, shrinking); at $\eta = 0.09$ by -1.25 (oscillating, growing). Meanwhile w_1 shrinks by only $1 - 0.07 = 0.93$ per step.

He initialisation. Width 256: $\sigma_w = \sqrt{2/256} = 0.088$. With Xavier, $\sigma_w = 0.0625$, the per-layer variance gain is $\frac{256}{2} \cdot 0.0625^2 = 0.5$; after 29 more layers the standard deviation falls by $0.5^{29/2} \approx 4 \times 10^{-5}$ — the lab measures 3.5×10^{-5} starting from 0.59.

11.7 Small implementation and checks

The lab `labs/ch11-backprop/neural.py` contains a 60-line reverse-mode engine and a hand-written backward pass:

```
class Tensor:
    def __matmul__(self, other):
        return Tensor(self.value @ other.value, (self, other),
                      lambda g: (g @ other.value.T, self.value.T @ g))  #
                                ↪ vector-Jacobian products

    def relu(self):
        mask = self.value > 0
        return Tensor(self.value * mask, (self,), lambda g: (g * mask,))

    def backward(self):
        ... # topological sort, then for t in reversed(order): push
        ↪ t.grad to t.parents

G_A = (P - Y) / B                                # softmax + cross-entropy
g["W2"] = H.T @ G_A;  G_H = G_A @ W2.T
G_S = G_H * (S > 0);  g["W1"] = X.T @ G_S
```

Checks, in `test_neural.py`: hand gradients agree with central differences (3.5×10^{-8}) and exactly with the autodiff engine; the scalar example gives 12 and 2; softmax-gradient rows sum to zero; the stability threshold is $2/\lambda_{\max}$; momentum beats gradient descent on the quadratic; He initialisation keeps scales level while small initialisation vanishes; training fits the spirals and a huge step size does not; random labels are memorised but do not generalise.

A debugging sequence that finds most bugs (do it in order):

1. Pass one batch through the model; print every shape and dtype.
2. Check the initial loss: for K balanced classes it should be about $\log K$ (1.10 for three).
3. Gradient-check on a tiny model and batch, in float64, with any randomness fixed.
4. Overfit a handful of examples to near-zero loss. If you cannot, the bug is in the data, the loss, or the update — not in the amount of data.
5. Log loss, gradient norm, update norm and parameter norm per step; a finite loss with zero updates means parameters are detached or not given to the optimiser.
6. Only then tune the learning rate (a sweep on a log grid), then the rest.

11.8 Experiment and expected observations

Run `python labs/ch11-backprop/neural.py` (seed 0; 300 training and 300 validation spiral points).

experiment	result
gradient checks	finite differences 3.5×10^{-8} ; autodiff identical

experiment	result
quadratic $\kappa = 25$: steps to $f < 10^{-3}$	GD 66, momentum 22, Adam 43
GD $\eta = 0.079 / 0.081$ (limit 0.080)	$\ w\ $ after 100 steps 0.08 / 11.8
spirals, 300 epochs: SGD 0.3 / momentum 0.03 / Adam 0.01	train acc 1.000 each; val acc 0.997 each
SGD, train loss after 200 epochs, $\eta = 0.003 \dots 10$	0.677, 0.503, 0.268, 0.107, 0.038, 0.009, 0.004, 2.727
30-layer ReLU, activation sd at layer 30	small $4.7\text{e-}29$, Xavier $3.5\text{e-}5$, He 1.16, Xavier + residual 1.75
backprop signal layer 1 / layer 30	small $4.3\text{e-}28$, Xavier $5.2\text{e-}5$, He 1.21, residual 1.34
true vs. random labels (10-D lifted spirals)	100% train at epoch 90 vs. 388; val acc 0.943 vs. 0.360

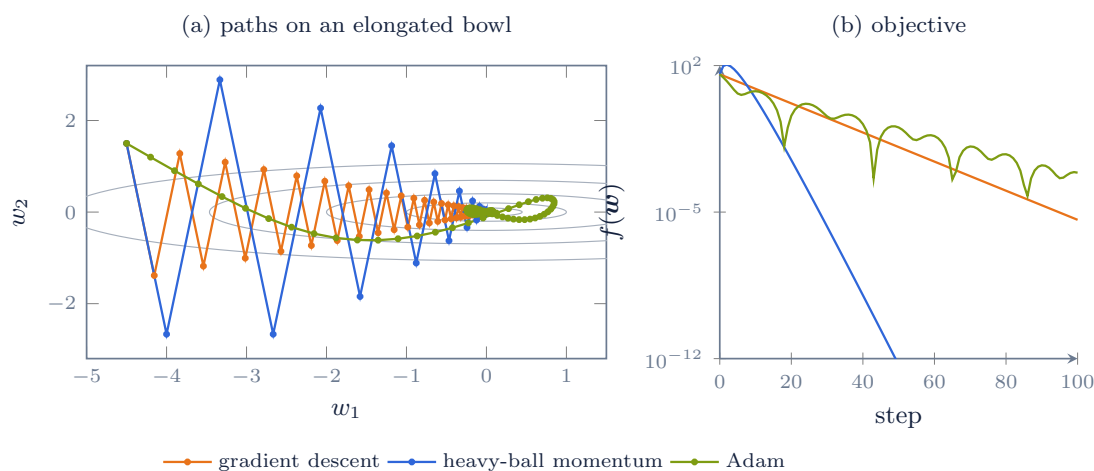


Figure 11.2: Gradient descent, heavy-ball momentum and Adam on a quadratic with condition number 25. Gradient descent zig-zags across the valley and creeps along it; momentum overshoots but converges in a third of the steps; Adam’s per-coordinate steps make fast early progress and then jitter.

What to notice:

1. **The best learning rate is just below instability.** The sweep improves monotonically up to $\eta = 3$ and fails at $\eta = 10$. Sweep on a logarithmic grid and pick the largest stable value, then back off by a factor of 2–3 for safety; a schedule then decays it.
2. **Optimisers differ in speed more than destination** here: all three reach 100% training and 99.7% validation accuracy on the spirals. Compare optimisers at their own tuned learning rates, never at one shared value.
3. **Depth multiplies small errors.** A per-layer gain of $\frac{1}{2}$ is harmless in one layer and fatal in thirty.
4. **Training accuracy can lie.** Memorising 300 random labels takes about four times longer than learning the true ones, but it happens.

11.9 Moving pieces: diagnosing and fixing

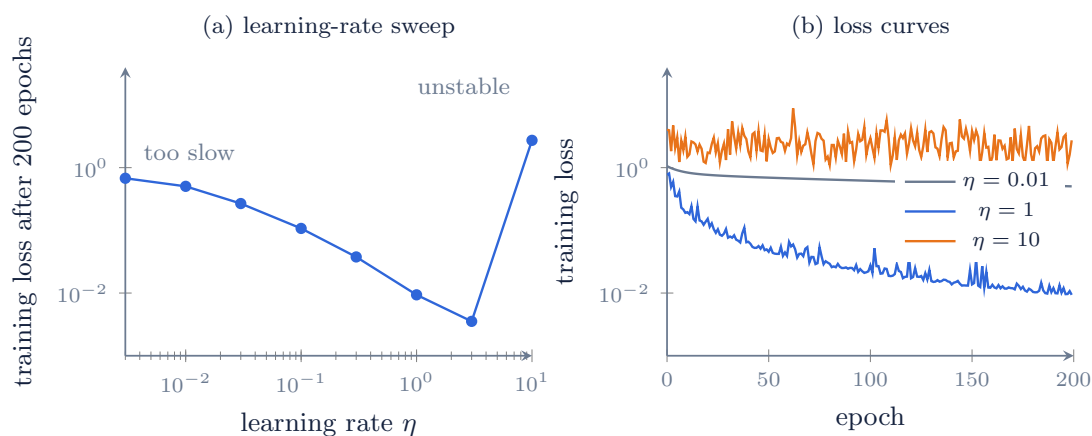


Figure 11.3: Plain SGD on the spiral classifier. (a) Final training loss falls as the learning rate grows — until the run becomes unstable. (b) Loss curves: too small is slow, a good rate is fast, too large bounces.

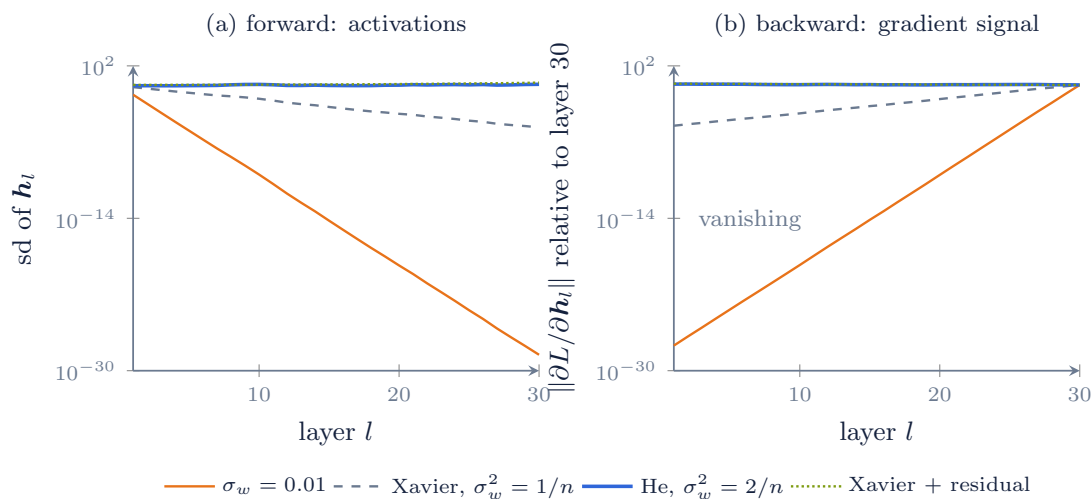


Figure 11.4: A 30-layer ReLU network at initialisation. With weights too small (0.01) or scaled for symmetric activations (Xavier), activations and backpropagated gradients shrink geometrically with depth; He initialisation keeps them level, as does a residual connection.

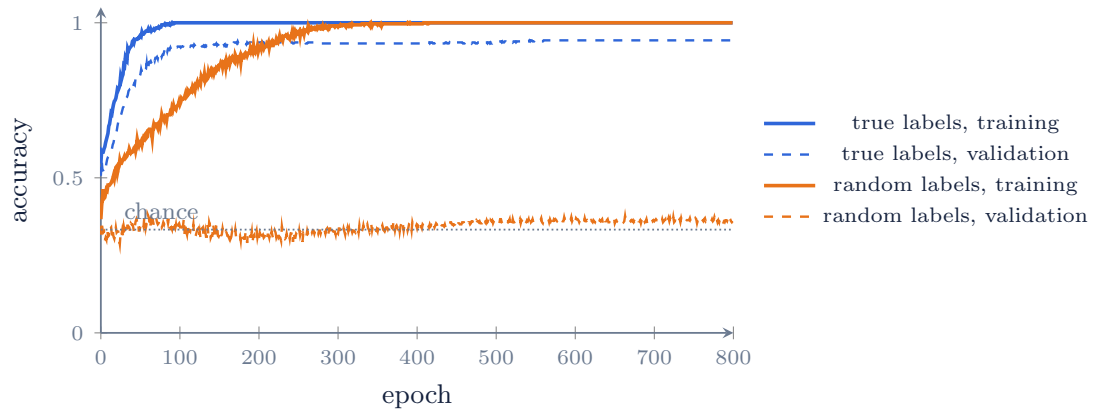


Figure 11.5: The same network trained on true and on randomly permuted labels. Both reach 100% training accuracy; validation accuracy stays at chance for random labels.

Moving piece	Symptom when it is wrong	How to confirm	Fix
Gradient code	loss decreases slowly or not at all;	gradient check on a tiny model, float64,	fix the backward rule; use autodiff
Learning rate	mysterious plateaus divergence or NaN (too high); slow linear-looking decrease (too low)	fixed randomness log-grid sweep; loss curve shape	and check its inputs largest stable rate; warm-up; decay schedule
Loss reduction	effective learning rate changes with batch size	compare sum vs. mean	average over the batch; scale η deliberately
Initialisation	loss stuck at $\log K$; activations near zero or huge	activation sd and gradient norm per layer at step 0	He (ReLU) or Xavier (tanh); zero-init the last layer of residual branches
Dead ReLUs	many units never active; capacity lost	fraction of units with zero activation over a batch	smaller learning rate; leaky ReLU/GELU; check biases
Exploding gradients	sudden spikes in loss and gradient norm	log gradient norm	gradient clipping (after finding the cause); normalisation; lower η
Normalisation mode	validation much worse than training for no reason	evaluate the training set in eval mode	switch to evaluation mode; check running statistics
Optimiser state	odd behaviour after resuming or changing parameters	check what the optimiser holds	save and restore optimiser state; re-create after changing parameter sets

Moving piece	Symptom when it is wrong	How to confirm	Fix
Weight decay with Adam	regularisation weaker than expected	compare L2-in-loss and decoupled decay	AdamW
Data pipeline	training accuracy at chance; overfitting impossible	overfit 10 examples; inspect inputs and labels	fix labels, shuffling, normalisation of inputs
Memorisation	training loss near zero, validation poor	gap between training and validation; random-label control	more data, augmentation, regularisation, early stopping (chapter 5)

First things to try: (1) check the initial loss against $\log K$; (2) overfit a tiny batch; (3) sweep the learning rate; (4) log gradient norms per layer; (5) only then change the architecture.

11.10 Application, limitations and failure cases

Application (educational case study). When the crop-health team compares a new architecture with a baseline, the comparison is only fair if both have been trained with tuned learning rates, the same number of epochs or compute, the same augmentation and the same stopping rule, and if several seeds are reported. Write down: optimiser and its hyperparameters, batch size and loss reduction, initialisation, schedule with warm-up, weight decay, gradient clipping, number of steps, and how the checkpoint was selected. Without these, “architecture A beats B” is often “recipe A beats recipe B”.

Limitations:

- The quadratic analysis is local; real loss surfaces have saddles, plateaus and changing curvature. It predicts the *shape* of failures, not exact thresholds.
- Initialisation formulas assume independence and symmetric pre-activations; correlated inputs, attention layers and normalisation change the details.
- Adam’s defaults work often but not always; it can generalise worse than well-tuned SGD with momentum on some vision tasks.
- Memorisation experiments show capacity, not what a network does on real data; real networks fit structured signal first.

11.11 Bridge to a research paper

He, Zhang, Ren and Sun (2015), *Delving deep into rectifiers: surpassing human-level performance on ImageNet classification* (ICCV 2015).

- **Problem:** very deep plain networks of rectified units (22–30 layers) failed to train from Xavier initialisation.
- **Mechanism:** derive the variance of pre-activations through ReLU layers (step 10) and set $\text{Var}[w] = 2/n$; also introduce the parametric ReLU, whose negative slope is learned.
- **Central claim:** with this initialisation a 30-layer plain network converges where Xavier does not; PReLU networks reached 4.94% top-5 error on ImageNet.

- **Assumptions to question:** independence of weights and inputs; that the forward and backward conditions (fan-in vs. fan-out) can both hold; whether initialisation still matters once batch norm and residual connections (published within a year) are used.
- **Affordable experiment:** extend the lab to *train* a 20-layer plain ReLU MLP (width 64) on the lifted spirals with Xavier, He and He + residual initialisations, five seeds each, and plot training loss against epoch. Does Xavier train at all? How much does the residual connection add once He initialisation is used?

11.12 Memory map



Figure 11.6: Memory map of chapter 11.

Summary. Backpropagation is the chain rule applied in reverse topological order over a computational graph: each node multiplies the incoming gradient by its local Jacobian (transposed) and reuses values stored on the forward pass, so the full gradient costs a

few forward passes whatever the number of parameters. For softmax cross-entropy the output gradient is prediction minus target. Gradient descent is limited in step size by the sharpest curvature ($\eta < 2/\lambda_{\max}$) and in speed by the flattest; momentum improves the dependence on the condition number to its square root, and Adam rescales each coordinate. Depth multiplies per-layer gains, so initialisation ($\sigma_w^2 = 2/n$ for ReLU), normalisation and residual connections are needed to keep signals and gradients at order one. Networks can memorise noise, so training loss is never evidence of generalisation.

11.13 Exercises

1. *Conceptual.* Why can ten affine layers without nonlinearities express no more than one affine map?
2. *Conceptual.* If the minibatch doubles in size and the loss is a mean, does the gradient double? What about a summed loss?
3. *Conceptual.* Why must the random draw be fixed during a gradient check of a stochastic objective (dropout, the VAE of chapter 10)?
4. *Derivation.* Derive the backward rules for $\mathbf{U} = \mathbf{Z}\mathbf{W}$ by writing one entry of $\mathbf{U}$ and differentiating, and check the shapes.
5. *Derivation.* Derive the gradient of layer normalisation $\hat{\mathbf{h}} = (\mathbf{h} - \bar{h})/\sqrt{\sigma^2 + \epsilon}$ with respect to $\mathbf{h}$, and show it is orthogonal to the all-ones vector.
6. *Derivation.* For heavy-ball momentum on a one-dimensional quadratic with curvature λ , write the update as a 2×2 linear system and find the condition on (η, β) for stability.
7. *Implementation.* Add a `tanh` network to the lab's depth study and find the weight scale that keeps activations level. Compare with Xavier's $1/n$.
8. *Implementation.* Carry out the experiment of section 11.11.
9. *Interpretation.* A model reaches 99.9% training accuracy and 61% validation accuracy. List three explanations and the experiment that distinguishes each.
10. *Open question.* Large language models are trained with AdamW, warm-up and cosine decay at batch sizes of millions of tokens. Which parts of this chapter's analysis would you expect to still apply, and which not?

11.13.1 Answer discussions

1. A composition of affine maps $\mathbf{x} \mapsto \mathbf{x}\mathbf{W}_1 + \mathbf{b}_1 \mapsto \dots$ is affine: $\mathbf{x} \prod \mathbf{W}_l + \text{const.}$
2. With a mean, no: the gradient's expectation is unchanged and its variance halves. With a sum, yes — which silently doubles the effective learning rate.
3. Otherwise $L(\theta+h)$ and $L(\theta-h)$ are evaluated with different noise, and their difference measures the noise, not the derivative.
4. $U_{ij} = \sum_k Z_{ik}W_{kj}$, so $\partial L/\partial Z_{ik} = \sum_j G_{U,ij}W_{kj} = (\mathbf{G}_U\mathbf{W}^\top)_{ik}$ and $\partial L/\partial W_{kj} = \sum_i Z_{ik}G_{U,ij} = (\mathbf{Z}^\top\mathbf{G}_U)_{kj}$.
5. $\mathbf{G}_h = \frac{1}{\sigma}(\mathbf{G}_{\hat{h}} - \overline{\mathbf{G}_{\hat{h}}}\mathbf{1} - \hat{\mathbf{h}}\overline{\mathbf{G}_{\hat{h}}\hat{h}})$, where bars are means over features; summing its entries gives zero because shifting $\mathbf{h}$ does not change $\hat{\mathbf{h}}$.
6. $\begin{pmatrix} w_{t+1} \\ w_t \end{pmatrix} = \begin{pmatrix} 1 + \beta - \eta\lambda & -\beta \\ 1 & 0 \end{pmatrix} \begin{pmatrix} w_t \\ w_{t-1} \end{pmatrix}$; stable iff both eigenvalues have modulus below one: $0 \leq \beta < 1$ and $0 < \eta\lambda < 2(1 + \beta)$.
7. Near zero tanh is the identity, so $\sigma_w^2 \approx 1/n$ keeps the gain near one, with a slow decline from saturation; ReLU needs twice that.
8. Expect Xavier to fail or crawl, He to train, and the residual variant to train fastest and most reliably across seeds.

9. Overfitting (gap shrinks with more data or regularisation); a distribution shift between training and validation (check a held-out split from the training distribution); leakage or label errors in training (inspect examples, use a group split — chapter 5).
10. Stability limits, warm-up for early curvature, per-coordinate scaling and decoupled decay still apply; the quadratic picture of a single valley, small-batch noise arguments and the memorisation regime apply less, because such models see each example about once.

11.14 Self-check and common mistakes

- ☐ I can derive every gradient of a two-layer softmax classifier with shapes.
- ☐ I can explain why reverse-mode differentiation is cheap and what it costs in memory.
- ☐ I can state and explain $\eta < 2/\lambda_{\max}$ and the role of κ .
- ☐ I can derive $\sigma_w^2 = 2/n$ for ReLU layers.
- ☐ I can list the debugging sequence in order.

Common mistakes: summing the loss over the batch and then using a learning rate tuned for the mean; computing softmax and log separately (overflow, $\log 0$); forgetting to sum bias gradients over the batch; comparing optimisers at one learning rate; adding L2 to the loss with Adam and calling it weight decay; leaving batch norm in training mode at evaluation; gradient-checking with live dropout; treating gradient clipping as a fix rather than a symptom; reading 100% training accuracy as success.

References

- I. Goodfellow, Y. Bengio and A. Courville. *Deep Learning*. MIT Press, 2016, sections 6.5 (back-propagation) and chapter 8 (optimisation).
- H. Robbins and S. Monro. A stochastic approximation method. *Annals of Mathematical Statistics*, 22(3):400–407, 1951.
- B. T. Polyak. Some methods of speeding up the convergence of iteration methods. *USSR Computational Mathematics and Mathematical Physics*, 4(5):1–17, 1964.
- S. Linnainmaa. The representation of the cumulative rounding error of an algorithm as a Taylor expansion of the local rounding errors. Master’s thesis, University of Helsinki, 1970.
- D. E. Rumelhart, G. E. Hinton and R. J. Williams. Learning representations by back-propagating errors. *Nature*, 323:533–536, 1986.
- X. Glorot and Y. Bengio. Understanding the difficulty of training deep feedforward neural networks. In *AISTATS*, 2010.
- D. P. Kingma and J. Ba. Adam: a method for stochastic optimization. In *ICLR*, 2015. arXiv:1412.6980.
- K. He, X. Zhang, S. Ren and J. Sun. Delving deep into rectifiers: surpassing human-level performance on ImageNet classification. In *ICCV*, 2015.
- S. Ioffe and C. Szegedy. Batch normalization: accelerating deep network training by reducing internal covariate shift. In *ICML*, 2015.
- J. L. Ba, J. R. Kiros and G. E. Hinton. Layer normalization. arXiv:1607.06450, 2016.
- P. Goyal et al. Accurate, large minibatch SGD: training ImageNet in 1 hour. arXiv:1706.02677, 2017.
- C. Zhang, S. Bengio, M. Hardt, B. Recht and O. Vinyals. Understanding deep learning requires rethinking generalization. In *ICLR*, 2017.
- I. Loshchilov and F. Hutter. Decoupled weight decay regularization. In *ICLR*, 2019.

- A. G. Baydin, B. A. Pearlmutter, A. A. Radul and J. M. Siskind. Automatic differentiation in machine learning: a survey. *JMLR*, 18(153):1–43, 2018.

Chapter 12

Convolutions and residual networks

Prerequisites: chapter 11 (backpropagation, initialisation, normalisation); chapter 1 (linear maps).

Learning outcomes: after this chapter you can

1. write a convolutional layer as a structured linear map, count its parameters and compute output shapes for any kernel, stride, padding and dilation;
2. derive the backward pass of a convolution and explain weight sharing as evidence pooled across positions;
3. distinguish translation equivariance from invariance, and explain how striding and pooling break equivariance;
4. compute theoretical receptive fields recursively and explain why effective receptive fields are much smaller;
5. explain residual connections, normalisation layers and multi-scale architectures (ResNet, U-Net, FPN) as answers to specific failures.

12.1 A concrete problem

A field camera produces 16×16 grey patches; each contains either a “+”-shaped or an “x”-shaped lesion pattern, anywhere in the patch, under heavy noise. With 100 labelled patches, how do we build a classifier that works on new ones?

An MLP on the 256 raw pixels (chapter 11) has 16,578 parameters, fits the 100 training patches perfectly — and is at chance on new ones (section 12.8). It must learn the pattern separately at every position, and 100 examples cover few positions. A **convolutional network** with 682 parameters gets 95% test accuracy from the same 100 patches. It builds in two facts about images: useful patterns are **local**, and the same pattern can appear **anywhere**.

This chapter derives the convolutional layer from those two assumptions, then follows the problems that appear when such layers are stacked deeply — vanishing signals, lost resolution, aliasing — and the architectural answers that dominate computer vision: residual networks, normalisation, and multi-scale encoder–decoders. Chapter 13 uses them for detection and segmentation; chapter 14 asks what changes when the locality assumption is dropped.

12.2 How we got here

Receptive fields in the brain (1959–1962). David Hubel and Torsten Wiesel recorded neurons in the cat visual cortex that respond to oriented edges in a small region of the visual field — the **receptive field** — with “complex” cells responding to the same edge over a range of positions. Local features, repeated across the visual field and pooled for tolerance to position, became the blueprint.

The Neocognitron and LeNet (1980–1998). Kunihiko Fukushima’s Neocognitron (1980) alternated feature-extracting and pooling layers with shared weights, but was trained without gradients. Yann LeCun and colleagues (1989) trained a convolutional network by backpropagation to read handwritten zip codes; LeNet-5 (1998) read millions of bank cheques.

AlexNet and the deep-learning turn (2012). Krizhevsky, Sutskever and Hinton trained an eight-layer convolutional network on two GPUs with ReLUs, dropout and data augmentation, and cut ImageNet top-5 error from 26% to 15%. VGG (Simonyan and Zisserman, 2014) showed that stacks of 3×3 convolutions work well; GoogLeNet (Szegedy and colleagues, 2014) used multi-branch “Inception” blocks and 1×1 convolutions.

The degradation problem and ResNet (2015). Deeper plain networks had *higher training* error than shallower ones — not overfitting, but failure to optimise. Kaiming He, Xiangyu Zhang, Shaoqing Ren and Jian Sun proposed learning **residuals**, $\mathbf{y} = \mathbf{x} + F(\mathbf{x})$, and trained 152-layer networks that won ILSVRC 2015 — this chapter’s paper bridge. Batch normalisation (Ioffe and Szegedy, 2015) appeared the same year.

Dense prediction (2015–2017). Fully convolutional networks (Long, Shelhamer and Darrell, 2015) produced per-pixel outputs; U-Net (Ronneberger, Fischer and Brox, 2015) added skip connections from an encoder to a decoder for biomedical segmentation with few images; feature pyramid networks (Lin and colleagues, 2017) gave detectors strong features at every scale.

Refinements and rivals (2017–2022). Dilated convolutions, depthwise-separable convolutions (MobileNet, 2017), EfficientNet’s compound scaling (2019), and anti-aliased downsampling (Zhang, 2019) refined the design. Vision transformers (2020; chapter 14) dropped the locality assumption, and ConvNeXt (Liu and colleagues, 2022) showed that a convolutional network modernised with transformer-era training recipes matches them — a reminder that architecture comparisons are also recipe comparisons.

12.3 Assumptions and notation

Symbol	Meaning
$\mathbf{X} \in \mathbb{R}^{C \times H \times W}$	input feature map: C channels of height H and width W (a batch adds a leading axis)
$\mathbf{W} \in \mathbb{R}^{O \times C \times k \times k}$, $\mathbf{b} \in \mathbb{R}^O$	kernel (filters) and biases
k, s, p, d	kernel size, stride, padding, dilation
$\mathbf{Y} \in \mathbb{R}^{O \times H' \times W'}$	output feature map
r_l, j_l	theoretical receptive field and jump (input pixels between adjacent units) at layer l
$F(\mathbf{x})$	a residual branch
τ_δ	translation by δ pixels

Assumptions built into a convolution:

1. **Locality.** An output depends only on a $k \times k$ neighbourhood of the input.
2. **Stationarity.** The same local pattern is useful everywhere, so the same weights are used at every position (weight sharing).
3. **Boundary handling.** Padding (usually zeros) defines what lies outside the image; near borders assumption 2 is only approximately true.

12.4 Mathematical formulation

Convolutional layer (deep-learning convention; strictly a cross-correlation, since the kernel is not flipped):

$$Y_{o,u,v} = b_o + \sum_{c=1}^C \sum_{i=0}^{k-1} \sum_{j=0}^{k-1} W_{o,c,i,j} X_{c,su+di-p,sv+dj-p}.$$

Output size along one axis:

$$H' = \left\lfloor \frac{H + 2p - d(k-1) - 1}{s} \right\rfloor + 1.$$

Parameters: $k^2CO + O$, independent of H and W .

Receptive field (recursively, from $r_0 = j_0 = 1$):

$$r_l = r_{l-1} + (k_l - 1) d_l j_{l-1}, \quad j_l = s_l j_{l-1}.$$

Residual block: $y = x + F(x)$ (with a 1×1 projection of x when shapes differ).

Normalisation of a feature x over a set $\mathcal{S}$ (batch and positions for batch norm; features of one example for layer norm): $\hat{x} = (x - \mu_{\mathcal{S}}) / \sqrt{\sigma_{\mathcal{S}}^2 + \epsilon}$, then $y = \gamma \hat{x} + \beta$.

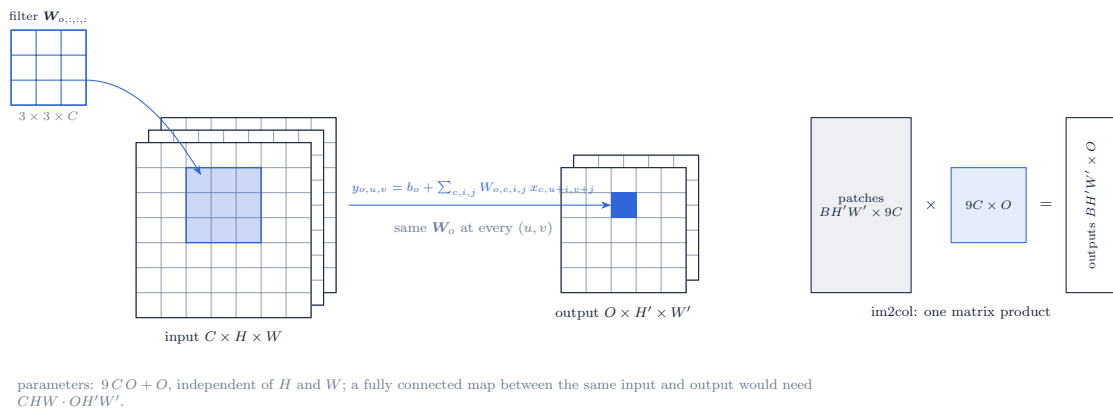


Figure 12.1: A convolutional layer. One filter of size $3 \times 3 \times C$ slides over the input; the same weights produce every position of one output channel. Unfolding all patches into rows (im2col) turns the layer into one matrix product.

12.5 Derivation, with reasons for each step

Step 1 — a convolution is a structured linear map. For fixed weights, $\mathbf{Y}$ is linear in $\mathbf{X}$: flatten both and $\text{vec}(\mathbf{Y}) = \mathbf{K} \text{vec}(\mathbf{X}) + \text{bias}$ for a matrix $\mathbf{K}$ of size $OH'W' \times CHW$. Locality makes $\mathbf{K}$ **sparse** (each row has k^2C non-zeros); sharing makes it **repetitive** (rows for different positions contain the same numbers, shifted) — for one channel in 1-D it is a banded Toeplitz matrix. A fully connected layer between the same shapes would have $CHW \cdot OH'W'$ free parameters; the convolution has k^2CO . For a 3×3 layer mapping 64 to 64 channels on 56×56 maps, that is 36,864 instead of about 4×10^{10} . The parameter saving is the statistical point: fewer parameters need fewer examples (section 12.8).

Step 2 — computing it: im2col. Unfold every $k \times k \times C$ input patch into a row of a matrix $\mathbf{P}$ of size $(BH'W') \times (k^2C)$, and reshape the kernel into $\mathbf{K}_{\text{mat}}$ of size $O \times k^2C$. Then all outputs are $\mathbf{P}\mathbf{K}_{\text{mat}}^\top$ — one matrix product, the operation hardware is best at. Production libraries use smarter algorithms (direct, FFT, Winograd), but im2col is the clearest statement of what is computed.

Step 3 — the backward pass, and why sharing pools evidence. In 1-D, $y_t = \sum_i w_i x_{t+i}$ with upstream gradient $g_t = \partial L / \partial y_t$:

$$\frac{\partial L}{\partial w_i} = \sum_t g_t x_{t+i}, \quad \frac{\partial L}{\partial x_j} = \sum_{t,i: t+i=j} g_t w_i.$$

Reason: w_i is used at every position t , so its gradient sums contributions from every position — one filter learns from evidence everywhere in the image. The input gradient is a **full** convolution of $\mathbf{g}$ with the flipped kernel: each input value receives gradient from every output it influenced. In im2col form, $\mathbf{G}\mathbf{K}_{\text{mat}} = \mathbf{G}_Y^\top \mathbf{P}$ and $\mathbf{G}_P = \mathbf{G}_Y \mathbf{K}_{\text{mat}}$, followed by “col2im”, which adds each patch gradient back into the positions it was copied from (chapter 11, step 2: a value used several times receives the sum of the gradients). Stride and padding only change which indices match; derive them on paper before optimising code.

Step 4 — equivariance, and how it is lost. Let τ_δ shift an image. Away from the borders, a stride-1 convolution commutes with shifts:

$$\text{conv}(\tau_\delta \mathbf{X}) = \tau_\delta \text{conv}(\mathbf{X}).$$

Reason: substitute $\mathbf{X}'_{c,u,v} = \mathbf{X}_{c,u-\delta_1,v-\delta_2}$ into the definition; the sum over the kernel is unchanged. Elementwise nonlinearities commute with shifts too, so a stack of stride-1 convolutions is **equivariant**: move the lesion, and its feature map moves with it. Global average or max pooling then gives **invariance**: the pooled features ignore position — exactly what a classifier wants, and exactly what a detector must not throw away.

Striding breaks it. Keeping every second pixel commutes only with shifts by even numbers of pixels. A one-pixel shift can change which samples survive — aliasing. In the lab, the globally pooled features of a stride-2 network change by 41% under a one-pixel shift, while the stride-1 feature map shifts exactly. Blurring before subsampling (a low-pass filter, Zhang 2019) reduces the change to 26%. Real CNNs are therefore only approximately shift-invariant, and their outputs can flip under one-pixel shifts.

Step 5 — receptive fields. A unit at layer l depends on an $r_l \times r_l$ window of the input. Each new $k \times k$ layer extends the window by $k - 1$ units of the *previous* layer, each of which is j_{l-1} input pixels apart — hence the recursion of section 12.4. Two 3×3 stride-1

layers see 5 pixels; with stride 2 first, 7. A ResNet-18-like stack (stem $7 \times 7/2$, max pool $/2$, then $16 \times 3 \times 3$ layers with three more stride-2 layers) has a theoretical receptive field of 435 pixels and a jump of 32. Two stacked 3×3 layers see as far as one 5×5 layer with $18C^2$ instead of $25C^2$ weights and an extra nonlinearity — VGG’s argument.

Step 6 — effective receptive fields are small and Gaussian. The theoretical field says which pixels *can* influence a unit, not how much. Back-propagate a gradient from one output unit to the input (Luo and colleagues, 2016): pixels near the centre have many paths through the network to the output, pixels near the edge of the field have few. By a central-limit argument the influence profile becomes Gaussian-like with a width that grows like $\sqrt{L}$, not L . The lab measures RMS radii of 1.8, 2.8 and 4.3 pixels at depths 2, 4 and 8, against theoretical half-widths 2, 4 and 8 (figure 12.2). Consequences: large objects need downsampling or dilation to be seen whole; and claims of the form “this unit sees the whole image” should be checked, not assumed.

Step 7 — pooling and downsampling trade resolution for context. Each stride-2 step doubles the jump, so later layers see more context with less compute, but small structures — a 3-pixel lesion, a distant pest — can vanish after four or five halvings. Max pooling routes the gradient to the selected maximum; average pooling spreads it equally. Neither is “correct”; both discard where exactly within the window a feature was, and how many there were.

Step 8 — residual connections. Chapter 11 showed that signals shrink or explode geometrically through plain stacks unless each layer’s gain is one. A residual block $\mathbf{y} = \mathbf{x} + F(\mathbf{x})$ has Jacobian

$$\frac{\partial \mathbf{y}}{\partial \mathbf{x}} = \mathbf{I} + \frac{\partial F}{\partial \mathbf{x}}, \quad \frac{\partial L}{\partial \mathbf{x}_l} = \frac{\partial L}{\partial \mathbf{x}_L} \prod_{m=l}^{L-1} \left(\mathbf{I} + \frac{\partial F_m}{\partial \mathbf{x}_m} \right).$$

Reason: chain rule. Expanding the product gives a sum over all subsets of blocks: one term is the identity, so the gradient at layer l always contains the output gradient unchanged, however deep the network. A residual network is like an ensemble of paths of many lengths (Veit and colleagues, 2016). Two further consequences: a block that is not useful can learn $F \approx 0$ and become the identity — so adding depth should not hurt training error, which addresses the degradation problem directly; and initialising the last layer (or normalisation scale) of each branch at zero makes the whole network start as the identity. It is not a theorem that gradients never vanish: if $\partial F / \partial \mathbf{x} \approx -\mathbf{I}$ the factor cancels.

Step 9 — bottlenecks and projections. In deep ResNets (50+ layers) each block is 1×1 (reduce channels by 4) $\rightarrow 3 \times 3 \rightarrow 1 \times 1$ (restore). The expensive 3×3 convolution runs on a quarter of the channels: about 70,000 weights per block at 256 channels instead of about 1.2 million for two 3×3 layers at full width. When a block changes resolution or width, the skip uses a strided 1×1 projection so that $\mathbf{x}$ and $F(\mathbf{x})$ have the same shape (figure 12.4).

Step 10 — normalisation layers. Batch normalisation standardises each channel using the mean and variance over the batch and all positions, then applies a learned scale and shift. It keeps each layer’s input distribution stable as earlier weights change, allows larger learning rates, and adds noise from batch composition (a mild regulariser). Three cautions: (1) at evaluation it uses running averages, so evaluation in training mode gives different — sometimes much worse — numbers; (2) with batches of 1–4 images (common in detection and segmentation) its statistics are noisy, and group normalisation or frozen statistics are used instead; (3) the original “internal covariate shift” explanation is contested; its benefits

are better understood as smoothing the optimisation landscape and controlling the scale of gradients. Layer normalisation (per example, over channels) avoids batch dependence and is standard in transformers (chapter 14).

Step 11 — getting resolution back: encoder–decoders and pyramids. Classification wants invariance; segmentation wants a label for every pixel, and detection wants objects of all sizes. Two designs dominate (figure 12.5):

- **U-Net:** a decoder upsamples step by step, and at each scale **concatenates** the encoder’s feature map of the same resolution. Fine detail comes from the early encoder layers; context comes from the deep ones. With heavy augmentation it trains on tens of images.
- **Feature pyramid network (FPN):** a top-down pathway upsamples the deepest (most semantic) features and **adds** a 1×1 -projected lateral map from the backbone at each scale. Every level is then both semantically strong and at its own resolution; detectors attach heads to every level, assigning small objects to fine levels and large ones to coarse levels (chapter 13).

12.6 Numerical worked example

Output size and parameters. $H = 32, k = 3, p = 1, s = 2, d = 1$: $H' = \lfloor (32 + 2 - 2 - 1)/2 \rfloor + 1 = 16$. A 3×3 layer from 8 to 16 channels has $3 \cdot 3 \cdot 8 \cdot 16 = 1152$ weights and 16 biases. The lab’s two-layer CNN has $72 + 8 + 576 + 8 + 16 + 2 = 682$ parameters.

A filter on a patch. A 3×3 patch with every row $(0, 1, 2)$ and a filter with every row $(-1, 0, 1)$: $3 \times (0 \cdot (-1) + 1 \cdot 0 + 2 \cdot 1) = 6$. A constant patch gives 0: the filter measures horizontal contrast.

Backward in 1-D. $\mathbf{x} = (1, 2, 3), \mathbf{w} = (1, -1)$: $\mathbf{y} = (1 - 2, 2 - 3) = (-1, -1)$. With upstream $\mathbf{g} = (2, 1)$: $\partial L / \partial w_1 = 2 \cdot 1 + 1 \cdot 2 = 4$, $\partial L / \partial w_2 = 2 \cdot 2 + 1 \cdot 3 = 7$; and $\partial L / \partial \mathbf{x} = (2 \cdot 1, 2 \cdot (-1) + 1 \cdot 1, 1 \cdot (-1)) = (2, -1, -1)$. The lab reproduces all three.

Receptive fields. Layers (k, s) : $(3, 1), (3, 1)$ give $r = 1 + 2 + 2 = 5, j = 1$. $(3, 2), (3, 1)$: $r_1 = 3, j_1 = 2$; $r_2 = 3 + 2 \cdot 2 = 7, j_2 = 2$. Dilation 2 with $k = 3$: $r = 1 + 2 \cdot 2 = 5$ with only nine weights.

Residual derivative. $y = x + ax$: $dy/dx = 1 + a$. At $a = 0$ the derivative is 1, where the plain $y = ax$ would give 0. At $a = -1$ even the residual derivative is 0 — the skip helps, but does not guarantee.

12.7 Small implementation and checks

The lab `labs/ch12-cnn/cnn.py` implements convolution with `sliding_window_view` and a matrix product:

```

xp = np.pad(x, ((0, 0), (0, 0), (pad, pad), (pad, pad)))
win = sliding_window_view(xp, (k, k), axis=(2, 3))[:, :, ::stride,
→ ::stride] # B,C,H',W',k,k
cols = win.transpose(0, 2, 3, 1, 4, 5).reshape(B * Ho * Wo, C * k * k)
→ # im2col
y = cols @ W.reshape(0, -1).T + b
→ # one product
# backward: gW = g2.T @ cols; gcols = g2 @ Wmat; then scatter-add each
→ of the k*k taps

```

Checks, in `test_cnn.py`: output shapes and receptive fields (including dilation); the 1-D worked example; one output equal to the direct triple sum; gradients against finite differences for stride 1 and 2 and through a residual CNN (relative error 10^{-8}); exact equivariance at stride 1, broken by subsampling and partly restored by blurring; effective receptive fields growing much more slowly than theoretical ones; and the CNN beating the MLP from 100 examples.

Two habits from this lab worth keeping: test layers on tiny inputs against a direct, slow implementation; and write every shape in a comment next to every reshape and transpose — most convolution bugs are transposed axes that still produce arrays of the right size.

12.8 Experiment and expected observations

Run `python labs/ch12-cnn/cnn.py` (seed 0).

experiment	result
gradient check (conv stride 1, 2; residual CNN)	relative error 1.0×10^{-8}
one-pixel shift: stride-1 feature map after re-shifting	max error 0
one-pixel shift: pooled features, stride 2 / blur then stride 2	relative change 0.405 / 0.258
effective RMS radius at depth 1, 2, 4, 8 (theoretical half-width 1, 2, 4, 8)	1.17, 1.81, 2.83, 4.31 px
ResNet-18-like stack, theoretical receptive field	435 px (jump 32)

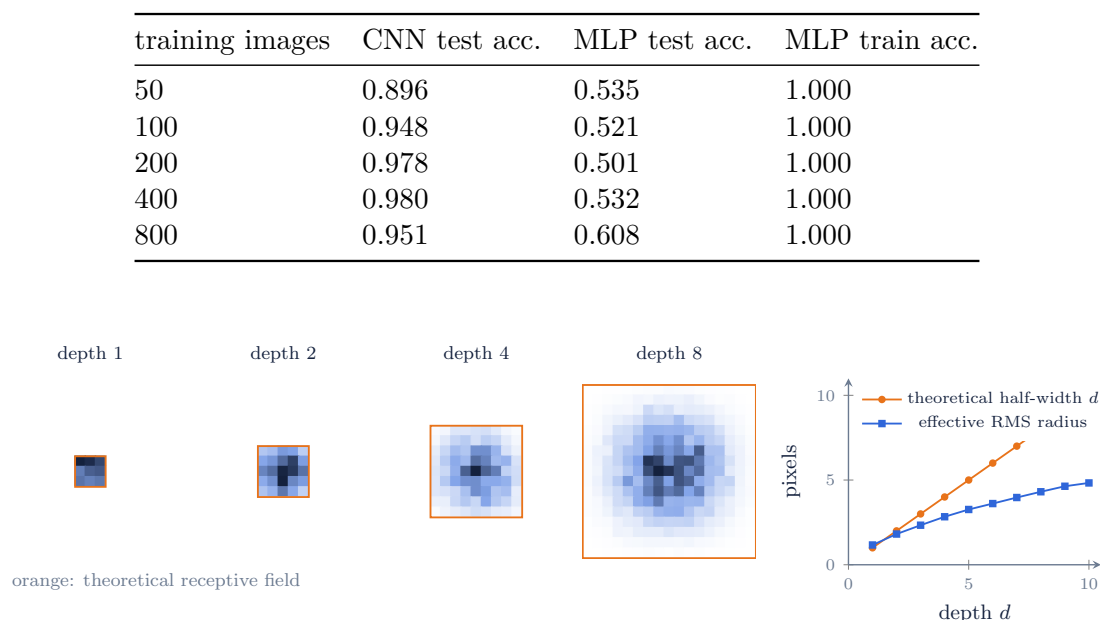


Figure 12.2: Effective receptive fields of random ReLU stacks of 3×3 convolutions: the influence of each input pixel on one central output, with the theoretical field outlined. The influence is concentrated near the centre, and its radius grows roughly like the square root of depth.

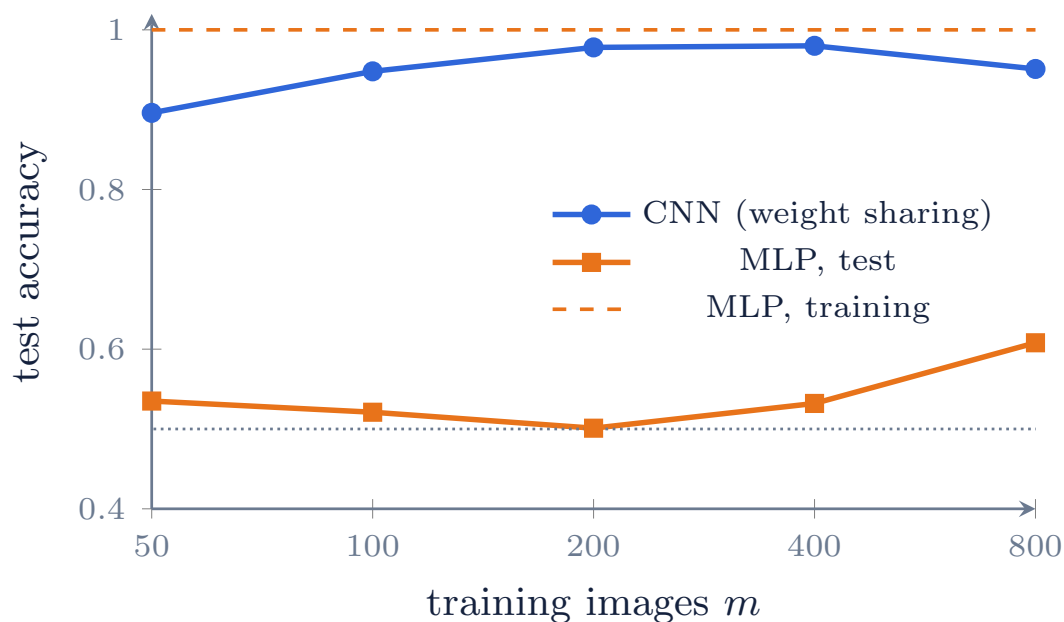


Figure 12.3: Test accuracy against training-set size: a 682-parameter CNN learns the position-independent pattern from 50 examples; a 16,578-parameter MLP memorises every training set and stays near chance.

What to notice:

1. **Inductive bias beats parameters.** The CNN has 24 times fewer parameters and wins by 40 points. Its assumptions (locality, stationarity) are true for this task; when they are false — for example when the *position* of a pattern carries the label — the same bias hurts.
2. **More data does not rescue the MLP quickly.** At 800 images it is still at 61%: it must learn the pattern at each of 144 positions separately.
3. **Shift invariance is approximate.** Even this tiny strided network changes its pooled features by 40% under a one-pixel shift.
4. **Most of a deep network’s receptive field is dim.** After eight layers, the RMS radius is about half the theoretical half-width, and the edges of the field contribute almost nothing.
5. **Optimisation noise is visible.** The CNN at 800 images (0.951) is slightly worse than at 400 (0.980) with the same number of steps; one seed and a fixed step budget are not enough to rank small differences (chapter 5).

12.9 Moving pieces: diagnosing and fixing

Moving piece	Symptom when it is wrong	How to confirm	Fix
Kernel/stride/padding arithmetic	shape errors; outputs misaligned with inputs by a pixel	print shapes; compare a one-pixel test pattern’s response position	recompute H' ; consistent “same” padding; align upsampling

Moving piece	Symptom when it is wrong	How to confirm	Fix
Downsampling	small objects missed; coarse masks	per-size recall (chapter 13); feature-map size at each stage	fewer stride-2 steps; dilation; FPN/U-Net skips
Aliasing	predictions flip under one-pixel shifts	consistency under small shifts	blur before subsampling; shift augmentation
Receptive field	large objects partly seen; context ignored	effective-RF map by back-propagation	more depth or downsampling; dilation; attention (chapter 14)
Depth without residuals	training error <i>rises</i> with depth	train shallow vs. deep with same recipe	residual connections; zero-init last layer of each branch
Normalisation	train/eval gap; poor results with tiny batches	evaluate in both modes; vary batch size	eval mode; group norm or frozen BN for small batches
Width and capacity	under-fitting (train error high)	training curve	wider or deeper; check learning rate first (chapter 11)
Border effects	errors concentrated near image edges	error map by position	reflect padding; crops with context; avoid evaluating on borders
Input normalisation	slow training; first-layer filters dominated by brightness	channel statistics	standardise inputs with training-set statistics
Inductive-bias mismatch	CNN worse than expected when position matters	test with position-dependent labels	add coordinates as channels; positional encodings

First things to try: (1) print every intermediate shape; (2) check a one-pixel shift of the input; (3) measure performance by object size; (4) evaluate in evaluation mode; (5) compare against a smaller model with the same training recipe before blaming depth.

12.10 Application, limitations and failure cases

Application (educational case study). For pest detection on sticky-trap photographs, insects are 10–30 pixels across in 4000-pixel images. A ResNet-50 backbone downsamples by 32 at its last stage, so an insect becomes less than one feature cell. Use an FPN and take detections from the finer levels, tile the image with overlap rather than resizing it down, and report recall separately for small objects. For leaf-disease classification from whole-leaf photographs, a standard backbone with global pooling is appropriate; check that the model is not using the background (chapter 5’s shortcut tests).

Limitations:

- Convolutions encode locality and stationarity; tasks with long-range dependencies (counting, relations between distant parts) need large effective receptive fields or attention.
- Equivariance holds for translation only; rotation and scale must be learned (augmentation) or built in specially.
- Batch statistics couple examples: a batch norm network’s output for one image depends on the others in its batch during training.

12.11 Bridge to a research paper

He, Zhang, Ren and Sun (2016), *Deep residual learning for image recognition* (CVPR 2016).

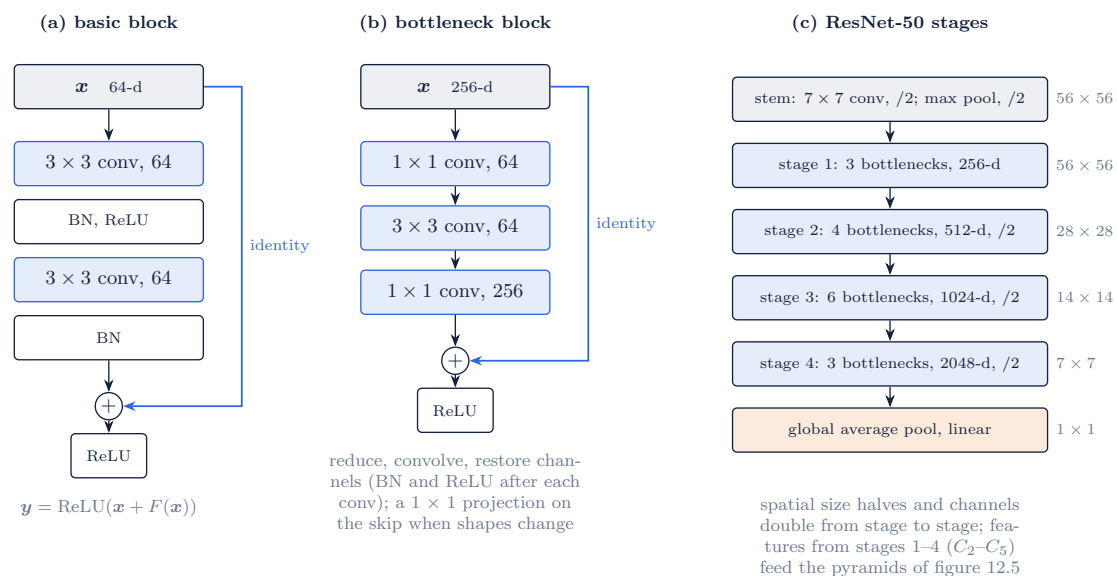


Figure 12.4: Residual blocks, redrawn after Figs. 2 and 5 of He et al. (2016): (a) the basic block of ResNet-18/34; (b) the bottleneck block of ResNet-50/101/152; (c) the stage layout of ResNet-50 (Table 1). Paper: [arXiv:1512.03385](https://arxiv.org/abs/1512.03385).

- **Problem:** the *degradation problem* — a 56-layer plain network has higher training and test error than a 20-layer one on CIFAR-10, although the deeper network could represent the shallower one by setting extra layers to the identity. The optimiser does not find that solution.
- **Mechanism:** reformulate each block to learn a residual $F(x)$ added to an identity shortcut (step 8); bottleneck blocks for depth (step 9); batch normalisation after each convolution; projection shortcuts where shapes change.
- **Central claim:** residual networks of 34, 50, 101 and 152 layers train well and improve with depth; an ensemble reached 3.57% top-5 error on ImageNet, and the features improved detection on COCO by 28% relative.
- **Assumptions to question:** is the benefit about optimisation (the paper’s argument) or about the implicit ensemble of paths? How much is due to batch normalisation and the training recipe, which changed at the same time?
- **Affordable experiment:** using the lab’s convolution, train plain and residual networks of depth 4, 8 and 16 (8 channels, He initialisation, residual branches scaled by $1/\sqrt{L}$) on the shapes task with plain SGD and the same learning rate, three seeds

each, for 2,000 steps. Plot final *training* loss against depth. (Expect several minutes of CPU time; reduce the image size to 12×12 to speed it up.) Does training loss rise with depth for the plain network, and not for the residual one?

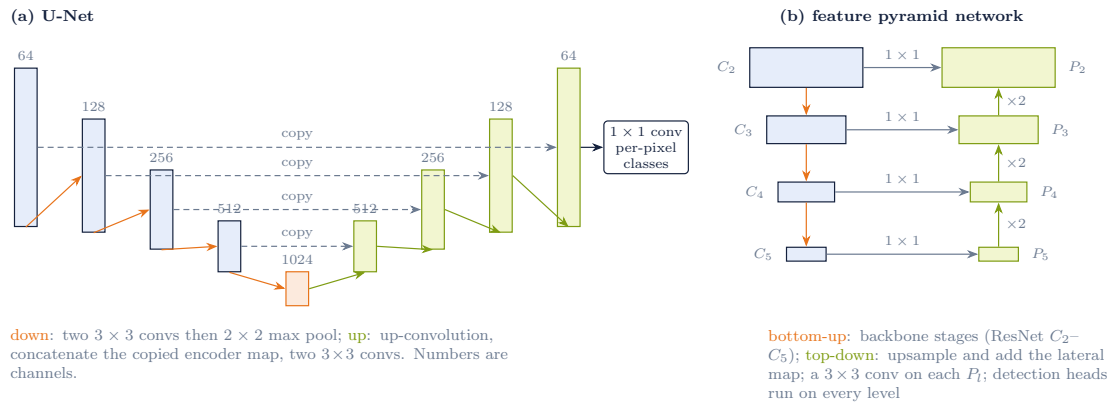


Figure 12.5: Recovering resolution for dense prediction. (a) U-Net, redrawn after Fig. 1 of Ronneberger et al. (2015): encoder features are copied and concatenated into the decoder at each scale. (b) Feature pyramid network, redrawn after Fig. 3 of Lin et al. (2017): a top-down pathway with lateral connections gives semantically strong features at every scale. Papers: [arXiv:1505.04597](https://arxiv.org/abs/1505.04597), [arXiv:1612.03144](https://arxiv.org/abs/1612.03144).

12.12 Memory map

Summary. A convolution is a linear map constrained by locality and weight sharing: k^2CO parameters regardless of image size, and gradients that pool evidence from every position. At stride 1 it is translation-equivariant; pooling turns equivariance into invariance, and striding breaks it through aliasing. Receptive fields grow linearly with depth in theory but their effective, Gaussian-like core grows only like the square root. Stacking many layers needs residual connections, whose identity path carries signal and gradient through any depth, plus normalisation; bottlenecks and projections make very deep networks affordable. Dense prediction recovers resolution with encoder–decoder skips (U-Net) or feature pyramids (FPN). These are the backbones of chapter 13 and the point of comparison for chapter 14.

12.13 Exercises

1. *Conceptual.* What is the output size for $H = 32$, $k = 3$, $p = 1$, $s = 2$, $d = 1$? For $H = 224$, $k = 7$, $p = 3$, $s = 2$?
2. *Conceptual.* A 3×3 layer maps 8 channels to 16. How many weights, excluding biases?
3. *Conceptual.* Why can forgetting evaluation mode change a validation score?
4. *Derivation.* Write a 1-D convolution with stride 2 and padding 1 as a matrix for $H = 6$, $k = 3$, and derive its backward pass as multiplication by the transpose. What does the transpose correspond to?
5. *Derivation.* Prove the receptive-field recursion by induction, including dilation.
6. *Derivation.* Compute the number of weights in a bottleneck block ($256 \rightarrow 64 \rightarrow 256$ channels) and in a basic block of two 3×3 convolutions at 256 channels.
7. *Implementation.* Add max pooling (forward and backward, with a tie rule) to the

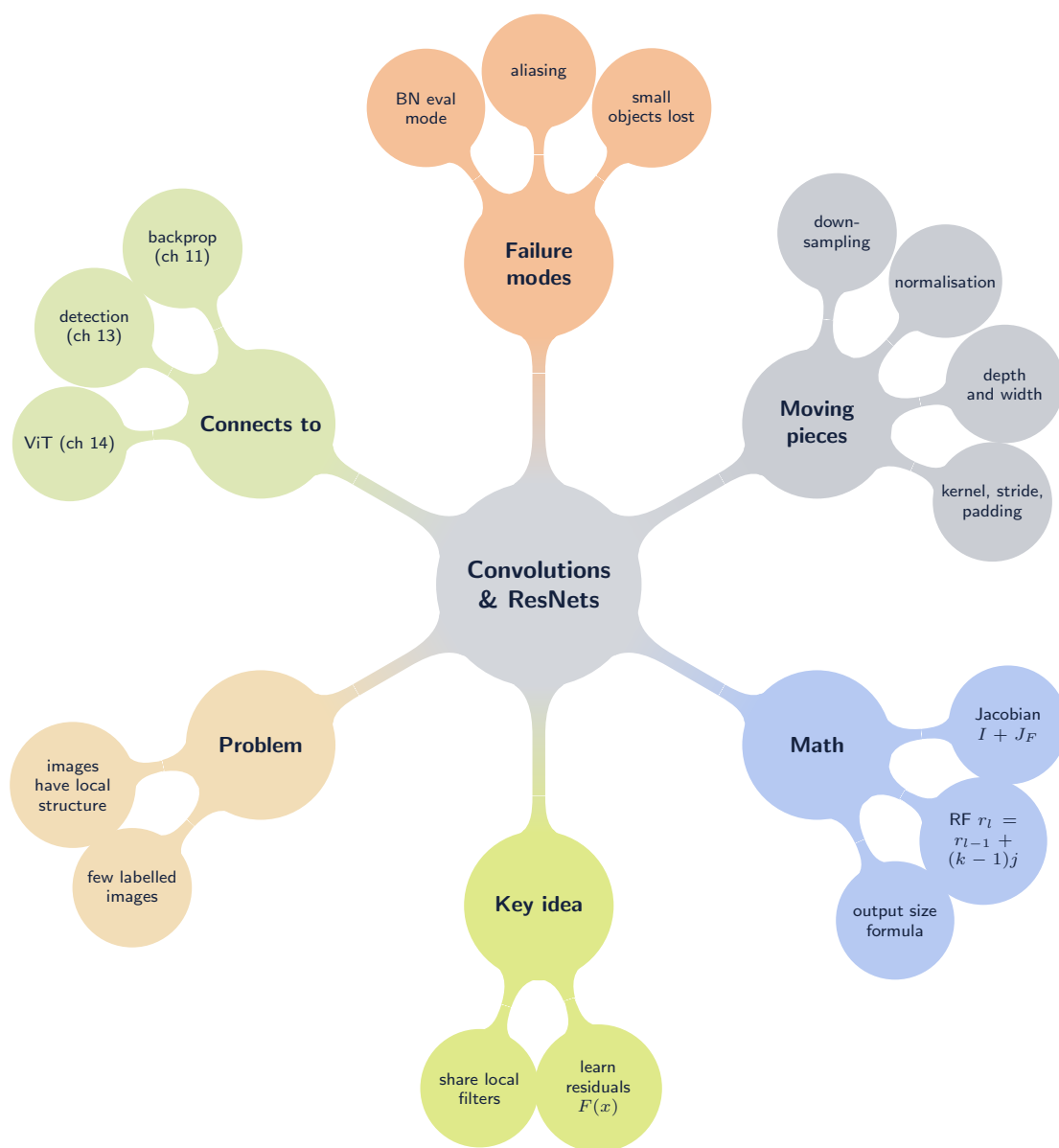


Figure 12.6: Memory map of chapter 12.

lab and gradient-check it.

8. *Implementation.* Carry out the experiment of section 12.11.
9. *Interpretation.* A detector misses most objects smaller than 16 pixels. Which moving pieces would you check first, and how?
10. *Open question.* Convolution builds in translation equivariance. Which other symmetries of images could be built in, and what would be gained or lost?

12.13.1 Answer discussions

1. $\lfloor (32 + 2 - 2 - 1)/2 \rfloor + 1 = 16$; $\lfloor (224 + 6 - 6 - 1)/2 \rfloor + 1 = 112$.
2. $3 \cdot 3 \cdot 8 \cdot 16 = 1152$.
3. Batch normalisation would use the current batch's statistics instead of running averages, and dropout would remain active; the evaluated function differs from the deployed one.
4. Each row of the 3×6 matrix holds (w_1, w_2, w_3) starting two columns later than the previous row (with padding columns removed). The input gradient multiplies by its transpose: a “transposed convolution”, the operation used for learned upsampling in decoders.
5. A unit at layer l spans k_l units of layer $l - 1$ spaced $d_l j_{l-1}$ input pixels apart, adding $(k_l - 1)d_l j_{l-1}$ pixels to the span of one layer- $(l - 1)$ unit.
6. Bottleneck: $256 \cdot 64 + 9 \cdot 64 \cdot 64 + 64 \cdot 256 = 69,632$. Basic at 256: $2 \cdot 9 \cdot 256^2 = 1,179,648$.
7. Route the gradient to the arg-max position of each window (choose the first maximum on ties); the finite-difference check fails exactly at ties, so test with distinct values.
8. Expect plain training loss to stop improving or rise with depth under one shared learning rate, and residual networks to train at all depths; report seeds.
9. Feature-map stride at the head that handles small objects, anchor or assignment sizes, input resizing, and the per-size recall; check with a synthetic image containing objects of known sizes.
10. Rotations and reflections (group-equivariant CNNs, Cohen and Welling, 2016) and scale. Gains: data efficiency where the symmetry holds (microscopy, satellite images). Losses: when orientation matters (text, faces upright), and computational cost.

12.14 Self-check and common mistakes

- ☐ I can compute output shapes, parameter counts and receptive fields for a given stack.
- ☐ I can derive the backward pass of a convolution in 1-D and explain col2im.
- ☐ I can explain equivariance vs. invariance and how striding breaks equivariance.
- ☐ I can explain the Jacobian argument for residual connections and its limits.
- ☐ I can draw U-Net and FPN and say what each skip carries.

Common mistakes: calling a CNN translation-invariant; confusing theoretical with effective receptive field; forgetting that padding changes border behaviour; using batch norm with batch size 2 and training-mode statistics; resizing high-resolution images down until small objects vanish; comparing architectures trained with different recipes; reading learned filters as human-interpretable edges without checking; reshaping instead of transposing axes.

References

- I. Goodfellow, Y. Bengio and A. Courville. *Deep Learning*. MIT Press, 2016, chapter 9 (convolutional networks) and section 8.7.1 (batch normalisation).

- D. H. Hubel and T. N. Wiesel. Receptive fields, binocular interaction and functional architecture in the cat's visual cortex. *Journal of Physiology*, 160(1):106–154, 1962.
- K. Fukushima. Neocognitron. *Biological Cybernetics*, 36(4):193–202, 1980.
- Y. LeCun, L. Bottou, Y. Bengio and P. Haffner. Gradient-based learning applied to document recognition. *Proceedings of the IEEE*, 86(11):2278–2324, 1998.
- A. Krizhevsky, I. Sutskever and G. E. Hinton. ImageNet classification with deep convolutional neural networks. In *NeurIPS*, 2012.
- K. Simonyan and A. Zisserman. Very deep convolutional networks for large-scale image recognition. In *ICLR*, 2015.
- K. He, X. Zhang, S. Ren and J. Sun. Deep residual learning for image recognition. In *CVPR*, 2016. arXiv:1512.03385.
- O. Ronneberger, P. Fischer and T. Brox. U-Net: convolutional networks for biomedical image segmentation. In *MICCAI*, 2015. arXiv:1505.04597.
- J. Long, E. Shelhamer and T. Darrell. Fully convolutional networks for semantic segmentation. In *CVPR*, 2015.
- T.-Y. Lin, P. Dollár, R. Girshick, K. He, B. Hariharan and S. Belongie. Feature pyramid networks for object detection. In *CVPR*, 2017. arXiv:1612.03144.
- W. Luo, Y. Li, R. Urtasun and R. Zemel. Understanding the effective receptive field in deep convolutional neural networks. In *NeurIPS*, 2016.
- A. Veit, M. Wilber and S. Belongie. Residual networks behave like ensembles of relatively shallow networks. In *NeurIPS*, 2016.
- R. Zhang. Making convolutional networks shift-invariant again. In *ICML*, 2019.
- Z. Liu, H. Mao, C.-Y. Wu, C. Feichtenhofer, T. Darrell and S. Xie. A ConvNet for the 2020s. In *CVPR*, 2022.

Chapter 13

Detection and segmentation

Prerequisites: chapter 12 (backbones, FPN, U-Net); chapter 5 (evaluation, grouped splits); chapter 3 (logistic regression and class weighting).

Learning outcomes: after this chapter you can

1. choose the output structure — label, set of boxes, per-pixel labels, per-object masks — that matches the question, and say what each implies for the loss and the metric;
2. compute IoU and Dice, encode boxes relative to anchors, and explain smooth- L_1 and focal losses;
3. compute average precision from scratch — matching, precision–recall, interpolation, IoU thresholds — and state exactly which variant is reported;
4. explain non-maximum suppression, two-stage detection (Faster R-CNN), set prediction (DETR) and promptable segmentation (SAM) as answers to specific problems;
5. diagnose the failures that averages hide: small objects, class imbalance, duplicates, empty masks and leakage between correlated images.

13.1 A concrete problem

The crop-health team now needs more than “is this leaf diseased?”. An agronomist wants to know **how many** lesions there are and **where** (to estimate severity), and a plant pathologist wants the exact **area** of each lesion (to follow growth over days). One image, three questions:

- *Is there disease?* — a label per image (classification).
- *Where are the lesions?* — a variable-size **set** of boxes with classes and scores (detection).
- *Which pixels are lesion?* — a label per pixel (semantic segmentation), or a separate mask for each lesion (instance segmentation).

Each question changes the output, the loss and — most importantly — the metric. A model that reaches 99% “pixel accuracy” can find no lesions at all (section 13.8), and “mAP” means different numbers in different papers. This chapter derives the losses and metrics of dense prediction, the two dominant detector designs, and the promptable segmentation models that are changing how masks are made.

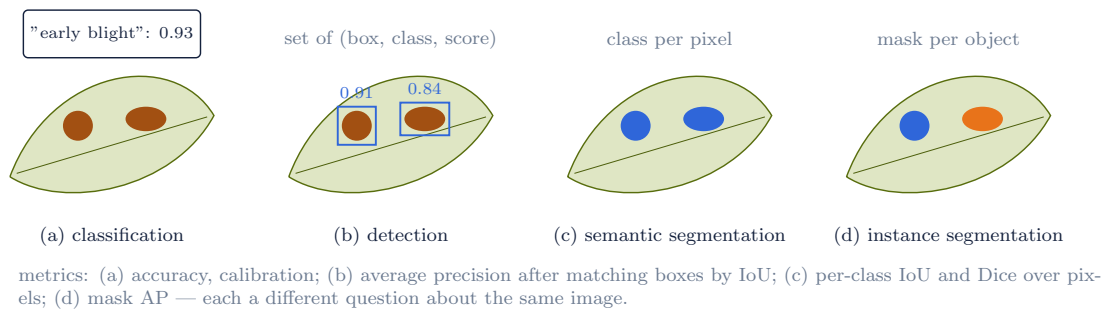


Figure 13.1: Four outputs for the same image. Classification returns a label; detection a set of (box, class, score) triples; semantic segmentation a class per pixel; instance segmentation a mask per object. Each needs its own loss and metric.

13.2 How we got here

Sliding windows and hand-made features (2001–2012). Viola and Jones (2001) detected faces in real time by scanning a cascade of boosted classifiers over every window. Dalal and Triggs (2005) used histograms of oriented gradients with a linear SVM for pedestrians; Felzenszwalb and colleagues (2008–2010) added deformable parts. PASCAL VOC (2005–2012) fixed the evaluation: a detection counts if its box overlaps a ground-truth box with $\text{IoU} \geq 0.5$, and performance is **average precision**.

Region-based CNNs (2014–2017). R-CNN (Girshick and colleagues, 2014) ran a CNN on about 2,000 proposed regions per image and nearly doubled VOC mAP. Fast R-CNN (2015) computed features once and pooled each region from the feature map; **Faster R-CNN** (Ren, He, Girshick and Sun, 2015) replaced external proposals with a learned **region proposal network** using **anchors**. Mask R-CNN (He and colleagues, 2017) added a mask branch for instance segmentation. FPN (chapter 12) handled scale.

One-stage detectors (2016–2020). YOLO (Redmon and colleagues, 2016) and SSD (Liu and colleagues, 2016) predicted boxes densely in one pass, fast but less accurate — until RetinaNet (Lin and colleagues, 2017) showed that the gap came from **class imbalance** among $\sim 100,000$ candidate locations, and fixed it with the **focal loss**. Anchor-free variants (CornerNet, FCOS, CenterNet, 2018–2019) predicted points or distances instead of anchor offsets.

Segmentation (2015–2018). Fully convolutional networks, U-Net and DeepLab’s dilated convolutions (2015–2018) made per-pixel prediction standard; COCO (Lin and colleagues, 2014) made **mAP over IoU thresholds 0.5–0.95** the detection standard and added mask AP.

Sets and prompts (2020–2024). DETR (Carion and colleagues, 2020) treated detection as direct **set prediction** with a transformer and one-to-one Hungarian matching, removing anchors and NMS. Segment Anything (Kirillov and colleagues, 2023) trained a **promptable** segmenter on 1.1 billion masks: a point or box in, a mask out — this chapter’s paper bridge. SAM 2 (2024) extended it to video. Open-vocabulary detectors (OWL-ViT, Grounding DINO) take text queries, connecting to chapters 19–20.

13.3 Assumptions and notation

Symbol	Meaning
$\mathbf{b} = [x_1, y_1, x_2, y_2]$	a box, with continuous corner coordinates ($x_2 > x_1, y_2 > y_1$)
$\text{IoU}(A, B) = A \cap B / A \cup B $	intersection over union of two boxes or masks
$\text{Dice}(A, B) = 2 A \cap B / (A + B)$	Dice coefficient (F1 of pixels)
$\mathbf{a}$	an anchor (reference box)
$\mathbf{t} = (t_x, t_y, t_w, t_h)$	box offsets relative to an anchor
TP, FP, FN	true positives, false positives, false negatives after matching
$P = TP / (TP + FP), R = TP / (TP + FN)$	precision and recall
AP_τ	average precision at IoU threshold τ
p_t	predicted probability of the true class

Assumptions and conventions to state in any report:

1. **Box convention.** Continuous corners here. An inclusive integer-pixel convention adds 1 to widths and changes IoU for small boxes.
2. **Matching rule.** Each ground-truth object may be matched at most once; predictions are matched in descending score order.
3. **The independent unit.** Images from one plant, field, patient or video are correlated; splits and confidence intervals must respect that (chapter 5).

13.4 Mathematical formulation

Overlap. For boxes or masks A, B :

$$\text{IoU} = \frac{|A \cap B|}{|A \cup B|}, \quad \text{Dice} = \frac{2|A \cap B|}{|A| + |B|} = \frac{2 \text{IoU}}{1 + \text{IoU}}.$$

Detection loss (per matched anchor or proposal): classification cross-entropy (or focal loss) plus a box-regression loss on offsets,

$$t_x = \frac{x - x_a}{w_a}, \quad t_y = \frac{y - y_a}{h_a}, \quad t_w = \log \frac{w}{w_a}, \quad t_h = \log \frac{h}{h_a},$$

$$\text{smooth}_{L_1}(r) = \begin{cases} \frac{1}{2}r^2/\beta, & |r| < \beta \\ |r| - \frac{1}{2}\beta, & \text{otherwise,} \end{cases}$$

where (x, y) is a box centre and (w, h) its size.

Focal loss (binary form): $\text{FL}(p_t) = -(1 - p_t)^\gamma \log p_t$; $\gamma = 0$ is cross-entropy.

Average precision. Sort all detections of a class across the dataset by score; walk down the list, marking each as TP or FP by the matching rule; compute P and R after each; replace precision by its **envelope** $\tilde{P}(r) = \max_{r' \geq r} P(r')$; AP is the area under $\tilde{P}$. COCO's AP averages $\tilde{P}$ at 101 recall points and then over IoU thresholds $\tau \in \{0.50, 0.55, \dots, 0.95\}$ and over classes.

Segmentation losses. Per-pixel cross-entropy $-\frac{1}{N} \sum_u \log p_{u,y_u}$, possibly class-weighted; soft Dice $1 - \frac{2 \sum_u p_u y_u + \epsilon}{\sum_u p_u + \sum_u y_u + \epsilon}$.

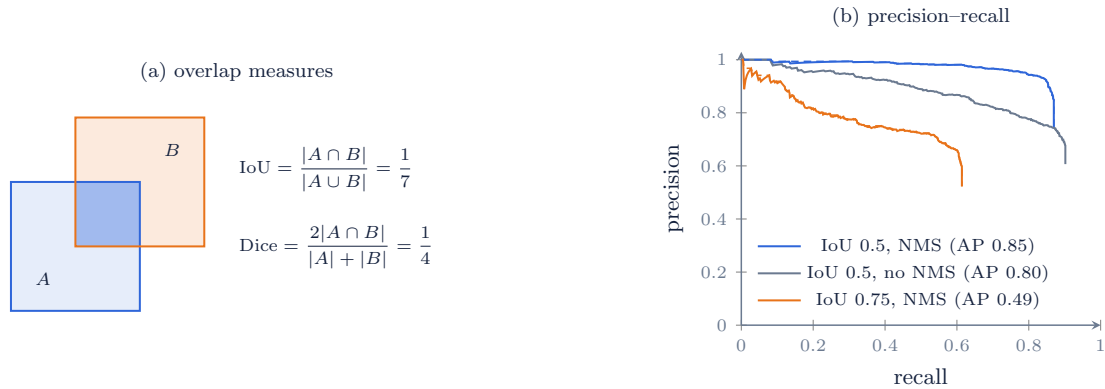


Figure 13.2: (a) IoU and Dice for two 2×2 boxes offset by one unit. (b) Precision-recall curves of a simulated detector: at IoU 0.5 with and without NMS, and at the stricter IoU 0.75; dashed lines are the precision envelopes whose areas are the APs.

13.5 Derivation, with reasons for each step

Step 1 — why outputs must be matched before they are scored. A detector returns a *set*: the number of boxes varies and their order is arbitrary. To compute a loss or a metric, each prediction must first be paired with at most one ground-truth object. Training uses an **assignment rule** (anchors with $\text{IoU} \geq 0.7$ to a ground-truth box are positive, < 0.3 negative, in between ignored, for Faster R-CNN’s RPN; DETR uses an optimal one-to-one matching); evaluation uses **greedy matching** by score. Without “at most once”, a detector could output ten copies of each box and count ten true positives.

Step 2 — IoU and Dice, and their relation. With $I = |A \cap B|$ and $U = |A \cup B| = |A| + |B| - I$: Dice = $2I/(|A| + |B|) = 2I/(U + I)$; divide numerator and denominator by U to get $2\text{IoU}/(1 + \text{IoU})$. Both are monotone in each other for a single pair, so they rank one image’s predictions identically, but averaged over images they differ (Dice is more forgiving at low overlap: IoU 0.33 is Dice 0.5). Both are scale-invariant: a 2-pixel error is catastrophic for a 4-pixel lesion and negligible for a 200-pixel leaf — the main reason small objects score lower at a fixed IoU threshold (section 13.8).

Step 3 — box parameterisation. Regressing raw corner coordinates mixes scales: an error of 5 pixels is small for a large box and huge for a small one. Encoding the centre shift **in units of the anchor’s size** and the size **as a log ratio** makes targets roughly scale-free and lets widths stay positive after decoding ($w = w_a e^{t_w}$). The log also makes doubling and halving symmetric. Smooth- L_1 is quadratic near zero (smooth gradients for fine adjustment) and linear far away (bounded gradients for badly matched anchors) — the Huber loss of robust statistics. Modern detectors often add an IoU-based loss (GIoU, DIoU) that directly optimises the overlap the metric uses.

Step 4 — class imbalance and the focal loss. A dense detector evaluates $\sim 100,000$ anchors per image, a handful of which are objects. Each easy background anchor has a small cross-entropy, but there are so many that they dominate the sum: in the lab’s simulation, easy negatives ($p < 0.1$) contribute 81% of the total cross-entropy. The focal loss multiplies each term by $(1 - p_t)^\gamma$, which is near zero for well-classified examples:

$$\frac{\text{FL}}{\text{CE}} = (1 - p_t)^\gamma : \quad p_t = 0.95, \gamma = 2 \Rightarrow 0.0025.$$

With $\gamma = 2$, the easy negatives’ share falls to 20%, so training concentrates on hard and positive examples. The same issue appears in segmentation, where background pixels dominate; class weighting and Dice losses are the usual remedies (step 7).

Step 5 — non-maximum suppression. Dense and two-stage detectors produce several overlapping boxes per object. Greedy NMS keeps the highest-scoring box, removes all boxes overlapping it by more than a threshold, and repeats. Its threshold is a real hyperparameter: too low and two adjacent lesions merge into one detection (lab: AP50 falls from 0.85 to 0.76 at threshold 0.3); too high and duplicates survive as false positives. Without NMS, the simulated detector’s AP50 drops from 0.85 to 0.80. DETR avoids NMS because its one-to-one matching *penalises* duplicates during training (figure 13.3b).

Step 6 — average precision, carefully. For one class, sort detections by score. Precision after the k -th detection is TP_k/k ; recall is TP_k/N_{gt} . Recall increases by $1/N_{\text{gt}}$ at each true positive. The raw precision curve zig-zags; the envelope $\tilde{P}(r) = \max_{r' \geq r} P(r')$ asks “what precision could I get if I chose a threshold giving at least this recall?”. Then:

$$\text{AP} = \int_0^1 \tilde{P}(r) dr \quad (\text{all-point, VOC 2010+}), \quad \text{AP}_\tau^{\text{COCO}} = \frac{1}{101} \sum_{r \in \{0, 0.01, \dots, 1\}} \tilde{P}(r).$$

Choices that change the number and must be reported: the IoU threshold (AP50, AP75, or averaged over 0.50:0.05:0.95), the interpolation, the maximum number of detections per image, how “difficult” or crowd objects are treated, and how classes are averaged. Small worked example: one object, two detections with scores 0.9 and 0.8. If the first is correct, $P = (1, \frac{1}{2})$, $R = (1, 1)$ and AP is 1 — the duplicate costs nothing because it comes *after* full recall. If only the second is correct, $P = (0, \frac{1}{2})$, $R = (0, 1)$ and AP is 0.5.

Step 7 — segmentation losses and imbalance. Per-pixel cross-entropy treats every pixel equally, so a model that lowers its loss by predicting background on ambiguous pixels sacrifices small lesions, which have few pixels. Two corrections:

- **Class-weighted cross-entropy** multiplies foreground terms by $w = (1 - f)/f$ for a foreground fraction f , balancing the total weight. It raises recall (lab: small-lesion recall from 0.42 to 0.65) at the cost of more false-positive pixels (Dice falls from 0.83 to 0.78).
- **Soft Dice** optimises an overlap ratio directly; its gradient for pixel u is $-(2y_u D - N)/D^2$ with $N = 2 \sum p y + \epsilon$, $D = \sum p + \sum y + \epsilon$, which depends on all pixels at once and does not reward the background. It is not exactly the hard Dice of thresholded masks, and its ϵ decides what happens for empty masks.

Neither is “correct”: they encode different costs of errors. Medical segmentation commonly uses cross-entropy + Dice.

Step 8 — empty masks and how averages are taken. If both prediction and ground truth are empty, IoU and Dice are 0/0. Conventions differ: score 1 (“correctly found nothing”), exclude the image, or report detection of presence separately. The choice can move a mean Dice by many points on datasets where many images have no lesion. Likewise, **global** Dice (pool all pixels) and **mean per-image** Dice differ: the lab’s cross-entropy

model has global Dice 0.83 and mean per-image Dice 0.77, because images with only small lesions score worse. Say which one you report.

Step 9 — two-stage detection: Faster R-CNN. We describe the modern form used in most libraries. The original Faster R-CNN (Ren et al., 2015) used a single backbone feature map and **RoI pooling**, which rounds proposal coordinates to the feature grid. Two later additions are now standard: a feature pyramid network (FPN; Lin et al., 2017) for scale, and **RoIAlign**, introduced with Mask R-CNN (He et al., 2017). A backbone (here with FPN) computes shared feature maps. The **region proposal network** slides a small network over every position and, for each of several anchors, predicts an objectness score and box offsets (step 3); the top few hundred proposals after NMS go to the second stage. **RoI Align** bilinearly samples a fixed 7×7 feature grid inside each proposal — no rounding of coordinates, which matters for masks — and a head classifies the region and refines its box. The design separates “is there something here?” from “what exactly is it?”, and each stage sees a more balanced problem (figure 13.3a).

Step 10 — set prediction: DETR. A transformer (chapter 14) decodes N learned **object queries** into N predictions of (class or “no object”, box). Training finds the one-to-one matching between predictions and ground-truth objects that minimises a matching cost (class probability plus box L_1 and GIoU) — the Hungarian algorithm — and applies the loss to matched pairs, with “no object” as the target for the rest. Because two predictions cannot both be matched to one object, duplicates are penalised by the loss itself and NMS is unnecessary. DETR trained slowly and struggled with small objects at first; Deformable DETR and DINO-DETR (2021–2022) fixed much of this (figure 13.3b).

Step 11 — promptable segmentation changes the contract. A promptable model receives extra information — a click, a box, a rough mask — and returns a mask for the object indicated (step through figure 13.5). Comparing it with an automatic segmenter without saying who supplied the prompts, how many, and from what (ground-truth boxes? a detector?) compares different tasks. One click is ambiguous (the lesion, the leaf, or the plant?), which is why SAM returns three masks with predicted quality scores.

13.6 Numerical worked example

Box IoU. $A = [0, 0, 2, 2]$, $B = [1, 1, 3, 3]$: each area 4, intersection $[1, 1, 2, 2]$ of area 1, union $4 + 4 - 1 = 7$: $\text{IoU} = 1/7 = 0.143$, $\text{Dice} = 2/8 = 0.25$. A correct class with this box fails AP50.

Box encoding. Anchor $[10, 10, 30, 50]$ (centre $(20, 30)$, size 20×40), ground truth $[12, 8, 36, 52]$ (centre $(24, 30)$, size 24×44): $t = (4/20, 0/40, \log 1.2, \log 1.1) = (0.200, 0, 0.182, 0.095)$. Decoding returns the box exactly. Smooth- L_1 ($\beta = 1$): 0.125 at $r = 0.5$, 1.5 at $r = 2$.

Precision and recall. $TP = 8$, $FP = 2$, $FN = 4$: $P = 0.8$, $R = 8/12 = 0.667$.

Mask overlap. Ground truth 20 pixels, prediction 30, overlap 15: $\text{IoU} = 15/(20+30-15) = 3/7 = 0.429$, $\text{Dice} = 30/50 = 0.6 = 2(3/7)/(1 + 3/7)$.

Focal weighting. An easy negative with $p = 0.02$ (so $p_t = 0.98$) has CE 0.020 and, with $\gamma = 2$, focal loss $0.0004 \cdot 0.020 = 8 \times 10^{-6}$; a hard positive with $p_t = 0.3$ has CE 1.20 and focal loss $0.49 \cdot 1.20 = 0.59$. The ratio between them grows from 60 to about 74,000.

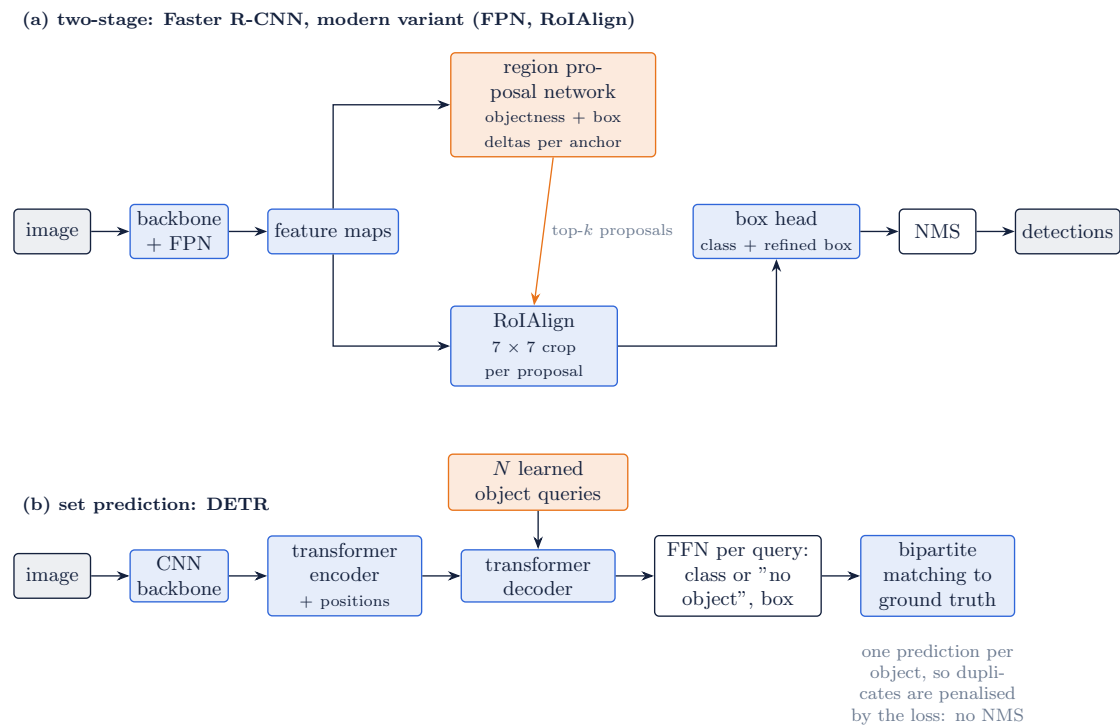


Figure 13.3: Two detector designs. (a) A modern Faster R-CNN variant. The two-stage design, redrawn after Fig. 2 of Ren et al. (2015), is a region proposal network on shared features, then per-region classification and refinement, then NMS; the original used RoI pooling on one feature map, while the FPN backbone (Lin et al., 2017) and RoIAlign (He et al., 2017, Mask R-CNN) are later additions. (b) DETR, redrawn after Fig. 2 of Carion et al. (2020): a transformer turns learned object queries into a set of predictions matched one-to-one to the ground truth. Papers: [arXiv:1506.01497](https://arxiv.org/abs/1506.01497), [arXiv:2005.12872](https://arxiv.org/abs/2005.12872).

13.7 Small implementation and checks

The lab `labs/ch13-dense/dense.py` implements every metric used in this chapter. The core of matching and AP:

```
def match(pred, scores, gt, thr=0.5):
    order = np.argsort(-scores); used = np.zeros(len(gt), bool); tp =
    → np.zeros(len(order), bool)
    ious = box_iou(pred[order], gt)
    for k in range(len(order)):                # best-scoring
        → first
        cand = np.where(~used & (ious[k] >= thr))[0] # unmatched objects
    → above threshold
        if len(cand):
            j = cand[np.argmax(ious[k, cand])]; used[j] = tp[k] = True
    return tp, scores[order]

precision = ctp / (ctp + cfp); recall = ctp / n_gt
env = np.maximum.accumulate(precision[::-1])[::-1] # precision envelope
ap = np.sum(np.diff(np.r_[0, recall]) * env)        # all-point AP
```

The segmentation model is fully convolutional but tiny: fixed features (the image and box-filtered means at three scales, and centre-surround differences) followed by a learned 1×1 classifier — logistic regression per pixel — trained with each loss.

Checks, in `test_dense.py`: the worked IoU; encode/decode round trip and smooth- L_1 values; NMS removes a duplicate and, at a low threshold, also a real neighbour; a duplicate is a false positive, and AP is 1 or 0.5 in the two worked cases; AP falls with a stricter IoU and without NMS; focal loss reduces the easy-negative share; the worked mask metrics and the empty-mask convention; a finite-difference check of the soft Dice gradient; and an all-background predictor with high pixel accuracy and zero Dice.

Use an independent implementation as a second check. Before publishing numbers, compare your AP with the official evaluation code of the benchmark (for COCO, `pycocotools`) on the same predictions; small differences in conventions are common and must be explained.

13.8 Experiment and expected observations

Run `python labs/ch13-dense/dense.py` (seed 0). The simulated detector sees 300 images with 1–6 objects of 8–60 pixels; it finds 90% of objects, with corner noise proportional to object size, a 30% chance of a duplicate, and on average one background false positive per image.

detector evaluation	AP50	AP75	COCO mAP@[.5:.95]
without NMS	0.804	0.497	0.462
with NMS (threshold 0.5)	0.852	0.488	0.478
NMS threshold 0.3 / 0.7	0.758 / 0.846	—	—
constant 1.5-pixel corner noise, objects < 24 px	0.370	0.050	—
constant 1.5-pixel corner noise, objects ≥ 40 px	0.543	0.313	—

Focal loss on 100,000 simulated anchors (100 positive): the share of the total loss from

easy negatives is 0.81 ($\gamma = 0$), 0.53 ($\gamma = 1$), 0.20 ($\gamma = 2$).

Segmentation: 60 training and 60 test images, 10.7% of pixels are lesion.

loss	pixel acc.	Dice (global)	IoU	mean per-image Dice	lesion recall, radius 1.5–3 / 3–5 / 5–8 px
all	0.895	0	0	0	0 / 0 / 0
background					
cross- entropy	0.967	0.833	0.713	0.769	0.42 / 0.64 / 0.89
weighted	0.944	0.779	0.638	0.727	0.65 / 0.91 / 1.00
cross- entropy					
soft Dice	0.967	0.841	0.725	0.785	0.42 / 0.79 / 0.96

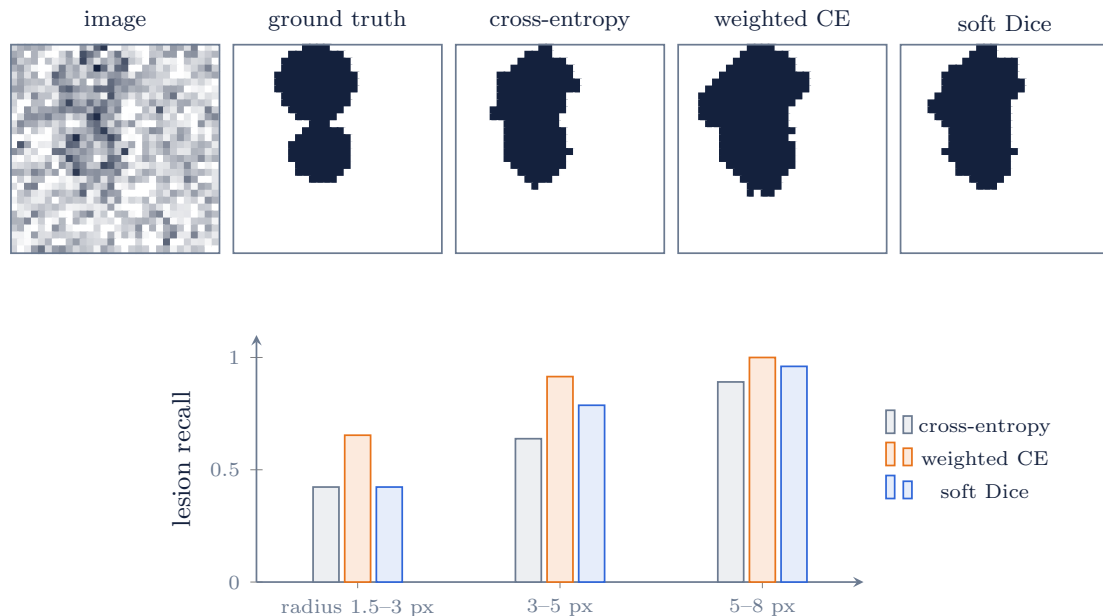


Figure 13.4: A per-pixel segmenter trained with three losses. Top: a crop of one test image, its mask and the predictions. Bottom: the fraction of lesions found, by radius. Class weighting finds more small lesions at the price of more false-positive pixels.

What to notice:

1. **Pixel accuracy is the wrong metric.** Predicting “no lesion” everywhere scores 89.5% while finding nothing.
2. **The loss sets the trade-off.** Weighted cross-entropy finds 65% of the smallest lesions instead of 42% but has a lower Dice; soft Dice has the best overlap but not the best small-lesion recall. Choose by the cost of a missed lesion versus a false alarm, then report both kinds of numbers.
3. **Averages hide sizes.** Recall for large lesions is near 1 for every loss; the differences are entirely in the small ones. Always break results down by object size.

4. **Strict IoU punishes small objects.** With the same 1.5-pixel localisation noise, AP75 is 0.05 for small objects and 0.31 for large: identical absolute errors are relatively larger for small boxes.
5. **NMS helps AP50 and slightly hurts AP75** here: suppressing a well-localised duplicate of a poorly localised top-scoring box can remove the only box that passes the strict threshold. Tune NMS for the metric that matters.

13.9 Moving pieces: diagnosing and fixing

Moving piece	Symptom when it is wrong	How to confirm	Fix
Metric definition	numbers not comparable with a paper	read the evaluation code; recompute with the official tool	report AP variant, IoU thresholds, interpolation, max detections
Matching rule	duplicates counted as hits; AP too high	count predictions per object	one match per object; greedy by score
NMS threshold	merged neighbours (low) or duplicates (high)	AP vs. threshold; inspect crowded scenes	tune on validation; soft-NMS; set prediction
Object size	small objects missed or poorly localised	AP or recall by size bucket	higher resolution, finer FPN levels, tiling, size-aware anchors
Anchor design / assignment	some sizes or shapes never matched as positives	IoU of each ground-truth box with its best anchor	anchor sizes and ratios from data; anchor-free heads
Class imbalance	high pixel accuracy, low recall	per-class recall; confusion matrix	focal loss, class weights, Dice, sampling
Box regression	correct classes, low AP75	IoU distribution of true positives	IoU-based losses; better features at the right scale
Empty images	mean Dice swings with convention	count empty ground truths and predictions	declare the convention; report presence detection separately
Annotation noise	ceiling on IoU; inconsistent edges	inter-annotator agreement on a subset	measure it; train with soft boundaries; report it as an upper bound
Correlated images	test far better than deployment	group split by plant, field, patient or video	grouped splits and grouped bootstrap (chapter 5)
Prompt protocol (SAM)	promptable model “beats” automatic one	check where prompts come from	report prompt source and count; compare like with like

First things to try: (1) recompute the metric with an independent implementation; (2) break results down by object size and by class; (3) visualise false positives and false negatives on 20 random images; (4) check that train and test do not share plants, fields or patients.

13.10 Application, limitations and failure cases

Application (educational case study): medical image segmentation. A team segments skin lesions in dermoscopy images from 300 patients, several images per patient. Correct practice:

- **Split by patient**, never by image: images of one patient share skin type, lesion and camera. A random image split lets the model recognise the patient and inflates Dice (chapter 5’s leakage).
- **Report per-patient** mean Dice with a confidence interval from a **patient-level bootstrap**, and the distribution, not only the mean; inspect the worst decile.
- **Measure annotator agreement** on a subset: if two dermatologists agree to Dice 0.85, a model at 0.90 against one of them is within annotation noise, not superhuman.
- **Declare** the empty-mask convention and the handling of images with several lesions.

The same principles apply to field images grouped by farm and date. These are educational illustrations of methodology, not claims about any deployed clinical or agricultural system.

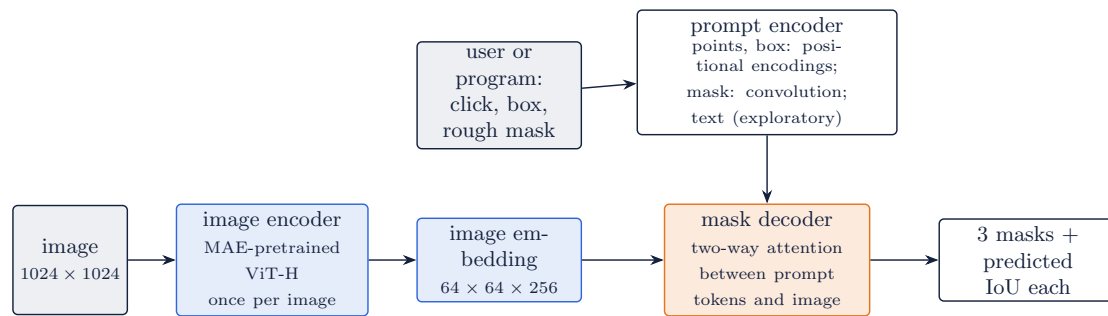
Limitations:

- AP summarises a whole curve; two detectors with equal AP can behave differently at the operating threshold you will deploy. Report precision and recall at that threshold too.
- IoU-based metrics are scale-invariant and treat all pixels of a mask alike; boundary metrics (boundary F-score, Hausdorff distance) are needed when edges matter.
- Synthetic results such as the lab’s illustrate mechanisms; they do not predict the size of effects on real data.

13.11 Bridge to a research paper

Kirillov, Mintun, Ravi, Mao, Rolland, Gustafson, Xiao, Whitehead, Berg, Lo, Dollár and Girshick (2023), *Segment Anything* (ICCV 2023).

- **Problem:** segmentation models are trained per dataset and per class list; a general “segment this object” tool needs a task that is defined for any object and a dataset large enough to learn it.
- **Mechanism:** define *promptable segmentation* — return a valid mask for any prompt; an MAE-pretrained ViT-H image encoder (chapters 14 and 16) computes an embedding once; a two-way attention decoder combines prompt tokens with it; three masks are output to handle ambiguity, trained with focal plus Dice loss on the best of the three (step 7); a **data engine** in which the model helps annotators produced 1.1 billion masks.
- **Central claim:** strong zero-shot performance from a single point or box on 23 segmentation datasets, often close to or better than interactive baselines, and useful zero-shot transfer to edge detection and proposal generation.
- **Assumptions to question:** what counts as a “valid” mask is defined by annotators of natural images; thin, low-contrast or textural structures (cracks, lesions, cells)



training: a "data engine" of three stages (model-assisted manual, semi-automatic, fully automatic) produced SA-1B, 1.1 billion masks on 11 million images; losses: focal + Dice on the best of the three masks, plus a regression loss for the predicted IoU. Ambiguity (a click on a shirt: shirt or person?) is handled by returning several masks.

Figure 13.5: Segment Anything, redrawn after Fig. 4 of Kirillov et al. (2023). A large image encoder runs once per image; a light prompt encoder and mask decoder run per prompt and return three candidate masks with predicted quality. Paper: [arXiv:2304.02643](https://arxiv.org/abs/2304.02643).

are harder; the results depend on how prompts are simulated; much of SA-1B was annotated *by the model*.

- **Affordable experiment:** on the lab's synthetic lesions, simulate a promptable segmenter by giving the per-pixel model one extra feature channel: a Gaussian centred at a click inside the lesion. Compare small-lesion recall and Dice with and without the click, with clicks from ground truth versus clicks from a noisy detector. How much of the gain is the prompt's information, and how much survives realistic prompts?

13.12 Memory map

Summary. Classification, detection and segmentation answer different questions and produce different structures: a label, a set of boxes, labels per pixel, masks per object. Sets must be matched before they can be scored; IoU and Dice measure overlap and are related by $\text{Dice} = 2\text{IoU}/(1 + \text{IoU})$. Detectors encode boxes relative to anchors with log-size offsets, fight extreme class imbalance with focal loss, and remove duplicates with NMS — or, in DETR, with one-to-one matching. AP is the area under the precision envelope and comes in several variants that must be named. Segmentation losses trade recall of small structures against overlap. Averages hide object size, empty images and correlated samples. Promptable models such as SAM add information through prompts, which must be reported when comparing.

13.13 Exercises

1. *Conceptual.* With $TP = 8$, $FP = 2$, $FN = 4$, compute precision and recall.
2. *Conceptual.* Why is duplicate handling necessary in detection evaluation?
3. *Conceptual.* What extra information must be documented when evaluating promptable segmentation?
4. *Derivation.* Prove $\text{Dice} = 2\text{IoU}/(1 + \text{IoU})$ and show that $\text{Dice} \geq \text{IoU}$ with equality only at 0 and 1.
5. *Derivation.* For a square box of side s shifted by δ pixels in x only, find IoU as a function of δ/s . What shift makes a box of side 8 fail IoU 0.75? Side 64?



Figure 13.6: Memory map of chapter 13.

6. *Derivation.* Derive the gradient of the soft Dice loss with respect to p_u and explain why it does not vanish for confident background pixels in the same way as cross-entropy's.
7. *Implementation.* Implement soft-NMS (decay scores by $e^{-\text{IoU}^2/\sigma}$ instead of removing boxes) in the lab and compare AP50 and AP75 on the simulated detector, especially on crowded images.
8. *Implementation.* Carry out the experiment of section 13.11.
9. *Interpretation.* A paper reports “mAP 45.2” without further detail. List what you need to know before comparing it with your 43.8.
10. *Open question.* DETR removes NMS by learning one-to-one assignments. Could segmentation benefit from an analogous set-based formulation, and what would replace per-pixel losses?

13.13.1 Answer discussions

1. $P = 8/10 = 0.8$, $R = 8/12 = 0.667$.
2. Otherwise a detector could inflate true positives by returning many copies of one object; matching at most once turns extra copies into false positives.
3. The prompt type, number and source (ground truth, a detector, a human), and any annotation information available to the prompt generator.
4. Algebra as in step 2. $\text{Dice} - \text{IoU} = \text{IoU}(1 - \text{IoU})/(1 + \text{IoU}) \geq 0$.
5. $\text{IoU} = (s - \delta)/(s + \delta)$; $\text{IoU} < 0.75$ when $\delta/s > 1/7$: about 1.1 pixels for $s = 8$ and 9.1 pixels for $s = 64$.
6. $\partial L/\partial p_u = -(2y_u D - N)/D^2$; for a background pixel ($y_u = 0$) it is $N/D^2 > 0$ regardless of p_u , so false-positive mass is always penalised in proportion to the overlap already achieved, rather than vanishing as $p_u \rightarrow 0$ like cross-entropy's.
7. Soft-NMS usually helps crowded scenes and AP at strict thresholds, at the cost of more low-score boxes.
8. Expect large gains with ground-truth clicks, especially for small lesions, and smaller gains with noisy clicks; the difference is the information supplied by the prompt.
9. The dataset and split, AP variant (IoU thresholds, interpolation, area ranges), maximum detections per image, test-time augmentation, input resolution, backbone and pretraining data, and whether the number is from the official evaluation server.
10. Mask-classification models (MaskFormer, Mask2Former) predict a set of masks with classes, matched one-to-one to ground-truth segments; per-pixel losses become mask losses on matched pairs.

13.14 Self-check and common mistakes

- ☐ I can compute IoU, Dice, precision, recall and AP by hand for a small example.
- ☐ I can encode and decode a box relative to an anchor.
- ☐ I can explain why focal loss and class weighting are needed.
- ☐ I can explain NMS, Faster R-CNN and DETR, and what replaces NMS in DETR.
- ☐ I can list the conventions a dense-prediction result must state.

Common mistakes: reporting pixel accuracy for segmentation; “mAP” without the variant; counting duplicates as true positives; tuning NMS on the test set; averaging Dice with an undeclared empty-mask rule; random image splits for data with patients, plants or videos; comparing a prompted model with an unprompted one; resizing images until small objects vanish; mixing inclusive and exclusive box conventions.

References

- I. Goodfellow, Y. Bengio and A. Courville. *Deep Learning*. MIT Press, 2016, section 12.2 (computer vision).
- P. Viola and M. Jones. Rapid object detection using a boosted cascade of simple features. In *CVPR*, 2001.
- M. Everingham, L. Van Gool, C. K. I. Williams, J. Winn and A. Zisserman. The PASCAL visual object classes (VOC) challenge. *IJCV*, 88(2):303–338, 2010.
- T.-Y. Lin et al. Microsoft COCO: common objects in context. In *ECCV*, 2014.
- R. Girshick, J. Donahue, T. Darrell and J. Malik. Rich feature hierarchies for accurate object detection and semantic segmentation. In *CVPR*, 2014.
- S. Ren, K. He, R. Girshick and J. Sun. Faster R-CNN: towards real-time object detection with region proposal networks. In *NeurIPS*, 2015. arXiv:1506.01497.
- K. He, G. Gkioxari, P. Dollár and R. Girshick. Mask R-CNN. In *ICCV*, 2017.
- T.-Y. Lin, P. Dollár, R. Girshick, K. He, B. Hariharan and S. Belongie. Feature pyramid networks for object detection. In *CVPR*, 2017.
- T.-Y. Lin, P. Goyal, R. Girshick, K. He and P. Dollár. Focal loss for dense object detection. In *ICCV*, 2017.
- J. Redmon, S. Divvala, R. Girshick and A. Farhadi. You only look once: unified, real-time object detection. In *CVPR*, 2016.
- N. Carion, F. Massa, G. Synnaeve, N. Usunier, A. Kirillov and S. Zagoruyko. End-to-end object detection with transformers. In *ECCV*, 2020. arXiv:2005.12872.
- F. Milletari, N. Navab and S.-A. Ahmadi. V-Net: fully convolutional neural networks for volumetric medical image segmentation (soft Dice loss). In *3DV*, 2016.
- A. Kirillov et al. Segment anything. In *ICCV*, 2023. arXiv:2304.02643.
- L. Maier-Hein et al. Metrics reloaded: recommendations for image analysis validation. *Nature Methods*, 21:195–212, 2024.

Chapter 14

Attention and vision transformers

Prerequisites: chapter 1 (matrix calculus, shapes); chapter 11 (backpropagation, gradient checking). Chapter 12 (convolutions) is useful for comparison, not required.

Bridge readings: sections 1.3–1.5; section 11.5.

Learning outcomes: after this chapter you can

1. write scaled dot-product attention and give the shape of every tensor;
2. explain the $1/\sqrt{d_k}$ scale, the stable softmax, and masking, and show what goes wrong without them;
3. derive the attention backward pass and verify it with finite differences;
4. explain why self-attention without positions is permutation-equivariant, and how position encodings, LayerNorm, residual connections and MLPs complete a transformer block;
5. turn an image into ViT tokens, count a ViT's parameters, compute and memory, and compare its inductive biases with a CNN's.

14.1 A concrete problem

To decide whether a patch in an aerial image shows a road, it helps to look at patches far away: roads are long and connected. A 3×3 convolution sees only neighbours. Its receptive field grows by two pixels per layer (chapter 12), so long-range context takes many layers, and information from far away arrives diluted through many intermediate steps.

We want an operation in which **every token can read from every other token**, with weights that depend on the *content* of the tokens rather than on fixed offsets. **Attention** is such an operation. A **vision transformer** (ViT) applies it to image patches.

14.2 How we got here

A bottleneck in translation (2014). Sequence-to-sequence networks (Sutskever, Vinyals and Le, 2014) translated a sentence by compressing it into one fixed-length vector and decoding from it. Long sentences did not fit. Dzmitry Bahdanau, Kyunghyun Cho and Yoshua Bengio (2014) let the decoder, at each output word, compute a weighted average over *all* encoder states, with weights from a learned relevance score — **attention**. The fixed bottleneck was gone, and the attention weights even showed which source words each output word used. Kelvin Xu and colleagues (2015) did the same for image captioning, attending over regions of a CNN feature map.

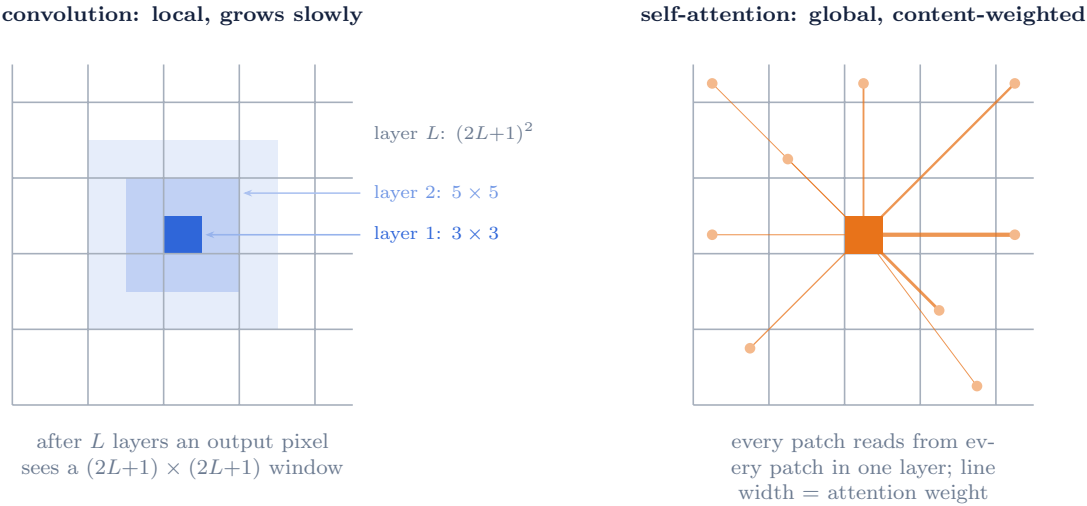


Figure 14.1: Left: a stack of 3×3 convolutions sees a window that grows by two pixels per layer. Right: one self-attention layer lets every patch read from every other patch, with weights that depend on content.

Attention is all you need (2017). Ashish Vaswani and colleagues dropped recurrence entirely. The **transformer** is a stack of blocks, each a multi-head self-attention layer and a position-wise MLP, wrapped in residual connections and layer normalisation, with position encodings added to the inputs. Removing recurrence made training parallel across the sequence, and the architecture scaled (BERT, GPT and their successors).

Into vision (2018–2021). Non-local networks (Wang et al., 2018) inserted self-attention into CNNs for video. DETR (Carion et al., 2020) used a transformer to output object boxes directly (chapter 13). Alexey Dosovitskiy and colleagues (2020) went all the way: cut the image into 16×16 patches, treat each as a token, and use an almost unmodified transformer. **ViT** matched or beat the best CNNs — but only when pretrained on very large datasets (hundreds of millions of images). On ImageNet alone it lost, because it lacks a CNN’s built-in locality and translation equivariance. DeiT (Touvron et al., 2021) closed much of that gap with strong augmentation and distillation, and Swin (Liu et al., 2021) restored some locality with windowed, hierarchical attention.

What broke, and what came next. *Quadratic cost* in the number of tokens: attention matrices are $\tau \times \tau$, which led to windowed and sparse variants and to FlashAttention (Dao et al., 2022), which computes exact attention without ever storing the full matrix. *Data hunger*: answered by self-supervised pretraining on unlabelled images (chapters 15–17) and by image–text pretraining (chapter 19). *Artifacts*: large ViTs develop a few high-norm background tokens used as scratch space; registers fix this (Darcet et al., 2024 — the paper bridge). Attention is now the common building block of almost every model in Parts IV and V.

14.3 Assumptions and notation

Symbol	Shape	Meaning
τ	scalar	number of tokens (sequence length)

Symbol	Shape	Meaning
$\mathbf{X}$	$\tau \times d$	input tokens of width d , one token per row $\mathbf{X}_{i,:}$
$\mathbf{W}_Q, \mathbf{W}_K$	$d \times d_k$	query and key projections
$\mathbf{W}_V$	$d \times d_v$	value projection
$\mathbf{Q} = \mathbf{XW}_Q, \mathbf{K} = \mathbf{XW}_K$	$\tau \times d_k$	queries, keys (self-attention)
$\mathbf{V} = \mathbf{XW}_V$	$\tau \times d_v$	values
$\mathbf{S}$	$\tau \times \tau'$	scores, $\mathbf{S} = \mathbf{QK}^\top / \sqrt{d_k}$
$\mathbf{A}$	$\tau \times \tau'$	attention weights, the row-wise softmax of $\mathbf{S}$
$\mathbf{O}$	$\tau \times d_v$	output $\mathbf{AV}$
h	scalar	number of heads; $d_k = d_v = d/h$
L	scalar	a loss; $\nabla_{\mathbf{Z}} L$ has the shape of $\mathbf{Z}$

In this chapter $\mathbf{K}$ is the key matrix, not a Gram matrix. For **cross-attention**, queries come from one sequence (τ tokens) and keys and values from another (τ' tokens). In **self-attention**, $\tau' = \tau$.

Assumptions:

1. A token is a vector. *What* the tokens are (patches, words, frames) is a modelling decision made before attention is applied.
2. Relevance between tokens is well modelled by a dot product after learned linear maps.
3. Position, if it matters, must be injected explicitly (section 14.5, step 4).

14.4 Mathematical formulation

Scaled dot-product attention is

$$\text{Attn}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax}\left(\frac{\mathbf{QK}^\top}{\sqrt{d_k}}\right) \mathbf{V},$$

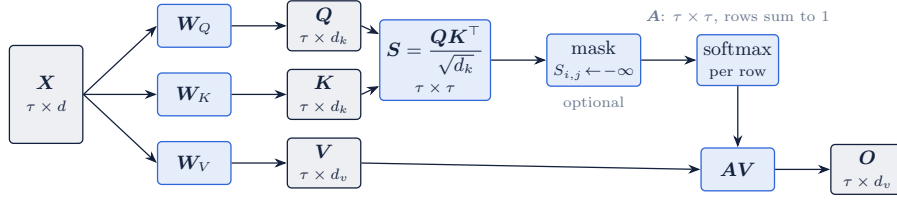
$$\text{softmax}(\mathbf{s})_j = \frac{\exp(s_j)}{\sum_{j'} \exp(s_{j'})} \quad \text{for each row } \mathbf{s}.$$

Each output row $\mathbf{O}_{i,:} = \sum_j A_{i,j} \mathbf{V}_{j,:}$ is a **convex combination of value rows**: the weights are non-negative and sum to one. Compare kernel smoothing (chapter 4): a weighted average of values with weights from a similarity — except that here the similarity $\exp(\mathbf{q}^\top \mathbf{k} / \sqrt{d_k})$ is computed between *learned* projections, so the model learns what “similar” means. **Multi-head attention** runs h attentions on lower-dimensional projections in parallel and mixes the results:

$$\text{MHA}(\mathbf{X}) = [\mathbf{O}^{[1]}, \dots, \mathbf{O}^{[h]}] \mathbf{W}_O, \quad \mathbf{O}^{[j]} = \text{Attn}(\mathbf{XW}_Q^{[j]}, \mathbf{XW}_K^{[j]}, \mathbf{XW}_V^{[j]}),$$

where $[\cdot, \dots, \cdot]$ concatenates along columns and $\mathbf{W}_O \in \mathbb{R}^{d \times d}$. The bracketed superscript indexes heads; ⁽ⁱ⁾ stays reserved for examples. Several heads can attend to different things

at once — one to a nearby edge, another to a distant object of the same colour — which a single softmax, forced to produce one weighted average, cannot.



Each output row $O_{i,:} = \sum_j A_{i,j} V_{j,:}$ is a convex combination of value rows. Multi-head: run h copies with $d_k = d_v = d/h$, concatenate $[O^{[1]}, \dots, O^{[h]}]$ ($\tau \times d$) and multiply by W_O ($d \times d$). Backward through softmax: $\nabla_S L = A \odot (\nabla_A L - ((\nabla_A L \odot A) \mathbf{1} \mathbf{1}^\top))$.

Figure 14.2: Scaled dot-product self-attention with the shape of every tensor, as defined by Vaswani et al. (2017). The mask is applied to the scores before the softmax.

A **transformer block** (pre-norm form) combines attention with an MLP, each wrapped in a residual connection:

$$\mathbf{X}' = \mathbf{X} + \text{MHA}(\text{LN}(\mathbf{X})), \quad \mathbf{X}'' = \mathbf{X}' + \text{MLP}(\text{LN}(\mathbf{X}')),$$

where LN normalises each token to zero mean and unit variance across its d features (then applies a learned scale and shift), and the MLP acts on each token separately ($d \rightarrow 4d \rightarrow d$ with a GELU nonlinearity in ViT). Attention *mixes information between tokens*; the MLP *processes each token*; the residual connections keep a running “residual stream” that every block reads from and adds to.

14.5 Derivation, with reasons for each step

Step 1: why scale by $\sqrt{d_k}$. Let $\mathbf{q}$ and $\mathbf{k}$ be one query row and one key row, and suppose their entries are independent with mean 0 and variance 1. Then $\mathbf{q}^\top \mathbf{k} = \sum_{l=1}^{d_k} q_l k_l$ has mean 0 and variance d_k (a sum of d_k independent terms of variance 1). With $d_k = 64$ the scores have standard deviation 8. The softmax then puts almost all its weight on one entry, and the gradient through it (step 3) is nearly zero. Dividing by $\sqrt{d_k}$ restores unit variance at initialisation. The lab measures this: at $d = 1024$ the raw variance is 1048 and the scaled variance is 1.02.

Step 2: stable softmax and masking. $\text{softmax}(\mathbf{s}) = \text{softmax}(\mathbf{s} - c\mathbf{1})$ for any constant c , because $\exp(-c)$ cancels between numerator and denominator. Choosing $c = \max_j s_j$ means $\exp(s_j - c) \leq 1$, so the exponential cannot overflow (log-sum-exp, section 1.5). To forbid token i from attending to token j (padding, or the future in causal models), set $S_{i,j} = -\infty$ before the softmax. Then $A_{i,j} = 0$ exactly, and the remaining weights still sum to one. Zeroing $A_{i,j}$ after the softmax would break that normalisation.

Step 3: the backward pass. Let a scalar loss L have upstream gradient $\nabla_O L$. Treat $\mathbf{O} = \mathbf{A}\mathbf{V}$, $\mathbf{A} = \text{softmax}(\mathbf{S})$ and $\mathbf{S} = \mathbf{Q}\mathbf{K}^\top / \sqrt{d_k}$ as three stages (chapter 11):

- $\mathbf{O} = \mathbf{A}\mathbf{V}$ is a matrix product, so $\nabla_V L = \mathbf{A}^\top \nabla_O L$ and $\nabla_A L = (\nabla_O L) \mathbf{V}^\top$.
- Softmax acts on each row independently. For one row $\mathbf{a} = \text{softmax}(\mathbf{s})$, the Jacobian is $\partial a_j / \partial s_l = a_j(\mathbf{1}_{j=l} - a_l)$. Hence

$$\frac{\partial L}{\partial s_l} = \sum_j \frac{\partial L}{\partial a_j} a_j(\mathbf{1}_{j=l} - a_l) = a_l \left(\frac{\partial L}{\partial a_l} - \sum_j \frac{\partial L}{\partial a_j} a_j \right),$$

or, for all rows at once, $\nabla_S L = \mathbf{A} \odot (\nabla_A L - ((\nabla_A L \odot \mathbf{A}) \mathbf{1}) \mathbf{1}^\top)$, where $\mathbf{1}$ is a vector of ones.

- $\mathbf{S} = \mathbf{Q}\mathbf{K}^\top / \sqrt{d_k}$ gives $\nabla_Q L = (\nabla_S L) \mathbf{K} / \sqrt{d_k}$ and $\nabla_K L = (\nabla_S L)^\top \mathbf{Q} / \sqrt{d_k}$.

Why derive this when autograd exists: the softmax term shows when learning stalls. If one $a_l \approx 1$, then every $\partial L / \partial s_l \approx 0$. This is the saturation problem that step 1 prevents. The lab’s `grad_check` compares these formulas with central differences and reports a maximum relative error of about 10^{-9} .

Step 4: permutation equivariance and positions. Let $\mathbf{P}$ be a permutation matrix. Then $(\mathbf{P}\mathbf{X})\mathbf{W}_Q = \mathbf{P}\mathbf{Q}$, so the scores become $\mathbf{P}\mathbf{S}\mathbf{P}^\top$. Row-wise softmax commutes with reordering rows and columns, so the weights become $\mathbf{P}\mathbf{A}\mathbf{P}^\top$, and the output is $\mathbf{P}\mathbf{A}\mathbf{P}^\top \mathbf{P}\mathbf{V} = \mathbf{P}(\mathbf{A}\mathbf{V})$ because $\mathbf{P}^\top \mathbf{P} = \mathbf{I}$. **Shuffling the input tokens shuffles the outputs in the same way, and nothing else changes.** Self-attention is **permutation-equivariant**; the MLP and LayerNorm act per token, so a whole transformer block is too. For images this means attention alone cannot tell a scrambled set of patches from the original image.

The fix is to add a **position encoding** $\mathbf{E}_{\text{pos}} \in \mathbb{R}^{\tau \times d}$ to the tokens. Three common choices:

- **Sinusoidal** (Vaswani et al., 2017): $E_{i,2k} = \sin(i/10000^{2k/d})$, $E_{i,2k+1} = \cos(i/10000^{2k/d})$. Each pair of coordinates rotates at its own frequency as the position i grows, so the encoding of position $i + \Delta$ is a fixed rotation of that of position i — offsets are linear functions of the encodings.
- **Learned** (ViT): one free vector per position, trained with the rest of the model. Must be interpolated when the number of patches changes at test time.
- **Relative and rotary** encodings inject offsets $i - j$ into the attention scores directly (rotary embeddings rotate $\mathbf{q}$ and $\mathbf{k}$ by position-dependent angles, so $\mathbf{q}_i^\top \mathbf{k}_j$ depends only on $i - j$).

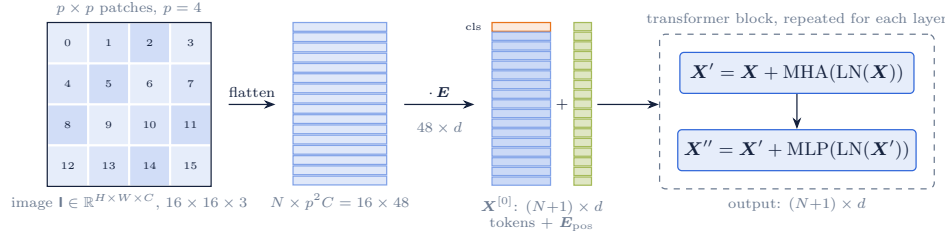
Step 5: from an image to tokens (ViT). Treat the image as a 3-D tensor $\mathbf{I} \in \mathbb{R}^{H \times W \times C}$. Cut it into non-overlapping $p \times p$ patches. That gives $N = HW/p^2$ patches, and each one flattens to a vector of length $p^2 C$. A shared linear map $\mathbf{E} \in \mathbb{R}^{p^2 C \times d}$ embeds each patch:

$$\mathbf{X}^{[0]} = \begin{bmatrix} \mathbf{x}_{\text{cls}}^\top \\ \text{patches}(\mathbf{I}) \mathbf{E} \end{bmatrix} + \mathbf{E}_{\text{pos}} \in \mathbb{R}^{(N+1) \times d},$$

so $\tau = N + 1$. The extra learned **class token** $\mathbf{x}_{\text{cls}}$ collects information through attention; its final state feeds the classification head. (Mean-pooling the patch tokens works about as well.) The patch embedding is the same as a convolution with kernel size p and stride p . That is the only spatial inductive bias a plain ViT has.

Step 6: why LayerNorm and residual connections. Deep stacks of blocks are hard to optimise (chapters 11–12). The **residual connection** gives every block an identity path, so the gradient reaches early layers without passing through every attention and MLP, and a block that is not useful can learn to add almost nothing. **LayerNorm** keeps each token’s scale under control so the softmax scores stay in the range step 1 arranged. Placing the norm *before* each sub-layer (“pre-norm”) keeps the residual stream un-normalised and trains more stably than the original “post-norm” arrangement, especially for deep models (Xiong et al., 2020).

Step 7: counting parameters, compute and memory. Per block, the attention projections $\mathbf{W}_Q, \mathbf{W}_K, \mathbf{W}_V, \mathbf{W}_O$ have $4d^2$ weights and the MLP $2 \cdot d \cdot 4d = 8d^2$: about $12d^2$ per block. ViT-Base ($d = 768$, 12 blocks) has $12 \cdot 12 \cdot 768^2 \approx 85$ million weights in its



Patch embedding = convolution with kernel p and stride p . Without E_{pos} , attention is permutation-equivariant: it sees a bag of patches. Classification reads the final cls token; dense tasks read the N patch tokens.

Figure 14.3: From an image to ViT tokens, with the shapes of the lab’s full run ($16 \times 16 \times 3$ image, $p = 4$, 16 patches), followed by a pre-norm transformer block. Redrawn after the architecture described in Dosovitskiy et al. (2021), figure 1.

blocks — close to its published 86M. Counting compute needs a convention. Ignoring biases, normalisation, the softmax and other small terms, one block costs about $12\tau d^2$ **multiply–accumulates** (MACs) for the projections and MLP (one per weight per token) plus $2\tau^2 d$ for the scores $\mathbf{QK}^\top$ and the weighted sum $\mathbf{AV}$. Counting a multiplication and an addition as two **FLOPs**, the same block costs $24\tau d^2 + 4\tau^2 d$ FLOPs. This book uses MACs unless it says FLOPs. Either way, the cost is **linear in τ for the dense layers and quadratic for attention**. Memory for the attention matrices is $h\tau^2$ numbers per block per image. At 224×224 with $p = 16$, $\tau = 197$; with $p = 8$, $\tau = 785$ — sixteen times more attention memory (section 14.8).

Step 8: inductive biases, CNN versus ViT.

Property	CNN	ViT
locality	built in (small kernels)	learned; one layer is global
translation equivariance	built in (shared weights)	only through the patch grid; positions must be learned
receptive field	grows layer by layer	global from layer 1
weights depend on content	no (fixed filters)	yes (attention weights)
data needed from scratch	modest	large, or strong augmentation / pretraining
cost in image size	linear in pixels	quadratic in tokens

Neither column is “better”. Built-in assumptions save data when they hold; learned ones win when there is enough data to learn them — the same trade-off as approximation versus estimation error (section 5.5).

14.6 Numerical worked example

One query and two keys with $d_k = 2$: $\mathbf{q} = (1, 0)^\top$, key rows $\mathbf{K}_{1,:} = (2, 0)$ and $\mathbf{K}_{2,:} = (0, 2)$, value rows $\mathbf{V}_{1,:} = (1, 0)$ and $\mathbf{V}_{2,:} = (0, 1)$.

- Scores: $\mathbf{q}^\top \mathbf{K}_{1,:}^\top / \sqrt{2} = 2/1.414 = 1.414$ and $\mathbf{q}^\top \mathbf{K}_{2,:}^\top / \sqrt{2} = 0$.
- Softmax: $\exp(1.414) = 4.11$, so $\mathbf{a} = (4.11, 1)/5.11 = (0.805, 0.195)$.
- Output: $0.805 \mathbf{V}_{1,:} + 0.195 \mathbf{V}_{2,:} = (0.805, 0.195)$.

Double $\mathbf{q}$ to $(2, 0)^\top$. The scores become $(2.83, 0)$ and the weights $(0.944, 0.056)$: attention gets *sharper*. This is the same mechanism that makes unscaled scores at large d_k saturate.

The softmax gradient by hand. For $\mathbf{a} = (0.805, 0.195)$ and upstream $\partial L / \partial \mathbf{a} = (1, 0)$: $\sum_j (\partial L / \partial a_j) a_j = 0.805$, so $\partial L / \partial \mathbf{s} = (0.805 \cdot (1 - 0.805), 0.195 \cdot (0 - 0.805)) = (0.157, -0.157)$. The entries sum to zero, as they must: adding a constant to all scores changes nothing.

Shape check for the lab’s full run: a $16 \times 16 \times 3$ image with $p = 4$ gives $N = 16$ patches of length 48. They embed to $\mathbf{X} \in \mathbb{R}^{16 \times 32}$. With $h = 4$ heads, each head works in $d_k = 8$, the weights form a $4 \times 16 \times 16$ tensor, and the output has shape 16×32 .

14.7 Small implementation and checks

The lab is `labs/ch14-attention/attention.py`: NumPy only, with no framework. The checks in `test_attention.py` are:

Check	What it guards against
rows of $\mathbf{A}$ sum to 1, softmax finite at scores of 1000	overflow; normalising the wrong axis
zero queries and keys give the mean of the value rows	wrong contraction order
permuting tokens permutes outputs	accidental position dependence
with sinusoidal positions, equivariance fails	position encodings not applied
causal mask leaves the upper triangle of $\mathbf{A}$ at 0	masking after the softmax
one head equals plain attention with projections	head split/merge reshape bugs
analytic vs. finite-difference gradients $< 10^{-6}$	wrong softmax Jacobian
large score scales saturate the softmax and shrink gradients	missing $1/\sqrt{d_k}$
LayerNorm gives zero-mean, unit-variance rows	normalising the wrong axis
patch order is row-major; token and memory counts	transposed patches; wrong cost estimates

The head split `reshape(tau, h, d_h).transpose(1, 0, 2)` is the most common source of silent bugs. A reshape with the wrong axis order still runs and still gives the right shapes, but it mixes features between heads. The one-head test does not catch this. Exercise 4 asks you to write a test that does.

14.8 Experiment and expected observations

`python labs/ch14-attention/attention.py` (seed 0, CPU, under 1 s) prints the shapes above. It confirms equivariance and masking, confirms that adding sinusoidal positions breaks equivariance, prints a gradient-check error of about 9×10^{-10} , and measures:

Score variance.

d	4	64	1024
$\text{Var}(\mathbf{q}^\top \mathbf{k})$	4.2	63.8	1048.2
$\text{Var}(\mathbf{q}^\top \mathbf{k} / \sqrt{d})$	1.05	1.00	1.02

Saturation. Multiply all queries by c ($\tau = 16$, so the maximum row entropy is $\log 16 = 2.77$):

scale c	0.1	1	10	100
mean	2.767	2.214	0.183	0.000
attention-row entropy				
$\ \nabla L\ $ w.r.t. the scaled queries	3.20	4.07	2.12	4.8×10^{-7}

At $c = 0.1$ attention is almost uniform (entropy at its maximum): every token averages everything. At $c = 100$ each row is one-hot and the gradient has vanished — the model can no longer learn *where* to attend. Step 1’s scaling keeps training in the middle of this table.

Memory. Storing every attention matrix of a ViT-Base forward pass on 224×224 images, batch 64, float32:

patch size	32	16	8
tokens τ	50	197	785
attention memory	0.1 GB	1.4 GB	22.7 GB

Halving the patch size quadruples the tokens and multiplies attention memory by about sixteen. This is why high-resolution dense prediction uses windowed attention, hierarchical designs, or memory-efficient kernels.

Extension experiment (optional single GPU). Train a tiny ViT (4 blocks, $d = 64$, patch size 4) on CIFAR-10 at 32×32 . Compare (a) learned position embeddings, (b) no position embeddings, and (c) a small CNN with a similar number of parameters. Expect (b) to lose accuracy, because it sees a bag of patches, and expect the CNN to win at this data scale. Run three seeds per setting and report the spread. Record the CIFAR-10 version and download source as data provenance. This is a toy study of an idea; it does not reproduce the ViT paper’s reported results.

14.9 Moving pieces: diagnosing and fixing

Moving piece	Symptom when it is wrong	How to confirm	Fix
Score scale ($1/\sqrt{d_k}$, temperature)	loss stalls early; attention maps one-hot from the start	attention entropy at initialisation	include $1/\sqrt{d_k}$; check initialisation scale

Moving piece	Symptom when it is wrong	How to confirm	Fix
Softmax axis	weights do not sum to 1 per query	<code>A.sum(-1)</code>	normalise over keys (last axis)
Masking	causal model “sees the future”; NaN rows	test a fully masked row; check order of mask and softmax	mask scores with $-\infty$ (or dtype minimum) before softmax; handle all-masked rows
Head split/merge	trains but underperforms; features mixed across heads	one-head vs. multi-head equivalence; exercise 4 test	reshape (<code>tau</code> , <code>h</code> , <code>d_h</code>) then transpose
Position encodings	bag-of-patches behaviour; failure at new resolution	shuffle patches at test time: does accuracy drop?	add positions; interpolate learned ones for new sizes
Normalisation and residuals	divergence or very slow training in deep stacks	gradient norms per layer	pre-norm blocks; residual connections; warm-up
Data scale	ViT underperforms a CNN of the same size	learning curves against data size	pretrained weights; strong augmentation; distillation; hybrid conv stem
Token count	out-of-memory at high resolution	$h\tau^2$ per block	larger patches; windowed attention; memory-efficient kernels
Artifacts	noisy attention maps; high-norm background tokens	histogram of output token norms	register tokens (section 14.11)
Reading attention maps	confident explanations that do not hold up	ablate the “attended” region	treat attention as a computation, not an explanation

First things to try when a transformer misbehaves: (1) check shapes and the softmax axis with a tiny input; (2) gradient-check one attention layer; (3) plot attention entropy per layer at initialisation and after a few steps; (4) shuffle input patches and see whether the model notices.

14.10 Application, limitations and failure cases

Application (educational case study). In remote-sensing segmentation, long and thin structures (roads, rivers, field boundaries) motivate long-range attention. Evaluate on geographically held-out tiles (chapter 5). A model that has memorised one region’s textures can look strong on a random split.

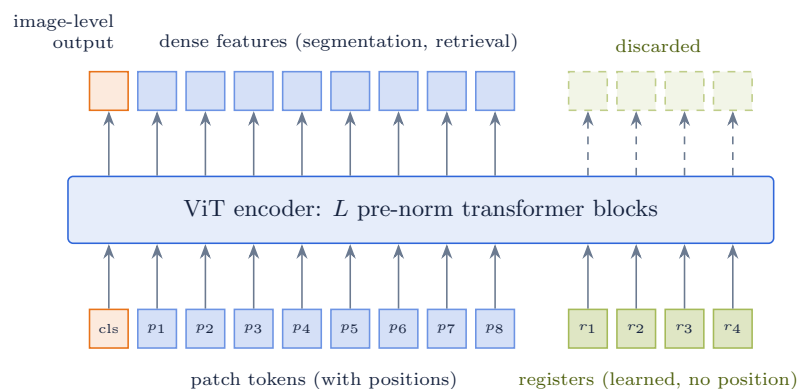
Failure cases:

- **Quadratic cost.** $\mathbf{A}$ is $\tau \times \tau$; memory, not FLOPs, usually runs out first.

- **Data hunger.** Without convolution’s locality bias, plain ViTs trained from scratch on small datasets underperform CNNs.
- **Attention maps are not explanations.** High attention weight does not mean causal importance, because values can be small and later layers mix everything.
- **Artifacts.** Large ViTs can produce a few high-norm tokens in background regions that act as global scratch space.
- **Resolution changes.** Learned position embeddings must be interpolated when the number of patches changes at test time, and accuracy can drop.

14.11 Bridge to a research paper

Vision Transformers Need Registers (Darcet, Oquab, Mairal and Bojanowski, ICLR 2024; [arXiv:2309.16588](https://arxiv.org/abs/2309.16588), verified 4 October 2026).



Without registers, large ViTs repurpose a few low-information background patch tokens as global scratch space; those tokens get very high norms and distort attention maps and dense features. Extra learnable tokens give the model that scratch space explicitly, and are thrown away at the output.

Figure 14.4: Register tokens, redrawn after the description and figure 1 of Darcet et al. (2024). Extra learnable tokens are appended to the input sequence and discarded at the output.

- **Inputs:** images; supervised and self-supervised ViTs (including DINOv2, chapter 16).
- **Outputs:** feature maps and attention maps.
- **Learned components:** the ViT, plus a few extra learnable “register” tokens that are discarded at the output.
- **Observed phenomenon:** high-norm outlier tokens appear in low-information background patches of large, long-trained models; they carry global information and their local information is lost.
- **Central claim:** adding register tokens removes these artifacts and gives smoother feature and attention maps, with better dense-prediction behaviour.
- **Assumptions to question:** do the artifacts appear in small models or short training runs? How much depends on the threshold used to call a token “high-norm”?
- **Affordable experiment:** load public pretrained checkpoints with and without registers. On 100 images from a public dataset, plot the distribution of output token norms, count tokens above a stated threshold, and compare attention maps. Record checkpoint names, image IDs and the threshold. This tests the observed phenomenon at small scale; it does not reproduce the paper’s benchmark numbers.

14.12 Memory map

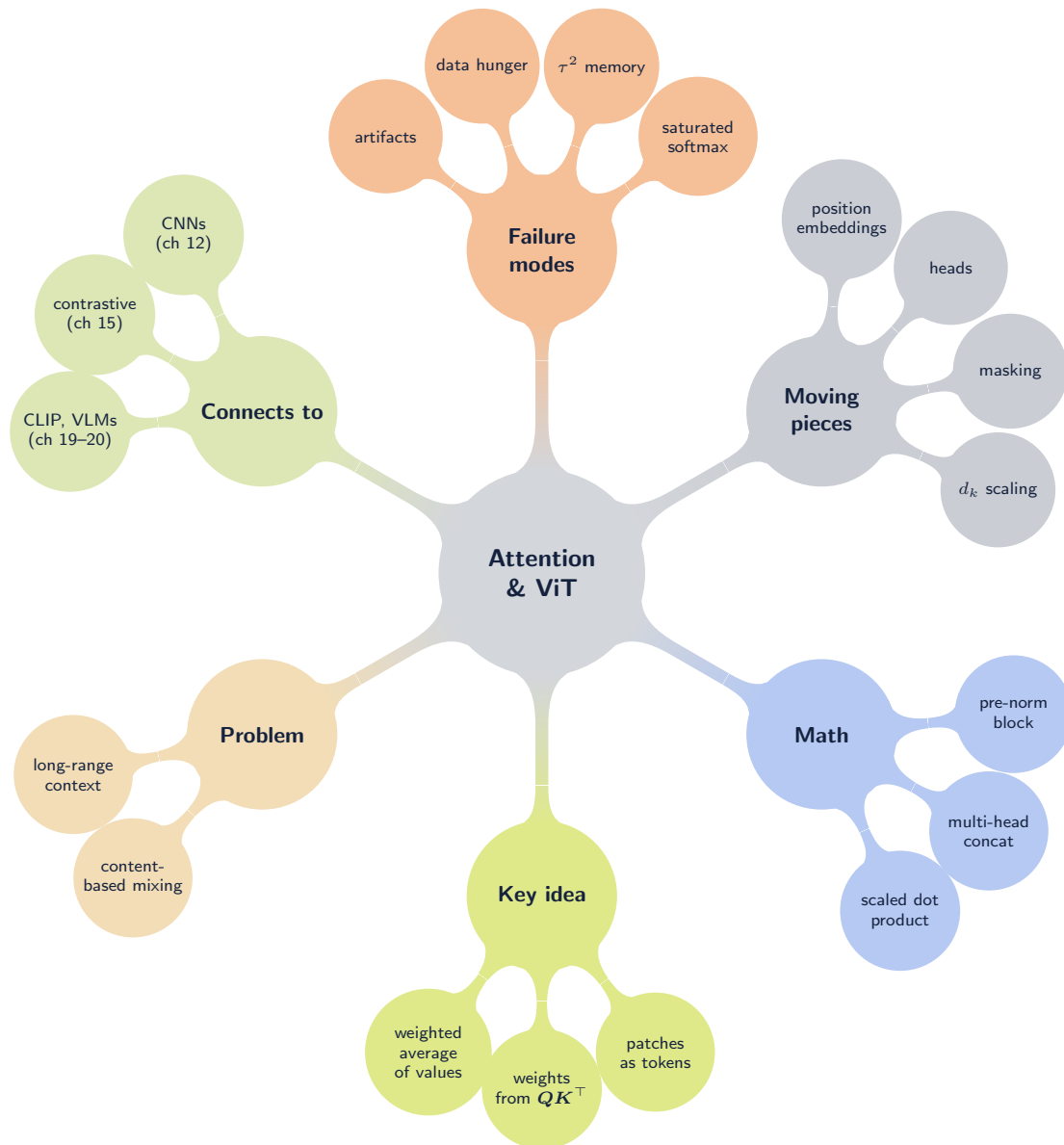


Figure 14.5: Memory map of chapter 14.

Summary. Attention replaces fixed, local mixing with a content-dependent weighted average over all tokens: scores from query–key dot products, scaled by $1/\sqrt{d_k}$, normalised by a stable, masked softmax, applied to values. Several heads attend to different things at once. Self-attention is permutation-equivariant, so positions must be added. A transformer block alternates attention (mixing between tokens) with an MLP (processing within tokens), around a residual stream kept in range by LayerNorm. A ViT turns patches into tokens and is otherwise the same block repeated; it trades a CNN’s built-in locality for flexibility, at a cost in data and in quadratic memory.

14.13 Exercises

1. *Conceptual.* Why can't attention weights be negative? Name one thing a convolution can compute in one layer that one attention layer, as defined here, cannot.
2. *Derivation.* Derive the softmax Jacobian $a_j(\mathbf{1}_{j=l} - a_l)$ and show that each row of $\nabla_{\mathbf{S}} L$ sums to zero. Why must it?
3. *Derivation.* Show that a causal mask makes output row i depend only on tokens $1, \dots, i$, and that the backward pass respects the same dependency.
4. *Implementation.* Write a test that fails if the head split uses `reshape(h, tau, d_h)` instead of `reshape(tau, h, d_h).transpose(1, 0, 2)`.
5. *Implementation.* Add learned position embeddings and verify that permutation equivariance fails; then implement bilinear interpolation of the embedding grid for a larger image.
6. *Derivation.* Show that for sinusoidal encodings, the pair $(E_{i+\Delta, 2k}, E_{i+\Delta, 2k+1})$ is a rotation of $(E_{i, 2k}, E_{i, 2k+1})$ by an angle depending only on Δ and k .
7. *Interpretation.* Using `attention_backward`, reproduce the saturation table of section 14.8 for $\tau \in \{16, 256\}$. How does the uniform-attention entropy change, and why?
8. *Failure analysis.* For a 224×224 image, compute the memory of all attention matrices in a 12-layer, 12-head ViT at $p \in \{32, 16, 8\}$ in float32. Which is infeasible on a 16 GB GPU at batch size 64?
9. *Open question.* Registers give the model scratch space. Propose a different intervention for the same artifact, and a measurement that would falsify it.

14.13.1 Answer discussions

1. They are softmax outputs. A convolution can compute a *difference* of neighbouring pixels (an edge filter with negative weights) in one layer; one attention layer produces convex combinations of values (though the value projection itself can have negative weights).
2. Differentiate $a_j = e^{s_j} / \sum e^s$. Row sums vanish because $\sum_l a_l (g_l - \sum_j g_j a_j) = 0$; shifting all scores leaves the output unchanged, so the gradient must be orthogonal to $\mathbf{1}$.
3. Masked weights are exactly zero, so $\mathbf{O}_{i,:}$ is a combination of $\mathbf{V}_{j,:}$ with $j \leq i$; the backward pass multiplies by the same zero weights.
4. Use $h = 2$ heads and weights that make head 1 attend uniformly and head 2 to one token; compare against an explicit per-head loop.
5. Equivariance fails as in the lab's sinusoidal test; interpolate the $\sqrt{N} \times \sqrt{N}$ grid of embeddings to the new grid size.
6. With $\omega_k = 10000^{-2k/d}$, the pair is $(\sin \omega_k i, \cos \omega_k i)$; adding Δ multiplies by the rotation matrix of angle $\omega_k \Delta$.
7. The maximum entropy is $\log \tau$, so uniform attention over 256 tokens has entropy 5.55; saturation needs larger scales when there are more keys competing.
8. 0.1, 1.4 and 22.7 GB (section 14.8): patch size 8 does not fit alongside the other activations.
9. For example, penalise high token norms, or add global pooling tokens only in later layers; falsify by showing artifacts persist (norm histogram unchanged) or dense metrics do not improve.

14.14 Self-check and common mistakes

- I can give the shapes of $\mathbf{Q}, \mathbf{K}, \mathbf{V}, \mathbf{S}, \mathbf{A}, \mathbf{O}$ for both self- and cross-attention.

- ☐ I know which axis the softmax normalises and why masking comes first.
- ☐ I can explain why ViT needs position embeddings and how sinusoidal ones encode offsets.
- ☐ I can count a transformer block's parameters and say which cost is quadratic in τ .
- ☐ I can list three inductive biases a CNN has and a ViT lacks.

Common mistakes: softmax over the wrong axis (columns instead of rows); masking with a huge constant such as -10^9 in float16, which overflows to $-\infty$ and turns a fully masked row into NaN (appendix C.3); forgetting $1/\sqrt{d_k}$; mixing heads by reshaping without transposing; forgetting to interpolate position embeddings at a new resolution; reading attention maps as feature importance.

References

- D. Bahdanau, K. Cho and Y. Bengio. Neural machine translation by jointly learning to align and translate. *ICLR*, 2015. [arXiv:1409.0473](#).
- K. Xu et al. Show, attend and tell: neural image caption generation with visual attention. *ICML*, 2015.
- A. Vaswani et al. Attention is all you need. *NeurIPS*, 2017. [arXiv:1706.03762](#).
- J. L. Ba, J. R. Kiros and G. E. Hinton. Layer normalization. 2016. [arXiv:1607.06450](#).
- X. Wang, R. Girshick, A. Gupta and K. He. Non-local neural networks. *CVPR*, 2018.
- R. Xiong et al. On layer normalization in the transformer architecture. *ICML*, 2020.
- A. Dosovitskiy et al. An image is worth 16x16 words: transformers for image recognition at scale. *ICLR*, 2021. [arXiv:2010.11929](#).
- H. Touvron et al. Training data-efficient image transformers and distillation through attention. *ICML*, 2021.
- Z. Liu et al. Swin transformer: hierarchical vision transformer using shifted windows. *ICCV*, 2021.
- T. Dao, D. Y. Fu, S. Ermon, A. Rudra and C. Ré. FlashAttention: fast and memory-efficient exact attention with IO-awareness. *NeurIPS*, 2022.
- J. Su et al. RoFormer: enhanced transformer with rotary position embedding. 2021. [arXiv:2104.09864](#).
- T. Darcet, M. Oquab, J. Mairal, P. Bojanowski. Vision transformers need registers. *ICLR*, 2024. [arXiv:2309.16588](#).
- I. Goodfellow, Y. Bengio and A. Courville. *Deep Learning*. MIT Press, 2016, section 6.5 (back-propagation) and the notation page.

Part IV

Visual representation learning

Chapter 15

Contrastive learning

Prerequisites: chapter 9 (KL, bounds on information); chapter 14 (encoders); chapter 11 (training).

Learning outcomes: after this chapter you can

1. write the InfoNCE (NT-Xent) loss as a softmax classification over candidates and derive its gradient, including through the L2 normalisation;
2. explain what temperature, batch size, negatives and the projection head each change;
3. state the bound $I \geq \log N - \mathcal{L}$, why it is capped at $\log N$, and why maximising mutual information is not the whole story;
4. read the loss as alignment plus uniformity, and explain how negatives prevent collapse;
5. design augmentations as statements about which features should be kept, and evaluate a representation with frozen probes rather than with its training loss.

15.1 A concrete problem

The crop-health team has 200,000 unlabelled field photographs and labels for only 2,000. The photographs vary in disease, but even more in things that do not matter: lighting, camera, background soil, time of day. A network trained on 2,000 labels overfits; features from an ImageNet model may not capture lesion textures. Can the unlabelled images teach an encoder which differences matter?

Contrastive learning turns the unlabelled set into a supervised task the network can solve: take two randomly augmented **views** of the same photograph and ask the encoder to recognise them as a pair among many views of other photographs. To succeed, the encoder must keep whatever survives the augmentations and distinguishes one photograph from another, and it may discard whatever the augmentations change. The augmentations are therefore the real specification: they say which differences are nuisance.

This chapter derives the loss, its gradient and its information-theoretic reading; shows with a lab what augmentations, temperature, batch size and negatives each do; and ends at SimCLR, the paper that made the recipe simple. Chapter 16 removes the negatives; chapter 19 applies the same loss to image-text pairs (CLIP).

15.2 How we got here

Agreement between views (1992–2006). Becker and Hinton (1992) trained two networks seeing adjacent patches of a stereogram to agree, maximising the mutual in-

formation of their outputs. Bromley and colleagues (1993) used “Siamese” networks to verify signatures. Hadsell, Chopra and LeCun (2006) introduced a contrastive loss that pulls similar pairs together and pushes dissimilar ones beyond a margin — and noted that without the push, all outputs collapse to a point.

Noise contrast (2010–2013). Gutmann and Hyvärinen (2010) proposed **noise-contrastive estimation**: learn a density by classifying data against noise samples. Word2vec’s negative sampling (Mikolov and colleagues, 2013) used the same idea to learn word vectors from co-occurrence.

Instances as classes (2014–2019). Dosovitskiy and colleagues (2014) treated each image (with its augmentations) as its own class. Wu and colleagues (2018) scaled “instance discrimination” with a memory bank of features. Van den Oord, Li and Vinyals (2018) introduced **InfoNCE** in Contrastive Predictive Coding and connected it to a lower bound on mutual information; Hjelm and colleagues (2019) and Tian and colleagues (2019) followed with information-maximising objectives.

The 2020 recipes. MoCo (He and colleagues, 2020) kept a queue of negatives from a slowly updated momentum encoder; **SimCLR** (Chen, Kornblith, Norouzi and Hinton, 2020) showed that strong augmentations, a projection head, a temperature-scaled loss and large batches suffice — this chapter’s paper bridge. Linear probes on self-supervised features approached supervised ImageNet accuracy. Wang and Isola (2020) explained the loss as **alignment** plus **uniformity**; Tschannen and colleagues (2020) showed that mutual information alone does not explain success; Khosla and colleagues (2020) used labels to remove false negatives (supervised contrastive learning).

After negatives (2020–2023). BYOL and SimSiam (2020) learned without negatives, DINO (2021) by self-distillation (chapter 16), and CLIP (2021) applied InfoNCE across modalities to 400 million image–text pairs (chapter 19). Contrastive losses remain central in retrieval, multimodal pretraining and re-identification.

15.3 Assumptions and notation

Symbol	Meaning
$\mathbf{x}$	an unlabelled image
$t \sim \mathcal{T}$	a random augmentation; $\tilde{\mathbf{x}} = t(\mathbf{x})$ a view
$f, \mathbf{h} = f(\tilde{\mathbf{x}})$	encoder (backbone) and its representation, used downstream
$g, \mathbf{z} = g(\mathbf{h})$	projection head and its output, used only by the loss
$\bar{\mathbf{z}} = \mathbf{z} / \ \mathbf{z}\ $	the L2-normalised projection
$s_{ik} = \bar{\mathbf{z}}_i^\top \bar{\mathbf{z}}_k$	cosine similarity
τ	temperature
B, N	images per batch ($2B$ views); number of candidates per anchor ($N = 2B - 1$ in SimCLR)
p_{ik}	softmax probability that candidate k is anchor i ’s positive
$I(\mathbf{x}; \mathbf{y})$	mutual information

Assumptions:

1. **Augmentations preserve what matters.** Two views of one image share the content we care about and differ in nuisance. If an augmentation changes the content (a crop that removes the lesion), the positive pair is wrong.
2. **Other images are mostly different.** Negatives are assumed to be different “things”; with few classes and large batches many negatives share the anchor’s class (false negatives).
3. **Evaluation is downstream.** The loss value is not the goal; frozen probes on labelled data measure the representation.

15.4 Mathematical formulation

For a batch of B images, make two views each ($2B$ views), compute normalised projections $\bar{z}_1, \dots, \bar{z}_{2B}$, and let $j(i)$ be the index of i ’s other view. The **NT-Xent** (normalised temperature-scaled cross-entropy, i.e. InfoNCE) loss is

$$\mathcal{L} = \frac{1}{2B} \sum_{i=1}^{2B} \ell_i, \quad \ell_i = -\log \frac{\exp(s_{i,j(i)}/\tau)}{\sum_{k \neq i} \exp(s_{ik}/\tau)}.$$

The anchor’s similarity with itself is excluded; the positive stays in the denominator. In MoCo the candidates are one positive key and K keys from a queue produced by a momentum encoder with parameters $\xi \leftarrow m\xi + (1-m)\theta$.

Evaluation: freeze f , compute h for labelled images, and report a linear-probe and/or k -nearest-neighbour accuracy, stating which layer and pooling were used.

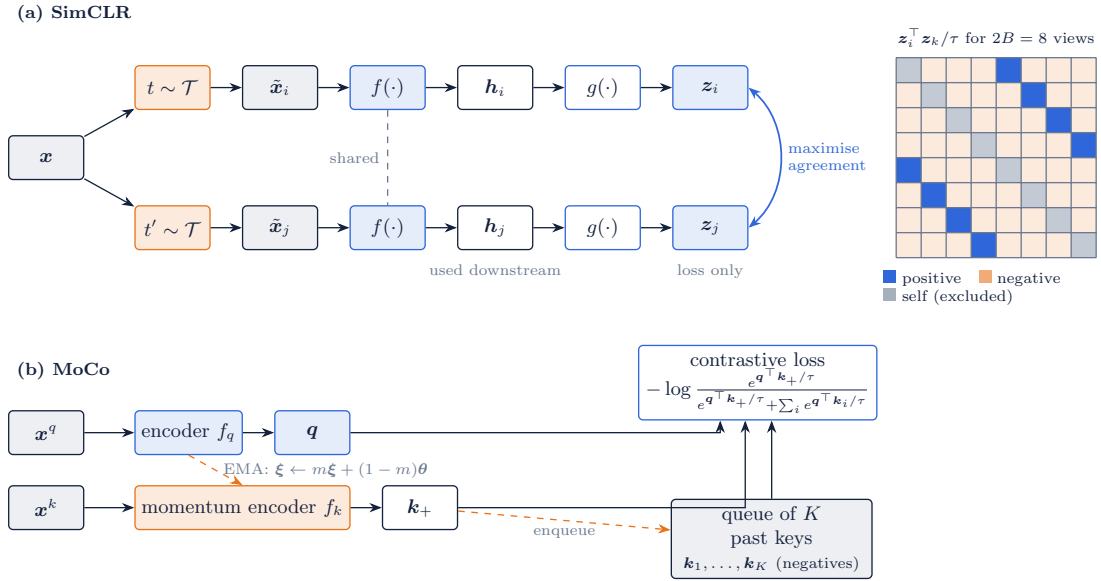


Figure 15.1: (a) SimCLR, redrawn after Fig. 2 of Chen et al. (2020): two augmentations per image, a shared encoder and projection head, and a loss that identifies each view’s partner among the other $2B - 1$ views; right, the similarity matrix of a batch with positives, negatives and excluded self-pairs. (b) MoCo, redrawn after Fig. 1 of He et al. (2020): keys from a momentum encoder stored in a queue. Papers: [arXiv:2002.05709](https://arxiv.org/abs/2002.05709), [arXiv:1911.05722](https://arxiv.org/abs/1911.05722).

15.5 Derivation, with reasons for each step

Step 1 — InfoNCE is a softmax classifier over candidates. Treat the similarities s_{ik}/τ as logits and the positive as the correct class: ℓ_i is exactly cross-entropy (chapter 11, step 4). With $p_{ik} = \exp(s_{ik}/\tau) / \sum_{r \neq i} \exp(s_{ir}/\tau)$,

$$\frac{\partial \ell_i}{\partial s_{ik}} = \frac{1}{\tau} (p_{ik} - \mathbb{I}[k = j(i)]).$$

Reading: the positive is pulled with force $(1 - p_{i,j(i)})/\tau$, and each negative is pushed with force p_{ik}/τ — proportional to how confusable it currently is. If all N candidates are equally likely, the loss is $\log N$; a perfectly collapsed encoder (all $\bar{z}$ equal) has exactly this loss, so the objective is never minimised by collapse when negatives are present.

Step 2 — through the normalisation. $\bar{z} = z/\|z\|$ has Jacobian $(\mathbf{I} - \bar{z}\bar{z}^\top)/\|z\|$, so

$$\nabla_z \mathcal{L} = \frac{1}{\|z\|} (\mathbf{I} - \bar{z}\bar{z}^\top) \nabla_{\bar{z}} \mathcal{L}.$$

Reason: differentiate $z/\|z\|$ by the quotient rule. The gradient is tangent to the sphere — it rotates z , never lengthens it — and shrinks as $\|z\|$ grows. Without normalisation the encoder could reduce the loss by inflating norms, which acts like lowering the temperature without learning anything. With $\mathbf{S} = \bar{\mathbf{Z}}\bar{\mathbf{Z}}^\top$ and $\mathbf{G} = \partial \mathcal{L} / \partial \mathbf{S}$ (zero diagonal), the gradient for all rows at once is $(\mathbf{G} + \mathbf{G}^\top)\bar{\mathbf{Z}}$; the lab uses exactly these two lines.

Step 3 — temperature sets how hard the negatives are. The push on negative k is p_{ik}/τ . As $\tau \rightarrow 0$, p_{ik} concentrates on the most similar negatives: the loss becomes a hardest-negative margin loss, sensitive to false negatives and label noise. As τ grows, all negatives are pushed almost equally, and the embedding is spread uniformly with little attention to local structure. Typical values are 0.05–0.5; the lab finds a broad optimum around 0.2–0.5 (figure 15.4a). Loss values at different temperatures are not comparable: the worked example’s loss falls from 0.551 to 0.240 when τ halves, with no change at all in the embedding.

Step 4 — the mutual-information bound and its cap. Let (x, y) be a positive pair and $y_2, \dots, y_N$ independent negatives. For any critic $f(x, y)$, the expected InfoNCE loss $\mathcal{L}_N = -\mathbb{E}[\log \frac{e^{f(x, y_1)}}{\sum_k e^{f(x, y_k)}}]$ satisfies

$$I(x; y) \geq \log N - \mathcal{L}_N.$$

Sketch: the best possible critic is $f^* = \log p(y | x) / p(y) + \text{const}$; for it, $\mathcal{L}_N$ is the cross-entropy of guessing which of N candidates was drawn from $p(y | x)$ rather than $p(y)$, and a Jensen step on the denominator gives $\mathcal{L}_N \geq \log N - I$ (van den Oord and colleagues, 2018; Poole and colleagues, 2019). Two consequences: the estimate can never exceed $\log N$, because $\mathcal{L}_N \geq 0$; and with N small relative to e^I the bound is loose. In the lab, two Gaussians with $I = 3.11$ nats give estimates of 0.67, 1.81, 2.61, 2.95 and 3.09 nats at $N = 2, 8, 32, 128, 512$ (figure 15.2). This is one reason more negatives help — but only up to a point, and representations that maximise this bound are not necessarily the most useful: an invertible scrambling of the features keeps mutual information and destroys linear readability (Tschannen and colleagues, 2020).

Step 5 — alignment and uniformity. Expand ℓ_i with $s_{i,j(i)}$ the positive similarity:

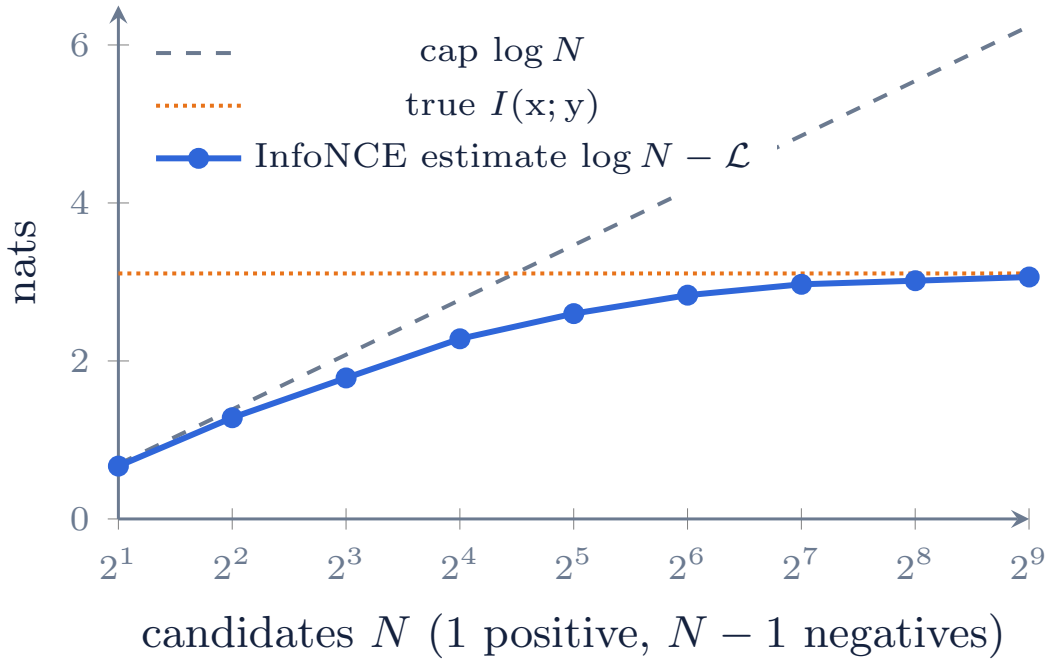


Figure 15.2: The InfoNCE estimate of mutual information with the optimal critic, for two correlated Gaussians with $I = 3.11$ nats. The estimate is capped by $\log N$ and needs many candidates to approach the truth.

$$\ell_i = -\frac{s_{i,j(i)}}{\tau} + \log \sum_{k \neq i} e^{s_{ik}/\tau}.$$

The first term rewards **alignment** — positives close together. The second, averaged over anchors, is minimised when similarities to other points are uniformly low — **uniformity** on the sphere. Wang and Isola (2020) showed that as $N \rightarrow \infty$ the loss converges to an alignment term plus a uniformity term $\log \mathbb{E} e^{-t\|\bar{z}-\bar{z}'\|^2}$, and that both predict downstream accuracy. Alignment alone is minimised by collapse; uniformity alone ignores the positives. The lab’s “positives only” run drives the alignment loss to 5×10^{-4} while every projection becomes the same vector (mean cosine 1.000) and k -NN accuracy falls below a random encoder’s.

Step 6 — augmentations choose the invariances. Suppose an image is generated from content $\mathbf{c}$ and nuisance $\mathbf{s}$, and augmentations resample $\mathbf{s}$ while keeping $\mathbf{c}$. Then the only information shared by the two views is $\mathbf{c}$, and an encoder that aligns views must, at the optimum, depend on $\mathbf{c}$ and not on $\mathbf{s}$ — under suitable conditions the content block is even identifiable (von K  gelgen and colleagues, 2021). The lab tests three policies on the same data:

- **resample the style** (what we want ignored): k -NN accuracy 0.845, nuisance almost gone from the projection ($R^2 = 0.24$);
- **noise only** (style shared by both views): style is the easiest way to tell images apart, so the encoder keeps it ($R^2 = 0.77$) and k -NN accuracy (0.395) falls *below* a random encoder’s (0.495);
- **destroy the content too** (an augmentation too strong): nothing reliable is shared, the loss barely moves ($5.55 \rightarrow 5.54$) and features stay near random (0.480).

The lesson is general: if colour distinguishes the disease, strong colour jitter removes the signal; if the task is counting lesions, crops that change the count are not label-preserving. There is no problem-independent augmentation recipe.

Step 7 — why a projection head, and which layer to evaluate. The loss makes $\bar{z}$ invariant to the augmentations, discarding information. Putting a small MLP g between the backbone and the loss lets z discard it while h keeps more. In the lab, nuisance R^2 is 0.77 in h and 0.24 in z under the style policy. SimCLR found linear probes on h better than on z by more than ten points; the information the head throws away is sometimes needed downstream (here, the “nuisance” would matter for a task about lighting).

Step 8 — batch size, negatives and false negatives. In SimCLR, the number of candidates grows with the batch ($N = 2B - 1$), which tightens the bound of step 4 and gives more hard negatives; the lab’s k -NN accuracy rises from 0.775 at $B = 8$ to 0.845 at $B = 128$ and then saturates. MoCo decouples N from B with a queue of keys from a momentum encoder: keys must come from a *slowly changing* encoder, or old and new keys are inconsistent. Large N has a cost: with C classes, a fraction about $1/C$ of negatives share the anchor’s class and are pushed away wrongly (false negatives). With labels, supervised contrastive learning treats all same-class samples as positives; without labels, debiased losses correct the expected false-negative mass.

Step 9 — a representation is judged by probes. The pretraining loss reflects the constructed task, not the downstream one. Freeze the encoder and report: linear-probe accuracy (how linearly accessible the information is), k -NN accuracy (how the metric geometry clusters classes), low-shot and fine-tuning results, and transfer to other datasets (chapter 18). A baseline with random features and one with supervised features belong in every table.

15.6 Numerical worked example

Loss and temperature. One positive with similarity 1, two negatives with similarity 0. At $\tau = 1$: $p_{\text{pos}} = e/(e + 2) = 0.576$, loss = 0.551. At $\tau = 0.5$: $p_{\text{pos}} = e^2/(e^2 + 2) = 0.787$, loss = 0.240. The embedding is identical; only the temperature changed.

Gradient. At $\tau = 1$, $\partial\ell/\partial s_{\text{pos}} = 0.576 - 1 = -0.424$ and each negative has $\partial\ell/\partial s_{\text{neg}} = (1 - 0.576)/2 = 0.212$: descent raises the positive similarity and lowers both negatives equally.

Uniform candidates and collapse. With four equally similar candidates, $\ell = \log 4 = 1.386$. A collapsed encoder in a SimCLR batch of $B = 128$ has $\ell = \log 255 = 5.54$ — the value the lab’s “destroy” run stays near.

Normalisation. $z = (3, 4)$, $\bar{z} = (0.6, 0.8)$, and a gradient $(1, 0)$ with respect to $\bar{z}$ becomes $\frac{1}{5}((1, 0) - 0.6(0.6, 0.8)) = (0.128, -0.096)$ with respect to z — perpendicular to z .

15.7 Small implementation and checks

The lab `labs/ch15-contrastive/contrastive.py` trains a two-layer MLP encoder with a linear projection head; the whole loss and its gradient are nine lines:

```
def info_nce(Z, tau):
    # Z: 2B x d, rows
    # L2-normalised; i and i+B are positives
    S = Z @ Z.T / tau; np.fill_diagonal(S, -np.inf)
    P = softmax_rows(S)
```

```

pos = np.r_[np.arange(B, 2 * B), np.arange(B)]
loss = -np.mean(np.log(P[np.arange(2 * B), pos]))
G = P.copy(); G[np.arange(2 * B), pos] -= 1; G /= 2 * B * tau
return loss, (G + G.T) @ Z          # then project onto the
    ↪ sphere's tangent plane

```

The synthetic world renders a content vector (20 classes) and a larger nuisance vector through a fixed nonlinear map into 32-dimensional “images”. Augmentations resample the nuisance, add noise and drop 20% of input features.

Checks, in `test_contrastive.py`: the worked example; identical embeddings give $\log(2B - 1)$; a gradient check through normalisation and encoder (4×10^{-8}); InfoNCE estimates never exceed $\log N$; alignment and uniformity extremes; good augmentations beat weak ones and remove nuisance from the projection; and removing negatives collapses the projection.

15.8 Experiment and expected observations

Run `python labs/ch15-contrastive/contrastive.py` (seed 0; 1,000 steps of Adam, batch 128, $\tau = 0.2$ unless varied; evaluation on 1,000 + 1,000 fresh images).

encoder	k -NN acc.	linear probe	nuisance R^2 in $\mathbf{h} / \mathbf{z}$	mean cosine between images
raw input	0.586	0.852	—	—
random encoder	0.495	0.762	—	—
contrastive, style	0.845	0.868	0.77 / 0.24	0.012
augmentation				
contrastive,	0.395	0.729	0.83 / 0.77	0.005
noise-only				
augmentation				
contrastive,	0.480	0.780	0.84 / 0.53	0.980
content- destroying				
augmentation				
positives only (no negatives)	0.398	—	—	1.000

temperature					
τ	0.05	0.1	0.2	0.5	1.0
k -NN	0.828	0.832	0.845	0.844	0.829
accuracy					
alignment /	0.105 /	0.194 /	0.224 /	0.160 /	0.134 /
uniformity	−1.54	−2.45	−2.82	−2.55	−2.30

batch size B	8	32	128	512
k -NN accuracy	0.775	0.810	0.845	0.846

What to notice:

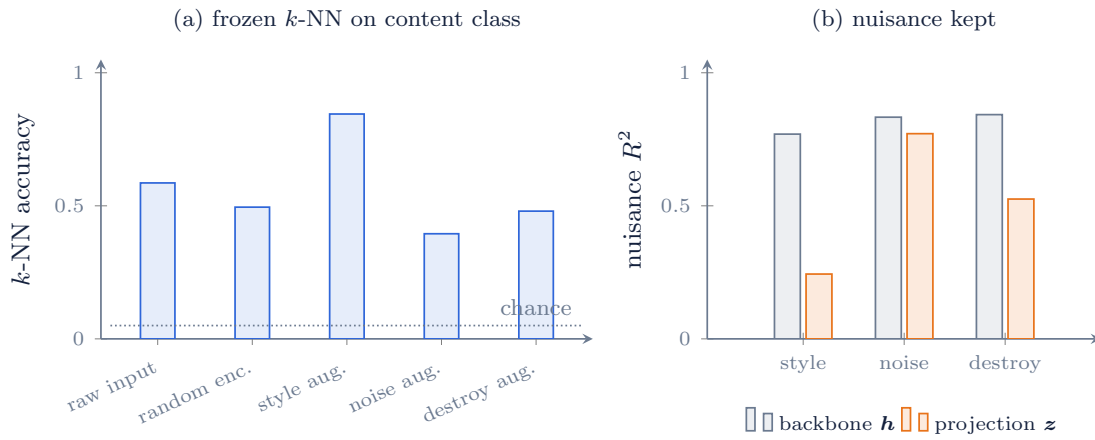


Figure 15.3: What the augmentations teach. (a) Frozen k -NN accuracy on the content class. (b) How much of the nuisance survives in the backbone and in the projection. Resampling the nuisance makes the projection discard it; leaving it shared makes it the most useful cue for the loss.

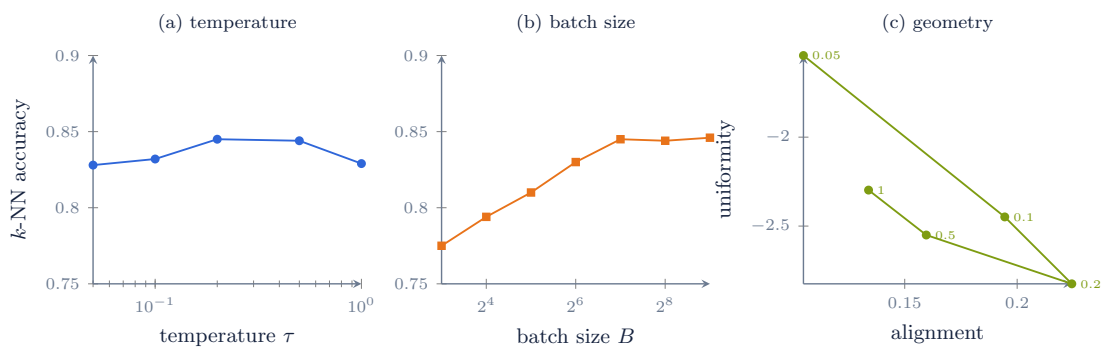


Figure 15.4: (a) k -NN accuracy against temperature and (b) against batch size. (c) Alignment against uniformity for five temperatures: low temperatures align positives tightly but spread the embedding less.

1. **The augmentation policy matters more than any hyperparameter.** It moves k -NN accuracy from 0.40 to 0.85; temperature moves it by 0.02.
2. **Contrastive learning can make features worse than random.** With noise-only augmentation the encoder learns exactly the shortcut we wanted to remove.
3. **Linear probes are forgiving.** The raw input already allows 0.852 by a linear probe, but only 0.586 by k -NN: what contrastive learning mostly buys here is a *metric* in which similar content is close — precisely what retrieval needs.
4. **More negatives help, then saturate.** Accuracy rises until about 128 images per batch and then flattens at this scale and number of classes.
5. **A falling loss is not progress.** The content-destroying run’s loss barely moves; the positives-only run’s loss reaches almost zero by collapsing. Neither is a good representation.

15.9 Moving pieces: diagnosing and fixing

Moving piece	Symptom when it is wrong	How to confirm	Fix
Augmentations	features cluster by camera, lighting or background; good loss, poor probes	nuisance probes; nearest neighbours of queries	resample exactly the nuisances; avoid augmentations that remove the label’s evidence
Temperature	collapse-like instability (too low) or diffuse embeddings (too high)	alignment/uniformity; probe across a log grid	tune τ (0.05–0.5); learnable τ with clipping
Negatives / batch size	weak features at small batch	probe vs. B	larger batch; MoCo queue; memory bank
False negatives	same-class images pushed apart; worse with few classes	fraction of same-class negatives	supervised contrastive with labels; debiased loss; fewer, more diverse negatives
Projection head	probing $\mathbf{z}$ underperforms; information missing downstream	probe $\mathbf{h}$ and $\mathbf{z}$	evaluate $\mathbf{h}$; MLP head
Normalisation	loss decreases by growing norms	track $\ \mathbf{z}\ $	L2-normalise before similarities
Collapse	all embeddings similar; mean cosine near 1	mean pairwise cosine; singular values	negatives, uniformity term, or the mechanisms of chapter 16
Momentum encoder (MoCo)	unstable training, inconsistent keys	key drift between iterations	momentum near 0.99–0.999; queue size vs. drift
Evaluation protocol	numbers not comparable	read layer, pooling, probe training, data	report probe details; include random and supervised baselines

Moving piece	Symptom when it is wrong	How to confirm	Fix
Pretraining data	features miss the target domain	probe on in-domain data	pretrain on in-domain unlabelled images

First things to try: (1) look at nearest neighbours of a few query images in feature space; (2) probe for nuisance variables (camera, site) as well as the target; (3) sweep augmentation strength before anything else; (4) evaluate the backbone, not the projection.

15.10 Application, limitations and failure cases

Application (educational case study). Pretrain a ResNet-50 with a SimCLR-style loss on the 200,000 unlabelled crop photographs, with crops large enough to keep lesions, mild colour jitter (colour carries disease information), blur and flips. Evaluate with a linear probe and k -NN on the 2,000 labelled images, split by farm (chapter 5), against ImageNet-supervised features and random features. Use the k -NN index for **retrieval**: an agronomist uploads a photograph and sees the most similar labelled cases. Probe the features for camera model and farm; if they are easy to predict, the retrieval will return images from the same camera rather than the same disease.

Limitations:

- Contrastive objectives learn what is shared between views; anything the augmentations destroy or leave shared is decided by the designer, not discovered.
- Performance depends heavily on batch size and compute; comparisons must match budgets.
- The mutual-information view explains the loss but not why representations are linearly useful.
- Fine-grained differences (two diseases with similar symptoms) may be treated as nuisance if the augmentations are too strong.

15.11 Bridge to a research paper

Chen, Kornblith, Norouzi and Hinton (2020), *A simple framework for contrastive learning of visual representations* (ICML 2020) — SimCLR (figure 15.1a; [arXiv:2002.05709](https://arxiv.org/abs/2002.05709)).

- **Problem:** self-supervised methods relied on specialised architectures or memory banks; which components actually matter?
- **Mechanism:** two random augmentations per image (crop-and-resize, colour distortion, blur), a ResNet encoder, an MLP projection head, NT-Xent with a temperature, large batches (up to 8,192) with LARS and long training.
- **Central claims:** the *composition* of augmentations is critical — crop plus colour distortion far outperforms either alone; a nonlinear projection head improves the representation *before* it by over ten points; larger batches and longer training help more than in supervised learning; a linear probe reached 76.5% top-1 on ImageNet, matching a supervised ResNet-50.
- **Assumptions to question:** the augmentation recipe is tuned for ImageNet’s object-centric photographs; the gains from batch size are partly gains from compute; linear-probe accuracy is one view of representation quality.

- **Affordable experiment:** in the lab, add a second nuisance vector that the augmentations do *not* resample but that is irrelevant to the label (a “camera signature”), at increasing strength. At what strength does k -NN accuracy start to fall, and does resampling it as an extra augmentation recover the loss? This is SimCLR’s colour-distortion finding in miniature: augmentations must remove the easy shortcuts.

15.12 Memory map

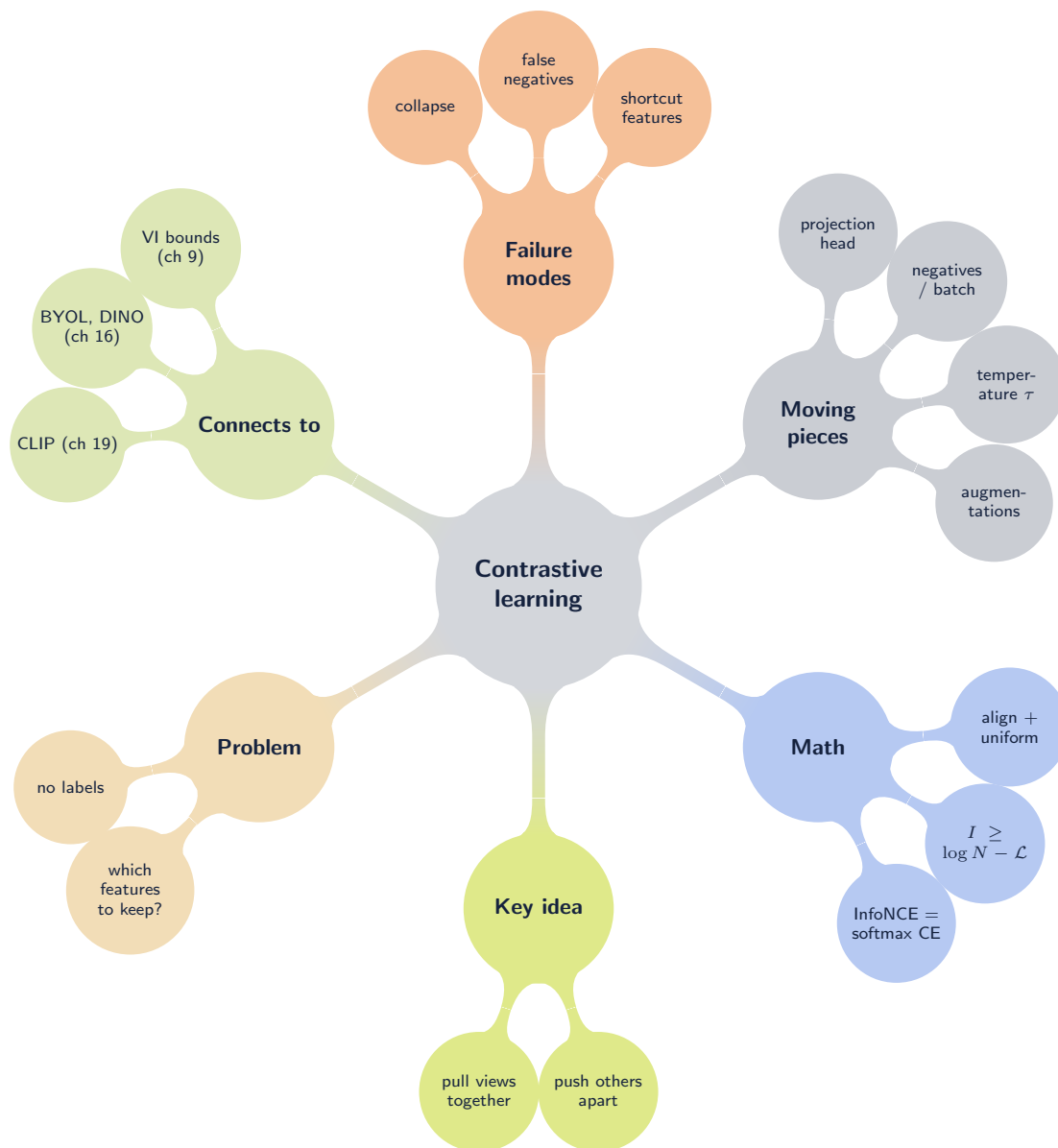


Figure 15.5: Memory map of chapter 15.

Summary. Contrastive learning turns unlabelled images into a classification task: identify each view’s partner among many candidates. The loss is softmax cross-entropy on temperature-scaled cosine similarities; its gradient pulls the positive and pushes each negative in proportion to how confusable it is, tangentially to the sphere. It lower-bounds mutual information by $\log N - \mathcal{L}$, a bound capped at $\log N$; it decomposes into alignment of positives and uniformity of the embedding, and negatives are what prevent collapse.

Augmentations specify the invariances, and are the most important design decision; temperature, batch size and the projection head tune the result. Representations are judged by frozen probes, never by the training loss.

15.13 Exercises

1. *Conceptual.* With four equally likely candidates including the positive, what is the loss?
2. *Conceptual.* Why is the positive included in the denominator?
3. *Conceptual.* Is a smaller contrastive loss always evidence of a better linear probe?
4. *Derivation.* Derive $\partial \ell_i / \partial s_{ik}$ and then $\partial \mathcal{L} / \partial \bar{\mathbf{Z}} = (\mathbf{G} + \mathbf{G}^\top) \bar{\mathbf{Z}}$.
5. *Derivation.* Show that the Jacobian of $\mathbf{z} \mapsto \mathbf{z} / \|\mathbf{z}\|$ is $(\mathbf{I} - \bar{\mathbf{z}} \bar{\mathbf{z}}^\top) / \|\mathbf{z}\|$.
6. *Derivation.* For one anchor, show that as $\tau \rightarrow 0$ the InfoNCE loss tends to $\max(0, \max_{k \neq j} s_{ik} - s_{ij}) / \tau$, a hardest-negative margin loss.
7. *Implementation.* Implement supervised contrastive learning (all same-class views are positives) in the lab and compare k -NN accuracy with 5 and with 50 classes.
8. *Implementation.* Carry out the experiment of section 15.11.
9. *Interpretation.* A contrastive model's nearest neighbours for a leaf image are all photographs from the same farm, of various diseases. Diagnose and propose two fixes.
10. *Open question.* CLIP uses the same loss between images and captions (chapter 19). What plays the role of the augmentation there, and what nuisances would you expect it to remove or keep?

15.13.1 Answer discussions

1. $\log 4 = 1.386$.
2. The denominator normalises a distribution over all candidates, the correct one included; without it the “probability” of the positive is not bounded by one and the loss is not a cross-entropy.
3. No: temperature, candidate composition and shortcuts change the loss without improving the information accessible to the probe (section 15.8, observations 2 and 5).
4. $\partial \ell_i / \partial s_{ik} = (p_{ik} - \mathbb{I}[k = j(i)]) / \tau$; since $s_{ik} = \bar{\mathbf{z}}_i^\top \bar{\mathbf{z}}_k$ appears in rows i and k , $\partial \mathcal{L} / \partial \bar{\mathbf{z}}_a = \sum_b (G_{ab} + G_{ba}) \bar{\mathbf{z}}_b$.
5. $\partial (z_a / \|\mathbf{z}\|) / \partial z_b = \delta_{ab} / \|\mathbf{z}\| - z_a z_b / \|\mathbf{z}\|^3$.
6. $\tau \log \sum_k e^{s_k / \tau} \rightarrow \max_k s_k$; subtract the positive's s_{ij} .
7. With 5 classes, false negatives are frequent and supervised contrastive helps a lot; with 50 classes the gain is smaller.
8. Expect a strength above which the camera signature becomes the easiest cue and accuracy falls; resampling it should restore accuracy.
9. The features encode farm (lighting, soil, camera). Fixes: augment the farm-specific cues (colour constancy, background masking or crops), sample negatives within farms, and evaluate with farm-grouped splits.
10. The pairing of an image with an independently written caption: anything not described in text (exact colours, layout, texture) is “nuisance” and may be discarded; anything both show (objects, scene) is kept.

15.14 Self-check and common mistakes

- ☐ I can write NT-Xent and its gradient, and explain the role of each term.

- ☐ I can state the InfoNCE bound and why it is capped at $\log N$.
- ☐ I can explain alignment and uniformity, and why negatives prevent collapse.
- ☐ I can design augmentations from the invariances a task needs.
- ☐ I can evaluate a representation with frozen probes and baselines.

Common mistakes: comparing loss values across temperatures or batch sizes; evaluating the projection instead of the backbone; colour jitter on a colour-defined task; crops that remove the object; no random-feature baseline; ignoring false negatives with few classes; reading mutual-information estimates above $\log N$; treating a decreasing loss as evidence that features improved.

References

- I. Goodfellow, Y. Bengio and A. Courville. *Deep Learning*. MIT Press, 2016, chapter 15 (representation learning) and section 18.6 (noise-contrastive estimation).
- S. Becker and G. E. Hinton. Self-organizing neural network that discovers surfaces in random-dot stereograms. *Nature*, 355:161–163, 1992.
- R. Hadsell, S. Chopra and Y. LeCun. Dimensionality reduction by learning an invariant mapping. In *CVPR*, 2006.
- M. Gutmann and A. Hyvärinen. Noise-contrastive estimation: a new estimation principle for unnormalized statistical models. In *AISTATS*, 2010.
- Z. Wu, Y. Xiong, S. X. Yu and D. Lin. Unsupervised feature learning via non-parametric instance discrimination. In *CVPR*, 2018.
- A. van den Oord, Y. Li and O. Vinyals. Representation learning with contrastive predictive coding. arXiv:1807.03748, 2018.
- B. Poole, S. Ozair, A. van den Oord, A. A. Alemi and G. Tucker. On variational bounds of mutual information. In *ICML*, 2019.
- K. He, H. Fan, Y. Wu, S. Xie and R. Girshick. Momentum contrast for unsupervised visual representation learning. In *CVPR*, 2020. arXiv:1911.05722.
- T. Chen, S. Kornblith, M. Norouzi and G. Hinton. A simple framework for contrastive learning of visual representations. In *ICML*, 2020. arXiv:2002.05709.
- T. Wang and P. Isola. Understanding contrastive representation learning through alignment and uniformity on the hypersphere. In *ICML*, 2020.
- M. Tschannen, J. Djolonga, P. K. Rubenstein, S. Gelly and M. Lucic. On mutual information maximization for representation learning. In *ICLR*, 2020.
- P. Khosla et al. Supervised contrastive learning. In *NeurIPS*, 2020.
- J. von Kügelgen et al. Self-supervised learning with data augmentations provably isolates content from style. In *NeurIPS*, 2021.
- A. Radford et al. Learning transferable visual models from natural language supervision. In *ICML*, 2021.

Chapter 16

Teacher–student and masked learning

Prerequisites: chapter 15 (contrastive learning, probes); chapter 14 (vision transformers); chapter 10 (autoencoders).

Learning outcomes: after this chapter you can

1. state precisely what an EMA teacher and a stop-gradient each do, and why they are different operations;
2. explain why BYOL and SimSiam do not collapse although a constant output minimises their loss, and test it by ablation;
3. derive DINO’s cross-entropy as entropy plus KL, and explain centring and sharpening as guards against two opposite collapses;
4. formulate masked autoencoding, explain why the loss is computed on masked patches and why high mask ratios need redundant data;
5. compare contrastive, distillation, masked-reconstruction and joint-embedding-prediction targets, and diagnose collapse with measurements rather than slogans.

16.1 A concrete problem

Chapter 15’s contrastive encoder needed **negatives**: many other images in each batch to push away. That created two problems for the crop-health team. Large batches need large hardware; and with only a dozen disease classes, many “negatives” are in fact the same disease (false negatives).

Can an encoder learn from unlabelled images without negatives? The obvious attempt — make the embeddings of two views of the same image equal — fails immediately: mapping every image to the same vector makes the loss zero. This chapter is about the two families of ideas that escape this trap and now dominate visual pretraining:

- **teacher–student self-distillation** (BYOL, SimSiam, DINO): the student predicts the output of a slowly moving copy of itself, with asymmetries that make collapse unstable;
- **masked modelling** (MAE, BEiT, iBOT): hide parts of the image and predict them, so there is no trivial answer to begin with.

Both lead to the foundation models used across the rest of the book — DINOv2 features, the MAE-pretrained encoder inside SAM (chapter 13), and the joint-embedding predictive architectures of chapter 17.

16.2 How we got here

Distillation and moving teachers (2015–2017). Hinton, Vinyals and Dean (2015) trained a small “student” to match a large teacher’s softened probabilities. Tarvainen and Valpola (2017) showed for semi-supervised learning that a teacher whose weights are an exponential moving average (EMA) of the student’s gives better targets than the student itself — the **mean teacher**.

Learning without negatives (2020–2021). BYOL (Grill and colleagues, 2020) added a predictor to the online network and used an EMA target with stop-gradient — and, to general surprise, did not collapse. SimSiam (Chen and He, 2021) showed the EMA was not essential: stop-gradient plus predictor suffices. SwAV (Caron and colleagues, 2020) clustered features online with balanced assignments. **DINO** (Caron and colleagues, 2021) applied self-distillation to vision transformers with centring and sharpening, and found attention maps that segment objects without labels. Barlow Twins (Zbontar and colleagues, 2021) and VICReg (Bardes, Ponce and LeCun, 2022) prevented collapse explicitly with variance and covariance terms.

Masked modelling (2008–2022). Denoising autoencoders (Vincent and colleagues, 2008) reconstructed corrupted inputs; context encoders (Pathak and colleagues, 2016) inpainted image regions. After BERT (Devlin and colleagues, 2019) made masked prediction the standard in language, iGPT (2020) and BEiT (Bao and colleagues, 2021) brought it to images. **MAE** (He and colleagues, 2022) masked 75% of patches, encoded only the visible ones, and reconstructed pixels with a small decoder — simple, fast and scalable. iBOT (Zhou and colleagues, 2022) and data2vec (Baevski and colleagues, 2022) predicted *features* of masked patches instead of pixels.

Foundation features (2023–2025). DINOv2 (Oquab and colleagues, 2023) combined DINO’s image-level loss, iBOT’s masked patch-level loss and a uniformity regulariser, and trained on 142 million curated images — this chapter’s paper bridge; its frozen features rival supervised and weakly supervised ones across tasks. Registers (Darcet and colleagues, 2024; chapter 14) and DINOv3 (2025) refined the recipe at larger scale.

16.3 Assumptions and notation

Symbol	Meaning
θ, ξ	student (online) and teacher (target) parameters
$m \in [0, 1)$	EMA momentum: $\xi \leftarrow m\xi + (1 - m)\theta$ (not the temperature)
$\text{sg}(\cdot)$	stop-gradient: the value is used, its derivative is treated as zero
f, g, q	encoder, projector, predictor
$\bar{\mathbf{v}} = \mathbf{v} / \ \mathbf{v}\ $	normalised vector
$K, \tau_s, \tau_t, \mathbf{c}$	DINO: number of prototypes, student and teacher temperatures, centre
$\mathbf{p}_s, \mathbf{p}_t$	student and teacher distributions over K prototypes
$\mathcal{M}, r$	set of masked patches and mask ratio

Assumptions:

1. **Views share content** (as in chapter 15): the teacher’s output on one view is a sensible target for the student’s output on another.
2. **For masking, the data are redundant**: missing parts are predictable from visible ones. Natural images are highly redundant in space; low-dimensional feature vectors are not.
3. **Collapse is possible** for every objective in this chapter; it is avoided by design choices whose effect must be measured.

16.4 Mathematical formulation

BYOL (and SimSiam with $m = 0$): for two views $\tilde{\mathbf{x}}, \tilde{\mathbf{x}}'$,

$$\mathcal{L}_{\text{BYOL}} = \|\bar{\mathbf{q}}_{\boldsymbol{\theta}}(\tilde{\mathbf{x}}) - \text{sg}(\bar{\mathbf{z}}_{\boldsymbol{\xi}}(\tilde{\mathbf{x}}'))\|^2 = 2 - 2 \cos(\mathbf{q}_{\boldsymbol{\theta}}, \text{sg}(\mathbf{z}_{\boldsymbol{\xi}})),$$

symmetrised over the two views, with $\mathbf{q}_{\boldsymbol{\theta}} = \mathbf{q} \circ \mathbf{g} \circ \mathbf{f}$ (online, with predictor) and $\mathbf{z}_{\boldsymbol{\xi}} = \mathbf{g} \circ \mathbf{f}$ (target, EMA weights, no predictor).

DINO: with logits $\mathbf{l}_s, \mathbf{l}_t \in \mathbb{R}^K$,

$$\mathbf{p}_s = \text{softmax}(\mathbf{l}_s / \tau_s), \quad \mathbf{p}_t = \text{sg}(\text{softmax}((\mathbf{l}_t - \mathbf{c}) / \tau_t)),$$

$$\mathcal{L}_{\text{DINO}} = - \sum_k p_{t,k} \log p_{s,k}, \quad \mathbf{c} \leftarrow \lambda \mathbf{c} + (1 - \lambda) \bar{\mathbf{l}}_t,$$

with $\tau_t < \tau_s$, the teacher an EMA of the student, global views for the teacher and global and local views for the student.

MAE: mask a random set $\mathcal{M}$ of a fraction r of the patches; encode the visible patches; decode encoded patches plus mask tokens; minimise

$$\mathcal{L}_{\text{MAE}} = \frac{1}{|\mathcal{M}|} \sum_{j \in \mathcal{M}} \|\hat{\mathbf{x}}_j - \mathbf{x}_j\|^2$$

(with per-patch normalised pixel targets in the original paper).

16.5 Derivation, with reasons for each step

Step 1 — EMA and stop-gradient are different operations. *Stop-gradient* acts within one step: the target’s value enters the loss, but the chain rule stops there, so no gradient reaches the target branch from this loss. *EMA* acts across steps: after the optimiser updates $\boldsymbol{\theta}$, the teacher moves a fraction $1 - m$ toward it. Index the steps so that $\boldsymbol{\theta}_t$ is the student after the t -th update, $\boldsymbol{\xi}_0$ is the initial teacher, and $\boldsymbol{\xi}_t = m\boldsymbol{\xi}_{t-1} + (1 - m)\boldsymbol{\theta}_t$ for $t \geq 1$. Unrolling t times,

$$\boldsymbol{\xi}_t = m^t \boldsymbol{\xi}_0 + (1 - m) \sum_{k=0}^{t-1} m^k \boldsymbol{\theta}_{t-k}.$$

The coefficients sum to one: $(1 - m) \sum_{k=0}^{t-1} m^k = 1 - m^t$, plus m^t on $\boldsymbol{\xi}_0$. So the teacher is a weighted average of the initial teacher and the past students, with weights decaying

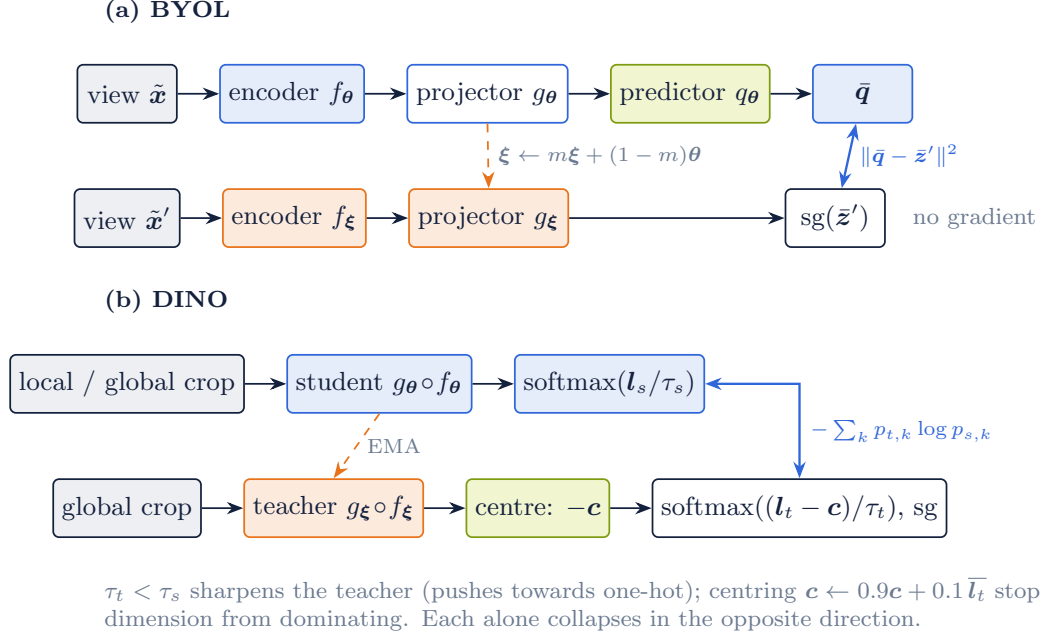


Figure 16.1: Two teacher–student designs without negatives. (a) BYOL, redrawn after Fig. 2 of Grill et al. (2020). (b) DINO, redrawn after Fig. 2 of Caron et al. (2021). Papers: [arXiv:2006.07733](https://arxiv.org/abs/2006.07733), [arXiv:2104.14294](https://arxiv.org/abs/2104.14294).

geometrically into the past. Once m^t is negligible, it behaves like an average with an effective memory of about $1/(1-m)$ steps: 100 steps for $m = 0.99$, 1,000 for 0.999. The teacher is a smoothed, slightly delayed, ensemble-like version of the student. A teacher can be stop-gradient without EMA (SimSiam: $m = 0$, same weights) or EMA with gradients (a valid but different algorithm).

Step 2 — why collapse is a solution, and why it need not be reached. If every image is mapped to the same unit vector $\mathbf{u}$ and the predictor is the identity, then $\bar{\mathbf{q}} = \bar{\mathbf{z}} = \mathbf{u}$ and $\mathcal{L}_{\text{BYOL}} = 0$. So collapse is a global minimum, and the question is about **dynamics**: does training reach it? Two observations:

- *Without stop-gradient*, the loss is differentiated through both branches, and its gradient pulls both outputs toward each other and toward any common value; nothing pushes images apart. In the lab the output spread falls to 0.02 and k -NN accuracy (0.382) drops below a random encoder’s (0.495).
- *With stop-gradient and a predictor*, the student chases a target it cannot move during the step. Chen and He (2021) read SimSiam as alternating optimisation: with an extra variable $\boldsymbol{\eta}_x$ per image, minimise $\mathbb{E}\|\mathcal{F}_\theta(t(\mathbf{x})) - \boldsymbol{\eta}_x\|^2$; the $\boldsymbol{\eta}$ -step sets $\boldsymbol{\eta}_x$ to the average representation of x over augmentations (the stop-gradient target), the $\boldsymbol{\theta}$ -step moves the network toward it — an EM-like scheme (chapter 8) that does not by itself drive different images together. Tian, Chen and Ganguli (2021) analysed linear networks and showed that the predictor learns to align with the correlation structure of the representation, which keeps the non-collapsed eigen-directions alive.

Neither argument is a guarantee for every architecture; the reliable test is the ablation. In the lab, removing the predictor collapses the output (spread 0.05), and removing stop-gradient collapses it further (0.02); BYOL and SimSiam keep it (0.66–0.67) and reach k -NN accuracy 0.80.

Step 3 — DINO’s loss is entropy plus KL. For any distributions,

$$-\sum_k p_{t,k} \log p_{s,k} = H(\mathbf{p}_t) + D_{\text{KL}}(\mathbf{p}_t \| \mathbf{p}_s).$$

Reason: add and subtract $\sum_k p_{t,k} \log p_{t,k}$. The teacher is fixed within a step, so the student minimises the KL — it matches the teacher. Two trivial ways to make the KL zero for every image:

- **uniform collapse:** teacher and student both output $1/K$ everywhere;
- **one-hot collapse:** both put all mass on the same prototype for every image.

Sharpening uses a low teacher temperature ($\tau_t < \tau_s$). Dividing logits by a smaller temperature amplifies the differences that already exist between them, so wherever the teacher’s logits differ, its target is peakier than it would be at τ_s . It cannot create differences: equal logits give a uniform softmax at any positive temperature, and the teacher and student logits are different functions, so a lower teacher temperature alone does not guarantee a peakier target than the student’s output. In the training dynamics, sharpening opposes drift toward uniform outputs, and if used alone it tends to favour one-hot collapse. **Centring** subtracts a running mean of the teacher logits, so a prototype that is high for every image is pulled down; it opposes one-hot collapse and, if used alone, tends to favour uniform outputs. Caron et al. (2021) report that both are needed; the lab’s ablation (step 4) shows the same pattern on a small problem, which is evidence about these settings rather than a proof that the two always balance.

Step 4 — measuring the balance: per-image versus marginal entropy. Let $\bar{\mathbf{p}}_t = \mathbb{E}_x \mathbf{p}_t(x)$ be the average teacher output. Healthy training has **low** per-image entropy $\mathbb{E}_x H(\mathbf{p}_t(x))$ (each image confidently assigned) and **high** marginal entropy $H(\bar{\mathbf{p}}_t)$ (prototypes used evenly). Their difference is the mutual information between image and prototype:

$$I(x; \mathbf{k}) = H(\bar{\mathbf{p}}_t) - \mathbb{E}_x H(\mathbf{p}_t(x)).$$

In the lab ($K = 32$, $\log K = 3.47$): full DINO has per-image entropy 0.86 and marginal entropy 3.05, so $I = 2.19$ nats; without centring both are about 0.02–0.04 ($I \approx 0$, one-hot collapse); without sharpening both are 3.45–3.47 ($I \approx 0$, uniform collapse). Logging these two numbers is the cheapest collapse detector there is.

Step 5 — explicit anti-collapse terms. Instead of asymmetry, one can penalise collapse directly. VICReg adds a **variance** term (hinge on the standard deviation of each embedding dimension across the batch) and a **covariance** term (off-diagonal covariance squared) to an invariance term; Barlow Twins drives the cross-correlation matrix of the two views’ embeddings toward the identity. DINOv2 adds the KoLeo regulariser $-\frac{1}{n} \sum_i \log \min_{j \neq i} \|\bar{\mathbf{z}}_i - \bar{\mathbf{z}}_j\|$, which spreads features uniformly (chapter 15’s uniformity, by nearest neighbours).

Step 6 — masked autoencoding: why the loss is on the masked patches. Visible patches pass through the encoder; if their reconstruction were scored, a large part of the loss could be reduced by copying the input, which teaches little. Scoring only the masked patches forces the network to infer them from context. In the lab, scoring all patches gives a lower reported MSE (0.93 vs. 1.18) but no better features — a reminder that the number reported depends on what is averaged.

Step 7 — the mask ratio and redundancy. The optimal ratio depends on how predictable the missing part is. Natural images are spatially redundant: a missing patch

can often be interpolated from its neighbours, so a low ratio makes the task trivial and teaches low-level interpolation; MAE found 75% best for images (and 90% for video, chapter 17). The lab’s synthetic “images” are 32 numbers with little redundancy: each patch carries information the others do not, and the best ratio is the *lowest* (0.125: k -NN 0.595; 0.875: 0.443). The general rule is not “mask 75%”, but “mask enough that copying fails and little enough that prediction is still possible”.

Step 8 — pixels or features as targets. Pixel targets demand detail — texture, exact colour, illumination — much of which is irrelevant to semantic tasks, which is one reason MAE features are weak under linear probes and k -NN (as in the lab: 0.48–0.60, against 0.80 for BYOL and 0.85 for contrastive learning) but strong after fine-tuning. Predicting **features** of the masked region (BEiT’s tokens, iBOT’s teacher features, data2vec, I-JEPA in chapter 17) moves the target toward semantics — at the price of re-introducing a teacher, and hence collapse risk.

Step 9 — the asymmetric encoder makes MAE cheap. MAE’s encoder processes only the visible 25% of tokens; attention cost is quadratic in tokens (chapter 14), so the encoder is roughly 3–16× cheaper per image than processing all tokens, and a small decoder handles the full set. This is why MAE scaled to very large ViTs, and why its pretrained encoder (ViT-H) became SAM’s image encoder (chapter 13).

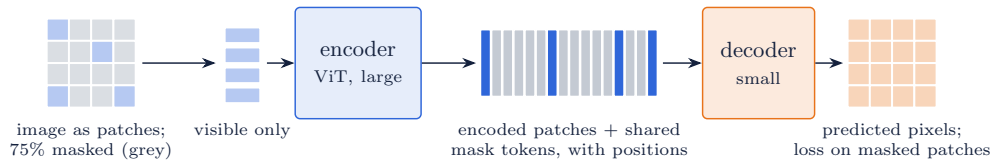
Step 10 — a map of targets. All four families predict something about one view from another; they differ in what is predicted and how trivial solutions are excluded:

Family	Target	Guard against triviality	Typical strength	Typical weakness
Contrastive (ch. 15)	which candidate is the positive	negatives	strong frozen features, retrieval	batch size, false negatives
Self-distillation (BYOL, DINO)	teacher’s embedding or distribution	predictor, stop-gradient, EMA, centering/sharpening	strong frozen features without negatives	silent collapse, many knobs
Masked reconstruction (MAE)	missing pixels	none needed: the target is the data	scalable, good after fine-tuning	weaker frozen features; pixel detail
Joint-embedding prediction (I-JEPA, ch. 17)	teacher features of missing regions	EMA teacher, stop-gradient	semantic targets, efficient	collapse risk, design of masks

16.6 Numerical worked example

EMA. Teacher parameter 2, student 4, $m = 0.9$: new teacher $0.9 \cdot 2 + 0.1 \cdot 4 = 2.2$. With $m = 0.99$ a student change is felt at 1% per step; after 100 steps of a constant student value, the teacher has covered $1 - 0.99^{100} = 63\%$ of the gap.

A collapsed BYOL output. If $\bar{\mathbf{q}} = \bar{\mathbf{z}}' = (1, 0)$ for every image, the loss is $\|(1, 0) - (1, 0)\|^2 = 0$. If two images have $\bar{\mathbf{q}} = (1, 0)$ and $\bar{\mathbf{z}}' = (0.6, 0.8)$, the loss is $0.4^2 + 0.8^2 = 0.8 = 2 - 2(0.6)$.



after pre-training the decoder is discarded; the encoder sees whole images for downstream tasks. Removing 75% of tokens makes the encoder about $3\times$ cheaper and the task hard enough that copying neighbours does not solve it — which relies on the redundancy of natural images.

Figure 16.2: Masked autoencoder, redrawn after Fig. 1 of He et al. (2022): the encoder sees only visible patches; a small decoder receives encoded patches and mask tokens and predicts the masked pixels. Paper: [arXiv:2111.06377](https://arxiv.org/abs/2111.06377).

DINO entropies. $K = 4$, teacher outputs for two images (0.97, 0.01, 0.01, 0.01) and (0.01, 0.97, 0.01, 0.01): per-image entropy 0.17 nats each, average output (0.49, 0.49, 0.01, 0.01) with entropy 0.79, so $I = 0.62$ nats. If both images give (0.97, 0.01, 0.01, 0.01) (one-hot collapse), the marginal entropy is also 0.17 and $I = 0$.

MAE loss. Masked targets (1, 3), predictions (2, 1): $\text{MSE} = ((2 - 1)^2 + (1 - 3)^2)/2 = 2.5$. Averaging visible patches into the same number changes the objective; visible patches are easy, so the number falls.

16.7 Small implementation and checks

The lab `labs/ch16-selfdistill/selfdistill.py` uses chapter 15's synthetic world and evaluation, so every number is comparable with contrastive learning. The BYOL step:

```
Z, cache = enc_forward(p, X1); Q = Z @ p["P"] # online:
→ encoder + linear predictor
T, _ = enc_forward(tgt, X2) # target: EMA
→ weights, no gradient
Qb, nq = normalise(Q); Tb, _ = normalise(T)
loss = np.sum((Qb - Tb) ** 2) / B
GQ = back_normalise(2 * (Qb - Tb) / B, Qb, nq) # gradient only
→ into the online branch
...; opt.step(p, grads); ema(tgt, p, m=0.99) # then move the
→ teacher
```

DINO's student gradient is $(\mathbf{p}_s - \mathbf{p}_t)/(\tau_s B)$ on the logits; the teacher is updated only by EMA and the centre by a running mean of teacher logits.

Checks, in `test_selfdistill.py`: the EMA worked example; a finite-difference check of the BYOL loss with and without stop-gradient; BYOL keeps spread and accuracy while the no-stop-gradient variant collapses; DINO without centring concentrates on few prototypes and without sharpening becomes uniform; and MAE's masked loss rises with the mask ratio.

16.8 Experiment and expected observations

Run `python labs/ch16-selfdistill/selfdistill.py` (seed 0; 1,000 Adam steps, batch 128; frozen evaluation as in chapter 15). For reference: random encoder k -NN 0.495, contrastive encoder (chapter 15) 0.845.

method	k -NN	linear probe	output spread	note
BYOL (predictor, EMA 0.99, stop-grad)	0.795	0.871	0.66	
SimSiam (predictor, stop-grad, no EMA)	0.801	0.865	0.67	
no predictor	0.599	0.811	0.05	collapsed output
no stop-gradient	0.382	0.685	0.02	collapsed, worse than random
DINO (centring + sharpening)	0.762	0.853	—	teacher entropy 0.86 per image, 3.05 marginal
DINO without centring	0.651	0.820	—	0.02 / 0.04: one-hot collapse
DINO without sharpening	0.606	0.803	—	3.45 / 3.47: uniform collapse

MAE						0.75, loss on all
mask ratio	0.125	0.25	0.5	0.75	0.875	
reported	0.247	0.417	0.696	1.181	1.446	0.932
MSE						
k -NN / probe	0.595 / 0.803	0.568 / 0.808	0.507 / 0.756	0.484 / 0.730	0.443 / 0.733	0.502 / 0.764

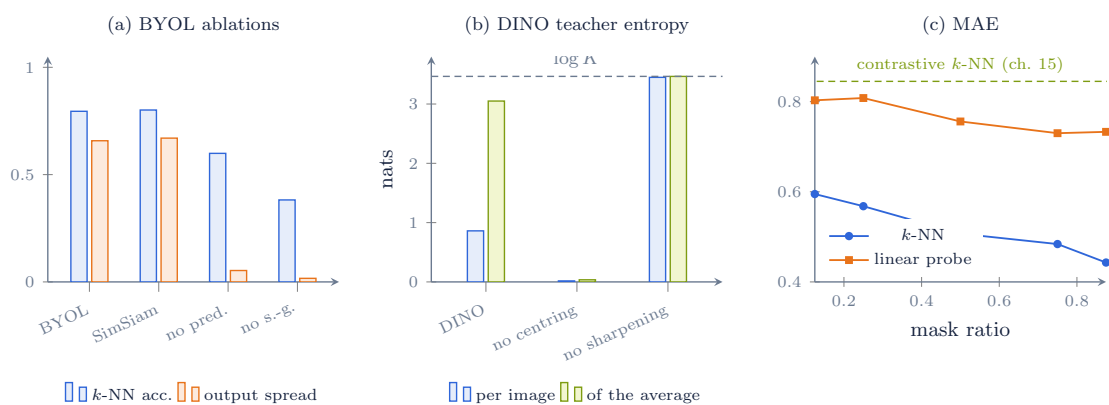


Figure 16.3: (a) BYOL-family ablations: accuracy and spread of normalised outputs. (b) DINO’s teacher entropy per image and of the average output: centring and sharpening each prevent one collapse. (c) MAE accuracy against mask ratio on non-redundant data, with the contrastive encoder for reference.

What to notice:

1. **Collapse can be partial.** The no-predictor run’s linear probe (0.811) still beats a random encoder; its collapse shows in the output spread and k -NN, not in every metric. Measure several.
2. **The loss lies.** The two collapsed BYOL variants reach the *lowest* losses (0.004 and 0.001) against BYOL’s 0.31.
3. **Each DINO guard has an opposite failure.** The two collapsed runs sit at the two ends of the entropy axis; the healthy run is low per image and high on average.
4. **Without negatives, almost as good.** BYOL/SimSiam reach k -NN 0.80 against contrastive 0.845 here — with no negatives at all.
5. **MAE’s recipe is data-dependent.** On data without spatial redundancy, high mask ratios hurt and frozen features are weak; the 75% rule comes from natural images.

16.9 Moving pieces: diagnosing and fixing

Moving piece	Symptom when it is wrong	How to confirm	Fix
Stop-gradient	loss falls to near zero; outputs identical	output spread; per-dimension std of normalised features	detach the target branch; check the code path
Predictor	slow collapse in BYOL/SimSiam	ablate the predictor; spread over training	keep a predictor; larger predictor learning rate
EMA momentum	instability (too low) or slow progress (too high)	teacher–student agreement over time	m 0.99–0.9995 with a cosine schedule toward 1
DINO centring	one-hot collapse: marginal entropy near 0	per-image and marginal teacher entropy	enable centring; tune its momentum
DINO temperatures	uniform collapse: both entropies near $\log K$	same	$\tau_t < \tau_s$; warm up τ_t from 0.04 to 0.07
Number of prototypes K	many prototypes unused	histogram of teacher argmax	K large (thousands at scale); KoLeo / uniformity terms
Mask ratio	trivial interpolation (too low) or impossible task (too high)	probe accuracy vs. ratio; reconstruction error	match to redundancy: ~75% images, ~90% video
Reconstruction target	features dominated by texture and lighting	probe for nuisance variables	normalised pixel targets; feature targets (iBOT, JEPA)
Loss reduction	reported MSE not comparable	which patches are averaged	report masked-only loss
Evaluation	MAE judged by linear probe only	fine-tune and probe	report both frozen and fine-tuned results

First things to try: (1) log output spread (or teacher entropies) every few hundred steps — collapse often begins quietly; (2) ablate one component at a time with the same data and budget; (3) evaluate frozen features with k -NN and a linear probe, and fine-tuning where the method expects it.

16.10 Application, limitations and failure cases

Application (educational case study). For the crop-health archive, take a publicly released DINOv2 checkpoint, extract frozen patch and image features, and evaluate (a) a linear probe for disease, (b) k -NN retrieval, and (c) a linear per-patch head for lesion segmentation, using farm-grouped splits (chapter 5). Compare with ImageNet-supervised features and with features after a short in-domain self-distillation fine-tune on the unlabelled images. Inspect the PCA of patch features — DINO-family features often separate leaf from soil without supervision — but treat such pictures as illustrations, not evidence; numbers come from the probes.

Limitations:

- No guarantee against collapse exists for these asymmetric methods; it is an empirical property of a recipe and must be monitored.
- Many interacting hyperparameters (momentum, temperatures, centring rate, crops, prototypes) make results hard to transfer across domains.
- Masked pixel reconstruction learns what is needed to paint pixels, which overlaps only partly with semantics.
- Large-scale results depend heavily on data curation, which is rarely fully reproducible.

16.11 Bridge to a research paper

Oquab, Darcet, Moutakanni and colleagues (2023), *DINOv2: learning robust visual features without supervision* ([arXiv:2304.07193](https://arxiv.org/abs/2304.07193); TMLR 2024).

- **Problem:** self-supervised features had not matched the general-purpose quality of text-supervised ones (CLIP, chapter 19) when used frozen across tasks.
- **Mechanism:** DINO’s image-level self-distillation loss plus iBOT’s masked patch-level loss (the student predicts the teacher’s features of masked patches), separate heads, KoLeo regularisation for uniform features, Sinkhorn-Knopp centring, high-resolution adaptation at the end, distillation into smaller models, and an automatic **data-curation** pipeline that retrieves 142 million images similar to curated datasets from a large uncured pool.
- **Central claim:** frozen DINOv2 features with linear heads match or exceed weakly supervised (image–text) features on classification, and are much better on dense tasks such as segmentation and depth.
- **Assumptions to question:** how much comes from the curated data rather than the objective; whether curation toward existing benchmarks inflates benchmark results; how features behave on very different domains (microscopy, fields, satellites).
- **Affordable experiment:** in the lab, add a KoLeo term to DINO’s student embeddings and sweep its weight (0, 0.01, 0.1). Measure k -NN accuracy, per-image and marginal teacher entropies, and the spread of features. Then repeat with $K = 8$ prototypes instead of 32: does the regulariser matter more when prototypes are scarce?

16.12 Memory map



Figure 16.4: Memory map of chapter 16.

Summary. Without negatives, matching two views is trivially solved by a constant output; self-distillation avoids reaching that solution through asymmetry. A predictor on the student, a stop-gradient on the target and (optionally) an EMA teacher make BYOL and SimSiam work; removing the predictor or the stop-gradient collapses them. DINO’s loss is entropy plus KL; sharpening prevents uniform outputs and centring prevents one prototype from winning, and the gap between marginal and per-image entropy measures whether the balance holds. Masked autoencoding avoids triviality by hiding the target: the loss is on masked patches, the mask ratio must match the redundancy of the data, and pixel targets favour detail over semantics. Modern foundation features (DINOv2) combine image-level distillation with masked feature prediction on curated data.

16.13 Exercises

1. *Conceptual.* Does stop-gradient mean the teacher can never change?
2. *Conceptual.* Why is zero matching loss compatible with a trivial constant representation?
3. *Conceptual.* Which measurements help detect collapse?
4. *Derivation.* Unroll the EMA recursion and find the weight on the student from k steps ago. What is the effective window for $m = 0.996$?
5. *Derivation.* Prove $-\sum_k p_k \log q_k = H(\mathbf{p}) + D_{\text{KL}}(\mathbf{p} \parallel \mathbf{q})$ and explain which term the student can change.
6. *Derivation.* For BYOL without stop-gradient, compute the gradient of $\|\bar{\mathbf{q}} - \bar{\mathbf{z}}'\|^2$ with respect to both branches and show that it never increases the distance between embeddings of *different* images.
7. *Implementation.* Add VICReg’s variance and covariance terms to the no-predictor BYOL variant and check whether they prevent its collapse.
8. *Implementation.* Carry out the experiment of section 16.11.
9. *Interpretation.* A team reports that their MAE encoder is “worse than random” under a linear probe but excellent after fine-tuning. Is this contradictory?
10. *Open question.* Masked prediction in feature space re-introduces a teacher. Is there a target that is both semantic and free of collapse risk?

16.13.1 Answer discussions

1. No: the EMA (or another rule) changes it between steps; stop-gradient only stops gradients within the current loss.
2. If prediction and target are the same constant, their difference is zero for every image.
3. Per-dimension standard deviation of normalised outputs, the covariance spectrum or effective rank, feature norms, DINO’s per-image and marginal entropies, nearest-neighbour diversity and downstream probes; no single number suffices.
4. Weight $(1 - m)m^k$; window $1/(1 - m) = 250$ steps.
5. Add and subtract $\sum p \log p$; only D_{KL} depends on the student.
6. The gradients pull $\bar{\mathbf{q}}$ and $\bar{\mathbf{z}}'$ toward each other along the sphere; no term involves pairs of different images, so nothing pushes them apart, and any shared component that reduces the loss is reinforced.
7. Variance and covariance terms should keep the per-dimension standard deviation up and restore accuracy.
8. Expect KoLeo to raise feature spread and marginal entropy, mattering more with few prototypes.
9. No. Linear probes measure linear accessibility of the target in frozen features; MAE features encode information nonlinearly and in detail, which fine-tuning can reorganise.
10. Open; candidates include targets from a frozen, separately trained encoder (no co-adaptation), or explicit regularisers; each moves the problem rather than removing it.

16.14 Self-check and common mistakes

- ☐ I can state the difference between EMA and stop-gradient precisely.
- ☐ I can explain why BYOL/SimSiam need a predictor and stop-gradient, and show it with an ablation.
- ☐ I can decompose DINO’s loss and say which collapse each of centring and sharpening

prevents.

- ☐ I can explain the choice of masked-only loss and mask ratio in MAE.
- ☐ I can compare the four families of self-supervised targets.

Common mistakes: confusing the EMA momentum with a temperature; forgetting to detach the target; reading a low loss as success; judging collapse by one statistic; using DINO without centring or with $\tau_t \geq \tau_s$; averaging visible patches into the MAE loss; copying the 75% mask ratio to data without spatial redundancy; evaluating MAE only by linear probe; attributing large-scale results to the objective when data curation changed too.

References

- I. Goodfellow, Y. Bengio and A. Courville. *Deep Learning*. MIT Press, 2016, chapter 14 (autoencoders, denoising).
- G. Hinton, O. Vinyals and J. Dean. Distilling the knowledge in a neural network. arXiv:1503.02531, 2015.
- A. Tarvainen and H. Valpola. Mean teachers are better role models. In *NeurIPS*, 2017.
- J.-B. Grill et al. Bootstrap your own latent: a new approach to self-supervised learning. In *NeurIPS*, 2020. arXiv:2006.07733.
- X. Chen and K. He. Exploring simple Siamese representation learning. In *CVPR*, 2021.
- Y. Tian, X. Chen and S. Ganguli. Understanding self-supervised learning dynamics without contrastive pairs. In *ICML*, 2021.
- M. Caron et al. Emerging properties in self-supervised vision transformers. In *ICCV*, 2021. arXiv:2104.14294.
- J. Zbontar, L. Jing, I. Misra, Y. LeCun and S. Deny. Barlow twins. In *ICML*, 2021.
- A. Bardes, J. Ponce and Y. LeCun. VICReg: variance-invariance-covariance regularization for self-supervised learning. In *ICLR*, 2022.
- P. Vincent, H. Larochelle, Y. Bengio and P.-A. Manzagol. Extracting and composing robust features with denoising autoencoders. In *ICML*, 2008.
- H. Bao, L. Dong, S. Piao and F. Wei. BEiT: BERT pre-training of image transformers. In *ICLR*, 2022.
- K. He, X. Chen, S. Xie, Y. Li, P. Dollár and R. Girshick. Masked autoencoders are scalable vision learners. In *CVPR*, 2022. arXiv:2111.06377.
- J. Zhou et al. iBOT: image BERT pre-training with online tokenizer. In *ICLR*, 2022.
- M. Oquab et al. DINOv2: learning robust visual features without supervision. *TMLR*, 2024. arXiv:2304.07193.

Chapter 17

JEPA and video

Prerequisites: chapter 16 (EMA teachers, stop-gradient, masked modelling); chapter 14 (tokens and attention cost).

Learning outcomes: after this chapter you can

1. specify a joint-embedding predictive architecture (context encoder, EMA target encoder, predictor) and say exactly which information each part sees during training and at inference;
2. explain why predicting features rather than pixels avoids modelling unpredictable detail, and what it risks instead;
3. design masks for images and video so that the task cannot be solved by copying neighbours, and count video tokens and attention cost;
4. evaluate temporal understanding with controls — order-aware probes, frame shuffling, balanced labels — instead of benchmark names or similarity statistics;
5. separate the claims of a video foundation model (features, anticipation, planning) and evaluate each.

17.1 A concrete problem

Sticky traps and field cameras record **video** of insects: some crawl toward the crop, some away; some species look alike but move differently. Two questions follow, and they are different:

1. *What* is it? (appearance — one frame may be enough)
2. *Which way, and how*, is it moving? (dynamics — needs the order of frames)

Hours of video are cheap; labels are not. Chapter 16’s methods learn from single images; video adds time, and with it two new opportunities and two new traps. The opportunity: the future is a free target — predict what happens next, and you must model how things move. The traps: neighbouring frames are nearly identical, so a model can “predict” by copying; and many video labels can be guessed from a single frame, so a model can score well while ignoring time altogether.

This chapter develops **joint-embedding predictive architectures (JEPAs)** — predict the *representation* of missing parts of an image or video from the visible parts — and the experimental discipline needed to tell temporal understanding from appearance shortcuts. Its paper bridge, V-JEPA 2, connects video representation learning to prediction and planning (chapter 23).

17.2 How we got here

Prediction as a learning signal (1999–2002). Rao and Ballard (1999) modelled the visual cortex as predicting its own inputs (“predictive coding”). Slow feature analysis (Wiskott and Sejnowski, 2002) found features that change slowly over time — invariances learned from temporal continuity.

Video recognition architectures (2014–2021). Two-stream networks (Simonyan and Zisserman, 2014) combined appearance with optical flow; 3-D convolutions (C3D 2015; I3D, Carreira and Zisserman, 2017, with the Kinetics dataset) learned spatio-temporal filters; SlowFast (Feichtenhofer and colleagues, 2019) used two frame rates. Video transformers (TimeSformer, ViViT, 2021) tokenised **tubelets** and factorised attention over space and time.

Predicting pixels (2014–2016). Early self-supervised video models predicted future frames in pixel space (Ranzato and colleagues 2014; Srivastava and colleagues 2015; Mathieu, Couprie and LeCun 2016). Predictions were blurry: with squared error, the best prediction of an uncertain future is the average of the possible futures.

Predicting representations (2018–2025). Contrastive predictive coding (van den Oord and colleagues, 2018) predicted future *latents* contrastively (chapter 15). LeCun’s position paper (2022) proposed the **joint-embedding predictive architecture** as a building block of world models: predict in representation space, so unpredictable details can be ignored. **I-JEPA** (Assran and colleagues, 2023) predicted target-block features of images from a context block; VideoMAE (Tong and colleagues, 2022) masked 90% of video tubes and reconstructed pixels; **V-JEPA** (Bardes and colleagues, 2024) predicted features of masked video tubes; **V-JEPA 2** (Assran and colleagues, 2025) scaled this to over a million hours of video and added an action-conditioned predictor for robot planning — this chapter’s paper bridge.

17.3 Assumptions and notation

Symbol	Meaning
$\mathbf{x}$	an image or clip; $\mathbf{x}_C$ the visible context, $\mathbf{x}_T$ the target region
$E_\theta, E_{\bar{\theta}}$	context encoder (trained) and target encoder (EMA of the context encoder)
P_ϕ	predictor
Δ_T	the positions (in space and time) of the target region
$\mathbf{h}_C, \mathbf{h}_T, \hat{\mathbf{h}}_T$	context features, target features, predicted target features
T, t, P	frames per clip, frames per tubelet, patch side
$N = (T/t)(H/P)(W/P)$	tokens per clip

Assumptions and declarations:

1. **Privileged targets.** During training the target encoder sees the *whole* image or clip, including what is hidden from the context encoder. That is supervision, not an inference input; a deployed system must not depend on it.

2. **Temporal labels need temporal evidence.** A label counts as “temporal” only if it cannot be predicted from any single frame or from frame statistics; this must be checked, not assumed.
3. **Collapse is possible**, exactly as in chapter 16, and must be monitored.

17.4 Mathematical formulation

$$\mathbf{h}_C = E_\theta(\mathbf{x}_C), \quad \mathbf{h}_T = \text{sg}(E_{\bar{\theta}}(\mathbf{x}))_T, \quad \hat{\mathbf{h}}_T = P_\phi(\mathbf{h}_C, \Delta_T),$$

$$\mathcal{L} = \frac{1}{|T|} \sum_{j \in T} \|\hat{\mathbf{h}}_j - \mathbf{h}_j\|_1 \quad (\text{V-JEPA; I-JEPA uses squared error}), \quad \bar{\theta} \leftarrow m\bar{\theta} + (1-m)\theta.$$

The target encoder encodes the full input and the features at the target positions are selected; the predictor receives the context features plus learnable mask tokens carrying the positional embeddings of Δ_T . After pretraining, the context encoder is kept and evaluated frozen (often with an attentive probe: a learned query cross-attending to all tokens).

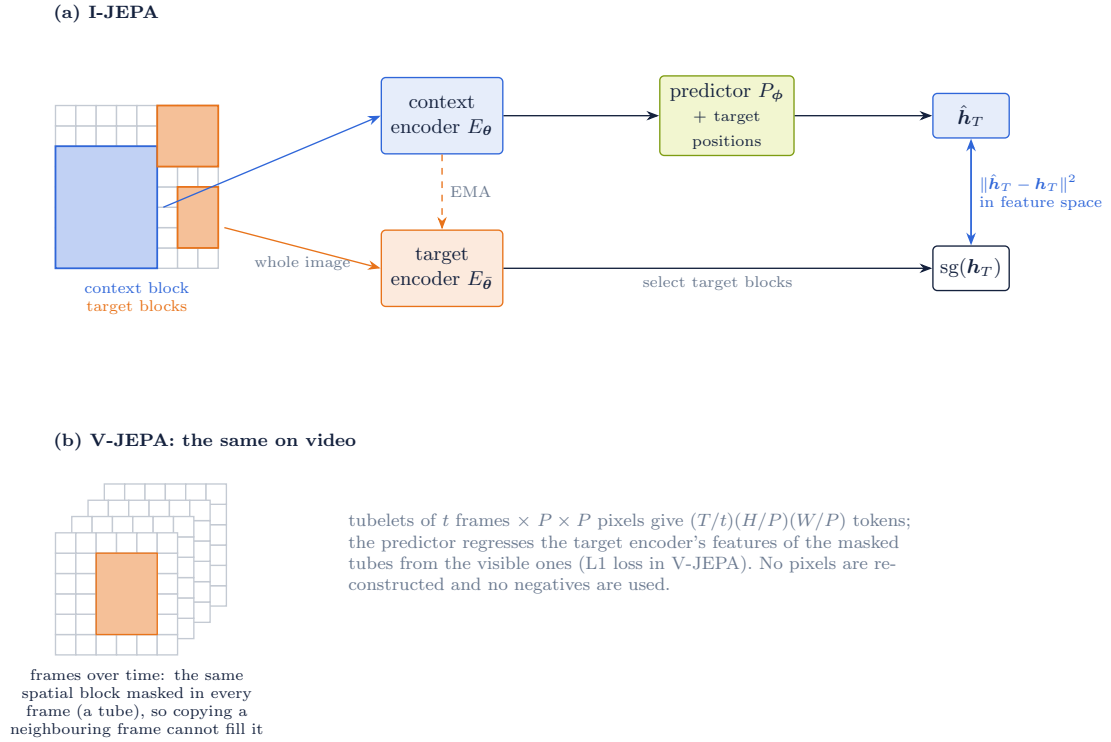


Figure 17.1: (a) I-JEPA, redrawn after Fig. 3 of Assran et al. (2023): a context block is encoded; a predictor predicts target-encoder features of several target blocks; the loss is in feature space. (b) V-JEPA (Bardes et al., 2024): the same with space–time tubes as targets. Papers: [arXiv:2301.08243](https://arxiv.org/abs/2301.08243), [arXiv:2404.08471](https://arxiv.org/abs/2404.08471).

17.5 Derivation, with reasons for each step

Step 1 — why pixel prediction blurs. Let the future frame $\mathbf{y}$ be uncertain given the past $\mathbf{x}$. The prediction minimising expected squared error is the conditional mean:

$$\arg \min_{\hat{\mathbf{y}}} \mathbb{E}[\|\mathbf{y} - \hat{\mathbf{y}}\|^2 \mid \mathbf{x}] = \mathbb{E}[\mathbf{y} \mid \mathbf{x}].$$

Reason: differentiate the expectation and set to zero (chapter 3). If a leaf may sway left or right, the mean frame shows a faint leaf in both places. Every unpredictable detail — exact texture, sensor noise, the precise flutter of a wing — must be averaged, and the model spends capacity on it. (With L_1 the optimum is the conditional median, which does not fix the problem.)

Step 2 — predicting in representation space. Replace the pixel target by an encoding $E(\mathbf{y})$. If the encoder discards what is unpredictable and keeps what is predictable (objects, positions, motion), the conditional mean of the representation is a meaningful target. The catch is that the encoder is learned too: the easiest way to make $E(\mathbf{y})$ predictable is to make it constant. This is chapter 16’s collapse again, and JEPAs use chapter 16’s remedies: a separate target encoder updated by EMA and a stop-gradient, plus an asymmetric predictor. In the lab, removing them collapses the features (spread 0.001, loss 0.001).

LeCun’s energy-based reading makes the trade explicit: the model defines a compatibility (energy) $\mathcal{E}(\mathbf{x}, \mathbf{y}) = \|P(E(\mathbf{x})) - E(\mathbf{y})\|$ that should be low for compatible pairs; collapse is the solution where energy is low *everywhere*. Contrastive methods push energy up on negatives; JEPAs rely on architecture and regularisation to keep low-energy regions small.

Step 3 — what the target encoder sees. The target features come from encoding the *whole* input, so they can depend on the context too; the predictor’s job is to infer the target’s representation from the context alone. This is legitimate supervision during training. It must be declared, because a downstream use that needs target-encoder features at inference would assume access to the very information being predicted.

Step 4 — mask design decides what is learned. The task must be solvable from context and unsolvable by local copying.

- In **images**, I-JEPA predicts several large target blocks (about 15–20% of the image each) from a context block covering most of the rest, with overlaps removed. Small random targets can be filled by interpolating neighbouring patches, which teaches texture; large blocks require inferring object parts.
- In **video**, adjacent frames are nearly identical, so masking random frames or random patches lets the model copy from the previous frame. **Tube masks** hide the same spatial region in every frame, and V-JEPA uses multi-block tube masks covering most of the clip. Video is even more redundant than images, which is why VideoMAE masks 90%.

The lab’s 1-D “video” shows the effect in miniature: masking a contiguous block of four frames gives better appearance (0.915 vs. 0.896) and motion (0.925 vs. 0.904) probes than masking four random frames, whose representations end up more similar between neighbouring frames (adjacent cosine 0.45 vs. 0.29).

Step 5 — tokens and cost. With tubelets of t frames and $P \times P$ patches, a clip of T frames at $H \times W$ has $N = (T/t)(H/P)(W/P)$ tokens. For $T = 16$, $t = 2$, 224×224 , $P = 16$: $8 \cdot 14 \cdot 14 = 1568$ tokens, and full self-attention compares $N^2 \approx 2.5$ million pairs per head per layer — eight times a single image’s 196 tokens, 64 times the cost. Remedies: encode only visible tokens (masking makes pretraining cheap, chapter 16), factorise attention into spatial and temporal parts, or lower frame rate and resolution. Record frame rate, stride, clip length and resolution: they define what motion the model can see.

Step 6 — pooling can erase time. Mean pooling $\bar{\mathbf{h}} = \frac{1}{T} \sum_t \mathbf{h}_t$ is invariant to permuting a fixed collection of feature vectors that have already been computed:

$$\frac{1}{T} \sum_t \mathbf{h}_{\pi(t)} = \frac{1}{T} \sum_t \mathbf{h}_t \quad \text{for every permutation } \pi.$$

Whether that erases time depends on how the $\mathbf{h}_t$ were computed. If the encoder processes each frame independently, as in this chapter’s lab, every $\mathbf{h}_t$ depends on one frame only, so the pooled vector is the same for a clip and for its reversal: such a probe cannot tell “opening” from “closing” except through something order-free in the data. A temporal encoder (a video transformer with temporal attention, or a 3-D convolution) is different: each $\mathbf{h}_t$ can already encode motion direction from neighbouring frames, reversing or shuffling the *input frames* changes the recomputed features, and their mean need not stay the same. A mean-pooled probe on such features can detect direction. Three operations should be kept apart:

- shuffling feature vectors *after* encoding, which mean pooling cannot see by construction;
- shuffling *input frames* and recomputing features, which tests what the whole encoder-plus-probe uses;
- attentive or concatenating probes with access to positions or to contextual temporal features, which can read order even from per-frame features.

In the lab’s unbalanced world, objects moving right start further left, so the average position reveals direction: a mean-pooled probe on random features “detects motion” with accuracy 0.777. In the balanced world (both directions cover the same span), mean pooling gives chance (0.49–0.50) for every encoder, while a probe on time-ordered features reaches 0.925 for the JEPA encoder. These results hold for this lab’s per-frame encoder and these synthetic clips; they show what mean pooling of per-frame features can and cannot do, not what it does for temporal video encoders. Evaluate temporal claims with order-aware probes (concatenation, attentive probes, temporal convolutions) **and** with a balanced dataset.

Step 7 — the shuffle control. Train the order-aware probe on ordered clips, then test it on clips whose input frames have been permuted (with a per-frame encoder, as here, this is the same as permuting the features). If accuracy stays high, the probe was not using order. In the lab every motion probe falls to chance on shuffled clips (0.43–0.48) — the test the evidence must pass. Publish the exact permutation, keep everything else fixed, and remember the limits: a small drop can also mean the task is appearance-driven or the perturbation is weak.

Step 8 — similarity statistics are ambiguous. Mean adjacent-frame cosine $C = \frac{1}{(T-1)} \sum_t \cos(\mathbf{h}_t, \mathbf{h}_{t+1})$ is high for static content, for temporally consistent features, for blurred features and for collapse. The collapsed lab encoder has the highest value of all (0.998). It is a description, not a score.

Step 9 — a collapse check that probes cannot fake. Linear probes standardise features; a collapsed encoder’s tiny residual variation can be scaled up and still classify (the lab’s collapsed encoder scores 0.868 on appearance). Add small noise to the features — a fixed absolute amount, sd 0.02 — before probing: healthy encoders barely change (0.893), the collapsed one falls to 0.155. Report feature spread and a noise-robust probe alongside accuracy.

Step 10 — conventions in the loss. “The prediction loss” is ambiguous without the norm and the reduction. L_1 summed over coordinates, L_1 averaged, and squared error summed differ by factors that depend on the feature dimension; L_1 ’s gradient is the sign of the residual, squared error’s is the residual. Match the paper exactly when reproducing it, and report which you used.

17.6 Numerical worked example

Loss conventions. Target $(1, -1)$, prediction $(0, 2)$: summed $L_1 = 1 + 3 = 4$; mean $L_1 = 2$; summed squared error $= 1 + 9 = 10$. The L_1 gradient is $(\text{sign}(-1), \text{sign}(3)) = (-1, +1)$ per coordinate; the squared-error gradient is $2(-1, 3) = (-2, 6)$.

Token count. $T = 16, t = 2, H = W = 224, P = 16$: 1568 tokens; at 90% masking the encoder sees about 157.

Pooling invariance. Two frames with features $(1, 0)$ then $(0, 1)$ (“moving right”) and $(0, 1)$ then $(1, 0)$ (“moving left”) both average to $(0.5, 0.5)$; concatenation gives $(1, 0, 0, 1)$ and $(0, 1, 1, 0)$, which a linear probe separates.

EMA target. With $m = 0.99$ the target encoder averages roughly the last 100 context-encoder states (chapter 16, step 1); V-JEPA anneals m toward 1 during training.

17.7 Small implementation and checks

The lab `labs/ch17-jepa/jepa.py` builds a JEPA in miniature. Each clip has 8 frames of 32 pixels. A per-frame MLP encodes frames; the predictor receives the encoded visible frames (zeros where masked), a mask indicator and a one-hot target index, and predicts that frame’s EMA-target features:

```
Hc = encode(p, X.reshape(-1, L)).reshape(m, T, D)           # context
→ encoder on every frame
ctx = Hc * (~M)[..., None]                                   # hide masked
→ frames
inp = np.c_[ctx.reshape(m, -1), M, onehot(t_target)]         # what the
→ predictor may use
pred = predictor(p, inp)
target = encode(tgt, X[np.arange(m), t_target])              # EMA target
→ encoder, no gradient
loss = np.mean(np.abs(pred - target)); G = np.sign(pred - target) /
→ pred.size
```

Checks, in `test_jepa.py`: token counting and loss conventions; the unbalanced world leaks direction to a mean-pooled probe and the balanced one does not; adjacent cosine is high for static content; block masks are contiguous; a gradient check of the predictor and encoder; and the JEPA encoder supports an order-aware motion probe that fails on shuffled clips, while the variant without EMA and stop-gradient collapses and fails the noise-robust probe.

17.8 Experiment and expected observations

Run `python labs/ch17-jepa/jepa.py` (seed 0; 1,500 Adam steps per method; frozen probes on 1,000 + 1,000 fresh clips; chance is 0.167 for appearance and 0.5 for motion).

encoder	appearance (noisy)	motion	motion, shuffled	mean- pooled motion	adjacent cosine	feature spread
raw pixels	0.605 (0.615)	0.569	0.454	0.490	0.262	—
random encoder	0.885 (0.870)	0.788	0.476	0.494	0.368	—
JEPA, block masking	0.915 (0.893)	0.925	0.434	0.504	0.292	0.219
JEPA, random- frame masking	0.896 (0.864)	0.904	0.444	0.485	0.453	0.204
pixel prediction, block masking	0.844 (0.832)	0.922	0.425	0.492	0.526	0.308
JEPA without EMA / stop- gradient	0.868 (0.155)	0.895	0.442	0.485	0.998	0.001

Unbalanced world (direction leaks through position), random encoder: mean-pooled motion 0.777.

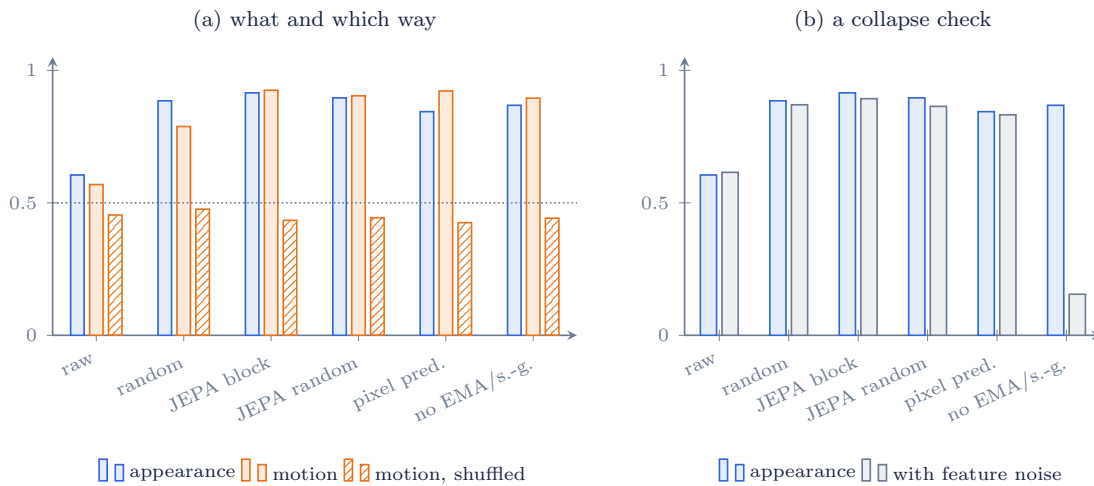


Figure 17.2: Frozen probes on the synthetic video world. (a) Appearance and motion; hatched bars test the motion probe on frame-shuffled clips, which drop to chance for every encoder. (b) The appearance probe with and without a little feature noise: only the collapsed encoder fails.

What to notice:

1. **Latent prediction with block masks gives the best features** for both questions

here; pixel prediction learns motion as well but appearance worse, because its encoder must also carry the background texture needed to paint pixels.

2. **Random-frame masking is easier and teaches less:** the masked frame can be inferred from its neighbours.
3. **Order matters only when the data and probe let it matter.** Mean pooling is at chance in the balanced world; in the unbalanced one it “detects motion” through position. Construct the control before trusting the number.
4. **The collapsed encoder looks fine** on the plain probes and has the most “temporally consistent” features. Feature spread and the noise-robust probe expose it.
5. **Random features are a strong baseline** (appearance 0.885). Always report one.

17.9 Moving pieces: diagnosing and fixing

Moving piece	Symptom when it is wrong	How to confirm	Fix
Mask geometry	features good at texture, poor at objects or motion	compare block/tube vs. random masks	large blocks in images; spatio-temporal tubes in video
Mask ratio	trivial task (low) or no signal (high)	probe accuracy vs. ratio	~75% images, ~90% video; tune on validation
Target encoder / EMA	collapse: tiny spread, near-zero loss	feature spread, noise-robust probe	EMA with stop-gradient; momentum schedule toward 1
Prediction space	blurry or texture-heavy targets (pixels)	compare latent vs. pixel targets on probes	predict features; normalise targets
Clip sampling	motion too slow or too fast to see	vary frame stride	document fps, stride, clip length; match to the task
Pooling in evaluation	temporal claims from order-invariant probes	permutation test of the probe	attentive or order-aware probes
Label shortcuts	“motion” solved by single frames or positions	single-frame and mean-pooled baselines	balance datasets; report static baselines
Token budget	out of memory, slow training	count tokens N and N^2	lower fps/resolution, tubelets, factorised attention, encode visible only
Privileged information	method needs future frames at test time	list inputs at inference	keep target-encoder use to training
Loss convention	results not comparable	norm and reduction in the code	state L_1/L_2 , sum/mean, target normalisation

First things to try: (1) a single-frame baseline and a shuffled-frame test for every

temporal claim; (2) log feature spread during pretraining; (3) count tokens before choosing clip length; (4) compare against random features.

17.10 Application, limitations and failure cases

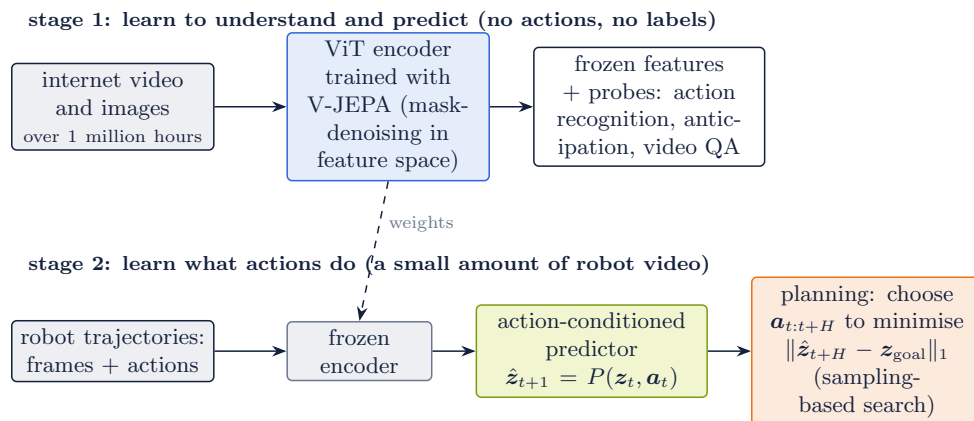
Application (educational case study). For insect-behaviour videos from field traps, extract frozen features from a public V-JEPA checkpoint and train two attentive probes: species (appearance) and behaviour (approaching versus leaving the crop). Build the evaluation so that behaviour cannot be read from a single frame — balance where insects appear and how they are oriented — and report single-frame and shuffled-frame baselines next to the video result. Split by trap and date (chapter 5). If behaviour accuracy barely changes under shuffling, the model is recognising appearance, not motion.

Limitations:

- Feature prediction avoids modelling unpredictable detail, but nothing guarantees the retained features are the ones a task needs; probes and controls are still required.
- Large-scale video pretraining is expensive and results depend on data mixtures, masks and clip sampling that are hard to reproduce.
- Accurate latent prediction is not evidence of physical understanding: the representation may encode only what makes prediction easy.
- Synthetic one-dimensional video illustrates mechanisms; real video adds camera motion, occlusion and long-range dependencies.

17.11 Bridge to a research paper

Assran, Bardes, Fan and colleagues (2025), *V-JEPA 2: self-supervised video models enable understanding, prediction and planning* ([arXiv:2506.09985](https://arxiv.org/abs/2506.09985)).



each stage is a separate system with its own evaluation: good frozen features (stage 1) do not by themselves make a planner; the planner's success depends on the predictor, the goal specification and the search (chapter 23).

Figure 17.3: The V-JEPA 2 pipeline, redrawn after Fig. 1 of Assran et al. (2025): action-free JEPA pretraining on internet-scale video, then an action-conditioned predictor trained on a small amount of robot video and used for planning toward a goal image.

- **Problem:** can a model learn to understand, predict and act in the physical world mostly from observation — video without labels — plus a small amount of interaction data?

- **Mechanism:** stage 1 scales V-JEPA pretraining (masked feature prediction on video tubes, EMA target encoder) to over a million hours of video and large ViT encoders, with progressive increases in resolution and clip length; frozen features are evaluated with attentive probes and aligned with a language model for video question answering. Stage 2 freezes the encoder and trains an action-conditioned predictor (V-JEPA 2-AC) on less than 62 hours of unlabelled robot trajectories; planning searches for actions whose predicted future embedding approaches the embedding of a goal image (model-predictive control in latent space).
- **Central claims:** strong results on motion-centred understanding (Something-Something v2) and action anticipation (Epic-Kitchens-100), competitive video question answering after alignment with a language model, and zero-shot pick-and-place with robot arms in new environments, planning from goal images.
- **Assumptions to question:** which gains come from data scale versus the objective; whether the probe protocol (attentive probes) favours some encoders; how much the planning success depends on the goal specification, the search and the task set rather than on the representation (chapter 23); and which evaluation results are measured on motion-sensitive subsets.
- **Affordable experiment:** extend the lab’s world with a third label, *speed* (one or two pixels per frame), and an action-conditioned predictor: given the encoded current frame and a “push” action (shift by -1 , 0 or $+1$), predict the next frame’s target features. Plan a sequence of three actions that brings the object’s predicted embedding closest to a goal frame’s embedding, by exhaustive search over the 27 sequences. Measure how often the plan’s real outcome matches the goal, and how this changes when the encoder is random, collapsed, or trained.

17.12 Memory map

Summary. A JEPA predicts the representation of hidden parts of an image or video from the visible parts, with a target encoder that is an EMA of the context encoder behind a stop-gradient. Predicting features rather than pixels avoids averaging over unpredictable detail, at the cost of collapse risk handled as in chapter 16. Masks decide what is learned: large blocks and space–time tubes defeat copying from neighbours. Video multiplies tokens and cost. Temporal understanding must be shown with order-aware probes, balanced labels, single-frame baselines and shuffled-frame controls; mean pooling of per-frame features erases order, similarity statistics are ambiguous, and a noise-robust probe exposes collapsed features that ordinary probes can rescue. V-JEPA 2 extends the idea from representations to prediction and planning, each of which needs its own evaluation.

17.13 Exercises

1. *Conceptual.* Why should a target encoder’s access to the full video during training be declared?
2. *Conceptual.* With $T = 16$, $t = 2$, $H = W = 224$, $P = 16$, how many tubelet tokens are there?
3. *Conceptual.* Does a higher adjacent-frame cosine prove better motion understanding?
4. *Derivation.* Show that the squared-error-optimal prediction is the conditional mean and the L_1 -optimal one the conditional median. What does each predict when the next frame is one of two equally likely images?
5. *Derivation.* Prove that any probe on time-averaged features is invariant to frame permutations, and give a pooling that is not.

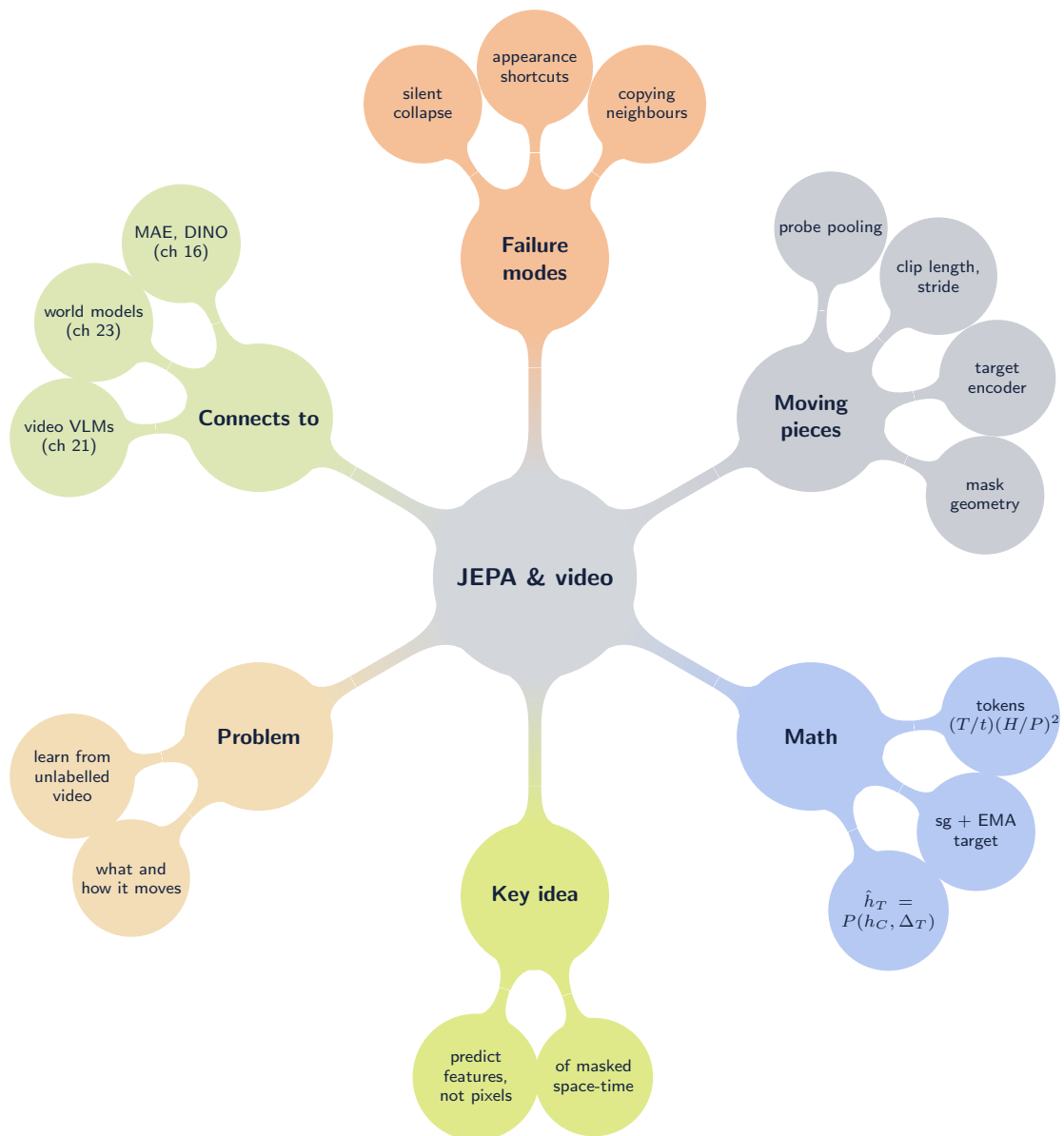


Figure 17.4: Memory map of chapter 17.

6. *Derivation.* Estimate the attention FLOPs of one self-attention layer for $N = 1568$ tokens and width 1024, and compare with a 196-token image.
7. *Implementation.* Add a “static” control to the lab — clips whose object does not move — and check the motion probe’s predictions on them.
8. *Implementation.* Carry out the experiment of section 17.11.
9. *Interpretation.* A video model reports high accuracy on an action dataset; its accuracy falls by only two points when frames are shuffled. List three explanations and how to distinguish them.
10. *Open question.* What would a JEPA need in order to represent uncertainty about the future, rather than a single predicted embedding?

17.13.1 Answer discussions

1. It is privileged supervision; declaring it prevents a deployed system from silently relying on information (future frames) that will not be available.
2. $8 \cdot 14 \cdot 14 = 1568$.
3. No: static content, blurred features and collapse also raise it; compare controls and task performance.
4. Differentiate the expected loss; the median of a two-point distribution is any value between the points (with ties), the mean is their average — a blend of the two images.
5. A permutation only reorders the terms of the sum. Concatenation, positional embeddings followed by attention, or temporal convolution are order-sensitive.
6. Scores and weighted sums cost about $2N^2d$ each: $2 \cdot 1568^2 \cdot 1024 \approx 5 \times 10^9$ multiply-adds per product, about 64 times the 196-token case.
7. A good motion probe should be uncertain (near 0.5) on static clips; confident predictions reveal reliance on other cues.
8. Expect planning to succeed far more often with the trained encoder than with random or collapsed ones, and to depend on whether the encoder represents position.
9. The task is appearance-driven; the probe pools away order; the shuffle preserves enough local order (e.g. only distant frames swapped). Test with single-frame baselines, order-sensitive probes and stronger permutations.
10. A distribution over embeddings (several hypotheses, a latent variable as in chapters 8–10, or an energy model sampled at inference), and an evaluation that rewards calibrated uncertainty.

17.14 Self-check and common mistakes

- ☐ I can write the JEPA equations and say what each encoder sees.
- ☐ I can explain why feature prediction avoids blur and what it risks.
- ☐ I can design image and video masks and count tokens.
- ☐ I can design a temporal evaluation with the right controls.
- ☐ I can separate representation, prediction and planning claims.

Common mistakes: calling latent prediction “generation”; masking random frames in video; mean-pooling per-frame features and claiming temporal understanding; reading adjacent-frame similarity as quality; trusting probe accuracy on possibly collapsed features; leaving positional or appearance shortcuts in “motion” labels; omitting fps, stride and clip length; comparing losses with different norms or reductions; treating a good frozen encoder as a planner.

References

- I. Goodfellow, Y. Bengio and A. Courville. *Deep Learning*. MIT Press, 2016, chapter 15 (representation learning) and section 20.10 (generative models of images).
- R. P. N. Rao and D. H. Ballard. Predictive coding in the visual cortex. *Nature Neuroscience*, 2(1):79–87, 1999.
- L. Wiskott and T. J. Sejnowski. Slow feature analysis. *Neural Computation*, 14(4):715–770, 2002.
- J. Carreira and A. Zisserman. Quo vadis, action recognition? A new model and the Kinetics dataset. In *CVPR*, 2017.
- M. Mathieu, C. Couprie and Y. LeCun. Deep multi-scale video prediction beyond mean square error. In *ICLR*, 2016.
- G. Bertasius, H. Wang and L. Torresani. Is space-time attention all you need for video understanding? In *ICML*, 2021.
- Y. LeCun. A path towards autonomous machine intelligence. OpenReview, 2022.
- Z. Tong, Y. Song, J. Wang and L. Wang. VideoMAE: masked autoencoders are data-efficient learners for self-supervised video pre-training. In *NeurIPS*, 2022.
- M. Assran et al. Self-supervised learning from images with a joint-embedding predictive architecture. In *CVPR*, 2023. arXiv:2301.08243.
- A. Bardes et al. Revisiting feature prediction for learning visual representations from video. *TMLR*, 2024. arXiv:2404.08471.
- M. Assran et al. V-JEPA 2: self-supervised video models enable understanding, prediction and planning. arXiv:2506.09985, 2025.

Chapter 18

Transfer and representation evaluation

Prerequisites: chapter 5 (risk, splits, calibration, bootstrap); chapters 15–17 (pretrained encoders); chapter 14 (tokens).

Learning outcomes: after this chapter you can

1. distinguish k -NN, linear probes, attentive probes and fine-tuning, and state the question each answers;
2. design a controlled transfer comparison and predict how the best method changes with the label budget;
3. measure calibration with NLL, Brier score and ECE, fix it with temperature scaling, and explain why calibration does not survive distribution shift;
4. compute uncertainty with the right resampling unit and report error slices;
5. read representation diagnostics (nearest neighbours, spectra, 2-D projections) without over-claiming.

18.1 A concrete problem

The crop-health team has five new diseases to recognise and only a handful of labelled images of each, from eight farms. A pretrained encoder exists — trained on ten *other* diseases, or self-supervised on unlabelled images (chapters 15–17). Next season, the model will be used on four *new* farms with different cameras and lighting.

Four decisions follow, and each needs evidence:

1. Freeze the encoder and train a small head, or fine-tune everything?
2. How should the token features be pooled?
3. How confident should the model be, and will that confidence still be right on new farms?
4. How uncertain is the accuracy estimate itself, when the test set has 1,500 images but only four farms?

This chapter treats evaluation as part of the method. It defines the protocols for using pretrained representations, shows how their ranking changes with the label budget and under shift, and turns chapter 5’s calibration and resampling tools into a reporting discipline for representation learning.

18.2 How we got here

Transfer by features (2013–2014). DeCAF (Donahue and colleagues, 2014) and Razavian and colleagues (2014) showed that features from an ImageNet-trained CNN, fed to a linear SVM, beat hand-designed features on many tasks — “CNN features off-the-shelf”. Yosinski and colleagues (2014) measured how transferable each layer is: early layers are general, later ones specific.

Fine-tuning becomes the default (2014–2019). Fine-tuning ImageNet-pretrained networks became standard for detection (R-CNN, chapter 13) and segmentation. Kornblith, Shlens and Le (2019) asked whether better ImageNet models transfer better and found that the answer depends on regularisation choices and the protocol (fixed features versus fine-tuning). He, Girshick and Dollár (2019) showed that with enough target data and training, pretraining mainly speeds up convergence.

Probes for self-supervised learning (2016–2021). Linear probes became the standard way to compare self-supervised representations (chapters 15–16), followed by k -NN evaluation (Wu and colleagues, 2018; DINO) and attentive probes for token-based encoders (V-JEPA, chapter 17). Each protocol rewards different properties.

Robustness and shift (2019–2022). ImageNet-C (Hendrycks and Dietterich, 2019) measured common corruptions; ImageNet-v2 (Recht and colleagues, 2019) showed accuracy drops on a freshly collected test set; WILDS (Koh and colleagues, 2021) collected real shifts (hospitals, cameras, countries). Guo and colleagues (2017) showed that modern networks are over-confident and that temperature scaling greatly reduces this in-distribution; Ovadia and colleagues (2019) showed that calibration degrades under shift. Kumar and colleagues (2022) showed that fine-tuning can distort good pretrained features and hurt out-of-distribution accuracy, and proposed linear probing then fine-tuning — this chapter’s paper bridge.

18.3 Assumptions and notation

Symbol	Meaning
E	pretrained encoder; $\mathbf{h}_1, \dots, \mathbf{h}_N$ its token features for one image
$\bar{\mathbf{h}} = \frac{1}{N} \sum_j \mathbf{h}_j$	mean-pooled feature
$\mathbf{q}, w_j = \text{softmax}_j(\mathbf{q}^\top \mathbf{h}_j / \sqrt{d})$	attentive probe’s learned query and token weights
$\mathbf{p}, \mathbf{y}$	predicted class probabilities and one-hot target
T	softmax temperature (logits divided by T)
NLL, BS, ECE	negative log-likelihood, Brier score, expected calibration error
k	labelled examples per class (the label budget)

Assumptions:

1. **The comparison is controlled:** same split, resolution, augmentation, checkpoint, layer, feature normalisation, probe family and tuning budget for every method compared.

2. **The test distribution is stated:** “new images from the same farms” and “new farms” are different questions.
3. **The independent unit is known** (image, plant, farm, patient, video), and resampling uses it.

18.4 Mathematical formulation

Protocols. With frozen E :

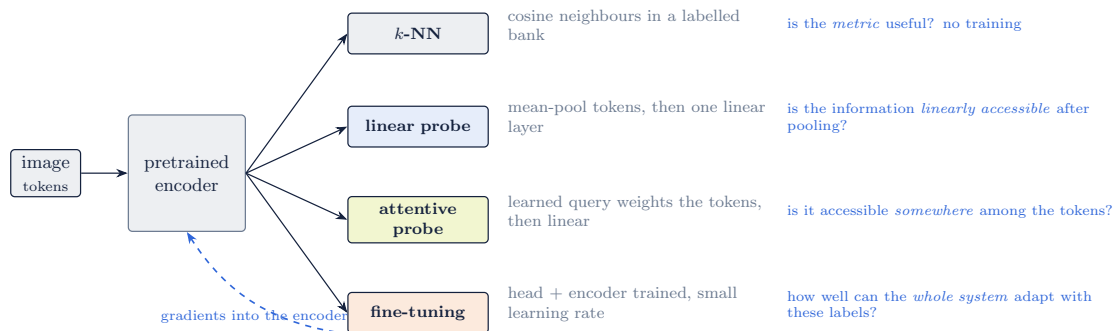
linear probe: $\hat{\mathbf{p}} = \text{softmax}(\mathbf{W}^\top \bar{\mathbf{h}} + \mathbf{b})$, attentive probe: $\hat{\mathbf{p}} = \text{softmax}\left(\mathbf{W}^\top \sum_j w_j \mathbf{h}_j + \mathbf{b}\right)$,

k -NN: vote among the k most cosine-similar labelled examples. **Fine-tuning** trains $\mathbf{W}$, $\mathbf{b}$ (and $\mathbf{q}$) together with E , usually with a smaller learning rate for E .

Proper scores and calibration for m test examples and K classes:

$$\text{NLL} = -\frac{1}{m} \sum_i \log p_{i, y_i}, \quad \text{BS} = \frac{1}{m} \sum_i \sum_k (p_{ik} - y_{ik})^2, \quad \text{ECE} = \sum_b \frac{|S_b|}{m} |\text{acc}(S_b) - \text{conf}(S_b)|,$$

with S_b the examples whose top probability falls in bin b . **Temperature scaling:** $\hat{\mathbf{p}}_T = \text{softmax}(\mathbf{a}/T)$, with $T > 0$ fitted by minimising NLL on held-out validation data.



fix for every method: split, resolution, augmentation, layer, normalisation, probe family and tuning budget. Each score answers its own question; none measures “representation quality” in general.

Figure 18.1: Four ways to use a pretrained encoder and the question each answers. Only fine-tuning sends gradients into the encoder.

18.5 Derivation, with reasons for each step

Step 1 — each protocol measures something different. A k -NN classifier has no trainable parameters: it measures whether the encoder’s *metric* places same-class images close together. A linear probe measures whether the label is *linearly* recoverable from pooled features. An attentive probe adds a learned pooling: is the information present *somewhere* in the tokens? Fine-tuning measures how well the *whole system* adapts with the given labels and compute. A representation can rank first on one and last on another; MAE-style encoders (chapter 16) are the classic example, weak under linear probes and

strong after fine-tuning. Report the protocol with the number, and do not call any one of them “representation quality”.

Step 2 — pooling can dilute the evidence. If only one of N tokens carries the object, mean pooling scales its contribution by $1/N$ and mixes it with $N - 1$ background tokens: the signal-to-noise ratio of $\bar{\mathbf{h}}$ falls roughly like $1/\sqrt{N}$ when background tokens vary independently. An attentive probe can weight the object token: the worked example’s tokens $(2, 0)$ and $(0, 2)$ average to $(1, 1)$, while weights $(0.9, 0.1)$ give $(1.8, 0.2)$. In the lab, the object occupies one of eight tokens; the mean-pooled probe reaches 0.655 with 50 labels per class while the attentive probe reaches 0.907. The attention weights do not show *why* a token was chosen, nor that the choice transfers.

Step 3 — the label budget decides the winner. Write the target error as approximation plus estimation error (chapter 5). A frozen encoder plus a small head has few parameters to estimate — low estimation error — but cannot change the features — approximation error fixed by pretraining. Fine-tuning and training from scratch can reduce approximation error but have many parameters to estimate. Hence:

- with very few labels, **frozen features with a small head** win (lab, 2 per class: attentive probe 0.656, fine-tune 0.505, scratch 0.438);
- as labels grow, **fine-tuning** overtakes (10 per class: 0.858 vs. 0.851; 50: 0.919 vs. 0.907);
- with many labels, **scratch** catches up (50 per class: 0.897), as He and colleagues (2019) found at scale.

The crossover points depend on how related source and target are; measure them, as a learning curve, rather than reporting one budget.

Step 4 — why fine-tuning can hurt out of distribution. Fine-tuning moves the encoder to fit the training distribution. If the head starts random, its large early gradients flow into the encoder and change features in directions that matter only in-distribution, while features useful for unseen domains are distorted (Kumar and colleagues, 2022). Training the head first (linear probe) and then fine-tuning everything with a small learning rate (LP-FT) reduces this distortion. Whether fine-tuning helps or hurts under shift is an empirical question about the features and the shift; in the lab’s world it helped (50 labels: 0.662 on unseen farms vs. 0.587 for the attentive probe), and section 18.11 asks you to test LP-FT.

Step 5 — proper scores. The NLL and the Brier score are *proper*: in expectation, they are minimised by reporting the true conditional probabilities. Accuracy is not — it ignores confidence. The Brier score is bounded (between 0 and 2 when summed over classes), so it is less dominated by a few confident errors than the NLL. Conventions differ by a factor of two between summing over both classes and scoring only the positive class in binary problems; state which.

Step 6 — temperature scaling preserves accuracy. For $T > 0$, dividing all logits by T is a monotone transformation of each logit, so $\arg \max_k a_k/T = \arg \max_k a_k$ and accuracy is unchanged. $T > 1$ softens over-confident predictions, $T < 1$ sharpens. Fitting T by validation NLL is a one-parameter maximum-likelihood problem (chapter 6); the lab fits $T = 1.84$, lowering in-domain ECE from 0.033 to 0.022 and NLL from 0.258 to 0.234 with accuracy unchanged.

Step 7 — calibration does not transfer. Temperature scaling fits one number on a finite validation set. It can reduce miscalibration substantially, but it cannot guarantee calibration even on the distribution it was fitted on: a single scale factor cannot fix errors

that differ between classes or confidence levels, and the fitted value has sampling error. Under a shift it can be far off. Under shift, accuracy falls but the model's confidence often does not, so it becomes over-confident. In the lab, on unseen farms ECE is 0.313 before and still 0.213 after the in-domain temperature, and NLL is $2.00 \rightarrow 1.28$. Report calibration separately for each test distribution, and if shifted validation data exist, recalibrate on them.

Step 8 — resampling at the level of the independent unit. The bootstrap estimates uncertainty by resampling the units that were sampled independently (chapter 5). The 1,500 shifted test images come from only **four** farms. If farms differ, resampling images treats them as 1,500 independent draws and gives a narrow interval; resampling farms reflects the real variability. In the lab: $[0.553, 0.602]$ by images versus $[0.304, 0.808]$ by farms. The second interval says what four farms can tell us — not much. The same applies to frames within videos and slices within patients.

Step 9 — error slices and diagnostics. Averages hide failures: report accuracy and calibration per farm, per class, per object size, per camera — slices defined before looking at results. Diagnostics such as nearest neighbours, feature norms, covariance spectra and 2-D projections (t-SNE, UMAP) help find problems, but projections distort distances and depend on hyperparameters: a well-separated plot is not evidence of a good representation, and an overlapping one is not evidence of a bad one. Held-out numbers decide.

18.6 Numerical worked example

Brier score. True class 1, prediction (0.6, 0.4): $(0.6 - 1)^2 + (0.4 - 0)^2 = 0.32$; the prediction is correct but the score is not zero. Prediction (1, 0): 0. Prediction (0.4, 0.6) (wrong): $0.36 + 0.36 = 0.72$.

Temperature. Logits (2, 0): $T = 1$ gives $p_1 = e^2 / (e^2 + 1) = 0.881$; $T = 2$ gives $e / (e + 1) = 0.731$; the top class is unchanged.

ECE with two bins. Ten predictions with confidence 0.9, of which 6 correct; ten with confidence 0.6, of which 6 correct: $\text{ECE} = \frac{1}{2}|0.6 - 0.9| + \frac{1}{2}|0.6 - 0.6| = 0.15$.

Bootstrap units. Four farms with accuracies 0.30, 0.50, 0.70, 0.90 and 250 images each: overall 0.60. Resampling images gives a standard error near $\sqrt{0.6 \cdot 0.4 / 1000} = 0.015$; resampling farms gives one near the standard deviation of farm accuracies divided by 2, about $0.22 / 2 = 0.11$ — seven times larger.

18.7 Small implementation and checks

The lab `labs/ch18-transfer/transfer.py` pretrains a token encoder on ten source classes and transfers it to five new ones. The attentive head and its gradient:

```
s = H @ q / np.sqrt(d); w = softmax(s, axis=1)           # one weight per
→ token
z = np.einsum("mj,mjd->md", w, H)                       # weighted
→ pooling
logits = z @ Wc + bc
# backward: dz = dlogits @ Wc.T; dH = w * dz + (softmax Jacobian) * q
Gw = np.einsum("mjd,md->mj", H, dz); Gs = w * (Gw - np.sum(Gw * w, 1,
→ keepdims=True)) / np.sqrt(d)
```

Checks, in `test_transfer.py`: the Brier and pooling examples; a gradient check of the attentive head and encoder; temperature preserves the argmax; the farm bootstrap is wider than the image bootstrap when farms differ; and a pretrained attentive probe beats training from scratch with three labels per class.

18.8 Experiment and expected observations

Run `python labs/ch18-transfer/transfer.py` (seed 0; test sets of 1,500 new images from the eight training farms and 1,500 from four unseen, more shifted farms; chance 0.2).

labels per class	scratch	linear probe (mean pool)	attentive probe	fine-tune
2 — training farms / unseen farms	0.438 / 0.286	0.392 / 0.238	0.656 / 0.404	0.505 / 0.355
10	0.781 / 0.513	0.486 / 0.233	0.851 / 0.542	0.858 / 0.583
50	0.897 / 0.625	0.655 / 0.314	0.907 / 0.587	0.919 / 0.662

Fine-tuned model, 50 labels per class, temperature fitted on in-domain validation ($T = 1.84$):

test set	accuracy	NLL ($T = 1 \rightarrow$ fitted)	Brier	ECE	95% CI, images	95% CI, farms
training farms	0.917	0.258 $\rightarrow$ 0.234	0.130 $\rightarrow$ 0.127	0.033 $\rightarrow$ 0.022	[0.902, 0.931]	[0.911, 0.922]
unseen farms	0.577	1.998 $\rightarrow$ 1.280	0.698 $\rightarrow$ 0.592	0.313 $\rightarrow$ 0.213	[0.553, 0.602]	[0.304, 0.808]

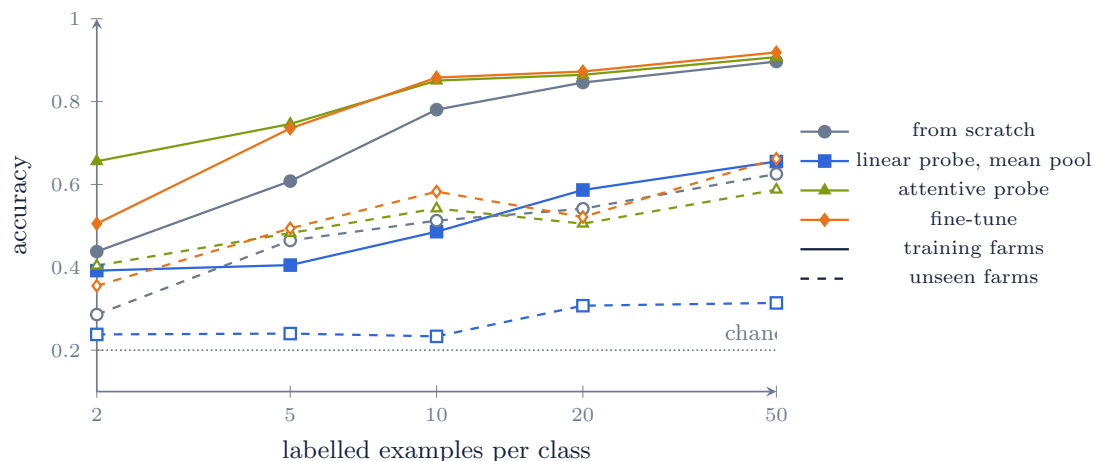


Figure 18.2: Accuracy against labels per class for four ways of using a pretrained encoder, on new images from the training farms (solid) and unseen farms (dashed). The best method changes with the label budget, and every method loses accuracy on unseen farms.

What to notice:

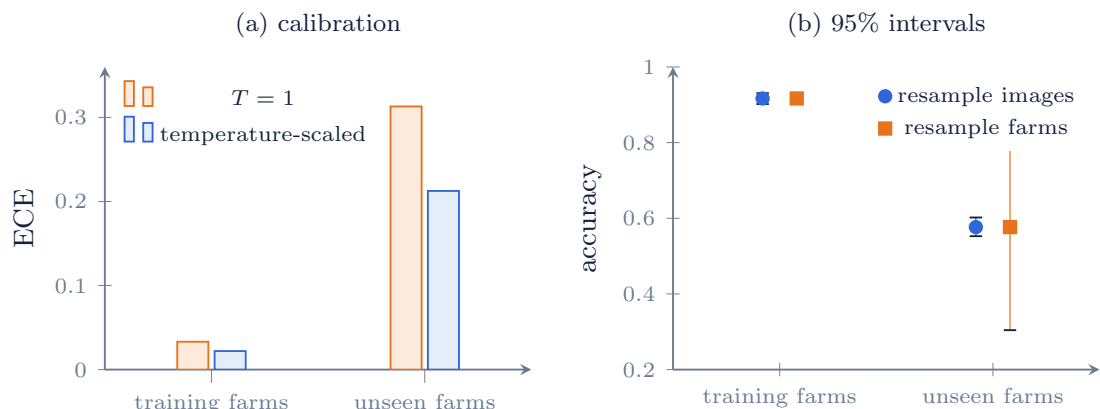


Figure 18.3: (a) Calibration error before and after temperature scaling: much reduced in-domain, much worse on unseen farms even after scaling. (b) 95% intervals for accuracy: resampling farms instead of images widens the unseen-farm interval nearly tenfold.

1. **No single winner.** Frozen features with an attentive head win at 2 labels; fine-tuning wins from 10; scratch approaches both at 50.
2. **Pooling is part of the evaluation.** The same frozen features score 0.655 or 0.907 depending only on pooling.
3. **Shift costs a third of the accuracy** for every method; the in-domain test set says nothing about it.
4. **Temperature scaling helps where it was fitted.** Unseen farms remain badly over-confident.
5. **Four farms are four data points.** The honest interval for unseen-farm accuracy spans 0.30–0.81.

18.9 Moving pieces: diagnosing and fixing

Moving piece	Symptom when it is wrong	How to confirm	Fix
Protocol	rankings flip between papers	list protocol, layer, pooling, budget	report several protocols; name the question
Pooling	frozen features look weak	attentive vs. mean probe	attentive or multi-query probes; CLS vs. patch tokens
Label budget	conclusions from one budget	learning curve over k	report curves; choose method per budget
Fine-tuning recipe	OOD accuracy drops after fine-tuning	compare LP, FT, LP-FT on shifted data	LP-FT; smaller encoder learning rate; layer-wise decay; early stopping
Hyperparameter tuning	method with more tuning wins	equal budgets and validation protocol	same search budget for every method

Moving piece	Symptom when it is wrong	How to confirm	Fix
Calibration	over-confidence, especially under shift	reliability diagram, ECE, NLL per test set	temperature scaling per distribution; ensembles
Resampling unit	intervals too narrow	count independent units	bootstrap farms/patients/videos
Error slices	average hides a failing subgroup	pre-registered slices	report slices; fix data for the weakest
Visual diagnostics	conclusions from t-SNE plots	compare with held-out metrics	treat plots as hypotheses, not evidence
Leakage	in-domain test too easy	check overlap of farms/plants between train and test	grouped splits (chapter 5)

First things to try: (1) a learning curve over label budgets for each protocol; (2) a separate test set from new farms (or cameras, sites); (3) reliability diagrams per test set; (4) bootstrap by the independent unit.

18.10 Application, limitations and failure cases

Application (educational case study). For the five new diseases, collect 10–20 labelled images per class from as many farms as possible (farm diversity matters more than images per farm). Compare an attentive probe on frozen DINOv2 features, LP-FT fine-tuning and a supervised ImageNet backbone, at several label budgets, with a test set from farms not used for training. Fit temperature on a validation set from the new farms if one can be labelled; otherwise report that calibration is unverified for them. Report per-farm accuracy with farm-level bootstrap intervals. A deployment decision should rest on the worst farms, not the average.

Limitations:

- Synthetic farm shifts (offsets and gains) are far simpler than real changes in camera, crop variety, season and disease presentation.
- Probes with more capacity measure something different from linear probes; comparisons must keep capacity fixed.
- A few target domains give little information about the next one; uncertainty about deployment remains large.
- Temperature scaling corrects only a global miscalibration; class- or region-specific errors need more.

18.11 Bridge to a research paper

Kumar, Raghunathan, Jones, Ma and Liang (2022), *Fine-tuning can distort pretrained features and underperform out-of-distribution* (ICLR 2022; [arXiv:2202.10054](https://arxiv.org/abs/2202.10054)).

- **Problem:** fine-tuning all layers usually gives the best in-distribution accuracy, but practitioners observed that linear probes sometimes generalise better out of distribution. Why, and can one get both?

- **Mechanism:** a theoretical analysis of two-layer linear networks shows that with a randomly initialised head, fine-tuning changes the features mostly in directions spanned by the training data, distorting features along the directions that matter for shifted data; the proposed **LP-FT** first trains the head with frozen features (linear probe), then fine-tunes everything starting from that head.
- **Central claim:** across ten distribution-shift datasets (including WILDS and ImageNet variants), LP-FT improves out-of-distribution accuracy over fine-tuning while matching or improving in-distribution accuracy.
- **Assumptions to question:** the theory is for linear models; the size of the effect depends on how good the pretrained features are for the shifted domain and on the fine-tuning learning rate and length; in the lab’s world plain fine-tuning already helped out of distribution.
- **Affordable experiment:** add LP-FT to the lab (train the attentive head on frozen features for 300 steps, then fine-tune everything with encoder learning rate 5×10^{-4}) and compare FT, LP and LP-FT at 10 and 50 labels per class on both test sets, with farm-level bootstrap intervals. Then make the shifted farms’ offsets larger: does the advantage of LP or LP-FT grow with the size of the shift?

18.12 Memory map

Summary. A pretrained representation can be used through k -NN, a linear probe, an attentive probe or fine-tuning, and each answers a different question. Which is best depends on the label budget: frozen features with a small head win with very few labels, fine-tuning with more, and training from scratch catches up with many; pooling choices can change frozen-feature results more than the encoder does. Fine-tuning can distort features useful out of distribution (LP-FT mitigates this). Proper scores and calibration error measure confidence; temperature scaling can improve calibration on the distribution it was fitted on, within the limits of one parameter and a finite validation set, and does not transfer reliably under shift. Uncertainty must be estimated by resampling the independent unit, and results reported by slice, with visual diagnostics treated as hypotheses.

18.13 Exercises

1. *Conceptual.* Can calibration improve while accuracy stays unchanged?
2. *Conceptual.* What changes when a linear probe is replaced by a large nonlinear head?
3. *Conceptual.* Why should resampling use videos rather than individual frames when videos are the independent units?
4. *Derivation.* Show that the Brier score is a proper scoring rule: for a fixed true distribution π , the expected score is minimised by $\mathbf{p} = \pi$.
5. *Derivation.* Show that temperature scaling with $T > 0$ preserves each example’s argmax. Can it change the ranking of examples by top-class confidence?
6. *Derivation.* For one object token among N i.i.d. background tokens with variance σ^2 per coordinate, compute the signal-to-noise ratio of the mean-pooled feature and of an ideal attentive pooling.
7. *Implementation.* Add a k -NN evaluation to the lab and compare it with the probes across label budgets.
8. *Implementation.* Carry out the experiment of section 18.11.
9. *Interpretation.* A paper shows a t-SNE plot in which its method’s classes form cleaner clusters than the baseline’s. What additional evidence would you require?
10. *Open question.* How would you decide, before deployment, whether a model will be

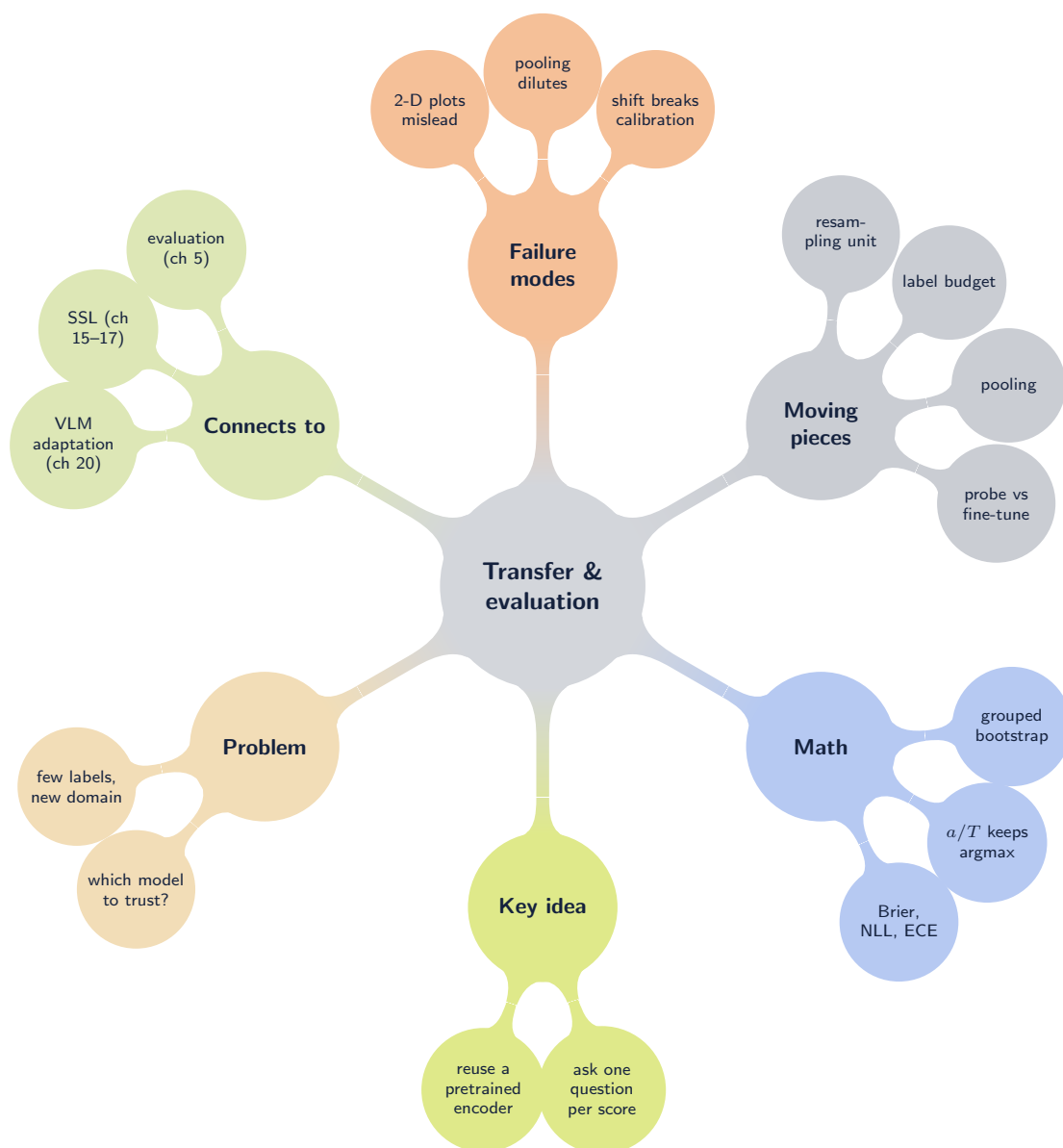


Figure 18.4: Memory map of chapter 18.

calibrated on a new farm with no labels?

18.13.1 Answer discussions

1. Yes: positive-temperature scaling preserves the argmax and can lower NLL and ECE.
2. The evaluation includes more readout capacity and can access nonlinearly encoded information; the score no longer answers the linear-accessibility question and must be compared only with heads of equal capacity.
3. Frames of one video are dependent; resampling frames treats them as independent and under-estimates uncertainty.
4. $\mathbb{E}_{y \sim \pi} \sum_k (p_k - y_k)^2 = \sum_k (p_k^2 - 2p_k\pi_k + \pi_k)$, minimised at $p_k = \pi_k$ (set the gradient to zero under $\sum p_k = 1$).
5. $a_j/T > a_k/T \iff a_j > a_k$ for $T > 0$, so the argmax is preserved. With two classes, confidence is a monotone function of the single logit gap, so the ranking of examples is preserved too; with more classes it depends on all the gaps, and the ranking of examples by confidence *can* change.
6. Mean pooling: signal $\mathbf{s}/N$, noise variance $\sigma^2(N-1)/N^2$, $\text{SNR} \approx \|\mathbf{s}\|^2/((N-1)\sigma^2)$; ideal attention selects the object token: $\text{SNR} \approx \|\mathbf{s}\|^2/\sigma_{\text{obj}}^2$.
7. k -NN should behave like the frozen probes at small budgets and need no training.
8. Expect LP-FT to sit between LP and FT in-domain and to help out of distribution when the shift is large.
9. Held-out accuracy under a fixed protocol, several seeds, quantitative neighbourhood metrics, and the same projection hyperparameters for both methods.
10. Unlabelled checks (prediction-confidence and feature-distribution shift, agreement between models), a small labelled audit from the new farm, and conservative decisions until it exists.

18.14 Self-check and common mistakes

- ☐ I can state what k -NN, linear probes, attentive probes and fine-tuning each measure.
- ☐ I can explain why the best transfer method depends on the label budget.
- ☐ I can compute NLL, Brier score and ECE and explain temperature scaling.
- ☐ I can choose the resampling unit for a bootstrap interval.
- ☐ I can design a transfer evaluation that includes distribution shift.

Common mistakes: reporting one protocol as “representation quality”; comparing methods with different tuning budgets; mean-pooling token features without trying attentive pooling; fitting temperature on the test set; assuming in-domain calibration holds under shift; bootstrapping images when farms or patients are the units; showing only averages; treating a t-SNE plot as evidence; fine-tuning with a random head and a large encoder learning rate.

References

- I. Goodfellow, Y. Bengio and A. Courville. *Deep Learning*. MIT Press, 2016, section 15.2 (transfer learning and domain adaptation).
- J. Donahue et al. DeCAF: a deep convolutional activation feature for generic visual recognition. In *ICML*, 2014.
- J. Yosinski, J. Clune, Y. Bengio and H. Lipson. How transferable are features in deep neural networks? In *NeurIPS*, 2014.

- S. Kornblith, J. Shlens and Q. V. Le. Do better ImageNet models transfer better? In *CVPR*, 2019.
- K. He, R. Girshick and P. Dollár. Rethinking ImageNet pre-training. In *ICCV*, 2019.
- C. Guo, G. Pleiss, Y. Sun and K. Q. Weinberger. On calibration of modern neural networks. In *ICML*, 2017.
- D. Hendrycks and T. Dietterich. Benchmarking neural network robustness to common corruptions and perturbations. In *ICLR*, 2019.
- Y. Ovadia et al. Can you trust your model’s uncertainty? Evaluating predictive uncertainty under dataset shift. In *NeurIPS*, 2019.
- P. W. Koh et al. WILDS: a benchmark of in-the-wild distribution shifts. In *ICML*, 2021.
- A. Kumar, A. Raghunathan, R. Jones, T. Ma and P. Liang. Fine-tuning can distort pretrained features and underperform out-of-distribution. In *ICLR*, 2022. arXiv:2202.10054.
- G. W. Brier. Verification of forecasts expressed in terms of probability. *Monthly Weather Review*, 78(1):1–3, 1950.

Part V

Multimodal and generative vision

Chapter 19

Image–text alignment

Prerequisites: chapter 15 (InfoNCE); chapter 14 (transformers); chapter 18 (evaluation protocols).

Learning outcomes: after this chapter you can

1. derive the symmetric image–text contrastive (CLIP) loss and its gradient, and say what it does and does not teach;
2. explain zero-shot classification as retrieval against text prompts — a linear classifier whose weights are written in language — and why prompt ensembling helps;
3. evaluate retrieval with $\text{recall}@k$ in both directions, with correct relevance sets and stated pool sizes;
4. explain why contrastively trained text encoders can behave like bags of words, and design hard-negative tests for order, relations and counting;
5. state what a softmax over prompts means (and does not mean) as a probability.

19.1 A concrete problem

An agronomy archive holds 500,000 photographs with free-text notes written by field staff (“yellowing lower leaves, rust pustules on the left leaf”). Two requests arrive:

1. **Search:** “find photos of rust pustules on wheat flag leaves” — images retrieved by a sentence.
2. **New categories:** classify photos into eight newly defined damage types, for which no labelled images exist — only names and short descriptions.

Neither fits supervised learning: the queries are open-ended and the categories have no training labels. What the archive *does* have is pairs of images and text. **Image–text alignment** trains an image encoder and a text encoder so that matching pairs land close together in one embedding space. Then search is nearest-neighbour retrieval, and a new category is just another sentence to compare against.

This chapter derives the CLIP objective (chapter 15’s InfoNCE, made symmetric across two modalities), turns zero-shot classification into retrieval, and develops the evaluation discipline that open-ended vision–language systems require: relevance sets, both retrieval directions, hard negatives that test word order and relations, and honest use of prompts. Chapter 20 connects such encoders to language models.

19.2 How we got here

Joint embeddings of images and words (2010–2018). WSABIE (Weston, Bengio and Usunier, 2010) embedded images and annotations in a shared space with a ranking loss. DeViSE (Frome and colleagues, 2013) mapped image features to word vectors and recognised classes never seen in training — early zero-shot learning. Kiros, Salakhutdinov and Zemel (2014) and VSE++ (Faghri and colleagues, 2018) learned visual–semantic embeddings for caption retrieval and showed that **hard negatives** matter.

Scale and natural language supervision (2020–2021). ConVIRT (Zhang and colleagues, 2020) applied a symmetric contrastive loss to chest X-rays and radiology reports. **CLIP** (Radford and colleagues, 2021) trained on 400 million image–text pairs collected from the web, with a 32,768-pair batch and a learnable temperature; its zero-shot ImageNet accuracy matched a supervised ResNet-50 without using any ImageNet training labels — this chapter’s paper bridge. ALIGN (Jia and colleagues, 2021) showed that 1.8 billion noisier pairs work too.

Refinements (2022–2024). LiT (Zhai and colleagues, 2022) locked a pretrained image encoder and trained only the text side; OpenCLIP and LAION-5B (Schuhmann and colleagues, 2022) made reproduction public; SigLIP (Zhai and colleagues, 2023) replaced the softmax with a pairwise sigmoid loss that does not need the whole batch for normalisation.

Limits become visible (2022–2024). Winoground (Thrush and colleagues, 2022) showed that models cannot match “a person sits and a dog stands” versus “a dog sits and a person stands”. Yuksekgonul and colleagues (2023) showed that contrastive vision–language models often behave like **bags of words** and proposed training with order-perturbed hard negatives. Liang and colleagues (2022) described a “modality gap”: image and text embeddings occupy separate cones of the sphere. CLIP-style encoders nevertheless became the visual front end of most vision–language models (chapter 20) and the text encoder of text-to-image diffusion (chapter 22).

19.3 Assumptions and notation

Symbol	Meaning
$(\mathbf{x}_i, \mathbf{t}_i)$	an image and its paired text
f, g	image and text encoders
$\mathbf{u}_i = \bar{f}(\mathbf{x}_i), \mathbf{v}_i = \bar{g}(\mathbf{t}_i)$	L2-normalised image and text embeddings
$S_{ij} = \mathbf{u}_i^\top \mathbf{v}_j / \tau$	scaled similarity; τ the temperature (CLIP learns $1/\tau$, the “logit scale”)
B	batch size (number of pairs)
$\mathcal{R}(q)$	the set of items relevant to query q
$R@k$	recall at k

Assumptions:

1. **Pairs are informative.** A caption describes something visible in its image. Web alt-text often does not; noisy pairs are tolerated at scale but limit what is learned.
2. **The other pairs in a batch are negatives.** Duplicate or near-duplicate captions make this false.
3. **Relevance is defined by the dataset**, not by the training pairing: several captions may be correct for one image, and several images for one caption.

19.4 Mathematical formulation

For a batch of B pairs, the **symmetric contrastive loss** is

$$\mathcal{L} = -\frac{1}{2B} \sum_{i=1}^B \left[\log \frac{e^{S_{ii}}}{\sum_j e^{S_{ij}}} + \log \frac{e^{S_{ii}}}{\sum_j e^{S_{ji}}} \right].$$

The first term classifies, for each image, which caption is its partner (a softmax over row i); the second, for each caption, which image is its partner (a softmax over column i). The targets are the diagonal.

Zero-shot classification with classes $c = 1, \dots, K$ and prompts $\mathbf{t}_c$ (or prompt sets $\mathcal{P}_c$):

$$\mathbf{w}_c = \frac{\sum_{\mathbf{t} \in \mathcal{P}_c} \bar{g}(\mathbf{t})}{\|\sum_{\mathbf{t} \in \mathcal{P}_c} \bar{g}(\mathbf{t})\|}, \quad \hat{y} = \arg \max_c \mathbf{u}^\top \mathbf{w}_c.$$

Retrieval ranks all candidates by similarity; $R@k$ is the fraction of queries with at least one relevant item among the top k .

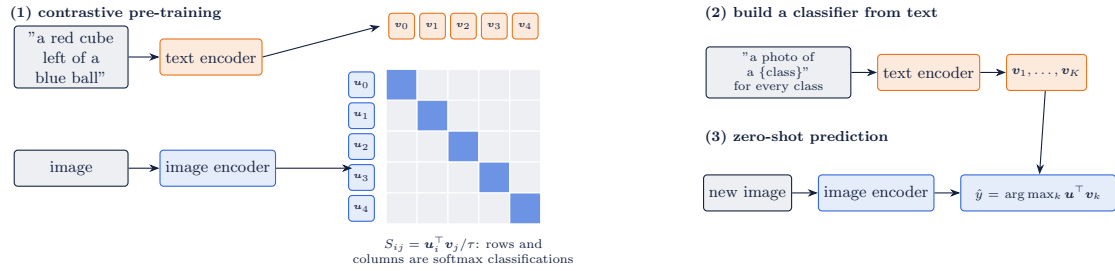


Figure 19.1: CLIP, redrawn after Fig. 1 of Radford et al. (2021): (1) contrastive pre-training with true pairs on the diagonal of the similarity matrix; (2) a classifier built from one text prompt per class; (3) zero-shot prediction as the most similar prompt. Paper: [arXiv:2103.00020](https://arxiv.org/abs/2103.00020).

19.5 Derivation, with reasons for each step

Step 1 — the gradient pulls pairs together and pushes the rest apart, in both directions. Let P^r be the row softmax of S and P^c the column softmax. Differentiating as in chapter 15, step 1,

$$\frac{\partial \mathcal{L}}{\partial S_{ij}} = \frac{1}{2B} \left[(P_{ij}^r - \delta_{ij}) + (P_{ij}^c - \delta_{ij}) \right], \quad \frac{\partial \mathcal{L}}{\partial \mathbf{u}_i} = \frac{1}{\tau} \sum_j \frac{\partial \mathcal{L}}{\partial S_{ij}} \mathbf{v}_j, \quad \frac{\partial \mathcal{L}}{\partial \mathbf{v}_j} = \frac{1}{\tau} \sum_i \frac{\partial \mathcal{L}}{\partial S_{ij}} \mathbf{u}_i,$$

followed by projection onto the sphere's tangent plane (chapter 15, step 2). *Why both directions?* The row term alone lets one caption sit close to many images (each image still picks it out among captions); the column term forbids that. Each direction is a different retrieval task, and both are trained.

Step 2 — shared dimension is not alignment. Two independently trained encoders with 512-dimensional outputs live in spaces whose coordinates mean different things; only joint training (or a learned mapping) makes $\mathbf{u}^\top \mathbf{v}$ meaningful. The temperature matters

as in chapter 15: CLIP learns $\log(1/\tau)$, initialised at $\log(1/0.07)$ and clipped at 100 for stability.

Step 3 — zero-shot classification is a linear classifier written in language. The prediction $\arg \max_c \mathbf{u}^\top \mathbf{w}_c$ is a linear classifier on image embeddings whose weight vectors $\mathbf{w}_c$ are text embeddings. Nothing is fitted on the class labels — but the *prompt* is a design choice on the weights. The candidate set defines the task: five classes are easier than five thousand, and adding a class changes the answer for images near the new class.

Step 4 — prompt ensembling averages out irrelevant words. A single prompt embedding contains the class *and* everything else in the sentence. Write $\bar{g}(\mathbf{t}) \approx \mathbf{a}_c + \mathbf{e}_t$, a class direction plus a prompt-specific component. Averaging over prompts that vary only the irrelevant parts shrinks $\mathbf{e}$ (if roughly independent, by $1/\sqrt{|\mathcal{P}_c|}$) and keeps $\mathbf{a}_c$. In the lab, classifying the left object’s shape with one prompt (which also names an arbitrary colour and right-hand object) reaches 0.490; averaging over all colours and right-hand objects reaches 0.919. CLIP’s paper reports a similar effect on ImageNet from 80 hand-written templates.

Step 5 — what a softmax over prompts means. Applying a softmax to $\{\mathbf{u}^\top \mathbf{w}_c / \tau\}$ gives numbers that sum to one, but they are probabilities *relative to the candidate set and the temperature*. They are not calibrated (chapter 18) unless calibrated on labelled data, and they change when candidates are added: with similarities 0.30 and 0.25 and $1/\tau = 100$, the first class gets 0.993; add a third candidate at 0.29 and it drops to 0.727. Treat zero-shot scores as rankings until validated.

Step 6 — why contrastive alignment can ignore word order. The loss only asks the encoders to tell the true caption apart from the *other captions in the batch*. Random captions almost never differ from the true one only in word order, so a text encoder that ignores order loses almost nothing. A bag-of-words encoder (mean of token embeddings) cannot represent order at all: “red cube left of blue ball” and “blue ball left of red cube” receive identical embeddings. In the lab, the bag-of-words model scores exactly chance (0.500) on a swapped-caption test, while a text encoder with positions scores 1.000. Large transformer text encoders *can* represent order, but nothing in the objective forces them to; tests such as Winoground and ARO measure whether they do. Training with **hard negatives** that differ only in order, relation or count puts those distinctions into the loss.

Step 7 — retrieval metrics need relevance sets. R@ k counts a query as a hit if *any* relevant item is in the top k . If the evaluation assumes only the paired item is relevant (the “diagonal”), a model is penalised for ranking an equally correct item first. In the lab’s world many scenes share a caption: with a pool of 1,000 there are on average 3.7 relevant images per caption, and text-to-image R@1 is 0.349 diagonal-only but 0.999 relevance-aware. Also report the **direction** (image→text and text→image differ) and the **pool size**: R@1 for the bag-of-words model falls from 0.850 to 0.586 when the pool grows from 100 to 1,000 candidates.

Step 8 — composition. If the encoders represent attributes separately (shape, colour) rather than memorising combinations, they generalise to unseen combinations. The lab holds out four shape–colour pairs from training: the positional model classifies the left shape of held-out combinations at 0.994 with a prompt ensemble. Real models compose far less reliably, especially for relations and counts; held-out combinations are a direct test.

Step 9 — sigmoid instead of softmax. SigLIP replaces the two softmaxes with independent binary decisions for every pair: $-\sum_{i,j} \log \sigma(z_{ij}(S_{ij} + b))$ with $z_{ij} = +1$ on the diagonal and -1 elsewhere. No normalisation over the batch is needed, which simplifies

large-scale distributed training and works better at small batch sizes. The evaluation lessons of this chapter apply unchanged.

19.6 Numerical worked example

Two pairs. $S = \begin{pmatrix} 2 & 0 \\ 0 & 2 \end{pmatrix}$: every correct-pair probability is $e^2/(e^2 + 1) = 0.881$, and the symmetric loss is $\log(1 + e^{-2}) = 0.1269$. With all similarities 0, both directions give probability $\frac{1}{2}$ and loss $\log 2 = 0.6931$.

Recall. Four queries with first relevant item at ranks 1, 2, 7, 3: $R@1 = 1/4$, $R@3 = 3/4$, $R@10 = 1$. Recall@ k is averaged over queries; it is not the fraction of retrieved items that are relevant (precision).

Candidate dependence. Two prompts with cosine similarities 0.30 and 0.25, logit scale 100: $\text{softmax}(e^{30}, e^{25})/(\cdot) = (0.993, 0.007)$. Add a prompt at 0.29: (0.727, 0.005, 0.268) — the image did not change, the “probability” of the first class did.

Gradient direction. With $B = 2$, $\tau = 1$, S as above: $P^r = P^c = \begin{pmatrix} 0.881 & 0.119 \\ 0.119 & 0.881 \end{pmatrix}$, so $\partial\mathcal{L}/\partial S_{11} = \frac{1}{4}(2 \cdot (0.881 - 1)) = -0.060$ and $\partial\mathcal{L}/\partial S_{12} = \frac{1}{4}(2 \cdot 0.119) = +0.060$: raise the diagonal, lower the rest.

19.7 Small implementation and checks

The lab `labs/ch19-clip/clip.py` trains an image MLP and a text encoder jointly:

```
U, ci = image_embed(p, X); V, ct = text_embed(p, Tok)           #
→ normalised embeddings
S = U @ V.T / tau
Pr = softmax(S, axis=1); Pc = softmax(S, axis=0)               #
→ image->text and text->image
loss = -0.5 * (np.mean(np.log(np.diag(Pr)))) +
→ np.mean(np.log(np.diag(Pc)))
G = 0.5 * ((Pr - I) + (Pc - I)) / B / tau                     # dL/d(U
→ V^T)
# image branch gets G @ V, text branch gets G.T @ U, then back through
→ normalisation
```

The bag-of-words text encoder averages token embeddings; the positional one concatenates them in order.

Checks, in `test_clip.py`: the worked examples; recall ranks on a small matrix; a gradient check of the symmetric loss through both encoders and the token embeddings, for both text encoders; that swapped captions contain the same words; the bag-of-words model scores exactly 0.5 on the swap test and the positional model above 0.9; and retrieval, relevance-aware recall and prompt ensembling behave as described after training.

19.8 Experiment and expected observations

Run `python labs/ch19-clip/clip.py` (seed 0; 2,000 Adam steps, batch 128, $\tau = 0.1$; four shape-colour pairs never appear in training).

text encoder	loss	i→t R@1 (pool 100 / 1000)	t→i R@1, diagonal (100 / 1000)	t→i R@1, relevance- aware (1000)	swap test
bag of words	1.78 → 0.72	0.850 / 0.586	0.690 / 0.153	0.490	0.500
positional	1.06 → 0.35	0.990 / 0.980	0.880 / 0.349	0.999	1.000

zero-shot left shape (chance 0.167)	one prompt	prompt ensemble	ensemble, held-out combinations
bag of words	0.305	0.519	0.566
positional	0.490	0.919	0.994

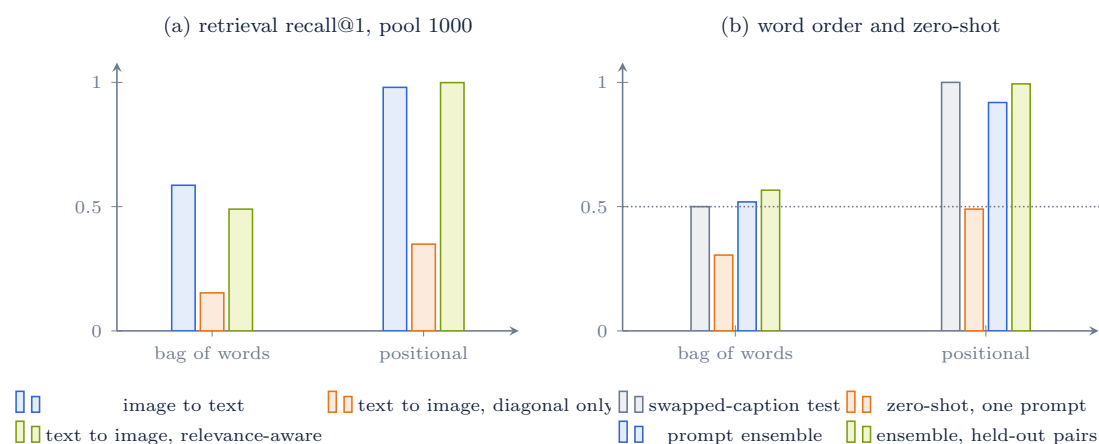


Figure 19.2: (a) **Recall@1** with 1,000 candidates: counting only the paired image hides that most “misses” retrieve another image with the same caption. (b) The bag-of-words model is at chance on the swapped-caption test; prompt ensembling raises zero-shot accuracy for both models, including on held-out combinations.

What to notice:

1. **The bag-of-words model is not useless** — it retrieves well when order does not matter — but it cannot distinguish mirror-image scenes, and its relevance-aware recall stalls at about half for that reason.
2. **Evaluation conventions move numbers by a factor of three** (t→i 0.349 diagonal vs. 0.999 relevance-aware).
3. **Prompts are weights.** The ensemble nearly doubles zero-shot accuracy without any training.
4. **Composition can work** when attributes are encoded separately: held-out combinations are classified as well as seen ones here. Test it on real models rather than assuming it.
5. **Pool size changes recall:** always state it.

19.9 Moving pieces: diagnosing and fixing

Moving piece	Symptom when it is wrong	How to confirm	Fix
Pair quality	retrieval good on easy queries, poor on specific ones	inspect random training pairs	filter or recaption data; deduplicate
Negatives	order, relations, counts ignored	swapped/negated/re-counted caption tests	hard negatives in training (NegCLIP); evaluate with Winoground/ARO-style sets
Duplicate captions	low diagonal recall, many “near misses”	count relevant items per query	relevance-aware evaluation; deduplicate batches
Temperature	unstable training or flat similarities	track learned logit scale	learnable τ with clipping
Batch size	weak alignment with small batches	recall vs. batch size	larger batches, queues, or sigmoid loss (SigLIP)
Prompts	zero-shot accuracy sensitive to wording	accuracy across templates	fixed, pre-registered prompt ensembles; tune on validation only
Candidate set	“probabilities” change when classes are added	rerun with different candidate lists	report the candidate set; calibrate on labelled data
Domain	poor on documents, small text, medical images	in-domain retrieval test	domain-specific pairs; fine-tune or LiT-style adaptation
Modality gap	image–image and text–text similarities incomparable with cross-modal ones	similarity histograms	only compare across modalities as trained; calibrate thresholds
Evaluation leakage	suspiciously high zero-shot results	overlap of evaluation images with web training data	deduplicate against training data; report possible contamination

First things to try: (1) evaluate both retrieval directions with correct relevance sets; (2) run swapped-word and counting tests; (3) compare a single prompt with a fixed ensemble; (4) check whether test images appear in the training pool.

19.10 Application, limitations and failure cases

Application (educational case study): visual search in an agronomy archive. Encode all 500,000 photographs once with a public CLIP-style model and store the normalised embeddings in an approximate nearest-neighbour index. Queries are encoded with the text encoder and answered by cosine similarity. Build an evaluation set of 200 queries written by agronomists, with relevance judged for the top 20 results of each system (pooled judging), and report $R@k$ and precision at k . Add targeted tests: lesion *position* (“on the

lower leaves”), *counts* (“a single pustule”), and *negation* (“no yellowing”), where contrastive models are known to be weak. For the eight new damage types, compare zero-shot prompts with a linear probe on the same embeddings trained on 10 labelled examples per class (chapter 18).

Limitations:

- Web-scale pairs carry the biases and errors of web text; models learn what captions mention, not what images contain.
- Natural-image encoders may not preserve small text, page structure or fine-grained differences.
- Contrastive alignment does not ground individual words in image regions; it aligns whole images with whole texts.
- Zero-shot scores are not calibrated probabilities.

19.11 Bridge to a research paper

Radford, Kim, Hallacy, Ramesh, Goh, Agarwal, Sastry, Aspell, Mishkin, Clark, Krueger and Sutskever (2021), *Learning transferable visual models from natural language supervision* (ICML 2021) — CLIP (figure 19.1; [arXiv:2103.00020](https://arxiv.org/abs/2103.00020)).

- **Problem:** supervised vision models recognise a fixed set of classes and transfer poorly to new ones; natural language is a much richer, freely available source of supervision.
- **Mechanism:** 400 million (image, text) pairs from the web; an image encoder (ResNet or ViT) and a transformer text encoder trained from scratch with the symmetric contrastive loss, batch size 32,768 and a learned temperature; zero-shot classification with prompt templates and ensembling.
- **Central claims:** zero-shot transfer to over 30 datasets, matching a supervised ResNet-50 on ImageNet without its training labels; much better robustness to natural distribution shifts (ImageNet variants) than ImageNet-trained models of equal ImageNet accuracy; linear probes on CLIP features competitive with the best supervised features.
- **Assumptions to question:** overlap between web training data and evaluation sets (the paper analyses it); dependence on prompt engineering; weaknesses the paper itself reports on counting, fine-grained classes and abstract tasks; social biases in learned associations.
- **Affordable experiment:** in the lab, add hard negatives to training — for each batch, append each caption’s swapped version as an extra candidate that must score lower than the true caption — and measure the swap test and retrieval for the positional encoder trained with fewer steps (where it has not yet learned order). Then add a counting attribute (one or two objects) and test whether prompts mentioning the count are matched correctly with and without counting hard negatives.

19.12 Memory map

Summary. CLIP-style alignment trains an image encoder and a text encoder with a symmetric contrastive loss so that paired images and texts are close and other pairs in the batch are far apart, in both retrieval directions. Zero-shot classification is retrieval among prompts: a linear classifier whose weights are text embeddings, improved by averaging prompts; its softmax scores depend on the candidate set and are not calibrated. Because random negatives rarely differ only in word order, relations or counts, such models can



Figure 19.3: Memory map of chapter 19.

behave like bags of words; hard-negative tests and training expose and fix this. Retrieval must be evaluated with relevance sets, both directions and stated pool sizes.

19.13 Exercises

1. *Conceptual.* Why does the loss include both rows and columns?
2. *Conceptual.* Does increasing feature dimension alone align two independently trained encoders?
3. *Conceptual.* What is [recall@2](#) for first-relevant ranks 1, 4, 2, 5?
4. *Derivation.* Derive $\partial\mathcal{L}/\partial S_{ij}$ for the symmetric loss and show that each row and each column of the gradient sums to zero.
5. *Derivation.* Show that zero-shot classification with normalised prompt embeddings is a linear classifier, and write prompt ensembling as averaging classifier weights.
6. *Derivation.* Write the SigLIP loss and its gradient with respect to S_{ij} ; compare the gradient on a hard negative with that of the softmax loss.
7. *Implementation.* Add relevance-aware image→text recall to the lab and explain why it differs less from diagonal recall than in the other direction.
8. *Implementation.* Carry out the experiment of section 19.11.
9. *Interpretation.* A team reports 95% zero-shot accuracy on a plant-disease benchmark using prompts chosen on the test set. What would you ask them to change?
10. *Open question.* What training signal, if any, would make a contrastive model ground each word in an image region?

19.13.1 Answer discussions

1. Rows supervise image→text retrieval, columns text→image; without columns, one caption could match many images.
2. No; alignment is a learned relationship between the spaces, not a matter of shape.
3. $2/4 = 0.5$.
4. $\partial\mathcal{L}/\partial S_{ij} = \frac{1}{2B}[(P_{ij}^r - \delta_{ij}) + (P_{ij}^c - \delta_{ij})]$; rows of $P^r - \mathbf{I}$ sum to zero and columns of $P^c - \mathbf{I}$ sum to zero, so each term separately has the stated property for its own direction.
5. $\hat{y} = \arg \max_c \mathbf{w}_c^\top \mathbf{u}$ is linear in $\mathbf{u}$; ensembling sets $\mathbf{w}_c$ to the normalised mean of prompt embeddings.
6. $\partial/\partial S_{ij} = -z_{ij} \sigma(-z_{ij}(S_{ij} + b))$; for a hard negative the sigmoid gradient depends only on that pair, while the softmax gradient depends on all other candidates in the row and column.
7. Captions are more varied than images in relevance terms here (each image has one caption), so duplicates mainly affect text→image.
8. Hard negatives should raise the swap test faster for the positional encoder; counting needs both a representation of count in the image encoder and negatives that require it.
9. Fix prompts before seeing test data (or choose them on a validation split), report the prompt list, compare with a few-shot linear probe, and check overlap of test images with the pretraining data.
10. Region-level supervision (boxes or masks with phrases), dense contrastive losses between words and patches, or generative objectives that must refer to locations (chapter 20) — each adds information that image-level pairs lack.

19.14 Self-check and common mistakes

- ☐ I can write the symmetric contrastive loss and its gradient.
- ☐ I can explain zero-shot classification as retrieval and prompt ensembling as weight averaging.
- ☐ I can compute $\text{recall}@k$ with relevance sets and state direction and pool size.
- ☐ I can design tests for word order, relations and counts.
- ☐ I can say what a softmax over prompts means.

Common mistakes: evaluating only one retrieval direction; assuming the paired item is the only relevant one; omitting the pool size; tuning prompts on test labels and calling the result zero-shot; reading prompt softmax scores as calibrated probabilities; assuming that contrastive models understand relations or counts; ignoring overlap between web training data and benchmarks; comparing similarities across modalities as if they were on one scale.

References

- I. Goodfellow, Y. Bengio and A. Courville. *Deep Learning*. MIT Press, 2016, section 15.6 (zero-shot learning).
- J. Weston, S. Bengio and N. Usunier. Large scale image annotation: learning to rank with joint word-image embeddings. *Machine Learning*, 81:21–35, 2010.
- A. Frome et al. DeViSE: a deep visual-semantic embedding model. In *NeurIPS*, 2013.
- F. Faghri, D. J. Fleet, J. R. Kiros and S. Fidler. VSE++: improving visual-semantic embeddings with hard negatives. In *BMVC*, 2018.
- Y. Zhang et al. Contrastive learning of medical visual representations from paired images and text. In *MLHC*, 2022 (arXiv 2020).
- A. Radford et al. Learning transferable visual models from natural language supervision. In *ICML*, 2021. arXiv:2103.00020.
- C. Jia et al. Scaling up visual and vision-language representation learning with noisy text supervision. In *ICML*, 2021.
- X. Zhai et al. LiT: zero-shot transfer with locked-image text tuning. In *CVPR*, 2022.
- X. Zhai, B. Mustafa, A. Kolesnikov and L. Beyer. Sigmoid loss for language image pre-training. In *ICCV*, 2023.
- T. Thrush et al. Winoground: probing vision and language models for visio-linguistic compositionality. In *CVPR*, 2022.
- M. Yuksekgonul, F. Bianchi, P. Kalluri, D. Jurafsky and J. Zou. When and why vision-language models behave like bags-of-words, and what to do about it. In *ICLR*, 2023.
- W. Liang et al. Mind the gap: understanding the modality gap in multi-modal contrastive representation learning. In *NeurIPS*, 2022.
- C. Schuhmann et al. LAION-5B: an open large-scale dataset for training next generation image-text models. In *NeurIPS Datasets and Benchmarks*, 2022.

Chapter 20

Instruction-tuned VLMs and adaptation

Prerequisites: chapter 19 (image–text alignment); chapter 14 (attention, cross-attention); chapter 11 (training); chapter 18 (evaluation protocols).

Learning outcomes: after this chapter you can

1. trace the learned components of a vision–language model (VLM) — vision encoder, projector or cross-attention, language model — and say which are trained in each stage;
2. write the conditional language-modelling objective, mask it to the response, and explain teacher forcing, exposure bias and decoding choices;
3. derive LoRA’s parameter count and gradients, explain its zero initialisation, and choose *where* to adapt;
4. test whether a VLM uses the image — text-only, shuffled-image and blank-image baselines — and measure hallucination;
5. evaluate free-text answers reproducibly: parsing, normalisation and fixed decoding.

20.1 A concrete problem

An extension service wants farmers to photograph a plant and ask, in their own words, “what is wrong with this leaf?”, “how many fruits are rotten?”, “is there any rust on the stem?”. The answer must be text, and it must be **about the photograph** — not a fluent guess from what such questions usually have as answers.

A CLIP-style model (chapter 19) can score how well a caption fits an image, but cannot answer a question. A language model can answer questions, but cannot see. A **vision–language model** connects the two: visual features are turned into something the language model can read, and the model is trained to generate answers conditioned on image and question. This chapter takes such a model apart — its components, its objective, how it is adapted with few parameters (LoRA) — and builds the evaluations that separate visual evidence from language priors: questions whose answers are guessable without the image are the norm, not the exception.

20.2 How we got here

Captioning and VQA (2014–2018). Encoder–decoder captioning (Vinyals and colleagues, 2015; Xu and colleagues with visual attention, 2015) generated sentences from CNN features. The VQA dataset (Antol and colleagues, 2015) posed free-form questions about images; within two years it was clear that models answered from **language priors** (“what colour is the banana?” $\rightarrow$ yellow), and VQA v2 (Goyal and colleagues, 2017) balanced the data so that each question had images with different answers. Bottom-up and top-down attention (Anderson and colleagues, 2018) attended over detected object regions.

Joint pretraining (2019–2021). ViLBERT, LXMERT and UNITER (2019–2020) pre-trained transformers on image–text pairs with masked-modelling and matching objectives, then fine-tuned for each task.

Frozen language models with eyes (2021–2023). Frozen (Tsimpoukelli and colleagues, 2021) fed visual tokens into a frozen language model. **Flamingo** (Alayrac and colleagues, 2022) inserted gated cross-attention layers between frozen language model blocks and learned from interleaved web text and images, enabling few-shot prompting. BLIP-2 (Li and colleagues, 2023) bridged a frozen image encoder and a frozen language model with a small “Q-Former”. **LLaVA** (Liu, Li, Wu and Lee, 2023) used a single projection from CLIP features into a language model’s input space and trained it on language-model-generated instruction–response conversations — **visual instruction tuning** — this chapter’s paper bridge. Qwen-VL, InternVL and later Qwen2.5-VL (2025) added higher and variable resolution, grounding with boxes, documents and video.

Parameter-efficient adaptation (2019–2023). Adapters (Houlsby and colleagues, 2019), prefix tuning (2021) and **LoRA** (Hu and colleagues, 2022) adapted large models by training a small number of added parameters; QLoRA (Dettmers and colleagues, 2023) combined LoRA with 4-bit quantised base weights so that large models could be fine-tuned on one GPU.

Hallucination becomes measurable (2018–2023). CHAIR (Rohrbach and colleagues, 2018) counted captioned objects absent from images; POPE (Li and colleagues, 2023) asked VLMs yes/no questions about object presence and found they often say “yes” to absent objects.

20.3 Assumptions and notation

Symbol	Meaning
$\mathbf{X}_v, \mathbf{X}_q, \mathbf{X}_a = (a_1, \dots, a_T)$	image, instruction (question) tokens, response tokens
g	vision encoder; $\mathbf{Z}_v = g(\mathbf{X}_v)$ its token features
$\mathbf{W}$ (projector)	maps $\mathbf{Z}_v$ into the language model’s embedding space: $\mathbf{H}_v = \mathbf{Z}_v \mathbf{W}$
f_ϕ	language model (autoregressive decoder)
$p_\theta(a_t \mid a_{<t}, \mathbf{X}_v, \mathbf{X}_q)$	next-token distribution
$\mathbf{W}_0, \mathbf{A}, \mathbf{B}, r, \alpha$	frozen weight, LoRA factors, rank, scaling

Assumptions:

1. **The answer is in the image.** Evaluation questions should require the image; this must be checked (section 20.5, step 8).
2. **Decoding is fixed** (greedy, or a stated sampling scheme with seeds) when models are compared.
3. **Answer parsing is part of the metric** and is specified before evaluation.

20.4 Mathematical formulation

The VLM defines an autoregressive distribution over the response:

$$p_{\theta}(\mathbf{X}_a \mid \mathbf{X}_v, \mathbf{X}_q) = \prod_{t=1}^T p_{\theta}(a_t \mid a_{<t}, \mathbf{X}_v, \mathbf{X}_q),$$

and **instruction tuning** minimises the negative log-likelihood of response tokens only:

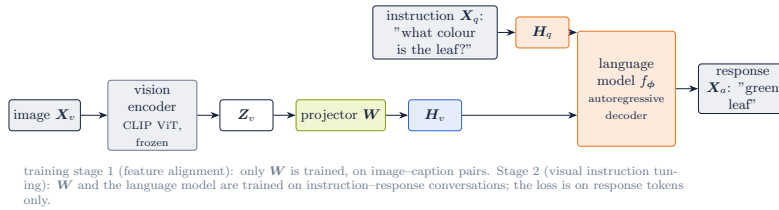
$$\mathcal{L}(\theta) = -\frac{1}{\sum_n T_n} \sum_n \sum_{t=1}^{T_n} \log p_{\theta}(a_t^{(n)} \mid a_{<t}^{(n)}, \mathbf{X}_v^{(n)}, \mathbf{X}_q^{(n)}).$$

LoRA replaces a frozen weight $\mathbf{W}_0 \in \mathbb{R}^{d_{\text{out}} \times d_{\text{in}}}$ by

$$\mathbf{W} = \mathbf{W}_0 + \frac{\alpha}{r} \mathbf{B} \mathbf{A}, \quad \mathbf{A} \in \mathbb{R}^{r \times d_{\text{in}}}, \mathbf{B} \in \mathbb{R}^{d_{\text{out}} \times r}, \mathbf{B}_{\text{init}} = \mathbf{0},$$

training only $\mathbf{A}$ and $\mathbf{B}$: $r(d_{\text{in}} + d_{\text{out}})$ parameters instead of $d_{\text{in}} d_{\text{out}}$.

(a) concatenate visual tokens (LLaVA)



(b) read visual tokens by cross-attention (Flamingo)

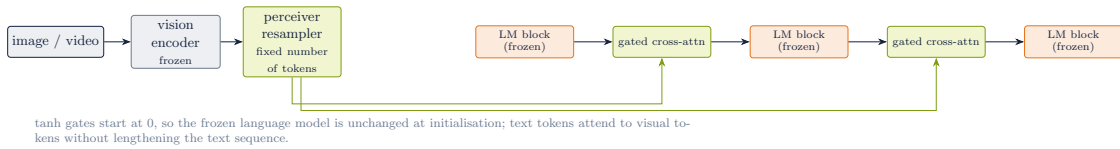


Figure 20.1: Two ways to connect vision to a language model. (a) LLaVA, redrawn after Fig. 1 of Liu et al. (2023): projected visual tokens are placed in the input sequence. (b) Flamingo (Alayrac et al., 2022): resampled visual tokens are read through gated cross-attention layers between frozen language-model blocks. Papers: [arXiv:2304.08485](https://arxiv.org/abs/2304.08485), [arXiv:2204.14198](https://arxiv.org/abs/2204.14198).

20.5 Derivation, with reasons for each step

Step 1 — three components, two interfaces. The **vision encoder** (usually a CLIP-style ViT, chapter 19) turns pixels into a grid of token features. The **connector** turns those features into something the language model can use: a linear or MLP **projector** whose outputs are placed in the language model’s input sequence as extra tokens (LLaVA), a resampler that compresses them to a fixed number of tokens (Flamingo’s perceiver, BLIP-2’s Q-Former), or **cross-attention** layers inside the language model (Flamingo). The **language model** generates the answer. The choice matters for cost: N_v visual tokens in the sequence add N_v positions to every self-attention layer (quadratic cost, chapter 14), while cross-attention adds a read of N_v keys without lengthening the text.

Step 2 — the loss covers the response only. The training sequence is [visual tokens, instruction, response]. Computing the loss on every position would also train the model to *predict the instruction* — a different objective that spends capacity on reproducing questions and can teach the model to continue the user’s text instead of answering it. Masking sets the loss weight of instruction and visual positions to zero. Reduction must be stated too: a two-token answer with token probabilities 0.8 and 0.5 has joint probability 0.40, summed NLL 0.916 and mean NLL 0.458 — and averaging per token versus per sequence weights long answers differently.

Step 3 — teacher forcing and exposure bias. Training conditions each step on the *true* previous tokens (teacher forcing), which makes all steps of a sequence trainable in parallel. Generation conditions on the model’s *own* previous tokens. An early mistake therefore changes everything after it, and the model never saw its own mistakes during training (“exposure bias”). Decoding is part of the system: greedy decoding, beam search and sampling (with temperature or nucleus truncation) give different answers from the same model, and comparisons must fix one. Numeric answers inherit this: “twelve”, “12” and “1 2” are different token sequences.

Step 4 — what a projector can and cannot fix. A trainable projector can re-map features that are present but expressed in the wrong coordinates — a different camera, a different encoder version. It cannot recover information the encoder discarded: if the encoder’s 224×224 input made small text unreadable, no projector restores it. Higher input resolution, image tiling (as in Qwen2.5-VL’s native dynamic resolution) or a different encoder is needed.

Step 5 — LoRA: why low rank, and why zero initialisation. Fine-tuning changes a weight by $\Delta \mathbf{W}$. Empirically the useful changes are often close to low rank, so LoRA constrains $\Delta \mathbf{W} = \frac{\alpha}{r} \mathbf{B} \mathbf{A}$. With $\mathbf{y} = \mathbf{W}_0 \mathbf{x} + \frac{\alpha}{r} \mathbf{B} \mathbf{A} \mathbf{x}$ and upstream gradient $\mathbf{g} = \partial L / \partial \mathbf{y}$:

$$\nabla_{\mathbf{B}} L = \frac{\alpha}{r} \mathbf{g} (\mathbf{A} \mathbf{x})^\top, \quad \nabla_{\mathbf{A}} L = \frac{\alpha}{r} \mathbf{B}^\top \mathbf{g} \mathbf{x}^\top.$$

Reason: chain rule through the two matrix products. With $\mathbf{B} = \mathbf{0}$ at the start, the adapted model is exactly the pretrained one, and $\nabla_{\mathbf{A}} L = \mathbf{0}$ on the first step; $\mathbf{B}$ learns first and then both factors move. After training, $\mathbf{B} \mathbf{A}$ can be merged into $\mathbf{W}_0$ so inference is no slower. The parameter count $r(d_{\text{in}} + d_{\text{out}})$ — 16,384 instead of 1,048,576 for a 1024×1024 layer at $r = 8$ — reduces optimiser state and storage per task; activation memory still scales with the base model.

Step 6 — where to adapt. Adaptation works where the mismatch is. In the lab, a new camera changes the frozen encoder’s outputs (a rotation and an offset of the feature space). Training the 896-parameter projector raises accuracy on the new camera from

0.377 to 0.713; LoRA on the projector improves with rank (0.420, 0.531, 0.620, 0.753 for $r = 1, 2, 4, 8$); LoRA of rank 8 on the *decoder* — more parameters, wrong place — reaches only 0.603. Full fine-tuning reaches the best new-camera accuracy (0.771) but **forgets** the old camera most (0.885 $\rightarrow$ 0.606). Choosing modules to adapt is a hypothesis about what changed.

Step 7 — rank, data and forgetting. A low rank limits how far the model can move, which both restricts adaptation and protects the old behaviour: in the lab, rank 1 keeps old-camera accuracy at 0.873 but helps the new camera little; rank 8 helps the new camera and costs more on the old one. With 300 examples, the number of trainable parameters is a regulariser like any other (chapter 3). Report accuracy on the original distribution after adaptation, not only on the new one.

Step 8 — does the model look? Grounding baselines. Three cheap controls:

- **Text-only model:** train the same architecture without images. Its accuracy is what the questions give away.
- **Shuffled images:** evaluate the trained VLM with images assigned to the wrong questions. If accuracy stays high, the image is not being used.
- **Blank images:** evaluate with an empty image; what remains is the model’s prior.

In the lab, colour questions have a strong prior (leaves are usually green): the text-only model answers 0.572 of them correctly, while the VLM with real images answers 1.000; with shuffled images the VLM drops to 0.291 — *below* the prior, because it now trusts misleading visual evidence — and to 0.424 with blank images. Count questions show the same pattern (0.772 real, 0.404 shuffled, 0.449 text-only). Report these numbers next to every VQA result.

Step 9 — hallucination. Generative VLMs can name objects that are not in the image, especially objects that co-occur with those that are (“fork” next to “plate”). Measure it directly: POPE-style yes/no questions about present and absent objects, with absent objects chosen at random, by popularity, and by co-occurrence; report the rate of “yes” for absent objects. Fluent, confident text is not evidence.

Step 10 — answer parsing is part of the metric. A free-text answer must be mapped to a label: exact match, normalised match (case, articles, punctuation), synonym lists, multiple-choice letter extraction, numeric parsing with units. Each choice changes scores, sometimes by more than the difference between models. Keep raw outputs, publish the parser, and count parse failures separately from wrong answers.

20.6 Numerical worked example

Sequence probability. Answer “green leaf” with $p(\text{green}) = 0.8$ and $p(\text{leaf} \mid \text{green}) = 0.5$: joint 0.40; summed NLL $-\log 0.8 - \log 0.5 = 0.916$; mean NLL 0.458.

LoRA parameters. $d_{\text{in}} = 256$, $d_{\text{out}} = 512$, $r = 4$: $4(256 + 512) = 3,072$ instead of 131,072. A model with 32 layers and LoRA of rank 16 on four 4096×4096 attention matrices per layer trains $32 \cdot 4 \cdot 16 \cdot 8192 \approx 16.8$ million parameters — about 0.25% of a 7-billion-parameter model.

LoRA gradient. $\mathbf{x} = (1, 2)^\top$, $\mathbf{A} = (0.5, -0.5)$ ($r = 1$), $\mathbf{B} = (0, 0)^\top$, $\alpha/r = 1$, upstream $\mathbf{g} = (1, -1)^\top$: $\mathbf{A}\mathbf{x} = -0.5$, so $\nabla_{\mathbf{B}} = \mathbf{g} \cdot (-0.5) = (-0.5, 0.5)^\top$ and $\nabla_{\mathbf{A}} = \mathbf{B}^\top \mathbf{g} \mathbf{x}^\top = \mathbf{0}$.

A prior’s ceiling. If 70% of leaves are green, always answering “green” to “what colour is the leaf?” scores 0.70 on those questions. A model at 0.72 may be using the image a

little — or not at all.

20.7 Small implementation and checks

The lab `labs/ch20-vlm/vlm.py` builds a VLM in miniature: a frozen random “camera” encodes four image slots; a question encoder produces a query; a trainable projector maps slot features into the model space; attention pools them; a two-step decoder generates the answer:

```
qv = np.tanh(E[Q].reshape(m, -1) @ Wq + bq)           # question
Vp = Vimg @ (P + A_P @ B_P) + pos                     # projector
→ (+ optional LoRA)
w = softmax(np.einsum("mjd,md->mj", Vp, qv) / np.sqrt(D)) # attend to
→ image slots
v = np.einsum("mj,mjd->md", w, Vp)
for t in range(2):                                     #
→ autoregressive decoder
    h = np.tanh(np.c_[qv, v, E[prev]] @ (Wd + A_Wd @ B_Wd) + bd); logits
→ = h @ Wo + bo
```

Checks, in `test_vlm.py`: parameter counts and the sequence probability; a LoRA-adapted model equals the base model at initialisation and $\mathbf{A}$'s gradient is zero while $\mathbf{B} = \mathbf{0}$; a gradient check through decoder, attention, projector and both LoRA modules; and that the VLM beats shuffled images and that adapting the projector helps on a new camera.

20.8 Experiment and expected observations

Run `python labs/ch20-vlm/vlm.py` (seed 0; 6,000 training questions, 2,500 Adam steps; exact match of both answer tokens).

model / input	all	colour	count	yes/no
VLM, real images	0.885	1.000	0.772	0.893
VLM, shuffled images	0.418	0.291	0.404	0.529
VLM, blank images	0.466	0.424	0.449	0.513
text-only model	0.499	0.572	0.449	0.487

Adapting to a new camera with 300 examples (800 steps); no adaptation: 0.377 on the new camera.

adapted modules	trainable parameters	new camera	old camera
projector	896	0.713	0.743
LoRA on projector, $r = 1/2/4/8$	56 / 112 / 224 / 448	0.420 / 0.531 / 0.620 / 0.753	0.873 / 0.815 / 0.819 / 0.678
LoRA on decoder, $r = 4/8$	512 / 1,024	0.578 / 0.603	0.791 / 0.741
everything	11,996	0.771	0.606

What to notice:

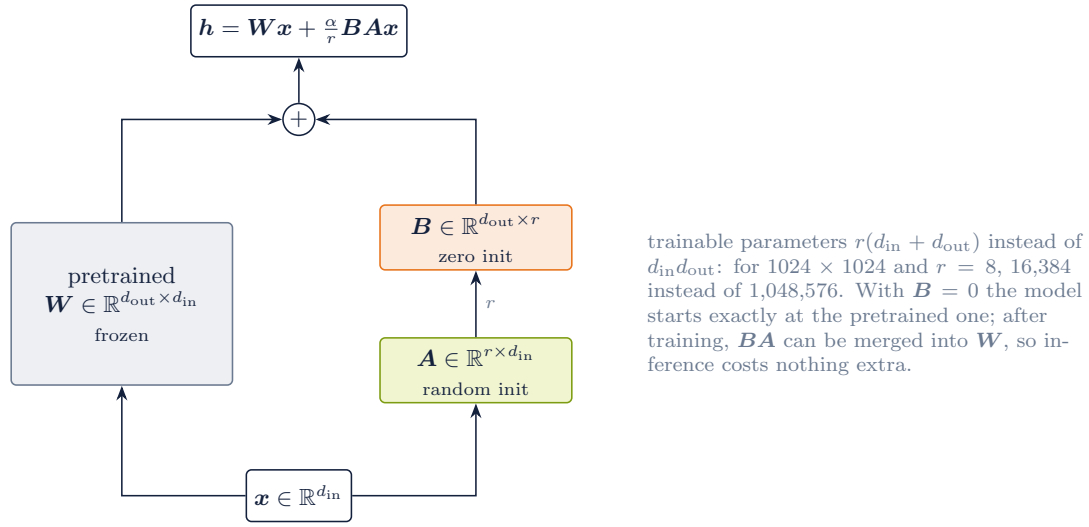


Figure 20.2: Low-rank adaptation, redrawn after Fig. 1 of Hu et al. (2022): a frozen weight plus a trainable low-rank update that starts at zero. Paper: [arXiv:2106.09685](https://arxiv.org/abs/2106.09685).

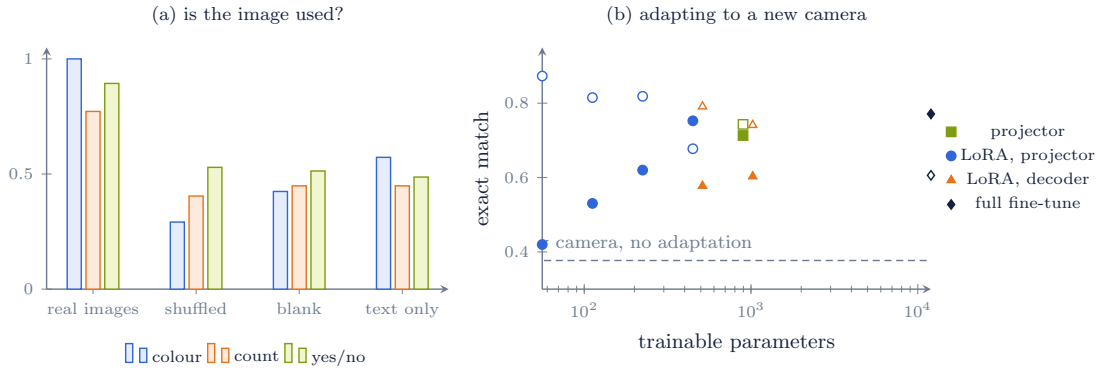


Figure 20.3: (a) Accuracy by question type with real, shuffled and blank images and for a model trained without images: the priors are strong, and only real images give the full accuracy. (b) Adapting to a new camera: accuracy on the new camera (filled) and the old one (open) against trainable parameters.

1. **Half the accuracy is free.** A model that never sees an image gets 0.50 here; the image is worth the other 0.39.
2. **Wrong images are worse than no image** for colour questions: the trained model trusts the visual evidence it is given.
3. **Adapt where the shift is.** The projector — small and in the right place — beats larger LoRA modules in the decoder.
4. **More trainable parameters, more forgetting.** Full fine-tuning gives the best new-camera score and the worst old-camera score.
5. **Rank is a dial**, not a free lunch: rank 1 barely adapts, rank 8 adapts and forgets.

20.9 Moving pieces: diagnosing and fixing

Moving piece	Symptom when it is wrong	How to confirm	Fix
Loss mask	model repeats or continues the question	inspect which positions carry loss	loss on response tokens only
Vision encoder resolution	fails on small text, small objects	accuracy by object/text size	higher resolution, tiling, dynamic resolution
Connector	visual tokens ignored or mis-mapped	shuffled-image and blank-image baselines	train the projector first (stage 1) on captions
Language prior	high accuracy without the image	text-only model; balanced question sets	balance data (VQA v2-style); evaluate on counter-prior images
Hallucination	objects named that are absent	POPE-style yes/no probes	negative examples in tuning; grounding with boxes; calibrated refusal
Decoding	results change between runs	fix decoding; vary seeds	greedy for evaluation; report sampling settings
Answer parsing	scores depend on formatting	audit parse failures	published parser; normalised matching; constrained output formats
Adaptation target	many parameters, little gain	adapt different modules	adapt where the shift is (projector, encoder, decoder)
LoRA rank / scale	no adaptation (too small) or forgetting (too large)	sweep r and α ; old-task accuracy	choose by validation on both distributions

Forgetting	old tasks degrade after adaptation	evaluate the original benchmark again	fewer trainable parameters, mix old data, regularise toward the base model
------------	---------------------------------------	--	--

First things to try: (1) text-only, shuffled-image and blank-image baselines; (2) accuracy by question type and by image property (size, count); (3) a hallucination probe; (4) a fixed decoding and parsing protocol before any comparison.

20.10 Application, limitations and failure cases

Application (educational case study): a plant-health assistant. Start from a public open-weight VLM (for example a Qwen2.5-VL or LLaVA-style checkpoint; record the exact version and its declared input resolution). Build an evaluation of 500 farmer-style questions on held-out photographs from unseen farms, with answers checked by agronomists, balanced so that common answers (“yes, it is diseased”) are not always correct. Report accuracy with text-only and shuffled-image baselines, a POPE-style probe for disease symptoms that are absent, and per-crop slices. Adapt with LoRA on a few hundred in-domain question–answer pairs, then re-run the full evaluation *and* a general benchmark to measure forgetting. The assistant’s answers should cite visible evidence (regions or boxes, where the model supports grounding) so that experts can check them.

Limitations:

- Fluent answers can be confidently wrong; generation quality is not evidence of perception.
- Benchmarks can leak into web-scale training data; results on public VQA sets may overstate generalisation.
- LoRA results depend on rank, scaling, learning rate and which modules are adapted; small changes can matter.
- The lab’s model is a few thousand parameters; it illustrates mechanisms, not the behaviour of billion-parameter models.

20.11 Bridge to a research paper

Liu, Li, Wu and Lee (2023), *Visual instruction tuning* (NeurIPS 2023) — LLaVA (figure 20.1a; [arXiv:2304.08485](https://arxiv.org/abs/2304.08485)).

- **Problem:** instruction-following language models had transformed text tasks; could the same recipe give a general-purpose visual assistant, given that no large set of visual instruction–response data existed?
- **Mechanism:** generate about 158,000 multimodal instruction-following examples by prompting a text-only language model (GPT-4) with captions and box annotations of COCO images; connect a CLIP ViT-L/14 encoder to a Vicuna language model with a linear projection; stage 1 trains only the projection on 595,000 image–caption pairs; stage 2 trains projection and language model on the instruction data with the loss on responses.
- **Central claims:** strong qualitative visual chat ability; a relative score of 85.1% compared with GPT-4 on a synthetic instruction-following benchmark; 92.5% accuracy on ScienceQA when combined with GPT-4.

- **Assumptions to question:** the instruction data were written by a model that never saw the images (only captions and boxes), so they may teach priors; GPT-4-judged scores depend on the judge; benchmarks may not require the image.
- **Affordable experiment:** in the lab, create “generated” training data that mimic this pipeline: answers derived from a caption that omits colour for 30% of objects, filled in by the prior (most likely colour for the shape). Train the VLM on this data and measure colour accuracy on real images, the gap to the text-only model, and how often the VLM answers with the prior colour when the image shows otherwise.

20.12 Memory map



Figure 20.4: Memory map of chapter 20.

Summary. A VLM combines a vision encoder, a connector (projector, resampler or cross-attention) and a language model, trained in stages and finally on instruction–response data with a loss on the response tokens. Generation is autoregressive, so decoding and

exposure to its own errors matter, and answers must be parsed by a stated rule. LoRA adapts a model with a low-rank update initialised at zero; its parameter savings are large, its effect depends on rank and on adapting the right module, and every adaptation should be checked for forgetting. VQA questions often have strong language priors; text-only, shuffled-image and blank-image baselines and hallucination probes are what show whether the model looks at the image.

20.13 Exercises

1. *Conceptual.* Why must instruction tokens be masked from the response loss?
2. *Conceptual.* Can a trainable projector recover a spatial detail absent from every input feature?
3. *Conceptual.* How many LoRA parameters are used for $d_{\text{in}} = 256$, $d_{\text{out}} = 512$, $r = 4$?
4. *Derivation.* Derive the LoRA gradients and show that with $\mathbf{B} = \mathbf{0}$ only $\mathbf{B}$ changes on the first step. What happens if both are initialised at zero?
5. *Derivation.* Show that teacher-forced training maximises the likelihood of the training responses, and give an example where greedy decoding returns a sequence that is not the most likely one.
6. *Derivation.* Compare the cost of N_v visual tokens in the sequence with N_v tokens read by cross-attention, for N_t text tokens and L layers.
7. *Implementation.* Add a POPE-style evaluation to the lab: yes/no questions about absent shapes, chosen at random and chosen to co-occur with present ones.
8. *Implementation.* Carry out the experiment of section 20.11.
9. *Interpretation.* A VLM scores 78% on a disease-identification benchmark and 71% when given shuffled images. What do you conclude?
10. *Open question.* How would you make a VLM say “I cannot tell from this image” when the evidence is insufficient, and how would you evaluate it?

20.13.1 Answer discussions

1. The intended objective is conditional response prediction; training on instruction tokens adds a different objective and can teach the model to continue questions.
2. Not reliably: it can only re-map what the features contain; any answer about the missing detail comes from priors.
3. $4(256 + 512) = 3,072$.
4. As in step 5. If both are zero, both gradients are zero forever: the update never starts.
5. Teacher forcing sums $\log p$ of the true tokens, which is the log-likelihood. Greedy picks the best first token; a sequence that starts with a slightly less likely token can have a higher joint probability (e.g. first-token probabilities 0.5/0.4 with second-token maxima 0.5/0.9: $0.25 < 0.36$).
6. In-sequence: self-attention over $N_t + N_v$ tokens costs $O(L(N_t + N_v)^2 d)$; cross-attention costs $O(L_x N_t N_v d)$ in the L_x cross-attention layers, with self-attention still $O(L N_t^2 d)$.
7. Expect more false “yes” answers for co-occurring absent shapes if the training data has co-occurrence structure.
8. Expect the VLM to inherit the prior: colour accuracy should drop on images that contradict it, and the gap to the text-only model should shrink.
9. Most of the accuracy (71 of 78 points) does not depend on the image; the benchmark is largely answerable from priors, and the 7-point gap is the evidence of visual use.
10. Train with questions whose answers are not visible and the target “cannot tell”, and evaluate on a balanced set with selective accuracy (accuracy against coverage) rather

than plain accuracy.

20.14 Self-check and common mistakes

- ☐ I can name the components of a VLM and say which are trained in each stage.
- ☐ I can write the masked response loss and explain teacher forcing.
- ☐ I can compute LoRA parameter counts and gradients.
- ☐ I can design text-only, shuffled-image and blank-image baselines and a hallucination probe.
- ☐ I can specify decoding and answer parsing for an evaluation.

Common mistakes: training on instruction tokens; comparing models with different decoding; reporting VQA accuracy without a text-only baseline; treating fluent explanations as grounding; adapting the decoder when the shift is in the images; forgetting to measure the original task after adaptation; summing and averaging sequence losses interchangeably; silently normalising answers; assuming a projector can recover information the encoder never kept.

References

- I. Goodfellow, Y. Bengio and A. Courville. *Deep Learning*. MIT Press, 2016, section 10.2 (teacher forcing) and 12.4 (natural language processing).
- O. Vinyals, A. Toshev, S. Bengio and D. Erhan. Show and tell: a neural image caption generator. In *CVPR*, 2015.
- S. Antol et al. VQA: visual question answering. In *ICCV*, 2015.
- Y. Goyal, T. Khot, D. Summers-Stay, D. Batra and D. Parikh. Making the V in VQA matter. In *CVPR*, 2017.
- P. Anderson et al. Bottom-up and top-down attention for image captioning and visual question answering. In *CVPR*, 2018.
- A. Rohrbach, L. A. Hendricks, K. Burns, T. Darrell and K. Saenko. Object hallucination in image captioning. In *EMNLP*, 2018.
- N. Housby et al. Parameter-efficient transfer learning for NLP. In *ICML*, 2019.
- E. J. Hu et al. LoRA: low-rank adaptation of large language models. In *ICLR*, 2022. arXiv:2106.09685.
- J.-B. Alayrac et al. Flamingo: a visual language model for few-shot learning. In *NeurIPS*, 2022. arXiv:2204.14198.
- J. Li, D. Li, S. Savarese and S. Hoi. BLIP-2: bootstrapping language-image pre-training with frozen image encoders and large language models. In *ICML*, 2023.
- H. Liu, C. Li, Q. Wu and Y. J. Lee. Visual instruction tuning. In *NeurIPS*, 2023. arXiv:2304.08485.
- Y. Li et al. Evaluating object hallucination in large vision-language models (POPE). In *EMNLP*, 2023.
- T. Detrmers, A. Pagnoni, A. Holtzman and L. Zettlemoyer. QLoRA: efficient finetuning of quantized LLMs. In *NeurIPS*, 2023.
- S. Bai et al. Qwen2.5-VL technical report. arXiv:2502.13923, 2025.

Chapter 21

Video and omni-modal perception

Prerequisites: chapter 17 (video tokens, temporal controls); chapter 20 (VLM components, grounding baselines); chapter 14 (attention cost).

Learning outcomes: after this chapter you can

1. describe each modality’s interface — sampling, encoding, timestamps — and say what evidence each sampling choice can and cannot capture;
2. compare early fusion, cross-attention and late fusion, and count the cost of joint attention over many streams;
3. design modality ablations with controls (mismatched and shuffled inputs), and explain why removing a modality is an intervention that may be out of distribution;
4. use modality dropout and explicit empty-modality conventions for missing inputs;
5. evaluate spatial and temporal outputs with appropriate metrics (circular error, event windows).

21.1 A concrete problem

A research station records **video with stereo audio** in a greenhouse to monitor pollinators and pests. The questions people ask mix modalities:

- *What* insect is on the flower? (video)
- *What* is buzzing — a bee or a hoverfly? (audio)
- *Where* is the sound coming from — left bench or right? (stereo audio)
- *How many* times did the sprayer flash on? Each flash lasts a fifth of a second. (the right video frames)

A model that “sees and hears” is easy to build; knowing *which evidence it used* is not. A question about a brief event cannot be answered from frames that missed it; a question about direction cannot be answered from mono audio; and a model may score well because of priors in the questions rather than anything it saw or heard. This chapter treats modality interfaces — sampling, timing, fusion — as part of the observation model, and builds the ablations needed to make claims about multimodal understanding. Its paper bridge is Qwen2.5-Omni, a model that takes text, images, audio and video and answers in text and speech.

21.2 How we got here

Multimodal deep learning (2011–2017). Ngiam and colleagues (2011) learned shared representations of audio and video of speech. SoundNet (Aytar, Vondrick and Torralba, 2016) transferred visual knowledge to sound; “Look, listen and learn” (Arandjelović and Zisserman, 2017) learned audio and visual features by predicting whether a frame and a sound clip correspond — a self-supervised correspondence task, later used to localise sounding objects. AudioSet (Gemmeke and colleagues, 2017) provided two million labelled sound clips.

Better audio encoders (2020–2023). wav2vec 2.0 (Baevski and colleagues, 2020) learned speech representations from raw audio by contrastive prediction; Whisper (Radford and colleagues, 2023) trained a speech recogniser on 680,000 hours of weakly labelled audio, using log-mel spectrograms as input. Audio spectrogram transformers treated spectrograms as images.

Many modalities in one model (2021–2025). Perceiver (Jaegle and colleagues, 2021) handled arbitrary inputs through a fixed latent array with cross-attention. ImageBind (Girdhar and colleagues, 2023) aligned six modalities to images contrastively. Video LLMs (2023) fed sampled frames to VLMs (chapter 20). Natively multimodal models (Gemini, 2023; GPT-4o, 2024) and open models such as **Qwen2.5-Omni** (Xu and colleagues, 2025) process audio and video jointly with time-aligned positions and generate speech as well as text — this chapter’s paper bridge.

Robustness to missing inputs (2015–). ModDrop (Neverova and colleagues, 2015) randomly dropped modalities during training so gesture recognition kept working when a sensor failed — the idea behind modality dropout.

21.3 Assumptions and notation

Symbol	Meaning
D, F	clip duration (s) and number of sampled frames; midpoint sampling $t_k = (k + \frac{1}{2})D/F$
f_s, C	audio sample rate and number of channels
N_v, N_a, N_t	numbers of video, audio and text tokens
z_v, z_a	pooled video and audio representations
$\emptyset$	the convention used for a missing modality (zeros, a learned token, a mask)
$d_o(a, b)$	circular error between angles

Assumptions and declarations:

1. **The evidence window is known.** For every question, record which modality contains the answer and when.
2. **Preprocessing is part of the model:** frame rate, resolution, crop, audio sample rate, channels, resampling, padding and truncation must be stated.
3. **An ablation is an intervention.** Removing or replacing a modality creates inputs that may never occur in training; the result describes the model’s response to that intervention.

21.4 Mathematical formulation

Sampling. Midpoint sampling of F frames over $[0, D]$ observes times $t_k = (k + \frac{1}{2})D/F$, $k = 0, \dots, F - 1$. An event of duration δ at a uniformly random time is seen by at least one frame with probability about $\min(1, F\delta/D)$.

Fusion. With modality encoders E_v, E_a :

$$\begin{aligned} \text{late fusion: } \hat{\mathbf{y}} &= h([\text{pool}(E_v(\mathbf{x}_v)), \text{pool}(E_a(\mathbf{x}_a))]), \\ \text{early fusion: } \hat{\mathbf{y}} &= h(\text{Transformer}([E_v(\mathbf{x}_v); E_a(\mathbf{x}_a); E_t(\mathbf{x}_t)])), \end{aligned}$$

and **cross-attention** lets queries from one stream attend to keys and values of another. Joint self-attention over all tokens computes $(N_v + N_a + N_t)^2$ scores per head per layer.

Ablations. For a model h and inputs $(\mathbf{x}_v, \mathbf{x}_a)$, compare $h(\mathbf{x}_v, \mathbf{x}_a)$ with $h(\mathbf{x}_v, \emptyset)$, $h(\emptyset, \mathbf{x}_a)$, $h(\mathbf{x}_v, \mathbf{x}'_a)$ (audio from another clip) and $h(\pi(\mathbf{x}_v), \mathbf{x}_a)$ (frames permuted).

Circular error (degrees): $d_o(a, b) = \min(|a - b| \bmod 360, 360 - (|a - b| \bmod 360))$.

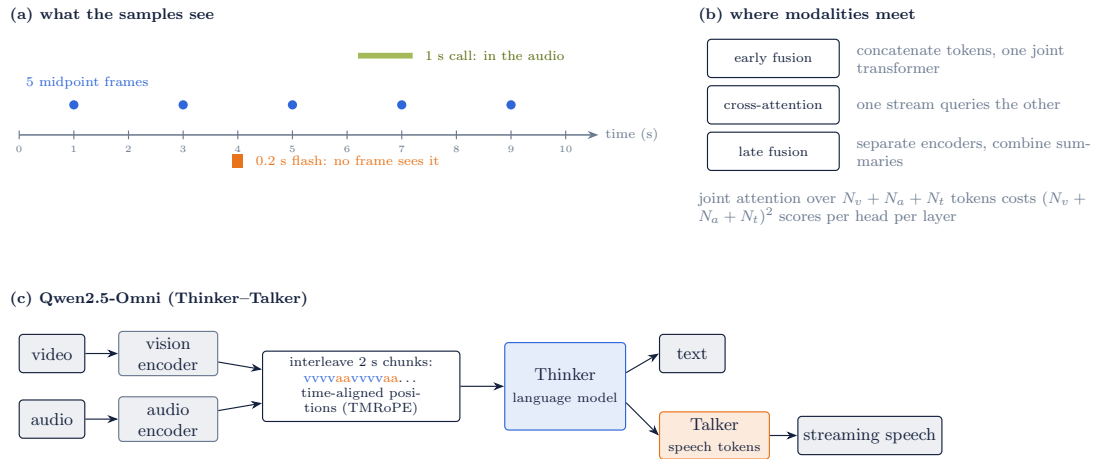


Figure 21.1: (a) Midpoint sampling of five frames misses a 0.2 s flash that the audio stream would contain. (b) Three places where modalities can meet. (c) Qwen2.5-Omni, redrawn after the architecture figure of Xu et al. (2025): modality encoders, audio and video tokens interleaved in time-aligned chunks, a Thinker that writes text and a Talker that produces streaming speech. Paper: [arXiv:2503.20215](https://arxiv.org/abs/2503.20215).

21.5 Derivation, with reasons for each step

Step 1 — sampling is an observation model. A video model never sees “the video”; it sees F frames at chosen times. A one-frame event in a 50-frame clip is in a sample of F midpoint frames with probability $F/50$ if its time is uniformly random. Counting two such events correctly requires seeing both: probability about $(F/50)^2$. In the lab, flash-count accuracy is 0.371, 0.425, 0.548 and 0.964 for $F = 5, 10, 25, 50$ (chance 0.333). Increasing F raises cost linearly in tokens and quadratically in joint attention; adaptive sampling (more frames around motion or sound) and event-window detection are the alternatives. When a model fails, first ask whether the evidence was sampled at all.

Step 2 — audio representations and what they keep. Raw waveforms keep everything at high cost (16,000 samples per second at 16 kHz). A **log-mel spectrogram**

— short-time Fourier transform, magnitudes pooled into mel-spaced frequency bands, then a logarithm — keeps the spectral envelope over time and discards phase; it is the standard input of speech and sound models. Learned discrete codes compress further. **Stereo** carries direction through interaural level and time differences; averaging channels to mono removes them. In the lab, a model trained on mono audio recognises calls perfectly (1.000) but locates them at chance (0.517); the stereo model locates them at 0.999. A transcript keeps words and drops tone, overlap, non-speech sounds and direction.

Step 3 — time alignment. If audio and video tokens are simply concatenated, position indices say nothing about *when*. Models that must relate a sound to a frame need positions that encode absolute time — Qwen2.5-Omni’s time-aligned rotary positions (TMRoPE) give audio and video tokens from the same moment related positions, and interleave the streams in short chunks so that attention can link co-occurring evidence. Offsets between streams (a few hundred milliseconds of lip-sync error) are a real failure mode; test robustness by shifting one stream.

Step 4 — fusion choices and their cost. **Early fusion** (one transformer over all tokens) lets every token interact, at $O((N_v + N_a + N_t)^2)$ cost: for 1,568 video tokens (chapter 17), 750 audio tokens and 64 text tokens, about 5.7 million scores per head per layer. **Cross-attention** costs $O(N_q N_k)$ per layer between two streams. **Late fusion** encodes each stream separately and combines pooled summaries: cheapest, and blind to fine-grained alignment (which sound goes with which object). Compression — resamplers, token merging, lower frame rates — trades detail for cost.

Step 5 — ablations as controlled interventions. Zeroing a modality asks “what does the model do with an input it has never seen?” In the lab’s late-fusion model, zeroing the audio drops flash-count accuracy from 0.964 to 0.356 — although flashes are purely visual — because a fused representation with an all-zero audio half is far from anything seen in training. The ablation measured the model’s fragility, not the audio’s contribution to counting. Better-designed interventions:

- **Mismatched modality** (audio from another clip): each audio track is still a real one, so the *marginal* distribution of audio is preserved, but the *joint* distribution of audio and video is broken: the pair is one the model may never have seen, and in real data it can be physically inconsistent (a bee on screen, a hoverfly on the track). The intervention removes the correct evidence without inventing an unnatural signal, which makes it a useful diagnostic, but it is not guaranteed to be in distribution, and a model that relies on cross-modal agreement may react to the inconsistency itself. In the lab, call accuracy falls to 0.268 and side to 0.508, while object and flashes are unaffected (1.000, 0.963). For this late-fusion model, whose audio and video questions are generated independently, that is a clear attribution; for models that fuse modalities early, or for data with correlated modalities, read it with the caveat above.
- **Shuffled frames**: tests whether order matters; here it does not (all scores unchanged), as expected for questions about identity and counts.
- **Train for the intervention**: with **modality dropout** (randomly zeroing a modality for 30% of training batches), zeroing audio at test time leaves flash counting intact (0.919) while call questions fall to chance — now the ablation measures what audio contributes.

Additivity does not hold: “video-only 60%, audio-only 55%, full 68%” does not mean audio contributes 8 points on each example; break results down by question type and by which modality contains the evidence.

Step 6 — empty-modality conventions. A zero vector, a learned “missing” token, silence, a black frame, and masking the modality’s tokens out of attention are different inputs with different effects. Declare which is used, train with it if missing inputs are expected in deployment, and do not compare results obtained with different conventions.

Step 7 — spatial and temporal outputs need their own metrics. For directions, absolute difference fails at the wrap-around: 179° and -179° are 2° apart, not 358° . For positions, state units and reference frame (camera-relative left is not world-relative left once the camera moves). For temporal answers, score event windows by temporal IoU (chapter 13’s IoU on intervals) rather than exact timestamps. Parse numeric answers by a published rule (chapter 20).

Step 8 — evidence-annotated evaluation. Annotate each test question with the modality and time window containing its answer. Then a wrong answer can be classified as *evidence not sampled*, *evidence sampled but misread*, or *answer not grounded*. A correct answer to a question whose evidence was not sampled reveals a prior. This bookkeeping is what turns a leaderboard number into an understanding of the system.

21.6 Numerical worked example

Missing a brief event. $D = 10$ s, $F = 5$: frames at 1, 3, 5, 7, 9 s. A 0.2 s flash at 4.0 s falls between frames 3 and 5 and is not observed; the audio (if the flash makes a sound) or a denser sample would contain it.

Probability of capture. One-frame flashes in 50 frames, $F = 10$: each flash is sampled with probability 0.2; both of two flashes with probability 0.04. Without seeing them, the best guess of the count is the prior.

Joint attention size. $N_v = 1568$, $N_a = 750$, $N_t = 64$: $(2382)^2 = 5,673,924$ scores per head per layer; at 16 heads and 32 layers, about 2.9 billion scores per example.

Circular error. $d_o(350, 10) = \min(340, 20) = 20$; $d_o(179, -179) = \min(358, 2) = 2$.

Reading an ablation table. Question-only 40%, video-only 60%, audio-only 55%, full 68%; mismatched audio 67%. The full model is better than video-only, but nearly all of the gain survives *wrong* audio — so the gain does not come from correctly aligned audio evidence (perhaps from audio statistics, or from more parameters being active).

21.7 Small implementation and checks

The lab `labs/ch21-omni/omni.py` builds a late-fusion model of sampled frames and stereo audio windows with four answer heads:

```
Vs, idx = sample_frames(V, F)                                # midpoint
→ sampling of F of 50 frames
Rv = relu(Vs @ Wv + bv); Ra = relu(A @ Wa + ba)              # per-frame
→ and per-window encoders
zv = np.c_[Rv.mean(1), Rv.sum(1) / 10]                       # mean and
→ sum pooling (sum can count)
za = np.concatenate([Ra.mean(2), Ra.sum(2) / 10], -1).reshape(m, -1) #
→ per channel: keeps stereo
h = relu(np.c_[zv, za] @ W1 + b1); logits = {q: h @ O[q] + c[q] for q in
→ heads}
```


A mono model averages the two channels before encoding; modality dropout zeroes one modality’s pooled vector for 30% of training batches.

Checks, in `test_omni.py`: midpoint times and sampled indices, circular error; a gradient check for stereo and mono models; and, after training, mismatched audio removes call accuracy but not object accuracy, mono audio loses the side, and five frames lose the flash count.

21.8 Experiment and expected observations

Run `python labs/ch21-omni/omni.py` (seed 0; 4,000 training clips, 1,000 Adam steps; 2,000 test clips). Chance: object 0.167, call 0.250, side 0.500, flashes 0.333.

model / intervention	object	call	side	flashes
full input	1.000	0.999	0.999	0.964
audio zeroed	0.940	0.231	0.515	0.356
video zeroed	0.160	0.788	0.979	0.319
mismatched audio	1.000	0.268	0.508	0.963
shuffled frames	1.000	0.999	0.999	0.964
mono-audio	1.000	1.000	0.516	0.957
model				
modality- dropout model, full	1.000	0.999	0.999	0.915
modality- dropout model, audio zeroed	1.000	0.253	0.502	0.919

frames sampled (of 50)	5	10	25	50
flash-count accuracy	0.371	0.425	0.548	0.964

What to notice:

1. **Zeroing a modality is not a neutral removal** for a model never trained without it.
2. **Mismatched inputs isolate evidence here**: exactly the audio questions fail. This relies on the lab’s independent audio and video and on late fusion; with correlated modalities the mismatch also creates inconsistent pairs.
3. **Representation choices delete information**: mono cannot localise however good the model.
4. **Sampling sets a ceiling**: no model counts flashes it never saw.
5. **Shuffling frames changes nothing here** — a correct null result for order-free questions, and a reminder that a temporal control only tests temporal claims.

21.9 Moving pieces: diagnosing and fixing

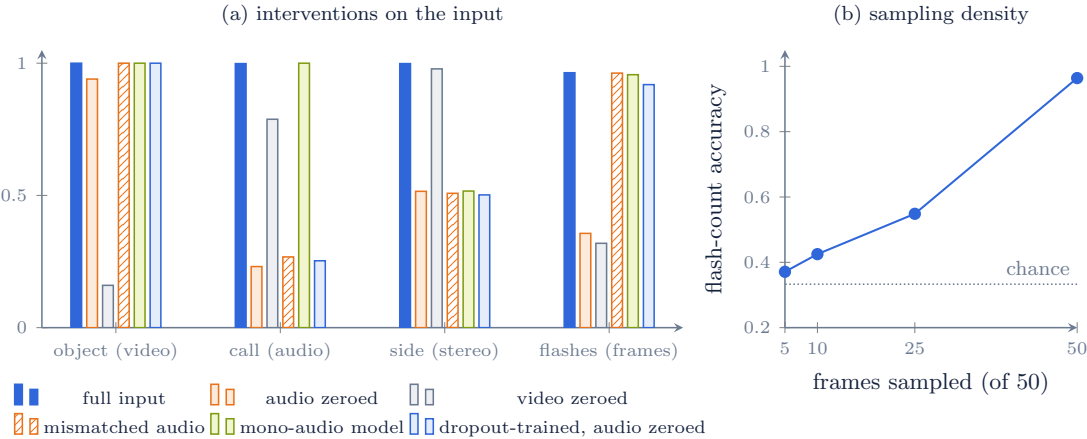


Figure 21.2: (a) Accuracy per question under interventions: zeroing audio also breaks the purely visual flash count unless the model was trained with modality dropout; mismatched audio removes exactly the audio answers in this late-fusion model; mono audio loses direction. (b) Flash counting needs dense sampling.

Moving piece	Symptom when it is wrong	How to confirm	Fix
Frame sampling	brief events missed; counts wrong	evidence-annotated errors; accuracy vs. F	denser or adaptive sampling; event detection; audio cues
Audio representation	direction, tone or non-speech sounds lost	probes for side, speaker, sound class	stereo/multichannel input; spectrograms rather than transcripts
Time alignment	sounds attributed to wrong objects or moments	shift one stream by ± 0.5 s	time-aligned positions; interleaved chunks
Fusion point	cannot link which sound goes with which object	targeted questions needing alignment	cross-attention or early fusion for fine-grained questions
Token budget	out of memory; truncated clips	count $N_v + N_a + N_t$	resampling, token merging, lower fps, shorter windows
Missing modality	collapse of unrelated answers when a stream is absent	zero/blank tests	modality dropout; learned missing tokens; masks
Ablation design	contributions misattributed	mismatched-input and shuffled controls	report interventions with controls, per question type
Priors	correct answers without evidence	question-only baseline; evidence annotations	balance questions; report evidence-conditional accuracy

Moving piece	Symptom when it is wrong	How to confirm	Fix
Spatial metrics	huge errors at wrap-around	check angle near $\pm 180^\circ$	circular error; declared reference frames
Parsing	numbers and directions misread	audit raw outputs	published parser; units; invalid-answer category

First things to try: (1) annotate where and when the evidence is for 100 test questions; (2) run mismatched-modality and question-only controls; (3) sweep the number of frames; (4) check what preprocessing does to audio channels.

21.10 Application, limitations and failure cases

Application (educational case study): an offline video-understanding study. For the greenhouse recordings, assemble 300 one-minute clips with questions annotated by modality and time window: species (video), buzz type (audio), side (stereo), sprayer events (video, brief). Evaluate a public omni-modal model, recording its frame-sampling and audio preprocessing settings, and a strong video-only VLM with a transcript of the audio. Report accuracy by question type, the mismatched-audio control, and whether failures on brief events coincide with events between sampled frames. Re-run with denser sampling restricted to annotated windows (an oracle) to separate “could not see” from “could not understand” — and report it as an intervention, not as the system’s ability.

Limitations:

- More modalities add more ways for priors and shortcuts to enter; they do not guarantee more understanding.
- Large omni-modal models are evaluated mostly on public benchmarks whose evidence windows are rarely annotated.
- Ablations on models never trained with missing inputs measure robustness, not contribution.
- The lab’s late-fusion model cannot test fine-grained alignment between specific sounds and specific objects.

21.11 Bridge to a research paper

Xu and colleagues (2025), *Qwen2.5-Omni technical report* ([arXiv:2503.20215](https://arxiv.org/abs/2503.20215)).

- **Problem:** build one model that perceives text, images, audio and video, and answers in text and natural speech, in a streaming fashion.
- **Mechanism:** modality-specific encoders for vision and audio processed block-wise; audio and video tokens interleaved in time-aligned chunks with **TMRoPE** positions that encode absolute time; a **Thinker** (a language model) that produces text and hidden representations; a **Talker** that turns them into streaming speech tokens, decoded to audio with a sliding-window decoder.
- **Central claims:** strong results across text, image, audio and video benchmarks, competitive with single-modality models of similar size, with real-time speech output; strong results on omni-modal benchmarks that require combining modalities.

- **Assumptions to question:** how much of the “omni-modal” benchmark performance requires *aligned* audio and video (mismatched controls are rarely reported); which frame and audio sampling settings were used for each benchmark; how evaluation questions were answered when the evidence fell between sampled frames.
- **Affordable experiment:** in the lab, add a fifth question that genuinely requires alignment — “on which side is the *object that flashed* when the call happened?” (flashes and calls co-occur on the same side for half the clips) — and compare late fusion with a model that fuses per-time-step features (concatenate frame and audio-window features at each time before pooling). Use mismatched audio and time-shifted audio as controls.

21.12 Memory map



Figure 21.3: Memory map of chapter 21.

Summary. Each modality reaches a model through choices — frame sampling, audio

representation, channels, timestamps — that decide which evidence exists at all: brief events between sampled frames and directions in averaged audio are gone before any learning. Modalities meet by early fusion, cross-attention or late fusion, with costs that grow with the square of the total number of tokens under joint attention; time-aligned positions let models relate co-occurring evidence. Claims about what a multimodal model uses require interventions with controls: zeroing a modality is out of distribution unless the model was trained with modality dropout, while mismatched inputs and shuffled frames help isolate specific evidence, at the cost of creating cross-modal pairs that may not occur in real data. Spatial and temporal outputs need circular, frame-aware and interval metrics, and evaluation sets should record where and when each answer’s evidence lies.

21.13 Exercises

1. *Conceptual.* What is the circular error between 350° and 10° ?
2. *Conceptual.* Why can a transcript-only baseline differ from an audio-only baseline?
3. *Conceptual.* If giving the model an oracle event window improves accuracy, what has been learned?
4. *Derivation.* For F uniformly spaced frames and an event of duration δ at a uniformly random time in $[0, D]$, derive the probability that at least one frame falls inside the event.
5. *Derivation.* A stereo source at angle θ reaches two microphones a distance d apart. Write the interaural time difference and explain why averaging channels removes it.
6. *Derivation.* Compare the attention cost of early fusion with that of late fusion plus one cross-attention layer, for $N_v = 1568$, $N_a = 750$, $N_t = 64$.
7. *Implementation.* Add a time-shift control to the lab: delay the audio by k windows and plot call and side accuracy against k .
8. *Implementation.* Carry out the experiment of section 21.11.
9. *Interpretation.* An omni-modal model scores 68% with audio and 67% with mismatched audio. What do you conclude, and what would you test next?
10. *Open question.* How should a benchmark be built so that improvements can be attributed to multimodal *integration* rather than to any single modality?

21.13.1 Answer discussions

1. 20° .
2. Transcripts omit non-verbal sounds, tone, overlap and direction, and add recognition errors.
3. That better event selection helps *under that intervention*; not that the deployed model can find the window.
4. $\min(1, \delta F/D)$ for spacing $D/F \geq \delta$: the event covers a frame time if its start falls within δ before one of the F times.
5. $\Delta t \approx d \sin \theta / c$ (speed of sound c); averaging the two channels yields one signal with no inter-channel difference to measure.
6. Early: $(2382)^2 \approx 5.7$ M scores; late fusion with cross-attention: within-stream $1568^2 + 750^2 + 64^2 \approx 3.0$ M plus cross terms $64 \cdot (1568 + 750) \approx 0.15$ M.
7. Expect accuracy to fall once the shift moves the call outside the window that the model associates with it — for a late-fusion model, perhaps not at all, which is itself informative.
8. Per-time fusion should solve the alignment question; late fusion should be near chance on it.

9. Almost none of the gain depends on the *correct* audio; test question-only and audio-statistics baselines, and break results down by question type.
10. Include questions whose answers require combining modalities (neither alone suffices), with evidence annotations, and report single-modality and mismatched-modality controls for each.

21.14 Self-check and common mistakes

- ☐ I can say what a given frame-sampling and audio-preprocessing choice can and cannot observe.
- ☐ I can compare fusion strategies and count their attention cost.
- ☐ I can design modality ablations with mismatched and shuffled controls.
- ☐ I can explain why zeroing a modality may be out of distribution, and what modality dropout changes.
- ☐ I can compute circular error and choose metrics for spatial and temporal answers.

Common mistakes: not recording frame sampling and audio preprocessing; averaging stereo to mono and then asking about direction; reading “video-only vs. full” differences as per-example contributions; zeroing modalities in models never trained without them; omitting mismatched-input controls; using absolute differences for angles; treating an oracle-window result as a system capability; claiming temporal understanding from tasks that need none.

References

- I. Goodfellow, Y. Bengio and A. Courville. *Deep Learning*. MIT Press, 2016, section 12.3 (speech recognition).
- J. Ngiam, A. Khosla, M. Kim, J. Nam, H. Lee and A. Y. Ng. Multimodal deep learning. In *ICML*, 2011.
- Y. Aytar, C. Vondrick and A. Torralba. SoundNet: learning sound representations from unlabeled video. In *NeurIPS*, 2016.
- R. Arandjelović and A. Zisserman. Look, listen and learn. In *ICCV*, 2017.
- J. F. Gemmeke et al. Audio Set: an ontology and human-labeled dataset for audio events. In *ICASSP*, 2017.
- N. Neverova, C. Wolf, G. Taylor and F. Nebout. ModDrop: adaptive multi-modal gesture recognition. *IEEE TPAMI*, 38(8):1692–1706, 2016.
- A. Baevski, H. Zhou, A. Mohamed and M. Auli. wav2vec 2.0: a framework for self-supervised learning of speech representations. In *NeurIPS*, 2020.
- A. Jaegle et al. Perceiver: general perception with iterative attention. In *ICML*, 2021.
- A. Radford et al. Robust speech recognition via large-scale weak supervision (Whisper). In *ICML*, 2023.
- R. Girdhar et al. ImageBind: one embedding space to bind them all. In *CVPR*, 2023.
- J. Xu et al. Qwen2.5-Omni technical report. arXiv:2503.20215, 2025.

Chapter 22

Diffusion in pixel and latent spaces

Prerequisites: chapter 9 (KL divergence, the ELBO); chapter 10 (VAEs, latent diffusion as a two-stage model); chapter 11 (training a network by hand).

Learning outcomes: after this chapter you can

1. write the forward noising process, its closed-form marginal and the reverse-time posterior, and explain why a learned reverse chain can generate data;
2. derive the simplified noise-prediction loss from the ELBO, and relate the optimal noise predictor to the score $\nabla_{\mathbf{x}} \log p_t(\mathbf{x})$;
3. implement DDPM ancestral sampling and deterministic DDIM sampling, and explain the trade-off between number of steps, cost and sample quality;
4. derive classifier-free guidance as sampling from a tilted distribution, and predict how it trades diversity for condition adherence;
5. explain what latent diffusion changes and what it does not, and diagnose common failures (blurry few-step samples, off-manifold drift, lost diversity, memorisation).

22.1 A concrete problem

A plant-pathology group has 40,000 photographs of healthy leaves and a few hundred of a rare disease. They want a model that can **generate** new, varied, realistic images — of leaves in general, and of the rare disease on request — to study what the classifier of chapter 12 has learned and, cautiously, to augment training data.

Chapter 10 met two difficulties. A model that maximises likelihood with a simple output distribution covers every mode but puts mass *between* them — blurry averages. A model that only has to fool a discriminator (a GAN) produces sharp samples but may drop modes. The data has many modes (leaf shapes, lighting, disease stages), each sharp.

Diffusion models take a third route. Destroying structure is easy: add a little Gaussian noise, many times, until only noise remains. If we can learn to undo **one small step** of that destruction — a denoising problem, solved by regression — then chaining the learned steps from pure noise produces data. Each step is nearly Gaussian, so it is easy to model; the chain as a whole can represent a distribution with many sharp modes. The price is cost: generating one sample takes tens to hundreds of network evaluations. This chapter derives the method, builds it in two dimensions where every number can be checked against an exact answer, and studies the two knobs practitioners turn most: the sampler and the guidance weight.

22.2 How we got here

Score matching and denoising (2005–2011). Hyvärinen (2005) showed how to fit the **score** $\nabla_{\mathbf{x}} \log p(\mathbf{x})$ of an unnormalised model without its normalising constant. Vincent (2011) showed that training a denoising autoencoder on Gaussian-corrupted data is equivalent to matching the score of the corrupted distribution. Tweedie’s formula (Robbins, 1956; Efron, 2011) had long related the best denoiser to that score.

Diffusion as a latent-variable model (2015). Sohl-Dickstein and colleagues (2015), inspired by non-equilibrium thermodynamics, defined a fixed forward chain that slowly turns data into noise and trained a reverse chain by maximising a variational bound — the framework of this chapter, though samples were then far from competitive.

Two lines meet (2019–2021). Song and Ermon (2019) trained one network to estimate scores at many noise levels and sampled with annealed Langevin dynamics. **Ho, Jain and Abbeel (2020)** — this chapter’s paper bridge — reparameterised the diffusion model to predict the added noise, simplified the bound to a mean-squared error, and produced image samples competitive with GANs. Song and colleagues (2021) unified both views as stochastic differential equations, with a deterministic “probability-flow” counterpart.

Faster and better samplers (2021–2022). DDIM (Song, Meng and Ermon, 2021) gave a deterministic sampler that skips steps. Nichol and Dhariwal (2021) introduced the cosine noise schedule and learned variances; Dhariwal and Nichol (2021) used a classifier’s gradient to steer samples, and Ho and Salimans (2021) obtained the same effect without a classifier — **classifier-free guidance**. Karras and colleagues (2022) separated the design choices (schedule, preconditioning, sampler) cleanly.

Scale (2022–). **Latent diffusion** (Rombach and colleagues, 2022; chapter 10) ran the process in an autoencoder’s latent space, making high-resolution text-to-image generation affordable. Imagen (Saharia and colleagues, 2022) introduced dynamic thresholding of the predicted image under strong guidance. Diffusion transformers (Peebles and Xie, 2023) replaced the U-Net with a ViT. Flow matching (Lipman and colleagues, 2023) and consistency models (Song and colleagues, 2023) reframed or distilled the process for fewer steps. Diffusion models also became world simulators for video (chapter 23).

22.3 Assumptions and notation

Symbol	Meaning
$\mathbf{x}_0 \sim p_{\text{data}}$	a data point (an image, a latent code, or a point in the plane in the lab)
$t = 1, \dots, T$	diffusion step; $T = 100$ in the lab, 1,000 in Ho et al. (2020)
$\beta_t, \alpha_t = 1 - \beta_t$	per-step noise variance and its complement
$\bar{\alpha}_t = \prod_{s \leq t} \alpha_s$	cumulative signal fraction; $\text{SNR}_t = \bar{\alpha}_t / (1 - \bar{\alpha}_t)$
$\epsilon \sim \mathcal{N}(\epsilon; \mathbf{0}, \mathbf{I})$	the noise added to make $\mathbf{x}_t$
$\hat{\epsilon}_{\theta}(\mathbf{x}_t, t, c)$	network predicting ϵ ; c a condition or $\emptyset$ (none)
$\hat{\mathbf{x}}_0$	predicted clean sample, computed from $\mathbf{x}_t$ and $\hat{\epsilon}$
p_t	marginal density of $\mathbf{x}_t$; $\nabla_{\mathbf{x}} \log p_t$ its score

Symbol	Meaning
w	guidance weight; $w = 0$ is plain conditional sampling

Assumptions and declarations:

1. **The forward process is fixed**, Gaussian and Markov; only the reverse process is learned.
2. **Each reverse step is modelled as Gaussian** with a fixed variance. This is accurate when steps are small; it is the reason few-step sampling needs care.
3. **Evaluation of samples is itself a model**. In the lab we can compare samples with the known data distribution; for images, metrics such as FID depend on a feature extractor and sample size, and must be declared.

22.4 Mathematical formulation

Forward process. Starting from data, add noise in T small steps:

$$q(\mathbf{x}_t | \mathbf{x}_{t-1}) = \mathcal{N}(\mathbf{x}_t; \sqrt{\alpha_t} \mathbf{x}_{t-1}, \beta_t \mathbf{I}).$$

Because Gaussians compose, any step can be reached in one jump:

$$q(\mathbf{x}_t | \mathbf{x}_0) = \mathcal{N}(\mathbf{x}_t; \sqrt{\alpha_t} \mathbf{x}_0, (1 - \bar{\alpha}_t) \mathbf{I}), \quad \mathbf{x}_t = \sqrt{\alpha_t} \mathbf{x}_0 + \sqrt{1 - \bar{\alpha}_t} \boldsymbol{\epsilon}.$$

The schedule is chosen so that $\bar{\alpha}_T \approx 0$: $\mathbf{x}_T$ is almost exactly standard normal, whatever the data. The lab uses the **cosine schedule** $\bar{\alpha}_t = f(t)/f(0)$ with $f(t) = \cos^2(\frac{t/T+s}{1+s} \frac{\pi}{2})$, $s = 0.008$.

Reverse process. A learned chain runs backwards from noise:

$$p_\theta(\mathbf{x}_{t-1} | \mathbf{x}_t) = \mathcal{N}(\mathbf{x}_{t-1}; \boldsymbol{\mu}_\theta(\mathbf{x}_t, t), \sigma_t^2 \mathbf{I}), \quad p(\mathbf{x}_T) = \mathcal{N}(\mathbf{x}_T; \mathbf{0}, \mathbf{I}).$$

Training objective (Ho et al., 2020). With the noise parameterisation below, the training loss is

$$L_{\text{simple}}(\boldsymbol{\theta}) = \mathbb{E}_{\mathbf{x}_0, t \sim \mathcal{U}\{1..T\}, \boldsymbol{\epsilon}} \left[\left\| \boldsymbol{\epsilon} - \hat{\boldsymbol{\epsilon}}_\theta(\sqrt{\alpha_t} \mathbf{x}_0 + \sqrt{1 - \bar{\alpha}_t} \boldsymbol{\epsilon}, t) \right\|^2 \right].$$

Predicted clean sample. Inverting the forward equation with the predicted noise:

$$\hat{\mathbf{x}}_0 = \frac{\mathbf{x}_t - \sqrt{1 - \bar{\alpha}_t} \hat{\boldsymbol{\epsilon}}_\theta(\mathbf{x}_t, t)}{\sqrt{\alpha_t}}.$$

Classifier-free guidance. Train one network with the condition c replaced by $\emptyset$ for a fraction of examples (0.2 in the lab); sample with

$$\tilde{\boldsymbol{\epsilon}} = (1 + w) \hat{\boldsymbol{\epsilon}}_\theta(\mathbf{x}_t, t, c) - w \hat{\boldsymbol{\epsilon}}_\theta(\mathbf{x}_t, t, \emptyset).$$

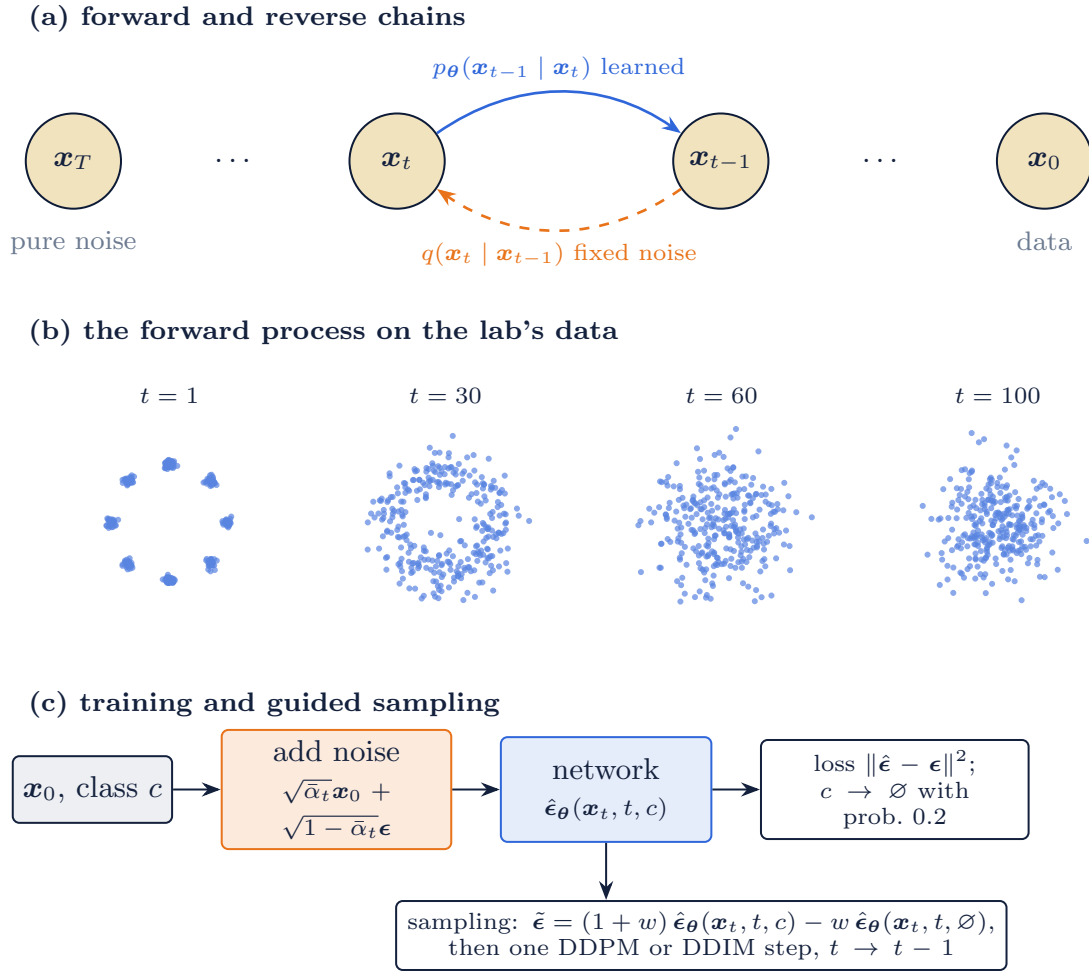


Figure 22.1: (a) The diffusion model as two chains, redrawn after Fig. 2 of Ho, Jain and Abbeel (2020): a fixed forward chain adds noise; a learned reverse chain removes it. (b) The forward process on the lab's eight-mode data: by $t = 60$ the modes are gone, and $\mathbf{x}_{100}$ is standard normal. (c) Training (predict the added noise; drop the condition 20% of the time) and guided sampling. Paper: [arXiv:2006.11239](https://arxiv.org/abs/2006.11239).

22.5 Derivation, with reasons for each step

Step 1 — the one-jump marginal. Write $\mathbf{x}_t = \sqrt{\alpha_t}\mathbf{x}_{t-1} + \sqrt{\beta_t}\boldsymbol{\epsilon}_t$ and substitute $\mathbf{x}_{t-1}$ from the step before: $\mathbf{x}_t = \sqrt{\alpha_t\alpha_{t-1}}\mathbf{x}_{t-2} + \sqrt{\alpha_t\beta_{t-1}}\boldsymbol{\epsilon}_{t-1} + \sqrt{\beta_t}\boldsymbol{\epsilon}_t$. The two noise terms are independent Gaussians, so their sum is Gaussian with variance $\alpha_t(1 - \alpha_{t-1}) + (1 - \alpha_t) = 1 - \alpha_t\alpha_{t-1}$. By induction, $\mathbf{x}_t = \sqrt{\alpha_t}\mathbf{x}_0 + \sqrt{1 - \alpha_t}\boldsymbol{\epsilon}$. *Why it matters:* training never simulates the chain; it samples a random t and jumps straight there, so each training example costs one network call.

Step 2 — the reverse posterior is Gaussian when $\mathbf{x}_0$ is known. The true reverse step $q(\mathbf{x}_{t-1} | \mathbf{x}_t)$ involves the unknown data distribution, but conditioning also on $\mathbf{x}_0$ gives, by Bayes' rule on Gaussians (chapter 6),

$$q(\mathbf{x}_{t-1} | \mathbf{x}_t, \mathbf{x}_0) = \mathcal{N}(\mathbf{x}_{t-1}; \tilde{\boldsymbol{\mu}}_t, \tilde{\beta}_t \mathbf{I}), \quad \tilde{\boldsymbol{\mu}}_t = \frac{\sqrt{\alpha_{t-1}}\beta_t}{1 - \bar{\alpha}_t}\mathbf{x}_0 + \frac{\sqrt{\alpha_t}(1 - \bar{\alpha}_{t-1})}{1 - \bar{\alpha}_t}\mathbf{x}_t, \quad \tilde{\beta}_t = \frac{1 - \bar{\alpha}_{t-1}}{1 - \bar{\alpha}_t}\beta_t.$$

Substituting $\mathbf{x}_0 = (\mathbf{x}_t - \sqrt{1 - \bar{\alpha}_t}\boldsymbol{\epsilon})/\sqrt{\bar{\alpha}_t}$ and simplifying gives the mean in terms of the noise: $\tilde{\boldsymbol{\mu}}_t = \frac{1}{\sqrt{\alpha_t}}(\mathbf{x}_t - \frac{\beta_t}{\sqrt{1 - \bar{\alpha}_t}}\boldsymbol{\epsilon})$. This motivates parameterising the model mean the same way, with $\hat{\boldsymbol{\epsilon}}_\theta$ in place of $\boldsymbol{\epsilon}$.

Step 3 — the ELBO becomes a sum of noise-prediction errors. Treat $\mathbf{x}_{1:T}$ as latent variables (chapter 9). The evidence lower bound on $\log p_\theta(\mathbf{x}_0)$ decomposes into a prior term (no parameters, because $q(\mathbf{x}_T | \mathbf{x}_0) \approx \mathcal{N}(\mathbf{x}_T; \mathbf{0}, \mathbf{I})$), a reconstruction term at $t = 1$, and for each $t \geq 2$ the term $\text{KL}(q(\mathbf{x}_{t-1} | \mathbf{x}_t, \mathbf{x}_0) \| p_\theta(\mathbf{x}_{t-1} | \mathbf{x}_t))$. Both distributions are Gaussian; with fixed variance σ_t^2 the KL is $\frac{1}{2\sigma_t^2}\|\tilde{\boldsymbol{\mu}}_t - \boldsymbol{\mu}_\theta\|^2$ plus a constant, and with the parameterisation of step 2,

$$\text{KL}_t = \frac{\beta_t^2}{2\sigma_t^2 \alpha_t(1 - \bar{\alpha}_t)} \|\boldsymbol{\epsilon} - \hat{\boldsymbol{\epsilon}}_\theta(\mathbf{x}_t, t)\|^2 + \text{const.}$$

Ho et al. dropped the weights in front and sampled t uniformly — L_{simple} . It is no longer the exact bound; it down-weights the small- t terms relative to the bound, which they found gives better samples. *Lesson:* the loss used in practice is a reweighted ELBO, and the weighting is a design choice (Kingma and colleagues, 2021, and Karras et al., 2022, study it).

Step 4 — the optimal noise predictor is the score. Under squared error the best predictor is the conditional mean, $\hat{\boldsymbol{\epsilon}}^*(\mathbf{x}_t, t) = \mathbb{E}[\boldsymbol{\epsilon} | \mathbf{x}_t]$. For $\mathbf{x}_t = \sqrt{\alpha_t}\mathbf{x}_0 + \sqrt{1 - \alpha_t}\boldsymbol{\epsilon}$, differentiating $p_t(\mathbf{x}_t) = \int q(\mathbf{x}_t | \mathbf{x}_0)p(\mathbf{x}_0)d\mathbf{x}_0$ gives Tweedie's formula,

$$\mathbb{E}[\mathbf{x}_0 | \mathbf{x}_t] = \frac{\mathbf{x}_t + (1 - \bar{\alpha}_t)\nabla_{\mathbf{x}} \log p_t(\mathbf{x}_t)}{\sqrt{\bar{\alpha}_t}}, \quad \hat{\boldsymbol{\epsilon}}^*(\mathbf{x}_t, t) = -\sqrt{1 - \bar{\alpha}_t} \nabla_{\mathbf{x}} \log p_t(\mathbf{x}_t).$$

So a noise predictor is a scaled **score** estimator: it points from $\mathbf{x}_t$ away from where data is likely. This is the bridge between the variational view (Ho et al.) and the score view (Song and Ermon). For a Gaussian mixture, p_t is itself a Gaussian mixture with means $\sqrt{\alpha_t}\boldsymbol{\mu}_k$ and variances $\bar{\alpha}_t s^2 + 1 - \bar{\alpha}_t$, so the score — and hence the **exact** optimal denoiser — is available in closed form. The lab uses it to test samplers separately from learning.

Step 5 — the predicted clean sample is an average. Read Tweedie's formula again: $\hat{\mathbf{x}}_0$ from the optimal denoiser is $\mathbb{E}[\mathbf{x}_0 | \mathbf{x}_t]$, the *posterior mean*. At high noise the posterior over $\mathbf{x}_0$ spreads over all modes and its mean is a blurry average — for the lab's ring of

modes, near the origin, where there is no data. A sampler that jumped from $\mathbf{x}_T$ to $\hat{\mathbf{x}}_0$ in one step would output that average. Diffusion avoids it by taking many small steps: each step commits a little, the posterior narrows, and the final $\hat{\mathbf{x}}_0$ is a mean over a single mode. *This is why steps matter*: with the exact denoiser, DDIM with two steps gives precision 0.000 in the lab — every sample lands at the centre.

Step 6 — DDPM ancestral sampling. Sample $\mathbf{x}_T \sim \mathcal{N}(\mathbf{x}_T; \mathbf{0}, \mathbf{I})$, then for $t = T, \dots, 1$:

$$\mathbf{x}_{t-1} = \frac{1}{\sqrt{\alpha_t}} \left(\mathbf{x}_t - \frac{\beta_t}{\sqrt{1 - \alpha_t}} \hat{\boldsymbol{\epsilon}}_{\theta}(\mathbf{x}_t, t) \right) + \sigma_t \mathbf{z}, \quad \mathbf{z} \sim \mathcal{N}(\mathbf{z}; \mathbf{0}, \mathbf{I}),$$

with $\sigma_t^2 = \tilde{\beta}_t$ (the lab) or β_t , and no noise at the last step. Cost: T network evaluations per sample. The injected noise has a useful side effect: an error in one step is partly washed out by the next steps, which see fresh noise and pull the sample back toward regions the network was trained on.

Step 7 — DDIM: a deterministic shortcut. Song, Meng and Ermon (2021) observed that L_{simple} only constrains the marginals $q(\mathbf{x}_t | \mathbf{x}_0)$, and many non-Markov forward processes share them. Choosing the deterministic one gives the update from t to any earlier t' :

$$\mathbf{x}_{t'} = \sqrt{\bar{\alpha}_{t'}} \hat{\mathbf{x}}_0 + \sqrt{1 - \bar{\alpha}_{t'}} \hat{\boldsymbol{\epsilon}}_{\theta}(\mathbf{x}_t, t) :$$

estimate the clean sample and the noise, then re-noise to the lower level using the *same* noise direction. The same trained network serves, with any subsequence of steps, so 10–50 calls replace 1,000. It is a discretisation of the probability-flow ODE, so it maps each $\mathbf{x}_T$ to one $\mathbf{x}_0$ (useful for editing and interpolation) — and, being deterministic, it does not wash out errors: inaccurate $\hat{\mathbf{x}}_0$ estimates at large t , where dividing by $\sqrt{\bar{\alpha}_t}$ amplifies errors in $\hat{\boldsymbol{\epsilon}}$, propagate to the end. **Clipping** $\hat{\mathbf{x}}_0$ to the data range and recomputing $\hat{\boldsymbol{\epsilon}}$ from it is a simple guard (Imagen’s dynamic thresholding is the adaptive version).

Step 8 — guidance as a tilted distribution. By Bayes’ rule $\nabla \log p(\mathbf{x} | c) = \nabla \log p(\mathbf{x}) + \nabla \log p(c | \mathbf{x})$. Classifier guidance (Dhariwal and Nichol) adds a scaled classifier gradient, $w \nabla \log p(c | \mathbf{x})$. Classifier-free guidance obtains the classifier gradient implicitly as $\nabla \log p(\mathbf{x} | c) - \nabla \log p(\mathbf{x})$, from one network trained with and without the condition. Since $\hat{\boldsymbol{\epsilon}} \propto -\nabla \log p_t$,

$$(1 + w) \nabla \log p(\mathbf{x} | c) - w \nabla \log p(\mathbf{x}) = \nabla \log [p(\mathbf{x} | c) p(c | \mathbf{x})^w],$$

the score of a distribution that up-weights points where the condition is *more certain*. Points that are compatible with several conditions lose mass: adherence rises, diversity falls. In the lab, training labels are only 75% pure (a quarter of the points from class-1 modes are labelled class 0), so $p(\mathbf{x} | c = 0)$ puts mass 0.75 on class-0 modes and 0.25 on class-1 modes, while $p(\mathbf{x})$ is uniform over modes. Because all modes have the same shape, the tilted distribution has the same modes with mass ratio $(0.75/0.25)^{1+w} = 3^{1+w}$, predicting adherence $3^{1+w}/(1 + 3^{1+w})$: 0.75, 0.90, 0.988 for $w = 0, 1, 3$. The lab measures 0.712, 0.891, 0.988. Two caveats. The guided noise estimate at each level is *not* exactly the score of the tilted distribution’s noisy marginals, so guided sampling only approximates it; at $w = 8$ samples start to leave the modes (precision 0.989 instead of 0.998 at $w = 3$, radial spread 0.080 instead of 0.071). And on images, the “modes” that lose mass are less typical compositions, styles and subjects — guidance trades diversity for fidelity to the prompt.

Step 9 — latent diffusion. Nothing above requires $\mathbf{x}$ to be pixels. Latent diffusion (chapter 10) trains an autoencoder first, then runs diffusion on its codes: a $512 \times 512 \times 3$ image (786,432 numbers) becomes a $64 \times 64 \times 4$ latent (16,384 numbers), 48 times smaller, and the denoising network spends its capacity on content rather than imperceptible detail. Two consequences: sample fidelity is capped by the decoder, and the latent’s statistics (scale, correlations) must be normalised so that the noise schedule means the same thing as in pixel space.

Step 10 — conditioning in large models. Text conditions enter through cross-attention from image tokens to text-encoder tokens (chapter 14), timesteps through embeddings added to every block, and spatial conditions (edges, depth, masks) through extra input channels or adapter networks. The guidance formula is unchanged; the null condition is the empty prompt.

22.6 Numerical worked example

Forward and back. One coordinate, $x_0 = 2$, $\bar{\alpha}_t = 0.64$, $\epsilon = 1$: $x_t = 0.8 \cdot 2 + 0.6 \cdot 1 = 2.2$. A perfect noise prediction $\hat{\epsilon} = 1$ recovers $\hat{x}_0 = (2.2 - 0.6 \cdot 1)/0.8 = 2.000$; an error of 0.5 ($\hat{\epsilon} = 0.5$) gives $\hat{x}_0 = (2.2 - 0.3)/0.8 = 2.375$. The error in $\hat{x}_0$ is the error in $\hat{\epsilon}$ multiplied by $\sqrt{1 - \bar{\alpha}_t}/\sqrt{\bar{\alpha}_t} = 0.75$ — and that factor grows without bound as $\bar{\alpha}_t \rightarrow 0$.

Schedule. The lab’s cosine schedule with $T = 100$: $\bar{\alpha}_1 = 0.9994$, $\bar{\alpha}_{50} = 0.494$ (signal-to-noise ratio 0.976 — the halfway point is where signal and noise are balanced), $\bar{\alpha}_{100} = 2.4 \times 10^{-7}$. At $t = T$ the factor $\sqrt{1 - \bar{\alpha}_t}/\sqrt{\bar{\alpha}_t}$ is about 2,000.

Guidance odds. Labels 75% pure: unguided odds of a class-0 mode are 3 : 1 (adherence 0.75). With $w = 1$, odds $3^2 = 9$, adherence 0.9; with $w = 3$, odds 81, adherence 0.988; with $w = 8$, odds 19,683, adherence above 0.9999 — in theory.

Latent compression. $512 \cdot 512 \cdot 3 = 786,432$; $64 \cdot 64 \cdot 4 = 16,384$; ratio 48. A U-Net step on the latent is roughly proportionately cheaper; the number of steps is unchanged.

Cost of a sample. DDPM with $T = 1000$: 1,000 network calls. DDIM with 50 steps: 50 calls. With classifier-free guidance: twice as many (conditional and unconditional), unless the two are batched. A VAE decoder or GAN generator: one call.

22.7 Small implementation and checks

The lab `labs/ch22-diffusion/diffusion.py` uses eight Gaussian modes (standard deviation 0.08) on a circle of radius 2; alternate modes belong to classes 0 and 1. The training step is the whole of L_{simple} :

```
t = rng.integers(1, T + 1, batch) # a random step
→ for each example
eps = rng.standard_normal((batch, 2))
xt = np.sqrt(ABAR[t - 1])[:, None] * x0 + np.sqrt(1 - ABAR[t - 1])[:,
→ None] * eps
c = np.where(rng.random(batch) < 0.2, NULL, labels) # drop the
→ condition 20% of the time
loss = np.mean(np.sum((net(p, xt, t, c) - eps) ** 2, 1)) # predict the
→ noise
```

and the two samplers are a few lines each:

```

# DDPM: T steps, fresh noise each step
x = (x - BETA[t-1] / np.sqrt(1 - ABAR[t-1]) * e) / np.sqrt(1 - BETA[t-1])
  ↪ + sigma_t * z
# DDIM: any subsequence of steps, deterministic
x0 = (x - np.sqrt(1 - ABAR[t-1]) * e) / np.sqrt(ABAR[t-1])
x = np.sqrt(ABAR[t_prev-1]) * x0 + np.sqrt(1 - ABAR[t_prev-1]) * e

```

The network is a two-hidden-layer MLP (128 units) taking $\mathbf{x}_t$, a sinusoidal embedding of t and a one-hot condition (class 0, class 1 or null), trained with Adam for 6,000 steps. Metrics: **precision** (fraction of samples within 0.3 of a mode centre), **coverage** (entropy of the histogram of nearest modes, divided by $\log 8$; 1 means all modes equally used), the **energy distance** to held-out data, and **class adherence** (fraction of samples near a mode of the requested class).

Checks, in `test_diffusion.py`: the schedule decreases from about 1 to about 0 and the inversion of step 1 is exact; the closed-form denoiser equals $-\sqrt{1 - \bar{\alpha}_t}$ times a finite-difference score; a gradient check of the network; the exact denoiser with DDPM and 25-step DDIM reproduces the data, while 5-step DDIM does not; after training, DDPM samples are precise and guidance raises adherence.

22.8 Experiment and expected observations

Run `python labs/ch22-diffusion/diffusion.py` (seed 0; 20,000 training points; 2,000 samples per row). The exact denoiser rows test the samplers alone; the learned rows add estimation error.

method	network calls	precision	coverage	energy distance
data (held out)	—	0.999	1.000	0.004
single Gaussian	—	0.067	0.982	0.092
(maximum likelihood)				
exact denoiser, DDPM	100	1.000	1.000	0.003
exact denoiser, DDIM	5 / 10 / 25	0.674 / 0.969 / 0.999	0.999	0.011 / 0.002 / 0.001
learned, DDPM	100	0.978	0.999	0.004
learned, DDIM	5 / 10 / 25	0.305 / 0.275 / 0.495	0.94 / 0.90 / 0.93	0.249 / 0.162 / 0.092
learned, DDIM with $\hat{\mathbf{x}}_0$ clipped to $[-2.5, 2.5]$	5 / 10 / 25	0.731 / 0.848 / 0.905	0.94 / 0.98 / 0.99	0.043 / 0.026 / 0.009

The training loss falls from 1.09 to 0.66. It does not fall to zero, and should not: at large t the noise cannot be predicted from $\mathbf{x}_t$, and the minimum of L_{simple} is the average conditional variance of ϵ given $\mathbf{x}_t$.

Guidance (class 0 requested, labels 75% pure, DDPM sampling):

w	0	1	3	8
class adherence	0.712	0.891	0.988	0.989

w	0	1	3	8
predicted $3^{1+w}/(1+3^{1+w})$	0.750	0.900	0.988	1.000
precision	0.975	0.986	0.998	0.989
coverage (all 8 modes)	0.946	0.815	0.689	0.647

Coverage cannot exceed $\log 4 / \log 8 = 0.667$ once only the four class-0 modes are used; at $w = 8$ it has reached that level, and the extra weight only pushes samples off the modes.

What to notice:

1. **Mass-covering is not sample quality.** The maximum-likelihood Gaussian has high coverage but only 6.7% of its samples near data.
2. **Separate the sampler from the model.** The exact denoiser shows what each sampler can do with perfect scores; the gap to the learned rows is estimation error. Without that control, “DDIM needs 100 steps here” would be misread as a property of DDIM.
3. **Deterministic samplers expose denoiser errors.** DDPM’s injected noise corrects; DDIM inherits every error. A cheap guard (clipping) recovers most of the loss.
4. **Guidance is a dial between diversity and adherence**, and its effect is predictable when the label noise is known.

22.9 Moving pieces: diagnosing and fixing

Moving piece	Symptom when it is wrong	How to confirm	Fix
Noise schedule	samples noisy (end too early) or wasted steps; poor detail at high resolution	plot $\bar{\alpha}_t$ and SNR; check $\bar{\alpha}_T \approx 0$	cosine schedule; shift the schedule toward more noise for larger images or latents
Loss weighting	good likelihood but poor samples, or the reverse	compare L_{simple} with the true ELBO	choose weighting deliberately; report which
Parameterisation	unstable training at very low or high t	loss per t bucket	ϵ -, $\mathbf{x}_0$ - or velocity-prediction; preconditioning
Timestep conditioning	samples ignore the noise level; mush at the end	evaluate the denoiser at fixed t against an oracle (2-D) or held-out loss per t	sinusoidal embeddings into every block
Number of steps / sampler	blurry or averaged samples; streaks	sweep steps; exact-denoiser control on toy data	more steps; higher-order solvers; stochastic sampler; distillation
$\hat{\mathbf{x}}_0$ range	saturated colours, samples outside data range	histogram of $\hat{\mathbf{x}}_0$ during sampling	clipping or dynamic thresholding

Moving piece	Symptom when it is wrong	How to confirm	Fix
Guidance weight	low diversity, oversaturated or off-manifold samples at high w ; ignored prompts at low w	adherence and diversity against w	choose w on a validation curve; guidance only on some steps
Condition dropout	guidance has no effect or is unstable	rate of $\emptyset$ in training	10–20% null-condition training
Latent autoencoder	fine detail and text are wrong whatever the diffusion model does	reconstruct real images through the autoencoder	better autoencoder; more latent channels
Evaluation	FID changes with sample count or feature extractor	report both; precision and recall separately	fixed protocol; human study for claims about quality
Memorisation	samples near-duplicate training images	nearest-neighbour search in pixel and feature space	deduplicate data; more data; report retrieval rates

First things to try: (1) on a toy version of the problem, compare the learned denoiser with an exact one; (2) plot loss per timestep; (3) sweep steps and guidance together and report precision-like and diversity-like metrics for each; (4) search the training set for nearest neighbours of generated samples.

22.10 Application, limitations and failure cases

Application (educational case study): synthetic examples of a rare class. For the leaf images, train a latent diffusion model (or fine-tune a public one with appropriate licence) conditioned on disease label, holding out a third of the rare-disease images entirely. Evaluate in three ways: (i) **realism and diversity** — precision and recall in a declared feature space, plus expert review of 100 samples per class without knowing which are real; (ii) **memorisation** — for every generated rare-class image, the nearest training image in pixel and feature space, flagging near-duplicates; (iii) **usefulness** — train the chapter 12 classifier with and without synthetic images and test on the held-out real rare-disease images, with confidence intervals (chapter 5). Report the guidance weight and sampler with every result, because they change all three answers. A gain from synthetic data is a claim about this classifier on this test set, not about the disease.

Limitations:

- Sampling is slow compared with one-shot generators; few-step methods trade quality, diversity or training cost.
- Guidance improves adherence by reducing diversity; synthetic sets made at high w under-represent atypical cases — often the ones augmentation was meant to supply.
- Diffusion models can reproduce training images (Carlini and colleagues, 2023), which matters for privacy, consent and licensing.
- Sample metrics such as FID depend on a feature extractor trained on other data; they do not measure correctness of details that matter to experts.

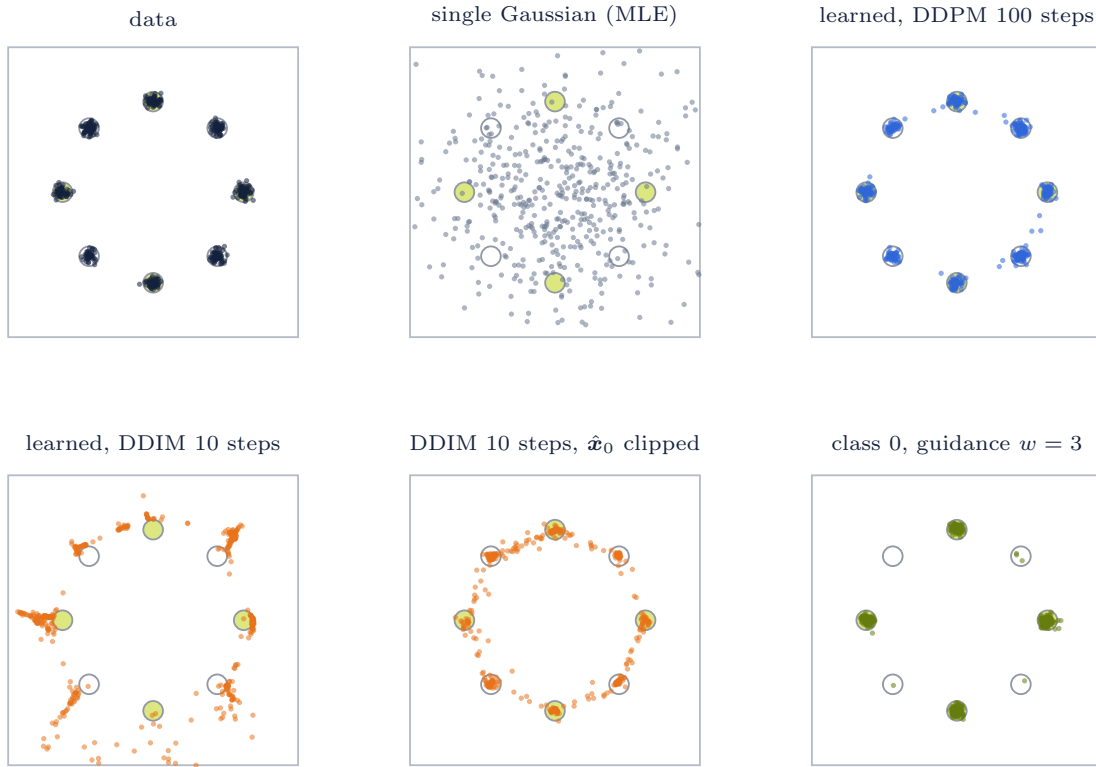


Figure 22.2: Five hundred samples from each method. The single Gaussian covers the modes and fills the space between them; DDPM sampling from the learned denoiser reproduces the sharp modes; 10-step DDIM with the same network scatters samples along streaks (errors from early, high-noise steps that are never corrected); clipping $\hat{x}_0$ removes most of them. With guidance $w = 3$, samples requested for class 0 land on class-0 modes (filled circles).

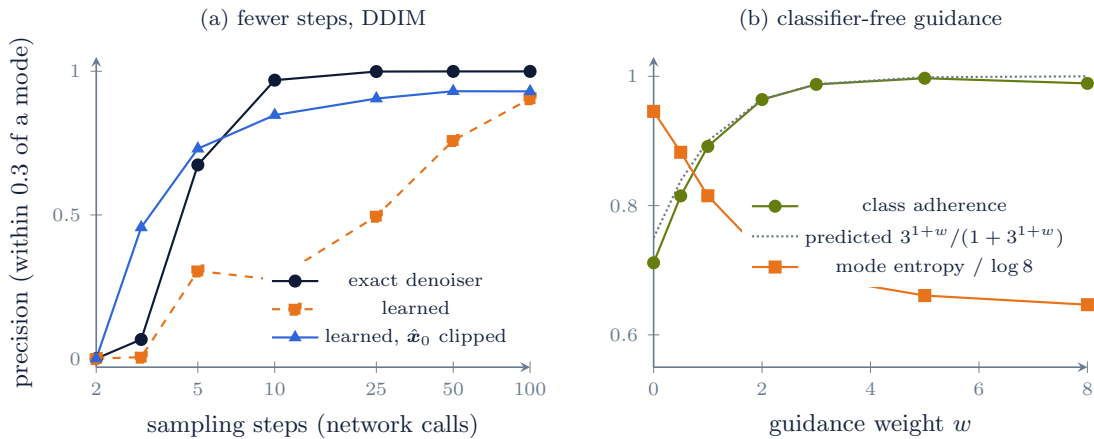


Figure 22.3: (a) Precision against the number of DDIM steps. With the exact denoiser, two steps give the posterior mean (the centre of the ring) and ten steps are nearly perfect; the learned denoiser needs far more steps unless $\hat{x}_0$ is clipped. (b) Classifier-free guidance follows the predicted adherence up to $w \approx 3$, while mode entropy falls toward the four-mode limit.

- The lab’s two-dimensional model has no spatial structure; U-Nets and transformers add inductive biases that the toy cannot test.

22.11 Bridge to a research paper

Ho, Jain and Abbeel (2020), *Denoising diffusion probabilistic models*, NeurIPS (arXiv:2006.11239).

- **Problem:** make diffusion probabilistic models (Sohl-Dickstein et al., 2015) produce high-quality images.
- **Mechanism:** a fixed linear β schedule from 10^{-4} to 0.02 over $T = 1000$ steps; the reverse mean parameterised through a noise predictor (step 2); the simplified loss L_{simple} (step 3); a U-Net with self-attention at low resolution and sinusoidal timestep embeddings; ancestral sampling with fixed variances.
- **Central claims:** on CIFAR-10, an Inception score of 9.46 and FID of 3.17, state of the art among unconditional models at the time; high-quality 256×256 LSUN samples; the noise-prediction objective connects diffusion models to denoising score matching with annealed Langevin sampling; log-likelihoods are competitive but not the best, and most lossless codelength goes to imperceptible detail.
- **Assumptions to question:** that FID on 50,000 samples measures what matters; that the simplified loss, chosen for sample quality, is the right objective when likelihood or calibration matters; that 1,000 steps are necessary (later work showed they are not, and DDIM uses the same trained model).
- **Affordable experiment:** in the lab, replace the cosine schedule with the paper’s linear schedule rescaled to $T = 100$ and compare (a) the per- t loss, (b) DDPM precision and coverage, and (c) DDIM precision against steps with the exact denoiser. Then train with the ELBO weights of step 3 instead of L_{simple} , and report which objective gives better samples.

22.12 Memory map

Summary. A diffusion model fixes a forward chain that adds Gaussian noise until data becomes noise, and learns the reverse chain. Because noised data has a closed-form marginal, training is regression: predict the added noise from the noisy input and the step, a reweighted form of the ELBO. The optimal noise predictor is the scaled score of the noisy data distribution, and the predicted clean sample is a posterior mean — which is why one-step generation gives averages and many small steps give sharp samples. DDPM sampling re-injects noise and forgives errors; DDIM is deterministic and skips steps, but exposes errors unless the predicted clean sample is kept in range. Classifier-free guidance samples approximately from $p(\mathbf{x} | c) p(c | \mathbf{x})^w$, raising adherence and lowering diversity in a predictable way. Latent diffusion runs the same process on autoencoder codes, which makes high resolution affordable and caps fidelity at the decoder.

22.13 Exercises

1. *Conceptual.* Why can training sample a random t and never simulate the forward chain?
2. *Conceptual.* Why does a one-step “denoise from pure noise” model produce blurry outputs, even with a perfect denoiser?

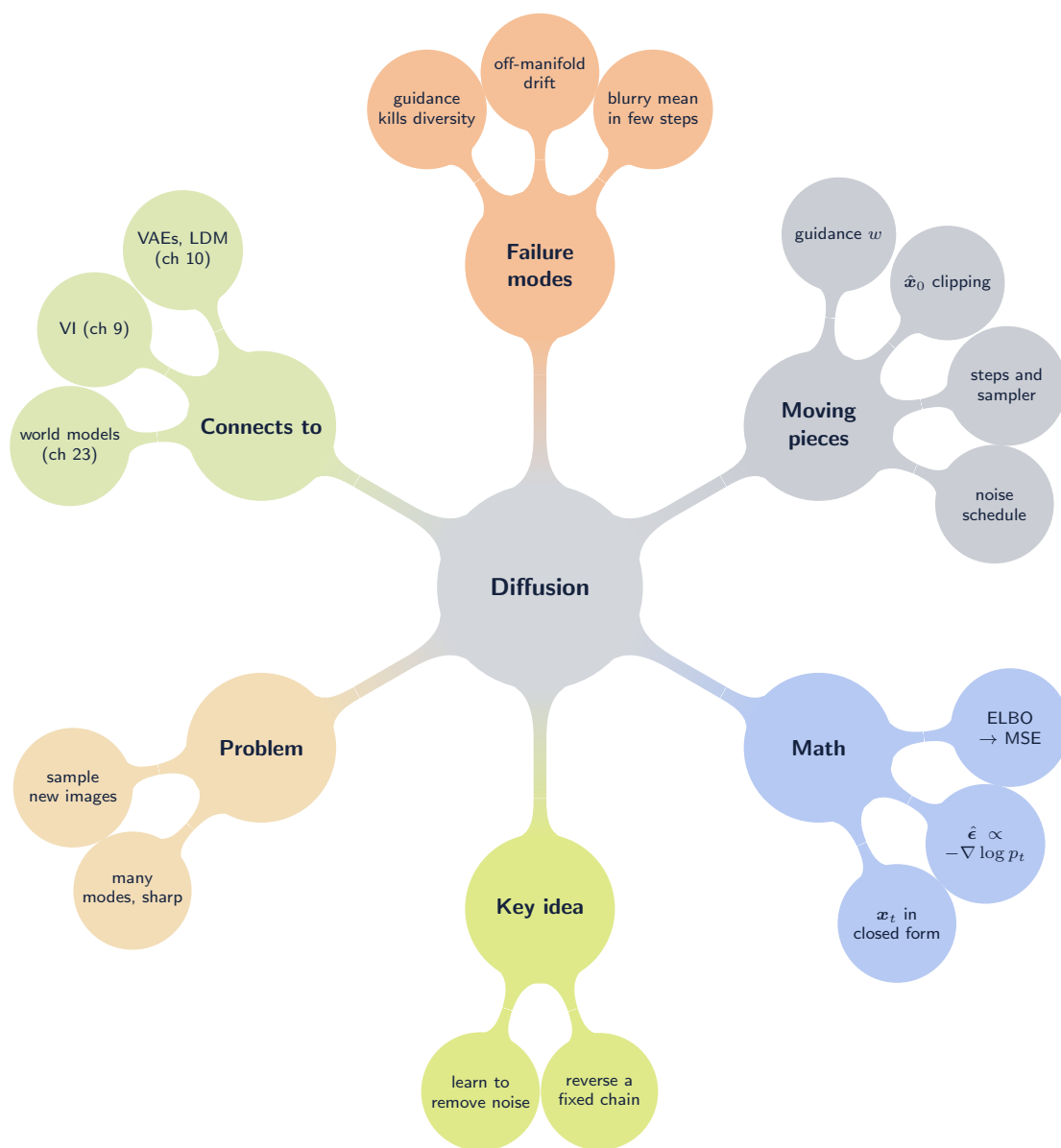


Figure 22.4: Memory map of chapter 22.

3. *Conceptual.* What does classifier-free guidance cost at sampling time, compared with unguided sampling?
4. *Derivation.* Show that $\mathbf{x}_t = \sqrt{\alpha_t}\mathbf{x}_{t-1} + \sqrt{\beta_t}\epsilon_t$ for all t implies $\mathbf{x}_t = \sqrt{\alpha_t}\mathbf{x}_0 + \sqrt{1 - \alpha_t}\epsilon$.
5. *Derivation.* Derive Tweedie's formula $\mathbb{E}[\mathbf{x}_0 | \mathbf{x}_t] = (\mathbf{x}_t + (1 - \bar{\alpha}_t)\nabla \log p_t(\mathbf{x}_t))/\sqrt{\bar{\alpha}_t}$ by differentiating $p_t(\mathbf{x}_t) = \int \mathcal{N}(\mathbf{x}_t; \sqrt{\alpha_t}\mathbf{x}_0, (1 - \bar{\alpha}_t)\mathbf{I}) p(\mathbf{x}_0) d\mathbf{x}_0$.
6. *Derivation.* With labels of purity ρ and two balanced classes of equally shaped modes, show that classifier-free guidance with weight w gives adherence $\rho^{1+w}/(\rho^{1+w} + (1 - \rho)^{1+w})$, under the assumption that guided sampling reaches the tilted distribution.
7. *Implementation.* Carry out the experiment of section 22.11.
8. *Implementation.* Add stochasticity to DDIM ($\eta > 0$, Song et al., 2021) and plot learned-model precision against η for 10 and 25 steps.
9. *Interpretation.* A team reports that its new sampler beats DDIM at 10 steps. What control would you ask for before believing the sampler, rather than the network, is responsible?
10. *Open question.* Guidance makes outputs match prompts better and makes them less diverse. How would you decide the right weight for generating training data, as opposed to generating images for people to look at?

22.13.1 Answer discussions

1. The marginal $q(\mathbf{x}_t | \mathbf{x}_0)$ is Gaussian in closed form, so $\mathbf{x}_t$ can be drawn directly from $\mathbf{x}_0$.
2. The squared-error-optimal output is the posterior mean $\mathbb{E}[\mathbf{x}_0 | \mathbf{x}_T]$, an average over all modes; many small steps let the posterior narrow to one mode before the mean is taken.
3. Two network evaluations per step (conditional and unconditional), usually batched together; the same number of steps.
4. Induction: the sum of two independent zero-mean Gaussians with variances $\alpha_t(1 - \bar{\alpha}_{t-1})$ and $1 - \alpha_t$ has variance $1 - \alpha_t\bar{\alpha}_{t-1} = 1 - \bar{\alpha}_t$.
5. $\nabla p_t = \int \mathcal{N}(\mathbf{x}_t; \sqrt{\alpha_t}\mathbf{x}_0, (1 - \bar{\alpha}_t)\mathbf{I}) \frac{\sqrt{\alpha_t}\mathbf{x}_0 - \mathbf{x}_t}{1 - \bar{\alpha}_t} p(\mathbf{x}_0) d\mathbf{x}_0$; divide by p_t to get $\nabla \log p_t = (\sqrt{\bar{\alpha}_t}\mathbb{E}[\mathbf{x}_0 | \mathbf{x}_t] - \mathbf{x}_t)/(1 - \bar{\alpha}_t)$ and rearrange.
6. Tilted mass on a correct mode is proportional to $\rho^{1+w}(1/2)^{-w}$, on a wrong one to $(1 - \rho)^{1+w}(1/2)^{-w}$; normalise. For $\rho = 0.75$ this gives $3^{1+w}/(1 + 3^{1+w})$.
7. Expect the linear schedule, designed for $T = 1000$, to destroy signal too quickly at $T = 100$ (most steps near pure noise), which hurts few-step DDIM most; the ELBO weighting emphasises small t and may give sharper but less well-placed samples.
8. Precision should improve with η for the learned model at a fixed number of steps, because injected noise corrects errors; with the exact denoiser it should change little.
9. The same comparison with an exact or much better denoiser (on a toy problem), and the same network and step budget for both samplers; ideally, sampling quality against steps for both.
10. For training data, measure the downstream classifier on real held-out data across w , including per-subgroup errors; for people, measure preference and prompt adherence. The weights are likely to differ, and lower w is often better for data.

22.14 Self-check and common mistakes

- ☐ I can write $q(\mathbf{x}_t | \mathbf{x}_0)$, the reverse posterior and the predicted clean sample from memory.
- ☐ I can derive L_{simple} from the ELBO and say what weighting was dropped.
- ☐ I can explain the relation between noise prediction, denoising and the score.

- ☐ I can implement DDPM and DDIM sampling and explain why they behave differently with an imperfect network.
- ☐ I can predict the effect of the guidance weight on adherence and diversity.

Common mistakes: evaluating a sampler only with a learned network; comparing samplers at different numbers of network calls; forgetting that guided sampling doubles the calls; reporting FID without sample count and feature extractor; reading a lower training loss at large t as a better model (the minimum is not zero); using a schedule tuned for $T = 1000$ with far fewer steps; claiming novelty of samples without nearest-neighbour checks; choosing the guidance weight by looking at a few images.

References

- I. Goodfellow, Y. Bengio and A. Courville. *Deep Learning*. MIT Press, 2016, chapter 20 (deep generative models).
- H. Robbins. An empirical Bayes approach to statistics. In *Proc. Third Berkeley Symposium*, 1956; B. Efron. Tweedie’s formula and selection bias. *JASA*, 106(496):1602–1614, 2011.
- A. Hyvärinen. Estimation of non-normalized statistical models by score matching. *JMLR*, 6:695–709, 2005.
- P. Vincent. A connection between score matching and denoising autoencoders. *Neural Computation*, 23(7):1661–1674, 2011.
- J. Sohl-Dickstein, E. Weiss, N. Maheswaranathan and S. Ganguli. Deep unsupervised learning using nonequilibrium thermodynamics. In *ICML*, 2015.
- Y. Song and S. Ermon. Generative modeling by estimating gradients of the data distribution. In *NeurIPS*, 2019.
- J. Ho, A. Jain and P. Abbeel. Denoising diffusion probabilistic models. In *NeurIPS*, 2020.
- Y. Song, J. Sohl-Dickstein, D. P. Kingma, A. Kumar, S. Ermon and B. Poole. Score-based generative modeling through stochastic differential equations. In *ICLR*, 2021.
- J. Song, C. Meng and S. Ermon. Denoising diffusion implicit models. In *ICLR*, 2021.
- A. Nichol and P. Dhariwal. Improved denoising diffusion probabilistic models. In *ICML*, 2021.
- P. Dhariwal and A. Nichol. Diffusion models beat GANs on image synthesis. In *NeurIPS*, 2021.
- D. P. Kingma, T. Salimans, B. Poole and J. Ho. Variational diffusion models. In *NeurIPS*, 2021.
- J. Ho and T. Salimans. Classifier-free diffusion guidance. *NeurIPS Workshop on Deep Generative Models*, 2021; arXiv:2207.12598.
- T. Karras, M. Aittala, T. Aila and S. Laine. Elucidating the design space of diffusion-based generative models. In *NeurIPS*, 2022.
- R. Rombach, A. Blattmann, D. Lorenz, P. Esser and B. Ommer. High-resolution image synthesis with latent diffusion models. In *CVPR*, 2022.
- C. Saharia et al. Photorealistic text-to-image diffusion models with deep language understanding (Imagen). In *NeurIPS*, 2022.
- W. Peebles and S. Xie. Scalable diffusion models with transformers. In *ICCV*, 2023.
- Y. Lipman, R. T. Q. Chen, H. Ben-Hamu, M. Nickel and M. Le. Flow matching for generative modeling. In *ICLR*, 2023.
- Y. Song, P. Dhariwal, M. Chen and I. Sutskever. Consistency models. In *ICML*, 2023.

- N. Carlini et al. Extracting training data from diffusion models. In *USENIX Security*, 2023.

Part VI

Entering research

Chapter 23

Geometry and world-model introduction

Prerequisites: chapter 1 (matrices, projections); chapter 2 (SVD, PCA); chapter 17 (JEPA, video representations).

Learning outcomes: after this chapter you can

1. write the pinhole camera model with intrinsics and pose, project points, and triangulate them from two views;
2. explain why monocular reconstruction has a scale ambiguity and why stereo depth error grows with the square of depth;
3. separate a latent world model into representation, prediction and planning, and design a diagnostic for each;
4. plan with a learned predictor using the cross-entropy method in a receding-horizon loop, and explain why the geometry of the latent space, not only its information, decides whether planning works;
5. read action-conditioned world-model papers (V-JEPA 2-AC) critically.

23.1 A concrete problem

A greenhouse robot must move a camera-mounted arm to a plant shown in a **goal photograph**. It has no map and no labelled coordinates — only images from its own camera, and the commands it sends to its motors. Two questions are hidden in this task.

The first is **geometric**: how do pixels relate to positions in the world? A camera turns 3-D points into 2-D pixels; depth is lost, and must be recovered from motion, from a second view, or from learned priors. How precisely, and at what range?

The second is about **world models**: can the robot learn, from its own experience, what its actions do to what it sees — and then use that knowledge to choose actions that make the current view look like the goal? Recent work (V-JEPA 2-AC, this chapter’s paper bridge) does exactly this with a frozen video encoder and a learned action-conditioned predictor.

This chapter introduces both, with one discipline throughout: break the system into parts — the camera model, the representation, the predictor, the planner — and test each part separately, so that a failure can be attributed. The lab makes the point concrete: a representation can contain all the information needed and still make planning fail.

23.2 How we got here

Projective geometry (Renaissance to 1990s). Perspective drawing (Brunelleschi, Alberti) and the camera obscura gave the pinhole model. Photogrammetry made it quantitative. In computer vision, Longuet-Higgins (1981) gave the eight-point algorithm for relative pose from point matches; Tsai (1987) and Zhang (2000) gave practical camera calibration; Hartley and Zisserman’s *Multiple View Geometry* (2000, 2004) systematised the field.

Reconstruction at scale (2006–2020). Structure from motion recovered cameras and sparse 3-D points from unordered photo collections (Snavely, Seitz and Szeliski, 2006, *Photo Tourism*; COLMAP, Schönberger and Frahm, 2016). Monocular depth networks learned depth from single images, with the scale ambiguity handled by supervision or scale-invariant losses (Eigen, Puhrsch and Fergus, 2014).

Neural scene representations (2020–2024). NeRF (Mildenhall and colleagues, 2020) fitted a network to a scene’s radiance field and rendered novel views by volume rendering; 3D Gaussian splatting (Kerbl and colleagues, 2023) made it fast. DUS3R (Wang and colleagues, 2024) regressed 3-D point maps directly from image pairs, with no calibration.

World models (1943–2025). Craik (1943) proposed that minds carry “small-scale models” of the world to try out actions. Model-based reinforcement learning learned dynamics and planned with them (Dyna, Sutton, 1991). Ha and Schmidhuber’s *World Models* (2018), PlaNet and Dreamer (Hafner and colleagues, 2019–2020) learned latent dynamics from pixels and planned or trained policies “in imagination”; MuZero (Schrittwieser and colleagues, 2020) planned with a model that predicts only what matters for value. LeCun (2022) argued for predicting in representation space rather than pixel space — the JEPA programme of chapter 17. DINO-WM (Zhou and colleagues, 2024) planned with a predictor over frozen DINOv2 features; **V-JEPA 2-AC** (Assran and colleagues, 2025) did so for real robot manipulation from a self-supervised video encoder. Video-generation models trained as world simulators (Genie, Bruce and colleagues, 2024; Cosmos, NVIDIA, 2025) took the pixel-space route (chapter 22).

23.3 Assumptions and notation

Symbol	Meaning
$\mathbf{x} \in \mathbb{R}^3$	a point in world coordinates; (X, Y, Z) in camera coordinates
$\mathbf{K}$	intrinsic matrix: focal length f (pixels), principal point (c_x, c_y)
$\mathbf{R}, \mathbf{t}$	camera pose: rotation and translation, world to camera
$\mathbf{P} = \mathbf{K}[\mathbf{R} \mid \mathbf{t}]$	3×4 camera matrix
B, d	stereo baseline (m) and disparity (pixels)
$\mathbf{s}_t, \mathbf{o}_t, \mathbf{a}_t$	true state, observation (image) and action at time t
$\mathbf{z}_t = E(\mathbf{o}_t)$	latent representation from an encoder E
$g(\mathbf{z}, \mathbf{a})$	action-conditioned predictor of the next latent
H	planning horizon (steps imagined per plan)

Assumptions and declarations:

1. **Pinhole cameras without lens distortion** in the geometry part; real lenses need a distortion model, estimated during calibration.
2. **The world is fully observed through one image** in the lab (the agent's position determines the image); real scenes have occlusion and hidden state.
3. **Goals are given as images**, so the planner needs no coordinates — but it then depends on the encoder's notion of similarity.

23.4 Mathematical formulation

Pinhole projection. In homogeneous coordinates,

$$\lambda \begin{bmatrix} u \\ v \\ 1 \end{bmatrix} = \mathbf{K} [\mathbf{R} \mid \mathbf{t}] \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}, \quad \mathbf{K} = \begin{bmatrix} f & 0 & c_x \\ 0 & f & c_y \\ 0 & 0 & 1 \end{bmatrix},$$

so that in camera coordinates $u = fX/Z + c_x$ and $v = fY/Z + c_y$; $\lambda = Z$ is the depth.

Two-view triangulation. Each view gives two linear equations in the homogeneous point $\tilde{\mathbf{x}}$, for example $u \mathbf{p}_3^\top \tilde{\mathbf{x}} - \mathbf{p}_1^\top \tilde{\mathbf{x}} = 0$ with $\mathbf{p}_i^\top$ the rows of $\mathbf{P}$. Stacking four equations in $\mathbf{A}\tilde{\mathbf{x}} = \mathbf{0}$ and taking the right singular vector of the smallest singular value (chapter 2) gives the **direct linear transform** (DLT) estimate. For a rectified stereo pair with baseline B : $Z = fB/d$.

Latent world model and planning. With encoder E , predictor g and a goal image $\mathbf{o}_g$, plan

$$\mathbf{a}_{t:t+H-1}^* = \arg \min_{\mathbf{a}_{t:t+H-1}} \sum_{h=1}^H \|\hat{\mathbf{z}}_{t+h} - E(\mathbf{o}_g)\|, \quad \hat{\mathbf{z}}_t = E(\mathbf{o}_t), \quad \hat{\mathbf{z}}_{t+h} = g(\hat{\mathbf{z}}_{t+h-1}, \mathbf{a}_{t+h-1}),$$

execute $\mathbf{a}_t^*$, observe, and plan again (**receding horizon**, or model-predictive control). The predictor is trained on logged transitions by $\min_g \mathbb{E} \|g(E(\mathbf{o}_t), \mathbf{a}_t) - E(\mathbf{o}_{t+1})\|^2$.

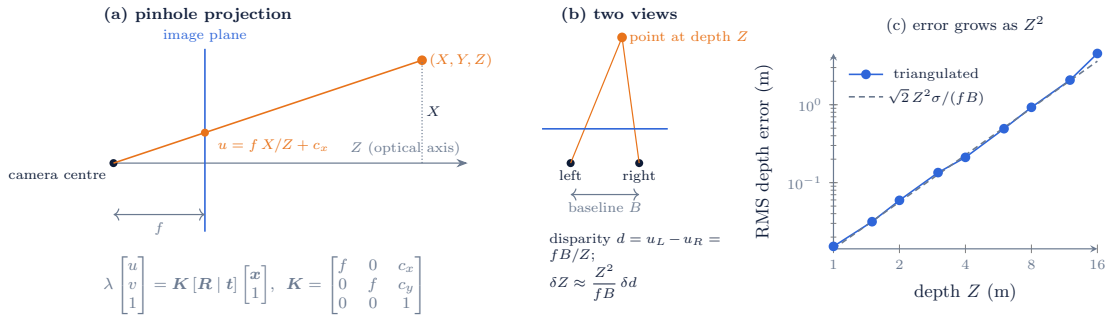


Figure 23.1: (a) The pinhole camera. (b) Two views with baseline B : depth is inversely proportional to disparity. (c) RMS depth error from triangulating pixels with 0.5-pixel noise ($f = 500$ px, $B = 0.1$ m), against the first-order prediction: error grows as Z^2 , and faster at long range where the linearisation fails.

23.5 Derivation, with reasons for each step

Step 1 — projection loses depth, and scale. $u = fX/Z + c_x$ is unchanged if (X, Z) are multiplied by the same factor: every point on a ray through the camera centre projects to the same pixel. With two images from a moving camera, the same holds for the whole scene: multiplying all points *and* the camera translation by k leaves every projection unchanged, because $\mathbf{R}(k\mathbf{x}) + k\mathbf{t} = k(\mathbf{R}\mathbf{x} + \mathbf{t})$ and the factor k cancels in the division by depth. A single moving camera therefore recovers structure **up to scale**; metric scale needs a known baseline, a known object size, another sensor, or a learned prior. In the lab, doubling the scene and the translation changes no pixel (maximum change 0.0).

Step 2 — triangulation is a null-space problem. Writing $\mathbf{P}\tilde{\mathbf{x}} = \lambda[u, v, 1]^\top$ and eliminating λ gives $u(\mathbf{p}_3^\top \tilde{\mathbf{x}}) = \mathbf{p}_1^\top \tilde{\mathbf{x}}$ and $v(\mathbf{p}_3^\top \tilde{\mathbf{x}}) = \mathbf{p}_2^\top \tilde{\mathbf{x}}$. Two views give $\mathbf{A} \in \mathbb{R}^{4 \times 4}$; with exact pixels, $\tilde{\mathbf{x}}$ spans its null space. With noise there is no exact null vector, and the minimiser of $\|\mathbf{A}\tilde{\mathbf{x}}\|$ over unit vectors is the last right singular vector — the same Rayleigh-quotient argument as PCA (chapter 2). The DLT minimises an algebraic error, not the pixel (reprojection) error; bundle adjustment refines it by minimising reprojection error over all points and cameras. In the lab, five noiseless points are recovered to 2×10^{-14} .

Step 3 — depth error grows as Z^2 . From $Z = fB/d$, $\partial Z/\partial d = -fB/d^2 = -Z^2/(fB)$. A disparity error δd therefore causes $\delta Z \approx Z^2 \delta d/(fB)$. Disparity is a difference of two pixel coordinates, each with noise σ , so δd has standard deviation $\sqrt{2}\sigma$. With $f = 500$ px, $B = 0.1$ m and $\sigma = 0.5$ px: 0.014 m at 1 m, 0.057 m at 2 m, 0.91 m at 8 m. The lab’s triangulated errors match up to 8 m (0.015, 0.059, 0.926) and exceed the prediction at 16 m (4.55 m against 3.62 m), where the disparity is only about 3 pixels and the linearisation of $1/d$ fails. *Consequences:* wider baselines and longer focal lengths help quadratically at range; depth from a small robot’s stereo pair is accurate only nearby; and learned depth networks inherit these limits wherever their training signal came from geometry.

Step 4 — a world model has three separable parts. For planning from pixels, the system must (i) **represent**: map an image to a latent that contains the state; (ii) **predict**: map a latent and an action to the next latent; (iii) **plan**: search for actions whose predicted latents approach the goal’s latent. Each can fail independently, and each has its own test:

- *representation*: can the state be recovered from the latent? A nonparametric (k-nearest-neighbour) **probe** measures information without requiring it to be linear;
- *prediction*: how fast do errors grow in open-loop rollouts of g , relative to the spread of latents?
- *planning*: does the cost slope toward the goal? Plot the **landscape** $\|E(\mathbf{o}(s)) - E(\mathbf{o}_g)\|$ over states, and run the planner with the **exact** dynamics scored by the same latent cost: if it still fails, the predictor is not to blame.

Step 5 — information is not geometry. A latent can contain the state perfectly yet have a distance that says little about how far apart two states are. In the lab the agent is rendered as a Gaussian blob on a 16×16 image, and the encoder is PCA of the images. When the blob is sharp, images of two positions more than a few blob widths apart do not overlap, and their distance is nearly constant: the landscape is a narrow well around the goal and a plateau elsewhere (figure 23.3). A k-NN probe still recovers the position ($R^2 = 0.999$) — the information is there — but a planner that descends latent distance sees no slope from far away. The correlation between latent and true distance to the goal is 0.27 for the sharp blob, 0.75 for a wide blob and 0.80 for a multi-scale code. *Lesson:* a representation for planning needs the right *metric*, not only the right content. Representations learned for invariance can also discard what the task needs: the lab’s

row-profile encoder (the image summed over columns) keeps the vertical position exactly ($R^2 = 1.000$) and the horizontal position not at all ($R^2 = -0.16$).

Step 6 — the cross-entropy method. The planning objective is non-differentiable in general (and the predictor may be a large network), so sample-based optimisation is common. The **cross-entropy method** (CEM) keeps a Gaussian over action sequences; each iteration samples a population (48 in the lab), scores each sequence by imagined rollouts, keeps the best few (6), and refits the mean and standard deviation to them. After a few iterations (3), the first action of the mean is executed. Cost per real step: population $\times$ iterations $\times$ horizon predictor calls — 720 in the lab, batched. Receding horizon makes the plan robust to model error: only one action is trusted before the world is observed again.

Step 7 — compounding prediction error. If each predicted step has error ϵ and the predictor amplifies errors by a factor ρ per step, the error after h steps is about $\epsilon(1 + \rho + \dots + \rho^{h-1})$, which grows exponentially when $\rho > 1$. Inputs that were never in training make ρ large: the predictor is extrapolating. Remedies: train on multi-step rollouts (V-JEPA 2-AC adds a rollout loss to teacher forcing), keep horizons short and replan, and collect data that covers the states the planner will visit. In the lab, a predictor trained only on transitions starting in the left 40% of the arena has a one-step error of 1.04 (relative to the latent spread) against 0.18 for the same encoder trained on the whole arena.

Step 8 — subgoals turn a long search into short ones. If the landscape is informative only near the goal, give the planner goals it can see from where it is: intermediate goal images along the way. V-JEPA 2-AC uses image subgoals for multi-stage pick-and-place. In the lab, four subgoal images along the straight line to the goal raise success from 0.22 to 0.99 for the sharp blob. Subgoals cannot restore missing information: the row-profile encoder stays at 0.11.

23.6 Numerical worked example

Projection. $f = 500$ px, $(c_x, c_y) = (320, 240)$, point $(0.2, -0.1, 2.0)$ m in camera coordinates: $u = 500 \cdot 0.2/2 + 320 = 370$, $v = 500 \cdot (-0.1)/2 + 240 = 215$.

Stereo. $B = 0.1$ m: disparity $fB/Z = 25$ px at 2 m and 5 px at 10 m. With $\delta d = \sqrt{2} \cdot 0.5 \approx 0.71$ px: $\delta Z \approx 4 \cdot 0.71/50 = 0.057$ m at 2 m and $100 \cdot 0.71/50 = 1.41$ m at 10 m.

Scale ambiguity. Points at depth 3–6 m seen from cameras 0.3 m apart produce exactly the same pixels as points at 6–12 m from cameras 0.6 m apart.

Planning budget. Population 48, 3 CEM iterations, horizon 5: $48 \cdot 3 \cdot 5 = 720$ predictor evaluations per executed action; 40 actions per episode: 28,800. With a 300-million-parameter predictor, that budget — not training — dominates the cost of acting.

Compounding. Per-step error 0.07 with amplification $\rho = 1.1$: after 16 steps $0.07 \cdot (1.1^{16} - 1)/0.1 \approx 2.5$ — several times the spread of the latents; with $\rho = 0.9$, about 0.57.

23.7 Small implementation and checks

The lab `labs/ch23-world/world.py` has two parts. The geometry part projects, triangulates and measures stereo error:

```

def triangulate(P1, P2, u1, u2):
    A = np.stack([u1[0] * P1[2] - P1[0], u1[1] * P1[2] - P1[1],
                  u2[0] * P2[2] - P2[0], u2[1] * P2[2] - P2[1]])
    X = np.linalg.svd(A)[2][-1] # right singular vector of the
    ↪ smallest singular value
    return X[:3] / X[3]

```

The world-model part has an agent in the unit square that moves by its action (at most 0.08 per axis), more slowly near the right wall; images are 16×16 renderings of a blob. Encoders: the true state (oracle), PCA of sharp-blob images (16 components), PCA of wide-blob images (8), a multi-scale code (PCA at three blob widths, concatenated) and the row profile. For each, a residual MLP predictor is trained on 24,000 random-action transitions, and planning uses CEM with a running latent cost:

```

for _ in range(iters): # cross-entropy
    ↪ method, all episodes at once
    A = np.clip(mu[:, None] + sd[:, None] * rng.standard_normal((E, pop,
    ↪ H, 2)), -STEP, STEP)
    x, total = np.repeat(x0, pop, 0), 0
    for h in range(H):
        x = model(x, A[:, :, h].reshape(E * pop, 2)); total = total +
    ↪ cost(x, ep)
        elite = np.take_along_axis(A, np.argsort(total.reshape(E, pop), 1)[: ,
    ↪ :6, None, None], 1)
        mu, sd = elite.mean(1), elite.std(1) + 1e-3
    action = mu[:, 0] # execute the
    ↪ first action, then replan

```

The “exact dynamics” control replaces `model` with the true environment step on the true state, while keeping the encoder’s latent cost — it isolates the landscape from the predictor.

Checks, in `test_world.py`: the worked projection; exact triangulation; invariance of pixels to scaling scene and translation together; stereo error within 20% of the first-order formula up to 4 m; blob rendering and encoder shapes; the row-profile probe loses x and keeps y ; landscape correlation orders the encoders; a gradient check of the predictor; planning with the state encoder succeeds; planning with the sharp-blob encoder fails with exact dynamics and succeeds with subgoals.

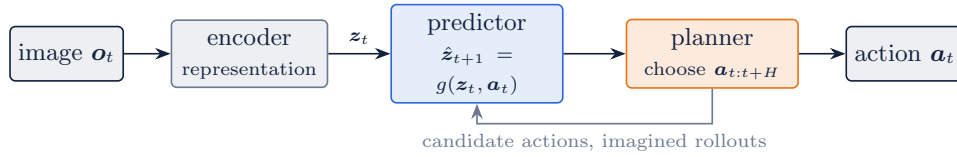
23.8 Experiment and expected observations

Run `python labs/ch23-world/world.py` (seed 0; 100 episodes with random start and goal positions; 40 steps; success means finishing within 0.08 of the goal). Random actions succeed in 3% of episodes.

encoder	probe R^2 (x, y)	landscape corr.	rollout error, 1 / 4 / 16 steps	learned predictor	exact dynamics	learned + 4 subgoals
state (oracle)	1.000, 1.000	1.000	0.014 / 0.039 / 0.099	1.000	1.000	1.000

encoder	probe R^2 (x, y)	landscape corr.	rollout error, 1 / 4 / 16 steps	learned predictor	exact dynamics	learned + 4 subgoals
PCA, sharp blob	0.999, 0.999	0.267	0.39 / 1.19 / 72	0.220	0.220	0.990
PCA, wide blob	1.000, 1.000	0.749	0.075 / 0.17 / 1.04	0.570	0.570	1.000
multi- scale	1.000, 1.000	0.802	0.18 / 0.39 / 1.59	0.720	0.710	1.000
row profile	− 0.162 , 1.000	0.301	0.25 / 0.47 / 1.08	0.110	0.230	0.110
multi- scale, predictor trained on $x < 0.4$	—	—	1.04 / 2.89 / 17.0	—	—	0.660

(a) three questions, three parts



(b) V-JEPA 2-AC (action-conditioned planning)

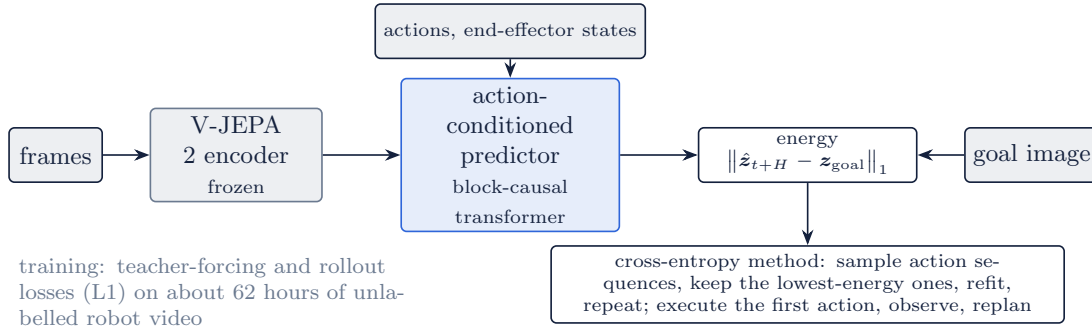


Figure 23.2: (a) A latent world model as three parts, each with its own test. (b) V-JEPA 2-AC, redrawn after the action-conditioned training and planning figures of Assran et al. (2025): a frozen V-JEPA 2 encoder, an action-conditioned predictor trained with teacher-forcing and rollout losses, and planning by the cross-entropy method toward a goal image’s embedding. Paper: [arXiv:2506.09985](https://arxiv.org/abs/2506.09985).

What to notice:

1. **Equal information, different planning.** Three encoders recover the state almost perfectly; their success rates are 0.22, 0.57 and 0.72. Their landscapes explain the difference.

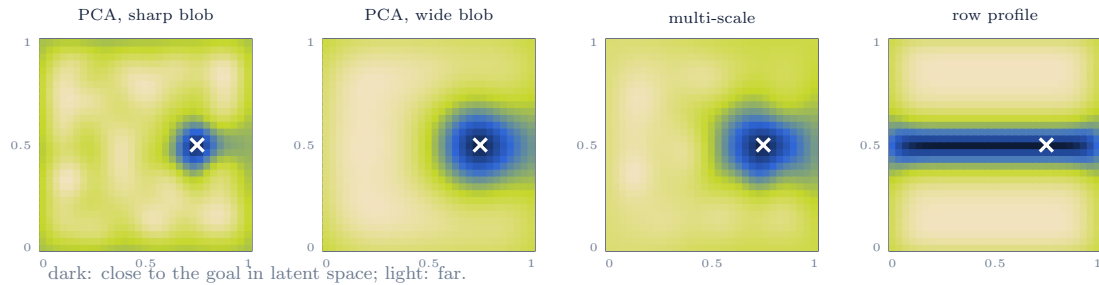


Figure 23.3: Latent distance to a goal (white cross) over all positions, for four encoders. The sharp-blob code is flat away from the goal; wider and multi-scale codes slope toward it; the row profile cannot tell positions on the same row apart.

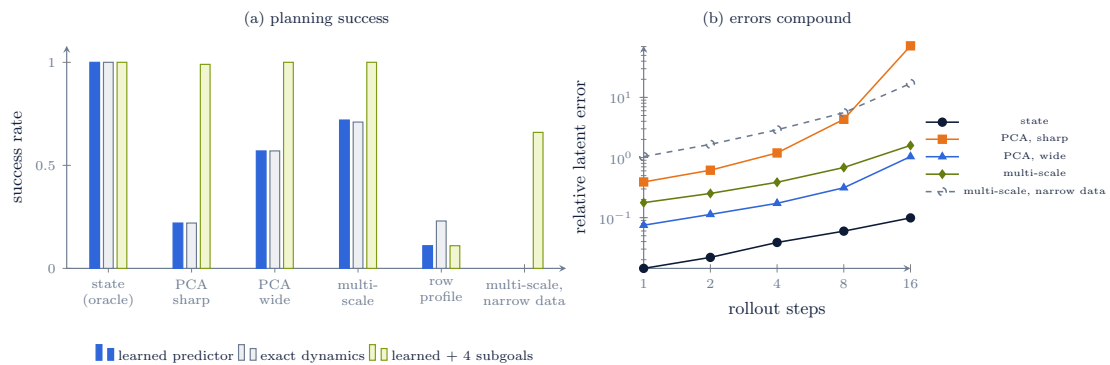


Figure 23.4: (a) Planning success. The learned predictor does as well as the exact dynamics for every encoder trained on full data — the landscape, not the predictor, limits planning; subgoals fix the landscape but not missing information; a predictor trained on narrow data limits planning even with subgoals. (b) Open-loop rollout error compounds with the number of steps.

2. **The exact-dynamics control assigns blame.** Replacing the learned predictor with the true dynamics changes nothing for these encoders, so improving the predictor would not help.
3. **Subgoals and data coverage are different fixes for different failures.** Subgoals rescue a flat landscape; they do not rescue a predictor that has never seen the right half of the arena (0.66), nor an encoder that discards x (0.11).
4. **Long rollouts are not needed to plan well with replanning.** The sharp-blob predictor’s 16-step error is enormous, yet with subgoals and a 5-step horizon it succeeds in 99% of episodes.

23.9 Moving pieces: diagnosing and fixing

Moving piece	Symptom when it is wrong	How to confirm	Fix
Intrinsics	straight lines bent; reprojection error large	calibration target; reprojection error per image	calibrate with a distortion model; fixed zoom and focus
Extrinsics / pose	triangulated points drift; scene scale wrong	reproject known points; check scale against a known length	bundle adjustment; metric reference
Baseline and range	depth noisy far away	error against Z ; compare with $Z^2 \delta d / (fB)$	wider baseline, longer focal length, nearer working range
Representation content	the planner cannot reach goals that differ in one factor	probe each state factor from the latent	encoder trained on data that varies that factor; less invariance
Representation geometry	succeeds near the goal, fails from far	landscape plot; exact-dynamics control	multi-scale or temporally-trained features; subgoals
Predictor coverage	plans that look good in imagination fail in reality	rollout error on states the planner visits	more diverse data; multi-step losses; uncertainty penalties
Horizon	myopic plans, or slow and noisy long plans	success against H	short horizon with replanning; subgoals
Search budget	inconsistent actions; jitter near the goal	success against population and iterations	larger population; warm-start from the previous plan
Cost definition	arbitrary first actions	terminal-only cost versus running cost	running cost; action penalties
Camera placement	actions move the arm the wrong way after the camera is moved	re-run with a shifted camera	fixed or calibrated cameras; train with varied viewpoints

First things to try: (1) probe the latent for each state factor; (2) plot the latent-distance landscape for a few goals; (3) run the planner with exact dynamics where you can (simulation, or replaying recorded outcomes); (4) measure rollout error on the states the planner actually visits.

23.10 Application, limitations and failure cases

Application (educational case study): a camera-to-plant reaching study. In a simulated greenhouse bench (a physics simulator with rendered images), collect a few hours of random arm motions with a wrist camera. Train an action-conditioned predictor on frozen features from a public self-supervised encoder (chapters 16–17). Goals are photographs of target leaves. Before reporting success rates, report the three diagnostics of this chapter: probes for arm position and target position, landscapes for ten goals, and the gap between planning with the learned predictor and with the simulator’s true dynamics. Then vary the camera position, the lighting and the bench layout, and report success under each. Stereo or depth measurements fix metric scale for the reaching distance; report the expected depth error at the working range.

Limitations:

- The lab’s world is fully observed, two-dimensional and deterministic; real manipulation involves contact, occlusion, other agents and uncertainty.
- Goal images specify *what it looks like when done*, not *what to do*; language or other goal specifications need further machinery.
- Image-based costs are sensitive to viewpoint, lighting and distractors that are irrelevant to the goal.
- Sample-based planning is expensive at every step, and its cost grows with horizon and action dimension.
- Safety: a predictor that is wrong off-distribution will still return confident plans. Physical systems need limits and monitoring outside the learned model.

23.11 Bridge to a research paper

Assran and colleagues (2025), *V-JEPA 2: self-supervised video models enable understanding, prediction and planning* ([arXiv:2506.09985](https://arxiv.org/abs/2506.09985)). Chapter 17 discussed its video pretraining; here the focus is the action-conditioned model, **V-JEPA 2-AC**.

- **Problem:** plan robot actions from images toward a goal image, without task-specific rewards or demonstrations of the task, using a world model learned from a small amount of unlabelled robot interaction.
- **Mechanism:** freeze the V-JEPA 2 video encoder; train an action-conditioned predictor (a transformer with block-causal attention over frames, actions and end-effector states) on fewer than 62 hours of robot video from the DROID dataset, with a teacher-forcing loss and a short rollout loss (L1 in representation space); at test time, minimise the L1 distance between the predicted future representation and the goal image’s representation with the cross-entropy method, execute the first action, and replan; use image subgoals for multi-stage pick-and-place.
- **Central claims:** zero-shot deployment on Franka arms in laboratories not seen in the training data, with reaching, grasping and pick-and-place from goal images; better success than baselines including a vision-language-action policy (Octo) and a video-generation world model (Cosmos), with much less planning time per action than the latter.

- **Assumptions to question:** how sensitive success is to camera placement (the paper reports that the model must infer action directions from the view, and that performance depends on the camera position); how much success depends on subgoal choice; whether the energy landscape is smooth enough for CEM beyond short horizons; how success rates vary across objects and scenes, given the number of trials.
- **Affordable experiment:** in the lab, add a “camera rotation”: render images in a frame rotated by an angle θ relative to the action axes, train the predictor at $\theta = 0$, and measure planning success as θ increases, with and without retraining the predictor on a few hundred rotated transitions.

23.12 Memory map

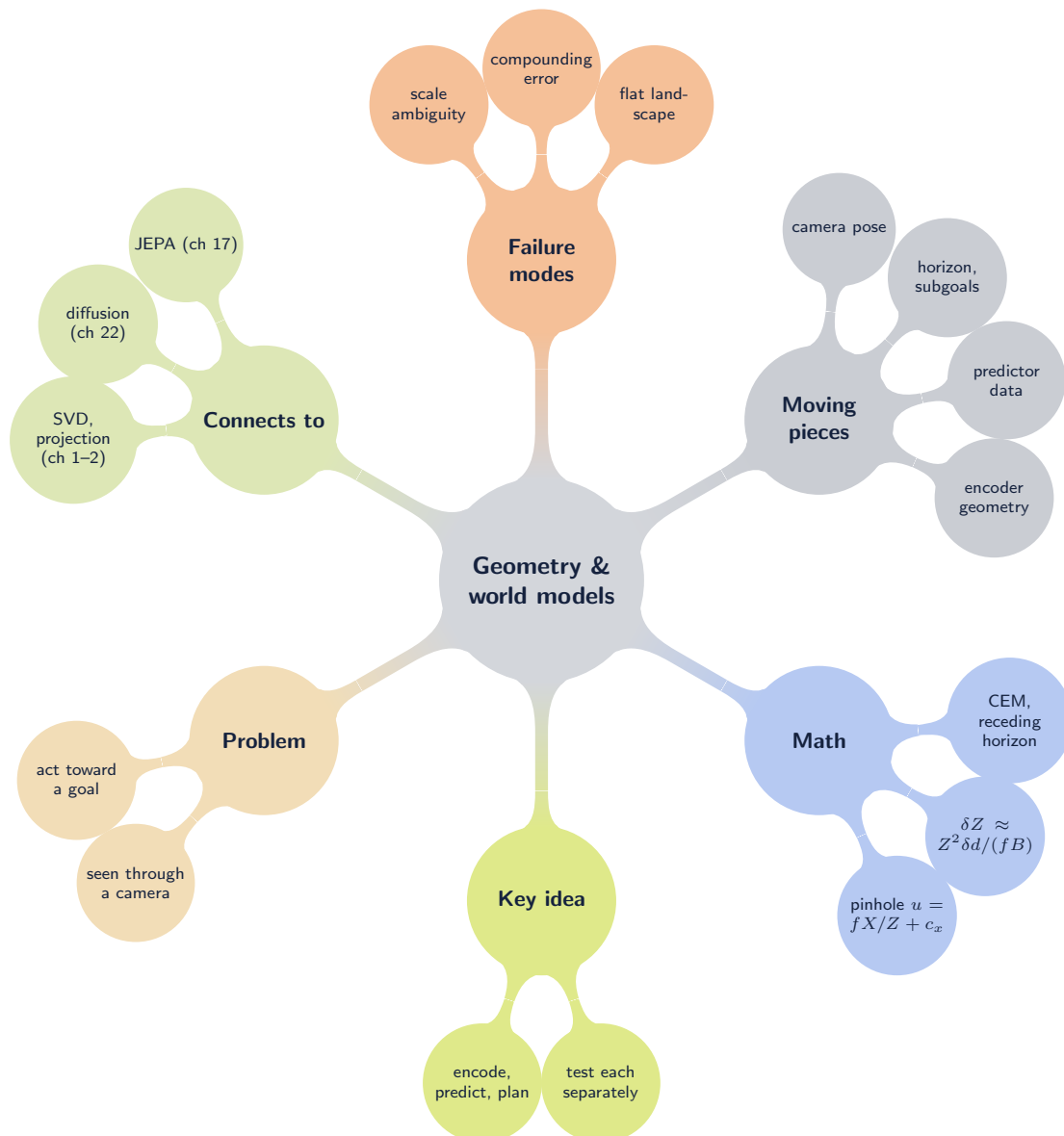


Figure 23.5: Memory map of chapter 23.

Summary. A pinhole camera maps 3-D points to pixels by $\mathbf{K}[\mathbf{R} \mid \mathbf{t}]$, losing depth along each ray; two views recover it by triangulation, as a null-space problem solved with the

SVD, with an error that grows as the square of the depth, and a single moving camera recovers structure only up to scale. A latent world model for planning has three parts — representation, prediction and planning — and each needs its own test: probes for information, rollout error for prediction, and landscapes and exact-dynamics controls for planning. The lab shows that information is not geometry: encoders that all contain the state can make planning succeed or fail depending on whether latent distance slopes toward the goal. Subgoals repair a flat landscape; data coverage limits the predictor; nothing repairs information the encoder discarded. V-JEPA 2-AC applies the same recipe — frozen self-supervised encoder, action-conditioned predictor, CEM toward a goal image — to real robot arms.

23.13 Exercises

1. *Conceptual.* Why can a single moving camera not determine the absolute size of a scene?
2. *Conceptual.* A k-NN probe recovers the state from a latent perfectly, but planning fails. Name two possible reasons.
3. *Conceptual.* Why does receding-horizon control tolerate an imperfect model better than executing a whole plan?
4. *Derivation.* Derive $\delta Z \approx Z^2 \delta d / (fB)$ and the RMS error for independent pixel noise of standard deviation σ in each view.
5. *Derivation.* Show that the DLT solution is the right singular vector of $\mathbf{A}$ with the smallest singular value.
6. *Derivation.* For a blob of width w and images compared by Euclidean distance, show that the squared distance between images of positions $\mathbf{s}$ and $\mathbf{s}'$ is proportional to $1 - \exp(-\|\mathbf{s} - \mathbf{s}'\|^2 / (4w^2))$ (ignoring image borders and pixelation). What does this imply for planning?
7. *Implementation.* Carry out the experiment of section 23.11.
8. *Implementation.* Replace CEM with gradient descent on the action sequence through the learned predictor. Compare success and cost with CEM for the wide-blob encoder.
9. *Interpretation.* A paper reports 80% task success for a world-model planner but no planning-with-true-dynamics control. What could you not conclude?
10. *Open question.* What training signal would make latent distance reflect “how many actions apart” two states are, without labels?

23.13.1 Answer discussions

1. Scaling all points and the camera translation by the same factor leaves every projection unchanged (step 1).
2. A flat or misleading latent landscape (geometry), or predictor errors on the states the planner visits; also a weak search.
3. Only the first action is executed before new evidence arrives, so errors in later imagined steps are never acted on.
4. $Z = fB/d \Rightarrow \delta Z = -(fB/d^2)\delta d = -(Z^2/fB)\delta d$; $\text{sd}(d) = \sqrt{2}\sigma$, so $\text{RMS}(\delta Z) \approx \sqrt{2}\sigma Z^2/(fB)$.
5. Minimise $\|\mathbf{A}\tilde{\mathbf{x}}\|^2 = \tilde{\mathbf{x}}^\top \mathbf{A}^\top \mathbf{A}\tilde{\mathbf{x}}$ subject to $\|\tilde{\mathbf{x}}\| = 1$: the minimiser is the eigenvector of $\mathbf{A}^\top \mathbf{A}$ with the smallest eigenvalue, the last right singular vector.
6. $\|b_s - b_{s'}\|^2 = \|b_s\|^2 + \|b_{s'}\|^2 - 2\langle b_s, b_{s'} \rangle$, and the overlap of two Gaussians of width w is proportional to $\exp(-\|\mathbf{s} - \mathbf{s}'\|^2 / (4w^2))$. The distance saturates beyond a few w : no gradient toward far goals; wider blobs or multi-scale codes extend the slope.
7. Expect success to fall as θ grows (the predictor’s action directions are wrong in the

rotated frame) and to recover with a little rotated data — the camera-placement sensitivity reported for V-JEPA 2-AC.

8. Gradient descent is cheaper per iteration but can stall on plateaus and local minima of the landscape; CEM is more robust, at higher cost.
9. Whether failures come from the model or from the planner/landscape, and hence what to improve; also whether the model is needed at all.
10. Temporal objectives — for example, contrastive or predictive objectives that make states a few steps apart close, or learning a distance that predicts the number of steps between frames (temporal distance learning).

23.14 Self-check and common mistakes

- ☐ I can project a point with $\mathbf{K}[\mathbf{R} \mid \mathbf{t}]$ and triangulate it from two views.
- ☐ I can explain scale ambiguity and compute stereo depth error at a given range.
- ☐ I can name the three parts of a latent world model and a test for each.
- ☐ I can implement CEM planning in a receding-horizon loop.
- ☐ I can explain why a representation with full information can still make planning fail.

Common mistakes: forgetting the principal point or mixing pixel and metric units; reporting monocular reconstructions in metres without a scale reference; trusting depth at ranges where disparity is a few pixels; judging a representation for planning by probe accuracy alone; blaming the predictor without an exact-dynamics control; using a terminal-only cost and reading the resulting jitter as model error; executing whole imagined plans open-loop; reporting success rates without the number of trials and the camera set-up.

References

- I. Goodfellow, Y. Bengio and A. Courville. *Deep Learning*. MIT Press, 2016, chapter 2 (SVD) and section 12.2 (computer vision).
- R. Hartley and A. Zisserman. *Multiple View Geometry in Computer Vision*, 2nd edition. Cambridge University Press, 2004.
- H. C. Longuet-Higgins. A computer algorithm for reconstructing a scene from two projections. *Nature*, 293:133–135, 1981.
- Z. Zhang. A flexible new technique for camera calibration. *IEEE TPAMI*, 22(11):1330–1334, 2000.
- N. Snavely, S. M. Seitz and R. Szeliski. Photo tourism: exploring photo collections in 3D. *ACM SIGGRAPH*, 2006.
- J. L. Schönberger and J.-M. Frahm. Structure-from-motion revisited. In *CVPR*, 2016.
- D. Eigen, C. Puhrsch and R. Fergus. Depth map prediction from a single image using a multi-scale deep network. In *NeurIPS*, 2014.
- B. Mildenhall et al. NeRF: representing scenes as neural radiance fields for view synthesis. In *ECCV*, 2020.
- B. Kerbl, G. Kopanas, T. Leimkühler and G. Drettakis. 3D Gaussian splatting for real-time radiance field rendering. *ACM TOG*, 2023.
- S. Wang, V. Leroy, Y. Cabon, B. Chidlovskii and J. Revaud. DUST3R: geometric 3D vision made easy. In *CVPR*, 2024.
- K. Craik. *The Nature of Explanation*. Cambridge University Press, 1943.
- R. S. Sutton. Dyna, an integrated architecture for learning, planning, and reacting. *SIGART Bulletin*, 2(4):160–163, 1991.

- R. Y. Rubinstein. The cross-entropy method for combinatorial and continuous optimization. *Methodology and Computing in Applied Probability*, 1:127–190, 1999.
- D. Ha and J. Schmidhuber. World models. arXiv:1803.10122, 2018.
- D. Hafner et al. Learning latent dynamics for planning from pixels (PlaNet). In *ICML*, 2019.
- J. Schrittwieser et al. Mastering Atari, Go, chess and shogi by planning with a learned model (MuZero). *Nature*, 588:604–609, 2020.
- Y. LeCun. A path towards autonomous machine intelligence. OpenReview, 2022.
- G. Zhou, H. Pan, Y. LeCun and L. Pinto. DINO-WM: world models on pre-trained visual features enable zero-shot planning. arXiv:2411.04983, 2024.
- J. Bruce et al. Genie: generative interactive environments. In *ICML*, 2024.
- M. Assran et al. V-JEPA 2: self-supervised video models enable understanding, prediction and planning. arXiv:2506.09985, 2025.

Chapter 24

Reproduction, ablations and scientific writing

Prerequisites: chapter 5 (risk, confidence intervals, experimental design).

Learning outcomes: after this chapter you can

1. state a bounded, falsifiable claim and fix a protocol before seeing results;
2. compare methods across seeds with paired intervals, and choose the number of seeds in advance;
3. design ablations that measure interactions, and recognise when leave-one-out ablations mislead;
4. recognise unequal tuning budgets and test-set selection as sources of spurious gains;
5. write results with their uncertainty, limitations and negative findings.

24.1 A concrete problem

A student reports that a new training recipe raises test accuracy from 53.4% to 75.4%. The recipe has three components. Which of them matter, would the gain hold with other seeds, and was the comparison fair? This chapter turns those questions into procedures, and runs each of them on a small task where every answer can be checked.

24.2 How we got here

Concerns about reproducibility grew with empirical machine learning. Henderson and colleagues (2018) showed that reinforcement learning results changed with the random seed and implementation details. Melis, Dyer and Blunsom (2018) found that, with equal tuning, older language-model architectures matched newer ones. Lucic and colleagues (2018) reached a similar conclusion for GANs. Bouthillier and colleagues (2021) measured the sources of variance in benchmarks and recommended randomising more of them. **Musgrave, Belongie and Lim (2020)** — this chapter’s paper bridge — re-ran metric-learning methods under one protocol and found far smaller improvements than reported. Pineau and colleagues (2021) introduced reproducibility checklists at NeurIPS.

24.3 Assumptions and notation

Symbol	Meaning
$a_i^{(M)}$	test accuracy of method M with seed i
$d_i = a_i^{(M)} - a_i^{(B)}$	paired difference for seed i
n	number of seeds; $\bar{d}$, s_d mean and standard deviation of d_i
δ	smallest effect worth detecting

A **seed** fixes all randomness of one run: data draw, split, initialisation and batch order. The test set is used once, after all choices are made.

24.4 Mathematical formulation

Paired interval. $\bar{d} \pm t_{n-1, 0.975} s_d / \sqrt{n}$ (we use 1.96 for simplicity). An unpaired interval uses $\sqrt{s_M^2/n + s_B^2/n}$ instead of $s_d/\sqrt{n}$; it is wider whenever the two methods’ results are positively correlated across seeds.

Number of seeds. To detect δ at level 0.05 with power 0.8, $n \approx (z_{0.975} + z_{0.8})^2 s_d^2 / \delta^2$ for a paired design and $n \approx 2(z_{0.975} + z_{0.8})^2 s^2 / \delta^2$ per arm for an unpaired one, with $z_{0.975} = 1.96$ and $z_{0.8} = 0.84$.

Factorial ablation. For two components A and B with cell means $m_\emptyset, m_A, m_B, m_{AB}$, the interaction is $m_{AB} - m_A - m_B + m_\emptyset$. Leave-one-out ablations attribute $m_{AB} - m_B$ to A; add-one ablations attribute $m_A - m_\emptyset$.

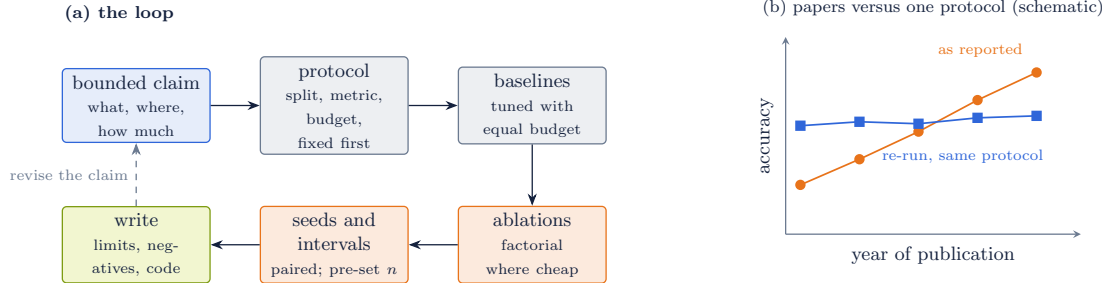


Figure 24.1: (a) The research loop of this chapter. (b) Schematic after the “papers versus reality” figure of Musgrave, Belongie and Lim (2020): reported accuracies rose over time, while re-running the methods under one protocol showed much smaller differences (shapes only, not the paper’s values). Paper: [arXiv:2003.08505](https://arxiv.org/abs/2003.08505).

24.5 Derivation, with reasons for each step

Step 1 — a bounded claim. “Our recipe is better” cannot be tested. “On task T, with budget N training steps, standardising the inputs and raising the learning rate tenfold improves test accuracy by at least two points, averaged over 20 seeds” can be. A bounded claim names the task, the comparison, the budget and the size of effect that would count.

Step 2 — why pairing helps. $\text{Var}(\bar{a}^{(M)} - \bar{a}^{(B)}) = (\sigma_M^2 + \sigma_B^2 - 2\rho\sigma_M\sigma_B)/n$. Running both methods with the same seeds makes ρ large, because a hard data draw is hard for both. In the lab, the 95% half-width for the effect of the jitter component C is 0.0101 unpaired and 0.0021 paired.

Step 3 — single seeds decide nothing. Component C has no real effect. Its paired difference is -0.0012 with interval $[-0.0032, +0.0009]$; yet C “wins” on 45% of seeds and “loses” on 50%. A single-seed comparison would report a gain or a loss almost as often as a coin toss.

Step 4 — ablations and interactions. If components interact, removing one from the full method and adding one to the baseline give different answers. With a positive interaction, leave-one-out attributions sum to more than the total gain and add-one attributions sum to less; only the factorial design shows the interaction itself. Its cost is 2^k cells for k components; for many components, use fractional factorial designs or test the interactions the mechanism suggests.

Step 5 — tuning budgets. If the proposed method’s hyperparameters are tuned and the baseline’s are not, the comparison measures tuning effort. Tune both with the same budget, on the validation set, and report the budget.

Step 6 — the test set is used once. Choosing among configurations or seeds by test accuracy selects noise and luck. Any choice made with the test set makes the reported number optimistic; a fresh test set or a pre-registered choice removes the bias.

Step 7 — reproducing an idea versus a number. Reproducing a reported number needs the original data, code and budget; reproducing an idea needs only a setting where the mechanism should act, and a prediction of what it should do there. The labs of this book are reproductions of ideas at small scale; their results support mechanisms, not the original papers’ numbers.

Step 8 — writing. A results section states the claim, the protocol, the numbers with intervals and the number of seeds, the ablations, and the limits: where the claim was not tested, which results were negative, and what would change the conclusion.

24.6 Numerical worked example

Seeds needed. Paired standard deviation 0.010 and effect 0.005: $n = (2.80)^2 \cdot 0.010^2 / 0.005^2 = 31.4$, so 32 seeds. Unpaired with standard deviation 0.020: $n = 2 \cdot (2.80)^2 \cdot 0.020^2 / 0.005^2 = 250.9$, so 251 seeds per arm.

Attribution. Cell means 0.534 (baseline), 0.614 (A), 0.564 (B), 0.755 (A+B). Total gain 0.221. Leave-one-out: A 0.191, B 0.141, sum 0.332. Add-one: A 0.081, B 0.030, sum 0.111. Interaction $0.755 - 0.614 - 0.564 + 0.534 = 0.111$.

24.7 Small implementation and checks

The lab `labs/ch24-reproduction/repro.py` trains a one-hidden-layer network to classify points from 10 features whose scales range from 0.3 to 20; the label depends on the two smallest-scale features, and 10% of labels are flipped. Component A standardises the inputs, B raises the learning rate tenfold, and C adds input noise of standard deviation 0.001. Each run takes about 20 ms, so hundreds of runs are cheap.

```
d = full - noC                                # paired: same seeds for
↪ both methods
m, lo, hi = mean_ci(d)                         # mean +/- 1.96 sd /
↪ sqrt(n)
inter = F[(1, 1)] - F[(1, 0)] - F[(0, 1)] + F[(0, 0)] # per-seed
↪ interaction
```

Checks, in `test_repro.py`: the seed formulas reproduce the worked example; the normal and bootstrap intervals agree on Gaussian data; runs are deterministic given their seed; the interaction is positive and leave-one-out over-attributes while add-one under-attributes; tuning only the method creates a gain that vanishes with equal tuning.

24.8 Experiment and expected observations

Run `python labs/ch24-reproduction/repro.py` (seeds 0–19; tuning and selection studies use seeds 0–9).

quantity	value
baseline, mean (sd) over seeds	0.534 (0.021)
A+B, mean (sd)	0.755 (0.017)
effect of C, paired, 95% CI	−0.0012 [−0.0032, +0.0009]
seeds on which C wins / loses	45% / 50%
A+B+C vs baseline	+0.213 [+0.200, +0.226]
interaction of A and B	+0.111 [+0.090, +0.132]
A+C tuned vs baseline untuned	+0.126 [+0.099, +0.152]
A+C tuned vs baseline tuned equally	+0.000 [−0.003, +0.003]
best of 8 training seeds, small test set / same run on fresh data / average run	0.771 / 0.770 / 0.759

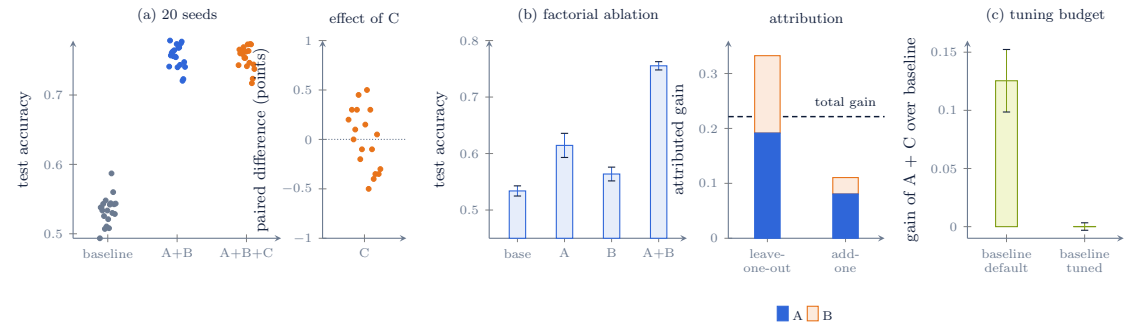


Figure 24.2: (a) Test accuracy for 20 seeds, and the paired difference made by C, centred on zero. (b) The factorial ablation, and how leave-one-out and add-one ablations attribute the total gain (dashed). (c) The apparent gain of a component with no effect disappears when the baseline is tuned equally.

What to notice: the spread across seeds (0.02) is larger than many reported improvements; pairing shrinks intervals fivefold; the factorial design reveals an interaction worth half the total gain; equal tuning removes a spurious 12.6-point gain; picking the best seed reports 1.2 points above a typical run, a gain no one else can reproduce.

24.9 Moving pieces: diagnosing and fixing

Moving piece	Symptom when it is wrong	How to confirm	Fix
Claim	results cannot contradict it	ask what result would refute it	name task, comparison, budget, effect size
Seeds	gains that vanish on re-run	spread across seeds	paired seeds; n chosen in advance
Intervals	overlapping claims of superiority	report intervals	paired or bootstrap intervals
Tuning budget	new method wins only when tuned	tune the baseline equally	equal, reported budgets
Ablation design	components “essential” in one analysis, useless in another	full factorial on two or three components	factorial or targeted interaction tests
Test-set use	reported number above fresh-data number	evaluate on a fresh split	validation for all choices; test once
Implementation	baseline weaker than published	reproduce the baseline’s published number first	use reference code; report the gap
Writing	readers cannot tell what was measured	can someone else re-run it from the paper?	protocol, code, seeds, limitations, negative results

24.10 Application, limitations and failure cases

Application (educational case study): a capstone reproduction. Choose one claim from a paper in chapters 12–23. Write a proposal (`research/capstone-proposal.md`) with the bounded claim, the protocol and the number of seeds, before running anything. Reproduce the baseline first, then the method, then a factorial ablation of at most two components. Record each run (`research/experiment-record.md`), and report in `research/capstone-report.md` with intervals, limitations and negative results.

Limitations: intervals over seeds do not cover variation over datasets or tasks; small-scale reproductions test mechanisms, not large-scale numbers; statistical significance is not practical importance; many comparisons need correction or a held-out confirmation.

24.11 Bridge to a research paper

Musgrave, Belongie and Lim (2020), *A metric learning reality check*, ECCV ([arXiv:2003.08505](https://arxiv.org/abs/2003.08505)).

- **Problem:** many deep metric learning papers reported steady improvements on image retrieval benchmarks (CUB-200-2011, Cars196, Stanford Online Products).
- **Mechanism:** identify flaws in common practice — comparisons with different backbones, embedding sizes and augmentation; hyperparameters chosen with test-set feedback; accuracy metrics that are uninformative — and re-run methods with the same architecture, a validation split, equal tuning budgets and a more informative metric ([MAP@R](https://arxiv.org/abs/2003.08505)).

- **Central claim:** under this protocol, improvements over older methods such as the contrastive and triplet losses were much smaller than reported, and in many cases absent.
- **Assumptions to question:** whether one protocol and one tuning procedure are fair to every method; whether the conclusions extend beyond the three datasets.
- **Affordable experiment:** in the lab, add a fourth component and compare the conclusions of leave-one-out, add-one and full factorial ablations, with 20 seeds each.

24.12 Memory map

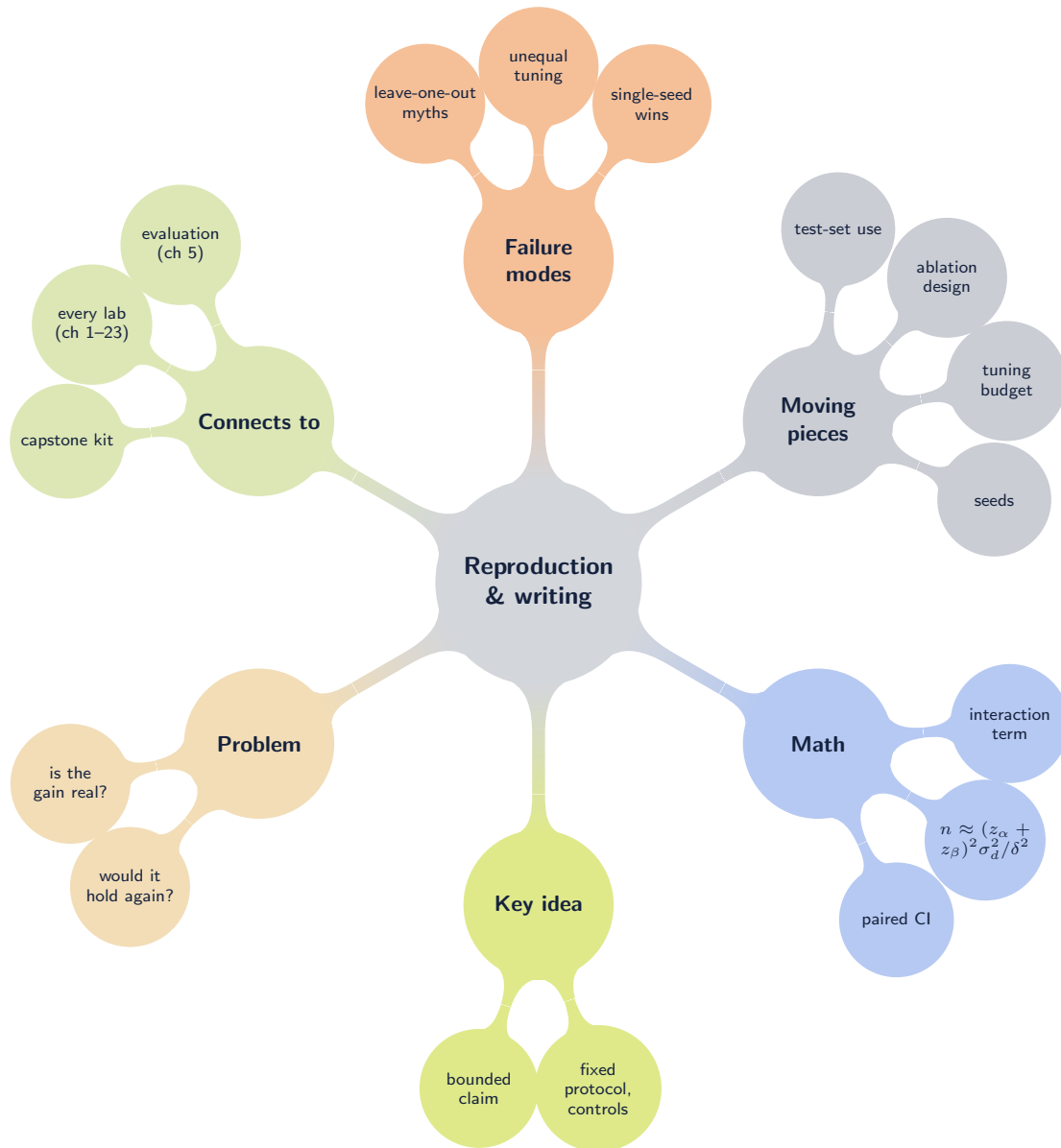


Figure 24.3: Memory map of chapter 24.

Summary. A research result is a claim, a protocol, and evidence with uncertainty. Bound the claim before running anything; compare methods over many seeds with paired intervals and choose the number of seeds in advance; tune baselines with the same budget; use the test set once; ablate with factorial designs when components may interact; and write down

limitations and negative results. On a small task, these rules separate a real gain from seed noise, unequal tuning and selection on the test set.

24.13 Exercises

1. *Conceptual.* Why is a single-seed comparison uninformative when the seed spread exceeds the effect?
2. *Conceptual.* Why can leave-one-out attributions sum to more than the total gain?
3. *Derivation.* Derive the variance of a paired difference and show when pairing helps.
4. *Derivation.* Derive the number of seeds for a paired comparison at level 0.05 and power 0.8.
5. *Implementation.* Carry out the experiment of section 24.11.
6. *Implementation.* Replace the normal interval with a paired bootstrap and a permutation test; compare.
7. *Interpretation.* A paper reports +0.8 points over a baseline with three seeds and no intervals. What would you ask for?
8. *Open question.* How should a field decide what counts as a fair tuning budget across methods?

24.13.1 Answer discussions

1. The sign of a single difference is then mostly noise (the lab: C wins on 45% of seeds with no effect).
2. With a positive interaction, each component's removal also removes the interaction, which is counted twice.
3. $\text{Var}(d) = \sigma_M^2 + \sigma_B^2 - 2\rho\sigma_M\sigma_B$; pairing helps when $\rho > 0$.
4. Require $\delta/(s_d/\sqrt{n}) \geq z_{0.975} + z_{0.8}$; solve for n .
5. Expect leave-one-out and add-one to disagree whenever components interact.
6. With 20 seeds the three should agree closely; the permutation test needs no normality.
7. Per-seed results, paired intervals, the baseline's tuning budget, and how the configuration was chosen.
8. A budget fixed in advance per method (number of trials or compute), reported with the results.

24.14 Self-check and common mistakes

- ☐ I can state a bounded claim and a protocol before running experiments.
- ☐ I can compute paired intervals and the number of seeds needed.
- ☐ I can design a factorial ablation and compute an interaction.
- ☐ I can recognise unequal tuning and test-set selection.

Common mistakes: single seeds; unpaired comparisons; tuning only the new method; selecting on the test set; leave-one-out ablations read as importance; omitting negative results.

References

- P. Henderson, R. Islam, P. Bachman, J. Pineau, D. Precup and D. Meger. Deep reinforcement learning that matters. In *AAAI*, 2018.
- G. Melis, C. Dyer and P. Blunsom. On the state of the art of evaluation in neural language models. In *ICLR*, 2018.

- M. Lucic, K. Kurach, M. Michalski, S. Gelly and O. Bousquet. Are GANs created equal? A large-scale study. In *NeurIPS*, 2018.
- K. Musgrave, S. Belongie and S.-N. Lim. A metric learning reality check. In *ECCV*, 2020.
- X. Bouthillier et al. Accounting for variance in machine learning benchmarks. In *MLSys*, 2021.
- J. Pineau et al. Improving reproducibility in machine learning research. *JMLR*, 22(164):1–20, 2021.
- B. Efron and R. J. Tibshirani. *An Introduction to the Bootstrap*. Chapman and Hall, 1993.

Appendix A. Prerequisites and self-diagnostic

The book assumes basic Python, matrix operations, derivatives and introductory probability. It does **not** assume a full undergraduate mathematics course. This diagnostic is for self-assessment, not a placement test. If you get stuck on more than two items in a block, read the matching material listed in A.5 before chapter 1. Notation follows the notation page: $\mathbf{x}$ is a vector, $\mathbf{A}$ a matrix, x a random variable.

A.1 Linear algebra

1. For $\mathbf{A} \in \mathbb{R}^{3 \times 5}$ and $\mathbf{B} \in \mathbb{R}^{5 \times 2}$, what is the shape of $\mathbf{AB}$? Is $\mathbf{BA}$ defined?
2. When is a square matrix invertible? Give two equivalent conditions.
3. Project $\mathbf{x} = (3, 4)^\top$ onto the unit vector $\mathbf{u} = (1, 0)^\top$. Then project it onto the line spanned by $(1, 1)^\top$.
4. What are the eigenvalues of $\begin{pmatrix} 2 & 0 \\ 0 & 3 \end{pmatrix}$, and why does a symmetric matrix have an orthonormal eigenbasis?

A.2 Calculus

5. Compute $\nabla_{\mathbf{w}} f$ for $f(\mathbf{w}) = (y - \mathbf{w}^\top \mathbf{x})^2$.
6. State the chain rule for $h(\mathbf{x}) = g(f(\mathbf{x}))$ where $f: \mathbb{R}^n \rightarrow \mathbb{R}^m$ and $g: \mathbb{R}^m \rightarrow \mathbb{R}$.
7. Why does a zero gradient not guarantee a minimum? Give an example.

A.3 Probability

8. If $p(x) = \mathcal{N}(x; \mu, \sigma^2)$ is the density of x , what is the density of $ax + b$?
9. State Bayes' rule and identify the prior, likelihood and posterior.
10. Two random variables are uncorrelated. Must they be independent ($a \perp b$)? Give a counterexample.
11. What are $\mathbb{E}[x]$ and $\text{Var}(x)$ for a fair six-sided die?

A.4 Python and NumPy

12. What does `X.mean(axis=0)` return for a design matrix `X` of shape `(m, n)`?
13. Predict the shape of `a[:, None] - b[None, :]` for `a` of shape `(3,)` and `b` of shape `(4,)`.
14. Why is `np.linalg.solve(A, b)` preferred to `np.linalg.inv(A) @ b`?

A.5 Where to refresh

This draft does not include its own refresher sections. For each block above, use:

- linear algebra and calculus: Goodfellow, Bengio and Courville (2016), chapters 2 and 4, and Deisenroth, Faisal and Ong (2020), chapters 2–5;
- probability: Goodfellow, Bengio and Courville (2016), chapter 3, and Deisenroth, Faisal and Ong (2020), chapter 6;
- Python and NumPy: the official NumPy user guide (<https://numpy.org/doc/stable/user/>), sections on array basics and broadcasting; question 14 is discussed in appendix C.

Full references:

- I. Goodfellow, Y. Bengio and A. Courville. *Deep Learning*. MIT Press, 2016 (same notation as this book).
- M. P. Deisenroth, A. A. Faisal and C. S. Ong. *Mathematics for Machine Learning*. Cambridge University Press, 2020 (freely available from the authors).
- G. Strang. *Introduction to Linear Algebra*, chapters 1–6, for a slower route through linear algebra.

Appendix C. Stable numerical computation

Four rules used throughout the labs.

C.1 Solve, do not invert

Use a factorisation and triangular solves (`np.linalg.solve`, `cholesky` plus `solve_triangular`) instead of `inv(A) @ b`. They cost less, are more accurate, and give the log-determinant for free: $\log \det(\mathbf{A}) = 2 \sum_i \log L_{i,i}$ when $\mathbf{A} = \mathbf{L}\mathbf{L}^\top$ (chapter 7).

C.2 Avoid squaring the condition number

For $\mathbf{X}$ with condition number $\kappa(\mathbf{X}) = s_{\max}/s_{\min}$, the Gram matrix has $\kappa(\mathbf{X}^\top \mathbf{X}) = \kappa(\mathbf{X})^2$. In float64, with about 16 significant digits, a $\kappa(\mathbf{X})$ of 10^8 leaves almost nothing of the smallest directions once squared. Use the SVD or QR of $\mathbf{X}$ itself for PCA and least squares (chapters 1–3).

C.3 Log-sum-exp and the stable softmax

$\log \sum_j \exp(s_j) = c + \log \sum_j \exp(s_j - c)$ with $c = \max_j s_j$. The same shift makes the softmax safe. Mask with $-\infty$ (or the most negative finite value of the dtype) **before** the softmax. In float16 a mask value such as -10^9 overflows to $-\infty$, and a row with every entry masked then gives NaN (chapter 14).

C.4 Jitter, seeds and checks

- Add a small diagonal jitter (10^{-8} – 10^{-6} times the mean diagonal) before a Cholesky factorisation of a nearly singular kernel matrix. Report the value you used.
- Fix and report random seeds. Report variation across seeds, not a single run.
- Check gradients with central differences $\frac{f(\mathbf{x} + \epsilon \mathbf{e}^{(i)}) - f(\mathbf{x} - \epsilon \mathbf{e}^{(i)})}{2\epsilon}$ in float64, with $\epsilon \approx 10^{-6}$. Compare relative, not absolute, errors.

Glossary

One-line definitions; the chapter in brackets gives the full treatment.

- **Attention** — a weighted average of value vectors, with weights from a softmax of query–key scores [14].
- **Calibration** — agreement between predicted probabilities or intervals and observed frequencies [6, 7].
- **Cholesky factorisation** — $\mathbf{A} = \mathbf{L}\mathbf{L}^\top$ with lower-triangular $\mathbf{L}$, for symmetric positive-definite $\mathbf{A}$ [7, App. C].
- **Coverage** — the fraction of held-out targets that fall inside a predictive interval [7].
- **Explained variance ratio** — the share of total variance captured by the top l principal components [2].
- **Gaussian process** — a distribution over functions whose finite marginals are jointly Gaussian [7].
- **Kernel** — a symmetric positive semidefinite similarity function [4].
- **Leakage** — test information that influences training or model selection [5].
- **Marginal likelihood** — $p(\mathbf{y} \mid \mathbf{X}; \boldsymbol{\theta})$ with the latent function integrated out [7].
- **Patch embedding** — a linear map from flattened image patches to tokens [14].
- **Permutation equivariance** — permuting the inputs permutes the outputs in the same way [14].
- **Principal direction** — a direction of maximal variance, orthogonal to the previous ones; a column of the PCA decoding matrix $\mathbf{D}$. The coordinates of an example along these directions are its **principal components** [2].
- **Schur complement** — $\mathbf{B} - \mathbf{C}^\top \mathbf{A}^{-1} \mathbf{C}$; the conditional covariance of a partitioned Gaussian [7].
- **Singular value decomposition** — $\mathbf{X} = \mathbf{U}\mathbf{S}\mathbf{V}^\top$ with orthonormal columns in $\mathbf{U}$ and $\mathbf{V}$ and non-negative diagonal $\mathbf{S}$ [2].
- **Toy study** — a small experiment on an idea. It is not a reproduction of a paper’s reported result [24].

References

Shared bibliography. Each chapter also lists its own references.

Books

- I. Goodfellow, Y. Bengio, A. Courville. *Deep Learning*. MIT Press, 2016. <https://www.deeplearningbook.org> (notation: https://github.com/goodfeli/dlbook_notation).
- M. P. Deisenroth, A. A. Faisal, C. S. Ong. *Mathematics for Machine Learning*. Cambridge University Press, 2020.
- K. P. Murphy. *Probabilistic Machine Learning: An Introduction*. MIT Press, 2022.
- C. E. Rasmussen, C. K. I. Williams. *Gaussian Processes for Machine Learning*. MIT Press, 2006.
- G. Strang. *Linear Algebra and Learning from Data*. Wellesley-Cambridge Press, 2019.

Paper bridges by chapter

Each full chapter ends with a “Bridge to a research paper” section built on one primary paper.

Ch.	Paper
1	A. G. Baydin, B. A. Pearlmutter, A. A. Radul, J. M. Siskind. Automatic differentiation in machine learning: a survey. <i>JMLR</i> 18(153), 2018. https://arxiv.org/abs/1502.05767
2	M. Turk, A. Pentland. Eigenfaces for recognition. <i>J. Cognitive Neuroscience</i> 3(1), 1991.
3	R. Tibshirani. Regression shrinkage and selection via the lasso. <i>JRSS B</i> 58(1), 1996.
4	A. Rahimi, B. Recht. Random features for large-scale kernel machines. NeurIPS 2007.
5	S. Beery, G. Van Horn, P. Perona. Recognition in terra incognita. ECCV 2018. https://arxiv.org/abs/1807.04975
6	D. J. C. MacKay. A practical Bayesian framework for backpropagation networks. <i>Neural Computation</i> 4(3), 1992.

Ch.	Paper
7	C. E. Rasmussen, C. K. I. Williams. <i>Gaussian Processes for Machine Learning</i> , chapters 2 and 5. MIT Press, 2006.
8	C. Stauffer, W. E. L. Grimson. Adaptive background mixture models for real-time tracking. CVPR 1999.
9	C. Blundell, J. Cornebise, K. Kavukcuoglu, D. Wierstra. Weight uncertainty in neural networks. ICML 2015. https://arxiv.org/abs/1505.05424
10	R. Rombach, A. Blattmann, D. Lorenz, P. Esser, B. Ommer. High-resolution image synthesis with latent diffusion models. CVPR 2022. https://arxiv.org/abs/2112.10752
11	K. He, X. Zhang, S. Ren, J. Sun. Delving deep into rectifiers. ICCV 2015. https://arxiv.org/abs/1502.01852
12	K. He, X. Zhang, S. Ren, J. Sun. Deep residual learning for image recognition. CVPR 2016. https://arxiv.org/abs/1512.03385
13	A. Kirillov et al. Segment anything. ICCV 2023. https://arxiv.org/abs/2304.02643
14	T. Darcet, M. Oquab, J. Mairal, P. Bojanowski. Vision transformers need registers. ICLR 2024. https://arxiv.org/abs/2309.16588
15	T. Chen, S. Kornblith, M. Norouzi, G. Hinton. A simple framework for contrastive learning of visual representations (SimCLR). ICML 2020. https://arxiv.org/abs/2002.05709
16	M. Oquab et al. DINOv2: learning robust visual features without supervision. TMLR 2024. https://arxiv.org/abs/2304.07193
17	M. Assran et al. V-JEPA 2: self-supervised video models enable understanding, prediction and planning. 2025. https://arxiv.org/abs/2506.09985
18	A. Kumar, A. Raghunathan, R. Jones, T. Ma, P. Liang. Fine-tuning can distort pretrained features and underperform out-of-distribution. ICLR 2022. https://arxiv.org/abs/2202.10054
19	A. Radford et al. Learning transferable visual models from natural language supervision (CLIP). ICML 2021. https://arxiv.org/abs/2103.00020

Ch.	Paper
20	H. Liu, C. Li, Q. Wu, Y. J. Lee. Visual instruction tuning (LLaVA). NeurIPS 2023. https://arxiv.org/abs/2304.08485
21	J. Xu et al. Qwen2.5-Omni technical report. 2025. https://arxiv.org/abs/2503.20215
22	J. Ho, A. Jain, P. Abbeel. Denoising diffusion probabilistic models. NeurIPS 2020. https://arxiv.org/abs/2006.11239
23	M. Assran et al. V-JEPA 2: self-supervised video models enable understanding, prediction and planning. 2025. https://arxiv.org/abs/2506.09985
24	K. Musgrave, S. Belongie, S.-N. Lim. A metric learning reality check. ECCV 2020. https://arxiv.org/abs/2003.08505

Other primary papers used across chapters

- A. Vaswani et al. Attention is all you need. NeurIPS 2017. <https://arxiv.org/abs/1706.03762>
- A. Dosovitskiy et al. An image is worth 16x16 words. ICLR 2021. <https://arxiv.org/abs/2010.11929>
- D. P. Kingma, M. Welling. Auto-encoding variational Bayes. ICLR 2014. <https://arxiv.org/abs/1312.6114>
- D. J. Rezende, S. Mohamed, D. Wierstra. Stochastic backpropagation and approximate inference in deep generative models. ICML 2014. <https://arxiv.org/abs/1401.4082>
- A. P. Dempster, N. M. Laird, D. B. Rubin. Maximum likelihood from incomplete data via the EM algorithm. *JRSS B* 39(1), 1977.
- O. Ronneberger, P. Fischer, T. Brox. U-Net: convolutional networks for biomedical image segmentation. MICCAI 2015. <https://arxiv.org/abs/1505.04597>
- T.-Y. Lin, P. Dollár, R. Girshick, K. He, B. Hariharan, S. Belongie. Feature pyramid networks for object detection. CVPR 2017. <https://arxiv.org/abs/1612.03144>
- S. Ren, K. He, R. Girshick, J. Sun. Faster R-CNN. NeurIPS 2015. <https://arxiv.org/abs/1506.01497>
- N. Carion et al. End-to-end object detection with transformers (DETR). ECCV 2020. <https://arxiv.org/abs/2005.12872>
- T.-Y. Lin, P. Goyal, R. Girshick, K. He, P. Dollár. Focal loss for dense object detection. ICCV 2017. <https://arxiv.org/abs/1708.02002>
- K. He, H. Fan, Y. Wu, S. Xie, R. Girshick. Momentum contrast for unsupervised visual representation learning (MoCo). CVPR 2020. <https://arxiv.org/abs/1911.05722>
- A. van den Oord, Y. Li, O. Vinyals. Representation learning with contrastive predictive coding. 2018. <https://arxiv.org/abs/1807.03748>
- J.-B. Grill et al. Bootstrap your own latent (BYOL). NeurIPS 2020. <https://arxiv.org/abs/2006.07733>
- M. Caron et al. Emerging properties in self-supervised vision transformers (DINO). ICCV 2021. <https://arxiv.org/abs/2104.14294>
- K. He, X. Chen, S. Xie, Y. Li, P. Dollár, R. Girshick. Masked autoencoders are

- scalable vision learners. CVPR 2022. <https://arxiv.org/abs/2111.06377>
- E. J. Hu et al. LoRA: low-rank adaptation of large language models. ICLR 2022. <https://arxiv.org/abs/2106.09685>
 - J.-B. Alayrac et al. Flamingo: a visual language model for few-shot learning. NeurIPS 2022. <https://arxiv.org/abs/2204.14198>
 - S. Bai et al. Qwen2.5-VL technical report. 2025. <https://arxiv.org/abs/2502.13923>
 - M. Assran et al. Self-supervised learning from images with a joint-embedding predictive architecture (I-JEPA). CVPR 2023. <https://arxiv.org/abs/2301.08243>
 - A. Bardes et al. Revisiting feature prediction for learning visual representations from video (V-JEPA). TMLR 2024. <https://arxiv.org/abs/2404.08471>
 - J. Song, C. Meng, S. Ermon. Denoising diffusion implicit models. ICLR 2021. <https://arxiv.org/abs/2010.02502>
 - J. Ho, T. Salimans. Classifier-free diffusion guidance. 2022. <https://arxiv.org/abs/2207.12598>
 - R. Rombach, A. Blattmann, D. Lorenz, P. Esser, B. Ommer. High-resolution image synthesis with latent diffusion models. CVPR 2022. <https://arxiv.org/abs/2112.10752>
 - R. Hartley, A. Zisserman. *Multiple View Geometry in Computer Vision*, 2nd edition. Cambridge University Press, 2004.
 - G. Zhou, H. Pan, Y. LeCun, L. Pinto. DINO-WM: world models on pre-trained visual features enable zero-shot planning. 2024. <https://arxiv.org/abs/2411.04983>

Each chapter’s own reference list gives the complete set of works it cites.

Figure and material credits

Figures. Every figure in this book is an original drawing made for it in TikZ/pgfplots; no figure, photograph or page from another work is reproduced. Plots show data produced by the book’s own labs from synthetic, seeded data. Diagrams of published architectures are new drawings of the published designs, marked “redrawn after” in their captions, with the source figure and a link:

Figure	Redrawn after
10.1 VAE training and generation paths	Kingma and Welling (2014); Rezende, Mohamed and Wierstra (2014)
10.5 Latent diffusion	Rombach et al. (2022), Fig. 3
12.4 Residual blocks and ResNet-50 stages	He et al. (2016), Figs. 2 and 5, Table 1
12.5 U-Net and feature pyramid network	Ronneberger et al. (2015), Fig. 1; Lin et al. (2017), Fig. 3
13.3 Faster R-CNN (modern variant) and DETR	Ren et al. (2015), Fig. 2, with FPN (Lin et al., 2017) and RoIAlign (He et al., 2017); Carion et al. (2020), Fig. 2
13.5 Segment Anything	Kirillov et al. (2023), Fig. 4
14.2 Scaled dot-product attention	Vaswani et al. (2017)
14.3 ViT tokenisation and block	Dosovitskiy et al. (2021)
14.4 Register tokens	Darcet et al. (2024), Fig. 1
16.1 BYOL and DINO	Grill et al. (2020), Fig. 2; Caron et al. (2021), Fig. 2
16.2 Masked autoencoder	He et al. (2022), Fig. 1
17.1 I-JEPA and V-JEPA	Assran et al. (2023), Fig. 3; Bardes et al. (2024)

Figure	Redrawn after
17.3 V-JEPA 2 pipeline	Assran et al. (2025), Fig. 1
19.1 CLIP	Radford et al. (2021), Fig. 1
20.1 LLaVA and Flamingo	Liu et al. (2023), Fig. 1; Alayrac et al. (2022)
20.2 LoRA	Hu et al. (2022), Fig. 1
21.1 Qwen2.5-Omni (panel c)	Xu et al. (2025), architecture figure
22.1 Diffusion chains (panel a)	Ho et al. (2020), Fig. 2
23.2 V-JEPA 2-AC (panel b)	Assran et al. (2025), action-conditioned training and planning figures
15.1 SimCLR and MoCo	Chen et al. (2020), Fig. 2; He et al. (2020), Fig. 1

Text. All text is written for this book. Short phrases quoted from other works are in quotation marks and attributed where they appear. Worked examples and exercises are original or adapted from standard textbook material, with the textbook cited in the chapter’s references.

Notation macros. `tex/dlbook/math_commands.tex` is copied unchanged from [goodfeli/dlbook_notation](#) (Goodfellow, Bengio and Courville), whose authors make it freely available for anyone to use; see `tex/dlbook/README.md`.

Data. No third-party dataset is included. Labs generate their data; chapters that discuss public datasets link to them and do not redistribute them.

Links verified 4 October 2026.

About the author

Ashish Papanai is a master's student in Machine Learning at the University of Tübingen and an ELIZA scholar. His research at the CVML Lab focuses on spatial audio-visual reasoning, multimodal systems, and world models. Previously, he worked as an Associate ML Scientist II at Wadhvani AI, developing computer vision systems for crop monitoring, pest detection, and yield estimation.

Website: <https://ashishpapanai.github.io/>